

第二十一屆組合數學與計算理論研討會論文集

Proceedings of the 21st Workshop on Combinatorial Mathematics and Computation Theory

algo 2004

2004/5/21~5/22

主辦單位：臺中健康暨管理學院資訊科技學系

協辦單位：國科會工程處工程科技推展中心

中央研究院資訊科學研究所

研討會網址：<http://it.thmu.edu.tw/algo2004>

A Frequency-Sensitive Competitive Learning Networks with Hadamard Transform Applied to Color Image Compression

Chi-Yuan Lin^{1,2} and Chin-Hsing Chen¹

¹Department of Electrical Engineering,
National Cheng Kung University, Tainan, 70101 Taiwan, R.O.C.

²Department of Electronic Engineering,
National Chin-Yi Institute of Technology, Taichung, 41111 Taiwan, R.O.C.
E-mail: cylin2.monica@msa.hinet.net

Abstract

The neural network is useful for data compression if the connection weights are chosen properly. In this paper, a Modified Frequency-Sensitive Competitive Learning (MFSCCL) network with Hadamard transform based on Vector Quantization (VQ) for color image compression is presented. The goal is apply a spread-unsupervised scheme based on the modified competitive learning networks so that on-line learning and parallel implementation for color image compression based on VQ in Hadamard Transform (HT) domain are feasible. In the MFSCCL network, each output neuron (codevector) incorporates a count of the number of times it has been the winner. The distortion measure used to determine the winner is updated to include the count number. The color image information transformed by HT operation was separated into RGB 3-plane DC value and AC coefficients. Then the AC coefficients for each plane were trained using the proposed MFSCCL method to generate the VQ codebook. The experimental results show that promising codebooks can be obtained using the presented spread MFSCCL network for color image compression in the transform domains.

Keywords: data compression, neural networks, Hadamard transform.

1 Introduction

A vector quantizer is a mapping of an input or test vector to an output or reproduction vector that is selected from a finite codebook of stored vectors. A number of vector quantization algorithms for data compression have proposed over the years [1-3]. The purpose of vector quantization is to create a codebook such that the average distortion between training vectors and their corresponding codevectors in the codebook is minimized. Codebook design can be considered as a clustering process in which each training vector is classified to a specific class. The clustering process updates the codebook iteratively such that the average distortion between training vectors and codevectors in the codebook becomes smaller and smaller.

Neural network are well suited to the problem of image compression due to their massively parallel and distributed architecture. Neural networks with competitive learning have been demonstrated capable of performing vector quantization [4-6]. A problem with the training procedure of competitive learning networks is that it sometimes leads to codevectors which are not distributed in an equiprobable manner [7]. The Frequency-Sensitive Competitive Learning (FSCL) [8] network is to overcome the limitations of the conventional competitive learning network. In Modified Frequency-Sensitive Competitive Learning

(MFSCl) network, the learning rule and stopping criterion of the FSCL network are modified to address the codebook design. The problem of VQ is regarded as a minimization process of an object function. This object function is defined as the average distortion between the training vectors in a divided image to the cluster centers represented by the codevectors in the codebook. The modified frequency-sensitive competitive learning network is simpler than that of the FSCL network, and is constructed as a two-layer fully interconnected array with the input neurons representing the training vectors and output neurons representing the codevectors in the codebook.

Color images comprise three planes: red, green, and blue. Thus, the most straightforward method to encode a color image by modified frequency-sensitive competitive learning networks is to split the RGB color image into 3 planes and compress them separately by treating each color component as a single gray-level image [9]. In this paper, the MFSCl network has been extended to Spread MFSCl network and applied to color image compression.

In the Hadamard Transform (HT) domain, this object function is defined as the average distortion between the training vectors in AC coefficients to the cluster centers represented by the codevectors in the codebook. The training vectors, constructed by AC coefficients, are directly fed into the MFSCl network. The MFSCl and spread MFSCl networks are demonstrated to have the capability of VQ in the transform domains.

The rest of this paper is organized as follows. Section 2 reviews the competitive learning networks. Section 3 presents the spread MFSCl network for color VQ. Section 4 presents the Hadamard transform and spread MFSCl network. Experimental

results are given in Section 5. Finally, conclusions are drawn in Section 6.

2 Competitive Learning Network

A competitive learning network is an unsupervised network which selects a winner based on similarity measure over the feature space. A proper neuron state is updated if and only if it wins the competition among all neurons. Many schemes for competitive learning networks have been proposed [10-12].

In the simple competitive learning network, the single output layer consists of cluster centers, each of which is fully connected to the inputs via interconnection strength. In the conventional competitive learning only one output unit is active at a time and the objective function is given by

$$J_c = \frac{1}{2} \sum_{j=1}^c \sum_{i=1}^n u_{i,j} \|\mathbf{x}_i - \mathbf{w}_j\|^2 \quad (1)$$

where n and c are the number of training vectors and the number of clusters respectively. $u_{i,j} = 1$ if $\mathbf{x}_i$ belongs to cluster j and $u_{i,j} = 0$ for all other clusters. The neuron that wins the competition is called the winner-take-all neuron. Thus $u_{i,j}$ indicates whether the input sample $\mathbf{x}_i$ activates neuron j to be a winner. $u_{i,j}$ is given by

$$u_{i,j} = \begin{cases} 1 & \text{if } \|\mathbf{x}_i - \mathbf{w}_j\| \leq \|\mathbf{x}_i - \mathbf{w}_k\|, \text{ for all } k; \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The incremental $\Delta \mathbf{w}_j$ is given by [10]

$$\langle \Delta \mathbf{w}_j \rangle = -\eta \frac{\partial J_c}{\partial \mathbf{w}_j} = \eta \sum_{i=1}^n (\mathbf{x}_i - \mathbf{w}_j) u_{i,j} \quad (3.a)$$

where $j = 1, 2, \dots, c$. Although Eq. (3.a) is written as a sum over all samples, practically it is usually used incrementally, i.e.

$$\Delta \omega_j = \eta (\mathbf{x}_i - \omega_j) \mu_{i,j}, \quad j = 1, 2, \dots, c. \quad (3.b)$$

The updating rule is given by

$$\omega_j(t+1) = \omega_j(t) + \Delta \omega_j(t). \quad (4)$$

In the above equation, the t is the training time index, η is the learning-rate parameter, and is typically reduced monotonically to zero as learning progresses. The simple competitive learning algorithm can be described as follows.

Step 1: Apply an input sample $\mathbf{x}_i$.

Step 2: Find the distortion for all output neural units.

Step 3: Select the output unit with the smallest distortion and label it as the winner and its weight vector as ω_j

Step 4: Update the selected weight vector by Eqs. (3) and (4).

Step 5: Repeat Steps 1 through 4 for all training samples.

3 Spread MFSCl network for Color VQ

Suppose an image is divided into n blocks (vectors of pixels) and each block consists of $\ell \times \ell$ pixels. A vector quantizer is a technique that maps the Euclidean $\ell \times \ell$ -dimensional space $\mathbf{R}^{\ell \times \ell}$ into a set $\{\omega_j, j = 1, 2, \dots, c\}$ of points in $\mathbf{R}^{\ell \times \ell}$, called a codebook. It looks for a codebook such that each training vector is approximated as close as possible by one of the code vectors in the

codebook. A codebook is optimal if the average distortion is at the minimum value. The average distortion $E[d(\mathbf{x}_i, \omega_j)]$ between an input sequence of training vectors $\{\mathbf{x}_i, i = 1, 2, \dots, n\}$ and its corresponding output sequence of code vectors $\{\omega_j, j = 1, 2, \dots, c\}$ is defined as

$$D = E[d(\mathbf{x}_i, \omega_j)] = \frac{1}{n} \sum_{i=1}^n d(\mathbf{x}_i, \omega_j) \quad (5)$$

A modified distortion measure for the training process is defined as

$$d^*(\mathbf{x}_i, \omega_j) = d(\mathbf{x}_i, \omega_j(t)) \times c_j(t) \quad (6)$$

where $c_j(t)$ is the total number of times that code vectors ω_j has been the winner during training. The winning code vector at each iteration of the training process is the vector with the minimum d^* .

The MFSCl network has the same architecture as the FSCL network. It is an unsupervised competitive learning network using the modified competitive learning rule and stopping criterion. Similar to the standard competitive learning rule in Eqs. (3.b) and (4), the least squared error solution can be obtained by

$$\omega_j(t+1) = \begin{cases} \omega_j(t) + \eta(\mathbf{x}_i - \omega_j) & \text{if } \|\mathbf{x}_i - \omega_j\| \leq \|\mathbf{x}_i - \omega_k\|, \text{ for all } k; \\ \omega_j(t) & \text{otherwise} \end{cases} \quad (7)$$

The MFSCl algorithm modifies only output neurons without updating the interconnection strengths. Instead of updating the interconnection strengths using the winner-take-all scheme in the FSCL network and for the purpose of simplifying the hardware architecture, the MFSCl network only modifies

the output states (codevectors).

We then map R, G and B plane training vectors of a color image to the spread MFSCCL neuron array that compresses them separately by treating each color plane as a single gray-level image. Therefore, the spread MFSCCL network based vector quantizer in the p th plane for the modified competitive learning rule can be modified as

$$\mathbf{w}_{j,p}(t+1) = \begin{cases} \mathbf{w}_{j,p}(t) + \eta(\mathbf{x}_{i,p} - \mathbf{w}_{j,p}) & \text{if } \|\mathbf{x}_{i,p} - \mathbf{w}_{j,p}\| \leq \|\mathbf{x}_{i,p} - \mathbf{w}_{k,p}\|, \text{ for all } k; \\ \mathbf{w}_{j,p}(t) & \text{otherwise} \end{cases} \quad (8)$$

Then, the steps of codebook design each plane using the spread MFSCCL network are given as follows.

- Step 1: Initialize the codevectors $\mathbf{w}_{j,p} (2 \leq j \leq c)$, learning rate η , maximum error (ME), total error (TE), and a threshold value ε .
- Step 2: Input a training vector $\mathbf{x}_{i,p}$ and find the winner's codevector based on the minimum d^* .
- Step 3: Apply Eq. (8) to update the winner's codevector and set $TE = TE + \|\mathbf{x}_{i,p} - \mathbf{w}_{j,p}\|$.
- Step 4: Repeat Step 2 and 3 for all input samples, then if $(ME - TE) / ME < \varepsilon$, go to step 5; otherwise replace ME content from TE , and go to Step 2.
- Step 5: Complete the codebook design in the p th plane ($p=1,2,3$).

4 Hadamard Transform and Spread MFSCCL network

Among various popular image transforms,

Hadamard Transform (HT) [13-14] takes the least computation overhead since its basis vectors content only +1 and -1. No multiplication is necessary and only fixed point arithmetic is required for computing the transformation. The forward and inverse 2-D Hadamard transform for an $N \times N$ image can be define as

Forward:

$$F(u, v) = \frac{1}{N} \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} f(j, k) (-1)^{\sum_{i=0}^{\ell-1} [b_i(u) b_i(j) + b_i(v) b_i(k)]} \quad (9)$$

Inverse:

$$f(j, k) = \frac{1}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} F(u, v) (-1)^{\sum_{i=0}^{\ell-1} [b_i(u) b_i(j) + b_i(v) b_i(k)]} \quad (10)$$

where $F(u, v)$ are the Hadamard transform coefficients, $f(j, k)$ are the pixel value at (j, k) , $N = 2^\ell$ for some ℓ , and $b_i(u)$ is the i th bit of u in the binary form.

For a 2-D $N \times N$ image, the input image will be divided into nonoverlapping blocks of equal size. The Hadamard transform is applied to transform each block into DC value and AC coefficients. Then the AC coefficients are fed into the spread MFSCCL algorithm for training vectors and generating better codebooks. In the decoder, indexes of codevectors are received to get AC information (codevectors). Then, the inverse Hadamard transform is used to reconstruct the image using DC value and AC coefficients.

5 Experimental Results

The codebook design is the primary problem in image compression based on vector quantization. In this paper, the images reconstructed from the LBG · MFSCCL · Hadamard Transform + MFSCCL (named by

HT+MFSCl) methods are compared. The training vectors were extracted from 256×256 with 8-bit gray level real images, each of which is divided into 4×4 blocks to generate 4096 non-overlapping 16-D training vectors. Two codebooks of size 128 and 256 were built by these training vectors. The resulting images were evaluated subjectively by the mean squared error (MSE) and peak signal to noise ratio (PSNR) that is defined for images of size $N \times N$ as

$$PSNR = 10 \log_{10} \frac{255^2}{MSE} \quad (11)$$

and

$$MSE = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N (x_{ij} - \hat{x}_{ij})^2 \quad (12)$$

where x_{ij} and $\hat{x}_{ij}$ are the pixel gray levels from the original and reconstructed images, and 255 is the peak gray level. Fig. 1 shows the PSNR and MSE of the “Girl” images reconstructed from two codebooks of size 128 and 256 designed by the LBG and the presented MFSCl and HT+MFSCl methods. From experimental results, the reconstructed images obtained from the presented MFSCl network are superior to those obtained from the LBG algorithm, and the performance of the presented HT+MFSCl algorithm is significantly better than the MFSCl algorithm.

In the color compression simulation, the color image information transformed by Hadamard transform operation was separated into RGB planes DC value and AC coefficients. Then the AC coefficients for each plane were trained using the proposed spread MFSCl methods to generate better codebook based on VQ. To show the reconstruction performance, the resulting images were evaluated by the average PSNR among three-color planes

$$PSNR_A = \frac{PSNR_R + PSNR_G + PSNR_B}{3} \quad (13)$$

where $PSNR_R, PSNR_G$, and $PSNR_B$ are the PSNR for red, green, and blue planes, respectively. The PSNRs of the “Tree” and “Girl” images calculated are shown in Fig. 2, and their reconstructed images using the HT + spread MFSCl network with 256 codevectors each plane are shown in Figure 3. Experimental results show that the HT + spread MFSCl method produce promising reconstructed images quality.

6 Conclusions

In this paper, a spread unsupervised parallel approach called spread MFSCl network with Hadamard transform based on vector quantization for color image compression has been presented. Instead of updating the interconnection strengths using the winner-take-all scheme in the FSCL network, the MFSCl algorithm only modifies output neurons and omits the updating of the interconnection strengths. Experimental results show that the MFSCl network is valid for codebook design and the HT+MFSCl method produce promising reconstructed images than those reconstructed by the MFSCl method. In addition, an important advantage of using neural networks for vector quantization is that the computation can be carried out in parallel by the neural units. Our future work includes VHDL implementation of MFSCl algorithm. The target technology will be FPGA design.

References

- [1] Y. Linde, A. Buzo, and R. M. Gray, “An Algorithm for Vector Quantizer Design,” *IEEE Trans. Commun.*, vol. COM-28, pp. 85-94, Jan., 1980.
- [2] R. M. Gray, “Vector Quantization,” *IEEE*

- ASSP Magazine, Vol.1, pp. 4-29, 1984.
- [3] A. Gersho, and R. M. Gray, *Vector Quantization and Signal Compression*, Kluwer Academic Publishers, Norwell, Ma., 1992.
- [4] M. H. Lin, S. S. Yang, J. S. Lin, M. H. Sheu, and C. Y. Lin, "A Modified Competitive Learning Network in Image Compression and VLSI Implementation," *The 15th Technological and Vocational Education Conference of R.O.C.*, pp. 189-198, 2000.
- [5] D. C. Park, "Centroid Neural Network for Unsupervised Competitive Learning," *IEEE Trans. on Neural Networks*, vol. 11, no. 2, pp. 520-528, 2000.
- [6] C. Y. Lin and C. H. Chen, "Image Compression Using Grey-Based Hopfield Neural Network and Block Truncation Coding," *Proceedings of the 20th Workshop on Combinatorial Mathematics and Computation Theory*, pp. 193-198, 2003.
- [7] R. Hecht-Nielsen, "Application of Counter-propagation Networks," *Neural Networks*, vol. 1, no. 2, pp. 131-141, 1988.
- [8] A. K. Krishnamurthy, S. C. Ahalt, D. E. Melton, and P. Chen, "Neural Networks for Vector Quantization of Speech and Images," *IEEE J. on Selected Areas in Commun.*, vol. 8, no. 8, pp. 1449-1457, 1990.
- [9] L. Moltedo, M. Nappi, D. Vitulano, and S. Vitulano, "Color image coding combining linear prediction and iterated function systems," *Signal Processing*, 63, 157-162, 1997.
- [10] J. S. Lin, K. S. Cheng, and C. W. Mao, "Segmentation of Multispectral Magnetic Resonance Image Using Penalized Fuzzy Competitive Learning Network," *Computers and Biomedical Research* 29, pp.314-326, 1996.
- [11] A. Bouzerdoum, "A Stochastic Competitive Learning Algorithm," *International Joint Conference on Neural Networks*, vol. 2, pp.908-913, 2001.
- [12] R. M. Chen, Y. M. Huang, and C. Y. Lin, "Competitive Neural Network to solve Real-Time Scheduling," *International Computer Symposium*, 2002.
- [13] K. R. Castleman, "Digital Image Processing," Prentice-Hall, New York, 1996.
- [14] C. K. Chan and L. M. Po, "Image Vector Quantization Using Hadamard Transform Subspace," *IEEE Region 10 Conference on Computer and Communication Systems*, 1990, pp. 476-480.

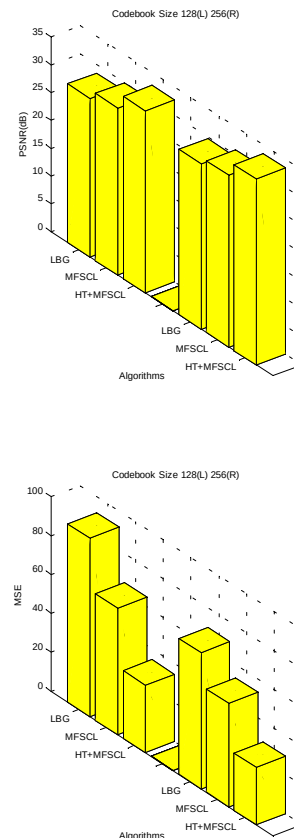


Fig. 1 PSNR and MSE of the "Girl" image reconstructed by the LBG and the presented MFSCl and HT+MFSCl algorithms.

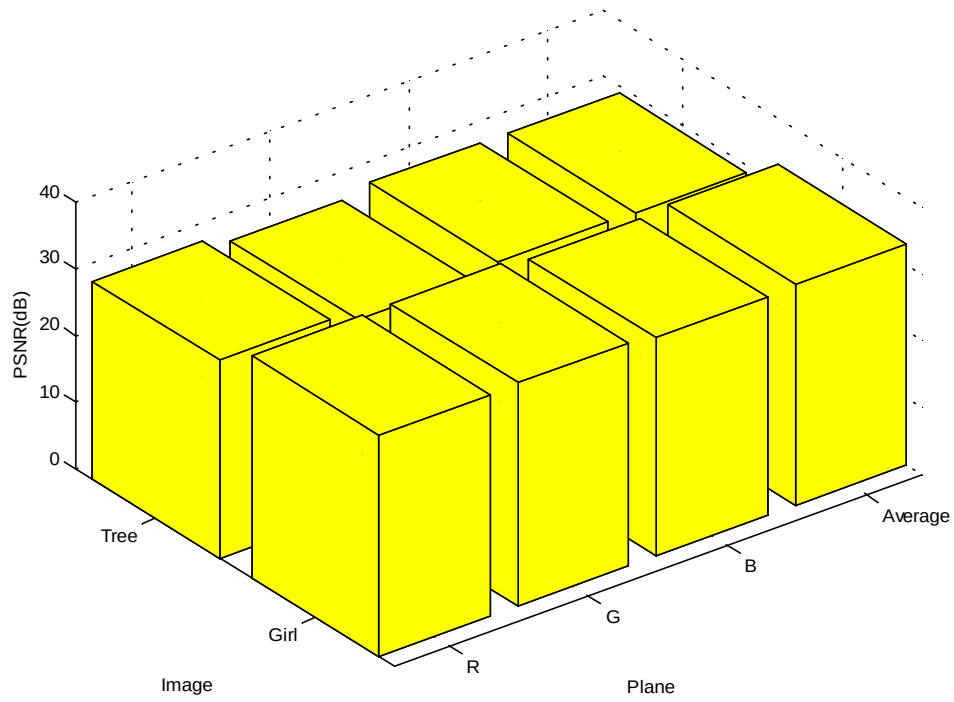


Fig. 2 PSNRs of the “Tree” and “Girl” images reconstructed by the Spread MFSCCL with 256 codevectors each plane.



Fig. 3 The “Tree” and “Girl” original color images (up row) and their reconstructed images (down row) using the HT + spread MFSCCL with 256 codevectors each plane.

Robust Watermarking Scheme Using Phase Shift Keying Embedding

Wen-Yuan Chen, Chio-Tan Kuo and Jiang-Nan Jow

Department of Electronic Engineering,

National Chin-Yi Institute of Technology, Taichung, Taiwan, R.O.C.

E-mail: cwy@chinyi.ncit.edu.tw

Abstract

Watermarking is a potential method for copyright protection and authentication of multimedia data on the internet. In this paper, a novel watermarking scheme using phase shift keying (PSK) modulation with amplitude boost (AB) and low amplitude block selection (LABS) is proposed to achieve superior performance in terms of robustness and imperceptibility. AB is hired to increase the robustness while LABS is employed to improve the imperceptibility.

In order to demonstrate the effectiveness of the proposed scheme, simulations under various conditions were conducted. The empirical results show that our proposed scheme can sustain most common attacks including JPEG compression, rotating, resizing, cropping, painting, noising and blurring etc.

Keyword: Spread Spectrum (SS), Phase Shift Keying (PSK), Discrete Fourier Transform (DFT), Pseudo-random Number sequence (PN), Joint Photographic Experts Group (JPEG).

1. Introduction

Watermarking techniques can be classified into two categories, one is processed in the spatial domain [1-2] and the other is accomplished in the transform domain. In the transform domain many approaches [3-7] are based on the Discrete Cosine Transform (DCT). Hsu and Wu [4] proposed a scheme by block-based image-dependent permutation of the watermarks in the middle band of the DCT coefficients and obtained good performance. Wu and Hsieh [6] used zerotree structure to embed watermark by rearranging the DCT coefficients in a way similar to the multi-resolution analysis (MRA) of wavelet transforms.

Another method in the transform domain is to hide watermarks in the discrete Fourier transform (DFT) coefficients of the host image. Ruanaidh et al. [13] presented a phase-based method in the DFT

domain and used an optimal detector for watermark recovery. Based on the Fourier-Mellin Transform Ruanaidh and Pun [14] presented a watermarking scheme that achieves rotation, scale and translation (RST) invariant. The scheme achieves robustness while sustains the RST attacks. Premaratne and Ko [15] proposed a new concept for embedding and detecting the watermark in the Discrete Fourier Transform. Since the embedding is independent of the image content, speedy embedding highly suitable for video streams can be achieved. Solachidis and Pitas [16] proposed a watermarking scheme which embeds a circularly symmetric watermark on a ring in the 2D DFT domain. The circularly symmetric watermark was used to solve the rotation invariance problem in the watermark detection in which a correlation operation was used.

In this paper, we proposed a DFT domain watermarking scheme using the phase shift keying (PSK). In our proposed scheme, the watermark bits are first expanded by spread spectrum and then concealed by PSK modulation in the DFT coefficients of the host image. The PSK embedding is employed due to its superior noise immunity. In the PSK embedding, the watermark information is embedded in the phase part of the host image. Thus, the threshold effect in which the quality of the recovered watermark plunges when the amplitudes of the DFT coefficients used for embedding the secret bits are below a threshold value may occur [18]. In this paper, a novel idea combining amplitude boost (AB) and low-amplitude block selection (LABS) is proposed to curb the threshold effect raised by PSK. We demonstrated that by properly combining AB and LABS robustness can be enhanced without sacrificing imperceptibility. Meanwhile, in our scheme neither the original host image nor the original watermark is required during the watermark detection process.

The remainder of this paper is organized as follows. In section 2, the proposed concealing algorithm is presented. The watermark extracting process is presented in section 3. Empirical results are presented in section 4. Finally, section 5 concludes this paper.

2. Concealing Algorithm

A robust watermarking scheme must survive all kinds of attacks, and at the same time sustain the virtual quality of the host image when the watermark is concealed. Besides, security is also an important factor required by a watermarking scheme. In order to construct a superior watermarking scheme, several skills are used in this paper to achieve the goal. The overall concealing process of our proposed scheme is shown in Fig. 1. The watermark W is first transformed to H by toral automorphism (TA) [18] using a pseudo random sequence (PN) generated by a private key to enhance the security. It is then spreaded by spread spectrum (SS) [19-20] to a binary random sequence $\{m\}$. On the other hand, the host image X is transformed to Z by DFT. Then a low amplitude block B is selected from Z using the LABS strategy. Two DFT coefficients ae^{jf} and

ae^{-jf} which from a complex conjugate pair are selected from B for the following AB process in which a is boosted to a' to combat attacks. In the PSK modulation, f is modulated by m into f' . After AB and PSK, $a'e^{jf'}$ and $a'e^{-jf'}$ in which f' contains the secret information are then embedded into Z at the selected block and coefficient pair locations by watermark bits embedding (WBE). The above embedding process is repeated for each secret bit in $\{m\}$ and its corresponding block. The resultant image Z' after embedding is then inversely transformed to obtain the watermarked image R . The details of each block of our embedding algorithm are presented in the following.

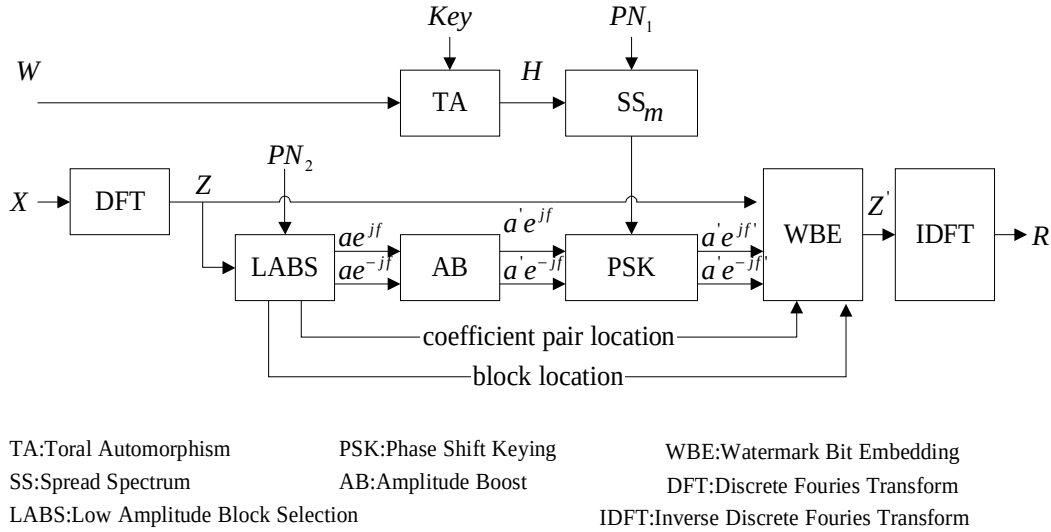


Fig. 1 The flow chart of the proposed embedding process

2.1 Toral Automorphism

For security, the watermark image is pre-permuted into noises by the toral automorphism with a user's key. The toral automorphism scatters the image shape in some iterated operations less than a specified number of times, and will return to the original shape while it is further iterated totally the specified number of times. The specified number is determined by the toral automorphism parameters and the image size. In this paper, the toral automorphism is used to transfer the shape of the original image into chaotic to protect the watermark from being stolen. The transfer function between H

and W is given by

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ k & k+1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \pmod{n} \quad (1)$$

Where (x, y) and (x', y') express the pixel locations of W and H respectively, k denotes the control parameter and n denotes the image size, respectively. An example of images transformed by toral automorphism is shown in Fig. 2.



Fig. 2 The images transformed by toral automorphism.

2.2 Spread Spectrum

SS is used to defense noises in many communication systems. A robust watermarking must be able to survive various attacks. In this paper, we hired the SS skill to enhance the robustness of the watermarking scheme. Spread Spectrum expands an information bit into several bits with random values. The expanded bits created when the information bit is high are the inverse of those created when the information bit is low. To randomize the expanded bits, PN sequences are used to accomplish the job. By raster scanning, H is converted into a bit sequence $d(j)$, $j=1,2,3\dots n$. For each $d(j)$, we generate a PN sequence $r_j(i)$, $i=1,2,3\dots l$, l is the length of expansion. By multiplying $d(j)$ by $r_j(i)$, a watermark-bearing bit is chopped up into chips. The expanded bit sequence is given by

$$m_j(i) = d(j) \cdot r_j(i), \quad i=1,2,3\dots l, \quad j=1,2,3\dots n. \quad (2)$$

2.3 Low-Amplitude Block Selection and Amplitude Boost

In this paper, two complementary strategies, amplitude boost (AB) and low-amplitude block selection (LABS) are employed to design a novel embedding scheme using the PSK modulation. The amplitude boost is a skill used to enhance robustness, while the low-amplitude block selection is used to preserve the imperceptibility as much as possible. The low amplitude block selection strategy selects blocks of low amplitude to embed the secret bits.

To make a watermarking scheme robust, a good strategy is to embed the watermark bits into the significant portion of the host signal, because this portion of the host data is highly sensitive to alteration. A watermark concealed in the high or middle frequency bands is easier to be removed or altered without affecting the host image by attacks [22]. In this paper, we embedded the secret data in lower frequency bands to enhance the robustness. The DFT of a block image B of size 8×8 is given by

$$B(u, v) = \frac{1}{\sqrt{64}} \sum_{x=0}^7 \sum_{y=0}^7 b(x, y) e^{-j2\pi xu/8 - j2\pi yv/8}, \quad 0 \leq u \leq 7, 0 \leq v \leq 7 \quad (3)$$

with the inverse transform given by

$$b(x, y) = \frac{1}{\sqrt{64}} \sum_{u=0}^7 \sum_{v=0}^7 B(u, v) e^{-j2\pi xu/8 - j2\pi yv/8}, \quad 0 \leq x \leq 7, 0 \leq y \leq 7 \quad (4)$$

In the PSK modulation, the watermark information is contained in the phase of the DFT coefficients. When a watermarked image is attacked, the DFT coefficients of the watermarked image are altered, which produces distortion when a watermark is recovered from the attacked watermarked image.

DFT coefficient due to an additive Gaussian noise is inversely proportional to the amplitude of the coefficient. In order to avoid the large distortion caused by DFT coefficients of small amplitudes, a novel strategy called AB is employed.

The AB strategy is to boost the amplitude of a selected DFT coefficient to a threshold value th when its value is below th so that the phase distortion under attacks can be keep below a certain level. In other words, after AB the amplitude of all the DFT coefficients used for embedding the secret bits are all above th . That is,

$$a' = \begin{cases} th & , \quad \text{if } a \leq th \\ a & , \quad \text{if } a > th \end{cases} \quad (5)$$

2.5 PSK Embedding

In the PSK modulation, the phase f is modified into f' according to

$$f' = a(f, s) + sb(m) \quad (6)$$

where s denotes the watermark strength factor, a and b denote the offset function and the embedding function respectively given by

$$a(f, s) = 2psi, \quad \text{if } |f - 2psi| \leq |f - 2psj|$$

$$\text{for } j = 0, 1, 2, \dots, \frac{1}{s} - 1, j \neq i$$
(7)

$$b(m) = \begin{cases} p/2, & \text{if } m = 0 \\ -p/2, & \text{if } m = 1 \end{cases}$$
(8)

Fig. 3 shows the coefficient pair selected within a block for embedding the secret bit and Fig. 4 shows the signal constellation of the PSK modulation for $s=1$, $s=1/2$ and $s=1/4$.

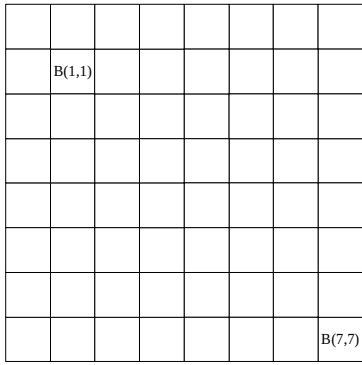


Fig. 3 The coefficient pair selected within a block for embedding the secret bit.

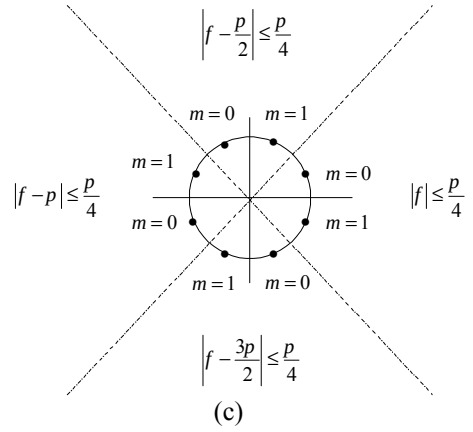
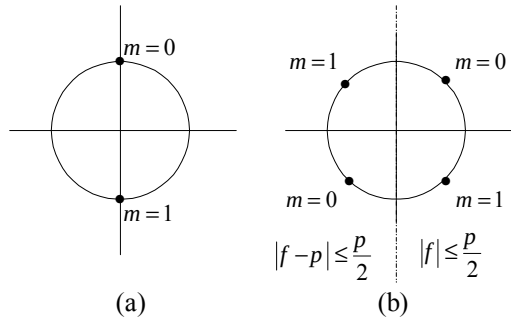
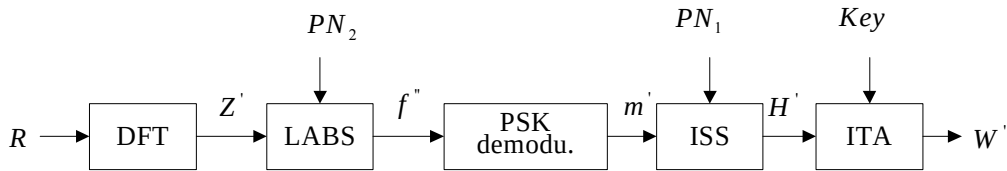


Fig. 4. The signal constellation of the PSK for (a) $s=1$, (b) $s=1/2$, and (c) $s=1/4$

3. Watermark Extraction Algorithm

A process which inverses the concealing process is used to recover the original watermark. The flow chart of the recovering process is shown in Fig. 5. The watermarked image R is transformed to Z' by DFT. The same PN sequence PN_2 used in the concealing process is used to select the embedded blocks from Z' for the PSK demodulation. In the PSK demodulation, the secret bit m' is extracted from the phase f'' of the selected DFT coefficient for each selected block. After all the secret bits are extracted from the PSK demodulation, they are contracted by inverse spread spectrum (ISS) and rearranged into the two-dimensional image H' . By passing H' through the inverse toral automorphism (ITA), the recovered watermark W' is obtained.



DFT: Discrete Fouries Transform

LABS: Low Amplitude Block Selection

PSK demodu.: Phase Shift Keying Demodulation

ISS: Inverse Spread Spectrum

ITA: Inverse Toral Automorphism

Fig. 5 The flow chart of watermark detection.

3.1 PSK Demodulation

In the process of the PSK demodulation the angle f'' is processed to recover the embedded secret bit

$$m' = \begin{cases} 0, & \text{if } |f'' - f_{1i}| \leq |f'' - f_{jk}|, \quad j=1,2; k=0,1,2,\dots,\frac{1}{s}-1, \quad k \neq i, \quad \text{when } j=1 \\ 1, & \text{if } |f'' - f_{2i}| \leq |f'' - f_{jk}|, \quad j=1,2; k=0,1,2,\dots,\frac{1}{s}-1, \quad k \neq i, \quad \text{when } j=2 \end{cases} \quad (9)$$

3.2 Inverse Spread Spectrum

Since SS expands watermark bits before embedding, they will be reconstructed by contraction. A bit $d'(j)$ in H' is obtained by contracting its expanded sequence $m'_j(i)$, $i=1,2,3\dots l$, using $r_j(i)$ generated by PN_1 . The value of $d'(j)$ is determined by

$$d'(j) = \begin{cases} 1, & \text{if } \sum_i m'_j(i) \oplus r_j(i) < \frac{l}{2} \\ 0, & \text{if } \sum_i m'_j(i) \oplus r_j(i) \geq \frac{l}{2} \end{cases} \quad j=1,2,\dots,n \quad (10)$$

Where $\oplus$ denotes the XOR operator.

4. Experimental Results

Imperceptibility is an important factor for watermarking. In this paper we employ the PSNR to indicate the degree of transparency. The PSNR of R is given by

$$PSNR = 10 \log_{10} \frac{255^2}{\frac{1}{N \times N} \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} (X(i,j) - R(i,j))^2} \quad (11)$$

Where $X(i,j)$ and $R(i,j)$, are the gray values at (i,j) of the host image X and the watermarked image R of size $n = N \times N$ respectively. The watermark similarity measurement is dependent on factors such as the knowledge of the experts, the experimental conditions, etc. Therefore a quantitative measurement is necessary to provide fair

judgment of the extracted fidelity. In this paper, we use the normalized correlation (NC) between the reference watermark W and the extracted watermark W' as the similarity measurement.

$$NC = \frac{\sum_i \sum_j [W(i,j)W'(i,j)]}{\sum_i \sum_j [W(i,j)]^2}, \quad (12)$$

NC is normalized by the reference watermark energy to give unity as the peak correction.

The images Lena (512×512) and Baboon (512×512) are used in simulation for demonstrating the performance of the proposed scheme. The logo image, National Chinyi Institute of Technology (32×32) and publications of department of electronic engineering (32×32) in Chinese were used as the watermark. The block size used is 8×8 . The values of parameters used in the simulations are: $k=5$, $s=\frac{1}{2}$, $l=3$, $b_1=3$, $b_2=1$, $th=9$ for the Baboon image and $th=13$ for the Lena image. The number of the secret bits after SS expansion is equal to $32 \times 32 \times l = 3072$. The total number of blocks is equal to $(512 \times 512) \div (8 \times 8) = 4096$. The number of blocks selected for concealing the secret bits is equal to $4096 \times \frac{b_1}{b_1 + b_2} = 3072$ which is equal to the

total number of the secret bits. The embedding scheme with AB and LABS is called WABLABS was used to test.

Fig. 6 shows the original image and the watermarked images of the WABLABS embedding scheme. The PSNR of the watermarked images are 35.35 and 34.12 for the Lena and Baboon images respectively. From Fig. 6, one could hardly perceive

the difference between the watermarked image and the original image. Figs. 7 and 8, show the comparison results from the JPEG lossy compression attacks using Photoshop 6.0 with quality levels from 0 to 12 for images Lena and Baboon respectively. Tables 1 and 2 show the empirical comparison results under the rotating, resizing, cropping, painting, noising and blurring attacks for the Lena and Baboon images respectively.

The normalized correlation and the extracted watermarks corresponding to Fig6 are shown in tables 3 and Fig. 7, as well as table 4 and Fig. 8 respectively. From the test results, the values of

NC are all above 0.88 and the extracted watermarks are clearly identified by the human vision.

Fig. 9 (a)-(f) show the 90-degree rotated, the resized, the cropped, the painted, the noised, and the blurred watermarked images and Fig. 9(g)-(l) show their corresponding extracted watermarks using the WABLABS scheme for the Lena image. The corresponding numbers of error bits of the extracted watermarks are listed in table 1. The visual quality of the extracted watermarks revealed in Fig. 9 (g)-(l) demonstrated that our proposed scheme can sustain all the above attacks.



Fig. 6 (a) the original image of Lena (b) the embedded image of Lena with the PSNR=35.35
(c) the original image of Baboon (d) the embedded image of Baboon with PSNR=34.12



Fig. 7 The extracted watermarks corresponding to JPEG compression attacks for the image Lena by photoshop 6.0.



Fig. 8 The extracted watermarks corresponding to JPEG compression attacks for the image Baboon by photoshop 6.0.



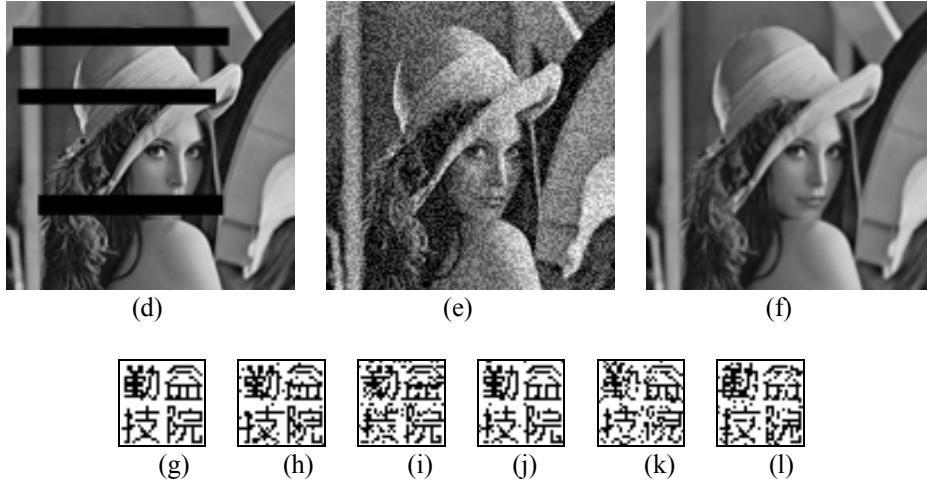


Fig. 9. The attacked watermarked images (a) – (f), (a) the 90-degree rotated (b) the resized (c) the cropped (d) the painted (e) the noised, and (f) the blurred. The corresponding extracted watermarks (g) – (l), (g) the 90-degree rotated (h) the resized (i) the cropped (j) the painted (k) the noised, and (l) the blurred.

Table 1 Comparison of the number of the error bits of the recovered watermark under various attacks for the Lena image.

Item	Parameter	WABLABS PSNR=35.35
Rotating	90 degree rotated	0
Resizing	From 512×512 to 192×192	52
Cropping	Cropping a quarter image	107
Painting	Painting three bars	29
Noising	15.06% noises contamination	103
Blurring	Gaussian blurring ratio 2.0	94

Table 2 Comparison of the number of the error bits of the recovered watermark under various attacks for the Baboon image.

Item	Parameter	WABLABS PSNR=34.12
Rotating	90 degree rotated	0
Resizing	From 512×512 to 192×192	108
Cropping	Cropping a quarter image	93
Painting	Painting three bars	33
Noising	15.06% noises contamination	97
Blurring	Gaussian blurring ratio 1.4	88

Table 3. The NC values corresponding to the number of error bits for the image Lena under JPEG compression attacks by photoshop 6.0.

Quality level	0	1	2	3	4	5	6	8	10	12
Error bits no	70	2	0	0	0	0	0	0	0	0
NC	0.92	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

Table 4. The NC values corresponding to the number of error bits for the image Baboon under JPEG compression attacks by photoshop 6.0.

Quality level	0	1	2	3	4	5	6	8	10	12
Error bits no	89	17	1	0	0	0	0	0	0	0
NC	0.88	0.95	0.99	1.0	1.0	1.0	1.0	1.0	1.0	1.0

5. Conclusions

Digital watermarking is a potential method to discourage unauthorized copying or attest origin of

digital data that includes audio, video and images. In this paper we present a robust watermarking scheme for still images using PSK with amplitude boost and low amplitude block selection. The amplitude boost strategy is used to enhance the robustness and the low amplitude block selection strategy is used to reduce the degradation of the host image caused by watermark concealing. Empirical results show that the proposed scheme can sustain attacks like JPEG lossy compression, rotation, resizing, cropping, painting, noising and blurring.

References

- [1] C. I. Podilchuk, and W. Zeng, Image-Adaptive Watermarking Using Visual Models, IEEE Trans. on selected area in communications 16 (4) (1998) 525-539.
- [2] W. N. Lie, and L. C. Chang, Spatial-Domain Image Watermarking by Data Embedding at Adaptive Bit Position. Proc. IPPR Conference on Computer Vision, Graphics and Image Processing, 1999, pp. 16-21.
- [3] S. C. Pei, Y. H. Chen and R. F. Torng, Digital Image and Video Watermarking Utilizing Just-Noticeable-Distortion Model. IPPR Conference on Computer Vision, Graphics and Image processing, 1999, pp. 174-182.
- [4] C. T. Hsu and J. L. Wu, Hidden Digital Watermarks in Images, IEEE Trans. on Image Processing, 8 (1) (1999) 58-68.
- [5] C. T. Hsu and J. L. Wu, DCT-Based Watermarking for Video, IEEE Trans. on Consumer Electronics, 44 (1) (1998) 206-216.
- [6] C. F. Wu and W. S. Hsieh, Digital Watermarking Using ZeroTree of DCT, IEEE Trans. on Consumer Electronics, 46 (1) (2000) 87-94.
- [7] G. C. Langelaar and R. L. Lagendijk, Optimal Differential Energy Watermarking of DCT Encoded Images and Video, IEEE Trans. on Image Processing, 10 (1) (2001) 148-158.
- [8] M. J. Tsai, K. Y. Yu and Y. Z. Chen, Joint Wavelet and Spatial Transformation for Digital Watermarking, IEEE Trans. on Consumer Electronics, 46 (1) (2000) 241-245.
- [9] Z. H. Wei, P. Qin and Y. Q. Fu, Perceptual Digital Watermark of Images Using Wavelet Transform, IEEE Trans. on Consumer Electronics, 44 (4) (1998) 1267-1272.
- [10] Z. M. Lu and S. H. Sun, Digital Image Watermarking Technique Based on Vector Quantization, ELECTRONICS LETTERS, 36 (4) (2000) 303-305.
- [11] H. Inoue, A. Miyazaki, A. Yamamoto and T. Katsura, A Digital Watermark Technique Based on the Wavelet Transform and Its Robustness on Image Compression and Transformation, IEICE Trans. Fundamentales, E82-A (1) (1999) 2-10.
- [12] N. Kaewkamnerd and K. R. Rao, Wavelet Based Image Adaptive Watermarking Scheme, ELECTRONICS LETTERS, 36 (4) (2000) 312-313.
- [13] J. J. K. Q Ruanaidh, W. J. Dowling and F. M. Boland, Phase Watermarking of Digital Images. IEEE International Conference on Image Processing, 3, 1996, pp. 239-242.
- [14] J. J. K. Q Ruanaidh and T. Pun, Rotation, Scale and Translation Invariant Digital Image Watermarking, IEEE International Conference on Image Processing 1, 1997, pp. 536-539.
- [15] P. Premaratne and C. C. Ko, A Novel Watermark embedding and Detection Scheme for Images in DFT Domain, IEEE International Conference on Image Processing and its Application, 2, 1999, pp. 780-783.
- [16] V. Solachidis and I. Pitas, Circularly Symmetric Watermark Embedding in 2-D DFT Domain, IEEE Trans. On Image Processing, 10 (465) (2001) 1741-1753.
- [17] Wen-Yuan Chen and Chin-Hsing Chen, "Robust Watermarking Scheme for Still Images Using Frequency Shift Keying with High-Variance Block Selection," *Optical Engineering*, Vol. 42, No. 6, pp.1826-1835, June 2003.
- [18] S. Haykin, *Communication System*, John Wiley & Sons, INC. New York, third edition, 1994, pp. 334-340.
- [19] G. Voyatzis and I. Pitas, Application of Toral automorphisms in Image Watermarking, IEEE International Conference on Image Processing, 3, 1996, pp. 237-240.
- [20] I. J. Cox, J. Kilian, F. T. Leighton and T. Shamoon, Secure Spread Spectrum Watermarking for Multimedia, IEEE Trans. on Image Processing, 6 (12) (1997) 1673-1687.
- [21] L. M. Marvel, C. G. Boncele, Jr. and C. T. Retter, Spread Spectrum Image Steganography, IEEE Trans. on Image Processing, 8 (8) (1999) 1075-1083.
- [22] S. Voloshynovskiy, S. Pereira, T. Pun, J. Eggers and J. K. Su, Attacks on Digital Watermarks: Classification, Estimation-Based Attacks, and Benchmarks, IEEE Communications Magazine, August 2001.

The Quantization Effects of CORDIC Arithmetic for Digital Signal Processing Applications

Tze-Yun Sung Yi-Hsun Sung

Electrical Engineering Department

Chung Hua University

Tel: 886-3-579-0902 Fax: 886-3-578-1150

E-mail: bobsung@btcx.com.tw, tom@btcx.com.tw

Abstract

CORDIC (COordinate Rotation DIgital Computer) is an algorithm for performing a sequence of iteration computation using the coordinate rotation. It can generate some powerful elementary function only realized by a simple set of adder and shifter. As such, it is suitable to implement high performance chips, which are applied in digital signal processing and image processing fields with high speed and massive function computation using VLSI technology.

In this paper, we discuss and analyze the computation error of the CORDIC algorithm. We analyze the CORDIC algorithm and categorize different types of computation error in order to derive from them systematically. Each type of error analysis includes rotation mode and vectoring mode according to characteristic of the CORDIC algorithm, and the computation error of each mode is made up of approximation error and truncation error. We have not only considered and promoted some neglected errors and formulas, but also revealed the outcomes of error analysis in the form, which is only related to the number of iteration and word length only. We also reveal the design reference with the error tolerance in tables.

Keywords: CORDIC, approximation error, truncation error, digital signal processing.

1. Introduction

CORDIC (COordinate Rotation DIgital Computer) is an iterative arithmetic algorithm for efficient computation of elementary functions [1]. Based on a rotation paradigm, CORDIC facilitates an efficient and unified computation scheme for the evaluation of many elementary functions such as trigonometric functions, hyperbolic functions, exponentials, logarithm, and square-roots [2]. This makes CORDIC particularly appealing to applications which require intensive evaluations of elementary functions.

Hardware implementation of CORDIC needs only shifters and adders, the simple hardware is very attractive. Haviland and Tuszynski [3] designed and implemented a full function CORDIC chip in 1980. T. Y. Sung and Y. H. Hu [4], [5] proposed doubly pipelined CORDIC array processors applying on digital signal processing and also implemented a CORDIC chip in 1987. In 1992, Y. H. Hu, H.E. Liao [6] proposed CORDIC algorithm applying on adaptive lattice filter. In 1985, T. Y. Sung and Y. H. Hu proposed CORDIC array processor applying on solving Toeplitz systems [7]. In 1987, T. Y. Sung and Y. H. Hu also proposed CORDIC arithmetic applying on Kalman filters [8]. S. S. Nikolaidis proposed CORDIC architecture applying on all-pass filters. J.

K. Hwang [9] proposed VLSI architecture of adaptive lattice filter basing on CORDIC, they improved the computation accuracy. In 1993, Metafas and Goutis [10] proposed floating-point computation architecture of CORDIC. X. Hu, Harber, and Bass [11] expanded convergence range of CORDIC and reduced input limits.

CORDIC is applied in signal processing widely today, but the research of computation error and accuracy analysis is rarely. Many designers of CORDIC algorithm might not understand the role of computation error in CORDIC design, they had just increased number of iteration, and even number of bits of mantissa is less considered. Error analysis of CORDIC algorithm was proposed by Y. H. Hu [12] who changed conventional design concept of CORDIC algorithm, but neglected an influence factor which was discussed about error of $\delta(i)$ sequence by X. Hu [13] and Kota [14]. Kota had just discussed in circular coordinate system, but the error analysis of CORDIC was not completely.

According to improving performance and saving hardware, the number of iteration and word length should be concerned. These concerns should cause computation error. Especially, the model of fixed point error analysis of the CORDIC algorithm would be proposed, and prediction of computation error with number of iteration and word length could be performed definitely. As knowing where the errors are, we can design the hardware with the error consideration in order to get the best cost-performance and the desired outcomes. An optimal design of CORDIC arithmetic for digital signal processing applications would be achieved.

2. The CORDIC Algorithm

Basic formula of CORDIC is as follows [2]:

$$x(i+1)=x(i)+m \delta(i)y(i)2^{-s(m,i)}, \quad (1)$$

$$y(i+1)=y(i)-\delta(i)x(i)2^{-s(m,i)}, \quad (2)$$

$$z(i+1)=z(i)-\delta(i)\alpha_m(i), \quad (3)$$

where $s(m,i)$ is a non-decreasing integer “shift sequence” satisfying

$$s(m,i) \leq s(m,i+1) \leq s(m,i)+1, \text{ and}$$

$$\alpha_m(i)=m^{-1/2}\tan^{-1}[m^{-1/2}2^{-s(m,i)}] \quad (4)$$

is the angle rotated in i -th iteration. $m=+1, -1$ or 0 determines whether the coordinate system is circular, hyperbolic, or linear, respectively. $\delta(i)$ determines the direction of rotation angle, $+1$ and -1 represent different direction respectively.

3. Domain of Convergence of CORDIC

CORDIC is a numerical algorithm with iterations, the domain and input maximum angle of convergence must be concerned. Walther proposed converging criterion [2], but he had not proved it yet. Now, we prove this criterion and establish two theorems for domain of convergence.

Theorem 1: Converging criterion :

$$\alpha(i) - \sum_{j=i+1}^{n-1} \alpha(j) \leq \alpha(n-1), \text{ for } i \geq 0.$$

Proof: In case of linear coordinate system, rotation angle α is represented as

$$\alpha(i) = 2^{-i}, \quad \alpha(i+1) = 2^{-(i+1)},$$

$$\text{then } 2^{-i} - 2^{-(i+1)} = 2^{-(i+1)} \leq 2^{-(i+1)}.$$

$$\text{Hence, } \alpha(i) - \alpha(i+1) \leq \alpha(i+1).$$

In case of circular coordinate system, rotation angle $\alpha(i)$ and $\alpha(i+1)$ are represented as $\alpha(i) = \tan^{-1}2^{-i} = \theta$ and $\alpha(i+1) = \tan^{-1}2^{-(i+1)} = \varphi$ respectively.

$$\tan(\theta - \varphi) \leq \tan \varphi = \frac{\tan \theta - \tan \varphi}{1 + \tan \theta \tan \varphi}$$

$$= \frac{2^{-(i+1)}}{1+2^{-2i-1}} \leq 2^{-(i+1)}, \text{ and } \theta - \varphi \leq \varphi.$$

Hence, $\alpha(i) - \alpha(i+1) \leq \alpha(i+1)$.

In case of hyperbolic coordinate system, rotation angle $\alpha(i)$ and $\alpha(i+1)$ are represented as

$$\alpha(i) = \tanh^{-1} 2^{-i} = \theta \quad \text{and} \quad \alpha(i+1) = \tanh^{-1} 2^{-(i+1)} = \varphi$$

respectively.

$$\tanh^{-1} 2^{-i} - \tanh^{-1} 2^{-(i+1)} \leq \tanh^{-1} 2^{-(i+1)}.$$

Hence, $\alpha(i) - \alpha(i+1) \leq \alpha(i+1)$.

According to above derivations, we summarize as

$$\alpha(i) - \alpha(i+1) - \alpha(i+2) - \dots - \alpha(n-1) \leq \alpha(n-1).$$

Finally, the convergence criterion is represented as

$$\alpha(i) - \sum_{j=i+1}^{n-1} \alpha(j) \leq \alpha(n-1). \quad (5) \quad \square$$

Lemma 1: $|z_{\max}(0)| = \alpha(n-1) + \sum_{j=0}^{n-1} \alpha(j)$.

Proof: Based on convergence criterion, the rotation angle should be ranged from 0 to $\alpha(n-1)$.

$|z(0)|$ could be represented as

$$|z(0)| \leq \sum_{j=0}^{n-1} \alpha(j) + \alpha(n-1).$$

Since $z(0)$ can be represented as range of input

angle, $|z_{\max}(0)|$ can be represented as

$$|z_{\max}(0)| = \alpha(n-1) + \sum_{j=0}^{n-1} \alpha(j). \quad (6)$$

Hence, $|z_{\max}(0)|$ represents a maximum input angle

or a maximum convergence angle. $\square$

Theorem 2: $|z(i)| \leq \alpha(n-1) + \sum_{j=i}^{n-1} \alpha(j)$, for $i \geq 0$. (7)

Proof: According to theorem 1, we obtain

$$|z(i+1)| < \alpha(n-1) + \sum_{j=i+1}^{n-1} \alpha(j).$$

Hence, $|z(i)| \leq \alpha(n-1) + \sum_{j=i}^{n-1} \alpha(j)$. $\square$

Lemma 2: Convergence range can be expanded [11];

in linear, circular, and hyperbolic coordinate system,

$\theta_{\max}$ can be represented as $\theta_{\max} = 2^{M+1}$,

$$\theta_{\max} = \tan^{-1}(2^{-n}) + 2\tan^{-1}(2^0) + \sum_{j=0}^n \tan^{-1} 2^{-j}, \text{ and}$$

$$\theta_{\max} = \sum_{j=-M}^0 \tanh^{-1}(1 - 2^{-2^{j+1}}) + \tanh^{-1} 2^{-n} +$$

$$\sum_{j=1}^n \tanh^{-1}(2^{-j}), \text{ respectively.}$$

Proof: In linear coordinate system, $\frac{\alpha(i)}{\alpha(i+1)} \leq 2$

satisfies convergence criterion, such that

$$|z(0)| \leq \alpha(n) + \sum_{j=-M}^n \alpha(j). \quad (8)$$

Hence, $|z(0)|$ is expressed as

$$|z(0)| \leq \theta_{\max} = 2^{M+1}. \quad (9)$$

In circular coordinate system, let iteration sequence of j is 0, 0, 0, 1, 2, ..., n , and

$$\frac{\alpha(i)}{\alpha(i+1)} \leq 2 \text{ satisfies convergence criterion. The}$$

design of CORDIC hardware avoids left shift in order to simplify hardware, so that iteration sequence of j is chosen as 0, 0, 0, 1, 2, ..., n . $\theta_{\max}$ is represented as [8]

$$\begin{aligned} \theta_{\max} = & \tan^{-1}(2^{-n}) + 2\tan^{-1}(2^0) \\ & + \sum_{j=0}^n \tan^{-1} 2^{-j} \cong 3.3141(189.9^\circ) > 180^\circ. \end{aligned} \quad (10)$$

According to eq. (10), the convergence range is expanded to $\pm 180^\circ$. Hence, the range of input can be unlimited.

In case of hyperbolic coordinate system, when j is a negative number, $\tanh^{-1}(2^{-j})$ becomes a complex number, and the number of negative iteration should be reduced in order to improve computation speed [11]. Now,

$$\alpha(i) = \tanh^{-1}(1 - 2^{-2^{i+1}}), \text{ for } i \leq 0,$$

should be chosen.

$$\frac{\alpha(i)}{\alpha(i+1)} = \frac{\tanh^{-1}(1-2^{-2^{-i+1}})}{\tanh^{-1}(1-2^{-2^{-(i+1)+1}})} \leq 2, \text{ for } i \leq 0,$$

also satisfies convergence criterion, such that

$$\theta_{\max} = \sum_{j=-M}^0 \tanh^{-1}(1-2^{-2^{j+1}}) + \tanh^{-1}2^{-n} + \sum_{j=1}^n \tanh^{-1}(2^{-j}). \quad (11) \quad \square$$

In this paper, the angle α of hyperbolic coordinate system with expanding convergence range would be defined as

$$\alpha(i) = \tanh^{-1}(1-2^{-2^{-i+1}}), \text{ for } i \leq 0, \quad (12)$$

$$\alpha(i) = \tanh^{-1}(1-2^{-i}), \text{ for } i > 0. \quad (13)$$

4. Error Analysis of Conventional CORDIC Algorithm

The basic CORDIC algorithm is specified in eqs. (1), (2) and (3), vector $\mathbf{v}$ is defined as $[x \ y]^T$. Three kinds of representation form are defined [3]; one is a mathematical representation with infinite precision, as mathematically defined, values of this representation are defined as $\tilde{x}, \tilde{y}, \tilde{z}, \tilde{\mathbf{v}}, \theta$, another one is a hypothetical representation, every iteration uses the same $\delta(i)$ obtained in a real implementation and word length is performed with an infinite number of bits, values of this representation are defined as $x, y, z, \mathbf{v}, \alpha$, and the other one is a real-world representation, iteration and word length are performed both finite, values of this representation are defined as $\hat{x}, \hat{y}, \hat{z}, \hat{\mathbf{v}}, \varphi$.

In rotation mode, truncation error is represented as

$$\|\mathbf{v}(n) - \hat{\mathbf{v}}(n)\| \text{ which is a norm [3].}$$

$$\|\mathbf{v}(n) - \hat{\mathbf{v}}(n)\| \leq \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| + \|\mathbf{v}(n) - \hat{\mathbf{v}}(n)\| \quad (14)$$

eq. (14) represents this overall approximation error [3].

In vectoring mode, truncation error is represented as $|\alpha - \varphi|$. Hence, this overall approximation error is represented as

$$|\theta - \varphi| \leq |\theta - \alpha| + |\alpha - \varphi|. \quad (15)$$

In case of rotation mode, absolute and relative approximation error are represented as

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| \quad \text{and} \quad \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\|$$

respectively. Hence, results of error analysis only depend on “ n ” iterations, and b fractional binary digits, and “ b_v ” binary digits of mantissa of x, y . If n, b_v and b are determined, the result of error would be computed.

According to $\left| \theta - \sum_{i=0}^{n-1} \delta(i) \alpha(i) \right| < \alpha(n-1)$ [2], we

obtain

$$\left| \theta - \sum_{i=0}^{n-1} \delta(i) Q[\alpha(i)] \right| < Q[\alpha(n-1)], \quad (16)$$

where $Q[\bullet]$ denotes the operator of quantization, and $Q[x(i)]$ is defined as $Q[x(i)] = x(i) - e(i)$, where $e(i)$ is a truncation error and $e(i) < 2^{-b}$.

Hence,

$$\alpha - \sum_{i=0}^{n-1} \delta(i) Q[\alpha(i)] = \sum_{i=0}^{n-1} \delta(i) \mu(i),$$

where $\alpha = \sum_{i=0}^{n-1} \alpha(i)$, and $\mu(i) = \alpha(i) - Q[\alpha(i)]$. (17)

Assuming that μ is a maximum error for each $\mu(i)$, we obtain

$$\left| \alpha - \sum_{i=0}^{n-1} \delta(i) Q[\alpha(i)] \right| < n\mu. \quad (18)$$

Hence,

$$|\theta - \alpha| \leq Q[\alpha(n-1)] + n\mu. \quad (19)$$

Due to truncation error, $Q[\alpha(n-1)]$ is obtained as $Q[\alpha(n-1)] \leq \alpha(n-1)$, and $\mu = 2^{-b}$, where b is

fractional binary digits.

Theorem 3: In circular, hyperbolic and linear coordinate system, the relative approximation error of rotation mode are represented as

$$\begin{aligned} \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| &\leq \tan^{-1} 2^{-(n+1)} + n \cdot 2^{-b}, \\ \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| &\leq \left| e^{\tanh^{-1} 2^{-(n-r)} + n \cdot 2^{-b}} - 1 \right|, \end{aligned}$$

and $\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq 2^{-n} + n \cdot 2^{-b}$, respectively.

Proof: In circular coordinate system, $\mathbf{M}$ is defined as

$$\mathbf{M} = \begin{bmatrix} \cos \theta - \cos \alpha & \sin \theta - \sin \alpha \\ -(\sin \theta - \sin \alpha) & \cos \theta - \cos \alpha \end{bmatrix}.$$

Hence, maximum eigenvalue of $\mathbf{M}^T \mathbf{M}$ is $2 - 2\cos(\theta - \alpha)$, and $\|\mathbf{M}\| = \sqrt{2 - 2\cos(\theta - \alpha)}$.

$$\text{Hence, } \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| \leq K_l \|\mathbf{v}(0)\| |\theta - \alpha|. \quad (20)$$

From eq. (19), $\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\|$ should be modified

$$\text{as } \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| \leq K_l \|\mathbf{v}(0)\| (Q[\alpha(n-1)] + n\mu).$$

According to eq. (20), we obtain

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq \tan^{-1} 2^{-(n+1)} + n \cdot 2^{-b}. \quad (21)$$

In hyperbolic coordinate system,

$$\|\mathbf{M}\| = |\cosh \theta - \cosh \alpha + \sinh \theta - \sinh \alpha| = |e^{\theta} - e^{\alpha}|,$$

$$\text{and } \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| = K_1 |e^{|\theta|} - e^{|\alpha|}| \|\mathbf{v}(0)\|. \quad (22)$$

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq |e^{|\alpha| + |\theta|} - 1| \leq |e^{Q[\alpha(n-1)] + n\mu} - 1|.$$

$$\text{Hence, } \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq |e^{\tanh^{-1} 2^{-(n-r)} + n \cdot 2^{-b}} - 1|, \quad (23)$$

where r is the number of repeated iteration.

In linear coordinate system,

$$\begin{aligned} \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| &= \left\| \begin{bmatrix} 0 & 0 \\ -(\theta - \alpha) & 0 \end{bmatrix} \right\| \|\mathbf{v}(0)\| \\ &= |\theta - \alpha| \|\mathbf{v}(0)\|. \end{aligned} \quad (24)$$

$\|\tilde{\mathbf{v}}(n)\|$ can be represented as

$$\|\tilde{\mathbf{v}}(n)\| = \left[\frac{\theta^2 + \sqrt{\theta^4 + 4\theta^2} + 2}{2} \right]^{\frac{1}{2}} \|\mathbf{v}(0)\|. \quad (25)$$

From eqs. (24) and (25), we obtain

$$0.29 |\theta - \alpha| < \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq |\theta - \alpha|, \quad (26)$$

where $-\pi \leq \theta \leq \pi$.

Finally, we obtain

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq 2^{-n} + n \cdot 2^{-b}. \quad (27) \quad \square$$

Theorem 4: In circular, hyperbolic, linear coordinate system, the fixed point truncation error of rotation mode are represented

$$\text{as } \|Q[\hat{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left[1 + \sum_{j=1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1 + 2^{-2i}} \right) \right],$$

$$\|Q[\hat{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left\{ 1 + \sum_{j=1}^{n-1} \left[\prod_{i=j}^{n-1} \sqrt{1 + 2^{-2i}} \right] \cdot e^{\sum_{i=j}^{n-1} \tanh^{-1} 2^{-i}} \right\},$$

$$\text{and } |Q[\hat{y}(n)] - y(n)| = \sum_{i=2}^{n+1} |e(i)| = n\varepsilon, \text{ respectively.}$$

Proof: In circular coordinate system, the iteration process relates the output vector $\mathbf{v}(i+1)$ to its input vector $\mathbf{v}(i)$ as $\mathbf{v}(i+1) = \mathbf{p}_1(i)\mathbf{v}(i)$, where $\mathbf{p}_1(i)$ is represented the rotation in i -th iteration as

$$\mathbf{p}_1(i) = \begin{bmatrix} 1 & \delta(i)2^{-i} \\ -\delta(i)2^{-i} & 1 \end{bmatrix}.$$

Hence,

$$\|Q[\hat{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq |e(n)| + \sum_{j=1}^{n-1} \left\| \prod_{i=j}^{n-1} \mathbf{p}_1(i) \right\| \cdot |e(j)|. \quad (28)$$

$$\|Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left[1 + \sum_{j=1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1+2^{-2i}} \right) \right], \quad (29)$$

where $|e(i)| = \sqrt{e_x^2(i) + e_y^2(i)} \leq \sqrt{2}\varepsilon$.

In hyperbolic coordinate system, from eq. (28), $Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)$ is represented as

$$\|Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left\{ 1 + \sum_{j=1}^{n-1} \left[\prod_{i=j}^{n-1} \sqrt{1-2^{-2i}} \right] \cdot e^{\sum_{i=j}^{n-1} \tan^{-1} 2^{-i}} \right\}. \quad (30)$$

In linear coordinate system,

$Q[\tilde{y}(i+1)] = Q[\tilde{y}(i)] - \delta(i)\alpha(i)Q[\tilde{x}(i)] - e(i+1)$, and

$$Q[\tilde{y}(n)] = y(0) - \alpha \cdot x(0) - \sum_{i=1}^n e(i). \quad (31)$$

$$\text{Hence, } |Q[\tilde{y}(n)] - y(n)| = \sum_{i=2}^{n+1} |e(i)| = n\varepsilon. \quad (32) \quad \square$$

Theorem 5: In circular, hyperbolic and linear coordinate system, the approximation error of vectoring mode are represented

as $|\theta - \alpha| < \sin^{-1}(Q[\alpha(n-1)] + n\mu)$,

$|\theta - \alpha| \leq \sinh^{-1} |e^{|\alpha|} - e^{|\theta|}| \leq \sinh^{-1} |e^{|\alpha-\theta|} - 1|$, and

$|\theta - \alpha| \leq Q[\alpha(n)] + n\mu$, respectively.

Proof: In circular coordinate system, using eq. (20) and $\tilde{\mathbf{y}}(n) = 0$, we obtain

$$|\theta - \alpha| < \sin^{-1}(Q[\alpha(n-1)] + n\mu). \quad (33)$$

In hyperbolic coordinate system, using eq. (22) and $\tilde{\mathbf{y}}(n) = 0$, we obtain

$$|\theta - \alpha| \leq \sinh^{-1} |e^{|\alpha|} - e^{|\theta|}| \leq \sinh^{-1} |e^{|\alpha-\theta|} - 1|, \quad (34)$$

In linear coordinate system, from eqs. (18), (24) and $\tilde{\mathbf{y}}(n) = 0$, we obtain

$$|y(n)| \leq |x(0)|(Q[\alpha(n-1)] + n\mu). \quad (35)$$

From eq. (35), and $|\theta - \alpha| = \left| \frac{y(n)}{x(0)} \right|$, we obtain

$$|\theta - \alpha| \leq Q[\alpha(n)] + n\mu. \quad (36) \quad \square$$

Theorem 6: In circular, hyperbolic and linear coordinate system, the fixed point truncation error of

vectoring mode are represented as

$$|\alpha - \phi| = \left| \alpha - \sum_{i=0}^{n-1} \delta(i)Q[\alpha(i)] \right| \leq n\mu, \text{ where } \mu = 2^{-b}.$$

Proof: In circular coordinate system, from eqs. (17) and (18), we obtain

$$|\alpha - \phi| = \left| \alpha - \sum_{i=0}^{n-1} \delta(i)Q[\alpha(i)] \right| \leq n\mu, \text{ where } \mu = 2^{-b}. \quad (37)$$

Truncation errors of hyperbolic and linear coordinate system are the same as eq. (37). $\square$

5. Error Analysis of CORDIC Algorithm with Expanding Convergence Range

The CORDIC algorithm with expanding convergence range had been proposed [11], the error analysis of linear, circular and hyperbolic coordinate system is discussed in order.

In case of linear coordinate system, the sequence of iteration is $-M, -M+1, \dots, 0, 1, 2, \dots, n$. Convergence range is expanded; $|z_{in}| \leq 1$ is replaced by $|z_{in}| \leq 2^{M+1}$, $|y_{in}/x_{in}| \leq 1$ is replaced by $|y_{in}/x_{in}| \leq 2^{M+1}$, z is an integer of 2^{M+1} , b is replaced by $b-M-1$, the truncation error with expanding convergence range is 2^{M+1} times of that without expanding convergence range. Besides, N -iteration is replaced by $(N+M+1)$ -iteration.

Lemma 3: In linear, circular, and hyperbolic coordinate system, the relative approximation error of rotation mode are represented as

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq Q[\alpha(n)] + (N+M+1) \cdot 2^{-b+M+1},$$

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| < (Q[\alpha(n-1)] + (N+2) \cdot 2^{-b+1}),$$

$$\text{and } \|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq \left| e^{Q[\alpha(n-r)] + (N+M+1)\mu} - 1 \right|,$$

respectively.

Proof: In linear coordinate system, from *theorem 3*, n is replaced by $(N + M + 1)$, and 2^{-b} is replaced by 2^{-b+M+1} , we obtain

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq Q[\alpha(n)] + (N+M+1) \cdot 2^{-b+M+1}. \quad (38)$$

In circular coordinate system, the convergence angle exceeds 180° , it makes input range unlimited, the sequence of iteration is $0, 0, 0, 1, 2, \dots, N$. Hence,

$$|z_{in}| \leq 3.3141 (189.9^\circ), \text{ and } K_1' = \prod_{i=0,0,0}^N \sqrt{1+2^{-2i}} = 2K_1.$$

For fixed word length, scale factor with expanding convergence range is two times of that without expanding convergence range, it causes that truncation error with expanding convergence range becomes two times of that without expanding convergence range, such that $\varepsilon = 2^{-b+1}$, $z = 3.3143$.

Hence, $b' = b - 1$, $\mu = 2^{-b'} = 2^{-b+1}$.

From eq. (38), $n = N + 2$, $\mu = 2^{-b+1}$, we obtain

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| < (Q[\alpha(n-1)] + (N+2) \cdot 2^{-b+1}). \quad (39)$$

In hyperbolic coordinate system, the analysis of approximation error of rotation mode is discussed as follows:

According to eq. (11), $\theta_{\max} = 2.0911550745$ ($M=0$), 3.8081486768 ($M=1$), 6.9263334719 ($M=2$), 12.8180806919 ($M=3$), 24.25550091711 ($M=4$).

$\theta_{\max}$ becomes larger, the number of bits of mantissa is reduced, such that $b' = b - M$, the truncation error with expanding convergence range becomes 2^M times of that of conventional CORDIC. Scale factor

$$K_h = \left[\prod_{i=-M}^0 \sqrt{1 - (1 - 2^{-2^{i+1}})^2} \right] \left[\prod_{i=1}^N \sqrt{1 - 2^{-2i}} \right].$$

$$K_h = 0.6614378278 < 2^0 \text{ } (M=0), 0.2301706229 < 2^{-2} \text{ } (M=1), 0.0203245237 < 2^{-5} \text{ } (M=2), 0.0001122778 < 2^{-13} \text{ } (M=3), 0.0000000024 < 2^{-28} \text{ } (M=4).$$

If $M=2$, number of bits of mantissa would be lose 5-bit. Hence, M is larger; the truncation error becomes much larger. Because of $b' < b$, $\varepsilon = 2^{-b'}$ becomes larger.

The sequence of iteration is $-M, -M+1, \dots, 0, 1, 2, 3, \dots, N$. n is replaced by $(N+M+1)$

and μ is represented as $\mu = 2^{-b'}$.

From *Theorem 3*, we obtain

$$\|\tilde{\mathbf{v}}(n) - \mathbf{v}(n)\| / \|\tilde{\mathbf{v}}(n)\| \leq \left| e^{Q[\alpha(n-r)] + (N+M+1)\mu} - 1 \right|, \quad (40)$$

where $\mu = 2^{-b'}$ and $b' = b - M$. $\square$

Lemma 4: In linear, circular, and hyperbolic coordinate system, the fixed point truncation error of rotation mode is represented as $|Q[\hat{y}(n)] - y(n)| = (N + M + 1)\varepsilon$,

$$|Q[\hat{\mathbf{v}}(n)] - \mathbf{v}(n)| \leq \sqrt{2}\varepsilon \left[1 + \sum_{j=0,0,1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1+2^{-2i}} \right) \right],$$

$$|Q[\hat{\mathbf{v}}(n)] - \mathbf{v}(n)| \leq \sqrt{2}\varepsilon \left\{ 1 + \sum_{j=-M+1}^n \left[\prod_{i=j}^n \sqrt{1-2^{-2i}} \right] \cdot e^{\sum_{i=j}^n \alpha(i)} \right\},$$

respectively.

Proof: In linear coordinate system, from *Theorem 4*, $Q[\hat{y}(n)]$ is represented

$$\text{as } Q[\hat{y}(n)] = y(-M) - \alpha_M \cdot x(-M) - \sum_{i=-M}^n e(i).$$

$$\text{Hence, } Q[\hat{y}(n)] = y(n) - \sum_{i=-M}^n e(i).$$

Assume that $e(i) = \varepsilon$, $|Q[\hat{y}(n)] - y(n)|$ is represented as $|Q[\hat{y}(n)] - y(n)| = (N + M + 1)\varepsilon$. (41)

In circular coordinate system, from *Theorem 4*, we obtain

$$\|Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left[1 + \sum_{j=0,0,1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1+2^{-2i}} \right) \right]. \quad (42)$$

In hyperbolic coordinate system, from *Theorem 4*, $\|Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)\|$ is represented as

$$\|Q[\tilde{\mathbf{v}}(n)] - \mathbf{v}(n)\| \leq \sqrt{2}\varepsilon \left\{ 1 + \sum_{j=-M+1}^n \left[\prod_{i=j}^n \sqrt{1-2^{-2i}} \right] \cdot e^{\sum_{i=j}^n \alpha(i)} \right\}. \quad (43) \square$$

Lemma 5: In linear coordinate system, the approximation error of vectoring mode is represented as $|\theta - \alpha| = Q[\alpha(n)] + (N + M + 1) \cdot 2^{-b+M+1}$, $|\theta - \alpha| < \sin^{-1}(Q[\alpha(n-1)] + (N + 2) \cdot 2^{-b+1})$, and $|\theta - \alpha| \leq \sinh^{-1}(e^{Q[\alpha(n-r)] + (N+M+1)\mu'} - 1)$,

where $\mu' = 2^{-b'}$, $b' = b - M$, respectively.

Proof: In linear coordinate system, from *Theorem 5*, $|\theta - \alpha| = Q[\alpha(n)] + n\mu$, n is replaced by $(N + M + 1)$, and 2^{-b} is replaced by 2^{-b+M+1} , we obtain $|\theta - \alpha| = Q[\alpha(n)] + (N + M + 1) \cdot 2^{-b+M+1}$. (44)

In circular coordinate system, from *Theorem 5* and *Lemma 3*, $|\theta - \alpha|$ is represented as $|\theta - \alpha| < \sin^{-1}(Q[\alpha(n-1)] + (N + 2) \cdot 2^{-b+1})$. (45)

In hyperbolic coordinate system, according to *Theorem 5*, and expanding convergence range, $|\theta - \alpha|$ is represented as

$$|\theta - \alpha| \leq \sinh^{-1}(e^{Q[\alpha(n-r)] + (N+M+1)\mu'} - 1), \quad (46)$$

where $\mu' = 2^{-b'}$, $b' = b - M$. □

Lemma 6: In linear, circular and hyperbolic coordinate system, the fixed point truncation error of vectoring mode are represented

as $|\alpha - \phi| \leq (N + M + 1)\mu_M$, where $\mu_M = 2^{-b'}$,

where $b' = b - (M + 1)$, $|\alpha - \phi| \leq (N + 2)\mu_0$,

where $\mu_0 = 2^{-b'}$, $b' = b - 1$, and

$$|\theta - \alpha| \leq (N + M + 1)\mu.$$

Proof: In linear coordinate system, $|\alpha - \phi|$ is

$$\text{represented as } |\alpha - \phi| = \left| \sum_{i=-M}^n \delta(i)\mu(i) \right| \leq \sum_{i=-M}^n \mu(i).$$

Assume that $\mu(i) = \mu_M$, one has

$$|\alpha - \phi| \leq (N + M + 1)\mu_M, \quad \text{where } \mu_M = 2^{-b'},$$

where $b' = b - (M + 1)$.

In circular coordinate system, the sequence of iteration is changed, such that $|\alpha - \phi| \leq \sum_{i=0,0,0,1}^{n-1} \mu(i)$.

Assume that $\mu(i) = \mu_0$, one has

$$|\alpha - \phi| \leq (N + 2)\mu_0, \quad (47)$$

where $\mu_0 = 2^{-b'}$, $b' = b - 1$.

In hyperbolic coordinate system, due to expanding convergence range, $|\theta - \alpha|$ is represented as

$$|\theta - \alpha| \leq \left| \sum_{i=-M}^n \delta(i)\mu(i) \right| \leq \sum_{i=-M}^n \mu(i),$$

where $\mu' = 2^{-b'}$, $b' = b - M$.

$$|\theta - \alpha| \text{ can be represented as } |\theta - \alpha| \leq (N + M + 1)\mu. \quad (48) \square$$

6. Summary and Conclusion

The approximation error of rotation mode is represented relative error model; this error is related to iterations and word length. The truncation error is not related to initial values; hence this error is represented absolute error model. If the overall error is represented as relative error model, the truncation error is divided by ideal value $\|\tilde{\mathbf{v}}(n)\|$. If the overall error is represented as absolute error model, the approximation error of rotation mode is multiplied by

ideal value $\|\tilde{v}(n)\|$.

In this paper, we analyze the computation error of CORDIC for different type, and derive error analysis formulas which consider error of $\delta(i)$ sequence and expanding convergence range. The result of overall error for each type is shown in Tables 1 and 2. Comparison of Sung, Y. H. Hu (Hu) [12], and K. Kota (Kota) [14] research is shown in Table 3.

The computation error of CORDIC with expanding convergence range is discussed and analyzed, and error analysis formulas are also

developed. We establish complete error model involving error of $\delta(i)$ sequence [12], which Kota had just discussed in circular coordinate system [14]. All of error analysis formulas, which consist of truncation error and approximation error, are only related to word length and iteration number. According to analysis formulas and design reference tables, we can develop a cost-effective special purpose processor with CORDIC. Small area and high performance of CORDIC processor will be achieved.

Linear Coordinate System:
a. Rotation Mode: $[2^{-n} + n \cdot 2^{-b}] + n \cdot 2^{-b_v}$
b. Vectoring Mode: $[2^{-n} + n \cdot 2^{-b}] + n \cdot 2^{-b}$
Circular Coordinate System:
a. Rotation Mode: $[\tan^{-1} 2^{-(n-1)} + n \cdot 2^{-b}] + \sqrt{2} \cdot 2^{-b_v} \left[1 + \sum_{j=1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1+2^{-2i}} \right) \right]$
b. Vectoring Mode: $[\sin^{-1} (\tan^{-1} 2^{-(n-1)} + n \cdot 2^{-b})] + n \cdot 2^{-b}$
Hyperbolic Coordinate System:
a. Rotation Mode: $\left[\left e^{\tanh^{-1} 2^{-(n-r)} + n 2^{-b}} - 1 \right + \sqrt{2} \cdot 2^{-b_v} \left\{ 1 + \sum_{j=2}^n \left[\prod_{i=j}^n \sqrt{1-2^{-2i}} \right] \cdot e^{\sum_{i=j}^n \tanh^{-1} 2^{-i}} \right\} \right]$
b. Vectoring Mode: $[\sinh^{-1} (e^{\tanh^{-1} 2^{-(n-r)} + n 2^{-b}} - 1)] + n \cdot 2^{-b}$

Table 1 Overall error of conventional CORDIC algorithm ($\|\tilde{v}(n)\| = 1$)

Linear Coordinate System:
a. Rotation Mode: $[2^{-n} + (n + M + 1) \cdot 2^{-b+M+1}] + (n + M + 1) \cdot 2^{-b_v}$
b. Vectoring Mode: $[2^{-n} + (n + M + 1) \cdot 2^{-b+M+1}] + (n + M + 1) \cdot 2^{-b+M+1}$
Circular Coordinate System:
a. Rotation Mode: $[\tan^{-1} 2^{-(n-1)} + (n + 2) \cdot 2^{-b+1}] + \sqrt{2} \cdot 2^{-b_v+1} \left[1 + \sum_{j=0,0,1}^{n-1} \left(\prod_{i=j}^{n-1} \sqrt{1+2^{-2i}} \right) \right]$
b. Vectoring Mode: $[\sin^{-1} (\tan^{-1} 2^{-(n-1)} + (n + 2) \cdot 2^{-b+1})] + (n + 2) \cdot 2^{-b+1}$
Hyperbolic Coordinate System:
a. Rotation Mode: $\left[\left e^{\tanh^{-1} 2^{-(n-r)} + (n+M+1) 2^{-b+M}} - 1 \right + \sqrt{2} \cdot 2^{-b_v'} \left\{ 1 + \sum_{j=M+1}^n \left[\prod_{i=j}^n \sqrt{1-2^{-2i}} \right] \cdot e^{\sum_{i=j}^n \tanh^{-1} 2^{-i}} \right\} \right]$
b. Vectoring Mode: $[\sinh^{-1} (e^{\tanh^{-1} 2^{-(n-r)} + (n+M+1) 2^{-b+M}} - 1)] + (n + M + 1) \cdot 2^{-b+M}$

Table 2 Overall error of CORDIC algorithm with expanding convergence range ($\|\tilde{v}(n)\| = 1$)

Type of error analysis of CORDIC	Sung	Hu[9]	Kota[11]
Conventional CORDIC Algorithm	Yes	*	@
CORDIC Algorithm with Expanding Convergence Range	Yes	No.	No.

Table 3 Comparison of related researches (*: Neglect error of $\delta(i)$ sequence. @: Only circular coordinate system)

References

- [1] J. E. Volder, "CORDIC Trigonometric Computing Technique", *IRE Transactions on Electronic Computers*, Vol. EC-8, No. 3, pp.330-334, 1959.
- [2] J. S. Walther, "A Unified Algorithms for Elementary Functions", *Proc. Spring Joint Computer Conf.*, pp. 379-385, 1979.
- [3] G. L. Haviland and A. A. Tuszynski, "A CORDIC Arithmetic Processor Chip", *IEEE Transactions on Computers*, Vol. 29, No. 2, pp.68-79, 1980.
- [4] T. Y. Sung, T. M. Parng, Y. H. Hu, "Design and Implementation of a VLSI CORDIC Processor," *IEEE Int'l Symp. on Circuits and Systems*, pp.934-935, May 1986.
- [5] T. Y. Sung, Y. H. Hu, H. J. Yu, "Doubly Pipelined CORDIC Array for Digital Signal Processing Algorithms", *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, Vol. 2, pp.1169-1172, Apr. 1986.
- [6] Y. H. Hu, H. E. Liao, "CALF: A CORDIC Adaptive Lattice Filter", *IEEE Transactions on Signal Processing*, Vol. 40, No. 4, pp.990-993, 1992.
- [7] T. Y. Sung, Y. H. Hu, H. J. Yu, "Doubly Pipelined CORDIC Array Processor for Solving Toeplitz Systems," *Proc. of the Twenty-Third Annual Allerton Conf. on Communication, Control, and Computing*, Monticello, IL., USA, Oct. 1985.
- [8] T. Y. Sung, Y. H. Hu, "Parallel VLSI Implementation of Kalman Filter," *IEEE Transactions on Aerospace and Electronic Systems*, Vol.23, No.2, pp.215-224, March, 1987.
- [9] S. S. Nikolaidis, D. E. Metafas, C. E. Goutis, "CORDIC Based Pipelined Architecture for All-Pass Filters", *1993 IEEE International Symposium on Circuits and systems*, Vol. 3, pp.1917-1920, 1993.
- [10] D. E. Metafas, C.E. Goutis, "A Floating-Point Advanced CORDIC Processing, *Journal of VLSI Signal processing*, Vol. 10, pp.53-65, 1995.
- [11] X. Hu, R. G. Harber, S. C. Bass, "Expanding the range of the Convergence of the CORDIC Algorithm", *IEEE Transactions on Computers*, Vol. 40, No. 1, pp.13-21, 1991.
- [12] Y. H. Hu, "The Quantization Effects of the CORDIC Algorithm", *IEEE Transactions on Signal Processing*, Vol. 40, No. 4, pp.834-844, 1992.
- [13] X. Hu, S. C. Bass, "A Neglected Error Source in the CORDIC Algorithm", *1993 IEEE International Symposium on Circuits and Systems*, Vol. 1, pp.766-769, 1993.
- [14] K. Kota, J. R. Callaro, "Numerical Accuracy and Hardware Trade-offs for CORDIC Arithmetic for Special-Purpose Processors", *IEEE Transactions on Computers*, Vol. 42, No. 7, pp.769-779, 1993.

使用二元樹表示法與基因演算法合成被動濾波電路

侯浩生¹ 張守進² 蘇炎坤

國立成功大學微電子工程研究所

¹hsh@mail.njtc.edu.tw

²changsj@mail.ncku.edu.tw

摘要

本文提出一種二元樹結構用以表示階梯狀的 RLC 電路，並描述如何透過走訪這種樹狀結構來作電路分析。根據這種樹狀表示法，作者提出一個基因演算法來合成被動濾波電路，這種方法可以同時進行電路元件個數、電路拓撲結構及元件值的演化。針對合成低通與高通兩種濾波電路所做的實驗顯示，這種方法是有效的而且比以前所曾提出的方法還要快。

第一節、簡介

所謂的電腦輔助設計 (CAD) 就是利用電腦強大的運算能力來作自動化的設計動作，而 CAD 在電路設計上的應用即是所謂的電路合成 (circuit synthesis)。電路合成的過程在本質上可以視為是一種最佳化 (optimization) 的過程，即在所有可能的元件、元件值及其拓撲組合中去找尋符合規格的最佳解。而作為最佳化問題的解決方案之一，取法自生物演化法則的基因演算法 (Genetic Algorithm, 以下簡稱 GA)，在許多領域的最佳化問題的應用上取得了極大的成功 [1]，其中之一即是電路合成的應用 [2]。

GA 的應用牽涉到三個主要部分：1. 表示法 (representation)，2. 基因運算 (genetic operations)，3. 適應度函數 (fitness function)。其中表示法是最重要的一點，特別是針對電路合成這類問題，一個好的表示法，必須符合以下的要求：1. 表示法所用到的資料結構最好是動態的而非固定長度的，因為最後合成出來的電路所需的元件數目很可能無法事先得知。2. 經過基因運算後產生的電路，必須保持是有效的電路，以免產生無效的電路拓撲結構，或是懸空的元件等，因而降低 GA 的效率。3. 表示法所用的資料結構，不僅要能方便地做基因運算，最好也能同時方便地進行電路分析 (circuit analysis)，以便作適應度計算。雖然之前已經有其他的表示法被提出 [3]-[5]，但是沒有一個能同時符合上述的要求，本篇文章提出一種二元樹的結構用以表示階梯

狀 (ladder) 的電路，這種新的表示法可以同時滿足上述的三個要求，以這種表示法為基礎的 GA 被用來合成濾波電路，實驗結果顯示這種方法是有效的，並且比先前其他作者所提出的方法快。

本文以下的內容組織如下：第二節、說明這種新的二元樹結構如何用來表示階梯狀的電路。第三節、說明如何經由走訪這種樹狀結構 (traveling tree)，來作電路分析的動作。第四節、提出以這種二元樹表示法為基礎的 GA，來執行電路合成的工作。第五節、說明適應度函數。第六節、將前述各節所提出的方法整合起來，運用在實際的低通與高通被動濾波電路的合成問題。第七節、結論與未來研究方向。

第二節、二元樹表示法

階梯狀電路廣泛地被用在電路設計上，圖 1 顯示階梯狀電路的拓撲結構，圖中每個長方形代表一個兩端點的被動元件 (如：電阻器、電感器、電容器，即 RLC)，而且每個長方形都可以擴充成被動元件的串、並聯組合，圖 2(a) 顯示一個擴充後的階梯狀 RLC 電路 (為了簡化起見，令所有的元件都是電阻器)，並以 V_{in} 的電壓源來推動。

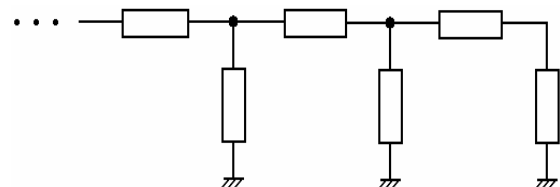


圖 1 階梯狀電路的拓撲結構。

受「資料結構」中算式表示樹 (expression tree) 的啟發 [6]，我們發現像圖 2(a) 這樣的階梯

狀電路可以用類似的二元樹來表示，圖 2(b)就是其對應的二元樹表示法。圖 2(b)中的節點(node)可以分成兩類，我們採取在文獻[7]中所用的術語，將所有的終端節點 (terminal node) 歸類為終端集合 (terminal set)，由被動元件 (即 RLC) 組成，每個終端節點對應到電路中的一個元件。另一類非終端節點屬於功能集合 (function set)，由兩種運算子 (operator) 組成，即所謂的串聯與並聯運算子 (分別以 + 及 // 表示)。經由比對圖 2(a)及圖 2(b)，我們可以很輕易地看出，這個電路的拓撲結構 (即被動元件串並聯的組合)，如何經由這種二元樹表示法描繪出來。

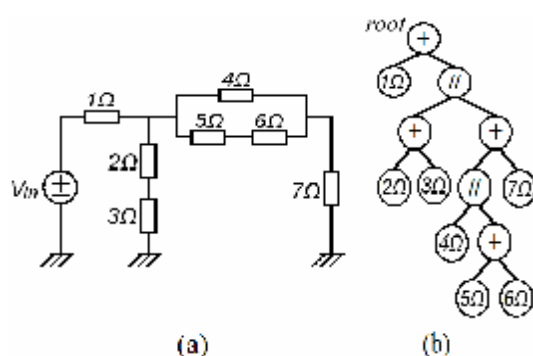


圖 2 一個階梯狀 RLC 電路的例子 (為了簡化起見，令所有的元件都是電阻器)。(a)電路圖 (b)二元樹表示法。

值得一提的是，傳統的階梯狀電路表示法，例如阻抗矩陣 (impedance matrix)，是屬於靜態的 (static) 資料結構，而這裡所提出的二元樹在本質上是一種動態的 (dynamic) 資料結構，這種特性使得它適用於電路元件數目不能事先確定的設計情況。此外，對二元樹作基因運算之後，得到的仍是有效的電路，這對 GA 的效率有極大的助益。

第三節、電路分析的演算法

電路分析牽涉到兩種電性的量的計算，即電壓 (voltage) 與電流 (current)。就一個給定電路的二元樹表示法來說，一般只有在終端節點上的元件值是已知的，而我們可以據此求出其對應的阻抗值 (impedance value)。一個完整的電路分析過程就是：根據這些終端節點上的阻抗值，將每個節點上的電壓與電流值計算出來。底下我們將描述電路分析的演算法，並且以像圖 2 這樣的階梯狀電路為例子，求其電壓增益 (voltage gain) 的頻率響應 (frequency response)。演算法如下：

- 1) 由下而上的程序：每一個終端節點將自己的阻抗值往父節點 (parent node) 傳，而每個非終端節點在收到子節點 (children node) 傳來的阻抗值後，根據自己的運算子是串聯

(+) 或是並聯 (//) 對收到的資料作運算，計算出自己的等效阻抗值，然後再將其往上傳，一直要到根節點 (root node) 的等效阻抗值求出時，整個程序才停止。順帶一提，此時根節點的等效阻抗即是從電壓源往整個階梯狀電路看進去的等效阻抗。

- 2) 由上而下的程序：首先將電壓源的電壓值除以根節點的等效阻抗值，以求得流入此一階梯狀電路的總電流，我們可以將其視為注入根節點的總電流，此一電流從根節點開始，根據以下的法則由上往下流：如果遇到串聯節點，則將流入的電流複製成兩份，分別流出至兩個子節點；如果遇到並聯節點，則將流入的電流根據兩個子節點的阻抗值反比例分配 (即所謂的分流定理)，流出至子節點；如果遇到的是終端節點，則停止。
- 3) 計算電壓增益：經過上述兩個程序後，我們已經將各個節點的等效阻抗與電流值算出來，接著，我們只需利用這些資料，將我們要求的節點的電壓值算出，再除以電壓源的電壓值，便可以算出其電壓增益。

事實上，上述第一與第二步驟，都可以利用走訪二元樹的演算法 (tree traversal algorithm) 來完成，並可以寫成遞迴形式的程式 (請參閱參考文獻[6])。對一個擁有 n 個元件的電路來說，上述演算法的複雜度 (complexity) 為 $O(n)$ ，此外，上述的電路分析方法適用於對階梯狀 RLC 電路作符號電路分析 (symbolic circuit analysis)。

第四節、基因演算法

我們提出一個以二元樹表示法為基礎的 GA，用以合成如圖 3 所示的 LC 濾波電路。

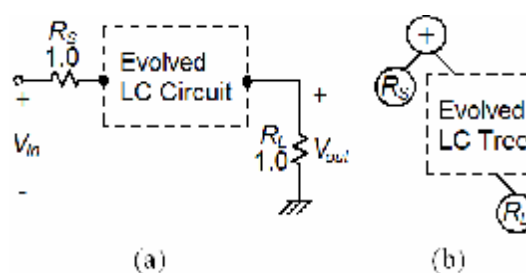


圖 3 LC 濾波電路的架構圖。虛線框起來的部分就是要利用基因演算法合成的電路部分，其兩邊則為電源與負載電阻，而其電阻值皆被正規化 (normalized) 為 1。(a) 電路圖 (b) 二元樹表示法。

演算法如下：

- 1) 初始化 (Initialization)：首先產生 100 個個體 (individuals) 作為初始群體 (initial population)，每個個體都是從圖 4 所示的四

種基材 (building blocks) 中隨機選取，其中負載電阻 (R_L) 的電阻值設定為正規化的 1，而 L (電感) 或 C (電容) 的值則在 0.2 到 5.0 的範圍內隨意選定。

- 2) 選擇 (Selection)：從目前的群體中隨意挑選兩個個體作競爭，其中適應度 (fitness) 較高的得以獲得交配權，同樣的程序用以產生另一個親代。有一個術語用來稱呼這種選擇的策略—tournament selection。
- 3) 交配 (Crossover)：在兩個親代的二元樹結構上隨機選取一個斷點 (cutting point)，然後互相交換其斷點之下的子樹 (subtree)，形成兩個新的子代 (offspring)。我們將交配率 (crossover rate) 設定為 0.9。
- 4) 突變 (Mutation)：每一個經過上述交配步驟形成的子代各自產生 10 個突變的個體，這些突變的個體與原始個體有相同的二元樹結構，只有終端節點上面的元件值會有隨機的變動，而變動值圍繞在原始值的附近。
- 5) 取代 (Replacement)：接著對這些原始及突變後的子代們作適應度的計算 (將在下一節介紹)，然後將表現最好的一個挑選出來與目前群體中最後一名作比較，若前者表現較好，則將後者取代。
- 6) 結束條件 (Termination Criteria)：如果符合規格的電路產生，或迴圈數超過 10,000 次的話，就停止程序；否則跳到步驟 2，繼續下一個迴圈。

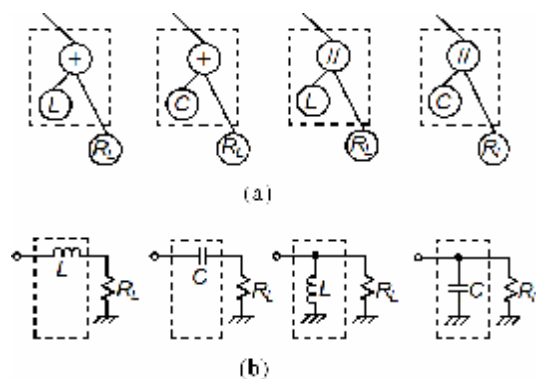


圖 4 建構 LC 濾波電路的四種基材 (building blocks)。(a) 二元樹表示法 (b) 電路圖。

第五節、適應度函數

要評估一個電路的適應度，首先便要對這個電路作分析，文獻[3]及[4]的作者採取的方法是將知名的電路模擬軟體 SPICE 納入其演算法中來作電路分析的工作，但是這樣的方式牽涉到表示法的轉換，也就是說，在做基因運算時用的是一種表示法，而要藉助 SPICE 作電路分析時又需將就另一種表示法 (即所謂的 netlist 格式)，這種轉換針對 GA 過程中每個個體都要執行，因此會降低其效率。而我們提出的二元樹表示法可

以直接作基因運算 (參考第四節)，同時也可以直接作電路分析 (參考第三節)，而不需任何轉換。此外第三節的演算法是直接於頻域 (frequency-domain) 作分析，比諸 SPICE——一種時域 (time-domain) 的模擬軟體——在高頻的應用上將較具優勢。

如下式(1)即為我們所採用的適應度函數：

$$Fitness = -\sum b_i |A(i) - T(i)| \quad (1)$$

其中， i 代表在頻率軸上的取樣點 (sampling points) 的索引值 (index)，本實驗取樣 50 個點； $A(i)$ 表示在第 i 個頻率時電路的實際頻率響應 (actual frequency response)，這個值可以利用第三節的電路分析演算法求得；而 $T(i)$ 代表目標頻率響應 (target frequency response)，即設計規格；而 b_i 是一個二進制位元 (bit)，當實際電路響應符合規格時，其值為 0，反之為 1。上述的式(1)，可以視為是一般所謂的錯誤函數 (error function) 前加負號，故錯誤值越大，適應度越負 (即越差)。

GA 會有產生較複雜電路的傾向，因為複雜的電路一般而言會有較好的適應度，在參考文獻[5]中，作者引介一個所謂的懲罰函數 (penalty function) 來懲罰較為複雜的電路，本文採用與其相似的懲罰函數如下式(2)所示：

$$penalty(n) = 1 + \exp(n - n_{max}) \quad (2)$$

其中 n 代表電路的元件數； n_{max} 代表初估的元件數。當 n 很小時，懲罰函數值為 1，而當 n 超過 n_{max} 後，懲罰函數的值以指數方式急速成長。我們將式(1)乘以式(2)形成新的適應度函數，使之能壓抑電路的元件數。

第六節、實驗結果

實驗一：低通濾波電路 (Low-pass filter)

在文獻[5]中，作者提出一種混合式 (hybrid) 的 GA 用以合成如下規格的低通濾波電路：

Pass-band edge:	1.0 rad/s
Stop-band edge:	1.5 rad/s
Maximum pass-band gain:	-6 dB
Minimum pass-band gain:	-7 dB
Maximum stop-band gain:	-52 dB

我們採用相同的規格，利用本文提出的方法，在個人電腦上作了 100 次的實驗，每次都能順利產生符合規格的電路，我們將產生電路所需的時間與文獻[5]作比較，並將結果列在表一，我們可以看出，雖然所使用的電腦 CPU 等級不同，但是我們的方法確實快很多。我們從 100 個符合規格的電路中挑出一個來，顯示在圖 5；而其頻率響應則顯示在圖 6。

表一 產生符合規格的低通濾波電路所需時間的比較。

	所使用的 CPU	合成電路所需的時間
文獻[5]的方法	Pentium II 300MHz	3 小時
本文提出的方法	P4 2.4GHz	平均 7.4 秒

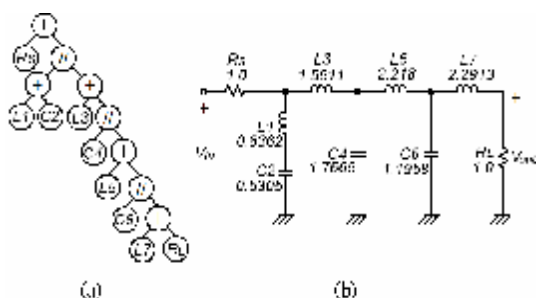


圖 5 GA 合成的低通濾波電路。(a) 二元樹表示法 (b) 電路圖 (含元件值)。

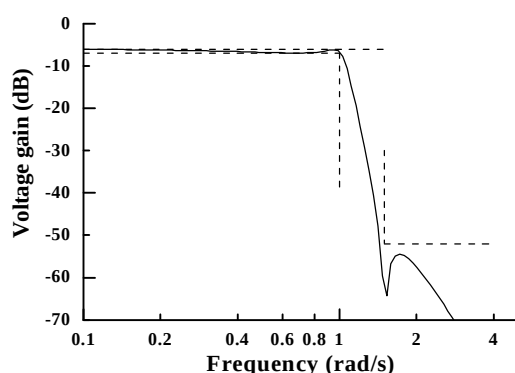


圖 6 圖 5 所示低通濾波電路的電壓增益 (頻率響應)。

實驗二：高通濾波電路 (High-pass filter)

接著我們將上述的規格稍微修改，使之變成高通濾波器的規格如下：

Pass-band edge:	1.5 rad/s
Stop-band edge:	1.0 rad/s
Maximum pass-band gain:	-6 dB
Minimum pass-band gain:	-7 dB
Maximum stop-band gain:	-52 dB

傳統上設計高通濾波器的流程是先設計低通濾波器，然後再作適當的轉換而成。在此，我們利

用本文所提出的方法，直接針對上述高通濾波器的規格，在個人電腦上作了 100 次的實驗，同樣地，每次都能順利產生符合規格的電路，我們從其中挑出一個來，顯示在圖 7；而其頻率響應則顯示在圖 8。

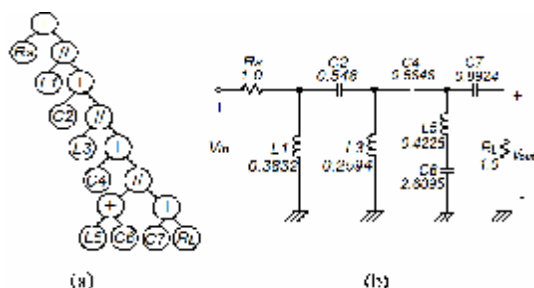


圖 7 GA 合成的高通濾波電路。(a) 二元樹表示法 (b) 電路圖 (含元件值)。

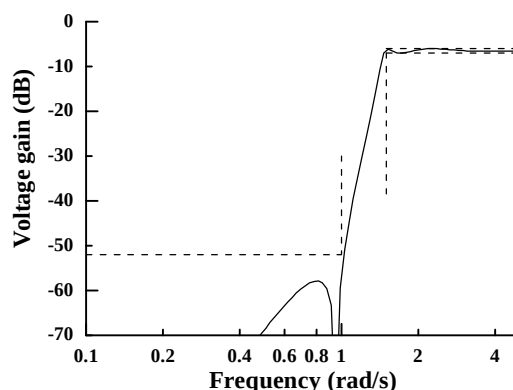


圖 8 圖 7 所示高通濾波電路的電壓增益 (頻率響應)。

第七節、結論與未來研究方向

本文提出一種新的二元樹表示法來描述階梯狀電路，這種二元樹的表示法不僅方便我們作電路分析，並且以這種二元樹表示法為基礎的 GA 在執行電路合成的工作時相較於過去的方式有若干的優點，使得它在速度上比較快，這點在我們的實驗中明確地獲得證明。

跟其他方法相比，GA 被認為在處理多目標最佳化 (multiple-objectives optimization) 問題上是較有效率的，這點使我們有信心在未來的研究中，除了考慮頻率響應之外，也可以同時將其他濾波電路的設計考量 (例如：phase delay optimization、component value sensitivity) 一併列入，達成多目標的最佳化設計。

參考文獻

- [1] D. E. Goldberg. Genetic Algorithms in Search, Optimization and Machine Learning. New York: Addison-Wesley, 1989
- [2] R. S. Zebulum, M. A. C. Pacheco, and M. Vellasco. Evolutionary electronics: automatic design of electronic circuits and systems by genetic algorithms. CRC Press, 2002.
- [3] J. R. Koza, F. H. Bennett III, F.H., D. Andre, M. A. Keane, and F. Dunlap. Automated synthesis of analog electrical circuits by means of genetic programming. *IEEE Trans. Evol. Comput.*, vol. 1, pp. 109-128, July 1997.
- [4] J. D. Lohn, and S. P. Colombano. A circuit representation technique for automated circuit design. *IEEE Trans. Evol. Comput.*, vol. 3, no. 3, pp. 205-219, Sept. 1999.
- [5] J. B. Grimbleby. Automatic analogue circuit synthesis using genetic algorithms. *IEE Proc.-Circuits Devices Syst.*, vol. 147, No. 6, pp. 319-323, Dec. 2000.
- [6] T. A. Budd. Classic data structures in C++. Addison-Wesley Publishing Company, 1994.
- [7] J. R. Koza. Genetic programming: On the programming of computers by means of natural selection. Cambridge, MA: MIT Press, 1992.

The Design of Improved Dynamic AES and Hardware Implementation Using FPGA

C.L. Lee ¹, Y.Z. Liang ², C.D. Lee ¹, J.Y. Hwang ^{*2}

¹ Department of Information Engineering, I-Shou University, Kaohsiung, Taiwan.
m9103010@stmail.isu.edu.tw; d9003002@stmail.isu.edu.tw

² Department of Technology Research, ASN Technology Corp., Southern Taiwan Science Park, Taiwan. donie@asn.com.tw; hjy@asn.com.tw

Abstract

ByteSub is a key module in the AES hardware architectures. Most of the ByteSub design is accomplished by look-up table method. To reduce the complexity of AES module and improve the performance and security, we propose a novel dynamic ByteSub generator, which is composed of finite field inverse and multipliers. In our design, look-up table method is no more essential to the ByteSub hardware architecture. The proposed design is implemented on Altera APEX20KE FPGA family of devices. It needs 44218 logic cell elements on FPGA chip and the performance can be achieve to 57.2 MHz. Our proposed design does not need any memory elements.

1 Introduction

In 2001, the National Institute of Standards and Technology (NIST) announced Rijndael algorithm to be Federal Information Processing Standard (FIPS) 197 [10] - Advanced Encryption Standard (AES). Hence, the AES replaced the Data Encryption Standard (DES) [11]. By replacing the DES, AES algorithm is the standard of most cryptography applications with software or hardware implementation since then. The goal of our design is to improve the performance and security integrity. In this paper, we proposed the dynamic AES algorithm hardware design by using the standard AES algorithm and finite field inverse algorithm circuit on FPGA devices. The details are described in the following sections.

2 The Advanced Encryption Standard

The AES algorithm is a symmetric block cipher. The length of the plaintext and ciphertext data block is 128 bits. The length of the key block can be 128,

192 or 256 bits. In this paper, the length of AES algorithm proposed is limited to 128-bit data and 128-bit key blocks. The block of data and key in AES algorithm can be presented as a 4×4 matrix of bytes. The encryption/decryption operation in AES is composed of multiple rounds, which is the basic operating unit in AES. Every round consists of four transformations except the final round in AES encryption/decryption operation. The four transformations are described as below:

- ByteSub - a single non-linear transformation, applied to each byte of the data.
- ShiftRow - considering the data as a 4×4 matrix of bytes, reorders the bytes of each row cyclic.
- MixColumn - considering the data as a 4×4 matrix of bytes, a linear transformation applied to columns of the matrix.
- AddRoundKey - XORed with a round key and the intermediate data block, which is produced after the previous three transformations.

The initial step of AES is XORed with original data and key block before any AES round operations. The MixColumn transformation is excluded in the final round operation of AES. The complete encryption/decryption operation of AES algorithm is shown in Figure 1.

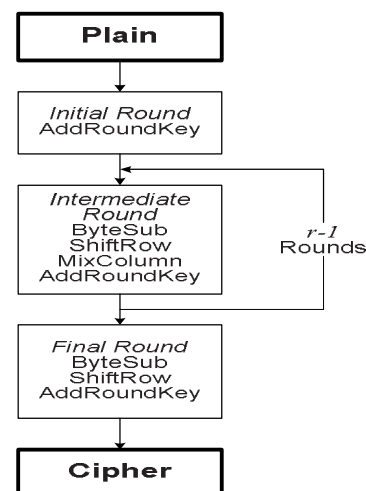


Figure 1. The complete operations of AES.

*Corresponding Author

A full detailed description of the AES algorithm can be found in the FIPS-197 [10] documents.

3 Improvement of AES

I. The multiplier and inverse in $GF(2^m)$

ByteSub transformation is critical to the performance of AES algorithm. Most of the traditional methods utilize look-up tables (S-Box table) to implement ByteSub transformation [3,8]. A single S-box look-up table required 256 bytes memory elements in it. In general, 8K bytes of look-up table construct the 32 S-Boxes for AES encryption and decryption operations with 128 bits length of data and key block. Hence, this kind of architecture is not suitable for area limited applications, such as the smart card chip design [4]. To avoid the use of look-up table, the finite field multiplier and inverse modules are used in the proposed designed.

Multiplier

The AES algorithm mathematical operation is based on $GF(2^8)$. There are 30 kinds of irreducible polynomials in $GF(2^8)$ [7,10]. The irreducible polynomial used in standard AES is $x^8+x^4+x^3+x+1$, which is presented as $m(x)$.

According to M.H. Jing et al. [5], we developed a novel 8-bit multiplier over $GF(2^8)$ with extra 8-tuples coefficients over $GF(2)$, which can appoint any chosen irreducible polynomial over $GF(2^8)$ by users. The architecture of this novel 8-bit multiplier is shown in Figure 2.

Inverse

Traditionally, the finite field inverse calculation can be achieved by a series of finite field multipliers [1,2,5]. Considering a finite field element over $GF(2^m)$ A , the inversion A^{-1} can be calculated by a series of power of A . According to Stephen B. Wicker [9], the finite field inversion can be expressed as

$$A \bullet A^{-1} = 1 = A^{2^m-1} = A \bullet A^{2^m-2} \quad (1)$$

Thus, the finite field $GF(2^8)$ multiplicative inversion A^{-1} , performed in ByteSub transformation, can be calculated by $A^{2^8-2} = A^{254}$. By using the structure of the systolic array, A^{254} can be represented by the multiplications of the square property of A and A^2 [9]. The block diagram of A^{-1} circuit, which uses square property, is shown in Figure 3.

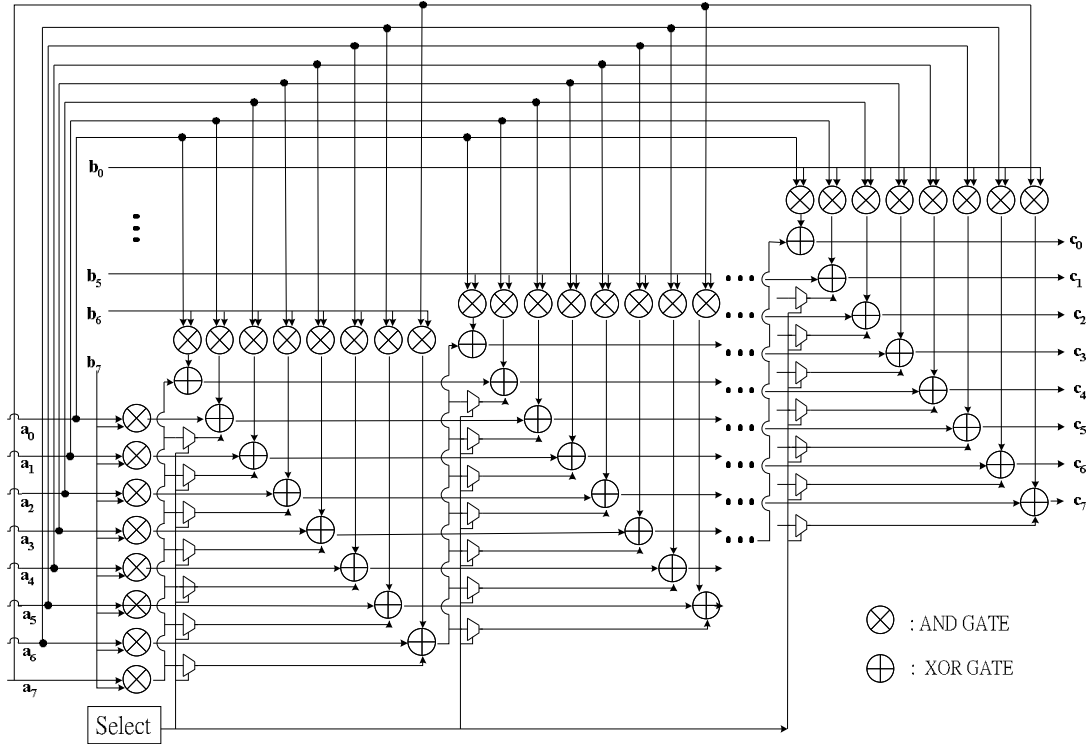


Figure 2. Implementation of an 8-bit multiplier.

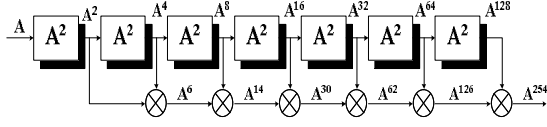


Figure 3. The finite field inverse circuit using square property.

According to the Figure 3, the inverse circuit has six multipliers in this kind of design. This inverse circuit is still a critical path in ByteSub module. However, the A^{254} can also be represented [4] as

$$A^{-1} = A^{2^8-1} = A^{254} = ((A^4) \cdot (A^3))^{32} \cdot ((A^3)^2 \cdot (A^3)^8) \quad (2)$$

Based on equation 2, the new inverse circuit architecture can be drawn and shown as Figure 4.

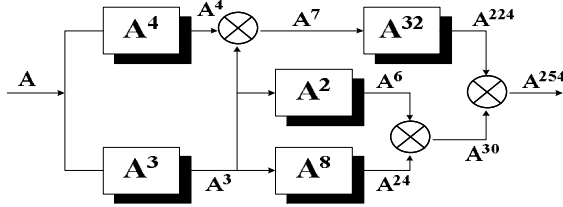


Figure 4. The new finite field inverse circuit.

As shown in Figure 4, the number of multipliers in this new finite field inverse circuit is reduced from 6 to 3 [4].

II. The Dynamic AES

The standard AES algorithm uses an irreducible binary polynomial of degree 8, $m(x) = x^8 + x^4 + x^3 + x + 1$, to perform the finite field multiplication in ByteSub transformation. According to R. Lidl and H. Niederreiter [7], we can apply the following theorem to count the number of irreducible polynomial [6] in $GF(2^m)$. In other words, we can use other irreducible polynomial over $GF(2^8)$ to construct different ByteSub transformations. Thus, the users can choose their own ByteSub transformation to perform the variant AES algorithm and increase the complexity.

Theorem : The number $N_q(n)$ of irreducible polynomial in $GF(2^m)$ of degree n is given by

$$N_q(n) = \frac{1}{n} \sum_{d|n} \mu(d) q^{n/d} \quad (3)$$

Möbius function μ :

$$m(n) = \begin{cases} 1 & \text{if } n = 1 \\ (-1)^k & \text{if } n \text{ is the product of } k \text{ distinct primes} \\ 0 & \text{if } n \text{ is divisible by the square of prime} \end{cases}$$

All of the 30 kinds of irreducible binary polynomial of degree 8 in $GF(2^8)$ were listed in Table 1.

We represent the 30 irreducible polynomials as a control bits set, then apply the corresponding control bits set (Table 1) to the proposed finite field multipliers (Figure 2) as the input signal. These finite field multipliers and inverse as shown in Figure 2 and 4 can construct the novel dynamic ByteSub generator with control bits set input signals.

4 The Implementation of Dynamic AES on FPGA and Cryptanalysis

In order to implement the proposed dynamic ByteSub generator, we apply the finite field inverse module and an Affine transformation module, which can be implemented directly on the bit-level. We also use a Field Programmable Gate Arrays (FPGA) chip to verify our novel hardware design. The Figure 5 gives the block diagram of this improved dynamic AES algorithm.

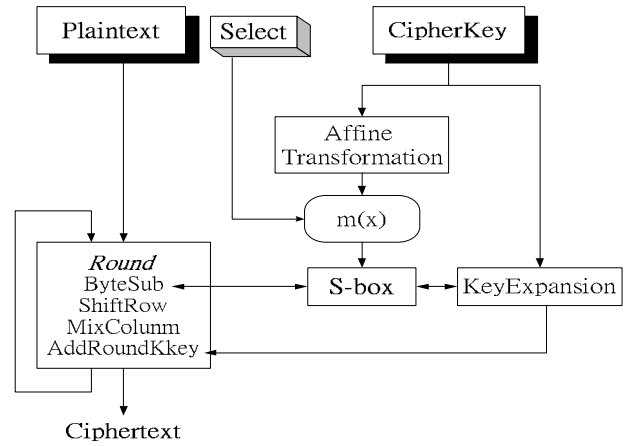


Figure 5. The block diagram of improved AES algorithm.

There is Electronic Code Book (ECB) mode and Cipher Block Chaining (CBC) mode [10,11] in this improved dynamic AES algorithm, with 128 bits data and key block. The $gmul_m$ signal, which uses the Table 1 control bits set as input bits, is a selector for 30 kinds of irreducible polynomials. The AES integral symbol is shown in Figure 6.

Table 1. All irreducible polynomial of degree 8 in GF(2⁸) and control component of ByteSub generator.

Index	Primitive	Irreducible Polynomial		Control Bit (binary)
0		$x^8+x^4+x^3+x+1$	1B	00000
1	*	$x^8+x^4+x^3+x^2+1$	1D	00001
2	*	$x^8+x^5+x^3+x+1$	2B	00010
3	*	$x^8+x^5+x^3+x^2+1$	2D	00011
4		$x^8+x^5+x^4+x^3+1$	39	00100
5		$x^8+x^5+x^4+x^3+x^2+x+1$	3F	00101
6	*	$x^8+x^6+x^3+x^2+1$	4D	00110
7	*	$x^8+x^6+x^4+x^3+x^2+x+1$	5F	00111
8	*	$x^8+x^6+x^5+x+1$	63	01000
9	*	$x^8+x^6+x^5+x^2+1$	65	01001
10	*	$x^8+x^6+x^5+x^3+1$	69	01010
11	*	$x^8+x^6+x^5+x^4+1$	71	01011
12		$x^8+x^6+x^5+x^4+x^2+x+1$	77	01100
13		$x^8+x^6+x^5+x^4+x^3+x+1$	7B	01101
14	*	$x^8+x^7+x^2+x+1$	87	01110
15		$x^8+x^7+x^3+x+1$	8B	01111
16	*	$x^8+x^7+x^3+x^2+1$	8D	10000
17		$x^8+x^7+x^4+x^3+x^2+x+1$	9F	10001
18		$x^8+x^7+x^5+x+1$	A3	10010
19	*	$x^8+x^7+x^5+x^3+1$	A9	10011
20		$x^8+x^7+x^5+x^4+1$	B1	10100
21		$x^8+x^7+x^5+x^4+x^3+x^2+1$	BD	10101
22	*	x^8+x^7+x+1	C3	10110
23	*	$x^8+x^7+x^6+x^3+x^2+x+1$	CF	10111
24		$x^8+x^7+x^6+x^4+x^2+x+1$	D7	11000
25		$x^8+x^7+x^6+x^4+x^3+x^2+1$	DD	11001
26	*	$x^8+x^7+x^6+x^5+x^2+x+1$	E7	11010
27		$x^8+x^7+x^6+x^5+x^4+x+1$	F3	11011
28	*	$x^8+x^7+x^6+x^5+x^4+x^2+1$	F5	11100
29		$x^8+x^7+x^6+x^5+x^4+x^3+1$	F9	11101

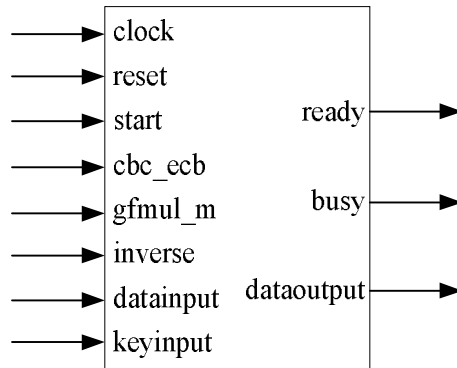


Figure 6. AES integral symbol.

The hardware description language we used in this paper is VHDL, and the synthesized device used is EP20K1500EBC652 of Altera's chip. There are 44218 logic cell elements in the simulations result and the performance achieves 57.2 MHz. Importantly, the proposed design does not need any memory bits elements. This feature can increase the security by preventing the exposure of S-Box data.

The Affine transformation has no impact on ByteSub transformation non-linearity properties. However, by proper chosen of Affine transformation, the S-Box will have a complex algebraic expression and the S-Box data will have no fixed points and opposite fixed point.

Cryptanalysis

To avoid the differential attack, the proposed novel dynamic AES preserve the original architecture and adds extra-modified parameter (gfmul_m signal) in ByteSub transformation. Furthermore, the systolic hardware design in the proposed one can be used to avoid power analysis attack and the timing attack.

5 Conclusions

In this paper, we implemented the dynamic AES on FPGA successfully. Using the proposed finite field inverse and multiplier modules, we add extra parameters in ByteSub transformation as the ByteSub generator. The users can choose one polynomial from the 30 kinds of irreducible polynomials freely, to generate their own version of S-Box tables. This proposed design can provide better performance and increase the security. In AES hardware implementation, we use the cell-based design to implement the proposed AES chip by using VHDL language. The future goal is to develop the dynamic modified Affine transformation and MixColumn transformation in subsequent study.

Reference

- [1] Y.H. Chen, "The Implementation of Reed-Solomon Code Using Inversionless Berlekamp Massey Algorithm," 1999.
- [2] Y.S. Cho and S.K. Park, "Design of GF(2m) Multiplier Using its Subfields," Elec. Letters, Vol. 37, No. 7, pp650-651, April 1998.
- [3] C.H. Hsieh, "Design and Implementation of Advanced Encryption Standard Chip," 2003.
- [4] J.Y. Hwang, "The Design, Implementation and Application of Advanced Encryption Standard Algorithm," 2001.
- [5] M.H. Jing, Y.W. Chen, Y.C. Chang, T.K. Troung, "A New VLSI for Implementing the Multiplication and Inverse in the RS-Code," 4TH multimedia technology and applications, Taiwan China, April 16 1999, pp.304.
- [6] Robert J. McEliece, "Finite Fields for Computer Scientists and Engineers," California Institute of Technology, 1987.
- [7] R. Lidl and H. Niederreiter, "Introduction to Finite Fields and Their Applications," Cambridge University Press, 1986.
- [8] P. Mroczkowski, "Implementation of the block cipher Rijndael using Altera FPGA," Public Comments on AES Candidate Algorithms-Rounds 2.
- [9] Stephen B. Wicker, "Error control System for Digital Communication and Storage," Prentice hall international, Inc., 1995.
- [10] "Announcing the Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication 197, November 26, 2001.
- [11] "Data Encryption Standard (DES)," Federal Information Processing Standards Publication 46-3, October 25, 1999.

Case Study of Protein Stability by Using Topological Properties *

Jiunn-I Shieh and Kuei-Jen Lee

Department of Information Management
Taichung Healthcare and Management University, Taichung, Taiwan
Department of Bioinformatics
Taichung Healthcare and Management University, Taichung, Taiwan

{jishieh, kjlee}@thmu.edu.tw

Abstract

This paper shows how to study the protein stability by using the topological properties. In our case study, this paper also shows that protein systems have power-law property. This means some atoms are very important to each protein system. We conjecture that there exists a binding site around them.

1 Introduction

Thermostability is one of the most important properties of enzymes in both science and industry. Thermostable proteins can be used to improve the efficiency of chemical processes in industry and provide a better understanding protein folding and stabilization, which are important for protein engineering, biomolecular design, and structural prediction[1]. In this work, we are focused on industrial enzymes which can hydrolyze the cellulose to glucose and is called cellulases.

Cellulases are glycoside hydrolases which can be found in bacteria and fungi. Bacterial and fungal cellulases have been widely used in detergent, textile, and food industries[2]. The enzymes that hydrolyze the cellulose to glucose can be divided in two classes[2]: endoglucanases (EC 3.2.1.4), which hydrolyze internal bonds in the cellulose chains, and exoglucanases (EC 3.2.1.91), which hydrolyze from the chain ends. Both of these enzymes are important for industry, due to the fact that the cellulose can be hydrolysed to glucose by these enzymes and the glucose is fermented to ethanol by yeast for the use of transportation fuel. Cellulases are glycoside hydrolases. In this study, we used glycoside hydrolase family 12 (Cel12) endoglucanases[3] as the studying materials, which are enzymes isolated from *Humicola grisea* (H. grisea Cel12A) and *Trichoderma reesei* (T. reesei Cel12A). H. grisea enzyme has a thermal denaturation midpoint of the transition temperature (T_m) of 68.7°C which is 14.3 °C higher than that of T. reesei enzyme[3]. The H. grisea enzyme has 43% sequence identity

* Research supported by NSC 92-2213-E-468-007

with the *T. reesei* enzyme by the global alignment[4], as shown in Fig.1. The three-dimensional (3-D) structures of the *H. grisea* enzyme and *T. reesei* enzyme have been obtained by X-ray diffraction and are now available in the Protein Data Bank (PDB) with PDB codes 1OLR (Fig.2) and 1H8V (Fig. 3), respectively.

This paper proposes a methodology of using the topological invariant to discriminate two different function proteins although they have high similarity. In Section 2, the definitions of topological invariants are introduced. A novel approach of topological properties is presented in Section 3. Section 4 presents the results of computation and compares the results with other approach. Finally, a conclusion is in the last section.

2 Topological invariants

A topological property is very important for classification in many aspects. The purpose of this paper is to find some good topological properties to classify protein function.

The topological properties (e.g., patterns of connectivity, topological complexity, cycle rank, etc) of graphs can be analyzed using spectral graph theory. Each graph has a Laplacian matrix

representation and a corresponding eigenvalue spectrum [6]; the number of eigenvalues is equal to the rank of the matrix. The eigenvalues are related to the connectivity pattern (or topology) of the graph. In particular, the second largest eigenvalue is a measure of the compactness of a graph. A large second eigenvalue indicates a compact graph, whereas a small eigenvalue implies an elongated topology[5].

Define the Laplacian matrix of a graph $L(G)$.

The Laplacian matrix marks the connections between nodes, such that $L(i, j) = -1$ if $i \neq j$ and i is adjacent to j . Also, $L(i, i) =$ the total number of connections between i and any other node. Thus the Laplacian matrix is both symmetric, and the values of all rows and columns add up to zero. It is known that the second eigenvalue of the Laplacian matrix of a graph, $\lambda_2(L(G))$, is the algebraic connectivity of graph G . Thus the vector $\mathbf{V}_2$ corresponding to $\lambda_2(L(G))$ is the eigenvector of algebraic connectivity, and has special properties.

Another topological property is “cycle rank”.

Let G be a graph with $c(G)$ components. The cycle rank of G , denoted $\beta(G)$, is the number of edges in the relative complement of a full spanning forest of G .

Thus, the cycle rank is $\beta(G) = |E(G)| - |V(G)| + c(G)$, where $|E(G)|$ is the number of edges in the graph G and $|V(G)|$ is the number of vertices in the graph G .

From the definition, it is easy to see that $\beta(G)$ is the minimum number of edges to be deleted for the induce subgraph being a spanning tree. This should be related to thermal property.

In this work the Laplacian matrix is used to calculate its corresponding eigenvalues. The total number of vertices in a protein secondary structure equals the total number of eigenvalues. The eigenvalue that helps to describe the protein topology is the second eigenvalue. The second eigenvalue describes the compactness of a graph. We believe that the compactness of a graph is also related to thermal property

3 Methods

First, find two proteins which have high similarity but different function. We check it by the global alignment. Then, we take the following processes:

- Find 3-dimension coordinates of all atoms in each protein.

- Construct a graph from the above. First, we take the atoms as the vertices of the graph. Then we get a trival graph with empty edge set. Given the threshold $\varepsilon > 0$, we add a edge between two atoms if the distance between the two atoms is greater ε , otherwise no edge is added between two atoms.

- Change the threshold from 4 unit to 30 unit by increasing 1 unit each time. Each time we get a graph(G). Note that the unit is 10^{-8} cm.

- For each graph we write down the Laplacian matrix $L(G)$ of the correspondent graph.

- For each Laplacian matrix, calculate the second eigenvalue $\lambda_2(L(G))$ of the Laplacian

matrix and $\beta(G)$ of the correspondent graph.

4 Data analysis

The data analysis is divided into three cases:

- Case 1: $L_2(L(G))$

From the computation results (Table 4), at many different scales we have that the value L_2 of H. grisea enzyme is greater than that of T. reesei enzyme. So the graph of H. grisea enzyme is more compact than that of T. reesei enzyme. Therefore, it is very reasonable to say that the thermal stability of H. grisea enzyme is higher than that of T. reesei enzyme.

- Case 2: $\beta(G)$.

From the computation results (Table 5), at many different scales we have that the value cycle rank of H. grisea enzyme is greater than that of T. reesei enzyme. It means that the graph of H. grisea enzyme needs more time to delete the edge to form a spanning tree than that of H. grisea enzyme. Therefore, it is also reasonable to say that the thermal stability of H. grisea enzyme is higher than that of T. reesei enzyme.

- Case 3:

From Table 4 and Table 5, we check the power-law property for each system by log-log regression. The results show that r^2 for L_2 of T. reesei enzyme and H. grisea enzyme are 0.992951 and 0.992849 as shown in Table 6 and Table 7, respectively. The r^2 for $\beta(G)$ of T. reesei enzyme and H. grisea enzyme are 0.98586 and 0.98708 as shown in Table 8 and Table 9, respectively. These show that each system is very good fit to power-law property.

To sum up, compare L_2 method and $\beta(G)$ method from data we analyzed. It seems that $\beta(G)$ method is better than L_2 method.

5 Conclusion

In this paper, we have shown that the topological properties can be used to discriminate two different function proteins although they have high similarity. These two protein system also have power-law property. This means that some atoms are very important to each protein system. We conjecture that there maybe a binding site around them. Currently, we are still working on

the issues of the study of protein stability by using topological properties within a shorter time and the algorithm for finding the binding sites.

References

- [1] Kim, Y.-W. et al., 2003, Directed Evolution of Thermus Maltogenic Amylase toward Enhanced Thermal Resistance, Appl. Environ. Microbiol. 8, 4866-4874.
- [2] Schulein, M, 2000, Protein engineering of cellulases, Biochimica et Biophysica Acta, 1543, 239-252.
- [3] Sandgren, M. et al., 2003, The Humicola grisea Cel12A enzyme structure at 1.2 Å resolution and the impact of its free cysteine residues on thermal stability, Protein

Science, 12, 2782-2793.

- [4] <http://www.ebi.ac.uk/emboss/align/>
- [5] Hin Hark Gan, Samuela Pasquali, and Tamar Schlick, Exploring the repertoire of RNA secondary motifs using graph theory with implications for RNA design, Nucleic Acids Res., 31, 2926-2943, 2003.
- [6] F.R.K. Chung, Spectral graph theory, Amer. Math. Soc. Providence, 1994.

Appendix

```
#=====
#
# Aligned_sequences: 2
# 1: 10LR
# 2: 1H8V
# Matrix: EBLSUM62
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 229
# Identity: 98/229 (42.8%)
# Similarity: 137/229 (59.8%)
# Gaps: 16/229 ( 7.0%)
# Score: 501.5
#
#
#=====
```

10LR	1	XIRSLCELYGYWSGNGYELLNNLWGKDTATSGWQCTYLDGTNNGGIQWST	50
1H8	1	. . . : : : . : : . . .	46

V		XTSCDQWATFTGNGYTVSNNLWGA - SAGSGFGCVTAVSL - SGGASWH A	
10L R	51	AWEWQGAPDNVKSYPYVGKQIQGRKISDINS MRTSVSWTYDRTDIRANV	100
1H8 V	47	. ::. .::.: : . :.. :: DWQWSGGQNNVKSYQNSQIAIPQKRTVNSISSMPTTASWSYSGSNIRANV	96
10L R	101	AYDVFTARDPDHPNWGGDYELMIWLARYGGIYPIGT FHSQVNLAGRTWDL	150
1H8 V	97	: .: :: . . :... : . : . AYDLFTAANPNHVITYSGDYELMIWLGKYGDIGPIGSSQGT VNVGGQSWTL	146
10L R	151	WTGYNGNM RVYSFLPPSGDIRDFSCDIKDFFNYLER NHGYPAREQN LIVY	200
1H8 V	147	:. . : :..: :...: . : : :... YYGYNGAMQVYSFVAQT - NTTNYSGDVKNFFNYLR DNKGYN AAGQYVLSY	195
10L R	201	QVGTECFTGGPARFTCRDFRADLW	224
1H8 V	196	 QFGTEPFTGSGT-----LNVASWTASIN	218
#-----			
#-----			
file:///F:/protein-stability/1LOR_1H8V.htm[2004/4/3 yd Uydy^08:56:40]			

Fig. 1. Global alignment of *H. grisea* enzyme (1OLR) and *T. reesei* enzyme (1H8V).

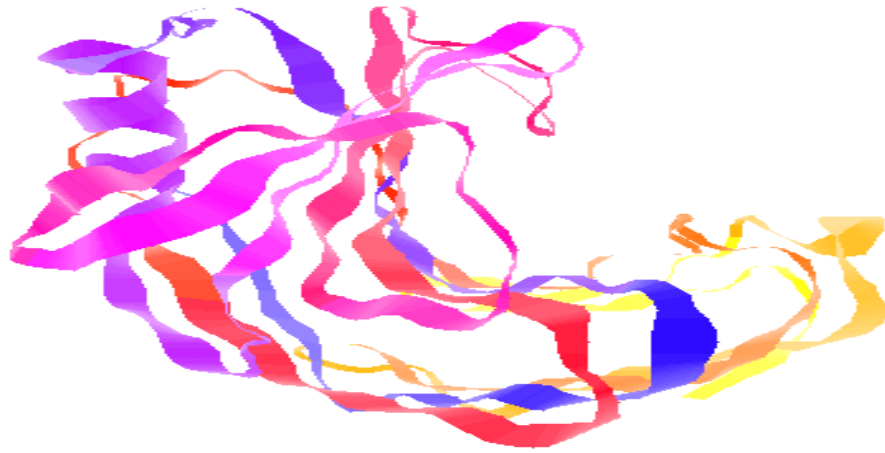


Fig. 2. Three-dimensional structure of the H. grisea enzyme

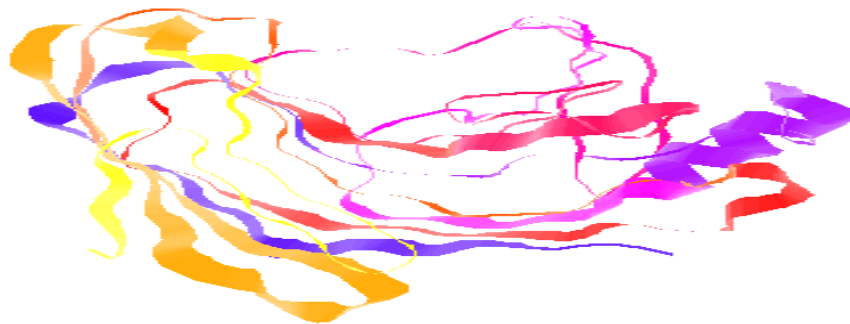


Fig. 3. Three-dimensional structure of the T. reesei enzyme.

ε	l_2 (T. reesei enzyme)	l_2 (H. grisea enzyme)		ε	Cycle rank (T. reesei enzyme)	Cycle rank (H. grisea enzyme)
4	0.0720	0.0724		4	8364	9284
5	0.2835	0.2835		5	18517	20602
6	0.7070	0.7250		6	32111	36024

7	1.5495	1.5812		7	49337	55492
8	2.9851	3.0645		8	69818	78801
9	5.3870	5.5512		9	95002	107423
10	9.2055	9.4029		10	125734	142228
11	14.8544	15.2237		11	161559	183373
12	22.7464	23.1884		12	201092	228431
13	33.1815	33.8253		13	243904	278035
14	47.0273	47.7713		14	291456	332716
15	64.9195	65.7445		15	343658	392409
16	86.7573	88.0187		16	397901	455325
17	113.0886	114.8249		17	453930	520824
18	144.2712	146.3088		18	511943	588210
19	180.4478	183.1114		19	571833	658189
20	222.1353	225.4988		20	632657	729787
21	268.4324	272.3763		21	693098	801556
22	318.9149	323.7338		22	752409	872813
23	374.5746	379.7790		23	811528	943894
24	434.5988	440.5198		24	869608	1014301
25	497.5034	504.7836		25	925416	1082854
26	563.8181	572.2726		26	978823	1148567
27	632.0749	641.4141		27	1028911	1210960
28	701.3060	713.3066		28	1076467	1270850
29	766.0013	787.7856		29	1120604	1327040
30	839.9631	858.7090		30	1160743	1378806
31	915.2513	919.9922		31	1197074	1425901

Table 4. l_2

Table5. β (G) (cycle rank)

SUMMARY

OUTPUT

<i>Regression Statistics</i>	
Multiple R	0.996469
R Square	0.992951

Adjusted R Square	0.99268
Standard Error	0.324813
Observations	28

ANOVA

	<i>df</i>	<i>SS</i>	<i>MS</i>	<i>F</i>	<i>Significance F</i>	
Regression	1	386.3821	386.3821	3662.276	1.65E-29	
Residual	26	2.743085	0.105503			
Total	27	389.1252				

	<i>Coefficients</i>	<i>Standard Error</i>	<i>t Stat</i>	<i>P-value</i>	<i>Lower 95%</i>	<i>Upper 95%</i>
Intercept	-12.1091	0.3021	-40.083	6.62E-25	-12.7301	-11.4881
X variable 1	4.553313	0.075241	60.51674	1.65E-29	4.398654	4.707973

Table 6. $\log_2(I_2) - \log_2 e$ (T. reesei enzyme)

SUMMARY

OUTPUT

<i>Regression Statistics</i>	
Multiple R	0.996418
R Square	0.992849
Adjusted R Square	0.992574
Standard Error	0.327155
Observations	28

ANOVA

	<i>df</i>	<i>SS</i>	<i>MS</i>	<i>F</i>	<i>Significance F</i>	
Regression	1	386.3547	386.3547	3609.767	1.99E-29	
Residual	26	2.78279	0.10703			
Total	27	389.1375				

	<i>Coefficients</i>	<i>Standard Error</i>	<i>t Stat</i>	<i>P-value</i>	<i>Lower 95%</i>	<i>Upper 95%</i>
Intercept	-12.0844	0.304279	-39.715	8.38E-25	-12.7099	-11.459
X variable 1	4.553152	0.075783	60.08133	1.99E-29	4.397377	4.708926

Table 7. $\log_2(I_2) - \log_2 e$ (H. grisea enzyme)

SUMMARY

OUTPUT

Regression Statistics						
Multiple R	0.992905					
R Square	0.98586					
Adjusted R Square	0.985316					
Standard Error	0.238165					
Observations	28					
ANOVA						
	<i>df</i>	<i>SS</i>	<i>MS</i>	<i>F</i>	<i>Significance F</i>	
Regression	1	102.8228	102.8228	1812.728	1.41E-25	
Residual	26	1.47479	0.056723			
Total	27	104.2976				
	<i>Coefficients</i>	<i>Standard Error</i>	<i>t Stat</i>	<i>P-value</i>	<i>Lower 95%</i>	<i>Upper 95%</i>
Intercept	8.976274	0.221512	40.5228	5E-25	8.52095	9.431598
X variable 1	2.348896	0.055169	42.57614	1.41E-25	2.235494	2.462298

Table 8. $\log_2(\beta(G)) - \log_2 \varepsilon$ (T. reesei enzyme)

SUMMARY

OUTPUT

<i>Regression Statistics</i>	
Multiple R	0.993519
R Square	0.98708
Adjusted R Square	0.986583
Standard Error	0.230687
Observations	28

ANOVA

	<i>df</i>	<i>SS</i>	<i>MS</i>	<i>F</i>	<i>Significance</i> <i>F</i>	
Regression	1	105.704	105.704	1986.307	4.36E-26	
Residual	26	1.383625	0.053216			
Total	27	107.0876				
	<i>Coefficients</i>	<i>Standard</i> <i>Error</i>	<i>t Stat</i>	<i>P-value</i>	<i>Lower 95%</i>	<i>Upper 95%</i>
Intercept	9.049588	0.214556	42.17821	1.79E-25	8.608561	9.490614
X variable 1	2.381577	0.053437	44.56801	4.36E-26	2.271736	2.491419

Table 9. $\log_2(\beta(G))$ -- $\log_2 \varepsilon$ (H. grisea enzyme)

Drawing Alternative Splicing Graphs

Yaw-Ling Lin*

Dept. Computer Science and Information Management

Providence University, Taiwan

E-mail: yllin@pu.edu.tw

Abstract

Alternative splicing of a single pre-mRNA can give rise to different mRNA transcripts. Alternative splicing of premessenger RNA is an important layer of gene expression regulation in eukaryotic cell. Consequently, alternative splicing is an important mechanism for generating protein diversity from a single gene. Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing a given alternative splicing graph on the plane.

Given an alternative splicing graph $G = (V, E)$, here in this paper we show that computing the crossing number of a given upper drawing ASG can be done in $O(|E| \log |E|)$ time. The optimal (fewest crossing) drawing of alternative splicing graphs is related to the long known difficult problem of crossing number. Here we propose an efficient branch-and-bound recursive algorithm with heuristic searching strategy. Some experimental results are also discussed in the paper.

Keywords: algorithm, computational biology, branch and bound, crossing number.

1 Introduction

RNA splicing is an essential, precisely regulated post-transcriptional process that occurs prior to mRNA translation. A gene is first transcribed into a pre-messenger RNA (pre-mRNA), which is a copy of the genomic DNA containing intronic regions destined to be removed during pre-mRNA processing (RNA splicing), as well as exonic sequences that are retained within the mature mRNA. Alternative splicing of eukaryotic pre-

mRNAs is a mechanism for generating potentially many transcript isoforms from a single gene. It serves versatile regulatory functions in controlling major developmental decisions and fine-tuning of gene function [9]. The majority of alternative splicing events give rise to different protein products. At least 35% of human genes undergo alternative splicing [6, 5]. The physiological functions of these splice variants may be similar, opposite, or unrelated. In addition, they may differ in other properties, such as stability, tissue and cellular localization, temporal expression pattern, and responses to agonists or antagonists. The presence or level of specific splice variants may cause and/or indicate pathological or normal conditions. A class example is the Prostate Specific Antigen, the most important marker available today for diagnosing and monitoring patients with prostate cancer [4].

Exons are areas of genomes that can be translated to produce mRNA and finally transcribed to proteins. Different combinations of exons can be spliced together to produce different mRNA isoforms of a gene, encoding structurally and functionally different protein products [10]. Note that an *alternative splicing form* (ASF) is composed by linking several alternative splicing sites. In other words, an alternative splicing form is just a list of exons, altogether constitute an expressed mRNA. Given a (hamiltonian) directed acyclic graph $G = (V, E)$ with a source vertex $s \in V$, where the vertex set V represents exons and E represents the (possible alternative splicing) transition between exons. It follows that a (connected) path starting from s is a putative RNA transcript for a given gene of interest. The *splicing graph* G is the directed graph on the set of transcribed sites V that contains a directed edge (u, v) if and only if u and v are consecutive positions in one of the transcripts. Every transcript can be viewed as a path in the splicing graph G , and the whole graph G is the union of all paths.

*This work is supported in part by the National Science Council, Taiwan, R.O.C, grant NSC-92-2213-E-126-011.

Definition 1 (Alternative Splicing Graph)

A (hamiltonian) directed acyclic splicing graph $G = (V, E)$ is an alternative splicing graph (ASG) if G has a single starting (source) vertex $s \in V$, where the vertex set V represents exons and E represents the (alternative splicing) transition between exons.

Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing (embedding) a given alternative splicing graph on the plane.

Since an exon is just a consecutive sequence of the genomic sequence, the (5' to 3') genomic positions thus constitute a natural linear ordering. It follows that the drawing of a alternative splicing graph usually places the vertices (exons) on a horizontal line from left to right. Furthermore, the transcribing transition edges are usually drawn as arcs connecting the corresponding exons. One way of embedding these transition edges is to draw them at the upper plane above the horizontal line. The corresponding embedding is called an (*upper drawing*) *alternative splicing graph* (ud-ASG). However, allowing only upper embedding often leads to too many crossing drawn arcs on the upper plane. The number of crossing arcs can be reduced dramatically by allowing some of these arcs being drawn on the lower plane; see Figure 1 for an illustration of a ud-ASG, and its optimal (fewest crossing number) two-way embedding.

Crossing Number

Given a graph $G = (V, E)$, the crossing number is the minimum possible number of crossings with which the graph can be drawn. The crossing number of G is usually denoted by $\nu(G)$, that attracts many researchers' interests for many years citeliebers-planarizing. A graph with crossing number 0 is a planar graph. Garey and Johnson [2] showed that determining the crossing number is an NP-complete problem.

In 1954 Zarankiewicz provided what he believed was a proof for the crossing number of the complete bipartite graph [11], where he claimed that $\nu(K_{n,m}) = \lfloor \frac{n}{2} \rfloor \lfloor \frac{n-1}{2} \rfloor \lfloor \frac{m}{2} \rfloor \lfloor \frac{m-1}{2} \rfloor$. The reasoning behind the conjecture is as follows. Place $\lfloor \frac{n}{2} \rfloor$ of the vertices on the positive x axis and $\lceil \frac{n}{2} \rceil$ on the negative x axis; similarly place $\lfloor \frac{m}{2} \rfloor$ of the vertices on the positive and $\lceil \frac{m}{2} \rceil$ on the negative y

axis. Connect the vertices on the x axis with those on the y axis by drawing nm straight line edges. Hence this is a drawing on $K_{n,m}$, and the number of crossings can be determined to be exactly as stated above. However, in 1969 R. K. Guy found an error in Zarankiewicz's proof [3]. Hence Zarankiewicz's calculation is not the exact crossing number for the complete bipartite graphs, but it is at least an upper bound to the exact value. In 1971, Kleitman [7], proved that Zarankiewicz's conjecture predicts the correct values for all complete bipartite graphs $K_{5,m}, K_{6,m}$ for all positive m .

Guy [3] initiated the search for the crossing number of the complete graphs offering a conjecture as to what the number might be. Guy conjectured that $\nu(K_n) = \frac{1}{4} \lfloor \frac{n}{2} \rfloor \lfloor \frac{n-1}{2} \rfloor \lfloor \frac{n-2}{2} \rfloor \lfloor \frac{n-3}{2} \rfloor$. The construction of this conjecture is a lot more involved and subtle than Zarankiewicz's. To summarize it, the vertices of K_n are mapped to top and bottom discs of a cylinder isomorphic to the unit sphere. Then these vertices are connected to form two complete graphs isomorphic to $K_{n/2}$ on the top and bottom discs. By considering the shortest helical paths on the cylinder between the vertices on the top and bottom discs, the upper bound for the crossing number of complete graph is then obtained.

Today Guy's conjecture on the crossing number of the complete graphs and Zarankiewicz's conjecture for complete bipartite graphs have neither been proven nor disproven. However, in 1983 Garey and Johnson proved the crossing number problem to be NP-Complete and conjectured it is likely to be intractable [2]. Many improved upper and lower bounds have been found for the crossing numbers of the complete and complete bipartite graphs, but no exact solution. [8].

2 Crossing Number of ud-ASG

Here we show that computing the crossing number of a given ud-ASG can be done in $O(|E| \log |E|)$ time.

Theorem 1 *Computing the crossing number of a given ud-ASG can be done in $O(n \log n)$ time*

Proof. We propose an $O(|E| \log |E|)$ time algorithm, CROSS-UD(V, E), shown in Figure 2. The algorithm essentially accumulate numbers of crossing edges of each edge that proceeds it.

Note that two edges (x, y) and (u, v) , in the lexicographic ordering, intersect each other if and

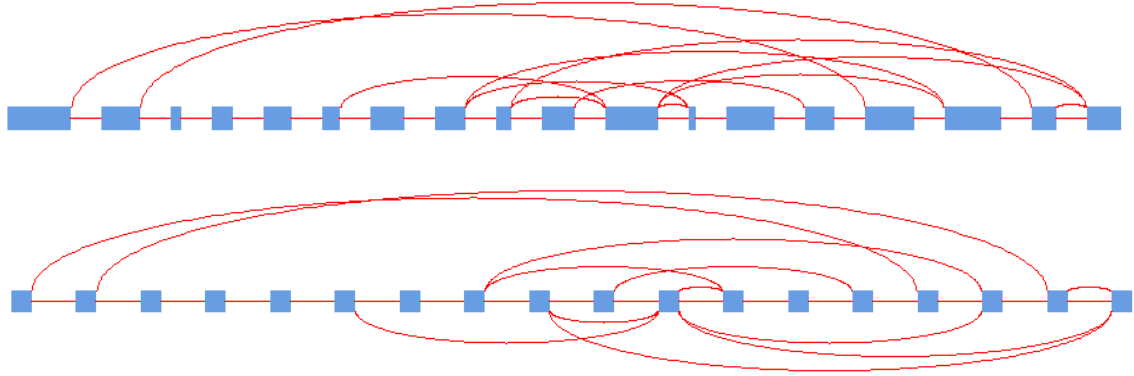


Figure 1: An (upper drawing) alternative splicing graph and its corresponding optimal (two-way) drawing.

only if $x < u < y < v$. It follows that Step 4 of the algorithm correctly discards irrelevant non-intersecting edges (x, y) 's since these edges have right endpoints $y \leq u$. It thus implies Step 5 correctly computes the number of intersecting edges (x, y) 's for the reason that all these edges (x, y) 's must have $x < u < y < v$. Note that the two edges (u, v) and (u, v') with $v < v'$ do not intersect each other since they are visited in the decreasing v 's ordering.

Clearly the sorting in Step 1 can be done in $O(|E| \log |E|)$ time. Furthermore, it is easily seen that Step 4 to Step 6 each can be computed in $O(\log |E|)$ time by maintaining some standard priority queue; e.g. a balanced binary tree with augmented data that can be used to compute the $\text{RANK}()$ operation for correctly reporting the $\text{NUM-PRED}(Q, v)$ function. Since the for loop is executed exactly $O(|E|)$ times, it follows that the algorithm finishes in $O(|E| \log |E|)$ time. $\square$

3 Optimal Drawing of ASG

As discussed in previous sections, allowing only upper embedding often leads to too many crossing drawn arcs on the upper plane. The number of crossing arcs can be reduced dramatically by allowing some of these arcs being drawn on the lower plane as illustrated in Figure 1.

Finding efficient ways of drawing alternative splicing graphs with fewest possible crossings is actually a very difficult problem. As an example, it is immediately seen that complete graphs K_n and complete bipartite graphs $K_{n,n}$ and $K_{n,n-1}$ are just a special cases of alternative splicing graphs. Thus an efficient algorithm for optimal drawing

of ASG immediately solve the long lasting open problems for computing the crossing numbers of complete graphs and complete bipartite graphs.

Here we propose a branch-and-bound algorithm for optimal drawing of ASG that efficiently finds the drawing with fewest crossings. The algorithm has been tested and implemented to incorporate with our web-based system. We undergo some experiments of the implemented programs and some benchmark data is shown later.

First we describe our branch-and-bound recursive algorithm, $\text{ASG-SEARCH}(k, B)$, in Figure 3. The ideas behind the algorithm is as following. First layout the corresponding vertices (exons) on a horizontal line as their natural ordering in the genomic sequence. Now the problem comes to decide whether a considered edge shall be placed in the upper plane or the lower plane. It follows the topological structure is decided once we figure out how to layout the edges in the upper or lower plane. Thus the embedding problem is equivalent to assign a boolean value, 0 or 1, to each edge. That is, consider the searching space $S = \{\text{str}[1..|E|] \mid \text{str}[i] = 1 \text{ or } 0\}$; i.e., $|S| = 2^{|E|}$. The algorithm essentially needs to find out the optimal element in S whose embedding produces the fewest crossing number.

A brute force approach would need to explore all $2^{|E|}$ configurations and then computing the crossing numbers for each layout resulting an $\Theta(2^{|E|} |E| \log |E|)$ time algorithm. To avoid unnecessary explorations of most inconceivably “bad” configurations, we use the idea of standard branch-and-bound technique with some “cleaver” heuristic searching strategy. In the recursive subroutine $\text{ASG-SEARCH}(k, B)$, as shown at Figure 3, the main theme can be viewed as searching a complete binary tree with depth $|E|$. The searching

CROSS-UD(V, E)

Input: A set of vertices V , and a set of linking arcs $E = \{(u, v) \mid u, v \in V\}$.

Output: The crossing number of the upper drawing alternative splicing graph, $G = (V, E)$.

```

1 Sort the edges  $(u, v)$ 's of  $E$  in increasing  $u$  (and then decreasing  $v$  when  $u$  is tight) ordering.
2  $k \leftarrow 0; Q \leftarrow \emptyset$   $\triangleright$  Priority Queue.
3 for each edge  $(u, v)$  in sorted order do
4   REMOVE-PRED( $Q, u$ )  $\triangleright$  Discard vertices at, or on the left of,  $u$ .
5    $k \leftarrow k + \text{NUM-PRED}(Q, v)$   $\triangleright$  crossing arcs.
6   INSERT( $Q, v$ )  $\triangleright$  Adding vertex  $v$ .
7 return  $k$ 

```

Figure 2: Computing the crossing number of a ud-ASG in $O(|E| \log |E|)$ time.

global $Opt[1..|E|], Temp[1..|E|]$ $\triangleright$ boolean string for drawing configuration of G
 $\triangleright Temp[i] = 0$ if and only if the edge i is drawn at the upper plane.
global $BOUND$ $\triangleright$ global upper bound for the optimal drawing

ASG-SEARCH(k, B) $\triangleright$ drawing the k -th edge of G
Input: k : the k -th edge; B : upper bound of crossings.

```

1 if  $k > |E|$  then  $\triangleright$  Terminates.
2 if  $B < BOUND$  then
3    $BOUND \leftarrow B; Opt \leftarrow Temp$ 
4 return
5  $U \leftarrow \text{UP-CROSS}(k); L \leftarrow \text{LOW-CROSS}(k)$ 
6 if  $U \leq L$  then  $\triangleright$  Greedy heuristic: smaller cost first.
7 if  $U + B \geq BOUND$  then return  $\triangleright$  Branching bound!
8  $Temp[k] \leftarrow 0; \text{ASG-SEARCH}(k + 1, U + B)$   $\triangleright$  recursive searching
9 if  $L + B \geq BOUND$  then return
10  $Temp[k] \leftarrow 1; \text{ASG-SEARCH}(k + 1, L + B)$ 
11 else  $\triangleright$  Symmetric cases
12 if  $L + B \geq BOUND$  then return  $\triangleright$  Branching bound!
13  $Temp[k] \leftarrow 1; \text{ASG-SEARCH}(k + 1, L + B)$   $\triangleright$  recursive searching
14 if  $U + B \geq BOUND$  then return
15  $Temp[k] \leftarrow 0; \text{ASG-SEARCH}(k + 1, U + B)$ 

```

Main program calls ASG-SEARCH(1, 0) with $BOUND \leftarrow \infty$.

Figure 3: A branch-and-bound algorithm for drawing ASG with fewest crossing with heuristic searching.

paths are encoded by the single boolean string $Temp[1..|E|]$. Note that a completely 0-1 filled string $Temp[]$ corresponds to a leaf node of the imaginary search tree. Thus finding the optimal configuration is equivalent to finding a leaf node with the minimum crossing number. Furthermore, once some values of some leaf nodes have been found, their minimum crossing number can be viewed as a bounded value for other unexploited paths. The currently obtained minimum crossing configuration is stored at the boolean array $Opt[]$ with the minimum crossing stored at the global variable $BOUND$.

It follows that, whenever a prefix path of some unfinished paths possess a prefix crossing number being greater than $BOUND$, the remaining paths (corresponding to some subset of the searching space S) do not need to be further exploited. Here the spirit of branching and bound is applied. It is easily seen that the Step 3 of the searching subroutine $ASG-SEARCH(k, B)$ set up the currently branching bound, $BOUND$. Also note that Steps 7, 9, 12, and 14 of $ASG-SEARCH$ are where the bounded branching take places. Most of the unneeded computation efforts are saved there.

Finally, for the branch-and-bound searching algorithm to be really efficient, it will be beneficial if we can reach some “good” answers as soon as possible. The idea is to find a relatively low value of the bounded valued, $BOUND$, as soon as possible. Thus much unwanted computation can be avoided. To do so, we employ the heuristic searching strategy at Step 5 and Step 6 of $ASG-SEARCH$. Note that Step 5 computes $UP-CROSS(k)$ and $LOW-CROSS(k)$, these are the crossing number caused by k -th edge when it is placed at the upper plane and lower plane. The algorithm then further searching downward which one comes smaller. The idea behind these two lines is the greedy approach. Whenever the k -th edge is in consideration, it would be wiser to place it in the (upper or lower) plane where it cause less cost.

4 Experimental Results

The resulting branch and bound searching routine is rather efficient. For example, in our implementation of the algorithm in C , that is run under the Red Hat Linux 7.2 system with experimental machine equipped with Pentium-4 CPU for 1000Mhz. Just as an example of showing the efficiency of the heuristic branch and bound algo-

rithm, we note that the program correctly finds the embedding of complete graph K_9 in 1 seconds; and the program correctly finds the optimal embedding of complete graph K_{10} in 83 seconds. The resulting crossing numbers $\nu(K_9) = 36, \nu(K_{10}) = 60$ are correctly reported by the program. In contrast, note that K_9 possess 36 edges and K_{10} has 45 edges; that is, the searching spaces have sizes $2^{36} = 68,719,476,736, 2^{45} = 35,184,372,088,832$. It is estimated that the program explores about 1/1000 over the whole searching space; the computation speed up is about 1,000 times faster comparing to the brute force approach in these cases. Here we summarize our experiments for searching the optimal embedding of some complete graphs complete bipartite graphs in Table 1.

Most of the program for finding the optimal embedding of the alternative splicing graphs have been incorporated with our quantitative alternative splicing analysis system web site at <http://bioinfo.cs.pu.eud.tw/ASF/dasg> [1]. The system with easy internet accessibility is mostly built by using the conventional PHP system, that has a long reputation of being easily integrated with HTML and CGI programs; the optimal embedding program is one of these CGI programs and have been equipped with PHP programs for drawing the resulted optimal embedded ASG for visualization.

5 Concluding Remarks

Due to all these important aspects of genomic alternative splicing mechanism that produces different mRNA's, biologists require means of analyzing and visualizations of the alternative splicing forms. In this paper, we consider ways of drawing a given alternative splicing graph on the plane.

Given a graph $G = (V, E)$, here in this paper we show that computing the crossing number of a given upper drawing ASG can be done in $O(|E| \log |E|)$ time.

The optimal (fewest crossing) drawing alternative splicing graphs is related to the long known difficult problem of crossing number problem [3, 2, 8]. Here we propose an efficient branch-and-bound recursive algorithm with heuristic searching strategy. Some experimental results are also discussed in the paper.

G	n	m	$\nu(G)$	T	G	n	m	$\nu(G)$	T
K_3	3	3	0	< 0.01					
K_4	4	6	0	< 0.01					
K_5	5	10	1	< 0.01	$K_{2,2}$	4	4	0	< 0.01
K_6	6	15	3	< 0.01	$K_{3,3}$	6	9	1	< 0.01
K_7	7	21	9	< 0.01	$K_{4,4}$	8	16	4	< 0.01
K_8	8	28	18	< 0.01	$K_{5,5}$	10	25	16	< 0.01
K_9	9	36	36	1.08	$K_{6,6}$	12	36	36	0.265
K_{10}	10	45	60	82.4	$K_{7,7}$	14	49	81	261.5
K_{11}	11	55	100	2332	$K_{8,8}$	16	64	144	261,979

Name: G : graph; n : number of nodes; m : number of edges;
 $\nu(G)$: crossing number; T : CPU time in seconds).

Table 1: Searching for the optimal embedding of some complete graphs.

Acknowledgements

The author would like to thank my graduate students Po-Shun Yu, Hsun-Chang Chang, and Tze-Wei Huang for their assistance of building up the Web system, and we thank many instructive discussions and information concerning the biological processes of alternative splicings within the Providence university Bioinformatic forum [1].

References

- [1] Providence University Bioinfo Forum. <http://bioinfo.cs.pu.edu.tw>.
- [2] M. R. Garey and D. S. Johnson. Crossing number is NP-complete. *SIAM Journal of Algebraic and Discrete Methods*, 4:312–316, 1983.
- [3] Richard K. Guy. Crossing numbers of graphs. In *Graph Theory and Applications*, Lecture Notes in Mathematics 303, pages 111–124. Springer-Verlag, 1972.
- [4] Nathalie Heuze, Sophie Olayat, Ninette Gutman, Marie-Louise Zani, and Yves Courty. Molecular cloning and expression of an alternative hklk3 transcript coding for a variant protein of prostate-specific antigen. *Cancer Res.*, 59:2820–2824, 1999.
- [5] Zhengyan Kan, Eric C. Rouchka, Warren R. Gish, and David J. Gene structure prediction and alternative splicing analysis using genomically aligned ests. *Genome Res.*, 11:889–900, 2001.
- [6] Zhengyan Kan, David States, and Warren Gish. Selecting for functional alternative splices in ests. *Genome Res.*, 12:1837–1845, 2002.
- [7] D. J. Kleitman. The crossing number of $K_{5,n}$. *Journal of Combinatorial Theory*, 9:315–323, 1971.
- [8] Annegret Liebers. Planarizing graphs - a survey and annotated bibliography. *Journal of Graph Algorithms and Applications*, 5(1):1–74, 2001.
- [9] A. J. Lopez. Alternative splicing of pre-mrna: developmental consequences and mechanisms of regulation. *Annu. Rev. Genet.*, 32:279–305, 1998.
- [10] Qiang Xu, Barmak Modrek, and Christopher Lee. Genome-wide detection of tissue-specific alternative splicing in the human transcriptome. *Nucleic Acids Res.*, 30:3754–3766, 2002.
- [11] K. Zarankiewicz. On a problem of P. Tur'an concerning graphs. *Fund. Math.*, 41:137–145, 1954.

Ordering Analysis of Gene Expression Dynamics

Chia-Lin Shen, Kuei-Jen Lee, Hsiang-Chuan Liu

Department of Bioinformatics
Taichung Healthcare and Management University
std91242004@ms1.thmu.edu.tw

Abstract

DNA microarrays technology can be used to discover the expression of thousands of genes and the regulatory relationship between them. This article presents an ordering method to analyze gene expression dynamics, and find the ordering relationship, such as the upstream or downstream between these genes. Our ordering method has two components: we use an ordering index (OI) combined with standard statistic significant test to analyze gene expression data. We use ordering index to measure the association of genes through the dynamics of gene expression. Because the characteristics of our ordering index are same to the 「conditional probability」, which make the model can present the direction of gene controlling. The hypothesis direction can provide us more information about the regulatory mechanisms of genes. And the standard statistic significant test can make the judgments objective.

1 Introduction

The human genomic project has provided us with a lot of data and biological knowledge, and the development of cDNA microarrays technology has given scientists more opportunities to study physiology and gene function in depth. One of the advantages of this technology is that enable investigators to simultaneously measure the expression of thousands of genes, and find the gene expression patterns that control our body. When analyzing the different gene expression levels, we can do systematical analysis about the main function of the genes and the cross relationship of between them through this new technology.

The technique of cDNA Microarrays allows us to analyze a great number of genes at a time. This is a very powerful tool when analyzing the whole genome. There are a lot of methods developed to analyze the gene expression data that obtained from cDNA Microarrays experiments. According the complex levels, we can roughly separate them

into about two levels.

First, by using the differences of gene expression level, we can decide the gene expression level if the statistical significance can be qualified on the cell situation in this time. Typically, the standard statistic procedure can be used. The statistical t-test is the most useful procedure, and the t-statistic results can be changed to become a score to evaluate the significance [1]. There is another method that can get more background message using the prior knowledge through the Bayesian statistics [2].

Secondly, a more complex way is used to analyze the mutual relationship between genes or a series of genes. Use the gene expression pattern and the level of gene expression to analyze the relationship between the genes that we are interesting. Methods using in this level are more complex than the first level. A common method uses the Pearson correlation coefficient to construct a gene similarity matrix, and then it can use the hierarchical clustering algorithm based on the average-linkage method to do the classification of the genes [3, 4].

On the other hand, the classification based on the Pearson correlation coefficient are easily influential by the data that obtained from the experiments, such as the outlier or the changing condition of experiment resulted the data can not present on the normal level. We can treat each experiment data as a random variable. To prevent the research bias because of the random variable, we can choose the entropy as the measurement of the gene expression level, because entropy does not influenced by the value of random variable which is the measurement of the uncertainty of the distribution. Algorithm using the maximum entropy can analyze the genome-wide expression patterns [5]. To construct relevance networks using the pair-wise mutual information of the gene can present the relationship more clearly [6, 7]. This method set up threshold mutual information (TMI=1.3) by experience to judge the associations of the genes exists or not. Although to set up a TMI and to construct a relevance network can be more easily to present the mutual relationship of

the genes, but it also has a disadvantage. That is, this method can not detect the association with the mutual information lower than the threshold.

Our method uses the 「ordering index」 (OI) which comes from the procedure that using mutual information divides to the marginal entropy, and replaces 「mutual information」 to measure the association of genes through the gene expression data. The result presents the aspect of the gene association. Because the characteristics of our ordering index are the same to the 「conditional probability」, which make the model having the ability to present the controlling direction of the genes. Because of our ordering index has the range between 0 to 1, and there are an exact relation between mutual information and the correlation coefficient derived from two variables [8]. So we can approximate change the value to the t-statistic because of OI is similar to the one-way correlation coefficient. Then use the right-tail hypothesis test to the one-way correlation relationship and determine if the OI is significant enough to say this pair of the genes existing ordering relationship.

The hypothesis direction can provide more information about the gene expression dynamics. And the statistic significant test can make the judgments more objective than the threshold TMI determined by subjective experience. Here, we use this hypothesis association to present the ordering relationship of the genes. Therefore, we can use the direction of OI to represent the controlling direction of the genes.

2 METHOD

First calculated the entropy of each gene and the pair-wise mutual information of the gene pair. Then we calculated OI of each gene pair as the measurement of the ordering relationship.

Secondly, the use of the statistical test replaces the critical value to judge OI, and take the pairs of genes with the statistical significance to bind the association network of genes. Because of the direction characteristic containing in 「ordering index」, somehow the result gets from the model has the ability to present the controlling direction of the genes.

Uniform distribution has the maximum entropy, so if the entropy of the gene expression data is higher, and it means that its expression levels are more randomly distributed.

In general, the entropy of gene A, $H(A)$, is defined by equation (1)

$$H(A) = -\sum_{i=1}^m p(a_i) \log_2 p(a_i) \quad (1)$$

By calculating the entropy as the measurement of gene expression level, we use a histogram technique [6] to calculate the range of value for each gene expression data, divide the range into 5 sub-interval ($m=5$), and then count the relative frequency $p(a_i)$ of data which lie in each sub-interval a_i .

The joint entropy of the two genes A and B, $H(A, B)$, is defined by equation (2)

$$H(A, B) = -\sum_{i=1}^m \sum_{j=1}^m p(a_i, b_j) \log_2 p(a_i, b_j) \quad (2)$$

Here the $p(a_i, b_j)$ represents the relative frequency which lies in the cell (a_i, b_j) . One advantage of using entropy is that we can present the variation of gene expression level, and it does not need to consider the question about normalization, so that it will not meet the question about suitable normalization equation.

Using the entropy to calculate the mutual information of pair-wise gene, as shown in equation (3)

$$MI(A, B) = H(A) + H(B) - H(A, B) \quad (3)$$

The higher mutual information shows the higher association between two genes. Because mutual information represents the association, so we choose $MI(A, B)$ to represent the association between genes. Although the value of mutual information will not be a negative value ($MI(A, B) \geq 0$), the value still could be higher than 1, which is not an ideal measurement. Considering the property of $MI(A, B) \leq H(B)$, we can extend the definition to the ordering index, $OI(A/B)$, as shown in equation (4)

$$OI(A/B) = \frac{MI(A, B)}{H(B)} \quad (4)$$

Because of our ordering index follow the form of the 「conditional probability」, so we can see the OI has the same characteristics with 「conditional probability」 which make our model can present ordering relationship between data. And we use the ordering relationship to represent the direction of gene controlling. The hypothesis direction can

provide us more information about the regulatory mechanisms of gene. Ordering index restricts the direction of regulatory association of the genes, so we use $OI(A/B)$ to represent the ordering relationship from gene A to gene B. $OI(A/B)$ is always not equal to $OI(B/A)$. It means the relation can not be reversible. There is another useful property of $OI(A/B)$, i.e. $0 \leq OI(A/B) \leq 1$.

The range of $OI(A/B)$ is approximate to the correlation coefficient. We can use statistical hypothesis test to judge the significant of the ordering relationship, i.e. $OI(A/B)$ [9]. Statistical hypothesis test provide us more objective information than the methods that using the critical value to be a threshold. Statistical hypothesis test can provide the estimation of false positive rate and false negative rate. Because of the value of $OI(A/B)$ just between 0 to 1, so we can change $OI(A/B)$ to t-statistic than use the right-tail t-test to assess the ordering relationship between gene A and gene B.

$$H_0 : r \leq 0 \text{ vs } H_1 : r > 0 \text{ at } \alpha = 0.05/0.01$$

$$t_{AB} = \frac{OI(A/B)}{\sqrt{\frac{1 - [OI(A/B)]^2}{n-2}}} \sim t(n-2) \quad (5)$$

If $t_{AB} > t_{1-\alpha}(n-2)$, it represents the existence of the association from A to B, where n = the number of sample.

In this paper, we use the hypothesis test of correlation coefficient to carry out the test of OI , because it approximates the correlation coefficient in which the degree of freedom is $(n-2)$. Through the above null hypothesis H_0 and the t-statistics, we can analyze the association of genes, and in theory, we can control the error rate under the level as we expect, through the choice of the value of α .

3 Experimental data set

We use the public data set obtained from the Stanford Microarray database SMD [10]. We obtain the stomach cancer data set, which we draw out 17 p53-related genes as a small data set, and each gene has 20 samples. And the entire gene name refers from the ExPASy Molecular Biology Server [11]. We use these genes to test the ability of the explanation and the reasonability of our model.

This small data set contains normal and tumor samples, which come from 20 patients with each gene expression level. The genes that we choose

are shown in Table I.

Table I

Gene Symbol		Gene Name
TP53BP1	G1	tumor protein p53 binding protein, 1
P5326	G2	hypothetical protein p5326
NM23-H6	G3	nucleoside diphosphate kinase type 6 (inhibitor of p53-induced apoptosis-alpha)
TP53TG3	G4	TP53TG3 protein
PIGPC1	G5	p53-induced protein PIGPC1
TP53INP1	G6	tumor protein p53 inducible nuclear protein 1
RRM2B	G7	ribonucleotide reductase M2 B (TP53 inducible)
MDM2	G8	**Mdm2, transformed 3T3 cell double minute 2, p53 binding protein (mouse)
TP53BP2	G9	**tumor protein p53 binding protein, 2
TP53	G10	tumor protein p53 (Li-Fraumeni syndrome)
TP53TG1	G11	TP53 target gene 1
REPRIMO	G12	candidate mediator of the p53-dependent G2 arrest
PP5395	G13	hypothetical protein PP5395
PIG11	G14	**p53-induced protein
DDA3	G15	p53-regulated DDA3
TP53BPL	G16	tumor protein p53-binding protein
PA26	G17	p53 regulated PA26 nuclear protein

All these show in Table I. are p53-related genes, which contain the upstream and downstream genes of the p53.

4 RESULTS

We use the method with $\alpha=0.05$ and $\alpha=0.01$ to analyze the association of these genes, and we obtain the relevance networks with hypothesis direction of the gene controlling direction. The value of α means the probability which we reject the hypothesis wrongly. When the value of α is chosen, we choose the probably error rate of the association when we consider this association is true.

In another word, we choose the false positive rate that we can bear. Hence, when we choose $\alpha=0.01$, this means we just want to bear 1% false positive rate.

We calculate both directions of OI , which

Table II.

		OI (A/ B) →																
$\alpha=0.05$		G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14	G15	G16	G17
←OI (B/ A)	G1		-	-	-	-	-	*	-	*	-	-	-	*	-	-	-	-
	G2	-		*	*	-	-	-	-	-	-	-	-	-	-	-	-	-
	G3	-	*		-	-	-	-	-	-	-	-	-	-	-	-	*	-
	G4	-	*	-		-	-	-	-	*	*	-	-	*	*	*	-	-
	G5	-	-	-	-		-	*	-	-	-	-	*	*	-	-	-	*
	G6	-	-	-	-	-		-	-	*	-	-	-	-	-	-	-	-
	G7	-	-	-	-	-	-		-	-	-	-	-	-	-	-	-	-
	G8	-	-	-	-	-	-	-		-	-	-	-	-	-	-	-	-
	G9	*	-	-	-	-	-	*	-		*	-	-	*	-	-	-	-
	G10	-	-	-	-	-	-	-	-	*		-	-	-	-	-	-	-
	G11	-	-	-	-	-	-	-	-	-	-		-	-	-	-	-	-
	G12	-	-	-	-	*	-	-	-	-	*	-		-	*	-	-	-
	G13	*	-	-	*	*	-	-	-	*	-	-	-		-	*	-	-
	G14	-	-	-	-	-	-	-	-	-	-	-	*	-		*	-	-
	G15	-	-	-	*	-	-	-	-	-	-	-	-	-	*		*	-
	G16	-	-	*	-	-	-	-	-	-	-	-	-	-	-	*		-
	G17	-	-	-	-	*	-	-	-	-	-	*	-	-	-	-	-	-

express by OI (A/B) and OI (B/A), and we can get pair-wise OI as we show below.

Table II presents the association in which OI is significant enough when we take the value of $\alpha = 0.05$. The direction of OI(A/B) contains 21 pairs of gene which is significant enough. As shown in Table II, we can see that these pairs of gene have relationship and the relationship has the direction A->B. Another direction OI(B/A) contains 18 pairs of genes which is significant enough and have the relationship with direction B->A. Fourteen of these have both direction significance. We use the symbol “*” to represent the pair of gene with significant OI under $\alpha = 0.05$.

We use the OI as the measurement of the association. This means we assume that the result of our experiment can give hypothesis direction of the gene. If the pair of the gene only has one-way association indicated these two genes may have the upstream and downstream relationship. This direction obeys the direction of OI. If both directions of the pair of genes are significance that may means these two genes interaction with each other or that may have another gene interaction with these two genes in the pathway. Next we will use the network that find on $\alpha = 0.01$ to analyze

above assumption. Table III presents the association in which OI is significant enough when we take the value of $\alpha = 0.01$. The direction of OI(A/B) contains 5 pairs of gene which are significant enough. We can see these pairs of gene have relationship and the relationship has the direction A->B. Another direction OI(B/A) contains 6 pairs of genes which are significant enough and have the relationship with direction B->A. 2 of these have both direction significant. We use the symbol “**” to represent the pair of gene with the significance of OI under $\alpha = 0.01$. We can use Figure I to present the frequencies of association when the value of $\alpha = 0.05$ and $\alpha = 0.01$.

Figure I

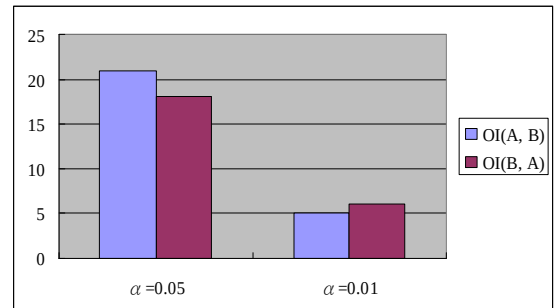


Table III

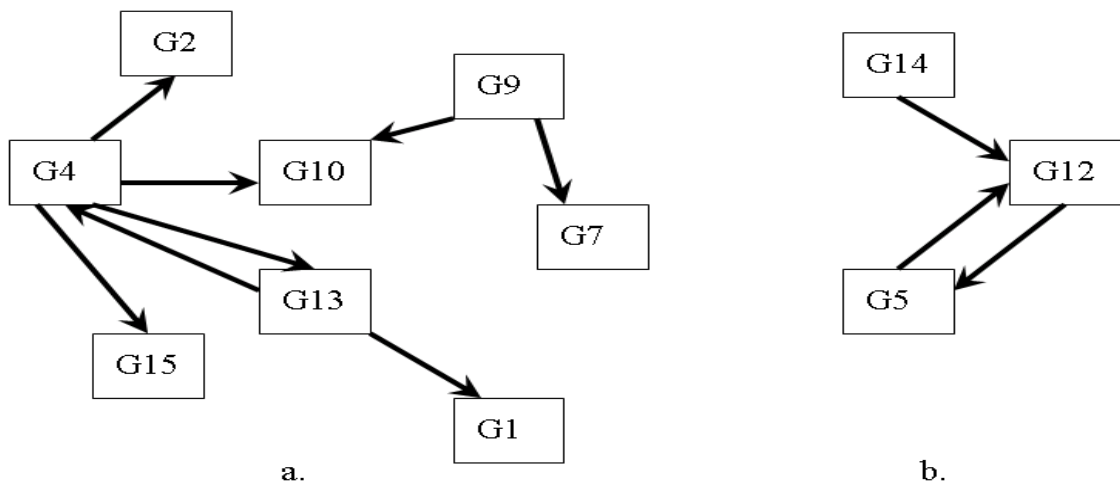
$\oplus$		OI (A/ B) $\rightarrow$																	
		$\alpha=0.01$	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14	G15	G16	G17
$\leftarrow$ OI (B/ A)	G1		-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	G2	-		-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	G3	-	-		-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	G4	-	**	-		-	-	-	-	-	**	-	-	**	-	**	-	-	-
	G5	-	-	-	-		-	-	-	-	-	-	**	-	-	-	-	-	-
	G6	-	-	-	-	-		-	-	-	-	-	-	-	-	-	-	-	-
	G7	-	-	-	-	-	-		-	-	-	-	-	-	-	-	-	-	-
	G8	-	-	-	-	-	-	-		-	-	-	-	-	-	-	-	-	-
	G9	-	-	-	-	-	-	**	-		**	-	-	-	-	-	-	-	-
	G10	-	-	-	-	-	-	-	-	-		-	-	-	-	-	-	-	-
	G11	-	-	-	-	-	-	-	-	-	-		-	-	-	-	-	-	-
	G12	-	-	-	-	**	-	-	-	-	-	-		-	-	-	-	-	-
	G13	**	-	-	**	-	-	-	-	-	-	-	-		-	-	-	-	-
	G14	-	-	-	-	-	-	-	-	-	-	-	**	-		-	-	-	-
	G15	-	-	-	-	-	-	-	-	-	-	-	-	-	-		-	-	-
	G16	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		-	-
	G17	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		-

Figure I shows when we reduced the false positive rate we can bear ($\alpha = 0.05$ change to $\alpha = 0.01$), we can find the less number of pair that is significant enough. If all genes in the data set have truly association, the value of α change may represent the distance of these associated genes. In fact, most of the association we found have lower MI than TMI = 1.3. However, these relationships

seem reasonable. That means using our method can find the association of gene more objective, and this method may find more pairs of gene which have association but do not have particular high MI. Also it can prevent higher rate of false negative.

We present the relevant network of Table III in Figure II.

Figure II.



As shown in Figure II, we can see the Figure II present two networks of gene, Figure II (a) and Figure II (b). Figure II (a) contains 8 genes, and we can see their relationship very easily and clearly. G9 represents the tumor suppressor p53-binding protein 2 (TP53BP2), which function is a regulator that plays a central role in regulation of apoptosis and cell growth via its interactions, which regulates TP53 by enhancing the DNA binding and transactivation function of TP53 on the promoters, belonging to the upstream gene of P53. G10 represents the tumor protein p53 which belongs to the downstream gene of G9. G7 represents the ribonucleoside diphosphate reductase M2, whose function provides the precursors necessary for DNA synthesis, which also belongs to the downstream gene of G9.

We can see that the network based on our model is a reasonable network.

Beside two hypothetical proteins G2 and G13, the rest of these genes contain G4 and G4 represents the TP53-inducible gene (TP53 target gene 3) which is suspected that this protein plays an important role in the TP53-mediated signaling pathway [12]. This gene may have interaction with P53. G15 represents p53-regulated DDA3 and G1 represents tumor suppressor p53-binding protein 1. This network seems reasonable.

In Figure II (b), G14 represents p53-induced protein 11 which is apoptosis regulator. G12 represents candidate mediator of the p53-dependent G2 arrest which is cell-cycle control gene, G5 represents P53-induced protein PIGPC1, and all belonging to downstream genes of P53 should have interaction with each other.

5 DISCUSSIONS

There are two main advantages in our model. One is that we can present a relation network with suggestive direction. Even some of them are still blurred, but this suggestive direction can indicate the gene regulatory direction under this specific situation maybe some kind of disease or some specific cell condition that depending on the data we got. Thus we can get more information than other method, and we can understand the relation of gene regulatory network more clearly.

The other is that we can use the statistical test to evaluate the significant level of OI and this is a more objective method, because in this test we can control the false positive rate and can prevent increasing false negative rate.

We suggest there are several questions and potential development still waiting for solving. One of these is to set up an automatic analysis procedure, allow this model can analyze mass data

automatically. This should be our short term work. When analyzing this model, we find the OI or t-statistics seem containing the distant meaning inside, and find a useful method to determine the relation distance which may be a potential work. Besides, the gene pair that contains two directions needs mass data to analyze to find the clear relationship. This may be an interesting work.

6 Acknowledge

Acknowledge professor Rong-Ming Chen and graduate student Guey-Jong Shyr in the multimedia labs of Taichung Healthcare and Management University. They help us to collect the data.

References

- [1] Virginia Goss Tusher, Robert Tibshirani, and Gilbert Chu. Significance analysis of microarrays applied to the ionizing radiation response. *Proc. Natl Acad. Sci. USA*, April 24, 2001 vol. 98, 5116–5121.
- [2] Pierre Baldi and Anthony D. Long. A Bayesian framework for the analysis of microarray expression data : regularized t-test and statistical inferences of gene changes. *Bioinformatics*, Vol. 17 no. 6 2001, Pages 509–519.
- [3] Eisen,M.B., Spellman,P.T., Brown,P.O. and Botstein,D. (1998). Cluster analysis and display of genome-wide expression patterns. *Proc. Natl Acad. Sci. USA*, 95, 14 863–14 868.
- [4] Xin Chen, Siu Tim Cheung, Samuel So, Sheung Tat Fan, Christopher Barry, John Higgins, Kin-Man Lai, Jiafu Ji, Sandrine Dudoit, Irene O.L. Ng, John Higgins, Kin-Man Lai, Jiafu Ji, Sandrine Dudoit, Irene O.L. Ng. Gene Expression Patterns in Human Liver Cancers. *Molecular Biology of the Cell* Vol. 13, 1929–1939, June 2002
- [5] Christopher James Langmead, C. Robertson McClung, Bruce Randall Donald. A Maximum Entropy Algorithm for Rhythmic Analysis of Genome-Wide Expression Patterns. *Proc. the IEEE Computer Society Bioinformatics Conference (CSB)*, Stanford University, Palo Alto, CA August 14 - 16, 2002.
- [6] A. J. Butte and I. S. Kohane. Mutual Information Relevance Networks: Functional Genomic Clustering Using Pairwise Entropy Measurements. *Pacific Symposium on Biocomputing* 5: 418-29 (2000).
- [7] Gregory t. Klus , Andrew Song , Ari Schick,

- Mattias Wahde and Zoltan Szallasi. Mutual information analysis as a tool to assess the role of aneuploidy in the generation of Cancer-associated differential gene expression patterns. Department of Pharmacology, Uniformed Services University of the Health Sciences, Bethesda, MD, Div. of Mechatronics, Chalmers University of Technology, Göteborg, Sweden
- [8] Wentian Li. Mutual information functions versus correlation functions. *Journal of Statistical Physics*, 60, 823-837 (1990).
 - [9] Hsiang-Chuan Liu. Ordering theory base on ordering index. *Journal of Educational Measurement and Statistics* Vol.14, 1-11. (2004) (in press).
 - [10] Stanford Micorarray Database <http://genome-www4.stanford.edu/MicroArray/SMD/>.
 - [11] ExPASy Molecular Biology Server <http://us.expasy.org/sprot/>.
 - [12] Ng CC, Koyama K, Okamura S, Kondoh H, Takei Y, Nakamura Y. Isolation and characterization of a novel TP53-inducible gene, TP53TG3. *Genes Chromosomes Cancer* 26:329-335, 1999. Copyright 1999 Wiley-Liss, Inc.

RNA Secondary Structure Alignment Based on Stem Representation *

Meng-Yi Wu, Chang-Biau Yang[†] and Kuo-Si Huang

Department of Computer Science and Engineering

National Sun Yat-sen University, Kaohsiung, Taiwan

[†]cbyang@cse.nsysu.edu.tw

Abstract

The comparison methods for RNA or protein molecules are important and basic tools in molecular biology. So far, most comparison methods, such as sequence alignment, are only applicable to the primary structures of biomolecules. Indeed, the functions of biomolecules have close relationship in their structures. The RNA secondary structure alignment problem is to align two given RNA structures to get the structure similarity. In addition, it is also helpful to predict the functions of biomolecules and to classify them. In this paper, we design a dynamic programming method for aligning two RNA secondary structures that do not contain any pseudoknot. The time complexity of our algorithm is $O(N^4)$, where N is the number of blocks contained in the given RNAs. We also apply our algorithm to real biomolecules, the tRNAs of Homo sapiens mitochondrion, to evaluate the practicability of our algorithm. We take three tRNA genes, TRNG, TRNA and TRNV, to test the performance of our algorithm. From the viewpoint of human eyes, the structure of TRNG is more similar to TRNA. Our algorithm also gets this result.

1 Introduction

It is well-known that RNAs play an important role for the biochemical interactions, such as RNA transcription, reverse transcription, protein translation, and mRNA splicing. Some RNAs are also ribozymes, and some of them contain genetic information of viruses, such as HIV, SARS, and flu. The functions of RNAs are corresponding to their structures and RNA-protein interactions.

Actually, it is not easy to get the exact structure of RNA, because the structure may be different in various environments, such as temperature and pressure. Because of the importance of RNA structure, some methods have been proposed to predict the RNA structures [7, 10, 13, 17, 20].

The functions of biomolecules have close relationship in their structures. Therefore, the structure comparison methods for RNA or protein molecules are important and they are basic tools in molecular biology. So far, most comparison methods are only applicable to the primary structures of biomolecules. In general, many RNA sequences have very high similarity, but their secondary structures or functions may not be very similar. In other words, many RNAs have similar secondary structures and functions, but their identities may not be high enough. Hence, this paper pays attention to the RNA secondary structure alignment problem. Our goal is to align two RNA molecules with given secondary structures to get the structure similarity.

For distinguishing different RNA structures, our important job is to construct the structure similarity evaluation method between RNAs. Since there are many sequence alignment methods, some researchers try to incorporate the secondary structure information into primary structure [1, 3, 9]. Lin et al. [9] proposed two novel operations, arc-altering and arc-breaking. In an RNA structure alignment, an arc-altering operation means that one nucleotide is deleted and another nucleotide is aligned to a nucleotide of the other RNA, and an arc-breaking operation means that two nucleotides identically are aligned to two nucleotides that are not a base pair of the other RNA. The above two operations make the problem harder, but the biological meanings of these operations may need more discussion.

In another way, an RNA structure may be represented as a tree. Several algorithms try to adopt

*This research work was partially supported by the National Science Council of the Republic of China under contract NSC-92-2213-E-110-005.

the methods for solving tree problems to reconstruct the alignment and to evaluate the similarity between two RNAs [4–6, 11, 14, 15, 18]. Some methods [5, 6, 14, 15] do not use RNA nucleotide information, because it is difficult to represent the stems and loops in a tree. Zhang [18] proposed a method to solve WLCS (weighted largest common sub-structure) problem on two RNA trees by applying the ordered tree edit distance algorithm [19].

Lu et al. proposed another measurement of edit distance between labeled trees [11], which can be applied on the comparison of RNA secondary structures. If RNA secondary structures can be represented as labeled trees, one may calculate the edit distances between these labeled trees for comparing and classifying RNA secondary structures. Their method is an alternative measurement of the similarity between RNA secondary structures.

From the viewpoint of RNA structures with pseudoknots, some researchers also studied the RNA structure alignment problem [2, 8, 12, 16]. For two given RNAs, which one structure is known and the other structure is unknown, Lenhof et al. [8] proposed a polyhedral approach to align the two RNAs. The approach can be applied to two RNA structures which have tertiary interactions or pseudoknot. If the structures of two RNAs are known, there are many methods to calculate the similarity or to align these two RNAs [2, 12, 16]. Ma et al. [12] proved that calculating edit distance of two RNAs is an NP-hard problem, where the structures of RNAs contain pseudoknots.

From the molecular biological or biomedical viewpoint, the stems and loops in an RNA structure may possess some functions or induce the interactions with other molecules, proteins and complexes. The stems and loops should be important characteristics for RNA structures. If we regard stems or loops as blocks, an RNA sequence can be represented as a sequence with stem blocks and loop blocks. The block-representing RNA sequences lead to a novel RNA structure similarity and alignment problem. In this paper, we propose a dynamic programming algorithm for calculating the similarity and alignment of two block-representing RNAs with stems and loops.

The rest of this paper is organized as follows. In Section 2, we introduce the RNA structures and related definitions. Our RNA secondary structure alignment algorithm based on stem representation is described in Section 3. We improve our algorithm to reduce the number of required indices in Section 4. Section 5 shows our experimental re-

sults and we conclude this paper in Section 6.

2 Preliminaries

RNA is a single strand, composed of nucleotides (bases) adenine (A), guanine (G), cytosine (C) and uracil (U). The sequence consisting of the nucleotides A, G, C and U is called the *primary structure* of RNA. The *secondary structure* of an RNA is formed by its primary structure which is folded back on itself. In a secondary structure of an RNA, there are three possible types of base pairs: $G \equiv C$, $A = U$, and $G - U$ with triple, double and single hydrogen bond, respectively.

An RNA sequence is represented as a string of N_R elements $R = r_1 r_2 \cdots r_i \cdots r_{N_R}$, where $r_i \in \{A, G, C, U\}$, $1 \leq i \leq N_R$. (r_i, r_j) denotes a base pair of a secondary structure of R , where $1 \leq i < j \leq N_R$. For convenience, given two nucleotides x and y in an RNA sequence, if x is closer to 5' and y is closer to 3', we call x is in the *left position* of y and y is in the *right position* of x .

We can cut an RNA secondary structure into a linear structure that contains the information of the secondary structure. In the linear structure, there are many blocks according to the information of the secondary structure. A *block* is defined to be a contiguous sequence (or string) that is composed of several nucleotides. Hence, the *block string* of N_B elements is denoted as $B = b_1 b_2 \cdots b_i \cdots b_{N_B}$, where b_i is a block, $1 \leq i \leq N_B$.

We define a *stem* as a group of stacked pairs, and the stem in an RNA secondary structure is contiguous among these stacked pairs. In a stem, the stacked pairs can be divided into two *stem blocks* according to their indices. The *left stem block* is closer to 5', and the *right stem block* is referred to as the block closer to 3'. We use $\langle b_i, b_j \rangle$ to denote the two stem blocks in the same stem, where b_i and b_j are the left and right stem blocks, respectively. Another group of contiguous unpaired nucleotides is called *loop block*. The block string B of an RNA secondary structure consists of loop blocks and stem blocks.

The following example illustrates an RNA structure.

$$R = A_1 G_2 G_3 C_4 C_5 A_6 A_7 U_8 U_9 G_{10} C_{11} U_{12} G_{13} C_{14} A_{15} \\ A_{16} G_{17} G_{18} C_{19} C_{20} U_{21} G_{22} C_{23} C_{24} C_{25} A_{26} C_{27} G_{28}$$

As shown in Figure 1, (G_2, U_9) and (G_3, U_8) are base pairs in R , where base pairs G_2 and G_3 are contiguous, U_8 and U_9 are also contiguous. The set of stacked pairs (G_2, U_9) and (G_3, U_8) form one stem. The left stem block is composed of G_2 and

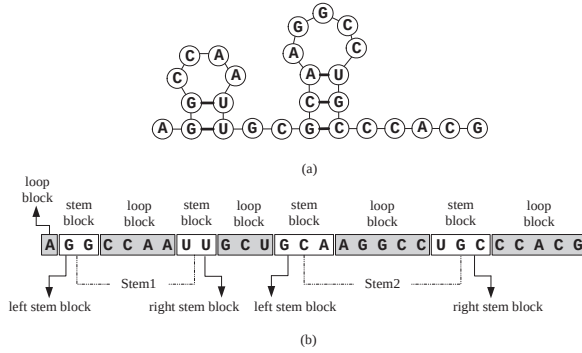


Figure 1: The secondary structure and the blocks of RNA sequence $R = \text{AGGCCAAUUGCUGCAAGGCCUGCCACG}$. (a) The secondary structure of R . (b) All loop blocks and all stem blocks of R , where a stem has left stem block and right stem block.

G_3 . The right stem block is composed U_8 and U_9 . Similarly, the base pairs (G_{13}, C_{23}) , (C_{14}, G_{22}) and (A_{15}, U_{21}) form another stem, where the left stem block is composed of G_{13} , C_{14} and A_{15} , and the right stem block is composed U_{21} , G_{22} and C_{23} . In addition, C_4 , C_5 , A_6 and A_7 are contiguous unpaired nucleotides. In fact, $C_4C_5A_6A_7$ is one of loop blocks in R .

The RNA secondary structure has two types named *non-pseudoknot* and *pseudoknot*. The *non-pseudoknot* type has no cross for each stem of an RNA secondary structure, the *pseudoknot* type may have crossing stems in an RNA secondary structure. For example, consider a string block $B = b_1b_2b_3b_4b_5b_6$, where b_2 and b_4 are loop blocks and other blocks are stem blocks. If $\langle b_1, b_6 \rangle$ and $\langle b_3, b_5 \rangle$ are two stems, we say B belongs to the non-pseudoknot type. If $\langle b_1, b_5 \rangle$ and $\langle b_3, b_6 \rangle$ are two stems, then they form a cross and the RNA B has a pseudoknot. Note that our algorithm only considers the RNA structures without any pseudoknot.

3 RNA Secondary Structure Alignment Algorithm

In this section, we shall propose a new dynamic programming method for performing an alignment on two RNA secondary structures. For convenience, we regard the two RNA sequences as the master sequence and the slave sequence.

The *master* sequence is of length N_R , denoted as $R = r_1r_2 \cdots r_{N_R}$, and the *slave* sequence is of length N_{R^*} , denoted as $R^* = r_1^*r_2^* \cdots r_{N_{R^*}}^*$. If there are N_B blocks contained in the secondary structure of R , the N_B blocks form a block string $B = b_1b_2 \cdots b_{N_B}$. Similarly, $B^* = b_1^*b_2^* \cdots b_{N_{B^*}}^*$, where R^* contains N_{B^*} blocks.

An RNA secondary structure contains a set of stems, each consisting of two stem blocks. Suppose there are N_S and N_{S^*} stem blocks in the secondary structures of R and R^* , respectively. Let S denote a stem block string and $S = s_1s_2 \cdots s_i \cdots s_{N_S}$, where s_i is a stem block, $1 \leq i \leq N_S$. Let S^* denote another stem block string and $S^* = s_1^*s_2^* \cdots s_j^* \cdots s_{N_{S^*}}^*$, where s_j^* is a stem block, $1 \leq j \leq N_{S^*}$. Let $p(s_i)$ denote the *stem partner* of stem block s_i . That is, $\langle s_i, p(s_i) \rangle$ and $\langle p(s_i), s_i \rangle$ are stems. We define a *segment* to be a group of contiguous nucleotides which appear within a block or across several blocks. The symbol $[x_s, x_e]$ represents the segment starting at nucleotide x_s and ending at nucleotide x_e . Similarly, the symbols with subscripts s and e represent the starting and ending positions, respectively.

We define a function $\rho(s_i, s_j)$ to decide whether two stem blocks s_i and s_j form a stem or not. If two stem blocks s_i and s_j form a stem, then $\rho(s_i, s_j) = 1$; otherwise, $\rho(s_i, s_j) = 0$. In addition, we denote the complement of $\rho(s_i, s_j)$ as $\overline{\rho(s_i, s_j)}$. For convenience, we regard $\rho(s_i, s_j)$ and $\overline{\rho(s_i, s_j)}$ as Boolean functions in our formulas. Let $\text{stemalign}(\langle s_i, s_j \rangle, \langle s_k^*, s_l^* \rangle)$ denote the alignment score function between stem $\langle s_i, s_j \rangle$ of R and stem $\langle s_k^*, s_l^* \rangle$ of R^* , where $1 \leq i < j \leq N_S$ and $1 \leq k < l \leq N_{S^*}$. Let $\text{segmentalign}(t, z)$ denote the segment alignment score function, where t is a segment of R and z is a segment of R^* . The operator $\oplus$ represents to find the next stem block and $\ominus$ represents to find the previous stem block.

Finally, let $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$ define the alignment score function in our algorithm, where x and y are the stem blocks of R , u and v are the stem blocks of R^* , α and β are segments of R , and γ and δ are segments of R^* . Therefore, the initial indices of the alignment score function $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$ of our algorithm are expressed in Figure 2.

Hence, the initialization of our algorithm is summarized as follows.

1. $x = s_1$ and $y = s_{N_S}$ in R .
2. $u = s_1^*$ and $v = s_{N_{S^*}}^*$ in R^* .

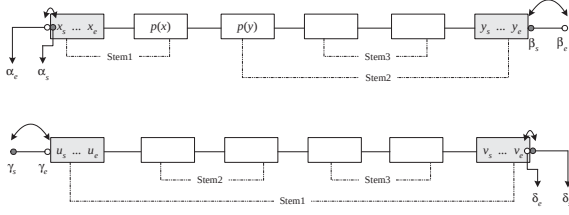


Figure 2: The illustration of initial indices of our algorithm.

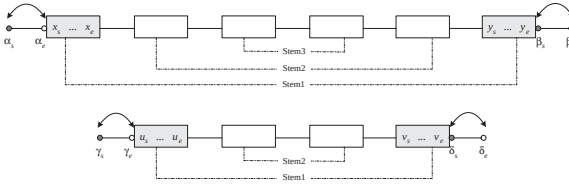


Figure 3: Illustration of Case 1.

3. In R , if α is not null (empty), then $\alpha = b_1$ according to B , $\alpha_s = (b_1)_s$ and $\alpha_e = (b_1)_e$, i.e., $[\alpha_s, \alpha_e] = [(b_1)_s, (b_1)_e]$. Otherwise, α is null, $\alpha_s = x_s$ and $\alpha_e = x_s - 1$, i.e., $[\alpha_s, \alpha_e] = [x_s, x_s - 1]$. It is similar for γ in R^* . If γ is not null, then $[\gamma_s, \gamma_e] = [(b_1^*)_s, (b_1^*)_e]$. Otherwise, γ is null, $[\gamma_s, \gamma_e] = [u_s, u_s - 1]$.
4. In R , if β is not null, then $\beta = b_{N_B}$ according to B , $\beta_s = (b_{N_B})_s$ and $\beta_e = (b_{N_B})_e$, i.e., $[\beta_s, \beta_e] = [(b_{N_B})_s, (b_{N_B})_e]$. Otherwise, β is null, $\beta_s = y_e + 1$ and $\beta_e = y_e$, i.e., $[\beta_s, \beta_e] = [y_e + 1, y_e]$. It is similar for δ in R^* . If δ is not null, then $[\delta_s, \delta_e] = [(b_{N_B^*}^*)_s, (b_{N_B^*}^*)_e]$. Otherwise, δ is null, $[\delta_s, \delta_e] = [v_e + 1, v_e]$.

In our algorithm, the input data are two RNA secondary structures without any pseudoknot. In other words, their secondary structures look like $\cdot(((((\cdot))))\cdot)$, where $"()"$ represents a stem and $"."$ represents a loop block. We divide our alignment algorithm into four cases according to the situations of stem blocks as follows.

Case 1: $\rho(x, y) = 1$ and $\rho(u, v) = 1$. Figure 3 illustrates Case 1. We can further divide the case into three subcases.

Case 1.1: Stem $\langle x, y \rangle$ aligns with stem $\langle u, v \rangle$. We must find a solution for

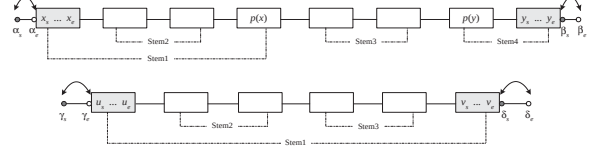


Figure 4: Illustration of Case 2.

stem blocks $x \oplus 1, y \ominus 1, u \oplus 1$ and $v \ominus 1$. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x \oplus 1, y \ominus 1, u \oplus 1, v \ominus 1, [\alpha_s + 1, (x \oplus 1)_s - 1], [(y \ominus 1)_e + 1, y_s - 1], [u_e + 1, (u \oplus 1)_s - 1], [(v \ominus 1)_e + 1, v_s - 1]) + \text{stemalign}(\langle x, y \rangle, \langle u, v \rangle) + \text{segmentalign}(\alpha, \gamma) + \text{segmentalign}(\beta, \delta)$.

Case 1.2: $\langle x, y \rangle$ is not aligned with $\langle u, v \rangle$, and $\langle x, y \rangle$ of R becomes a part of a segment. We must find a solution for stem blocks $x \oplus 1, y \ominus 1, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x \oplus 1, y \ominus 1, u, v, [\alpha_s, (x \oplus 1)_s - 1], [(y \ominus 1)_e + 1, \beta_e], \gamma, \delta)$.

Case 1.3: $\langle x, y \rangle$ is not aligned with $\langle u, v \rangle$, and $\langle u, v \rangle$ of R^* becomes a part of a segment. We must find a solution for stem blocks $x, y, u \oplus 1$ and $v \ominus 1$. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, u \oplus 1, v \ominus 1, \alpha, \beta, [\gamma_s, (u \oplus 1)_s - 1], [(v \ominus 1)_e + 1, \delta_e])$.

Case 2: $\rho(x, y) = 0$ and $\rho(u, v) = 1$. Figure 4 illustrates Case 2. This case can be further divided into three subcases as follows.

Case 2.1: Find the partner $p(y)$ of stem block y . We must find a solution for stem blocks $x, p(y) \oplus 1, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, p(y) \oplus 1, u, v, \alpha, [(p(y) \oplus 1)_e + 1, \beta_e], \gamma, \delta)$.

Case 2.2: Find $p(x)$ of x . We must find a solution for stem blocks $p(x) \oplus 1, y, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(p(x) \oplus 1, y, u, v, [\alpha_s, (p(x) \oplus 1)_s - 1], \beta, \gamma, \delta)$.

Case 2.3: Stem $\langle u, v \rangle$ of R^* is a part of a segment. We must find a solution for stem blocks $x, y, u \oplus 1$ and $v \ominus 1$. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, u \oplus 1, v \ominus 1, \alpha, \beta, [\gamma_s, (u \oplus 1)_s - 1], [(v \ominus 1)_e + 1, \delta_e])$.

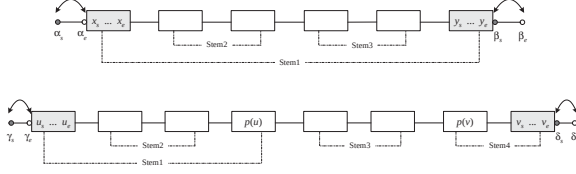


Figure 5: Illustration of Case 3.

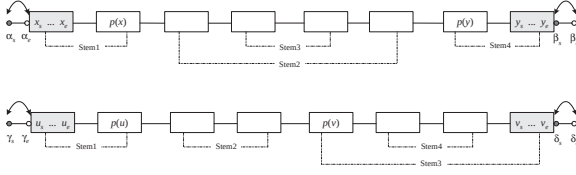


Figure 6: Illustration of Case 4.

Case 3: $\rho(x, y) = 1$ and $\rho(u, v) = 0$. Figure 5 illustrates Case 3. We can further divide the case into three subcases.

Case 3.1: Find $p(v)$ of v . We must find a solution for stem blocks x, y, u and $p(v) \oplus 1$. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, u, p(v) \oplus 1, \alpha, \beta, \gamma, [(p(v) \oplus 1)_e + 1, \delta_e])$.

Case 3.2: Find $p(u)$ of u . We must find a solution for stem blocks $x, y, p(u) \oplus 1$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, p(u) \oplus 1, v, \alpha, \beta, [\gamma_s, (p(u) \oplus 1)_s - 1], \delta)$.

Case 3.3: Stem $< x, y >$ of R is a part of a segment. We must find a solution for stem blocks $x \oplus 1, y \oplus 1, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x \oplus 1, y \oplus 1, u, v, [\alpha_s, (x \oplus 1)_s - 1], [(y \oplus 1)_e + 1, \beta_e], \gamma, \delta)$.

Case 4: $\rho(x, y) = 0$ and $\rho(u, v) = 0$. Figure 6 illustrates Case 4. We can further divide the case into six subcases as follows.

Case 4.1: Find $p(y)$ of y . We must find a solution for stem blocks $x, p(y) \oplus 1, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, p(y) \oplus 1, u, v, \alpha, [(p(y) \oplus 1)_e + 1, \beta_e], \gamma, \delta)$.

Case 4.2: Find $p(v)$ of v . We must find a solution for stem blocks x, y, u and $p(v) \oplus 1$. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, u, p(v) \oplus 1, \alpha, \beta, \gamma, [(p(v) \oplus 1)_e + 1, \delta_e])$.

Case 4.3: Find $p(y)$ of y and $p(v)$ of v . Thus, we divide this subcase into two mapping groups. The first group is for stem blocks $x, p(y) \oplus 1, u$ and $p(v) \oplus 1$. The second group is for stem blocks $p(y), y, p(v)$ and v . We must find the solution for both groups. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, p(y) \oplus 1, u, p(v) \oplus 1, \alpha, [(p(y) \oplus 1)_e + 1, p(y)_s - 1], \gamma, [(p(v) \oplus 1)_e + 1, p(v)_s - 1]) + f(p(y), y, p(v), v, [p(y)_s, p(y)_s - 1], \beta, [p(v)_s, p(v)_s - 1], \delta)$.

Case 4.4: Find $p(x)$ of x . We must find a solution for stem blocks $p(x) \oplus 1, y, u$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(p(x) \oplus 1, y, u, v, [\alpha_s, (p(x) \oplus 1)_s - 1], \beta, \gamma, \delta)$.

Case 4.5: Find $p(u)$ of u . We must find a solution for stem blocks $x, y, p(u) \oplus 1$ and v . Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, y, p(u) \oplus 1, v, \alpha, \beta, [\gamma_s, (p(u) \oplus 1)_s - 1], \delta)$.

Case 4.6: Find $p(x)$ of x and $p(u)$ of u . Thus, we divide the subcase into two mapping groups. The first group is for stem blocks $x, p(x), u$ and $p(u)$. The second group is for stem blocks $p(x) \oplus 1, y, p(u) \oplus 1$ and v . We must find a solution for the two groups. Thus, $f(x, y, u, v, \alpha, \beta, \gamma, \delta) = f(x, p(x), u, p(u), \alpha, [p(x)_e + 1, p(x)_e], \gamma, [p(u)_e + 1, p(u)_e]) + f(p(x) \oplus 1, y, p(u) \oplus 1, v, [p(x)_e + 1, (p(x) \oplus 1)_s - 1], \beta, [p(u)_e + 1, (p(u) \oplus 1)_s - 1], \delta)$.

4 Improvement on Space

The function $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$ has an 8-dimensional argument, and the maximum range of each argument in one dimension is the maximum number of blocks in the block strings (i.e., $\max(N_B, N_{B^*})$). One can easily observe that both time complexity and space complexity of our algorithm are $O((\max(N_B, N_{B^*}))^8)$.

For practical implementation in computers, the formula is infeasible when the number of RNA blocks is large. Accordingly, we have to try to reduce the required memory space. That is, we

will try to decrease the dimensions (i.e., the number of indices) of the dynamic programming formula $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$. It will lead to the design of a new dynamic programming formula $\hat{f}(i, j, k, l)$ whose function is the same as that of $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$.

Before introducing formula $\hat{f}(i, j, k, l)$, we need modify the symbols, terms and functions defined in formula $f(x, y, u, v, \alpha, \beta, \gamma, \delta)$, and define some more. Let $I(x, S, B)$ denote the block in the block string B which is corresponding to the stem block x of stem block string S , where $1 \leq x \leq N_S$ and $1 \leq I(x, S, B) \leq N_B$. In other words, if $s_x = b_i$, then $I(x, S, B) = i$. Let $I(x, B, S)$ denote the stem block of S corresponding to block x of B , but $I(x, B, S) = \text{null}$ if we can not find the corresponding stem block. Let $L(x, i, j)$ denote the maximal block between blocks i and x , where $I(L(x, i, j), B, S) \neq \text{null}$ and $i \leq p(L(x, i, j)) \leq j$. In other words, $L(x, i, j) = \max_{i \leq y \leq x} \{y | y \text{ is a block of } B, I(y, B, S) \neq \text{null and } i \leq p(y) \leq j\}$. Similarly, $R(x, i, j) = \min_{x \leq y \leq j} \{y | y \text{ is a block of } B, I(y, B, S) \neq \text{null and } i \leq p(y) \leq j\}$. In addition, we define $LN(x, i, j) = \max_{i \leq y \leq x} \{y | y \text{ is a block of } B, I(y, B, S) \neq \text{null and } i \leq p(y) \leq j\}$. Similarly, $RN(x, i, j) = \min_{x \leq y \leq j} \{y | y \text{ is a block of } B, I(y, B, S) \neq \text{null and } i \leq p(y) \leq j\}$. $LN(x, i, j)$ and $RN(x, i, j)$ are used for finding the next stems. The above functions for S^* and B^* are defined similarly.

In the new dynamic programming algorithm, the initial values of the variables in $\hat{f}(i, j, k, l)$ are set that i and j are the block indices in B , and $i = 1$ and $j = N_B$ in R ; k and l are the block indices in B^* , and $k = 1$ and $l = N_{B^*}$ in R^* . Then formula $\hat{f}(i, j, k, l)$ is given in Figure 7.

$\text{segmentalign}(i, j)$ represents the segment alignment score function, where i is a segment of R and j is a segment of R^* . The calculation of $\text{segmentalign}(i, j)$ is to find the minimal length between segments i and j . Indeed, one can easily modify the function $\text{segmentalign}(i, j)$ to align segments i and j . We do not consider whether the aligned nucleotides are the same or not, because the RNA secondary structure alignment appreciates the alignment of the structural shape. We shall consider only the segment length here. Hence, $\text{segmentalign}(i, j)$ is given as follows.

$$\text{segmentalign}(i, j) = \min(n_R(i), n_{R^*}(j)) \times \phi, \quad (8)$$

where ϕ is the score for aligning two nucleotides and it is a constant, and $n_R(i)$ and $n_{R^*}(j)$ are the lengths of segments i and j , respectively.

For example, consider two segments $i = \text{CGAAGUC}$ and $j = \text{AAUGAGCUG}$, where segments i and j have 7 and 9 nucleotides, respectively. If $\phi = 1$ in Equation 8, then $\text{segmentalign}(i, j) = \min(7, 9) \times 1 = 7$. When $\phi = 0$, our algorithm only aligns the stem part of RNA secondary structures. Here, we will set ϕ to 1 when discussing the performance of our algorithm.

In the stem alignment problem, we consider only base pairs here. That is, we only align the base pairs in both stems. Let x be one stem of the secondary structure of R and $n_{bp}(x)$ be the number of base pairs in x . Similarly, let y be one stem of the secondary structure of R^* and $n_{bp^*}(y)$ be the number of base pairs in y . Hence, $\text{stemalign}(x, y)$ is given as follows.

$$\text{stemalign}(x, y) = (\min(n_{bp}(x), n_{bp^*}(y)))^\lambda \times \tau, \quad (9)$$

where λ and τ are constants.

For example, suppose two stems $x = \text{AAG} \parallel \text{CUU}$ and $y = \text{AGGCC} \parallel \text{GGCCU}$, stem x has 3 base pairs and stem y has 5 base pairs. If $\lambda = 2$ and $\tau = 4$ in Equation 9, then $\text{stemalign}(x, y) = (\min(3, 5))^2 \times 4 = 36$.

We can see that there are four indices in the improved formula $\hat{f}(i, j, k, l)$, and the number of all cases are fixed. Therefore, it is trivial that the time complexity of our improved dynamic programming algorithm is $O(N^4)$, where N is the number of blocks contained in the given RNAs (i.e. $N = \max(N_B, N_{B^*})$).

5 Experimental Results

In this section, we will show our experimental results and analyze the performance of our algorithm. Our algorithm is implemented by C++ using Borland C++ Compiler 5.5 on PC with AMD DuronTM processor 800 MHz and 256 MB RAM. Our testing data are obtained from NCBI (<http://www.ncbi.nlm.nih.gov/>). There are 22 tRNAs in Homo sapiens mitochondrion. In our experiments, we use the method in [7] to predict the secondary structures of tRNAs.

$$\widehat{f}_1(i, j, k, l) = \max_{\substack{RN(i, i, j) \leq x \leq LN(j, i, j) \\ RN(k, k, l) \leq y \leq LN(l, k, l)}} \left\{ \widehat{f}(i, x-1, k, y-1) + \widehat{f}(x, p(x), y, p(y)) + \widehat{f}(p(x)+1, j, p(y)+1, l) \right\} \quad (1)$$

$$\begin{aligned} \widehat{f}_2(i, j, k, l) = & \text{segmentalign}([i_s, (R(i, i, j) - 1)_e], [k_s, (R(k, k, l) - 1)_e]) \\ & + \text{segmentalign}([(L(j, i, j) + 1)_s, j_e], [(L(l, k, l) + 1)_s, l_e]) \\ & + \text{stemalign}(< R(i, i, j), L(j, i, j) >, < R(k, k, l), L(l, k, l) >) \\ & + \widehat{f}(R(i, i, j) + 1, L(j, i, j) - 1, R(k, k, l) + 1, L(l, k, l) - 1) \end{aligned} \quad (2)$$

$$\widehat{f}_3(i, j, k, l) = \max \left\{ \begin{aligned} & \widehat{f}(i, R(i, i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(R(i, i, j), LN(p(L(j, i, j))), i, j, R(k, k, l), L(l, k, l)) \\ & + \widehat{f}(LN(p(L(j, i, j))), i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, RN(p(R(i, i, j))), i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(RN(p(R(i, i, j))), i, j, L(j, i, j), R(k, k, l), L(l, k, l)) \\ & + \widehat{f}(L(j, i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, R(i, i, j) - 1, k, RN(R(k, k, l), k, l) - 1) + \widehat{f}(R(i, i, j), L(j, i, j), RN(R(k, k, l), k, l), LN(L(l, k, l), k, l)) \\ & + \widehat{f}(L(j, i, j) + 1, j, LN(L(l, k, l), k, l) + 1, l) \end{aligned} \right. \quad (3)$$

$$\widehat{f}_4(i, j, k, l) = \max \left\{ \begin{aligned} & \widehat{f}(i, R(i, i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(R(i, i, j), L(j, i, j), R(k, k, l), LN(p(L(l, k, l))), k, l) \\ & + \widehat{f}(L(j, i, j) + 1, j, LN(p(L(l, k, l))), k, l) + 1, l) \\ & \widehat{f}(i, R(i, i, j) - 1, k, RN(p(R(k, k, l))), k, l) - 1) + \widehat{f}(R(i, i, j), L(j, i, j), RN(p(R(k, k, l))), k, l, L(l, k, l)) \\ & + \widehat{f}(L(j, i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, RN(R(i, i, j), i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(RN(R(i, i, j), i, j), LN(L(j, i, j), i, j), R(k, k, l), L(l, k, l)) \\ & + \widehat{f}(LN(L(j, i, j), i, j) + 1, j, L(l, k, l) + 1, l) \end{aligned} \right. \quad (4)$$

$$\widehat{f}_5(i, j, k, l) = \max \left\{ \begin{aligned} & \widehat{f}(i, R(i, i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(R(i, i, j), LN(p(L(j, i, j))), i, j, R(k, k, l), L(l, k, l)) \\ & + \widehat{f}(LN(p(L(j, i, j))), i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, R(i, i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(R(i, i, j), L(j, i, j), R(k, k, l), LN(p(L(l, k, l))), k, l) \\ & + \widehat{f}(L(j, i, j) + 1, j, LN(p(L(l, k, l))), k, l) + 1, l) \\ & \widehat{f}(i, p(L(j, i, j)) - 1, k, p(L(l, k, l)) - 1) + \widehat{f}(p(L(j, i, j)), j, p(L(l, k, l)), l) \\ & \widehat{f}(i, RN(p(R(i, i, j))), i, j) - 1, k, R(k, k, l) - 1) + \widehat{f}(RN(p(R(i, i, j))), i, j, L(j, i, j), R(k, k, l), L(l, k, l)) \\ & + \widehat{f}(L(j, i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, R(i, i, j) - 1, k, RN(p(R(k, k, l))), k, l) - 1) + \widehat{f}(R(i, i, j), L(j, i, j), RN(p(R(k, k, l))), k, l, L(l, k, l)) \\ & + \widehat{f}(L(j, i, j) + 1, j, L(l, k, l) + 1, l) \\ & \widehat{f}(i, p(R(i, i, j)), k, p(R(k, k, l))) + \widehat{f}(p(R(i, i, j)) + 1, j, p(R(k, k, l)) + 1, l) \end{aligned} \right. \quad (5)$$

$$\widehat{f}_6(i, j, k, l) = \begin{cases} \text{segmentalign}([i_s, j_e], [k_s, l_e]) & \text{if } i = j \text{ and } k \leq l \quad \text{or } i \leq j \text{ and } k = l \\ 0 & \text{if } i > j \text{ or } k > l \end{cases} \quad (6)$$

$$\widehat{f}(i, j, k, l) = \begin{cases} \max \left\{ \begin{aligned} & \widehat{f}_1(i, j, k, l) \\ & (\rho(i, j) \times \rho(k, l) + \rho(R(i, i, j), L(j, i, j)) \\ & \quad \times \rho(R(k, k, l), L(l, k, l))) \times \widehat{f}_2(i, j, k, l) \\ & (\overline{\rho(i, j)} \times \rho(k, l) + \overline{\rho(R(i, i, j), L(j, i, j))}) \\ & \quad \times \rho(R(k, k, l), L(l, k, l))) \times \widehat{f}_3(i, j, k, l) \\ & (\rho(i, j) \times \overline{\rho(k, l)} + \rho(R(i, i, j), L(j, i, j)) \\ & \quad \times \rho(R(k, k, l), L(l, k, l))) \times \widehat{f}_4(i, j, k, l) \\ & (\overline{\rho(i, j)} \times \overline{\rho(k, l)} + \overline{\rho(R(i, i, j), L(j, i, j))}) \\ & \quad \times \rho(R(k, k, l), L(l, k, l))) \times \widehat{f}_5(i, j, k, l) \end{aligned} \right. & \text{if } i < j \text{ and } k < l \\ \widehat{f}_6(i, j, k, l) & \text{otherwise} \end{cases} \quad (7)$$

Figure 7: The new dynamic programming formula $\widehat{f}(i, j, k, l)$.

Some parameters are used to measure the LCS (longest common subsequence) identity of the two sequences. $Min(L)$ denotes the alignment length of the minimal LCS alignment. The LCS identity is defined as follows.

$$LCS\ identity = \frac{LCS\ length}{Min(L)} \times 100\%$$

Then, some parameters are used to measure the *structure similarity* of the secondary structures of R and R^* . $Score(R, R^*)$ denotes the score of the secondary structure alignment of R and R^* by using our algorithm. $Max(R, R^*)$ denotes the maximal score between the two scores which R and R^* are aligned with themselves, respectively. That is, $Max(R, R^*) = Max\{Score(R, R), Score(R^*, R^*)\}$. Then the structure similarity is defined as follows.

$$structure\ similarity = \frac{Score(R, R^*)}{Max(R, R^*)} \times 100\%$$

We shall change values of some parameters in Equation 9. λ advances (makes) that the alignment score of base pairs between two stems is nonlinear enhancement and τ denotes the stem score contributed by aligning two base pairs in two stems. We test structure similarities of each pair of 22 tRNAs by using our algorithm, where $\lambda = 1, 1.5$ and 2, and $\tau = 2, 3, 4, 5$ and 6. Consequently, we have come to the conclusion that λ can be set to 1.5 and τ can be set to 4.

Though many RNA sequences share very high identity, their secondary structures or functions may not be very similar. Meantime, the secondary structures and functions of many RNA sequences are very similar, but they may not share high sequence identity. Hence, we will discuss the performance of our algorithm according to the similarities of sequences and structures.

Table 1 shows the result based on $\lambda = 1.5$ and $\tau = 4$, where the input data is TRNG and TRNA for Case A, and TRNG and TRNV for Case B. They are tRNAs of Homo sapiens mitochondrion. In Table 1, we can see that LCS identity is 47.62 % and 54.43 % of Case A and Case B, respectively. That is, the LCS identity of Case B is higher than Case A.

The RNA secondary structures of Case A are predicted by using a dynamic programming method [7]. Figure 8 and Figure 9 show the secondary structures of Case A and Case B, respectively. In Figure 8 and Figure 9, we can see that the two secondary structures of Case A are more similar than those of Case B.

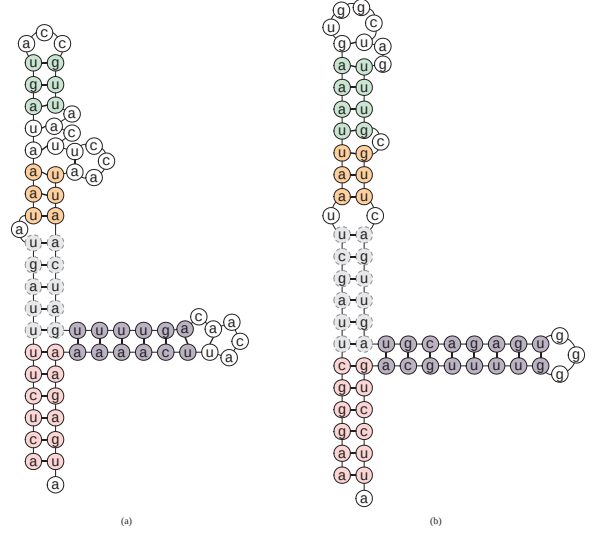


Figure 8: The optimal alignment for Case A by using our algorithm and $\lambda = 1.5$ and $\tau = 4$. (a) The secondary structure of TRNG. (b) The secondary structure of TRNA. Note: The secondary structures are predicted by using a dynamic programming method [7].

In Table 1, we can see that the structure similarity of Case A is 76.21 % and structure similarity of Case B is 50.44 %. In other words, the structure similarity of Case A is higher than Case B by applying our algorithm, but the LCS identity of Case B is higher than Case A. Hence, our algorithm is a good approach for RNA secondary structure alignment.

6 Conclusion

In this paper, we propose an RNA secondary structure alignment algorithm to evaluate the secondary structure similarity of two RNAs based on blocks for dealing with stems and loops. For the practical implementation in computers, the eight indices of our original formula make the program infeasible when the number of RNA blocks is large. We improve the formula into a four-index one. We can see that there are four indices in the improved formula, both time complexity and space complexity of our improved dynamic programming algorithm are reduced to $O(N^4)$, where N is the number of blocks contained in the given RNAs.

For the experiment of the biological data, we

Table 1: The experimental results of Case A and Case B based on $\lambda = 1.5$ and $\tau = 4$. Case A: TRNG and TRNA. Case B: TRNG and TRNV.

λ	τ	Case	LCS length	$Min(L)$	LCS identity	$Score(R, R^*)$	$Max(R, R^*)$	Structure similarity
1.5	4	A	40	84	47.62 %	205	269	76.21 %
		B	43	79	54.43 %	114	226	50.44 %

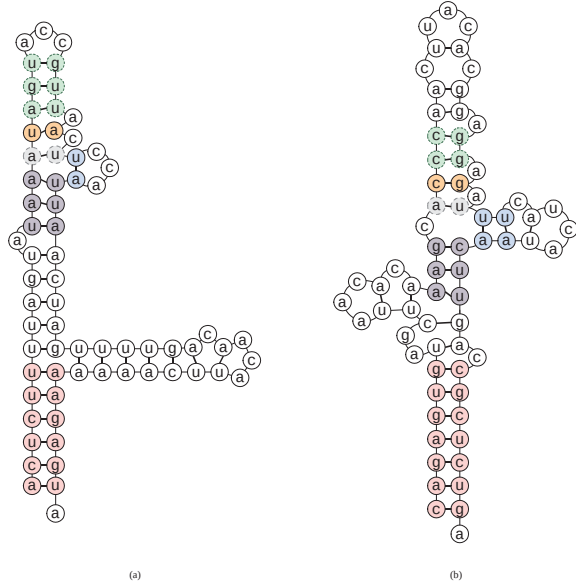


Figure 9: The optimal alignment for Case B by using our algorithm and $\lambda = 1.5$ and $\tau = 4$. (a) The secondary structure of TRNG. (b) The secondary structure of TRNV. Note: The secondary structures are predicted by using a dynamic programming method [7].

take three of tRNAs, that are TRNG, TRNA, and TRNV, as the input data of our algorithm. In the result, we see that the structure similarity of TRNG and TRNA is higher, but their LCS identity is lower than TRNG and TRNV. Hence, our block concept and our algorithm should be a good approach and method for RNA secondary structure alignment. In addition, it is also helpful to predict the functions of biomolecules and to classify them. In the future, we will test more real data to verify the practicality of our algorithm. The pseudoknots are not considered in our algorithm, it is a worth way to develop new RNA structure alignment method to deal with pseudoknots.

References

- [1] V. Bafna, S. Muthukrishnan, and R. Ravi, "Computing similarity between RNA strings," *Proceedings of the 6th Annual Symposium on Combinatorial Pattern Matching*, pp. 1–16, 1995.
- [2] G. D. Collins, S. Le, and K. Zhang, "A new method for computing similarity between RNA structures," *Information Sciences*, Vol. 139, pp. 59–77, 2001.
- [3] F. Corpet and B. Michot., "RNAAlign program: Alignment of RNA sequences using both primary and secondary structures," *Computer Applications in the Biosciences*, Vol. 10, No. 4, pp. 389–399, 1994.
- [4] T. Jiang, L. Wang, and K. Zhang, "Alignment of trees - an alternative to tree edit," *Theoretical Computer Science*, Vol. 143, No. 1, pp. 137–148, 1995.
- [5] S. Y. Le, R. Nussinov, and J. V. Maizel, "Tree graphs of RNA secondary structures and their comparisons," *Computers and Biomedical Research*, Vol. 22, pp. 461–473, 1989.
- [6] S. Y. Le, J. Owens, R. Nussinov, J. H. Chen, B. A. Shapiro, and J. V. Maizel, "RNA secondary structures: Comparison and determination of frequently recurring substructures by consensus," *Computer Applications in the Biosciences*, Vol. 5, No. 3, pp. 205–210, 1989.
- [7] R. C. T. Lee, "Computational biology," <http://www.csie.ncnu.edu.tw/>, Department of Computer Science and Information Engineering, National Chi-Nan University, 2001.
- [8] H. P. Lenhof, K. Reinert, and M. Vingron, "A polyhedral approach to RNA sequence structure alignment," *Journal of Computational Biology*, Vol. 5, No. 3, pp. 517–530, 1998.

- [9] G. H. Lin, B. Ma, and K. Zhang, "Edit distance between two RNA structures," *Proceedings of the fifth Annual International Conference on Computational Biology*, pp. 211–220, 2001.
- [10] M. C. Lin, C. B. Yang, and K. S. Huang, "Prediction of RNA secondary structures by genetic algorithms," *Proceedings of the 6th World Multiconference on Systemics, Cybernetics and Informatics, SCI 2002*, Vol. 12, pp. 439–444, 2002.
- [11] C. L. Lu, Z. Y. Su, and C. Y. Tang, "A new measure of edit distance between labeled trees," *Proceedings of the 7th Annual International Computing and Combinatorics Conference (COCOON 2001)*, Vol. 2108, pp. 338–348, 2001.
- [12] B. Ma, L. Wang, and K. Zhang, "Computing similarity between RNA structures," *Theoretical Computer Science*, Vol. 276, pp. 111–132, 2002.
- [13] C. Notredame, E. O'Brien, and D. G. Higgins, "RAGA: RNA sequence alignment by genetic algorithm," *Nucleic Acids Research*, Vol. 25, No. 22, pp. 4570–4580, 1997.
- [14] B. A. Shapiro, "An algorithm for comparing multiple RNA secondary structures," *Computer Applications in the Biosciences*, Vol. 4, No. 3, pp. 387–393, 1988.
- [15] B. A. Shapiro and K. Zhang, "Comparing multiple RNA secondary structures using tree comparisons," *Computer Applications in the Biosciences*, Vol. 6, No. 4, pp. 309–318, 1990.
- [16] Z. Wang and K. Zhang, "Alignment between two RNA structures," *Proceedings of the 26th International Symposium on Mathematical Foundations of Computer Science*, pp. 690–702, 2001.
- [17] M. S. Waterman and T. F. Smith, "RNA secondary structure: A complete mathematical analysis," *Mathematical Bioscience*, Vol. 42, pp. 257–266, 1978.
- [18] K. Zhang, "Computing similarity between RNA secondary structures," *In Proceedings of IEEE International Joint Symposia on Intelligence and Systems*, pp. 126–132, 1998.
- [19] K. Zhang and D. Shasha, "Simple fast algorithms for the editing distance between trees and related problems," *SIAM Journal on Computing*, Vol. 18, No. 6, pp. 1245–1262, 1989.
- [20] M. Zuker and D. Sankoff, "RNA secondary structures and their prediction," *Mathematical Bioscience*, Vol. 46, pp. 591–621, 1984.

Constrained Sequence Alignment: A Dedicated Version and Its Applications

Yun-Sheng Chung* and Chuan Yi Tang

Department of Computer Science

National Tsing Hua University, Hsinchu, Taiwan

g914342@oz.nthu.edu.tw, cytang@cs.nthu.edu.tw

Abstract

In this paper, we study a problem that arises naturally in biological applications. Given two sequences, along with a sequence of patterns, we want to align the two sequences such that the specified patterns are aligned together. This is the constrained sequence alignment problem and is defined in [14]. The multiple sequence version is called CMSA. In this paper, we focus on the pair-wise version where one of the sequences is annotated with the correct occurrence positions of the patterns. This version solves many applications that one would use constrained alignment to solve, and the algorithm proposed here is more efficient than that for the original version. In addition, we propose an improved approximation algorithm for CMSA. Our algorithm improves that proposed in [3] from $O(Ck^2mn^2)$ time and $O(k^2mn^2)$ space to $O(Ck^2n^2)$ time and $O(kn)$ space, where k is the number of sequences and C is the maximum number of valid “constrained lists” (defined later).

1 Introduction

Sequence alignment is essential in computational biology. Classical methods consider the alignment of characters on the sequences. This problem has been extensively studied in the past (see, e.g., [10]). There are some properties that can be deduced directly from the characters, e.g. hydrophilicity and polarity of amino acids. Methods dealing with simple characters can naturally be used to treat this kind of properties. In [11], the amino acid sequences are transformed into these volumes so that FFT can be utilized to accelerate the alignment procedure. As biological knowledge, predictions and hypotheses grow rapidly, it turns out to be important and desirable

to incorporate richer biological information into alignment procedures, instead of merely comparing characters or corresponding quantities. This motivates the study of annotated sequence alignment (analysis) [4], since such information often can be represented as some kind of annotations on the sequences. In the past, the study of annotated sequence alignment focuses on the annotation of RNA structure information, e.g., [12, 9] (each predicted hydrogen bond is represented as an arc linking two characters). In addition to arc annotations, Evans [4] also studied color annotations on sequences and applied them in database searches. Taylor used a different approach [15] for annotating and comparing motifs.

Tang et al. [14] studied a problem called constrained multiple sequence alignment (CMSA). This problem can also be recognized as one kind of annotated sequence alignment. In [14], a constraint is a sequence of residues that is considered necessary for a protein family. They considered RNases for an example. RNases contain three conserved His12 (H), Lys41 (K) and His119 (H) active site residues as their major structural feature [14]. Thus if we are aligning RNase sequences, it is desirable that each of these conserved residues appears in the same columns in the resulting alignment. If standard alignment is used, those active site residues are not guaranteed to align together, which is not reasonable if we are given the knowledge. Furthermore, the order of “H, K, H” should be preserved. In the terminology in [14], “H, K, H” is the constraint to be satisfied in this example.

In a follow-up work by Chin et al. [3], a more efficient algorithm for the pair-wise constrained alignment is proposed. Their algorithm runs in $O(mn^2)$ time and space. For CMSA, they applied the center-star technique in Gusfield’s well known 2-approximation algorithm for (standard) multiple sequence alignment [6] and get a 2-approximation algorithm.

*To whom correspondence should be addressed.

Tsai et al. [16] generalized the definition of patterns, from single residues to substrings. Their algorithm is equally efficient as that in [3]. They applied their method to detect if SARS-TW1 virus forms in some structural elements involving in RNA replications.

In this paper, we discuss the case when the positions of occurrences of the given patterns on one of the sequences are known. For the other sequence, we do not know at the beginning whether these patterns exist, or where they are. This is often the case in applications but is not discussed in [14, 3, 16]. Biologists are generally able to annotate important regions on a protein sequence known to have some function. Given a sequence whose function is to be predicted, we can align it with the annotated sequence to see if it can have the same function as the annotated sequence. Important regions can have different degrees: necessary, relevant, irrelevant, etc. For example, it has been shown that in Dihydropyrimidinases, there is a sequence of six residues responsible for the binding of metal [1]. In general, if a reasonable multiple sequence alignment is available for a protein family, we can extract conserved regions from the alignment, and the degree of conservation (e.g. perfectly conserved, strongly conserved, etc.) can be seen as an indication of importance. This suggests that an effective tool should be able to consider both necessary and relevant regions. Previous results either focuses only on necessary regions [14, 3, 16, 15], or only on relevant regions [4]. The algorithm presented in this paper deals with necessary regions, with additional constraints on ranges between patterns. In fact, the incorporation of regions of less important regions into our algorithm is straightforward and we discuss this later. Our algorithm runs in $O(n^2)$ time and $O(n)$ space. The result applies to the general cost measure as in [14, 3, 16].

The patterns we consider in this paper admits gaps and degenerate forms in them, which are not considered in [14, 3, 16, 15]. The method in [4] does not guarantee that all necessary regions are aligned together.

The constrained sequence alignment problem is in some sense similar to the color annotated sequence comparison problem [4]. But the color annotation (layered-string version) on a string forms in a tree structure and by definition any two color blocks in the same level can not overlap. This property enables one to solve this problem recursively by considering each colored substring a "hyper-character". But in our problem, this prop-

erty no longer exists, since a pattern can be of length more than one and can occur in any position.

The remainder of this paper is organized as follows. In Sec. 2 we give some definitions. Section 3 gives the algorithm for the one-annotated version. Section 4 devotes to the improvement of the approximation algorithm in [3]. Finally, concluding remarks are given in Sec. 5.

2 Preliminaries

Let Σ be the alphabet where the characters of sequences are drawn. For a sequence X , let $X(i)$ be the i th character on X , and denote the string $X(i)X(i+1) \dots X(j)$ as $X[i..j]$. Denote the length of X as $|X|$.

An alignment of strings $S_1, \dots, S_k$ is a set of strings $(S'_1, \dots, S'_k)$, where S'_i is obtained by inserting some spaces "-" into S_i , and $|S'_i| = |S'_j|$ for all i, j . A pairwise alignment of two strings is a special case when $k = 2$. For convenience, define $\#spaces(c; S'_i)$ to be the number of space characters "-" in S'_i at or before position c . Also define $\#spaces(I; S'_i)$ be the number of -'s in the interval I in S'_i .

Let $\delta : (\Sigma \cup \{-\})^2 \rightarrow \mathbf{R}$ be the cost function. Let (S'_1, S'_2) be an alignment. The score of this alignment is given by $\sum_l \delta(S'_1(l), S'_2(l))$.

Now we define some terms used in this paper.

Definition 1 A pattern P is a language (set of strings) $P(1)P(2) \dots P(|P|)$, where $P(l) \subset \Sigma$ for $1 \leq l \leq |P|$. We say that P occurs at position i in string X if $X[i+l-1] \in P(l)$ for $1 \leq l \leq |P|$. If P occurs in X at positions $i_1 < \dots < i_c$, then i_x is denoted as $start_x(X; P)$. Let $end_x(X; P) = i_x + |P| - 1$. We also say that P occurs at interval $[start_x(X; P), end_x(X; P)]$.

As can be seen, the patterns are defined as languages instead of single residues. This definition is beneficial for dealing with degenerate forms, e.g., motifs that are represented in IUPAC code [5]. For a pattern P_l , define $cost(P_l)$ to be the cost of matching two occurrences of P_l . Later we refer to this kind of pattern as type-1 pattern.

We now give another type of pattern to be considered in this paper. This is useful to allow gaps in pattern occurrences.

Definition 2 A pattern P is a string $P(1)P(2) \dots P(|P|)$, where $P(l) \in \Sigma$ for

$1 \leq l \leq |P|$. We say that P occurs at interval $[i, j]$ in string X if $d_E(X[i..j], P) \leq t$, where t is a threshold value and d_E is the weighted edit distance between two strings. Suppose that P occurs at intervals $[i_1, j_1], [i_2, j_2], \dots, [i_c, j_c]$, where $i_1 \leq i_2 \leq \dots \leq i_c$. Then we denote i_x as $start_x(X; P)$ and j_x as $end_x(X; P)$.

We will refer to this kind of pattern as type-2 pattern, for distinction. The definition for patterns adopted in [16] uses Hamming distance and does not allow gaps in occurrences. It is a special case of the definition given here.

Definition 3 A motif constraint is a sequence (rather than merely a set) of patterns $P_1, \dots, P_m$. We use $\mathcal{P} = \langle P_1, \dots, P_m \rangle$ to denote a motif constraint. Also let $\mathcal{P}_l = \langle P_1, \dots, P_l \rangle$.

Definition 4 An alignment $S'_1, \dots, S'_k$ satisfies motif constraint $P_1, \dots, P_m$ (either type-1 or type-2) if there exists intervals $[c_1, c'_1], \dots, [c_m, c'_m]$ where $c_1 < \dots < c_m$ such that for all $1 \leq i \leq k$ and all $1 \leq l \leq m$, P_l occurs at interval $[c_l, c'_l]$ in S_i and all $[c_l, c'_l]$'s are disjoint (referred to as the non-overlapping condition). When $k = 2$ and if (S'_1, S'_2) is optimal, the interval $[c_l - \#spaces(c_l; S'_1), c_l - \#spaces(c_l; S'_2)]$ is said to be a “correct” interval of occurrence of P_l in S_i .

In this paper, the following problem is studied.

One-annotated constrained sequence alignment Given two strings S_1 and S_2 and a constraint $\mathcal{P} = \langle P_1, \dots, P_m \rangle$, along with m “correct” intervals $[c_1, c'_1], \dots, [c_l, c'_l]$ of occurrences (one for each pattern) in S_1 for each P_l such that the non-overlapping condition is satisfied. Find an alignment of S_1 and S_2 satisfying $\mathcal{P}$ such that the score is minimized.

The following is the original problem studied in [14, 3, 16] (in fact generalized here). In Sec. 4, a more efficient approximation algorithm over that in [3] is proposed.

Constrained multiple sequence alignment (CMSA) Given a set of sequences and a constraint $\mathcal{P}$, find an alignment satisfying $\mathcal{P}$ with the minimum score.

Since the multiple sequence alignment problem is NP-hard [17, 2], it is clear that CMSA is also NP-hard. Tang et al. [14] and Tsai et al. [16] solved this problem by a progressive method, and Chin et al. [3] adopted an approximation algorithm. The method proposed in Sec. 3 improves Chin et al.’s 2-approximation algorithm.

3 The algorithm

In this section, the algorithm for the one-annotated version is developed. Let $\mathcal{P} = \langle P_1, \dots, P_m \rangle$ be a motif constraint. Suppose that we are given S_1 and S_2 and want to find the optimal alignment satisfying $\mathcal{P}$. Assume that the correct occurrence positions in S_1 is given. First we find on S_2 the occurrence positions for the patterns in $\mathcal{P}$. This is discussed in Sec. 3.1 and Sec. 3.2. In Sec. 3.3, the main algorithm is presented.

3.1 Annotation of type-1 patterns

For each pattern P_l in the constraint, we use $|\Sigma|$ bits to represent $P_l(i)$. For example, if $\Sigma = \{A, T, C, G\}$ (the order is relevant) and $P_l(i) = \{A, C\}$, then $P_l(i)$ is represented as $(1, 0, 1, 0)$. These bits can be stored compactly in computer words to enhance practical performance. Hence all the patterns can be converted into this form in $O(L|\Sigma|)$ time and space, where $L = \sum_{i=1}^m |P_i|$. It is then easy to determine all positions of occurrences for all patterns by sliding window (and simple arithmetic and bit operations) in $O(Ln)$ time. The overall space needed is $O(L|\Sigma| + r)$, where r is the total number of occurrences of all patterns in S_2 . Let $j_1 \leq \dots \leq j_{c(S_2; P_l)}$ be the positions of occurrences of P_l in S_2 . Let $start_x(S_2; P_l) = j_x$. Also define $end_x(S_2; P_l) = j_x + |P_l| - 1$. We keep the list $occur(S_2; P_l) = \langle [start_1(S_2; P_l), end_1(S_2; P_l)], \dots, [start_{c(S_2; P_l)}(S_2; P_l), end_{c(S_2; P_l)}(S_2; P_l)] \rangle$, where $c(S_2; P_l)$ is the number of occurrences of P_l in S_2 .

Since each type-1 pattern is in fact a set of strings, one may want to apply suffix tree technique or Aho-Corasick algorithm (see, e.g., [7]) to find positions of occurrences. But it turns out to be uneconomical to enumerate all possible strings in P_l , which is needed for a direct application of these two methods. For an extreme example, if $P_l = \Sigma^{|P_l|}$, then the number of strings in P_l is $|\Sigma|^{|P_l|}$, which grows exponentially in $|P_l|$. On the other hand, it is a restricted regular expression, and the use of general regular expression matching algorithms [7] is not necessary.

3.2 Annotation of type-2 patterns

First of all, we give the following lemma that helps to ensure some property of the occurrence list. It is mentioned in [13] using the notion of grid graph. Define $f(x; X) = \min\{i : x - \#spaces(i; X) = x\}$ for a string X and index x .

Lemma 1 *Let (X', Y') be an alignment (not necessarily optimal) of X and Y , and let (X'', Y'') be an alignment of X and $Y[i..j]$. Then there exists a pair of indices (x, y) such that (X', Y') can be decomposed as $(X'[1..f(x; X')], X'[f(x; X') + 1..|X'|], Y'[1..f(y; Y')], Y'[f(y; Y') + 1..|Y'|]$, and (X'', Y'') can be decomposed as $(X''[1..f(x; X'')], X''[f(x; X'') + 1..|X''|], Y''[1..f(y; Y'')], Y''[f(y; Y'') + 1..|Y''|]$.*

For type-2 patterns, we use simple dynamic programming to find the occurring intervals. For each pattern P_l , $1 \leq l \leq m$, we compute $D(i, j)$, the smallest weighted edit distance d_E between $P_l[1..i]$ and all suffixes of $S_2[1..j]$. Let $S_2[j'..j]$ be such a suffix. Then define $src(i, j) = j'$. If $D(i, j) = d_E(P_l[1..i], S_2[j'..j]) = d_E(P_l[1..i], S_2[j''..j])$, then let $src(i, j) = \max(j', j'')$ (or other tie-breaking rule). After the computation of all entries of $D(i, j)$, $1 \leq i \leq |P_l|$, $1 \leq j \leq |S_2|$, we keep all intervals $[src(|P_l|, j), j]$ such that $d_E(P_l, S_2[src(|P_l|, j)..j]) = D(|P_l|, j) \leq t$ in list $occur(S_2; P_l)$, where t is the threshold value specified by the user, and $c(S_2; P_l)$ is the number of j such that $D(|P_l|, j) \leq t$. By Lemma 1 and the tie-breaking rule, if $end_x > end_y$, then $start_x \geq start_y$. Note that here we are finding occurrence intervals, so we concern only about edit distance (the score), not how P_l aligns with S_2 . This processing takes $O(Ln)$ time and $O(n + r)$ space, where $r = \sum_{l=1}^m c(S_2; P_l)$.

3.3 The main algorithm

Before the presentation of the algorithm, it should be noted that the problem can be solved by the $O(mn^2)$ time and space algorithm in [3] (with some modifications). However, it is natural to ask whether the additional information given in this version (i.e., the positions of occurrences in one of the strings) can be utilized to improve the time and space bounds. The answer turns out to be positive, and is explored in this section.

To find the optimal constrained alignment of S_1 and S_2 , we compute table T using the following recurrences:

1. $T[0, 0] = 0$. $T[0, j] = T[0, j - 1] + \delta(-, S_1[j])$ if $0 < j < start(S_1; P_l)$. $T[i, 0] = T[i - 1, 0] + \delta(S_2[i], -)$ if $i > 0$.
2. If $j \notin [start(S_1; P_l), end(S_1; P_l)]$ for all l , then
 $T[i, j] = \min\{T[i - 1, j] + \delta(S_2[i], -), T[i - 1, j - 1] + \delta(S_2[i], S_1[j]), T[i, j - 1] + \delta(-, S_1[j])\}$.

3. If $j = start(S_1; P_l)$ for some l ,

$$T[i, j] = \begin{cases} T[i - 1, j - 1] & \text{if } i = start_x(S_2; P_l) \\ & \text{for some } 1 \leq x \leq c(S_2; P_l); \\ \text{null} & \text{otherwise.} \end{cases}$$

4. If $j = end(S_1; P_l)$ for some l ,

$$T[i, j] = \begin{cases} \infty & \text{if } i = 0; \\ \min \begin{cases} T[start_x(S_2; P_l), start(S_1; P_l)] + cost(P_l) \\ T[i - 1, j] + \delta(S_2[i], -) \end{cases} & \text{if } i = end_x(S_2; P_l); \\ T[i - 1, j] + \delta(S_2[i], -) & \text{otherwise.} \end{cases}$$

5. If $j \in (start(S_1; P_l), end(S_1; P_l))$ for some l , then $T[i, j] = \text{null}$ (it can be left undefined).

Observe that T has a property that if $j \in [end(S_1; P_l), start(S_1; P_{l+1}))$ for some $l < m$, or if $j \geq end(S_1; P_l)$ for $l = m$, then $T[i, j]$ is the minimum score of aligning $S_2[1..i]$ and $S_1[1..j]$ satisfying the (sub-)constraint $\mathcal{P}_l$. It follows that $T[|S_2|, |S_1|]$ is the optimal score of constrained alignment of S_1 and S_2 satisfying the constraint $\mathcal{P}$.

In the above recurrences, it may seem that to determine to which interval j belongs, it takes time more than just a constant. It turns out to be unnecessary; if the computation proceeds either row-wise or column-wise, then by the sorted-property of the list $occur(S_2; P_l)$, each such determination takes only $O(1)$ time. Also note that the annotation process yields no ambiguity for the determination of $start_x(S_2; P_l)$ given $end_x(S_2; P_l)$.

With the above recurrences, we can now easily compute the optimal constrained alignment of S_1 and S_2 . It is clear that $O(n^2)$ time suffices. To reduce space requirements to $O(n)$, a little care is needed. First, it is easy to see that if we are only to compute the score of the optimal constrained alignment, instead of the alignment itself, then the above recurrences can be applied and takes only $O(n)$ space. Besides, the columns in T that satisfy the condition in 5 are negligible; this simplifies the situation one faces. If some care is taken, Hirschberg's well-known divide-and-conquer technique [8] can be modified to apply here (divide the columns). This reduces the required space to $O(n)$.

Recall that the occurrence-finding phase takes $O(Ln)$ time and $O(L|\Sigma| + r)$ space for type-1 pat-

terns, and $O(Ln)$ time and $O(n+r)$ space for type-2 patterns, where L is the total length of the patterns in the constraint and r is the total number of occurrences of all patterns in S_2 . Hence it takes $O(n^2 + Ln)$ time and $O(n + L|\Sigma|)$ space for type-1 constraints, and the same time and $O(n + r)$ space for type-2 constraints. Let's take a closer look at these bounds. First, $L = O(n)$ by the non-overlapping condition. Also, it is common to assume that $|\Sigma|$ is a constant. In addition, it is clear that r is bounded above by $O(mn)$. Hence the time and space needed are essentially $O(n^2)$ and $O(n+r)$, respectively. Note that if constraints are defined as in [3], $r = O(n)$ and the space complexity is $O(n)$.

To incorporate the analysis for important regions that are considered not necessary, a simple weighting scheme can be used, as in [4]. Specifically, for $x, y \in \Sigma \cup \{-\}$, $\delta(x, y)$ can be substituted by $\delta(x, y) \times w_1(x) \times w_2(y)$, where w_1 and w_2 are weights associated with the importance of characters in the first and second string, respectively.

4 A more efficient approximation algorithm for CMSA

Chin et al. [3] extended Gusfield's well-known center-star approximation algorithm for the multiple sequence alignment problem [6] to deal with CMSA, giving a $(2 - \frac{2}{k})$ approximation ratio. The approximation ratio is guaranteed if δ satisfies triangle inequality [6, 3]. It turns out that the algorithm developed in the previous section improves the performance of the 2-approximation algorithm in [3], although the definition of patterns is more general here. We now explore this.

Suppose we are given k strings $S_1, \dots, S_k$, along with a constraint $\mathcal{P}$. Note that the occurrence intervals are not given for this problem. For convenience, let $n = \max\{|S_1|, \dots, |S_k|\}$. The modified algorithm is outlined below.

1. **Occurrence-finding phase** Find all intervals of occurrences of all patterns P_l on all strings.
2. **Best center sequence selection phase** Looping over all k sequences, consider each the center sequence once. Suppose that S_i is now the center sequence. Then for each possible list of occurrence intervals $[start_{c_1}(S_i; P_1), end_{c_1}(S_i; P_1)], \dots, [start_{c_m}(S_i; P_m), end_{c_m}(S_i; P_m)]$ ($(c_1, \dots, c_m)$ is referred to as "constrained list" later) of patterns

$P_1, \dots, P_m$ on S_i , $1 \leq c_l \leq c(S_i; P_l)$, S_i is pair-wise aligned with all strings in $\{S_1, \dots, S_k\} \setminus S_i$ with respect to this constrained list. The sum of these $k-1$ pair-wise scores is kept for later comparison. Note that this is exactly the case when the "correct" occurrence intervals on S_i are given; the alignment can be done by the algorithm in the previous section.

We then determine for what choices of S_i and constrained list, the sum of pair-wise scores is minimized. Such S_i and $(c_1, \dots, c_m)$ is used in the following step. This S_i is referred to as the best center sequence and $(c_1, \dots, c_m)$ as the best constrained list.

3. **Merging phase** The $k-1$ pair-wise alignments of the best center sequence to all other sequences are merged, under the best constrained list, in the same way as in [3].

The first phase takes $O(kLn)$ time and $O(L|\Sigma| + R)$ space, where R is the total number of occurrences of all patterns in all strings. It is clear that R is bounded by $O(kmn)$. The second phase takes $O(Ck^2n^2)$ time and $O(n + r)$ additional space, since in this phase we concern only about the scores of alignments, and we need not store the alignment results. C is the maximum number of valid constrained lists in a single input string; the additional $O(r)$ space is due to the enumeration of valid constrained lists for all sequences, where $r = \max_{1 \leq i \leq k} \{\sum_{l=1}^m c(S_i; P_l)\}$. The third phase takes $O(kn^2)$ time and $O(kn)$ additional space. Hence the total time and space taken are $O(Ck^2n^2)$ and $O(kn + R)$, respectively. In particular, if the definition for patterns in [3] is adopted, our algorithm takes $O(Ck^2n^2)$ time and $O(kn)$ space, as opposed to $O(Cmk^2n^2)$ time and $O(mk^2n^2)$ space for the algorithm in [3].

5 Conclusions

In this paper, we consider a case of pair-wise constrained sequence alignment, where occurrences of patterns in one of the string are known. This is useful when determining whether a query sequence has some properties known or hypothesized by biologists, so that it could have some function. We propose an algorithm taking $O(n^2)$ time and $O(n+r)$ space, where r is the number of occurrences of the patterns in the constraint. The analysis admitted here is thus more general than

previous constrained sequence alignment results [14, 3, 16]. Also, the algorithm improves previous approximation algorithm for CMSA [3], whose effectiveness had been demonstrated in [3], both in generality and efficiency, allowing one to conduct more practical and larger scale analysis.

We are currently developing efficient algorithms that can incorporate more kinds of constraints into the alignment. The first we would consider is arc annotation, which is useful for representing RNA secondary structures. Arc annotations and motif constraints together make the prediction and analysis of TIM barrel structure more plausible, since such structure contains alpha helices and beta sheets. On the other hand, we have completed an algorithm that can incorporate the information about ranges between patterns. It is hoped that combining all these factors, the analysis will be more realistic and flexible for biologists.

References

- [1] Abendroth, J., Niefind, K., Schomburg, D.: X-ray Structure of a Dihydropyrimidinase from *Thermus* sp. at 1.3Å Resolution. *J. Mol. Biol.* **320** (2002) 143–156
- [2] Bonizzoni, P., Vedova, G. D.: The Complexity of Multiple Sequence Alignment with SP-score that is a metric. *Theoret. Comput. Sci.* **259** (2001) 63–79
- [3] Chin, F. Y. L., Ho, N. L., Lam, T. W., Wong, P. W. H., Chan, M. Y.: Efficient Constrained Sequence Alignment with Performance Guarantee. In: *Proc. Comput. Syst. Bioinfo. (CSB'03)*. IEEE (2003) 337–346
- [4] Evans, P. A.: Algorithms and Complexity for Annotated Sequence Analysis. Ph. D Thesis, Department of Computer Science, University of Victoria, Canada (1999)
- [5] Faisst, S., Meyer, S.: Compilation of Vertebrate-encoded Transcription Factors. *Nucleic Acids Research* **20** (1992) 3–26
- [6] Gusfield, D.: Efficient Methods for Multiple Sequence Alignment with Guaranteed Error Bounds. *Bul. Math. Biol.* **30** (1993) 141–154
- [7] Gusfield, D.: Algorithms on Strings, Trees, and Sequences. Cambridge University Press (1997)
- [8] Hirschberg, D. S.: A Linear Space Algorithm for Computing Maximal Common Subsequences. *Comm. ACM.* **18** (1975) 341–343
- [9] Jiang, T., Lin, G., Ma, B., Zhang, K.: A General Edit Distance between RNA Structures. *J. Comput. Biol.* **9** (2002) 371–388
- [10] Jiang, T., Xu, Y., Zhang, M. Q. (ed.): Current Topics in Computational Molecular Biology. The MIT Press (2002)
- [11] Katoh, K., Misawa, K., Kuma, K.-I., Miyata, T.: MAFFT: A Novel Method for Rapid Multiple Sequence Alignment Based on Fast Fourier Transform. *Nucleic Acids Res.* **30** (2002) 3059–3066
- [12] Lin, G.-H., Chen, Z.-Z., Jiang, T., Wen, J.: The Longest Common Subsequence Problem for Sequences with Nested Arc Annotations. *J. Comput. Syst. Sci.* **65** (2002) 465–480
- [13] Schmidt, J. P.: All Highest Scoring Paths in Weighted Grid Graphs and Their Application to Finding All Approximate Repeats in Strings. *SIAM J. Comput.* **27** (1998) 972–992
- [14] Tang, C. Y., Lu, C. L., Chang, M. D. T., Sun, Y. J., Tsai, Y. T., Chang, J. M., Chiou, Y. H., Wu, C. M., Chang, H. T., Chou, W. I., Chiang, S. C.: Constrained Sequence Alignment Tool Development and its Application to RNase Family Alignment. *J. Bioinfo. Comput. Biol.* **1** (2003) 267–287
- [15] Taylor, W. R.: Motif-biased Protein Sequence Alignment. *J. Comput. Biol.* **1** (1994) 297–310
- [16] Tsai, Y. T., Lu, C. L., Yu, C. T., Huang, Y. P.: MuSiC: A Tool for Multiple Sequence Alignment with Constraints. *Bioinformatics* (to appear)
- [17] Wang, L., Jiang, T.: On the Complexity of Multiple Sequence Alignment. *J. Comput. Biol.* **1** (1994) 337–348

Topological Properties, Optimal Routing, and Embedding on the K -valent Graph

Sun-Yuan Hsieh* and Tien-Te Hsiao

Department of Computer Science and Information Engineering,
National Cheng Kung University, 701 Tainan, Taiwan

Abstract

This paper introduces a new family of Cayley graphs, named the k -valent graph, for building interconnection networks. We show that the k -valent graph possesses many valuable topological properties, such as regular with the node-degree k , logarithmic diameter subject to the number of nodes for some fixed $k \geq 6$, and maximally fault tolerance. We also show that the k -valent graph contains trivalent graphs defined in [P. Vadapalli and P. K. Srimani, "Trivalent Cayley graphs for interconnection networks," Information Processing Letters, vol. 54, pp. 329–335, 1995.] as its subclass of graphs. An algorithm is proposed to determine a shortest path between arbitrary two nodes. Besides, embedding of cycles or cliques on the k -valent graph is also discussed.

Keywords: Cayley graphs, diameter, embedding, interconnection networks, shortest path routing.

1 Introduction

Design of interconnection networks is an important issue of the parallel processing or distributed system. Many interconnection networks have been proposed in the literature [1, 2, 4, 5, 7, 6, 8, 9, 10, 11, 13, 15, 16, 19, 21]. The choice of network topologies significantly determines the performance of these kinds of systems. Several important measurements, such as connectivity, node-degree, diameter, symmetry, fault tolerance and so on, are discussed for building interconnection networks [24]. The interconnection network is often modelled as an undirected graph, in which the nodes correspond to the processors and the edges correspond to the communication channels. Thus

the terms, graph and network, node and processor, edge and channel, are used interchangeably throughout this paper.

A class of Cayley graphs was shown to be very suitable for designing interconnection networks [2]. These graphs are based on permutation groups. There are quite a few well-known interconnection networks, including hypercubes [4], star graphs [1, 2], pancake graphs [2], and so on, which belong to Cayley graphs. All of these graphs are regular and symmetric. However, the degree of the nodes increases with the size of the graph. It can make the use of these graphs prohibitive for networks with large number of nodes. Therefore, the fixed node-degree networks are important from VLSI implementation point of view [16]; there are applications where the computing nodes in the interconnection network can have only a fixed number of I/O ports [5]. Therefore, designing of an interconnection networks to keep fixed node-degree when the number of nodes increases may be of interests by itself.

P. Vadapalli and P. K. Srimani [19] proposed the trivalent Cayley graphs with the fixed node-degree 3. This class of graphs was shown to be regular, to have logarithmic diameter, and to be maximally fault-tolerant. Several topological properties and shortest path routing algorithm were discussed in [14, 18, 19, 20, 22]. However, both the node-degree and the node-connectivity of trivalent graphs are limited by 3, and thus the probability of failure get increasing when the size of the network is extended. In this paper, we generalize the work of [19] to propose a new family of Cayley graphs, named the k -valent graphs $G_{k,n}$. It possesses a nice property that the degree of each node is fixed by k without regarding the size of the network. We show that the trivalent graph is a subclass of the k -valent graph when $k = 3$. We also show that the k -valent graph have logarithmic diameter when $k \geq 6$, maximally fault tolerance, and can

*Corresponding author: hsiehsy@mail.ncku.edu.tw

embed disjoint cycles or cliques. Besides, a shortest path routing algorithm which is more complicated than that of the trivalent Cayley graph is also presented.

The rest of this paper is organized as follows: In the next section, the k -valent Cayley graph is introduced. In Section 3, a shortest path routing algorithm is proposed. Based on the algorithm, the diameter is established in this section. Embedding and node-connectivity are discussed in Section 4 and Section 5, respectively. Finally, some concluding remarks are given in Section 6.

2 The k -valent Graph

The following is a formal definition of the k -valent Cayley graph.

Definition 1. A k -valent graph $G_{k,n}$ is an undirected graph with $N = n(k-1)^n$ vertices for any integers $n \geq 2$ and $k \geq 3$. Each node v of $G_{k,n}$ has the form $s_0 s_1 \dots s_{m-1} \tilde{s}_m s_{m+1} \dots s_{n-1}$ corresponding to a string of n symbols selected from $\{0, 1, \dots, k-2\}$ such that exactly one symbol $\tilde{s}_m$ is in *marked* form and the others are in *unmarked* form. We sometimes use v^m for representing a node v with the marked symbol on position m , and thus the notations v^m and v are used interchangeably throughout this paper. Let $s_i^* = s_i$ or $\tilde{s}_i$. Each edge is of the type $(v, \delta(v))$, where $\delta \in \{f, f^{-1}, g_1, g_2, \dots, g_{k-2}\}$ is a *generator* defined as follows:

- $f(u^m) = v^{(m-1) \bmod n}$, where $u^m = s_0^* s_1^* \dots s_{n-1}^*$, $v^{(m-1) \bmod n} = s_1^* s_2^* \dots s_{n-1}^* \alpha^*$ and $\alpha = (s_0 + 1) \bmod (k-1)$;
- $f^{-1}(u^m) = v^{(m+1) \bmod n}$, where $u^m = s_0^* s_1^* \dots s_{n-1}^*$, $v^{(m+1) \bmod n} = \beta^* s_0^* s_1^* \dots s_{n-2}^*$ and $\beta = (s_{n-1} - 1) \bmod (k-1)$;
- $g_i(u^m) = v^m$, where $u^m = s_0^* s_1^* \dots s_{n-1}^*$, $v^m = s_0^* s_1^* \dots s_{n-2}^* \gamma^*$ and $\gamma = (s_{n-1} + i) \bmod (k-1)$ for $1 \leq i \leq k-2$.

For example, $\{\tilde{0}0, \tilde{0}1, \tilde{1}0, \tilde{1}1, \tilde{0}\tilde{0}, \tilde{0}\tilde{1}, \tilde{1}\tilde{0}, \tilde{1}\tilde{1}\}$ is the node set of $G_{3,2}$. Since f (respectively, f^{-1}) is the inverse of f^{-1} (respectively f), and the inverse of g_i equals g_{k-i-1} for $1 \leq i \leq k-2$, the edges of $G_{k,n}$ are bidirectional. Moreover, according to Definition 1, it is clear that $G_{k,n}$ belongs to Cayley graphs.

Proposition 1. $G_{k,n}$, $n \geq 2$ and $k \geq 3$, is a regular graph of node-degree k , and has $\frac{kn(k-1)^n}{2}$ edges.

Proof. Straightforward. $\square$

P. Vadapalli and P. K. Srimani [19] defined the *trivalent Cayley graphs* TC_n as follows.

Definition 2. [19] Each node of a *trivalent Cayley graph* TC_n , $n \geq 2$, corresponds to a circular permutation in lexicographic order of n symbols, $t_1, t_2, \dots, t_n$, complemented or uncomplemented. The node set of TC_n is defined as $V(TC_n) = \{t_j^* t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots t_{j-1}^* | t_i^* = t_i \text{ or } \bar{t}_i \text{ for all } i, j \in \{1, \dots, n\}\}$. Let $\bar{\bar{j}} = j$. Each edge is of the type $(v, \delta(v))$, where $\delta \in \{f_{TC}, f_{TC}^{-1}, g_{TC}\}$, defined in the following way:

$$\begin{aligned} f_{TC}(t_j^* t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots t_{j-1}^*) &= t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots t_{j-1}^* \bar{t}_j^* \\ f_{TC}^{-1}(t_j^* t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots t_{j-1}^*) &= \bar{t}_{j-1}^* t_j^* \dots t_n^* t_1^* t_2^* \dots t_{j-2}^* \\ g_{TC}(t_j^* t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots t_{j-1}^*) &= t_{j+1}^* \dots t_n^* t_1^* t_2^* \dots \bar{t}_{j-1}^* \end{aligned}$$

We can use English alphabets for representing symbols of TC_n . For example, when $n = 4$, $t_1 = a, t_2 = b, t_3 = c$, and $t_4 = d$ with lexicographic order $a < b < c < d$.

Two graphs G_1 and G_2 are *isomorphic* if there is a one-to-one function π from $V(G_1)$ onto $V(G_2)$ such that $(u, v) \in E(G_1)$ if and only if $(\phi(u), \phi(v)) \in E(G_2)$ [23]. We next show that k -valent graphs contain trivalent Cayley graphs as its subclass.

Theorem 2. $G_{3,n}$ and TC_n are isomorphic.

Proof. Let $t_1, t_2, \dots, t_n$ be a lexicographic order of n symbols for TC_n . For each node $s_0 s_1 \dots s_{m-1} \tilde{s}_m s_{m+1} \dots s_{n-1}$ of $G_{3,n}$, where $s_i \in \{0, 1\}$, we define a function π mapping $V(G_{3,n})$ to $V(TC_n)$ as follows:

$$\pi(s_0 s_1 \dots s_{m-1} \tilde{s}_m s_{m+1} \dots s_{n-1}) = t'_{n-m+1} t'_{n-m+2} \dots t'_n t'_1 \dots t'_{n-m}, \quad (1)$$

where $t'_i = t_i$ if $s_{(m+i-1) \bmod n} = 0$ and $t'_i = \bar{t}_i$ if $s_{(m+i-1) \bmod n} = 1$, for $i \in \{1, \dots, n\}$. The function π is obviously one-to-one and onto.

Suppose that $u^m = s_0^* s_1^* \dots s_{n-1}^*$ and $v^{m'}$ are two adjacent nodes in $G_{3,n}$. Let $\pi(u^m) = t'_{n-m+1} t'_{n-m+2} \dots t'_n t'_1 \dots t'_{n-m}$ according to Equation 1. There are three cases.

Case 1. $f(u^m) = v^{m'}$. Then, $v^{m'} = v^{(m-1) \bmod n} = s_1^* s_2^* \dots s_{n-1}^* \alpha^*$, where $\alpha = (s_0 + 1) \bmod 2$. Then, $\pi(v^{(m-1) \bmod n}) = t'_{n-m+2} \dots t'_n t'_1 \dots t'_{n-m} \bar{t}'_{n-m+1}$. Thus, $\pi(u^m)$ and $\pi(v^{m'})$ are adjacent in TC_n because $f_{TC}(\pi(u^m)) = \pi(v^{m'})$.

Case 2. $f^{-1}(u^m) = v^{m'}$. Then, $v^{m'} = v^{(m+1) \bmod n} = \beta^* s_0^* s_1^* \dots s_{n-2}^*$, where $\beta = (s_{n-1} - 1) \bmod 2$. So $\pi(v^{(m+1) \bmod n}) = t'_{n-m} t'_{n-m+1} \dots t'_n t'_1 \dots t'_{n-m-1}$. Thus, $\pi(u^m)$ and $\pi(v^{m'})$ are adjacent in TC_n because $f_{TC}^{-1}(\pi(u^m)) = \pi(v^{m'})$.

Case 3. $g_1(u^m) = v^{m'}$. Then, $v^{m'} = v^m = s_0^* s_1^* s_2^* \dots \gamma^*$, where $\gamma = (s_{n-1} + 1) \bmod 2$. So $\pi(v^m) = t'_{n-m+1} t'_{n-m+2} \dots t'_n t'_1 \dots t'_{n-m-1} t'_{n-m}$. Thus, $\pi(u^m)$ and $\pi(v^{m'})$ are adjacent in TC_n because $g_{TC}(\pi(u^m)) = \pi(v^{m'})$.

On the other hand, if $\pi(u^m)$ and $\pi(v^{m'})$ are adjacent in TC_n , then $f_{TC}(\pi(u^m)) = \pi(v^{m'})$ or $f_{TC}^{-1}(\pi(u^m)) = \pi(v^{m'})$ or $g_{TC}(\pi(u^m)) = \pi(v^{m'})$. With the proofs similar to that of cases 1–3, we can show the following results to ensure that u^m and $v^{m'}$ are adjacent in $G_{3,n}$: (1) If $f_{TC}(\pi(u^m)) = \pi(v^{m'})$, then $f(u^m) = v^{m'}$; (2) If $f_{TC}^{-1}(\pi(u^m)) = \pi(v^{m'})$, then $f^{-1}(u^m) = v^{m'}$; (3) If $g_{TC}(\pi(u^m)) = \pi(v^{m'})$, then $g_1(u^m) = v^{m'}$. Therefore, since $(u^m, v^{m'}) \in E(G_{3,n})$ if and only if $(\pi(u^m), \pi(v^{m'})) \in E(TC_n)$, $G_{3,n}$ and TC_n are isomorphic. We complete the proof. $\square$

3 Shortest path routing and diameter

A graph is said to be *simple* if it contains no loops and multiple edges. A *path* is a simple graph whose vertices can be ordered so that two vertices are adjacent if and only if they are consecutive in the list. Define the distance of u and v to be the length of a shortest path between u and v . Since $G_{k,n}$ is a Cayley graph, it is node-symmetric [2]. Thus the distance between two arbitrary nodes is equal to that between some node (source node) and the identity node $\tilde{0}0\dots 0$. Our shortest path routing algorithm aims at constructing a shortest path from a given source node to the identity node. We begin with the following definitions.

For $v = s_0^* s_1^* \dots s_{n-1}^*$, let $v[i] = s_i^*$, $0 \leq i \leq n-1$, and let $v[i, j] = s_i^* s_{i+1}^* \dots s_j^*$, $0 \leq i < j \leq n-1$. If we view $v = s_0^* s_1^* \dots s_{n-1}^*$ as a string, then $v[i, j]$ is a *substring*. A *0-substring* is a substring whose symbols are all zero.

Definition 3. Consider an arbitrary node $v = s_0 s_1 \dots s_{m-1} \tilde{s}_m s_{m+1} \dots s_{n-1}$ in $G_{k,n}$. The marked symbol $\tilde{s}_m$ divides the string into two parts: the *left part* $s_0 s_1 \dots s_{m-1}$ and the *right part* $\tilde{s}_m s_{m+1} \dots s_{n-1}$. We also define the following notations for v :

- $NSL_v[x] = |\{s_i : s_i \equiv x \pmod{k-1} \text{ for } 0 \leq i \leq m-1\}|$.
- $NSR_v[x] = |\{s_i : s_i \equiv x \pmod{k-1} \text{ for } m \leq i \leq n-1\}|$.
- Let LZL_v be the length of a longest 0-substring in the left part, i.e. $LZL_v = \max\{l \geq 0 : s_j = s_{j+1} = \dots = s_{j+l-1} = 0, 0 \leq j \leq m-1\}$. Also let j_{L_v} indicate the position of the left-end of the leftmost longest 0-substring in the left part. In particular, define $j_{L_v} = 0$ if $LZL_v = 0$.
- Let LZR_v be the length of a longest 0-substring in the right part, i.e. $LZR_v = \max\{l \geq 0 : s_j = s_{j+1} = \dots = s_{j+l-1} = 0, m \leq j \leq n-1\}$. Also let j_{R_v} indicate the position of the left-end of the leftmost longest 0-substring in the right part. In particular, define $j_{R_v} = m$ if $LZR_v = 0$.

Throughout this paper, the subscript v can be omitted from the notations $NSL_v[x]$, $NSR_v[x]$, LZL_v , LZR_v , j_{L_v} , and j_{R_v} if no ambiguity arises.

For example, consider a node 1002100 of $G_{4,7}$. Then, $NSL[0] = NSR[0] = 2$; $NSL[1] = NSR[1] = 1$; $NSL[2] = 1$; $LZL = LZR = 2$; $j_L = 1$ and $j_R = 5$.

We next define the distance function that is useful for our routing algorithm.

Definition 4. Consider an arbitrary node v in $G_{k,n}$. Define:

1. $DIST_1(v) = \begin{cases} 2n + m - 2LZL - NSL[0] - NSR[1] - 2 & \text{if } (m - j_L - LZL) > 0 \\ 2n + m - 2LZL - NSL[0] - NSR[1] & \text{otherwise.} \end{cases}$
2. $DIST_2(v) = \begin{cases} 3n - m - 2LZR - NSR[0] - NSL[-1] - 2 & \text{if } (n - j_R - LZR) > 0 \\ 3n - m - 2LZR - NSR[0] - NSL[-1] & \text{otherwise.} \end{cases}$
3. $DIST_3(v) = 2n + m - NSL[-2] - NSR[-1]$.
4. $DIST_4(v) = 3n - m - NSR[2] - NSL[1]$.

The *distance function* of v is defined as $DIST(v) = \min_{1 \leq i \leq 4} \{DIST_i(v)\}$.

The four terms $DIST_i(v)$'s, $1 \leq i \leq 4$, defined in Definition 4 represent lengths of four possible paths of $G_{k,n}$ from v to the identity node. Those

paths of lengths $DIST_i(v)$, $1 \leq i \leq 4$, are constructed using algorithms DIST_1–DIST_4, respectively. As it will be shown later, the distance function $DIST(v)$ computes the length of a shortest path from v to the identity node. For ease of understanding our shortest path routing algorithm, we first present Algorithm DIST_1 to generate a path from v to the identity node with the length $DIST_1(v)$.

Algorithm 1 DIST_1(v, n, m, LZZ, j_L)

```

1: for  $i = 0$  to  $(j_L - 1)$  do
2:    $v \leftarrow f(v)$ 
3: end for
4: for  $i = 0$  to  $(j_L - 1) + (n - m)$  do
5:   if  $v[n - 1] \neq 1$  then
6:      $v \leftarrow g_{k-v[n-1]}(v)$ 
7:   end if
8:    $v \leftarrow f^{-1}(v)$ 
9: end for
10: if  $(m - j_L - LZZ) > 0$  then
11:   for  $i = 1$  to  $(m - j_L - LZZ - 1)$  do
12:      $v \leftarrow f^{-1}(v)$ 
13:   end for
14:    $v \leftarrow g_{k-v[n-1]-1}(v)$ 
15:   for  $i = 1$  to  $(m - j_L - LZZ - 1)$  do
16:      $v \leftarrow f(v)$ 
17:     if  $v[n - 1] \neq 0$  then
18:        $v \leftarrow g_{k-v[n-1]-1}(v)$ 
19:     end if
20:   end for
21: end if

```

Since the concepts of algorithms DIST_2–DIST_4 are similar to that of Algorithm DIST_1, we presented them in Appendix. Below is our shortest path routing algorithm.

Algorithm 2 OPTMAL_ROUTE

```

1: compute the value  $l \in \{1, 2, 3, 4\}$  such that
    $DIST_l(v)$  is the minimum of  $DIST_i(v)$ 's for
   all  $1 \leq i \leq 4$ .
2: execute Algorithm DIST_ $l$ 

```

The following lemma can help in proving the correctness of our routing algorithm.

Lemma 3. *For an arbitrary node v in $G_{k,n}$, $DIST(v) - DIST(v') = \pm 1$ or 0, where $v' = \delta(v)$ and $\delta \in \{f, f^{-1}, g_1, g_2, \dots, g_{k-2}\}$.*

Proof. Assume that $v = s_0 s_1 \dots s_{m-1} \tilde{s}_m s_{m+1} \dots s_{n-1}$ and $\delta(v) = v' = t_0 t_1 \dots t_{m'-1} \tilde{t}_{m'} t_{m'+1} \dots t_{n-1}$. Let NSL' , NSR' , LZZ' , and LZR'

be the corresponding parameters (defined in Definition 3) of v' . By the definition of the distance function, we can utilize the fact that $\min_{1 \leq i \leq 4} \{DIST_i(v)\} - DIST_p(v') \leq DIST(v) - DIST(v') \leq DIST_q(v) - \min_{1 \leq i \leq 4} \{DIST_i(v')\}$, for some $1 \leq p, q \leq 4$, to simplify the computation.

Case 1. $\delta \in \{g_1, g_2, \dots, g_{k-2}\}$. In this case, we have that $m' = m$, $NSL'[x] = NSL[x]$, and $LZZ' = LZZ$. According to Definition 4, the following results can be obtained.

(1) $DIST_1(v) - DIST_1(v') = -NSR[1] + NSR'[1] = \pm 1$ or 0.

(2) Let $q = -2LZR - NSR[0] + 2LZR' + NSR'[0]$. According to Definition 4, we have the following results:

i. If $v[n-1] \neq 0$ and $v'[n-1] \neq 0$, then $q = 0$ and $DIST_2(v) - DIST_2(v') = q = 0$;

ii. If $v[n-1] \neq 0$, $v'[n-1] = 0$, and $LZR = LZR'$, then $q = 1$ and $DIST_2(v) - DIST_2(v')$ may equal $q = 1$;

iii. If $v[n-1] \neq 0$, $v'[n-1] = 0$, and $LZR \neq LZR'$, then $q = 3$ and $DIST_2(v) - DIST_2(v') = q - 2 = 1$;

iv. If $v[n-1] = 0$, $v'[n-1] \neq 0$, and $LZR = LZR'$, then $q = -1$ and $DIST_2(v) - DIST_2(v') = q = -1$;

v. If $v[n-1] = 0$, $v'[n-1] \neq 0$, and $LZR \neq LZR'$, then $q = -3$ and $DIST_2(v) - DIST_2(v') = q + 2 = -1$.

(3) $DIST_3(v) - DIST_3(v') = -NSR[-1] + NSR'[-1] = \pm 1$ or 0.

(4) $DIST_4(v) - DIST_4(v') = -NSR[2] + NSR'[2] = \pm 1$ or 0.

Combining (1)–(4), we have that $DIST(v) - DIST(v') = \pm 1$ or 0.

Case 2. $\delta = f$.

Case 2.1. $m \neq 0$. In this case, we have that $m' = m - 1$.

(1) By definition 4, $DIST_1(v) - DIST_1(v') = 1 - 2LZZ - NSL[0] - NSR[1] + 2LZZ' + NSL'[0] + NSR'[1]$. Then, $DIST_1(v) - DIST_1(v') = \begin{cases} 1 - 2LZZ + 2LZZ' = \pm 1 & \text{if } v[0] = 0; \\ 1 & \text{otherwise.} \end{cases}$

(2) Let $q = -1 - 2LZR - NSR[0] - NSL[-1] + 2LZR' + NSR'[0] + NSL'[-1]$. By definition 4, $DIST_2(v) - DIST_2(v')$ may equal q or $q - 2$ or $q + 2$ in this case. Then, $DIST_2(v) - DIST_2(v') = \begin{cases} -1 - 2LZR + 2LZR' = \pm 1 & \text{if } v[0] = k - 2; \\ -1 & \text{otherwise.} \end{cases}$

(3) By Definition 4, $DIST_3(v) - DIST_3(v') = 1 - NSL[-2] - NSR[-1] + NSL'[-2] + NSR'[-1]$. By discussing the situations that $v[0] = k-3$ or $v[0] \neq k-3$, we conclude that $DIST_3(v) - DIST_3(v') = 1$.

(4) By Definition 4, $DIST_4(v) - DIST_4(v') = -1 - NSR[2] - NSL[1] + NSR'[2] + NSL'[1]$. By discussing the situations that $v[0] = 1$ or $v[0] \neq 1$, we have that $DIST_4(v) - DIST_4(v') = -1$.

Combining (1)–(4), we have that $DIST(v) - DIST(v') = \pm 1$ or 0.

Case 2.2. $m = 0$. In this case, we have that $m' = n - 1$, $NSL[x] = 0$, and $LZL = 0$. Thus, $DIST_1(v) = 2n - NSR[1]$.

$$DIST_2(v) = \begin{cases} 3n - 2LZR - NSR[0] - 2 & \text{if } (n - j_R - LZR) > 0; \\ 3n - 2LZR - NSR[0] & \text{otherwise.} \end{cases}$$

$$DIST_3(v) = 2n - NSR[-1].$$

$$DIST_4(v) = 3n - NSR[2] \geq 2n.$$

Note that we do not need to consider $DIST_4(v)$ in computing $DIST(v)$. On the other hand, we have that

$$DIST_1(v') = \begin{cases} 3n - 1 - 2LZL' - NSL'[0] - NSR'[1] - 2 & \text{if } (m' - j_{L_{v'}} - LZL') > 0; \\ 3n - 1 - 2LZL' - NSL'[0] - NSL'[1] & \text{otherwise.} \end{cases}$$

$$DIST_2(v') = \begin{cases} 2n + 1 - 2LZR' - NSR'[0] - NSL'[-1] - 2 & \text{if } (n - j_{R_{v'}} - LZR') > 0; \\ 2n + 1 - 2LZR' - NSR'[0] - NSL'[-1] & \text{otherwise.} \end{cases}$$

$DIST_3(v') = 3n - 1 - NSL'[-2] - NSR'[-1] \geq 2n - 1 \geq DIST_2(v')$ (this term can be ignored in computing $DIST(v')$);

and $DIST_4(v') = 2n + 1 - NSR'[2] - NSL'[1]$.

Then, the following results hold:

(1) Since $DIST_1(v) - DIST_4(v') = -1 - NSR[1] + NSR'[2] + NSL'[1]$, we can show that $DIST_1(v) - DIST_4(v') = -1$ without regarding whether $v[0] = 1$ or not.

(2) Since $DIST_2(v) - DIST_1(v') = 1 - 2LZR - NSR[0] + NSR'[1] + 2LZL' + NSL'[0]$, we have that $DIST_2(v) - DIST_1(v') =$

$$\begin{cases} \pm 1 & \text{if } v[0] = 0; \\ 1 & \text{otherwise.} \end{cases}$$

(3) Clearly, $DIST_3(v) - DIST_2(v') =$

$$\begin{cases} -1 - NSR[-1] + 2LZR' + NSR'[0] & \text{if } v[0] = k-2; \\ 1 - NSR[-1] + 2LZR' + NSR'[0] & \text{otherwise.} \end{cases}$$

It is not difficult to verify that $DIST_3(v) - DIST_2(v') = 1$ without regarding whether $v[0] = k-2$ or not.

From (1)–(3), $DIST(v) - DIST(v') = \pm 1$ or 0.

Case 3. $\delta = f^{-1}$. Then, $v' = f^{-1}(v) \Leftrightarrow f(v') = v$. This is symmetric to Case 2. $\square$

Lemma 4. For the identity node I in $G_{k,n}$, $DIST(I) = 0$ and $DIST(\delta(I)) = 1$ for any $\delta \in \{f, f^{-1}, g_1, g_2, \dots, g_{k-2}\}$.

Proof. Straightforward. $\square$

Theorem 5. Given an arbitrary node v in $G_{k,n}$, Algorithm OPTIMAL_ROUTE correctly generates an optimal (shortest) path of length $DIST(v)$ from v to the identity node.

Proof. Clearly, Algorithm OPTIMAL_ROUTE constructs a path of length $DIST(v)$ for a given node v . By Lemma 3, since an application of any one generator can decrease the value of $DIST(v)$ at best by one, the algorithm does construct a shortest path from v to the identity node. $\square$

Corollary 6. For an arbitrary node v in $G_{k,n}$, $DIST(v)$ equals the distance between v and the identity node.

Proof. It follows directly from Theorem 5. $\square$

We can utilize our shortest path routing algorithm to compute the diameter $diam(G_{k,n})$ for $k \geq 6$.

Theorem 7. For an arbitrary node v in $G_{k,n}$, $DIST(v) \leq \max\{2n, \frac{5n}{2} - 2\}$. Moreover, $diam(G_{k,n}) = \lfloor \frac{5n}{2} \rfloor - 2$ when $k \geq 6$ and $n \geq 4$; and $diam(G_{k,n}) = 2n$ when $k \geq 6$ and $n = 2, 3$.

Proof. By Definition 4, we have that $DIST(v) \leq \min\{2n + m - 2, 3n - m - 2, 2n + m, 3n - m\} = \min\{2n + m - 2, 3n - m - 2\} \leq \frac{5n}{2} - 2$ for $m > 0$, and $DIST(v) \leq \min\{2n, 3n - 2\} \leq 2n$ for $m = 0$. Hence the upper bound of the diameter is $\max\{\frac{5n}{2} - 2, 2n\}$. Note that it is $\frac{5n}{2} - 2$ for $n \geq 4$.

We next show the lower bound. For $k \geq 6$ and $n \geq 4$, consider the node v in $G_{k,n}$ such that $v[i] \notin \{0, 1, k-2, k-3\}$ and $m = \lfloor n/2 \rfloor$. Thus $NSL[0] = NSL[1] = NSL[-1] = NSL[-2] = NSR[0] = NSR[1] = NSR[-1] = NSR[-2] = 0$ and $LZL = LZR = 0$. By Definition 4, $DIST(v) = \min\{2n + \lfloor \frac{n}{2} \rfloor - 2, 3n - \lfloor \frac{n}{2} \rfloor - 2\} = \lfloor \frac{5n}{2} \rfloor - 2$. Therefore, $\lfloor \frac{5n}{2} \rfloor - 2 \leq diam(G_{k,n}) \leq \frac{5n}{2} - 2$ and $diam(G_{k,n}) = \lfloor \frac{5n}{2} \rfloor - 2$. The case of $k \geq 6$ and $n = 2, 3$ can be shown similarly. $\square$

4 Embedding

In this section, we show that the k -valent graph can embed two types of structures, cycles and cliques.

A *cycle* is a graph with an equal number of nodes and edges whose nodes can be placed around a circle so that two vertices are adjacent if and only if they appear consecutively along the circle [23]. Two cycles are *node-disjoint* if they have no common node. An edge defined by the generator f (or f^{-1}) is an $\mathcal{F}$ -edge. Any cycle in $G_{k,n}$ consisting of only $\mathcal{F}$ -edges is an $\mathcal{F}$ -cycle. For example, the cycle $(\tilde{0}0, 0\tilde{1}, \tilde{1}1, 1\tilde{0})$ in $G_{3,2}$ is an $\mathcal{F}$ -cycle. For non-negative integer i , we recursively define $f^i(v) = \begin{cases} v & \text{if } i = 0, \\ f(f^{i-1}(v)) & \text{if } i > 0. \end{cases}$ The notation $f^{-i}(v)$ can be defined similarly.

Theorem 8. $G_{k,n}$ can embed $(k-1)^{n-1}$ node-disjoint $\mathcal{F}$ -cycles of length $n(k-1)$.

Proof. Given an arbitrary node v in $G_{k,n}$, it is not difficult to verify that $f^{n(k-1)}(v) = v$ and $f^i(v) \neq f^j(v)$, where $1 \leq i, j \leq n(k-1)$ and $i \neq j$. Thus, from an arbitrary node v , a cycle of length $n(k-1)$ in $G_{k,n}$ can be generated by a functional iteration $f^{n(k-1)}(v)$. Since $G_{k,n}$ has $n(k-1)^n$ nodes, there are totally $(k-1)^{n-1}$ $\mathcal{F}$ -cycles of each length $n(k-1)$ can be generated. These cycles are node-disjoint by the fact that $f(u) = f(v)$ if and only if $u = v$. $\square$

Each $\mathcal{F}$ -cycle has a unique node v such that $v[i] = s$ and $m = 0$ for two given integers $0 \leq i \leq n-1$ and $0 \leq s \leq k-2$. Therefore, for each $\mathcal{F}$ -cycle C , we can define C -leader (leader for short if no ambiguity arises) to be the unique node v of C satisfying that $v[0] = 0$ and $m = 0$. Note that each leader uniquely determines a specific $\mathcal{F}$ -cycle. Two $\mathcal{F}$ -cycles C_1 and C_2 are said to be *adjacent* if there exists a node $v_1 \in C_1$ and a node $v_2 \in C_2$ such that $v_1 = g_i(v_2)$ for some $1 \leq i \leq k-2$.

Theorem 9. Each $\mathcal{F}$ -cycle of $G_{k,n}$ is adjacent to $n(k-2)$ (respectively, $k-2$) different $\mathcal{F}$ -cycles for $n > 2$ (respectively, $n = 2$).

Proof. We first consider $n > 2$. For an arbitrary $\mathcal{F}$ -cycle C with the C -leader $v = \tilde{0}s_1s_2 \dots s_{n-1}$,

$$f^{-i} \circ (g_j \circ f^i(v)) = \tilde{0}s_1s_2 \dots s' \dots s_{n-1}, \quad (2)$$

where $\circ$ means function composition and $s' = (s_{i-1} + j) \bmod (k-1)$ for all $i \neq 1$ and $1 \leq j \leq k-2$. Thus $(n-1)(k-2)$ leaders of $(n-1)(k-2)$ distinct $\mathcal{F}$ -cycles can be arrived from v . This implies

that the given $\mathcal{F}$ -cycle is adjacent to $(n-1)(k-2)$ distinct $\mathcal{F}$ -cycles. Moreover, the other $(k-2)$ leaders can be arrived from v by

$$f^{-1-jn} \circ (g_j \circ f(v)) = \tilde{0}s'_1s'_2 \dots s'_{n-1}, \quad (3)$$

where $s'_i = (s_i - j) \bmod (k-1)$. Therefore, each $\mathcal{F}$ -cycle is adjacent to $n(k-2)$ different $\mathcal{F}$ -cycles.

The case of $n = 2$ can be shown using either Equation 2 or Equation 3. $\square$

The proof of the above theorem leads to the following result.

Corollary 10. An $\mathcal{F}$ -cycle with its leader $v = \tilde{0}x_1x_2 \dots x_{n-2}$ is adjacent to following $\mathcal{F}$ -cycles with leaders $\tilde{0}y_1y_2 \dots y_{n-1}$ such that one of the following conditions hold.

1. $x_i \neq y_i$ for some $1 \leq i \leq n-1$, and $x_j = y_j$ for all $1 \leq j \leq n-1$ and $j \neq i$;
2. $x_1 - y_1 \equiv x_2 - y_2 \equiv \dots \equiv x_{n-1} - y_{n-1} \pmod{k-1}$.

We next show that $G_{k,n}$ can embed cliques. An edge of $G_{k,n}$ defined by the generator g_i is called $\mathcal{G}$ -edge. A *clique* in a graph is a set of pairwise adjacent nodes. A clique is said to be *maximal* if it is not contained in another clique. A $\mathcal{G}$ -clique is a maximal clique in $G_{k,n}$ such that each pair of adjacent nodes is connected by a $\mathcal{G}$ -edge. For example, $\{\tilde{0}0, \tilde{0}1\}$ is a $\mathcal{G}$ -clique in $G_{3,2}$. Note that a $\mathcal{G}$ -clique contains $k-1$ nodes. Given some fixed $0 \leq s \leq k-2$, each $\mathcal{G}$ -clique has a unique node v satisfying that $v[n-1] = s$. Therefore, for a $\mathcal{G}$ -clique K , we can define the K -leader (leader for short if no ambiguity arises) to be the unique node $v \in K$ satisfying that $v[n-1] = 0$. We use such a node to represent its corresponding $\mathcal{G}$ -clique for convenience. Two $\mathcal{G}$ -cliques are *node-disjoint* if they have no common node.

Theorem 11. $G_{k,n}$ can embed $n(k-1)^{n-1}$ node-disjoint $\mathcal{G}$ -cliques of size $k-1$.

Proof. Clearly, there are $n(k-1)^{n-1}$ leaders in $G_{k,n}$, which correspond to $n(k-1)^{n-1}$ $\mathcal{G}$ -cliques. It is not difficult to verify that these $\mathcal{G}$ -cliques are pairwise node-disjoint. Moreover, since each $\mathcal{G}$ -clique contains $k-1$ nodes, the $n(k-1)^{n-1}$ $\mathcal{G}$ -cliques form a partition of the vertices of $G_{k,n}$. $\square$

Two $\mathcal{G}$ -cliques K_1 and K_2 are said to be *adjacent* if there exists a pair of vertices $u \in K_1$ and $v \in K_2$ such that either $v = f(u)$ or $v = f^{-1}(u)$.

Theorem 12. *Each $\mathcal{G}$ -clique in $G_{k,n}$ is adjacent to $2(k-1)$ (respectively, $k-1$) different $\mathcal{G}$ -cliques when $n > 2$ (respectively, $n = 2$).*

Proof. We first consider that $n > 2$. Given an arbitrary $\mathcal{G}$ -clique with its leader $v = s_0^* s_1^* \dots s_{n-2}^* 0^*$, we can apply $f^i \circ g_j(v)$ for $i = \pm 1$ and $1 \leq j \leq k-2$, to reach $2(k-2)$ distinct $\mathcal{G}$ -cliques, and apply $f^{\pm 1}(v)$ to reach another two distinct $\mathcal{G}$ -cliques. Therefore, each $\mathcal{G}$ -clique is adjacent to $2(k-1)$ distinct $\mathcal{G}$ -cliques.

When $n = 2$, we can apply $f^1 \circ g_j(v)$ and $f^1(v)$ (or $f^{-1} \circ g_j(v)$ and $f^{-1}(v)$) to reach $k-1$ distinct $\mathcal{G}$ -cliques. Note that the $\mathcal{G}$ -cliques reached by $\{f^1 \circ g_j(v), f^1(v)\}$ are the same with those reached by $\{f^{-1} \circ g_j(v), f^{-1}(v)\}$. $\square$

5 Maximally fault tolerance

The node fault tolerance of an undirected graph is measured by the node-connectivity of the graph [24]. The node-connectivity of a graph G , written $\kappa(G)$, is the minimum size of a node set S such that $G - S$ is disconnected or has only one node [23]. A graph G is ξ -connected if its node-connectivity is at least ξ . Obviously, the node-connectivity of a graph G cannot exceed the minimum degree of a node in G ; thus $\kappa(G_{k,n}) \leq k$ since $G_{k,n}$ is k -regular. A graph is said to be *maximally fault-tolerant* if node-connectivity equals the minimum node-degree of the given graph. A set of paths from u to v are *pairwise internally disjoint* if any two of them have no common internal nodes. In this section, we show that $\kappa(G_{k,n}) = k$ and hence this class of graphs is maximally fault-tolerant.

Lemma 13. [19] $\kappa(RG_{3,n}) = 3$ for $n \geq 2$.

Since the node-connectivity of $G_{k,n}$ for $k = 3$ was shown in [19], we only consider $k \geq 4$ in the reminder of this section.

Definition 5. For a k -valent graph $G_{k,n}$, the *reduced graph* $RG_{k,n}$ of $G_{k,n}$ can be constructed using the following two steps:

1. For each $\mathcal{F}$ -cycle with its leader $\tilde{0}s_1s_2 \dots s_{n-1}$, shrink it into a single node, namely *super node*, and label it with $s_1s_2 \dots s_{n-1}$;
2. Connect two arbitrary super nodes by an undirected edge, namely *super edge*, if the corresponding $\mathcal{F}$ -cycles are adjacent in $G_{k,n}$.

For convenience, we use the notation $P[x, y]$ to denote a path from x to y .

Lemma 14. $\kappa(RG_{k,n}) \geq k$ for $k \geq 4$ and $n \geq 3$.

Proof. Consider a subgraph $R_{k,n}$ of $RG_{k,n}$ defined as follows: The node set of $R_{k,n}$ is the same with $RG_{k,n}$; and two node $x_1x_2 \dots x_{n-1}$ and $y_1y_2 \dots y_{n-1}$ of $R_{k,n}$ are adjacent iff there is exactly one i such that $x_i \neq y_i$ and $x_j = y_j$ for all $j \neq i$.

We show the lemma by showing that there are k pairwise internally disjoint paths between two arbitrary distinct nodes in $R_{k,n}$. The proof is by induction on n .

BASIS: Prove that the result is true for $n = 3$. Consider two arbitrary nodes u and v in $R_{k,3}$.

Case 1. u and v have exactly one different symbol. Without loss of generality, assume that $u = 0x$ and $v = 0y$. According to Corollary 10 and the construction of $R_{k,n}$, k pairwise internally disjoint paths from u to v can be constructed as follows: $(0x, 0y)$, $(0x, 1x, 1y, 0y)$, $\dots$, $(0x, (k-2)x, (k-2)y, 0y)$ and $(0x, 0z, 0y)$ for $z \in \{0, 1, \dots, k-2\} \setminus \{x, y\}$.

Case 2. u and v have two different symbols. Assume that $u = u_1u_2$ and $v = v_1v_2$. Then, according to Corollary 10 and the construction of $R_{k,n}$, $2k-4$ ($\geq k$ when $k \geq 4$) pairwise internally disjoint paths from u to v can be constructed as follows: (u_1u_2, u_1v_2, v_1v_2) , (u_1u_2, v_1u_2, v_1v_2) , $(u_1u_2, xu_2, xv_2, v_1v_2)$, and $(u_1u_2, u_1x, v_1x, v_1v_2)$ for all $x \in \{0, 1, \dots, k-2\} \setminus \{u_1, v_1\}$.

Thus the base case is true.

INDUCTION STEP: Assume that the result is true for $n-1$ and then show it is true for $n \geq 4$. It is not difficult to verify that $R_{k,n}$ has a recursive structure which can be decomposed into $k-1$ copies of $R_{k,n-1}$. Each $R_{k,n-1}$ is a subgraph of $R_{k,n}$ induced by nodes in $x_1x_2 \dots x_{n-2}y$ for some fixed symbol y and $x_i \in \{0, 1, \dots, k-2\}$. Consider two arbitrary node u and v . If u and v are in the same $R_{k,n-1}$, then there are k pairwise internally disjoint paths from u to v by induction hypothesis. Otherwise, u and v are in different $R_{k,n-1}$'s. Assume that $u = u_1u_2 \dots u_{n-2}0$ and $v = v_1v_2 \dots v_{n-2}1$. By the induction hypothesis, there are k pairwise internally disjoint paths $P_1, P_2, \dots, P_k$ from u to the node $v' = v_1v_2 \dots v_{n-2}0$ because u and v' are in the same $R_{k,n-1}$. Let $n_1, n_2, \dots, n_k$ be the neighbors of v' which belong to paths $P_1, P_2, \dots, P_k$, respectively. According to Corollary 10 and all n_i 's are in the

same $R_{k,n-1}$, we have that $n_i = t_1^i t_2^i \dots t_{n-2}^i 0$ in which $t_j^i \neq v_j$ for some j and $t_l^i = v_l$ for all $1 \leq l \leq n-2$ and $l \neq j$. Let n'_i be the neighbor of n_i which is located in the same $R_{k,n-1}$ with v , that is, $n'_i = t_1^i t_2^i \dots t_{n-2}^i 1$. Note that n'_i , for all $1 \leq i \leq k$, are adjacent to v . Also let $P_i[u, n_i]$ be the subpath of P_i from u to n_i , $1 \leq i \leq k$. Then, k pairwise internally disjoint paths from u to v can be constructed as $P_i[u, n_i] + (n_i, n'_i) + (n'_i, v)$ for all $i = 1, 2, \dots, k$, where “+” means the path-concatenation operation¹. $\square$

The following lemma is useful to prove Lemma 16.

Lemma 15. [12] *Given two sets of nodes V_1 and V_2 such that $|V_1| = |V_2| = \xi$ in a ξ -connected graph, there are ξ node disjoint paths connecting the nodes from V_1 to V_2 .*

Lemma 16. $\kappa(G_{k,n}) \geq k$ for $k \geq 4$ and $n \geq 2$.

Proof. We show the lemma by showing that there are k pairwise internally disjoint paths between two arbitrary nodes u and v in $G_{k,n}$. There are two cases.

Case 1. $n = 2$. Recall that in this case, $RG_{k,2}(= R_{k,2})$ is a complete graph on $k-1$ nodes. Thus there are totally $k-1$ $\mathcal{F}$ -cycles in $G_{k,2}$.

Case 1.1. u and v are in the same $\mathcal{F}$ -cycle C . This cycle is a concatenation of two internally disjoint paths from u to v , denoted by P_1 and P_2 , which contribute two paths. Let $C_1, C_2, \dots, C_{k-2}$ be the other $\mathcal{F}$ -cycles of $G_{k,2}$. Then, the other $k-2$ paths can be constructed using the following three steps:

1. For each C_i , identify the *entry node* $g_{j_i}(u) \in C_i$ and the *exit node* $g_{l_i}(v) \in C_i$, where $1 \leq j_i, l_i \leq k-2$.²
2. For each C_i , construct a path Q_i from $g_{j_i}(u)$ to $g_{l_i}(v)$ using the $\mathcal{F}$ -edges of corresponding $\mathcal{F}$ -cycle.
3. Construct the desired k internally disjoint paths: $P_1, P_2, (u, g_{j_1}(u)) + Q_1 + (g_{l_1}(v), v), (u, g_{j_2}(u)) + Q_2 + (g_{l_2}(v), v), \dots, (u, g_{j_{k-2}}(u)) + Q_{k-2} + (g_{l_{k-2}}(v), v)$.

Case 1.2. u and v are in different $\mathcal{F}$ -cycles. Let F_1 and F_2 be the $\mathcal{F}$ -cycles to which u and v belong, respectively. We first construct three pairwise internally disjoint paths from u to v using F_1 and F_2 . If u and v are adjacent, then the desired paths can be constructed as follows:

¹In the remainder of this paper, the notation “+” is used to mean the path-concatenation.

²It is not difficult to show that there exist j_i and l_i such that $g_{j_i}(u)$ and $g_{l_i}(v)$ are in the same $\mathcal{F}$ -cycle C_i .

A. Find two nodes u_1 and u_2 (respectively, v_1 and v_2) different from u (respectively, v) in F_1 (respectively, F_2) such that $g_i(u_1) = v_1$ and $g_i(u_2) = v_2$ for some $1 \leq i \leq k-2$.

B. Let $P[u, u_1]$ and $P[u, u_2]$ (respectively, $Q[v_1, v]$ and $Q[v_2, v]$) be two paths in F_1 (respectively, F_2) whose nodes are all different but u (respectively, v). Then, $(u, v), P[u, u_1] + (u_1, v_1) + Q[v_1, v]$, and $P[u, u_2] + (u_2, v_2) + Q[v_2, v]$ form the desired three paths.

Otherwise, if u and v are not adjacent, then the desired three paths can be constructed as follows:

I. Find two nodes u_1 and u_2 (respectively, v_1 and v_2) different from u (respectively, v) in F_1 (respectively, F_2) such that $g_i(u_1) = v, g_j(u_2) = v_1$, and $g_l(u) = v_2$, where $1 \leq i, j, l \leq k-2$.

II. Let $P'[u, u_1]$ and $P'[u, u_2]$ (respectively, $Q'[v_1, v]$ and $Q'[v_2, v]$) be two paths in F_1 (respectively, F_2) whose nodes are all different but u (respectively, v). Then, $P'[u, u_1] + (u_1, v), (u, v_2) + Q'[v_2, v]$, and $P'[u, u_2] + (u_2, v_1) + Q'[v_1, v]$ form the desired three paths.

The other $k-3$ internally disjoint paths from u to v can be constructed from the other $k-3$ $\mathcal{F}$ -cycles using the method similar to that of Case 1.1.

Case 2. $n \geq 3$. If u and v are in the same $\mathcal{F}$ -cycle, there exist k internally disjoint paths from u to v : two paths are constructed in the same $\mathcal{F}$ -cycle, and the other $k-2$ paths are constructed from adjacent $k-2$ $\mathcal{F}$ -cycles based on the method similar to that of Case 1. If u and v are in different $\mathcal{F}$ -cycles, then the set of nodes $\{g_1(u), g_2(u), \dots, g_{k-2}(u), g_1(f(u)), g_1(f^{-1}(u))\}$ (respectively, $\{g_1(v), g_2(v), \dots, g_{k-2}(v), g_1(f(v)), g_1(f^{-1}(v))\}$) which are in k different $\mathcal{F}$ -cycles, denoted by $\{C_1, C_2, \dots, C_k\}$ (respectively, $\{C'_1, C'_2, \dots, C'_k\}$), and can be reached from u (respectively, v). By Lemma 14 and Lemma 15, there are k node-disjoint paths connecting $\{C_1, C_2, \dots, C_k\}$ and $\{C'_1, C'_2, \dots, C'_k\}$ in $RG_{k,n}$. This implies that there exist k pairwise internally disjoint paths from u to v in $G_{k,n}$. $\square$

We conclude this section with the following result.

Theorem 17. $\kappa(G_{k,n}) = k$ for $k \geq 3$ and $n \geq 2$.

Proof. By Lemma 13 and Lemma 16, we have that $\kappa(G_{k,n}) \geq k$. By the fact that $G_{k,n}$ is a k -regular graph, we have that $\kappa(G_{k,n}) \leq k$. Thus the result holds. $\square$

6 Concluding remarks

In this paper, we generalize the concept of the trivalent graph and propose a new family of k -valent graphs $G_{k,n}$ for designing interconnection networks. The proposed graph is regular with the node-degree k , where k can be any integer larger than 3. We also show that $G_{k,n}$ for some fixed integer $k \geq 6$, has logarithmic diameter subject to the number of nodes, and has maximally fault tolerance. Besides, a shortest path routing algorithm is also presented.

We also show that $G_{k,n}$ can embed cycles. A cycle structure, which is a fundamental topology for parallel and distributed processing, is suitable for local area networks and for the development of simple parallel algorithms with low communication cost [17]. A cycle structure can also be used as a control/data flow structure for distributed computation in arbitrary networks. As with our result, $(k-1)^{n-1}$ node-disjoint cycles can be emulated in $G_{k,n}$. Previously, a number of parallel algorithms that can be executed on cycles have been developed [3]. Those algorithms can be executed as well on $G_{k,n}$.

References

- [1] S. B. Akers and B. Krishnamurthy, "The star graph: an attractive alternative to n -cube," in *Proceedings of International Conference on Parallel Processing*, St. Charles, IL, pp. 555–556, 1987.
- [2] S. B. Akers and B. Krishnamurthy, "A group-theoretic model for symmetric interconnection networks," *IEEE Transactions on Computers*, 38(4):555–556, 1989.
- [3] S. G. Akl, *Parallel Computation: Models and Methods*. Prentice Hall, 1997.
- [4] L. Bhuyan and D. P. Agrawal, "Generated hypercube and hyperbus structure for a computer network," *IEEE Transactions on Computers*, bf33:323–333, 1984.
- [5] C. Chen, D. P. Agrawal, and J. R. Burke, "dBCube: a new class of hierarchical multiprocessor interconnection networks with area efficient layout," *IEEE Transactions on Parallel and Distributed Systems*, 4:1332–1344, 1993.
- [6] G. H. Chen, J. S. Fu, and J. F. Fang, "Hypercomplete: a pancyclic recursive topology for large-scale distributed multicomputer systems," *Networks*, 35(1):56–69, 2000.
- [7] K. Day and A. Tripathi, "Arrangement graphs: a class of generated star graphs," *Information Processing Letters*, 42:235–241, 1992.
- [8] K. Ghose and K. R. Desai, "Hierarchical cubic networks," *IEEE Transactions on Parallel and Distributed Systems*, 6:427–435, 1995.
- [9] W. J. Hsu, "Fibonacci cubes: a new interconnection topology," *IEEE Transactions on Parallel and Distributed Systems*, 4:3–12, 1993.
- [10] K. Hwang and J. Ghosh, "Hypernet: a communication-efficient architecture for constructing massively parallel computers," *IEEE Transactions on Computers*, C-36:1450–1466, 1987.
- [11] Q. M. Malluhi and M. A. Bayoumi, "The hierarchical hypercube: a new interconnection topology for massively parallel systems," *IEEE Transactions on Parallel and Distributed Systems*, 5:17–30, 1994.
- [12] K. Menger, "Zur allgemeinen Kurventheorie," *Fund. Math.*, 10:95–115, 1927.
- [13] S. Öhring and S. K. Das, "Folded Petersen cube networks: new competitors for the hypercubes," *IEEE Transactions on Parallel and Distributed Systems*, 7:151–168, 1996.
- [14] S. Okawa, "Correction to the diameter of trivalent Cayley graphs," *IEICE Transactions on Fundamentals*, E84-A:1269–1272, 2001.
- [15] F. P. Preparata and J. Vuillemin, "The cube-connected cycles: a versatile network for parallel computation," *Communication of the ACM*, 24:300–309, 1981.
- [16] M. R. Samatham and D. K. Pradhan, "The de Bruijn multiprocessor network: a versatile parallel processing and sorting networks for VLSI," *IEEE Transactions on Computers*, C-38:567–581, 1989.
- [17] G. Tel, *Topics in Distributed Algorithms*. Cambridge Int'l Series on Parallel Computation, Cambridge University Press, 1991.
- [18] C.-H. Tsai, C.-N. Hung, L.-H. Hsu, and C.-H. Chang, "The correct diameter of trivalent Cayley graphs," *Information Processing Letters*, 72:109–111, 1999.
- [19] P. Vadapalli and P. K. Srimani, "Trivalent Cayley graphs for interconnection networks," *Information Processing Letters*, 54:329–335, 1995.
- [20] P. Vadapalli and P. K. Srimani, "Shortest routing in trivalent Cayley graph network," *Information Processing Letters*, 57:183–188, 1996.

- [21] P. Vadapalli and P. K. Srimani, "A new family of Cayley graph interconnection networks for constant degree four," *IEEE Transactions on Parallel and Distributed Systems*, 7:26–32, 1996.
- [22] M. D. Wagh and J. Mo, "Hamilton cycles in Trivalent Cayley graphs," *Information Processing Letters*, 60:177–181, 1996.
- [23] Douglas B. West, *Introduction to graph theory*, Prentice-Hall, Upper Saddle River, NJ 07458, 2001.
- [24] Junming Xu, "Topological structure and analysis of interconnection networks," Kluwer Academic, 2001.

A Appendix

Algorithm 3 $\text{DIST_2}(v, n, m, LZR, j_R)$

```

1: for  $i = 0$  to  $(n - 2 - j_R - LZR)$  do
2:    $v \leftarrow f^{-1}(v)$ 
3: end for
4: if  $(n - j_R - LZR) > 0$  then
5:    $v \leftarrow g_{k-v[n-1]-1}(v)$ 
6:   for  $i = 0$  to  $(n - 2 - j_R - LZR)$  do
7:      $v \leftarrow f(v)$ 
8:     if  $v[n - 1] \neq 0$  then
9:        $v \leftarrow g_{k-v[n-1]-1}(v)$ 
10:    end if
11:  end for
12: end if
13: for  $i = 0$  to  $(m - 1)$  do
14:    $v \leftarrow f(v)$ 
15:   if  $v[n - 1] \neq 0$  then
16:      $v \leftarrow g_{k-v[n-1]-1}(v)$ 
17:   end if
18: end for
19: for  $i = 0$  to  $j_R - m - 1$  do
20:    $v \leftarrow f(v)$ 
21: end for
22: for  $i = 0$  to  $j_R - m - 1$  do
23:   if  $v[n - 1] \neq 1$  then
24:      $v \leftarrow g_{k-v[n-1]}(v)$ 
25:   end if
26:    $v \leftarrow f^{-1}(v)$ 
27: end for

```

Algorithm 4 $\text{DIST_3}(v, n, m)$

```

1: for  $i = 0$  to  $(m - 1)$  do
2:    $v \leftarrow f(v)$ 
3:   if  $v[n - 1] \neq k - 2$  then
4:      $v \leftarrow g_{k-2-v[n-1]}(v)$ 
5:   end if
6: end for
7: for  $i = m$  to  $(n - 1)$  do
8:    $v \leftarrow f(v)$ 
9:   if  $v[n - 1] \neq 0$  then
10:     $v \leftarrow g_{k-1-v[n-1]}(v)$ 
11:   end if
12: end for
13: for  $i = 0$  to  $(m - 1)$  do
14:    $v \leftarrow f(v)$ 
15: end for

```

Algorithm 5 $\text{DIST_4}(v, n, m)$

```

1: for  $i = m$  to  $(n - 1)$  do
2:   if  $v[n - 1] \not\equiv 2 \pmod{k - 1}$  then
3:      $v \leftarrow g_{k-v[n-1]+1}(v)$ 
4:   end if
5:    $v \leftarrow f^{-1}(v)$ 
6: end for
7: for  $i = 0$  to  $(m - 1)$  do
8:   if  $v[n - 1] \neq 1$  then
9:      $v \leftarrow g_{k-v[n-1]}(v)$ 
10:   end if
11:    $v \leftarrow f^{-1}(v)$ 
12: end for
13: for  $i = m$  to  $(n - 1)$  do
14:    $v \leftarrow f^{-1}(v)$ 
15: end for

```

On the C_4 -destroying of Bipartite Permutation Graphs

(Extended Abstract)

*Sheng-Lung Peng, Jian-Hong Lee, Jun-Rong Lin, Chih-Shan Liu
Department of Computer Science and Information Engineering
National Dong Hwa University
Hualien, Taiwan, R.O.C.

Abstract

In this paper we propose a succinct representation for bipartite permutation graphs. Using this representation, we propose a unified approach on a given bipartite permutation graph G for the following problems: (1) adding the minimum number of edges on G such that the resulting graph has no C_4 (the minimum fill-in problem); (2) adding edges on G such that the resulting graph has no C_4 and its maximum clique size is as small as possible (the treewidth problem).

1 Introduction

Let $G = (V, E)$ be a finite, simple, and undirected graph where V and E denote the vertex and edge sets of G , respectively. For a vertex set $W \subseteq V$, let $G(W)$ denote the induced subgraph of G which is induced by W . A *biclique* is a complete bipartite graph. Let $N(v) = \{w | (v, w) \in E\}$ and $N[v] = N(v) \cup \{v\}$.

A graph $G = (V, E)$ is *bipartite* if V can be partitioned into two sets S and T such that every edge has its ends in different sets: vertices in the same set must not be adjacent. Equivalently, G is bipartite if and only if it is 2-colorable and is denoted as $G = (S, T, E)$.

A graph G is a *permutation graph* if there is some pair P, Q of permutations of the vertex set such that there is an edge between vertices x and y if and only if x precedes y in one of P, Q , while y precedes x in the other. A permutation graph can also be described by a permutation diagram. Let the vertices in the order of P be on a line and the vertices in the order of Q be on a parallel line. Draw an edge between two vertices that having the same assigned number. Then, there is an edge

(x, y) in G if and only if the line segment for x intersects the line segment for y .

Bipartite permutation graphs simultaneously have both of the properties of bipartite graphs and permutation graphs. Bipartite permutation graphs are first proposed by Spinrad, Brandstadt, and Stewart in 1985 [19].

A graph is *chordal* if it does not contain a chordless cycle of length greater than three. The *treewidth* (respectively, *minimum fill-in*) of a graph is the smallest maximum clique size (respectively, the minimum number of additional edges) of its all possible chordal supergraphs. The *treewidth problem* (respectively, *minimum fill-in problem*) of a graph is to find its treewidth (respectively, minimum fill-in).

The treewidth problem is NP-complete on cobipartite graphs (complement of bipartite graphs) [1], bipartite graphs [15], and graphs of maximum degree at most 9 [6]. For some special classes of graphs, it has been shown that the treewidth can be computed in polynomial time, such as cographs [5], circular-arc graphs [20], chordal bipartite graphs [16], permutation graphs [3], circle graphs [13], d -trapezoid graphs [4], and distance hereditary graphs [8].

The minimum fill-in problem is NP-complete on cobipartite graphs [22] and bipartite graphs [21]. The known graph classes for which the minimum fill-in can be computed by a polynomial time algorithm are cographs [10], d -trapezoid graphs [4], bipartite permutation graphs [19], chordal bipartite graphs [14, 9], and circle and circular-arc graphs [18].

In this paper, we first propose a representation for bipartite permutation graphs. That is, we show that a bipartite permutation graph can be obtained from a sequence of bicliques, e.g., $B_1, B_2, \dots, B_r$. Using this structure, we show that the treewidth and minimum fill-in problems can

*Corresponding author: lung@csie.ndhu.edu.tw

be solved in $O(r^2)$ time on bipartite permutation graphs. In the case of $r = o(n)$, it is an improvement where n is the number of vertices in the input graph.

The rest of this paper is organized as follows. In Section 2, we give a new representation for bipartite permutation graphs. Our unified algorithm for minimum fill-in and treewidth problems on bipartite permutation graphs is shown in Section 3. Finally, we give the concluding remarks in the last section.

2 Biclique structure of bipartite permutation graphs

Let $G = (S, T, E)$ be a bipartite graph. An ordering of the vertices in S has the *adjacency property* if for each vertex x in T , $N(x)$ consists of which are consecutive in the ordering of S . An ordering of the vertices in S has the *enclosure property* if for every pair of vertices v, w in T such that $N(v)$ is a subset of $N(w)$, vertices in $N(w) \setminus N(v)$ occur consecutively in the ordering of S . A *strong ordering* of the vertices of $G = (S, T, E)$ consists of an ordering of S and an ordering of T such that for all $(s, t), (s', t')$ in E where s, s' are in S and t, t' are in T , $s < s'$ and $t > t'$ imply (s, t') and (s', t) are in E . In [19], Spinrad et al. give the following characterization of a bipartite permutation graph.

Theorem 2.1 [19] The following statements are equivalent:

1. $G = (S, T, E)$ is a bipartite permutation graph.
2. There is a strong ordering of $S \cup T$.
3. There exists an ordering of S which has the adjacency and enclosure properties.

For a biclique $B = (X, Y, E')$, $E' = \{(x, y) \mid x \in X \text{ and } y \in Y\}$ is completely determined by X and Y . Thus, in the following, we use $B = (X, Y)$ to denote a biclique. Let $first(X, k)$ (respectively, $last(X, k)$) be the first (respectively, last) k vertices of X with ordering. For example, assume that $X = \{x_1, x_2, \dots, x_r\}$. Then $first(X, k) = \{x_1, x_2, \dots, x_k\}$ and $last(X, k) = \{x_{r-k+1}, x_{r-k+2}, \dots, x_{r-1}, x_r\}$.

For two bicliques $B_1 = (X_1, Y_1)$ and $B_2 = (X_2, Y_2)$ and two positive integers k and l , we define a concatenating operation $\oplus$ on B_1 and B_2 as follows:

Let $G = (S, T, E) = B_1 \oplus_l^k B_2$ where $S = X_1 \cup X_2$ by letting $last(X_1, k) = first(X_2, k)$ and $T = Y_1 \cup Y_2$ by letting $last(Y_1, l) = first(Y_2, l)$ with E being the union of edges in B_1 and B_2 and k, l satisfying the following two conditions.

1. $k + l \geq 1$,
2. neither $|X_1| = |X_2| = k$ nor $|Y_1| = |Y_2| = l$.

By Condition 1, G is connected. By Condition 2, it is impossible to have $|X_1| = |X_2| = k$ (or $|Y_1| = |Y_2| = l$). Otherwise, $B_1 \oplus_l^k B_2$ becomes a biclique. Note that B_1 and B_2 are two maximal bicliques of G . For convenience, we say that $last(X_1, k) \cup last(Y_1, l) = first(X_2, k) \cup first(Y_2, l)$ is the intersection of B_1 and B_2 .

For convenience, if there is no confusion, then $B_1 \oplus_l^k B_2$ is briefed as $B_1 \oplus B_2$. For a sequence of bicliques $B_1, B_2, \dots, B_r$, we define G_i as follows:

- (1) $G_1 = B_1$,
- (2) let $G_i = B_1 \oplus B_2 \oplus \dots \oplus B_i$ be the graph by concatenating $B_1, B_2, \dots, B_i$,
- (3) $G_{i+1} = G_i \oplus B_{i+1}$ where $\oplus$ is applied on B_i and B_{i+1} .

Lemma 2.2 G_r is a bipartite permutation graph.

Proof. We prove this lemma by induction.

- (i) G_1 is a biclique which is a bipartite permutation graph.
- (ii) Assume that $G_i = B_1 \oplus B_2 \oplus \dots \oplus B_i$ is a bipartite permutation graph. Now, we consider $G_{i+1} = G_i \oplus_l^k B_{i+1} = B_1 \oplus B_2 \oplus \dots \oplus B_i \oplus_l^k B_{i+1}$ where $B_i = (X_i, Y_i)$ and $B_{i+1} = (X_{i+1}, Y_{i+1})$. It is not hard to see that the neighborhood is changed only in the vertices of $B_i \cap B_{i+1}$. By the definition, vertices in $B_i \cap B_{i+1}$ satisfy the adjacency property. It is also not hard to see that they satisfy the enclosure property. That is, G_{i+1} is a bipartite permutation graph by Theorem 2.1.

Thus, by induction, G_r is a bipartite permutation graph. $\square$

If G is obtained by $B_1 \oplus_{l_1}^{k_1} B_2 \oplus_{l_2}^{k_2} \dots \oplus_{l_{r-1}}^{k_{r-1}} B_r$, then we say that $B_1, B_2, \dots, B_r$ is a *generating base* of G with respect to $(\oplus_{l_1}^{k_1}, \oplus_{l_2}^{k_2}, \dots, \oplus_{l_{r-1}}^{k_{r-1}})$. On the other hand, given a bipartite permutation graph $G = (S, T, E)$ with a strong ordering, we can obtain a generating

base as follows. Assume that $S = \{s_1, s_2, \dots, s_a\}$ and $T = \{t_1, \dots, t_b\}$. Let the largest index in $N(s_1)$ (respectively, $N(t_1)$) be j (respectively, i). Let $X_1 = \{s_1, \dots, s_j\}$ and $Y_1 = \{t_1, \dots, t_i\}$. Then $B_1 = (X_1, Y_1)$ is a maximal biclique of G . After B_1 is determined, we delete s_1 (respectively, t_1) and the other vertices whose neighborhood is $N(s_1)$ (respectively, $N(t_1)$). It is not hard to see the remaining graph is still a bipartite permutation graph and the ordering of vertices is still strong. By repeating this process, we can obtain $B_2, B_3, \dots, B_r$.

Finally, we have the following theorem.

Theorem 2.3 *G is a bipartite permutation graph if and only if there exists a generating base for G .*

3 A unified algorithm

Given a bipartite permutation graph $G = (S, T, E)$ with vertices of S and T in strong ordering, each crossing of (u, y) and (w, x) is contained in a cycle u, y, w, x, u . That is, each crossing pair represents a 4-cycle in G . By the definition, a graph is chordal if every cycle of length greater than three has a chord. Therefore, to form a chordal graph, every 4-cycle must be destroyed (by adding edges). The set of all the additional edges is called C_4 -destroying set. A previous investigation by Spinrad et al. [19] showed that the union of G and any minimal C_4 -destroying set is chordal. For convenience, the union is called a *triangulation* of G . We denote that E' is a minimal C_4 -destroying set and $G' = (S \cup T, E \cup E')$ is a chordal graph. For a vertex set W , let $E_W = \{(u, v) \in E \mid u, v \in W\}$. Spinrad et al. showed the following lemma.

Lemma 3.1 [19] *If (u, w) is in E'_S (E'_T), then for all v in S (T) such that $u < v < w$, the edges (u, v) and (v, w) are also in E'_S (E'_T).*

By Lemma 3.1 we have the following lemma.

Lemma 3.2 *Let $B = (X, Y)$ be a biclique in bipartite permutation graph G , then either X or Y must form a clique in any C_4 -destroying set of G .*

Intuitively, Lemma 3.2 tells us that we need to find all the possible maximal bicliques of G for C_4 -destroying. Let $G = (S, T, E)$ be a bipartite permutation graph with its generating base $\{B_1, \dots, B_r\}$ where $B_i = (X_i, Y_i)$ for all $i \in \{1, \dots, r\}$. Assume that $T(G_{i-1})$ is a triangulation of G_{i-1} . Consider G_i and we define six operations for triangulating G_i as follows:

- OU_i^1 : in the case that Y_{i-1} is a clique in $T(G_{i-1})$, vertices in X_i form a clique;
- OU_i^2 : in the case that X_{i-1} is a clique in $T(G_{i-1})$, vertices in $X_j \cup X_i$ form a clique where j is the smallest index such that $X_j \cap X_i \neq \emptyset$;
- OU_i^3 : in the case that X_{i-1} is a clique in $T(G_{i-1})$, vertices in X_i and $Y_{i-1} \cap Y_i$ form respective cliques;
- OD_i^1 : in the case that X_{i-1} is a clique in $T(G_{i-1})$, vertices in Y_i form a clique;
- OD_i^2 : in the case that Y_{i-1} is a clique in $T(G_{i-1})$, vertices in $Y_j \cup Y_i$ form a clique where j is the smallest index such that $Y_j \cap Y_i \neq \emptyset$;
- OD_i^3 : in the case that Y_{i-1} is a clique in $T(G_{i-1})$, vertices in Y_i and $X_{i-1} \cap X_i$ form respective cliques.

It is not hard to see that if G_{i-1} is triangulated, then G_i can be triangulated by applying any operation defined above. If G_i is triangulated by OX_i^j , we call it Type Z_i^j of triangulation where $Z \in \{U, D\}$ and $j \in \{1, 2, 3\}$.

Lemma 3.3 *If there is a way to triangulate G_{i-1} , then G_i can be triangulated using each of the six operators.*

Proof. Let $B_{j \cap i}^{j \cup i}$ (respectively, $B_{j \cup i}^{j \cap i}$) be the biclique $B = (X_j \cup X_i, Y_j \cap Y_i)$ (respectively, $B = (X_j \cap X_i, Y_j \cup Y_i)$) with $Y_j \cap Y_i \neq \emptyset$ (respectively, $X_j \cap X_i \neq \emptyset$). By the concatenating operator, the maximal bicliques occurred in G_i but not in G_{i-1} are B_i , $B_{p \cap i}^{p \cup i}$, and $B_{q \cup i}^{q \cap i}$ for all possible p and q .

For each maximal biclique, at least one partite must become a clique by Lemma 3.1. Assume that $B_{i-1} = K_q^p$ and $B_i = K_s^r$. Since G_{i-1} has been triangulated, we only consider the new bicliques after adding B_i . It is not hard to see that the new maximal bicliques are B_i , are triangulated by the definition. In the case of K_{q+s-l}^k (respectively, K_l^{p+r-k}), vertices in $S \cap K_{q+s-l}^k$ (respectively, $T \cap K_l^{p+r-k}$) are contained in $S \cap K_s^r$ (respectively, $T \cap K_q^p$). Thus K_{q+s-l}^k (respectively, K_l^{p+r-k}) is triangulated. Hence type OU_i^1 is a triangulation of G_i . The other cases are similar. $\square$

By the definition, we have the following theorem.

Theorem 3.4 *For $i \in \{1, \dots, r\}$, we have the following statements.*

1. $U_i^1 = \min\{D_{i-1}^1 + OU_i^1, D_{i-1}^2 + OU_i^1, D_{i-1}^3 + OU_i^1\};$
2. $U_i^2 = \min\{U_{i-1}^1 + OU_i^2, U_{i-1}^2 + OU_i^2, U_{i-1}^3 + OU_i^2\};$
3. $U_i^3 = \min\{U_{i-1}^1 + OU_i^1, U_{i-1}^2 + OU_i^1, U_{i-1}^3 + OU_i^1\};$
4. $D_i^1 = \min\{U_{i-1}^1 + OD_i^1, U_{i-1}^2 + OD_i^1, U_{i-1}^3 + OD_i^1\};$
5. $D_i^2 = \min\{D_{i-1}^1 + OD_i^2, D_{i-1}^2 + OD_i^2, D_{i-1}^3 + OD_i^2\};$
6. $D_i^3 = \min\{D_{i-1}^1 + OD_i^1, D_{i-1}^2 + OD_i^1, D_{i-1}^3 + OD_i^1\}.$

Let $\text{mintri}(G_i)$ be the optimal triangulation of G_i . We have the following theorem.

Theorem 3.5 $\text{mintri}(G_i) = \min\{U_i^1, U_i^2, U_i^3, D_i^1, D_i^2, D_i^3\}.$

It is not hard to see that $U_1^1 = U_1^2 = U_1^3$ and $D_1^1 = D_1^2 = D_1^3$. We propose a unified algorithm for treewidth and minimum fill-in problems on bipartite permutation graphs using dynamic programming approach in the following.

Algorithm: Triangulating;

Input: $G = B_1 \oplus_1 \dots \oplus_{r-1} B_r$ where $r > 1$, $B_i = K_{q_i}^{p_i}$ and $\oplus_i = \oplus_{l_i}^{k_i}$ for all i ;

Output: The optimal value Opt ;

1. **initialize** $U_1^1, U_1^2, U_1^3, D_1^1, D_1^2, D_1^3$;
2. **for** $i = 2$ **to** r **do begin**
3. compute $U_i^1, U_i^2, U_i^3, D_i^1, D_i^2, D_i^3$;
4. **end**;
5. $Opt^+(G) = \min\{U_r^1, U_r^2, U_r^3\}$;
6. $Opt^-(G) = \min\{D_r^1, D_r^2, D_r^3\}$;
7. $Opt(G) = \min\{Opt^+(G_r), Opt^-(G_r)\}$;

For treewidth, the variables and formula are as follows.

1. $U_1^1 = U_1^2 = U_1^3 = p_1$;
2. $D_1^1 = D_1^2 = D_1^3 = q_1$;
3. $U_i^1 = \min\{\max\{D_{i-1}^1, p_i + l_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}, \max\{D_{i-1}^2, p_i + l_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}, \max\{D_{i-1}^3, p_i + l_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}\};$

4. $U_i^2 = \min\{\max\{U_{i-1}^1, p_i + \sum_{t=j}^{i-1} (p_t - k_t) - 1\}, \max\{U_{i-1}^2, p_i + \sum_{t=j}^{i-1} (p_t - k_t) - 1\}, \max\{U_{i-1}^3, p_i + \sum_{t=j}^{i-1} (p_t - k_t) - 1\}\};$
5. $U_i^3 = \min\{\max\{U_{i-1}^1, p_i + l_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}, \max\{U_{i-1}^2, p_i + l_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}, \max\{U_{i-1}^3, p_i + l_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}\};$
6. $D_i^1 = \min\{\max\{U_{i-1}^1, q_i + k_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}, \max\{U_{i-1}^2, q_i + k_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}, \max\{U_{i-1}^3, q_i + k_{i-1} - 1, p_{i-1} + l_{i-1} - 1\}\};$
7. $D_i^2 = \min\{\max\{D_{i-1}^1, q_i + \sum_{t=j}^{i-1} (q_t - l_t) - 1\}, \max\{D_{i-1}^2, q_i + \sum_{t=j}^{i-1} (q_t - l_t) - 1\}, \max\{D_{i-1}^3, q_i + \sum_{t=j}^{i-1} (q_t - l_t) - 1\}\};$
8. $D_i^3 = \min\{\max\{D_{i-1}^1, q_i + k_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}, \max\{D_{i-1}^2, q_i + k_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}, \max\{D_{i-1}^3, q_i + k_{i-1} - 1, q_{i-1} + k_{i-1} - 1\}\};$

Similarly, the variables and formula for minimum fill-in can also be defined as a similar way. We omit the detail. Finally, we have the following theorem.

Theorem 3.6 *Algorithm Triangulating can be implemented in $O(r^2)$ time.*

4 Conclusion

Given a graph, computing the treewidth and minimum fill-in are interesting problems. The latter problem has an $O(n^5)$ -time algorithm on bipartite permutation graphs [19]. Both problems have been solved in $O(n^2)$ time on permutation graphs [3, 4]. In this paper, we propose a unified $O(r^2)$ -time algorithm for the treewidth and minimum fill-in problems on bipartite permutation graphs where r is the size of a generating base of the input graph. Note that r ranges from $O(1)$ to $O(n)$. In the case of $r = o(n)$, then our result is an improvement of others.

References

- [1] S. Arnborg, D. G. Corneil, and A. Proskurowski. Complexity of finding embeddings in a k -tree. *SIAM Journal of Algebraic and Discrete Methods*, 8:277–284, 1987.

- [2] H. L. Bodlaender. A tourist guide through treewidth. *Acta Cybernetica*, 11:1–23, 1993.
- [3] H. L. Bodlaender, T. Kloks, and D. Kratsch. Treewidth and pathwidth of permutation graphs. *SIAM Journal on Graph Discrete Mathematics*, 8:606–616, 1995.
- [4] H. L. Bodlaender, T. Kloks, D. Kratsch, and H. Müller. Treewidth and minimum fill-in on d -trapezoid graphs. *Journal of Graph Algorithms and Applications*, 2:1–23, 1998.
- [5] H. L. Bodlaender and R. H. Möhring. The pathwidth and treewidth of cographs. *SIAM Journal on Discrete Mathematics*, 6:181–188, 1993.
- [6] H. L. Bodlaender and D. M. Thilikos. Treewidth and small separators for graphs with small chordality. *Technical Report UU-CS-1995-02, Department of Computer Science, Utrecht University, Utrecht*, 1995.
- [7] A. Brandstadt, V. B. Le, and J. P. Spinrad, *Graph Classes: A Survey*, SIAM, 1999.
- [8] H. Broersma, E. Dahlhaus, and T. Kloks. A linear time algorithm for minimum fill in and treewidth for distance hereditary graphs. *Scientific program 5th Twente Workshop on Graphs and Combinatorial Optimization*, 48–50, 1997.
- [9] M. S. Chang. Algorithms for maximum matching and minimum fill-in on chordal bipartite graphs. *Proceedings of the 7th International Symposium Algorithms and Computation*, 146–155, 1996.
- [10] G. A. Dirac. Cographs: recognition, applications and algorithms. *Congressus Numerantium*, 43:249–258, 1984.
- [11] M. R. Garey and D. S. Johnson, *Computer and Intractability: A Guide to the Theory of NP-completeness*, W.H. Freeman, San Francisco, 1979.
- [12] M. C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, New York, 1980.
- [13] T. Kloks. Treewidth of circle graphs. *Proceedings of the 4th International Symposium on Algorithms and Computation*, 108–117, 1993.
- [14] T. Kloks. Minimum fill-in for chordal bipartite graphs. *Technical Report RUU-CS-93-11, Department of Computer Science, Utrecht University, The Netherlands*, 1993.
- [15] T. Kloks. Treewidth—computations and approximations. *Lecture Notes in Computer Science* 842, 1994.
- [16] T. Kloks and D. Kratsch. Treewidth of chordal bipartite graphs. *Journal of Algorithms*, 19:266–281, 1995.
- [17] T. Kloks, D. Kratsch, and J. Spinrad. On treewidth and minimum fill-in of asteroidal triple-free graphs. *Journal of Theoretical Computer Science*, 175:309–335.
- [18] T. Kloks, D. Kratsch, and C. K. Wong. Minimum fill-in on circle and circular-arc graphs. *Proceedings of the 23 International Colloquium on Automata, Languages and Programming*, 256–267, 1996.
- [19] J. Spinrad, A. Brandstadt, and L. Stewart, Bipartite permutation graphs, *Discrete Applied Mathematics*, 18:279–292, 1987.
- [20] R. Sundaram, K. S. Singh, and C. P. Rangan. Treewidth of circular arc graphs. *SIAM Journal on Discrete Mathematics*, 7:647–655, 1994.
- [21] R. E. Tarjan. Decomposition by clique separators. *Discrete Mathematics*, 55:221–232, 1985.
- [22] M. Yannakakis. Computing the minimum fill-in is NP-complete. *SIAM Journal on Algebraic Discrete Methods*, 2:77–79, 1981.

Balancing Minimum Spanning Trees and Multiple-Source Minimum Routing Cost Spanning Trees on Metric Graphs

Chung-Ming Lin* Yin Te Tsai† Chuan Yi Tang‡

Abstract

The building cost of a spanning tree is the sum of weights of the edges used to construct the spanning tree. The routing cost of a source vertex s on a spanning tree T is the total summation of distances between the source vertex s and all the vertices d in T . Given a source vertices set S , the multiple-source routing cost of a spanning tree T is the summation of the routing costs for source vertices in S . Both the building cost and the multiple-source routing cost are important considerations in construction of a network system. A spanning tree with minimum building cost among all spanning trees is called a minimum spanning tree (MST), and a spanning tree with minimum k -source routing cost among all spanning trees is called a k -source minimum routing cost spanning tree (k -MRCT). Usually a k -MRCT of a graph G with respect to k sources is not a MST of G , and vice versa. This paper proposes an algorithm to construct a spanning tree T for a metric graph G with a source vertex set S such that the building cost of T is less than $1 + 2/(\alpha - 1)$ times of that of a MST of G , and the k -source routing cost of T is less than $\alpha(1 + \frac{2(k-1)(n-2)}{k(n+k-2)})$ times of that of a k -MRCT of G with respect to S , where $\alpha > 1$, $k = |S|$ and n is the number of vertices of G .

1 Introduction

Two important problems in network design and many other fields are the minimum spanning tree (MST) problem and the k -source minimum routing cost spanning tree (k -MRCT) problem. Let $G = (V, E, w)$ be a connected and undirected

graph, where V , E and w denote the vertex set, the edge set and the nonnegative weight function on E , respectively. The MST problem is to find a spanning tree T of G such that the total length of the edges of T is a minimum one among all spanning trees of G . Given k vertices of V as sources and all vertices of V as destinations, the k -MRCT problem is to find a spanning tree T of G such that the total distance from sources to destinations on the tree T is minimized among all spanning trees of G .

Both the MST problem and the k -MRCT problem are widely studied problems. For the first problem, Kruskal's algorithm [5] runs in $O(|E| \log |V|)$ time and Prim's algorithm [6] runs in $O(|E| + |V| \log |V|)$ time. (See the article by Graham and Hell [2] for a history of MST problem.) Note that 1-MRCT problem is exactly the shortest path tree (SPT) problem and can be solved by Dijkstra's algorithm [1] in $O(|V|^2)$ time. Wu *et al.* [7] have shown that 2-MRCT problem is NP-hard, and proposed a polynomial time approximation scheme (PTAS) algorithm for it. The $|V|$ -MRCT problem has been proven by Johnson *et al.* [3] to be NP-hard, and Wu *et al.* [10] have proposed a PTAS algorithm for it.

In fact, the k -MRCT problem is a special case of the sum-requirement optimal communication spanning tree (SROCT) problem. Given a non-negative weight for each vertex of V , the weighted routing cost of a vertex is the product of the weight of the vertex multiplied by the total distance from the vertex to all the other vertices. The SROCT problem is to find a spanning tree T of G such that the total weighted routing cost of V on the tree T is a minimum one among all spanning trees of G . The k -MRCT problem is a special case of the SROCT problem in which the weight of each source vertex is one and the weights of the other vertices are zeros. Since some special cases of the SROCT problem are NP-hard, the SROCT problem is NP-hard. Wu *et al.* [8] have proposed a 2-approximation algorithm for it.

The balancing MST and k -MRCT problem,

*Department of Computer Science, National Tsing-Hua University, Taiwan, ROC.

†Department of Computer Science and Information Management, Providence University, Taiwan, ROC. E-mail: yttsai@pu.edu.tw

‡Corresponding author: Department of Computer Science, National Tsing-Hua University, Taiwan, ROC. E-mail: cytang@cs.nthu.edu.tw

$\text{Balance}(x, y, k)$, is to find a spanning tree T of G such that (1) the total tree length of T is less than x times to the length of a minimum spanning tree and (2) the total k -source routing cost on T is less than y times to the optimal k -source routing cost among all possible spanning trees of G . In 1995, Khuller *et al.* [4] proposed algorithm Find-Last to solve the $\text{Balance}(1 + \frac{2}{\alpha-1}, \alpha, 1)$ problem in $O(n)$ time when a MST and a SPT of G are given, where $\alpha > 1$ and $n = |V|$. In 2002, Wu *et al.* [9] solved the $\text{Balance}(f(i) + \frac{2}{\alpha-1}, \frac{i+3}{i+1}\alpha, n)$ problem in $O(n^{i+1})$ time, where $\alpha > 1$ and $f(1) = 1$, $f(2) = 2$ and $f(i) = 3$ when $i > 3$. In this paper, we give an algorithm to solve the $\text{Balance}(1 + \frac{2}{\alpha-1}, \alpha(1 + \frac{2(k-1)(n-2)}{k(n+k-2)}), k)$ problem for a metric graph G in $O(kn)$ time when a MST of G and SPT's of G rooted at the k source vertices are given. These results are summarized in Table 1.

The remaining sections are organized as follows. Some basic definitions and notations are given in Section 2. The algorithm for the balancing MST and k -MRCT problem is presented in Section 3. Finally, we give the conclusions in Section.

2 Basic Definitions and Notations

In this section, we give some basic definitions and notations that will be used in the discussion of the following sections. Let $G = (V, E, w)$ denote a connected graph with nonnegative edge weights, where V , E and w denote the vertex set, the edge set and the nonnegative weight function on edges, respectively. First of all, we give the following definitions:

1. A spanning tree $T = (V_T, E_T, w_T)$ of $G = (V, E, w)$ is a tree that connect all vertices of G ; i.e. $V_T = V$, $E_T \in E$, and $w_T(e) = w(e)$ for all $e \in E_T$.
2. The building cost of a graph $G = (V, E, w)$, denoted by C_G , is the summation of the weights of the edges of G ; i.e. $C_G = \sum_{e \in E} w(e)$.
3. The distance for a pair of vertices $s, d \in V$ on a graph G , denoted by $D_G(s, d)$, is the summation of the weights of the edges on the shortest path from s to d on G .
4. A minimum spanning tree of G , denoted by $\text{MST}(G)$, is a spanning tree with minimum building cost among all spanning trees of G . A shortest path tree of G rooted at r , denoted

by $\text{SPT}(G, r)$, is a spanning tree of G such that $D_G(r, v) = D_{\text{SPT}(G, r)}(r, v)$ for any vertex v in G .

5. The routing cost of a source vertex $s \in V$ on $G = (V, E, w)$ is the summation of the distances from s to all the vertices $d \in V$. The routing cost of a set S of source vertices on G , denoted by $R_G(S)$, is the summation of the routing costs for all the source vertices in S ; i.e. $R_G(S) = \sum_{s \in S} \sum_{d \in V} D_G(s, d)$.
6. A minimum routing cost spanning tree of G for a set S of source vertices, denoted by $\mathcal{T}_G(S)$, is a spanning tree of G with minimum routing cost among all spanning trees of G .
7. Graph $G = (V, E, w)$ is a *metric graph* if and only if G is a complete graph in which $D_G(p, q) \geq 0$ and $D_G(p, q) + D_G(q, r) \geq D_G(p, r)$ for all $p \neq q \neq r \in V$.

3 Balancing MSTs and k -MRCTs on Metric Graphs

In this section, we present a method to construct a spanning tree that balances the building cost and the k -source routing cost for a given metric graph with k source vertices. Recall that algorithm Find-Last in [4] can find out a spanning tree that balances the building cost and the single-source-all-destination shortest path for a given graph with a source vertex. We use Algorithm Find-Last(G, s, α, T_M, T_S) to denote the algorithm proposed in [4] by Khuller, and the input and output of this algorithm are described as follows:

Algorithm Find-Last(G, s, α, T_M, T_S)
Input: metric graph $G = (V, E, w)$ with a source vertex s of V , $\alpha > 1$, $\text{MST}(G)$ T_M , $\text{SPT}(G, s)$ T_S
Output: a spanning tree T of G

The following lemma states the performance about a tree produced by Find-Last.

Lemma 1 [4] Algorithm Find-Last(G, s, α, T_M, T_S) finds out a spanning tree T of G in $O(n)$ time, where $n = |V|$, such that
(1) $C_T \leq (1 + \frac{2}{\alpha-1}) C_{\text{MST}(G)}$, and
(2) $D_T(s, d) \leq \alpha D_G(s, d)$ for all $d \in V$.

Algorithm 1 described in Figure 1 is to find out a spanning tree for balancing MST and k -MRCT.

Problem	Time	Source	Remark
Balance($1 + \frac{2}{\alpha-1}, \alpha, 1$)	$O(n)$	[4]	metric graphs
Balance($f(i) + \frac{2}{\alpha-1}, \frac{i+3}{i+1}\alpha, n$)	$O(n^{i+1})$	[9]	metric graphs
Balance($1 + \frac{2}{\alpha-1}, \alpha(1 + \frac{2(k-1)(n-2)}{k(n+k-2)}), k$)	$O(kn)$	This paper	metric graphs

Table 1: The performance ratios and running times of related results for the balancing MST and k -MRC_T problem, where $\alpha > 1$, $n = |V|$, $f(1) = 1$, $f(2) = 2$ and $f(i) = 3$ for $i > 3$.

First of all, the algorithm finds a spanning tree T_i for each source vertex s_i by Find-Last, and then output the tree with minimum routing cost. Since algorithm Find-Last needs $O(n)$ time, the total time of Algorithm 1 is $O(kn)$ time, where $n = |V|$ and $k = |S|$. The following lemma comes from Lemma 1 and Algorithm 1 immediately:

Lemma 2 If T is a tree found by Algorithm 1, then we have $C_T \leq (1 + \frac{2}{\alpha-1})C_{\text{MST}(G)}$.

Lemma 3 If T is a tree found by Algorithm 1, then we have

$$R_T(S) \leq \alpha(1 + \frac{2(k-1)(n-2)}{k(n+k-2)})R_{T_G(S)}(S)$$

where $n = |V|$ and $k = |S|$.

Proof:

According to Algorithm 1, T is one of T_i with the minimum routing cost for S . Then we have

$$R_T(S) \leq \frac{1}{k} \sum_{i=1}^k R_{T_i}(S) \quad (1)$$

Next we shall find an upper bound for $\sum_{i=1}^k R_{T_i}(S)$. First of all, we find an upper bound for $R_{T_i}(S)$. Let s and d be two vertices in V . Then we have the following inequality:

$$\begin{aligned}
R_{T_i}(S) &= \sum_{s \in S} \sum_{d \in V} D_{T_i}(s, d) \\
&\leq \sum_{s \in S} \sum_{d \in V} (D_{T_i}(s, s_i) + D_{T_i}(s_i, d)) \\
&\quad (\text{By triangle inequality}) \\
&\leq \sum_{s \in S} \sum_{d \in V} (\alpha D_G(s, s_i) + \alpha D_G(s_i, d)) \\
&\quad (\text{By Lemma 1}) \\
&= \alpha \sum_{s \in S} \sum_{d \in V-S} (D_G(s, s_i) + D_G(s_i, d)) + \\
&\quad \alpha \sum_{s \in S} \sum_{d \in S} (D_G(s, s_i) + D_G(s_i, d))
\end{aligned}$$

$$\begin{aligned}
&= (n-k)\alpha \sum_{d \in S} D_G(s_i, d) + k\alpha \sum_{d \in V-S} D_G(s_i, d) \\
&\quad + (k-1)\alpha \sum_{d \in S} D_G(s_i, d) \\
&\quad + (k-1)\alpha \sum_{d \in S} D_G(s_i, d) \\
&= k\alpha \sum_{d \in V-S} D_G(s_i, d) \\
&\quad + \alpha(n+k-2) \sum_{d \in S} D_G(s_i, d) \quad (2)
\end{aligned}$$

By the above inequality (2), we have

$$\begin{aligned}
&\sum_{i=1}^k R_{T_i}(S) \\
&\leq k\alpha \sum_{s \in S} \sum_{d \in V-S} D_G(s, d) + \\
&\quad \alpha(n+k-2) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
&= k\alpha R_G(S) \\
&\quad + \alpha(n-2) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \quad (3)
\end{aligned}$$

Let $d_1, d_2, \dots, d_{n-k}$ be the vertices in $V-S$, then we have the following two inequalities:

$$\begin{aligned}
&(n-k) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
&= (n-k) \sum_{i=1}^k \sum_{j=1}^k D_G(s_i, s_j) \\
&= \sum_{i=1}^k \sum_{j=1}^{k-1} \sum_{m=1}^{n-k} D_G(s_i, s_{(i+j) \bmod k}) \\
&\leq \sum_{i=1}^k \sum_{j=1}^{k-1} \sum_{m=1}^{n-k} D_G(s_i, d_m) \\
&\quad + \sum_{i=1}^k \sum_{j=1}^{k-1} \sum_{m=1}^{n-k} D_G(s_{(i+j) \bmod k}, d_m) \\
&= \sum_{i=1}^k \sum_{j=1}^{k-1} \sum_{m=1}^{n-k} D_G(s_i, d_m)
\end{aligned}$$

Algorithm 1

Input: Metric graph $G = (V, E, w)$ with a set $S = \{s_1, s_2, \dots, s_k\} \subseteq V$ of source vertices, $\alpha > 1$, $\text{MST}(G)$, T_M , $\text{SPT}(G, s_i)$, $T_S^{s_i}$ for $1 \leq i \leq k$

Output: A spanning tree T of G

1. // use each s_k as source vertex respectively to run Find-Last
for $(i = 1 \text{ to } k)$
 $T_i = \text{Find-Last}(G, s_i, \alpha, T_M, T_S^{s_i});$
2. // output the spanning tree with smallest k -source routing cost
 $c = R_{T_1}(S)$, $j = 1$;
for $(i = 2 \text{ to } k)$
if $(c > R_{T_i}(S))$
 $c = R_{T_i}(S)$, $j = i$;
output T_j ;

Figure 1:

$$\begin{aligned}
& + \sum_{i=1}^k \sum_{j=1}^{k-1} \sum_{m=1}^{n-k} D_G(s_i, d_m) \\
& = 2(k-1) \sum_{i=1}^k \sum_{m=1}^{n-k} D_G(s_i, d_m) \\
& = 2(k-1) \sum_{s \in S} \sum_{d \in V-S} D_G(s, d) \quad (4)
\end{aligned}$$

$$\begin{aligned}
& (n+k-2) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
& = 2(k-1) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
& \quad + (n-k) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
& \leq 2(k-1) \sum_{s \in S} \sum_{d \in S} D_G(s, d) \\
& \quad + 2(k-1) \sum_{s \in S} \sum_{d \in V-S} D_G(s, d) \\
& \quad \text{(By Inequality (4))} \\
& = 2(k-1) \sum_{s \in S} \sum_{d \in V} D_G(s, d) \\
& = 2(k-1) R_G(S) \quad (5)
\end{aligned}$$

Based upon Inequality (5), we can rewrite inequality (3) as follows:

$$\begin{aligned}
& \sum_{i=1}^k R_{T_i}(S) \\
& \leq k\alpha R_G(S) \\
& \quad + \alpha \left(\frac{2(k-1)(n-2)}{n+k-2} \right) R_G(S)
\end{aligned}$$

By the above inequality and $R_G(S) \leq R_{T_G}(S)$,

Inequality (1) can be rewritten as follows:

$$\begin{aligned}
R_T(S) & \leq \frac{1}{k} \sum_{i=1}^k R_{T_i}(S) \\
& \leq \alpha \left(1 + \frac{2(k-1)(n-2)}{k(n+k-2)} \right) R_{T_G(S)}(S)
\end{aligned}$$

□

By Lemma 2 and Lemma 3, the main theorem of this section is as follows:

Theorem 1 Given a metric graph $G = (V, E, w)$ with a set of source vertices $S = \{s_1, s_2, \dots, s_k\} \in V$, an $\alpha > 1$ and a MST of G , $\text{SPT}(G, s_i)$ for $1 \leq i \leq k$, a spanning tree T of G can be found in $O(kn)$ time, where $n = |V|$ and $k = |S|$, such that

$$\begin{aligned}
(1) \quad C_T & \leq \left(1 + \frac{2}{\alpha-1} \right) C_{\text{MST}(G)} \\
(2) \quad R_T(S) & \leq \alpha \left(1 + \frac{2(k-1)(n-2)}{k(n+k-2)} \right) R_{T_G(S)}(S)
\end{aligned}$$

4 Conclusion

We have propose an to construct a spanning tree T for a metric graph $G = (V, E, w)$ with and a source vertex set $S \subseteq V$, where V , E and w denote the vertex set, the edge set and the nonnegative weight function on edges respectively, such that the building cost of T is less than $1 + 2/(\alpha - 1)$ times of that of a MST of G , and the k -source routing cost of T is less than $\alpha(1 + \frac{2(k-1)(n-2)}{k(n+k-2)})$ times of that of a k -MRCT of G with respect to S , where $k = |S|$, $n = |V|$ and $\alpha > 1$. The optimal algorithm for such problem on metric graphs is still open now.

References

- [1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [2] R. L. Graham and Pavol Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57, 1985.
- [3] D. S. Johnson, J. K. Lenstra, and A. H. G. Rinnooy Kan. The complexity of the network design problem. *Networks*, 8:279–285, 1978.
- [4] S. Khuller, B. Raghavachari, and N. Young. Balancing minimum spanning trees and shortest-path trees. *Algorithmica*, 14:305–321, 1995.
- [5] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7:48–50, 1956.
- [6] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36:1389–1401, 1957.
- [7] B. Y. Wu. A polynomial time approximation scheme for the two-source minimum routing cost spanning trees. *Journal of Algorithms*, 44(2):359–378, 2002.
- [8] B. Y. Wu, K. M. Chao, and C. Y. Tang. Approximation algorithms for some optimum communication spanning tree problems. *Discrete Applied Mathematics*, 102:245–266, 2000.
- [9] B. Y. Wu, K. M. Chao, and C. Y. Tang. Light graphs with small routing cost. *Networks*, 39(3):130–138, 2002.
- [10] B. Y. Wu, G. Lancia, V. Bafna, K. M. Chao, R. Ravi, and C. Y. Tang. A polynomial time approximation scheme for minimum routing cost spanning trees. *SIAM Journal on Computing*, 29(3):761–778, 1999.

Mutually Independent Hamiltonian Paths in Star Networks

Cheng-Kuan Lin^a, Hua-Min Huang^a, Lih-Hsing Hsu^{b,*}, and Sheng Bau^{c,†}

^aDepartment of Mathematics

National Central University, Chung-Li, Taiwan

^bDepartment of Computer Science

National Chiao Tung University, Hsinchu, Taiwan

^cSchool of Mathematics, University of Natal

Pietermaritzburg, South Africa

Abstract

Let $P_1 = \langle v_1, v_2, \dots, v_k \rangle$ and $P_2 = \langle u_1, u_2, \dots, u_k \rangle$ be any two hamiltonian paths of G . We say that P_1 and P_2 are *independent* if $u_1 = v_1$, $v_k = u_k$, and $v_i \neq u_i$ for $1 < i < k$. We say a set of hamiltonian paths $P_1, P_2, \dots, P_s$ of G are *mutually independent* if any two different hamiltonian paths are independent. We prove that there exist $(n - 2)$ mutually independent hamiltonian paths in S_n between any two nodes from different bipartite sets if $n \geq 4$. Moreover, this result is optimal.

Keywords: hamiltonian, hamiltonian laceable.

1 Introduction

For definitions and notations we follow [2]. $G = (V, E)$ is a *graph* if V is a finite set and E is a subset of $\{(u, v) \mid (u, v) \text{ is an unordered pair of } V\}$. We say that V is the *node set* and E is the *edge set*. For a node u , $N(u)$ denotes the *neighborhood* of u which is the set $\{v \mid (u, v) \in E\}$. For any node x of V , denote the *degree* of x by $\deg_G(x) = |N(x)|$. Two nodes u and v are *adjacent* if $(u, v) \in E$. A *path* is represented by $\langle v_1, v_2, \dots, v_k \rangle$ where $v_i \neq v_j$ for $i, j \in \{1, 2, \dots, k\}$ and $i \neq j$ and $(v_i, v_{i+1}) \in E$. Denote by $Q(i)$ the i -th node v_i of path $Q = \langle v_1, v_2, \dots, v_k \rangle$. We also write the path $\langle v_1, v_2, \dots, v_k \rangle$ as $\langle v_1, Q_1, v_i, v_{i+1}, \dots, v_j, Q_2, v_t, \dots, v_k \rangle$, where Q_1 is the path $\langle v_1, v_2, \dots, v_i \rangle$ and Q_2 is the path

*Corresponding author: lhhsu@cc.nctu.edu.tw

[†]The work of this author was supported in parts by the University of Natal URF fund and the National Chiao Tung University of Taiwan. The great hospitality of Professor Lih-Hsing Hsu and the Department of Computer Science and Department of Applied Mathematics of National Chiao Tung University is gratefully acknowledged.

$\langle v_j, v_{j+1}, \dots, v_t \rangle$. We use $d(u, v)$ to denote the *distance* between u and v , i.e., the length of the shortest path joining u and v .

A path P is a *hamiltonian* path if $V(P) = V(G)$. Let $P_1 = \langle v_1, v_2, \dots, v_k \rangle$ and $P_2 = \langle u_1, u_2, \dots, u_k \rangle$ be two paths of G . We say that P_1 and P_2 are *independent* if $u_1 = v_1$, $v_k = u_k$, and $v_i \neq u_i$ for $1 < i < k$. We say a set of hamiltonian paths $P_1, P_2, \dots, P_s$ of G are *mutually independent* if any two different paths in the set are independent.

An interconnection network connects the processors of the parallel computer. Its architecture can be represented as a graph in which the nodes corresponds to processors and the edges to connections. Hence we use graphs and networks interchangeably. There are many mutually conflicting requirements in designing the topology for computer networks. The n -cube is one of the most popular topologies [7]. The star network S_n was proposed in [1] as "an attractive alternative to the n -cube" topology for interconnecting processors in parallel computers.

Since its introduction, the network received considerable attention. Some interesting properties of star networks are studied in [3, 4, 5, 6, 8]. For example, it is known that the star network is a bipartite graph and there is a hamiltonian path joining nodes from different partite sets. In particular, Fragopoulou and Akl [4, 5] study the embedding of $(n - 1)$ directed edge-disjoint spanning trees on the star network S_n . These spanning trees are used in communication algorithms for star networks. In this paper, we prove that there exist $(n - 2)$ mutually independent hamiltonian paths of S_n between any two nodes from different bipartite sets. Obviously, these mutually independent paths can be used for communication algorithms for star networks.

In the following section, we give the definition of the star networks and some of the previous work that are used in this paper. In section 3, we prove that there exist $(n - 2)$ mutually independent hamiltonian paths of S_n between any two nodes from different bipartite sets. Moreover, we prove that the value $(n - 2)$ is tight.

2 Basic properties of the star networks

Let n be a positive integer. We use $\langle n \rangle$ to denote the set $\{1, 2, \dots, n\}$. The n -dimensional star network, denoted by S_n , is a graph with the node set $V(S_n) = \{u_1 u_2 \dots u_n \mid u_i \in \langle n \rangle \text{ and } u_i \neq u_j \text{ for } i \neq j\}$. The edges are specified as follows: $u_1 u_2 \dots u_i \dots u_n$ is adjacent to $v_1 v_2 \dots v_i \dots v_n$ through an edge in dimension i with $2 \leq i \leq n$ if $v_j = u_j$ for $j \notin \{1, i\}$, $v_1 = u_i$ and $v_i = u_1$. By definition, S_n is an $(n - 1)$ -regular graph with $n!$ nodes. Moreover, it is node transitive and edge transitive.

We use bold face letters to denote nodes in S_n . Hence $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_n$ is a sequence of nodes in S_n . It is known that the connectivity of S_n is $n - 1$. We use $\mathbf{e}$ to denote the element $12 \dots n$. It is known that S_n is a bipartite graph with one partite set containing those nodes corresponding to odd permutations and the other partite set containing those nodes

corresponding to even permutations. We will use white nodes to represent nodes for even permutations and black nodes to represent nodes for odd permutations. Let $\mathbf{u} = u_1 u_2 \dots u_n$ be any node of the star network S_n . We say that u_i is the i -th coordinate of $\mathbf{u}$, denoted by $(\mathbf{u})_i$, for $1 \leq i \leq n$. By the definition of S_n , there is exactly one neighbor $\mathbf{v}$ of $\mathbf{u}$ such that $\mathbf{u}$ and $\mathbf{v}$ are adjacent through an edge in the i -th dimension, $2 \leq i \leq n$. For this reason, we use $(\mathbf{u})^i$ to denote the unique i -neighbor of $\mathbf{u}$. Obviously, $((\mathbf{u})^i)^i = \mathbf{u}$. For $1 \leq i \leq n$, let S_n^i denote the subgraph of S_n induced by those nodes $\mathbf{u}$ with $(\mathbf{u})_n = i$. Obviously, S_n can be decomposed into n subgraphs S_n^i , $1 \leq i \leq n$, and each S_n^i is isomorphic to S_{n-1} . This furnishes a recursive definition (construction) for star networks. For $1 \leq i \neq j \leq n$, we use $E^{i,j}$ to denote the set of edges between S_n^i and S_n^j .

Lemma 1 $|E^{i,j}| = (n-2)!$ for any $1 \leq i \neq j \leq n$. Moreover, there are $\frac{(n-2)!}{2}$ edges joining black nodes of S_n^i to white nodes of S_n^j .

Proof. Let $i \neq j$. For each of $(n-2)!$ permutations of $\langle n \rangle - \{i, j\}$ there is exactly one transposition that represents an edge between a node of S_n^i and a fixed node of S_n^j . Hence $|E^{i,j}| = (n-2)!$. Exactly half of these correspond to transpositions of an odd permutation in S_n^i to an even permutation in S_n^j . $\square$

Lemma 2 Let $\mathbf{u}$ and $\mathbf{v}$ be two distinct nodes of S_n with $(\mathbf{u})_n = (\mathbf{v})_n$ such that $1 \leq d(\mathbf{u}, \mathbf{v}) \leq 2$. Then $((\mathbf{u})^n)_n \neq ((\mathbf{v})^n)_n$.

Theorem 1 [8] S_n is hamiltonian laceable if and only if $n \neq 3$.

Theorem 2 Assume that $n \geq 5$. Let $\{a_1, a_2, \dots, a_r\}$ be a subset of $\langle n \rangle$ for some $r \leq n$. For any white node $\mathbf{u}$ of $S_n^{a_1}$ and any black node $\mathbf{v}$ of $S_n^{a_r}$, there exists a path P as $\langle \mathbf{x}_1, P_1, \mathbf{y}_1, \mathbf{x}_2, P_2, \mathbf{y}_2, \dots, \mathbf{x}_r, P_r, \mathbf{y}_r \rangle$ joining $\mathbf{u}$ and $\mathbf{v}$ such that $\mathbf{x}_1 = \mathbf{u}$, $\mathbf{y}_r = \mathbf{v}$, and P_i is a hamiltonian path of $S_n^{a_i}$ joining $\mathbf{x}_i$ to $\mathbf{y}_i$ for $1 \leq i \leq r$. Moreover, P is a hamiltonian path of the subgraph induced by $\cup_{i=1}^r S_n^{a_i}$ joining $\mathbf{u}$ to $\mathbf{v}$.

Proof. We set that $\mathbf{x}_1$ as $\mathbf{u}$ and set that $\mathbf{y}_r$ as $\mathbf{v}$. Since $S_n^{a_1}$ is isomorphic to S_{n-1} , by Theorem 1, this theorem holds for $r = 1$. Assume that $r \geq 2$. By Lemma 1, we choose $(\mathbf{y}_i, \mathbf{x}_{i+1}) \in E^{a_i, a_{i+1}}$ with $\mathbf{y}_i$ is a black node and $\mathbf{x}_{i+1}$ is a white node for $1 \leq i \leq r-1$. Obviously, $\mathbf{x}_i \neq \mathbf{y}_i$ in $S_n^{a_i}$. Since $S_n^{a_i}$ is isomorphic to S_{n-1} , by Theorem 1, there is a hamiltonian path P_i of $S_n^{a_i}$ joining $\mathbf{x}_i$ to $\mathbf{y}_i$. Obviously, $\langle \mathbf{x}_1, P_1, \mathbf{y}_1, \mathbf{x}_2, P_2, \mathbf{y}_2, \dots, \mathbf{x}_r, P_r, \mathbf{y}_r \rangle$ is the required path. $\square$

Lemma 3 Let $\mathbf{w}$ be any black node and $\mathbf{u}, \mathbf{v}$ be any two distinct white nodes of S_n with $n \geq 4$. Then there exists a hamiltonian path in $S_n - \{\mathbf{w}\}$ joining $\mathbf{u}$ to $\mathbf{v}$.

Proof. We prove this lemma by induction. For $n = 4$, we can set $\mathbf{w} = 1234$ because S_4 is node transitive. The corresponding hamiltonian path between $\mathbf{u}$ and $\mathbf{v}$ in $S_4 - \{\mathbf{w}\}$ are listed below.

Then $\langle \mathbf{u}, P, \mathbf{s}, (\mathbf{s})^n, Q, \mathbf{t}, (\mathbf{t})^n, R, \mathbf{v} \rangle$ forms the desired path.

Case 2. $\mathbf{v} \in S_n^{n-1}$. Let $\{a_1, a_2, \dots, a_{n-2}\} = \langle n-1 \rangle - \{n-1\}$ with $a_1 = n$. Since S_n^i is isomorphic to S_{n-1} with $i \in \{a_1, a_2, \dots, a_{n-2}\}$, by Theorem 2, there exists a hamiltonian path R of the subgraph induced by $\cup_{i=1}^{n-1} S_n^{a_i}$ joining $\mathbf{u}$ to a black node $\mathbf{s}$ with $(\mathbf{s})_n = n-1$ and $(\mathbf{s})^n \neq \mathbf{v}$. Since S_n^{n-1} is isomorphic to S_{n-1} , by induction, there exists a hamiltonian path Q of $S_n^{n-1} - \{\mathbf{w}\}$ joining the white node $(\mathbf{s})^n$ to $\mathbf{v}$. Then $\langle \mathbf{u}, R, \mathbf{s}, (\mathbf{s})^n, Q, \mathbf{v} \rangle$ forms the desired path.

Case 3. $\mathbf{v} \in S_n^n$. Note that $|N(\mathbf{v}) - (\mathbf{v})^n| \geq 3$. We can choose $\mathbf{x}$ as a neighbor of $\mathbf{v}$ with $(\mathbf{x})_n \neq n-1$. Since S_n^n is isomorphic to S_{n-1} , there exists a hamiltonian path P of $S_n^n - \{\mathbf{x}\}$ joining $\mathbf{u}$ to $\mathbf{v}$. Without loss generality, we can write P as $\langle \mathbf{u}, P', \mathbf{s}, \mathbf{u} \rangle$. Since $d(\mathbf{s}, \mathbf{x}) \leq 2$, by Lemma 2, $((\mathbf{s})^n)_n \neq ((\mathbf{x})^n)_n$. Obviously, $\mathbf{x}$ and $\mathbf{s}$ both are black nodes.

Assume that $(\mathbf{s})_n = n-1$. Let $\{a_1, a_2, \dots, a_{n-2}\} = \langle n-2 \rangle$ with $a_{n-2} = (\mathbf{x})_n$. Since S_n^{n-1} are isomorphic to S_{n-1} , by induction, there exists a hamiltonian path Q of $S_n^{n-1} - \{\mathbf{w}\}$ joining the white node $(\mathbf{s})^n$ to a white node $\mathbf{t}$ with $(\mathbf{t})_n = a_1$. By Theorem 2, there exists a hamiltonian path R of the subgraph induced by $\cup_{i=1}^r S_n^{a_i}$ joining the black node $(\mathbf{t})^n$ to the white node $(\mathbf{x})^n$. Then $\langle \mathbf{u}, P', \mathbf{s}, (\mathbf{s})^n, Q, \mathbf{t}, (\mathbf{t})^n, R, (\mathbf{x})^n, \mathbf{b}, \mathbf{v} \rangle$ forms the desired path.

Assume that $(\mathbf{s})_n \neq n-1$. Let $\{a_1, a_2, \dots, a_{n-2}\} = \langle n-2 \rangle$ with $a_{n-3} = (\mathbf{y})_n$ and $a_{n-2} = (\mathbf{s})_n$. Since $S_n^{a_{n-2}}$ is isomorphic to S_{n-1} , by Theorem 1, there exists a hamiltonian path Q of $S_n^{a_{n-2}}$ joining the white node $(\mathbf{s})^n$ to a black node $\mathbf{t}$ with $(\mathbf{t})_n = n-1$. Since S_n^{n-1} is isomorphic to S_{n-1} , by induction, there exists a hamiltonian path H of $S_n^{n-1} - \{\mathbf{w}\}$ joining the white node $(\mathbf{t})^n$ to a white node $\mathbf{z}$ with $(\mathbf{z})_n = a_1$. By Theorem 2, there exists a hamiltonian path R of the subgraph induced by $\cup_{i=1}^r S_n^{a_i}$ joining the black node $(\mathbf{z})^n$ to the white node $(\mathbf{x})^n$. Then $\langle \mathbf{u}, P', \mathbf{s}, (\mathbf{s})^n, Q, \mathbf{t}, (\mathbf{t})^n, H, \mathbf{z}, (\mathbf{z})^n, R, (\mathbf{x})^n, \mathbf{x}, \mathbf{v} \rangle$ form the desired path. $\square$

Theorem 3 Let $\mathbf{u}$ be any white node of S_n with $n \geq 4$ and $\{a_1, a_2, \dots, a_{n-1}\}$ be an $n-1$ subset of $\langle n \rangle$. Then there exist hamiltonian paths $P_1, P_2, \dots, P_{n-1}$ of S_n joining $\mathbf{u}$ to some black node $\mathbf{z}_i$ such that

- (1) $(\mathbf{z}_i)_1 = (P_i(n!))_1 = a_i$ for $1 \leq i \leq n-1$, and
- (2) $|\{P_1(i), P_2(i), \dots, P_{n-1}(i)\}| = n-1$ for $2 \leq i \leq n!$.

Proof. Since S_n is node transitive, we may assume that $\mathbf{u} = \mathbf{e}$. We prove the theorem holds on S_4 by exhibiting three required hamiltonian paths in S_4 as follows:

$\{a_1, a_2, a_3\} = \{1, 2, 3\}$
(1234)(2134)(3124)(1324)(2314)(3214)(4213)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(4312)(3412)(2413)(1423)(4123)(2143)(3142)(4132)(1432)(1234)(3214)(2314)(4312)(1342)(3142)(4132)(1432)(3412)(2413)(4213)(1243)(2143)(4123)(1423)(3421)(2431)(4231)(3241)(2341)(4321)(1324)(3124)(2134)(1234)(4231)(2431)(1432)(3412)(4312)(1342)(3142)(4132)(1432)(4132)(2134)(3124)(1324)(2314)(3214)(4213)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(4312)(3412)(2413)(1423)(4123)(2143)(3142)(4132)(1432)(1234)(2134)(3124)(1324)(2314)(3214)(4213)(2413)(1423)(4123)(2143)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(3142)(4132)(1432)(3412)(4312)
$\{a_1, a_2, a_3\} = \{1, 2, 4\}$
(1234)(4231)(2431)(3421)(4321)(2341)(3241)(1243)(2143)(3142)(1342)(4312)(3412)(1432)(4132)(2134)(3124)(4123)(1423)(2413)(4213)(3214)(2314)(1324)(1234)(3214)(4213)(1243)(3241)(4231)(2431)(1432)(3412)(2413)(1423)(3421)(4321)(2341)(1342)(4312)(2314)(1324)(3124)(4123)(2143)(3142)(4132)(2143)(1234)(2134)(3124)(1324)(2314)(3214)(4213)(2413)(1423)(4123)(2143)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(3142)(4132)(1432)(3412)(4312)
$\{a_1, a_2, a_3\} = \{2, 3, 4\}$
(1234)(3214)(4213)(1243)(3241)(4231)(2431)(1432)(3412)(2413)(1423)(3421)(4321)(2341)(1342)(4312)(2314)(1324)(3124)(4123)(2143)(3142)(4132)(2143)(1234)(4231)(2431)(1432)(3412)(4312)(1342)(3142)(4132)(2134)(3124)(1324)(2314)(3214)(4213)(2413)(1423)(4123)(2143)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(3142)(4132)(1432)(3412)(4312)

Assume that the theorem holds for S_k , $4 \leq k < n$. Without loss of generality, we suppose that $a_1 < a_2 < \dots < a_{n-1}$. We set that $a'_i = a_i$ for $1 \leq i \leq n-3$, $a'_{n-2} = a_{n-1}$, and $a'_{n-1} = a_{n-2}$. Obviously, $(\mathbf{e})^n$ is a black node in S_n^1 . We set that

$$b_i = \begin{cases} i+3 & \text{if } 1 \leq i \leq n-4 \\ i-n+4 & \text{if } n-3 \leq i \leq 2n-5 \\ i+2n+5 & \text{if } 2n-4 \leq i \leq 2n-3 \end{cases}$$

By induction hypothesis, there exist hamiltonian paths $H_1, H_2, \dots, H_{n-2}$ of S_n^n joining $\mathbf{e}$ to some black node $\mathbf{v}_i$ such that

- (1) $(\mathbf{v}_i)_1 = (H_i((n-1)!))_1 = i+3$ for $1 \leq i \leq n-4$, $(\mathbf{v}_{n-3})_1 = (H_{n-3}((n-1)!))_1 = 1$, $(\mathbf{v}_{n-2})_1 = (H_{n-2}((n-1)!))_1 = 3$, and
- (2) $|\{H_1(i), H_2(i), \dots, H_{n-2}(i)\}| = n-2$ for $2 \leq i \leq (n-1)!$.

Since $|\{\mathbf{v} \in S_n^{i+2} \mid \mathbf{v} \text{ is a black node with } (\mathbf{v})_1 = a'_i\}| = \frac{(n-2)!}{2} \geq 2$, we choose a black node $\mathbf{z}_i \in S_n^{b_i+n-2}$ with $(\mathbf{z}_i)_1 = a'_i$ for $1 \leq i \leq n-3$. Since S_n^i is isomorphic to S_{n-1} for $1 \leq i \leq n$, by Theorem 2, there exists a path W_i as $\langle \mathbf{x}_1^i, T_1^i, \mathbf{y}_1^i, \mathbf{x}_2^i, T_2^i, \mathbf{y}_2^i, \dots, \mathbf{x}_{n-1}^i, T_{n-1}^i, \mathbf{y}_{n-1}^i \rangle$ joining the white node $(\mathbf{v}_i)^n$ to the black node $\mathbf{z}_i$ such that $\mathbf{x}_1^i = (\mathbf{v}_i)^n$, $\mathbf{y}_{n-2}^i = \mathbf{z}_i$, and T_j^i is a hamiltonian path of $S_n^{b_i+j-1}$ joining $\mathbf{x}_j^i$ to $\mathbf{y}_j^i$ for $1 \leq j \leq n-1$, and $1 \leq i \leq n-2$. Again, there exists a path W_{n-2} as $\langle \mathbf{x}_1^{n-2}, T_1^{n-2}, \mathbf{y}_1^{n-2}, \mathbf{x}_2^{n-2}, T_2^{n-2}, \mathbf{y}_2^{n-2}, \dots, \mathbf{x}_{n-1}^{n-2}, T_{n-1}^{n-2}, \mathbf{y}_{n-1}^{n-2} \rangle$ joining the white node $(\mathbf{v}_{n-2})^n$ to the black node $\mathbf{z}_{n-2}$ such that $\mathbf{x}_1^{n-2} = (\mathbf{v}_{n-2})^n$, $\mathbf{y}_{n-1}^{n-2} = \mathbf{z}_{n-2}$, and T_j^{n-2} is a hamiltonian path of $S_n^{b_i+n-2}$ joining $\mathbf{x}_j^{n-2}$ to $\mathbf{y}_j^{n-2}$ for $1 \leq j \leq n-1$. We set that P_i as $\langle \mathbf{e}, H_i, \mathbf{v}_i, (\mathbf{v}_i)^n, W_i, \mathbf{z}_i \rangle$ for $1 \leq i \leq n-2$.

By Theorem 2, there exists a path W as $\langle \mathbf{x}_1, Q_1, \mathbf{y}_1, \mathbf{x}_2, Q_2, \mathbf{y}_2, \dots, \mathbf{x}_{n-1}, Q_{n-1}, \mathbf{y}_{n-1} \rangle$ joining the black node $(\mathbf{e})^n$ to the white node $((\mathbf{e})^{n-1})^n$ such that $\mathbf{x}_1 = (\mathbf{e})^n$, $\mathbf{y}_{n-1} = ((\mathbf{e})^{n-1})^n$, and Q_i is a hamiltonian path of S_n^i joining $\mathbf{x}_i$ to $\mathbf{y}_i$ for $1 \leq i \leq n-1$. By Lemma 1, $|\{\mathbf{v} \in S_n^n \mid \mathbf{v} \text{ is a black node with } (\mathbf{v})_1 = a'_{n-1}\}| = \frac{(n-2)!}{2} \geq 2$, we choose a black node $\mathbf{z}_{n-1} \in S_n^1$ with $(\mathbf{z}_{n-1})_1 = a'_{n-1}$ and $\mathbf{z}_{n-1} \neq (\mathbf{e})^{n-1}$. Since S_n^n is isomorphic to S_{n-1} , by Lemma 3, there exists a hamiltonian path R of $S_n^n - \{\mathbf{e}\}$ joining the black node $(\mathbf{e})^{n-1}$ to $\mathbf{z}_{n-1}$. We set that P_{n-2} as $\langle \mathbf{e}, W, ((\mathbf{e})^{n-1})^n, (\mathbf{e})^{n-1}, R, \mathbf{z}_{n-1} \rangle$.

Then $\{P_1, P_2, \dots, P_{n-1}\}$ forms the desired paths.

We illustrate the proof of Theorem 3 with $n = 5$ as follows: Obviously, $a_1 \neq 3$, $a_2 \neq 4$, $a_3 \neq 5$, and $a_4 \neq 2$. By induction hypothesis, there exist hamiltonian paths H_1, H_2, H_3 of S_5^5 joining $\mathbf{e}$ to some black node $\mathbf{v}_i$ such that

- (1) $(\mathbf{v}_1)_1 = (H_1(24))_1 = 4$, $(\mathbf{v}_2)_1 = (H_2(24))_1 = 1$, $(\mathbf{v}_3)_1 = (H_3(24))_1 = 3$, and
- (2) $|\{H_1(i), H_2(i), H_3(i)\}| = 3$ for $2 \leq i \leq 24$.

Then we choose a black node $\mathbf{z}_1$ in S_5^3 with $(\mathbf{z}_1)_1 = a_1$, a black node $\mathbf{z}_2$ in S_5^4 with $(\mathbf{z}_2)_1 = a_2$, a black node $\mathbf{z}_3$ in S_5^2 with $(\mathbf{z}_3)_1 = a_4$, and a black node $\mathbf{z}_4$ in S_5^2 with $(\mathbf{z}_4)_1 = a_3$. By Theorem 2, there exists a path W_1 as $\langle \mathbf{x}_1^1, T_1^1, \mathbf{y}_1^1, \mathbf{x}_2^1, T_2^1, \mathbf{y}_2^1, \mathbf{x}_3^1, T_3^1, \mathbf{y}_3^1, \mathbf{x}_4^1, T_4^1, \mathbf{y}_4^1 \rangle$ joining

the white node $(\mathbf{v}_1)^5$ to the black node $\mathbf{z}_1$ such that $\mathbf{x}_1^1 = (\mathbf{v}_1)^5$, $\mathbf{y}_4^1 = \mathbf{z}_1$, and T_j^1 is a hamiltonian path of $S_n^{b_j}$ joining $\mathbf{x}_j^1$ to $\mathbf{y}_j^1$ for $1 \leq j \leq 4$, and a path W_2 . Furthermore, a path W_3 .

By Theorem 2, there exists a path W as $\langle \mathbf{x}_1, Q_1, \mathbf{y}_1, \mathbf{x}_2, Q_2, \mathbf{y}_2, \mathbf{x}_3, Q_3, \mathbf{y}_3, \mathbf{x}_4, Q_4, \mathbf{y}_4 \rangle$ joining the black node $(\mathbf{e})^5$ to the white node $((\mathbf{e})^4)^5$ such that $\mathbf{x}_1 = (\mathbf{e})^5$, $\mathbf{y}_4 = ((\mathbf{e})^4)^5$, and Q_i is a hamiltonian path of S_5^i joining $\mathbf{x}_i$ to $\mathbf{y}_i$ for $1 \leq i \leq 4$. Since S_5^5 is isomorphic to S_4 , by Lemma 3, there exists a hamiltonian path R of $S_5^5 - \{\mathbf{e}\}$ joining the black node $(\mathbf{e})^4$ to $\mathbf{z}_4$. We set that P_3 as $\langle \mathbf{e}, W, ((\mathbf{e})^4)^5, (\mathbf{e})^4, R, \mathbf{z}_4 \rangle$.

Thus $\{P_1, P_2, P_3, P_4\}$ forms the desired paths. See Figure 1 for illustration. $\square$

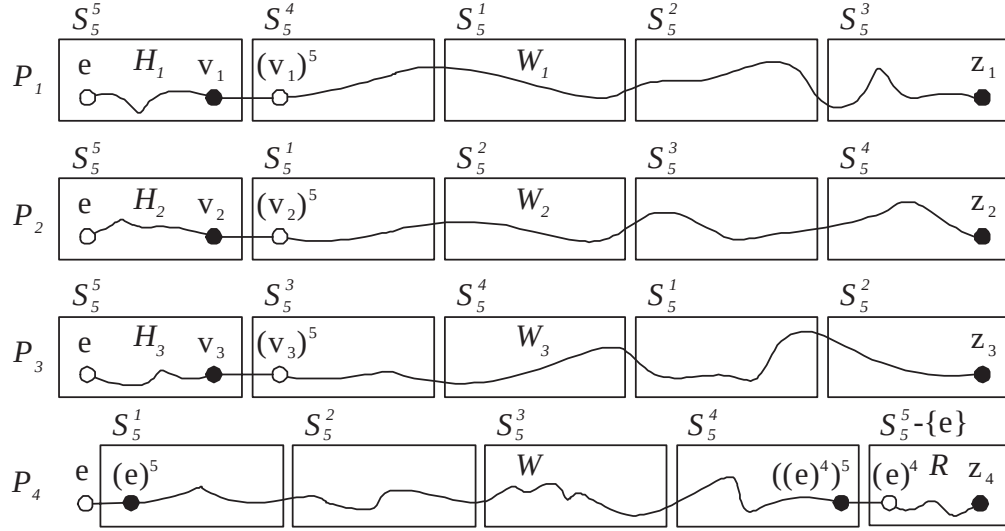


Figure 1: Illustration for Theorem 3 on S_5 .

3 Mutually independent hamiltonian paths

Theorem 4 Assume that $n \geq 4$. Then there exist $(n-2)$ mutually independent hamiltonian paths of S_n between any two nodes $\mathbf{u}$ and $\mathbf{v}$ from different bipartite sets. Moreover the value $(n-2)$ is tight.

Proof. Since star networks are node transitive, we may assume that $\mathbf{u} = \mathbf{e}$. We prove this theorem by induction. For $n \geq 4$, the required hamiltonian paths are list below:

$\mathbf{v} = (2134)$ (1234)(4231)(2431)(1432)(4132)(3142)(2143)(1243)(3241)(2341)(1342)(4312)(3412)(2413)(4213)(3214)(2314)(1324)(4321)(3421)(1423)(4123)(3124)(2134) (1234)(3214)(4213)(2413)(3412)(4312)(2314)(1324)(3124)(4123)(1423)(3421)(4321)(2341)(1342)(3142)(2143)(1243)(3241)(4231)(2431)(1432)(4132)(2134)
$\mathbf{v} = (3214)$ (1234)(2134)(3124)(1324)(2314)(4312)(3412)(1432)(4132)(3142)(1342)(2341)(4321)(3421)(2431)(4231)(3241)(1243)(2143)(4123)(1423)(2413)(4213)(3214) (1234)(4231)(2431)(3421)(4321)(2341)(3241)(1243)(4213)(2413)(1423)(4123)(2143)(3142)(1342)(4312)(3412)(1432)(4132)(2134)(3124)(1324)(2314)(3214)
$\mathbf{v} = (4231)$ (1234)(2134)(4132)(1432)(2431)(3421)(4321)(1324)(3124)(4123)(1423)(2413)(3412)(4312)(2314)(3214)(4213)(1243)(2143)(3142)(1342)(2341)(3241)(4231) (1234)(3214)(2314)(4312)(3412)(2413)(4213)(1243)(3241)(2341)(1342)(3142)(2143)(4123)(1423)(3421)(4321)(1324)(3124)(2134)(4132)(1432)(2431)(4231)
$\mathbf{v} = (1432)$ (1234)(2134)(4132)(3142)(1342)(2341)(4321)(1324)(3124)(4123)(2143)(1243)(3241)(4231)(2431)(3421)(1423)(2413)(4213)(3214)(2314)(4312)(3412)(1432) (1234)(3214)(2314)(4312)(3412)(2413)(4213)(1243)(3241)(4231)(2431)(3421)(1423)(4123)(2143)(3142)(1342)(4312)(3412)(1432)(4132)(2134)(3124)(1324)(2314)(3214)
$\mathbf{v} = (1243)$ (1234)(4231)(3241)(2341)(1342)(4312)(3412)(1432)(2431)(3421)(4321)(1324)(2314)(3214)(4213)(2413)(1423)(4123)(3124)(2134)(4132)(3142)(2143)(1243) (1234)(2134)(4132)(3142)(2143)(4123)(3124)(1324)(2314)(3214)(4213)(2413)(1423)(3421)(4321)(2341)(1342)(4312)(3412)(1432)(2431)(4231)(3241)(1243)
$\mathbf{v} = (1324)$ (1234)(2134)(3124)(4123)(1423)(2413)(3412)(1432)(4132)(3142)(2143)(1243)(4213)(3214)(2314)(4312)(1342)(2341)(3241)(4231)(2431)(3421)(4321)(1324) (1234)(4231)(2431)(3421)(4321)(2341)(3241)(1243)(4213)(3214)(2314)(4312)(1342)(3142)(2143)(4123)(1423)(2413)(3412)(1432)(4132)(2134)(3124)(1324)
$\mathbf{v} = (2341)$ (1234)(2134)(4132)(3142)(1342)(4312)(2314)(3214)(4213)(1243)(2143)(4123)(3124)(1324)(4321)(3421)(1423)(2413)(3412)(1432)(2431)(4231)(3241)(2341) (1234)(3214)(4213)(1243)(3241)(4231)(2431)(3421)(1423)(2413)(3412)(1432)(4132)(2134)(3124)(4123)(2143)(3142)(1342)(4312)(2314)(1324)(4321)(2341)
$\mathbf{v} = (3142)$ (1234)(3214)(2314)(4312)(1342)(2341)(3241)(4231)(2431)(1432)(3412)(2413)(4213)(1243)(2143)(4123)(1423)(3421)(4321)(1324)(3124)(2134)(4132)(3142) (1234)(4231)(2431)(1432)(4132)(2134)(3124)(4123)(1423)(3421)(4321)(1324)(2314)(4312)(2413)(3412)(4312)(1342)(2341)(3241)(1243)(2143)(3142)
$\mathbf{v} = (4123)$ (1234)(4231)(2431)(3421)(1423)(2413)(4213)(3214)(2314)(1324)(4321)(2341)(3241)(1243)(2143)(3142)(1342)(4312)(3412)(1432)(4132)(2134)(3124)(4123) (1234)(3214)(2314)(1324)(3124)(4132)(3412)(1342)(4312)(4321)(2341)(4231)(3241)(4321)(4231)(3241)(2341)(4321)(4231)(1423)(2413)(4213)(1243)(2143)(4123)
$\mathbf{v} = (4312)$ (1234)(4231)(3241)(2341)(1342)(3142)(4132)(2134)(3124)(1324)(4321)(3421)(2431)(1432)(3412)(2413)(1423)(4123)(2143)(1243)(4213)(3214)(2314)(4312) (1234)(2134)(3124)(1324)(2314)(3214)(4213)(2413)(1423)(4123)(2143)(1243)(3241)(4231)(2431)(3421)(4321)(2341)(1342)(3142)(4132)(1432)(3412)(4312)
$\mathbf{v} = (2413)$ (1234)(2134)(3124)(4123)(1423)(3421)(2431)(4231)(3241)(1243)(2143)(3142)(4132)(1432)(3412)(4312)(1342)(2341)(4321)(1324)(2314)(3214)(4213)(2413) (1234)(4231)(3241)(1243)(4213)(3214)(2314)(4312)(1342)(2341)(4321)(1324)(3124)(4132)(3142)(2143)(4123)(1423)(3421)(4321)(2431)(1432)(3412)(2413)
$\mathbf{v} = (3421)$ (1234)(3214)(4213)(2413)(1423)(4123)(3124)(2134)(4132)(1432)(3412)(4312)(2314)(1324)(4321)(2341)(1342)(3142)(2143)(1243)(3241)(4231)(2431)(3421) (1234)(2134)(4132)(1432)(2431)(4231)(3241)(2341)(1342)(3142)(2143)(1243)(4213)(3214)(2314)(4312)(3412)(2413)(1423)(4123)(3124)(1324)(4321)(3421)

Assume that the theorem holds for S_k , $4 \leq k < n$. Since S_n is node transitive and edge transitive, we assume that $\mathbf{u} = \mathbf{e} \in S_n^n$ and $\mathbf{v} \in S_n^{n-1}$. We set $b_i = i$ for $1 \leq i \leq n-2$ and $b_i = i - n + 2$ for $n-1 \leq i \leq 2n-5$. By Theorem 3, there exist hamiltonian paths $Q_1, Q_2, \dots, Q_{n-2}$ of S_n^n joining $\mathbf{e}$ and some black node $\mathbf{s}_i$ such that (1) $(\mathbf{s}_i)_1 = b_i$ for $1 \leq i \leq n-2$, and (2) $|\{Q_1(i), Q_2(i), \dots, Q_{n-2}(i)\}| = n-2$ for $2 \leq i \leq (n-1)!$. Again, there exist hamiltonian paths $R_1, R_2, \dots, R_{n-2}$ of S_n^{n-1} joining $\mathbf{v}$ and some white node $\mathbf{y}_i$ such that (1) $(\mathbf{t}_i)_1 = b_{i+n-2}$ for $1 \leq i \leq n-2$, and (2) $|\{R_1(i), R_2(i), \dots, R_{n-2}(i)\}| = n-2$ for $2 \leq i \leq (n-1)!$.

By Theorem 2, there exists a path W_i as $\langle \mathbf{x}_1^i, T_1^i, \mathbf{y}_1^i, \mathbf{x}_2^i, T_2^i, \mathbf{y}_2^i, \dots, \mathbf{x}_{n-2}^i, T_{n-2}^i, \mathbf{y}_{n-2}^i \rangle$ joining the white node $\mathbf{x}_1^i$ to the black node $\mathbf{y}_{n-2}^i$ such that $\mathbf{x}_1^i = (\mathbf{s}_i)^n$, $\mathbf{y}_{n-2}^i = (\mathbf{t}_i)^n$, and T_j^i is a hamiltonian path of $S_n^{b_{i+j-1}}$ joining $\mathbf{x}_j^i$ to $\mathbf{y}_j^i$ for $1 \leq j \leq n-1$, and $1 \leq i \leq n-2$. Hence $\{P_1, P_2, \dots, P_{n-2}\}$ form the desired paths. See Figure 2 for illustration the case $n = 5$.

Suppose that $\mathbf{u} = \mathbf{e}$ and $\mathbf{v} = (\mathbf{e})^n$. Let $P_1, P_2, \dots, P_s$ be a set of mutually independent hamiltonian path between $\mathbf{u}$ and $\mathbf{v}$. Obviously, $\{P_i(2) | 1 \leq i \leq s\} \subseteq \{(\mathbf{u})^i | 2 \leq i \leq n\}$. Thus $s \leq n-2$. Thus, the value $(n-2)$ is tight. $\square$

References

- [1] S. B. Akers and B. Krishnamurthy, "A group-theoretic model for symmetric interconnection networks," *IEEE Trans. Comput.*, Vol. 38, pp. 555–566, 1989.
- [2] J. A. Bondy and U. S. R. Murty, *Graph Theory with Applications*, North Holland, New York, (1980).

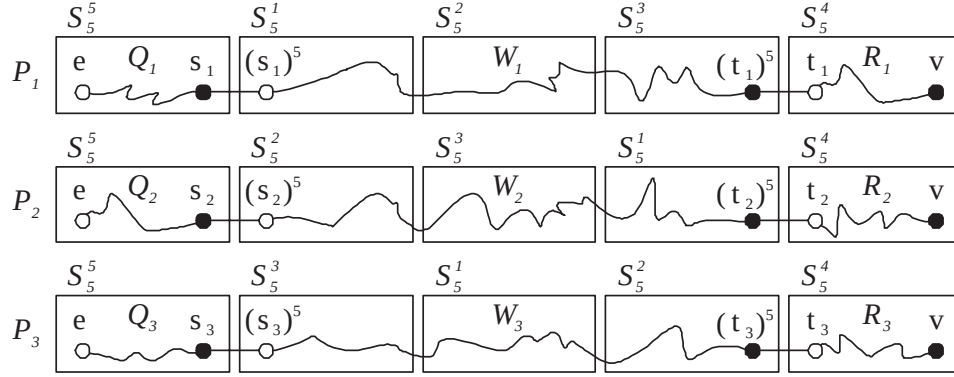


Figure 2: Illustration for 3 mutually independent hamiltonian paths on S_5 .

- [3] A. Bouabdallah, M. C. Heydemann, J. Opatmy, and D. Sotteau, “Embedding complete binary trees into star and pancake graphs”, *Theory Comput. Syst.*, pp. 279–305, 1998.
- [4] P. Fragopoulou and S.G. Akl, “Optimal communication algorithms on the star graphs using spanning tree constructions”, *Journal of Parallel and Distributed Computing*, Vol. 23, 55–71, 1995.
- [5] P. Fragopoulou and S.G. Akl, “Edge-disjoint spanning trees on the star networks with applications to fault tolerance”, *IEEE Transactions on Computers*, Vol. 45, 174–185, 1996.
- [6] L. Gargano, U. Vaccaro, and A. Vozella, “Fault tolerant routing in the star and pancake interconnection networks,” *Information Processing Letters*, pp. 315–320, 1993.
- [7] F.T. Leighton, *Introduction to Parallel Algorithms and Architectures: Arrays · Trees · Hypercubes*, Morgan Kaufmann, San Mateo, CA 1992.
- [8] S.Y. Hsieh, G.H. Chen, and C.W. Ho, “Hamiltonian-laceability of star graphs”, *Networks*, vol. 36, no. 4, pp. 225-232, 2000.

Use Knowledge Bases and HMM Approach to Predict Protein Secondary Structure

Wen-Lung Shu, Morris Chou, Johnson Chen

Computer Science & Information Engineering Department,

Chung-Hua University, Hsin-Chu, Taiwan.

wlshu@chu.edu.tw

ABSTRACT

In this paper, knowledge bases and HMM approach are used to predict protein secondary structure. This system contains the databases of primary, secondary and tertiary structures of proteins. PSI-BLAST is used to find all the domain profiles of proteins in EC2. Then these similar domains are grouped and built into domain profile databases.

Next, the Hidden Markov Model's (HMM) parameters can be trained by using the data of the proteins with recognized structure. The 2D structure of the query protein's domain can be identified, if it's matched homology profile in the databases is found. Then the 2D structures of remaining fragments can be predicted by using HMM approach. The accuracy of this method is above 80% for enzyme protein.

1 INTRODUCTION

Functional characterization of a protein sequence is the most frequent problem in biology. This task is usually facilitated with accurate three-dimensional (3D) structure of the studied protein. The structure of protein gives important information, such as the location of probable function (or interaction sites), identification of distantly related proteins, and important regions of the protein

sequence that are involved in maintaining the structure. However, it is difficult and time consuming to experimentally determine protein structure. For these reasons, protein structure predicting methods [1,2] have received a lot of attention.

Predicting the secondary structure of a protein from its sequence is a fundamental step in predicting the overall quaternary structure of a protein. Secondary structure prediction [3] is often limited to three components: α -helix, extended β -sheet, and coil. These forms are further folded together to form the tertiary, or globular, structure of a protein, and the quaternary structure is the result of the interaction between independently folded tertiary units.

Many methods have been proposed for protein secondary structure prediction. They can be categorized into four broad groups. The first group contains statistical based methods that make use of parameters obtained from analysis of known sequences and 3D structures [4, 5]. The predictive accuracy of this group of methods stays around 50 - 70%. The second group is "nearest neighbor" methods [6, 7].

These methods use sequence alignments to find sequences of known structure that have similar segments to the

query sequence. The predictive accuracy of this group of methods stays 68 -75%. The third group of methods is based on machine learning algorithms (HMM [8,9] and neural network models [10, 11]). The performance is in the range of 64 –75%. The fourth group of methods [12] combines previous methods, and its accuracy can reach to 72.9%.

In this paper, we develop a system that contains the databases of primary sequence, secondary and tertiary structure of protein. These databases are converted from Brookhaven's Protein Data Bank (PDB) [13, 14, 15]. Then, the Hidden Markov Model (HMM) [16, 17] is set up by statistically evaluating parameters from the proteins with recognized secondary structure. The forward, backward, and Baum-Welch algorithm [18, 19, 20] are used to train data, and Viterbi algorithm [21] is applied for finding optimal path. The accuracy of using HMM, Viterbi and to predict enzyme protein [22] is 64.34%.

To improve predicting accuracy, PSI-BLAST [23, 24, 25] is used to find 22,316 similar domain profiles between 1,025 enzyme proteins and 18,000 structure-recognized proteins. These domain profiles are further grouped into 2,926 groups with similarity $\geq 30\%$. These groups are stored into domain profile databases [26]. The 2D structure of the query protein's domain can be identified, if it's matched homology profile in the domain profile databases is found [27, 28, 29]. In final step, the 2D structures of remaining fragments can be predicted by using HMM approach. The accuracy of this method is above 80% for enzyme protein.

In the Section 2, HMM approach is described to predict 2D structure and it's performance is discussed also. Domain profile databases and HMM are used to increase the accuracy of prediction, and

resulting performance is analyzed in Section3. Finally the conclusion is given in Section 4.

2 USING HMM APPROACH TO PREDICT

An HMM is established for predicting 2D structure in Section 2.1. ER model of protein structure database is built in Section 2.2 first. Then this database is used to train HMM data in Section 2.3. Viterbi's forward and backward algorithm and Baum-Welch algorithm are applied in this section for data training also. The resulting HMM parameters are evaluated in Section 2.4. Finally, the performance of this approach is given in Section 2.5.

2.1 Our Hidden Markov model:

Our Hidden Markov model is shown in Figure 2.1 which are discussed below:

- Helix unit: represents the secondary structure of this amino acid is helix..
- Sheet unit: represents the secondary structure of this amino acid is sheet.
- Coil unit: represents the secondary structure of this amino acid is coil.
- P(ALA)..P(TYR) in each box: are the probability of amino acid that occurs in this type of secondary structure . They are also called emission probabilities.
- P(x1_x2): are transition probability from state x1 to state x2.

The emission probabilities and transition probabilities will be trained using forward, backward and Baum-Welch algorithm in Section 2.4. It is noted that sequence can starts from any secondary component (helix, Sheet or coil) in this design, and then Viterbi algorithm will be adopted to find the most possible path in Section.2.5

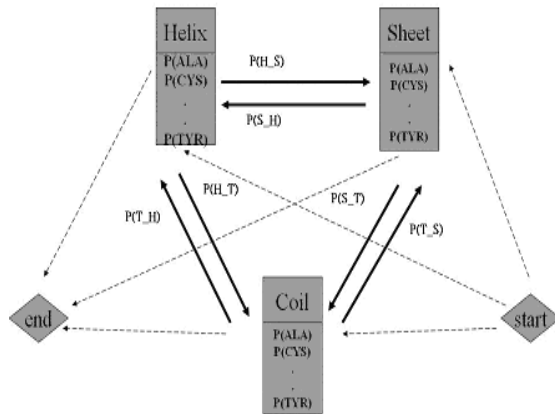


Figure 2.1 An Hidden Markov model for predicting 2D structure

2.2 E/R Model of protein structure database:

Figure 2.2 is the E/R model of our protein structure database. The database is build from 17945 protein data files.

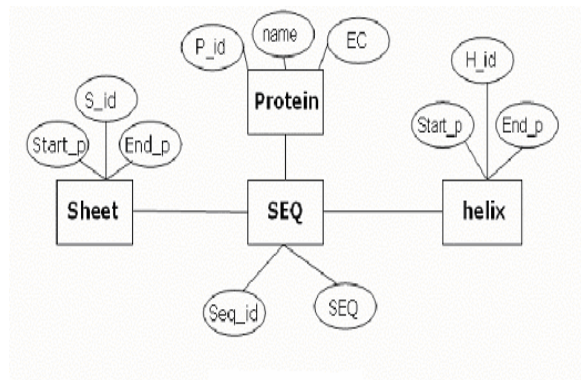


Figure 2.2 E/R Model for HMM training data.

2.3 HMM Data Training:

We choose the enzyme protein data in EC2 to train HMM parameters iteratively using Baum-Welch algorithm. The EC2 enzyme protein contains 2261 records in our database. The figure2.3 shows the flowchart of training HMM parameters.

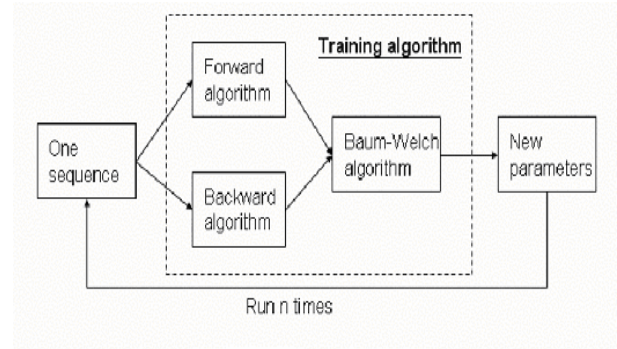


Figure2.3 HMM training process.

Let the variables are defined below:

A : A set of state transition probability $A = \{a_{ij}\}$.

B : is a emission probability set in each of the state $B = \{b_j(k)\}$.

a_{kl} : is transition probability from state k to state l.

$e_l(b)$: is the probability that symbol b is seen in state l.

x_i : is the i-th symbol in the sequence.

Θ : represents the entire current parameter values (all probabilities).

Baum-Welch algorithm can be applied to train HMM parameters. This algorithm is described as:

Initialization : Pick arbitrary model parameters.

Recurrence:

Set all the A and E variables to their pseudo-count values r(or to zero).

For each sequence $j=1 \dots n$:

Calculate $f_k(i)$ for sequence j using the forward algorithm.

$$\text{Initiation (} i=0 \text{) : } f_0(0) = 1, f_k(0) = 0 \quad K > 0$$

$$\text{Recursion (} i=1 \dots L \text{) : } f_i(i) = e_i(x_i) \sum_k f_k(i-1) a_{ki}$$

$$\text{Termination : } P(x) = \sum_k f_k(L) a_{k0}$$

Calculate $b_k(i)$ for sequence j using the backward algorithm:

Initiation ($i=L$) : $b_k(L)=a_{k0}$ for all k

Recursion ($i=L...1$) : $b_k(i)=\sum_l a_{kl}e_l(x_{i+1})b_l(i+1)$

Termination : $P(x)=\sum_l a_{0l}e_l(x_1)b_l(1)$

Add the contribution of sequence j to A and E:

$$A_{kl} = \sum_j \frac{1}{P(x^j)} \sum_i f_k^j(i) a_{kl} e_l(x_{i+1}^j) b_l^j(i+1)$$

$$E_k(b) = \sum_j \frac{1}{P(x^j)} \sum_{\{i/x_i^j=b\}} f_k^j(i) b_k^j(i)$$

Calculate the new model parameters:

$$a_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}}, \text{ and } e_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

Calculate the new log likelihood of the model.

$$\log P(x^1, \dots, x^n | \Theta) = \sum_{j=1}^n \log P(x^j | \Theta)$$

Termination :

Stop if the change in log likelihood is less than some predefined threshold or the maximum number of iterations is exceeded.

2.4 Result of HMM parameters:

After implementing the Baum-Welch algorithm, HMM parameters defined in Figure 2.2 can be trained completely. These parameters are listed in table 2.1.

Table 2.1 The resulting parameters of our HMM.

Data source : Enzyme_EC2			
2D Structure: HELIX			
Amino acid	P	Amino acid	P
H_A	0.1009	H_M	0.0273
H_C	0.0089	H_N	0.0352
H_D	0.0617	H_P	0.0238
H_E	0.0817	H_Q	0.0527
H_F	0.045	H_R	0.0506
H_G	0.0385	H_S	0.0456
H_H	0.0318	H_T	0.0439
H_I	0.0525	H_V	0.0547
H_K	0.0689	H_W	0.0185
H_L	0.1219	H_Y	0.0358

Data source : Enzyme_EC2			
2D Structure: HELIX			
Amino acid	P	Amino acid	P
H_A	0.1009	H_M	0.0273
H_C	0.0089	H_N	0.0352
H_D	0.0617	H_P	0.0238
H_E	0.0817	H_Q	0.0527
H_F	0.045	H_R	0.0506
H_G	0.0385	H_S	0.0456
H_H	0.0318	H_T	0.0439
H_I	0.0525	H_V	0.0547
H_K	0.0689	H_W	0.0185
H_L	0.1219	H_Y	0.0358

Data source : Enzyme_EC2			
2D Structure: TURN			
Amino acid	P	Amino acid	P
T_A	0.0685	T_M	0.0185
T_C	0.0124	T_N	0.0483
T_D	0.0742	T_P	0.0841
T_E	0.0564	T_Q	0.0292
T_F	0.0365	T_R	0.0497
T_G	0.1051	T_S	0.0562
T_H	0.0348	T_T	0.0532
T_I	0.0406	T_V	0.0524
T_K	0.0615	T_W	0.0144
T_L	0.0705	T_Y	0.0335

Transition	
Three state	
Transition	P
H_H	0.8982
H_S	0.0075
H_T	0.0943
S_H	0.018
S_S	0.8154
S_T	0.1667
T_H	0.0804
T_S	0.069
T_T	0.8506

2.5 Evaluating the performance of HMM approach:

We randomly choose 30 protein sequences in order to test system's performance. The resulting HMM parameters and Viterbi algorithm are used to predict 2D structure of protein, and the accuracy of every protein sequence are listed in Table 2.2. The most widely used way to determine prediction quality is defined in Q3 :

$$Q3 = [(PH + PS + PT) / N] \times 100\%$$

where PH , PS and PT are the numbers of correctly predicted states for three secondary components. N is the total number of residues.

Table 2.2 The performance of HMM approach for enzyme proteins

Using HMM to predict Enzyme_EC2							
ID	Protein name	Chain	P	ID	Protein name	Chain	P
1	10MH	A	0.587	16	1BKO	A	0.70576
2	1BHJ	A	0.611	17	1D2H	B	0.64981
3	1BJ4	A	0.628	18	1AQI	B	0.67663
4	1BWD	A	0.776	19	1BC5	A	0.7177
5	1C9Y	A	0.755	20	1EQ2	A	0.61553
6	1BQ2		0.576	21	1HNN	B	0.6212
7	1LXA		0.67	22	1HVV	A	0.75972
8	1ORT	J	0.553	23	1HW3	A	0.47591
9	1VZD		0.679	24	1NJE		0.52906
10	1ORT	C	0.702	25	1VZE		0.59272
11	1XVA	A	0.56	26	1QDI	A	0.6659
12	6JDW	A	0.643	27	1C9Y	A	0.6502
13	1D2C	B	0.716	28	1HZW	B	0.63242
14	1BHJ	D	0.685	29	1JKX	D	0.68297
15	1VID		0.69031	30	1KHH	B	0.49553
Total Accuracy			64.34%	Total PDB			30

3 IMPROVING OUR PREDICTION METHOD

The concept of our prediction

method is improved in Section 3.1. PSI-BLAST is used to search entire similar domain profiles in Section 3.2. These profiles are merged and stored into profile database in Section 3.3. Prediction performance is evaluated in Section 3.4 for EC2 proteins, and is evaluated in Section 3.5 for all structure recognized proteins.

3.1 The concept of our approach:

Although the EC2 group has 1633 protein records, many records are identical to others. So we reduce the unique protein records to 1025. Then we use PSI-BLAST to search similar domain profiles for each enzyme protein of EC2. Figure 3.1 shows the flowchart of predicting 2D structure.

There are two steps for finding domain profiles. The first step is to find the relative domain sequences of query protein in Gene Bank using PSI-BLAST. The second step is to search the corresponding group in domain profile databases. After that, get the most similar domain within this group using global alignment algorithm, and corresponding 2D structures of these domains can be retrieved from domain profile databases. The hidden markov model built in Section 2 can be applied to predict the remaining fragments.

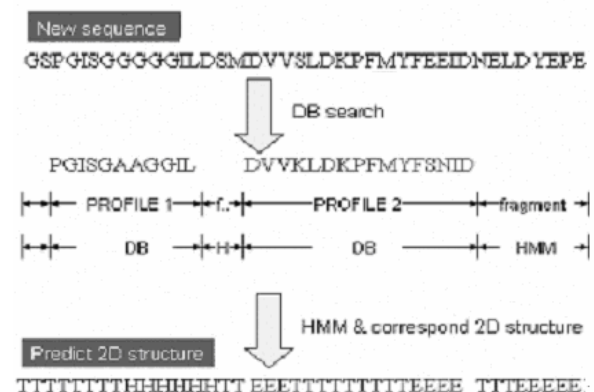


Figure 3.1 The Flowchart of Prediction 2D Structure

3.2 Search similar domains using PSI-BLAST:

The principal design goals in developing the Position-Specific Iterated BLAST (PSI-BLAST) were speed, simplicity and automatic operation. It is shown in Figure 3.2. The procedure of PSI-BLAST can be summarized in five steps:

- (1) PSI-BLAST takes a single protein sequence as an input and compares it to protein database, using the gapped BLAST program.
- (2) This program constructs a multiple alignment and generates a profile, if any significant local alignment is found. The original query sequence serves as a template for the multiple alignment and profile. Different numbers of sequences can be aligned in different template positions.
- (3) The profile is compared to the protein database in order to seek local alignments. After a few minor modifications, the BLAST algorithm can corrects and uses this profile directly.
- (4) PSI-BLAST estimates the statistical significance of the local alignments, using profile substitution scores that are related to gap scores. The statistical theory and parameters for gapped BLAST alignments still remain applicable to profile alignments.
- (5) Finally, PSI-BLAST iterates, by returning to Step (2), an arbitrary number of times or until convergence.

Profile-alignment statistics allow PSI-BLAST to proceed as a natural extension of BLAST; the results produced in iterative search steps are comparable to those data produced from the first pass. Unlike most profile-based search methods, PSI-BLAST runs as one program, starting with a single protein sequence, and the

intermediate steps of multiple alignment and profile construction are invisible to the user.

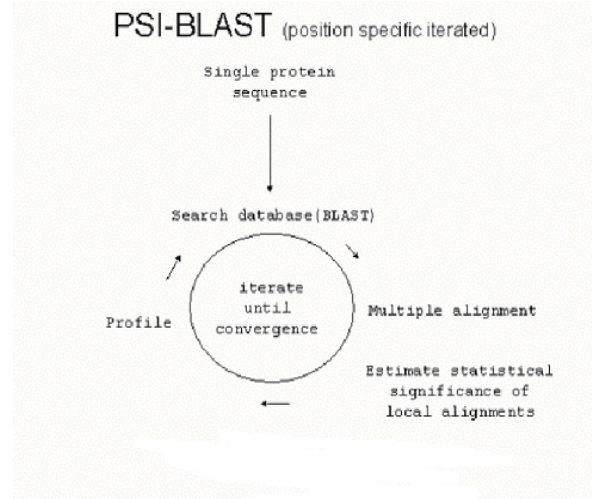


Figure 3.2 Search related profiles using PSI-BLAST.

3.3 Merge similar profiles:

There are 22316 profiles found from 1025 EC2 proteins. We use global alignment algorithm to classify all profiles which are collected from PSI- BLAST program. Figure 3.3 is the process of producing domain profile databases. Each step is described below:

- (1) In order to reduce time complexity, we only compare two sequences with length difference $< 30\%$ of maximum sequence length.

- (2) We use global alignment algorithm to count identical codes between two sequences.

If the number of identical codes $> 30\%$ of sequence length, then merge these two profiles into one group. These data are represented as domain profile databases.

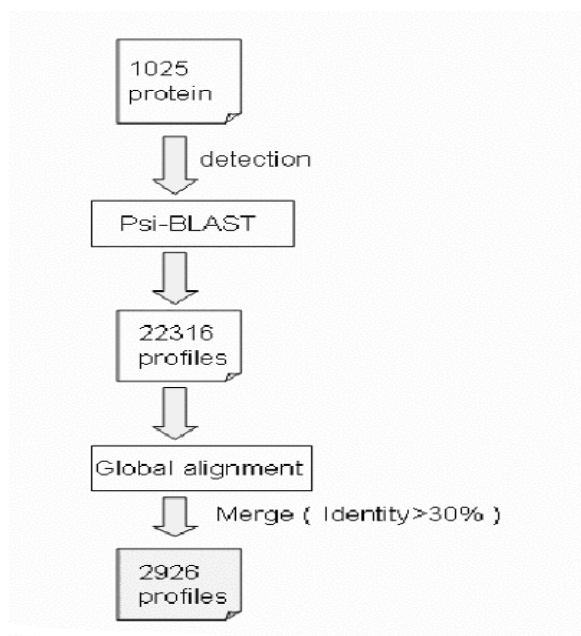


Figure 3.3 Collecting domain profile databases from PSI-BLAST files.

3.4 Evaluate prediction performance for enzyme proteins:

We choice 30 protein sequences from 1025 records of EC2 randomly, and use our approach to predict 2D structure. Evaluating procedure are given below:

- (1) Use PSI- BLAST to detect subsequence of target sequence.
- (2) Use global alignment to count the identical code number between target subsequence and every profile sequence. Then choose a best profile sequence which has maximum number of identical codes with target subsequence. The 2D structure of target subsequence will be substitute with this profile sequence structure.
- (3) After that, several fragments may still remain between subsequences provided in (1) Then we can use HMM approach to predict 2D structure of remaining fragments.
- (4) The accuracy of predicted structure can be evaluated by comparing with

recognized structure. The performance result for enzyme proteins is shown in Table 3.1.

Table 3.1 Evaluate the performance for enzyme proteins only.

Pdb_name	Chain	Accuracy(%)	Pdb_name	Chain	Accuracy(%)
2TPS	A	95.02	1JOW	A	81.10
1BHN		92.23	3THI	A	83.56
1VP3		91.66	2NMT		80.81
1RRB		87.94	100S		80.01
1NGS	A	84.56	1A25	A	89.26
1NOK	A	88.09	1FAQ		73.07
1HKB	A	84.41	1E2D		90.23
1KBR	A	80.12	1DTY	A	81.35
1EGH	B	85.00	1IB1	E	70.50
1DTY	A	81.35	1GCX		80.12
1DL6	A	91.17	1UBV		92.10
1C3Q	A	80.12	1B5		75.70
1BJN	A	84.44	1S66		81.33
1BTK	A	84.82	2ECK	A	80.37
1JSU	C	73.81	1NAW	A	83.77
TOTAL ACCURACY		80.12	TOTAL PDB		30

3.5 Evaluate performance for all structure recognized proteins:

If we choice 30 protein sequences from 18,000 structure recognized proteins, and use our method to predict 2D structure. The result is evaluated in Table 3.2 below. The accuracy is 68.46%. It is noted that our domain profile databases are collected only from enzyme proteins. These databases can be extended by collecting all PSI-BLAST files of the structure recognized proteins. It is also suggested that all the structure recognized proteins are classified into different classes based on their function. Each class can has it's own HMM parameters. The most possible class for a target protein can be predicted by finding the highest Viterbi probability from different sets of HMM parameters.

Table 3.2 Evaluate the performance for all structure recognized proteins.

Pdb_name	Chain	Accuracy(%)	Pdb_name	Chain	Accuracy(%)
1C9M	A	68.31	1HNI	A	67.81
1AZI		72.54	1GCW	B	70.12
1116		62.92	1RDS		70.91
2MYC		68.63	1DQM	H	68.54
1QIB	A	70.12	1COY		53.66
1HOK	B	81.43	1VRE	A	59.62
1K62	A	63.66	1G1V	A	80.13
1FPS		74.23	1K86	A	71.16
1IEZ	A	79.44	1GGH	A	65.42
1BOF		62.21	1EOE	C	64.32
1ES0		67.84	1I27		63.44
1JMK	A	65.94	1DAX		72.16
1NIB	B	76.67	1FF5	A	72.31
1JE3	A	71.58	1SNM		63.12
1TIU		69.81	1QKN	A	69.74
TOTAL ACCURACY		68.46	TOTAL PDB		30

4 CONCLUSION

Predicting the secondary structure of a protein using sequence is a fundamental step in predicting the overall (quaternary) structure of a protein. In this paper, we have developed a knowledge base system and HMM approach to predict protein 2D structure. To predict the structure of a target sequence, this sequence can get similar domain 2D structure from PSI-BLAST and our domain profile databases. The remaining fragments can be predicted by using HMM approach. The average accuracy of our method is 80.12% for enzyme protein.

5 REFERENCE

- [1] Geoffrey J. Barton, "Protein Sequence Alignment and Database Scanning", *Protein Structure Prediction - a practical approach*, IRL Press at Oxford University Press, 1996.
- [2] D. Higgins and W. Taylor,

Bioinformatics: Sequence, Structure, and databank, Oxford University Press, 2000.

- [3] Josie Chandler, "Methods of Secondary Structure Prediction", *Protein Sequence Analysis 8-006*, Literature Review Paper, Dec. 2003.
- [4] P. Y. Chou and G. D. Fasman, "Prediction of the Secondary Structure of Proteins from Their Amino Acid Sequence", *Adv. Enzymol Relat. Areas Mol. Biol.*, **47**:45-148, 1978.
- [5] R. D. King, M. Saqi, R. Sayle and M.J. Sternberg, "DSC: public domain protein secondary structure predication", *Comput Appl Biosci*, **13**: 473-4, 1997.
- [6] T. M. Yi and E. S. Lander, "Protein Secondary Structure Prediction Using Nearest-Neighbor Methods", *J. Mol. Biol.*, **232**:1117-1129, 1993.
- [7] D. Frishman and P. Argos, "Seventy-five percent accuracy in protein secondary structure prediction", *Proteins*, **27**: 329-35, 1997.
- [8] Anders Korogh, Bjorn Larsson, Gunnar von Heijne, Erik L. L. Sonnhammer, "Predicting Transmembrane Protein Topology with a Hidden Markov Model", *Application to Complete Genomes*, Academic Press, 2001.
- [9] V. Di Francesco, J. Garnier, P. J. Munson, "Protein Topology Recognition from Secondary Structure Sequences", *Application of the Hidden Markov Models to the Alpha Class Proteins*, Academic Press Limited, 1997.
- [10] B. Rost and C. Sander, "Combining evolutionary information and neural networks to predict protein secondary structure", *Proteins* **19**: 55-72, 1994.
- [11] J. M. Chandonia and M. Karplus. "New methods for accurate prediction of protein secondary structure", *Proteins*, **35**: 293-306, 1999.
- [12] J. A. Cuff and G.J. Barton, "Application of multiple sequence

- alignment profiles to improve protein secondary structure prediction”, *Proteins*, **40**: 502-511, 2000.
- [13] H. M. Berman , J. Westbrook, Z. Feng, G. Gilliland, T.N. Bhat, H. Weissig, “The Protein Data Bank”, *Nucleic Acids Res.*, **28**: 235-242, 2000.
- [14] Protein Data Bank < <http://www.rcsb.org/pdb/> >
- [15] PDB Format Description Version 2.2. <http://www.rcsb.org/pdb/docs/format/pdbguide2.2/guide2.2_frame.html >
- [16] S. L. Salzberg, D. B. Searls, S. Kasif, “An Introduction to Hidden Markov Models for Biologic Sequences”, *Computational Methods in Molecular Biology*, PP. 45-63, 1998.
- [17] Christopher Bystroff, Vesteinn Thorsson and David Baker, *HMMSTR: a Hidden Markov Model for Local Sequence-Structure Correlations in Proteins*, Academic Press, 2000.
- [18] A Kehagias, ” The Local Backward-Forward Algorithm”, *Proceedings of the International Joint Conference on Neural Networks*, PP. 321-326, Seattle, WA, 1991.
- [19] Baum L. E., “An inequality and association maximization technique in statistical estimation for probabilistic functions of a Markov process.”, *Inequalities*, **3**:1-8, 1972.
- [20] Richard Durbin, Sean R. Eddy, Anders Krogh and Graeme Mitchison, *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acid*, Cambridge University Press, 1998.
- [21] G. D. Fornely, " The Viterbi Algorithm", *Proceedings of the IEEE*, 61(3):268--278, 1973.
- [22] Bairoch A, “The ENZYME data bank”, *Nucleic Acids Res.*, **21**:3155-3156, 1993.
- [23] BLOSUM62 Substitution Matrix < <http://www.uky.edu/Classes/BIO/520/BIO520WWW/blosum62.htm> >
- [24] S. F. Altschul, etc., “Gapped BLAST and PSI-BLAST: a new generation of protein database search programs”, *Nucl. Acids Res.*, **25**:3389-3402, 1997.
- [25] S. B. Needleman, and C. D. Wunsch, “A General Method Applicable to the Search for Similarities in the Amino Acid Sequences of two Proteins”, *J. Mol. Biol.*, **48**: 443-453, 1970.
- [26] Arne Elofsson, “A Study on Protein Sequence Alignment Quality”, *PROTEINS: Structure, Function, and Genetics*, **46**:330–339, 2002.
- [27] Kenji Satou, Emiko Furuichi, Kyoko Takiguchi, Satoru Kuhara, Toshihisa Takagi, “Application of Deductive Database System PACADE toward Discovery of Clusters of Similar Structure in Protein”, *Proceedings of Twenty-Seventh Annual Hawaii International Conference on System Sciences*, 1994.
- [28] Inge Jonassen, Ingvar Eidhammer, Darrell Conklin, William R. Taylor, *Structure motif discovery and mining the PDB*, Oxford University Press, 2001.
- [29] Mohammed J. Zaki, Shan Jin , Chris Bystroff, “Mining Residue Contacts in Proteins Using Local Structure Predictions”, *IEEE International Symposium on Bioinformatics and Biomedical Engineering*, PP. 168-175, November 2000.

Prediction of Specific 70-mer Oligonucleotide with Neural Network Model

Chun-Chi Liu¹, Wen-Shyen E. Chen¹, Gwoboa Horng¹, Jeremy J.W. Chen^{2,†} and Pei-Chin Chang^{3,*,†}

¹Institute of Computer Science, ²Institute of Biomedical Sciences, and Molecular Biology,
National Chung-Hsing University, Taichung, Taiwan, ROC.

³Department of Bioinformatics,
Taichung Healthcare and Management University, Taichung, Taiwan, ROC.

Abstract

Motivation: Mammalian genomes are highly complex. To identify the specific 70-mer oligonucleotides of each gene in a mammalian gene database is a computation intensive task. With the advent of parallel genetic analysis methods such as microarrays and the availability of more and more whole genome sequences of organisms, an algorithm allowing speedy prediction of the specific oligonucleotides with a neural network will be a very useful tool.

Result: We present a computational model for specific 70-mer oligonucleotides prediction based on a Multi-Layer Backpropagation neural network using a broad range of input parameters. The algorithm was applied to the assemblies of gene sequences and was highly efficient for the TIGR Rat and *S.cerevisiae* TC database. The confidence can achieve 94.47% with coverage 22.52% in RGI test performance and it can achieve 96.00% with coverage 79.04% in SCGI test performance. Even the confidence of cross-species neural network can achieve 95.67% with coverage 93.49%. On the other hand, the trained neural network process 1,000 70-mer oligos that just costs 1.74 seconds.

Contact:

PC Chang <pcchang@thmu.edu.tw>

JJW Chen <jwchen@dragon.nchu.edu.tw>

1 Introduction

DNA microarrays could generate false positive data due to cross-hybridization between highly similar sequences (Xu et al., 2001, Evertsz et al., 2001, Modrek and Lee, 2002). Several studies have concluded that a threshold of around 70% sequence similarity can be used as a reference for cross-hybridization prediction (Kane et al., 2000, Xu et al., 2001, Hughes et al., 2001). Literature data (Hughes et al., 2001) indicate that longer oligonucleotides (60-80mer) provide significant sensitivity.

The algorithm described in this paper predicts the 70-mer specific oligos that have less than 70% similarity to the non-target sequences in a short time. The specific oligos can be applied as candidate probes for gene-specific expression analysis with minimal cross-hybridization problem.

A popular machine learning approach that effectively handles noise and complex relationships in a robust way is the artificial neural network (Baldi, 1998). Artificial neural networks have commonly been used as classifiers. Wu et al. have devised a neural network system for the automatic classification of protein sequences according to super-families (C. Wu, 1992). They have also been used to predict antisense oligo (Alistair M. Chalk 2002).

Two of the best-known heuristic algorithms of sequence alignment are FASTA and BLAST. In this paper, we present a data compilation of 70-mer oligos with the determined similarity value by WU-BLAST (Gish, 1996 - 2004), and apply a neural network model to predict oligo specificity. The execution time of BLAST is much longer than the trained neural network execution time.

2 Database

In this paper, we used the assembled Rat tentative consensus (RGI TC release 11.0) database (Quackenbush, 2001) from The TIGR Gene Index Databases and applied the algorithm to drive the neural network training set. We randomly selected 10 TCs from TC41xxxx (Range TC410000 ~ TC419999) for training data source and 100 TCs from TC42xxxx (Range TC420000 ~ TC429999) for test data source. We chose the small training set because the neural network training needs to learn 10,000 epochs.

General training set: We randomly selected 100 “70-mer oligo” at every TC from training data source. So the general training set has 1,000 oligos.

Average training set: We randomly selected 500 low-similarity (less than 70% similarity) and 500 high-similarity (great than 70% similarity) “70-mer oligo” from training data source. The average training set also has 1,000 oligos.

Test set: We randomly selected 100 “70-mer

*To whom correspondence should be addressed.

† JJW Chen and PC Chang contributed equally to this work and are joint corresponding authors.

oligo” at every TC from test data source. The test set will have 10,000 oligos.

3 Derived parameters as network input

We created 6 ~ 12-mer subsequence frequencies distributions for the whole RGI TCs then used average frequencies for low frequencies threshold and a half of average frequencies for very low frequencies threshold. The average frequencies are presented in Table 1.

Every l -mer subsequence is encoded by the following formula:

$$code = \sum_{i=1,l} c_i \times 4^{i-1}$$

Where c_i is 0, 1, 2, or 3 for A, C, G or T at the i -th base of the sequence segment and l is the length of the subsequence. For example, a sequence such as ACGT has the numerical code of $0 \times 4^0 + 1 \times 4^1 + 2 \times 4^2 + 3 \times 4^3 = 228$, and $l = 4$.

N-mer	Average frequencies	Half average frequencies
6-mer	18691.71	9345.86
7-mer	4661.17	2330.59
8-mer	1162.38	581.19
9-mer	289.88	144.94
10-mer	72.31	36.16
11-mer	18.44	9.22
12-mer	5.52	2.76

Table 1. The average frequencies

A set of parameters was derived from the features contained in the 70-mer-oligo selections, including values for 5 groups:

(1) Shannon’s entropy of the 6-mer and 8-mer subsequence frequencies distributions:

$$H(\cdot) = - \sum f \log_2 f$$

Where $H(\cdot)$ is the Shannon entropy of the relative frequencies of symbol occurrences $\{f\}$ in the corresponding subsequence (P. Bernaola Galvan, 1996; Shannon, 1948).

(2) The number of low frequencies subsequences (less than average frequencies) for 6, 7, 8, 9, 10, 11, and 12-mer. And the number of very low frequencies subsequences (less than half average frequencies) for 6, 7, 8, and 9-mer.

(3) The number of unique subsequences (frequency is equal to 1) for 10, 11, and 12-mer.

(4) The sum of the frequencies of the subsequences for 6, 7, 8, 9, 10, 11, and 12-mer.

(5) The similarity value by WU-BLAST (Gish, 1996-2004). The high sensitivity of similarity is requirement in the training set. We apply the parameters “word length $W = 7$, matching score $M = 1$, mismatching score $N = -1$, no gap, significance threshold $E = 10$ ” to WU-BLAST.

So the training and test set have 23 input parameters and one desired output item (similarity value from WU-BLAST).

4 Neural Network

A simple outline of the operation for neural network training and prediction was shown in Fig 1. We use the Java Object Oriented Neural Network (JOONE, 2004) that is an open source project. JOONE offers a highly adaptable neural network for Java programmers. The JOONE project source code is covered by a Lesser GNU Public License (LGPL).

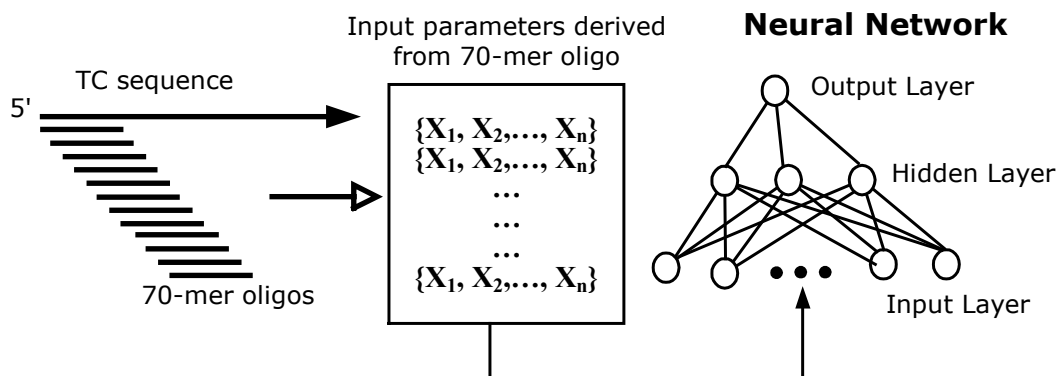


Fig 1. Simple outline of the operation for neural network training and prediction

The training function we used is a batch mode training algorithm. The training is terminated when the number of iterations exceeds the number of epochs (10,000) or when the Root Mean-Square Error (RMSE) value is less than 0.01. We employed the sigmoid layer as both input and output layer. Sigmoid function's output is smoothly limited within the range 0 and 1. The hidden layer is the logarithmic layer. The behavior of logarithmic layer permits to avoid the saturation of the processing elements of a layer in presence of a lot of input synapses connected, or in presence of input values very near to the limits 0 and 1.

Logarithmic activation function:

$$\begin{aligned} y &= \log(1 + x) & \text{if } x \geq 0 \\ y &= \log(1 - x) & \text{if } x < 0 \end{aligned}$$

Sigmoid activation function:

$$y = \frac{1}{1 + e^{-x}}$$

For the current application, the learning rate was set to 0.2, with momentum 0.5 at all simulations. In present study, a backpropagation neural network with one hidden layer was selected. The number of hidden layer nodes is selected based on the classification performance on training data. We trained 4 networks (10, 12, 14 and 16 hidden layer nodes) at all simulation and selected the best performance according to RMSE.

5 Result

The neural network's output value can indicate the specificity of the tested 70-mer oligo. We suppose that an oligo will be the specific oligo (similarity less then 70%) if its score from neural network's output is less than "score threshold". Then we calculated the criterions according to the blast similarities.

We assume that the neural network predicts N specific oligos, M predicted oligos are wrong in the N oligos, and there are P determined specific oligos (similarity less then 70%) by blast. We defined two criterions confidence and coverage to measure the test performance:

$$\text{Confidence} = M/N$$

$$\text{Coverage} = (N-M)/P$$

The results are shown in Table 2, Table 3, Fig 1 and Fig 2. As we can see, the confidence with general training set cannot great than 88% but the confidence with average training set can achieve 94.47% with coverage 22.52%. The average training set is better in this point of view.

Score threshold	Confidence (%)	Coverage (%)
0.80	87.29	96.37
0.81	87.25	96.52
0.82	87.23	96.70
0.83	87.19	96.98
0.84	87.13	97.25
0.85	87.04	97.55
0.86	86.94	97.78
0.87	86.92	98.03
0.88	86.88	98.29
0.89	86.89	98.53
0.90	86.87	98.89
0.91	86.78	99.10
0.92	86.77	99.51
0.93	86.65	99.71
0.94	86.52	99.87

Table 2. Test performance with general training set in a 23-14-1 neural network. Training RMSE = 0.1257.

Score threshold	Confidence (%)	Coverage (%)
0.80	93.40	10.32
0.81	93.40	12.44
0.82	94.06	15.38
0.83	94.38	18.67
0.84	94.47	22.52
0.85	94.15	26.26
0.86	94.16	30.60
0.87	94.06	35.55
0.88	93.68	40.12
0.89	93.48	45.12
0.90	92.82	50.25
0.91	92.03	56.02
0.92	91.46	97.15
0.93	90.75	99.11
0.94	89.96	99.83

Table 3. Test performance with average training set in a 23-16-1 neural network. Training RMSE = 0.1082.

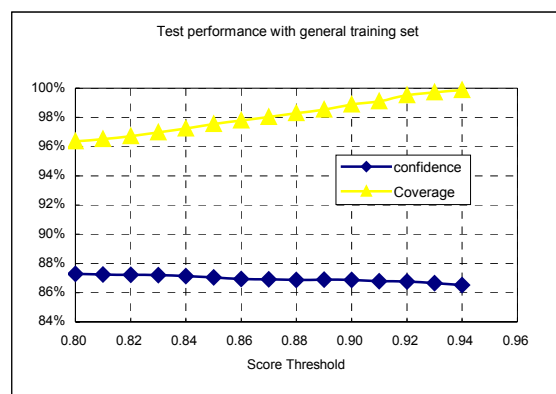


Fig 2. Test performance with general training set

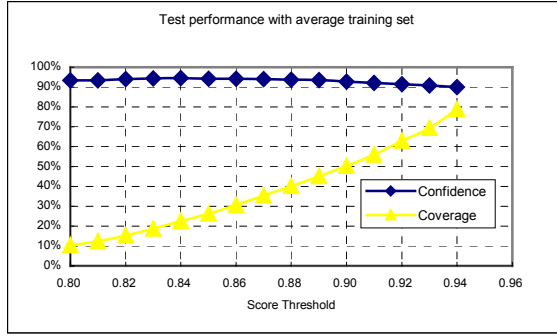


Fig 3. Test performance with average training set

The execution time of blast is much longer than the trained neural network execution time. The neural network may spend long time to discover the elegant network in training stage but it has great efficiency in test stage. The blast execution time of 1,000 oligos is 2,214.76 seconds. The neural network test 1,000 oligos time includes input parameters calculation (25.03 seconds) and process oligos (1.74 seconds). It is 26.77 seconds (1.02% of blast execution time).

6 Discussion

We considered the general training set has low performance because the distribution of randomly selected oligos has a bias. According to the test performance, using average training set can resolve the issue of bias training set and improve the performance.

When considering a multitude of organism sequence databases we asked ourselves the question: “Can the neural network trained by one species apply to another species?” We then used the assembled *S.cerevisiae* tentative consensus (SCGI TC release 3.0) database (Quackenbush, 2001) from The TIGR Gene Index Databases and applied the algorithm to drive SCGI training set and test set by previous methods. The SCGI sequence database has 4,107 TCs and the RGI sequence database has 51,916 TCs. So the SCGI database is related small and it has higher percentage of low similarity oligos than RGI.

To compare the test performance with different species, we use the same test set (SCGI test set) and two different trained neural networks. The results are shown in Table 4 and Table 5. As we can see, the confidence with native network (trained by SCGI) can achieve 96.00% with coverage 79.04% but the confidence with cross-species network can achieve 95.67% with coverage 93.49%. The cross-species network is good enough to perform prediction.

We trained the neural network in 10,000 epochs. It costs long time to try to discover the best network. If we can train neural network once, then

we use it on all of species. It will save much time to train the neural network for every species.

The future directions for the future research might be improvement of training performance by input parameters analysis or study of various N-mer specific oligos prediction.

Score threshold	Confidence (%)	Coverage (%)
0.80	95.95	71.38
0.81	96.00	79.04
0.82	95.97	83.61
0.83	95.96	88.27
0.84	95.91	91.11
0.85	95.89	93.13
0.86	95.89	94.68
0.87	95.79	95.85
0.88	95.67	97.21
0.89	95.41	97.93
0.90	95.34	98.61
0.91	95.29	98.90
0.92	95.22	99.17
0.93	95.17	99.32
0.94	95.03	99.46

Table 4. Test performance of SCGI test set in a SCGI trained 23-12-1 neural network. Training RMSE = 0.0608.

Score threshold	Confidence (%)	Coverage (%)
0.80	95.73	86.37
0.81	95.66	88.06
0.82	95.62	89.71
0.83	95.66	91.50
0.84	95.67	93.49
0.85	95.52	95.01
0.86	95.48	95.90
0.87	95.46	96.61
0.88	95.31	97.36
0.89	95.13	98.10
0.90	94.78	98.54
0.91	94.35	98.82
0.92	93.96	99.05
0.93	93.45	99.13
0.94	92.84	99.63

Table 5. Test performance of SCGI test set by RGI trained 23-16-1 neural network. Training RMSE = 0.1082.

References

- Alistair M. Chalk and Erik L.L. Sonnhammer. (2002) Computational antisense oligo prediction with a neural network model. *Bioinformatics*, 18, 1567-1575.
- Baldi,P.F. and Brunak,S. (1998) *Bioinformatics: The machine learning approach*. MIT Press, Cambridge, MA, pp. 91–104.
- Cuticchia,A.J. and Timberlake,W.E. (1993) Probe choice and analysis package-a set of program to aid in choosing synthetic oligomers for contig mapping. *CABIOS*, 9, 201-203.
- C. Wu, G. Whitson, J. McLarty, A. Ermongkonchai, and T.C. Chang, (1992) *Protein Classification Artificial Neural System*, Protein Science, vol. 1, 1992, pp. 667–677.
- Drmanac,S., Stavropoulos,N.A., Labat,I., Vonau,J., Hauser,B., Soares,M.B. and Drmanac,R. (1996) Gene-representation cDNA clusters defined by hybridization of 57419 clones from infant brain libraries with short oligonucleotide probes. *Genomics*, 37, 29-40.
- Evertsz,E.M., Au-Young,J., Ruvolo,M.V., Lim,A.C. and Reynolds,M.A. (2001) Hybridization cross-reactivity within homologous gene families on glass cDNA microarrays. *Biotechniques*, 31, 1182, 1184, 1186.
- Fu,Y.X., Timberlake,W.E. and Arnold,J. (1992) On the design of genome mapping experiments using short synthetic oligonucleotides. *Biometrics*, 48, 337-359.
- Gish, W. (1996-2004) <http://blast.wustl.edu>.
- Herwig,R., Schmitt,A.O., Steinfath,M., O'Brien,J., Seidel,H., Meier-Ewert,S., Lehrach,H. and Radelof,U. (2000) Information theoretical probe selection for hybridization experiments. *Bioinformatics*, 16, 890-898.
- Hughes,T.R., Mao,M., Jones,A.R., Burchard,J., Marton,M.J., Shannon,K.W., Lefkowitz,S.M., Ziman,M., Schelter,J.M., Meyer,M.R. et al. (2001) Expression profiling using microarrays fabricated by an ink-jet oligonucleotide synthesizer. *Nat. Biotechnol.*, 19, 342-347.
- Java Object Oriented Neural Network (JOONE). (2004) <http://joone.sourceforge.net/>.
- Kane,M.D., Jatke,T.A., Stumpf,C.R., Lu,J., Thomas,J.D. and Madore,S.J. (2000) Assessment of the sensitivity and specificity of oligonucleotide (50mer) microarrays. *Nucleic Acids Res.*, 28, 4552-4557.
- Li,F. and Stormo,G.D. (2001) Selection of optimal DNA oligos for gene expression arrays. *Bioinformatics*, 17, 1067-1076.
- Lockhart,D.J., Dong,H., Byrne,M.C., Follettie,M.T., Gallo,M.V., Chee,M.S., Mittmann,M., Wang,C., Kobayashi,M., Horton,H. and Brown,E.L. (1996) Expression monitoring by hybridization to high-density oligonucleotide array. *Nat. Biotechnol.*, 14, 1675-1680.
- Modrek,B. and Lee,C. (2002) A genomic view of alternative splicing. *Nat. Genet.*, 30, 13-19.
- P. Bernaola Galvan, R. Roman Roldan, J.L. Oliver. (1996) Compositional Segmentations and long-range fractal correlations in DNA Sequences. *Physical Review E* 53, pp 5181-5189.
- Pei-chin Chang and Konan Peck. (2002) Design and assessment of a fast algorithm for identifying specific probes for human and mouse genes. *Bioinformatics*, 19, 1311-1317.
- Quackenbush,J., Cho,J., Lee,D., Liang,F., Holt,I., Karamycheva,S., Parvizi,B., Pertea,G., Sultana,R. and White,J. (2001) The TIGR Gene Indices: analysis of gene transcript sequences in highly sampled eukaryotic species. *Nucleic Acids Res.*, 29, 159-164.
- Religio,A., Schwager,C., Richter,A., Ansorge,W. and Valcarcel,J. (2002) Optimization of oligonucleotide-based DNA microarrays. *Nucleic Acids Res.*, 30, e51.
- Rouillard,J.M., Herbert,C.J. and Zuker,M. (2002) OligoArray : genome-scale oligonucleotide design for microarrays. *Bioinformatics.*, 18, 486-487.
- SantaLucia,J.,Jr. (1998) A unified view of polymer, dumbbell, and oligonucleotide DNA nearest-neighbor thermodynamics. *Proc. Natl. Acad. Sci. USA*, 95, 1460-1465.
- Shannon, C. E. (1948) A mathematical theory of communication. *The Bell System Technical Journal*. 27:379–423; 623–656.
- Snehasis Mukhopadhyay, Changhong Tang, Jeffery Huang And Mathew Palakal. (2003) Genetic sequence classification and its application to cross-species homology detection. *Journal of VLSI Signal Processing* 35, 273–285.
- The TIGR Gene Index Databases, The Institute for Genomic Research, Rockville, MD 20850 (URL: <http://www.tigr.org/tdb/tgi>)
- Tijssen,P. (1993) *Hybridization with nucleic acid probes, Part I: Theory and nucleic acid preparation*. Amsterdam; Elsevier, New York, pp. 19-79.
- Xu,W., Bak,S., Decker,A., Paquette,S.M., Feyereisen,R. and Galbraith,D.W. (2001) Microarray-based analysis of gene expression in very large gene families: the cytochrome P450 gene superfamily of Arabidopsis thaliana. *Gene*, 272, 61-74.
- Yuan,Q., Ouyang,S., Liu,J., Suh,B., Cheung,F., Sultana,R., Lee,D., Quackenbush,J. and Buell,C.R. (2003) The TIGR rice genome annotation resource: annotating the rice genome and creating resources for plant biologists. *Nucleic Acids Res.*, 31, 229-233.

利用微陣列基因表現資料分析癌症相關基因之研究

沈詩珊¹ 石貴中¹ 梁綺雯¹ 劉峰銘¹ 陳榮銘^{2*} 張建國³ 蔡進發¹ 施河^{1,2}

¹ 臺中健康暨管理學院生物資訊研究所

² 臺中健康暨管理學院資訊與設計學系

³ 中國醫藥大學附設醫院

*E-mail: rmchen@thmu.edu.tw

摘要

癌症的形成是多步驟性的，其中包括致癌基因的過度活化、抑癌基因的不活化、DNA 修補基因以及其他相關因子的突變，造成細胞生長、分化、繁殖及死亡過程產生改變。而其中又以致癌基因的過度表現在許多癌症發生的起始階段扮演了非常重要的角色。此外，抑癌基因是可抑制致癌症的發生，如果抑癌基因的正常功能喪失，就會增加癌症的發生率。目前有許多抑癌基因已被鑑定出來，但致癌基因只是冰山一角，因此等待我們去研究。在人類基因中尋找導致基因突變的原因是很難的，尤其是使用傳統的分析方法。而目前不同癌細胞中各種基因表現量的變化已可利用 cDNA Microarray 進行分析。這方面的研究，在過去已累積了相當多的資料。本研究收集分析這些 Microarray 的資料，運用統計的方法進行差異基因的搜尋與篩選，來尋找與癌症有關的基因，如此能縮小基因實驗的範圍，確定實驗的方向，避免一些不必要的實驗，以便協助生物學家更快地找出與癌症相關的基因。

1 前言

癌症是一種很複雜且是家族性的疾病。從分子生物學的觀點來看，癌症是由不正常的基因表現所造成的基因疾病，而基因表現的交綜複雜可說是與 DNA 不穩定有直接的關係[1]-[5]。癌症的形成是多步驟性的，包括致癌基因的過度活化、抑癌基因的不活化、DNA 修補基因以及其他相關因子的突變，造成細胞生長、分化、繁殖及死亡過程產生改變。而其中又以致癌基因的過度表現在許多癌症發生的起始階段扮演了非常重要的角色。此外，因為抑癌基因可抑制致癌基因的生長，如果抑癌基因的功能喪失，就會增加癌症腫瘤的發生。有許多抑癌基因已被鑑定出來[6]-[7]，但致癌基因的研究只是冰山一角，因此等待我們去研究[8]-[12]。在人類基因中來尋找導致基因突變是很難的，尤其是使用傳統的分析方法，一次只針對一個或少數基因研究，對大部分未知功能的人類基因而言，這似乎是一條遙遠而崎嶇的長路。基因為一切生命現象的基礎，回顧過去幾十年來應用於篩選差異表現基因的方法，例如免疫篩選法 (immuno-screening)、刪減篩選法 (subtractive screening)、差異性篩選法 (differential screening)、演化保存區域篩選法 (conserve region between species or gene

family)、蛋白質序列推衍法 (oligonucleotide probe derived from protein sequence) 及差異性顯示法 (differential display) 等[13]-[19]。這些方法或可單獨進行或可結合其他實驗步驟以使篩選基因更加有效率，但仍須耗費大量人力、物力及時間，實驗結果亦往往得到偽陽性訊號(false positive)，而無法以行更進一步的實驗。在監測(monitors)基因表現上，傳統的方式是以北方墨點雜交法，一次一個或少數個基因來進行實驗。對於多基因參與的生化過程或基因網路而言，則需花費相當長的時間才可能完成，或甚至無法進行。而現在有一些高科技的發展，研究人員可透過生物資訊的方法，一次可同時分析很多的基因。因此若能夠運用生物資訊的技術來分析與癌症相關基因的研究，那就能鎖定實驗的方向，也就是降低搜尋空間，以便協助生物學家更快地找出癌症相關的基因。

2 方法

基因晶片之所以能成為本世紀最受矚目的功能基因研究工具之一，乃其具有 (1) 大量樣本；(2) 同時進行；(3) 自動化分析等三項特點及 (1) 基因定量；(2) 差異基因搜尋兩項基本特質，交互運用而衍生出許多應用，不僅於生物醫學上，在微生物及植物界也同樣具有廣大的應用範圍，本研究是針對差異表現基因來做篩選。

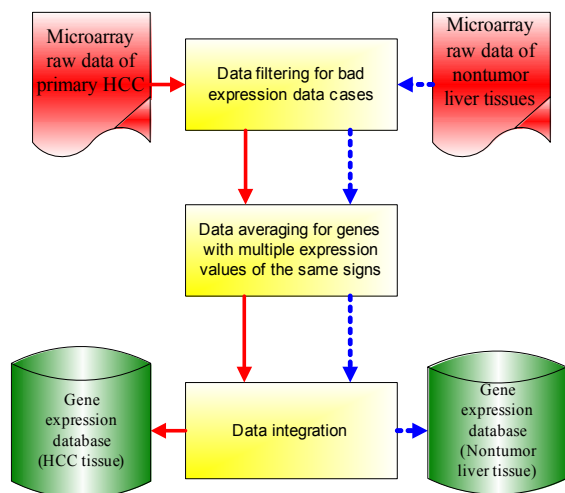
2.1 資料前置處理

Microarray 可提供成千上萬個基因同時進行分析工作，進而產生相當龐大的基因表現資料。但是在眾多的基因表現資料當中，有可能因為人為或環境的因素造成資料的誤差、不正確性。所以在進行 Microarray 差異性基因表現資料的分析前，必須對資料做前置篩選動作，資料前處理流程如圖一所示。過濾掉不正確的資料，以確立資料的正確性，並將多筆 Microarray 基因表現資料表合併成單一資料表，以方便進行分析。以下是我們所做的資料前置處理步驟說明。

(1) 從 Stanford Genomic Resources 網站中的 Stanford Microarray Database 中收集肝癌的 Microarray Raw Data (<http://genome-www.stanford.edu/>)，在此研究中

我們分別取得 HCC (hepatocellular carcinoma) 組織與正常組織的所有檢體樣本的原始資料。

- (2) 刪除所有檢體資料內的無效資料樣本。在基因表現資料裡面，無基因表現度和無基因名稱的欄位並無法幫助我們進行分析。而且這些是最容易發生錯誤的資料，在一開始便可以將這些資料進行刪除動作。
- (3) 建立新合併資料表的目的是將眾多基因表現資料表合而為一，以便分析基因表現的差異度。在一開始只先建立幾項基本固定欄位(NAME、Clone ID、Gene Symbol、Gene Name、Cluster ID、Accession)，這些欄位是固定欄位，並不會做變更或新增的動作。在基本欄位建立後，才會插入每個檢體樣本資料表的基因表現度欄位。
- (4) 由於在 Microarray 實驗過程中，經常將同一基因的表現度做數次的實驗，因此同一基因產生多個基因表現值。然而，同一基因的表現度有高低、有正有負。因此必須做第二次的資料篩檢動作。在單一檢體樣本的 Microarray 中，同一基因會有正負兩種表現值存在時，我們並不能判斷該基因表現度是過度或不足表現，理論上兩種表現值中只有一種為正確表現值，所以這種情況我們必須放棄該基因的表現值資料。在基因表現都是相同的極性時，因為表現度的值有高低。所以我們將該基因表現資料做平均動作，取平均值作為參考值。
- (5) 匯入合併資料庫。在取得平均後的基因表現度資料後，我們以 Gene Symbol 為 y 軸座標、以基因表現度欄位為 x 軸座標，將該值填入欄位內，經過此處理後會產生癌症組織與正常組織兩個資料庫。



圖一、資料前置處理流程圖

2 差異表現基因篩選

差異性比較是針對在 Microarray 中的各個癌症相

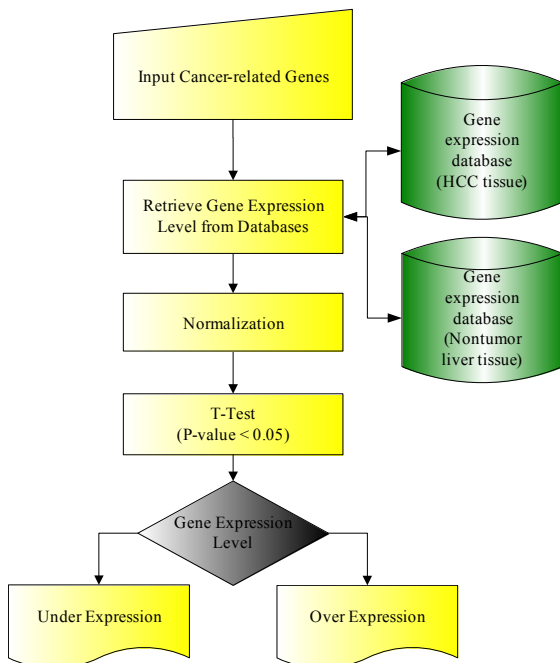
關基因來做研究，探討基因在癌症組織與正常組織中表現的差異。例如比較正常細胞與癌細胞之差異，可大量篩選腫瘤標記基因 (tumor marker) 或應用於具不同癌症轉移能力之細胞株以獲得癌症轉移相關基因。圖二是整個差異表現基因篩選的流程圖，以下我們說明如何處理差異表現的基因。

- (1) 經由文獻探討取得與癌症相關的基因，我們將它分成致癌基因與抑癌基因兩類。
- (2) 從肝癌 Microarray 資料庫中取得基因表現資料，其中包含 102 個 primary HCC (由 82 個病人中取得) 與 74 個 non-tumor liver tissues (由 72 個病人中取得)。
- (3) 篩選出在肝癌微陣列基因表現資料中癌症組織與正常組織皆出現的檢體。
- (4) 在肝癌微陣列基因表現資料中，取每筆微陣列基因表現資料經過 $\log_2$ 轉換計算後紅光與綠光基因表現量的差。
- (5) 將已知的致癌基因與抑癌基因放置到我們建好的肝癌 Microarray 資料庫中做更進一步的搜尋。
- (6) 將已搜尋到的致癌基因與抑癌基因的基因表現資料留在癌症組織與正常組織上皆有出現的檢體。
- (7) 將癌症組織檢體的基因表現資料減正常組織檢體的基因表現資料來做個別基因表現的差異性計算。
- (8) 分別求出致癌基因與抑癌基因在癌症組織與正常組織基因表現上的平均數與標準差。
- (9) 透過致癌基因與抑癌基因在癌症組織與正常組織基因表現上的平均數與標準差來做 T-test 檢定。
- (10) 我們篩選 P 值 < 0.05 的基因，以具有統計上的意義。

在本研究中，我們選擇基因表現資料中癌症組織與正常組織皆出現的檢體全部共 75 個檢體樣本，但是其中也有檢體樣本表現值是不具有公信力，如在同一個檢體有多個點，而表現值有正有負，我們會排除這種檢體，因為此種資料會影響分析結果，所以有些基因的檢體樣本會小於 75 個。

3 測試資料

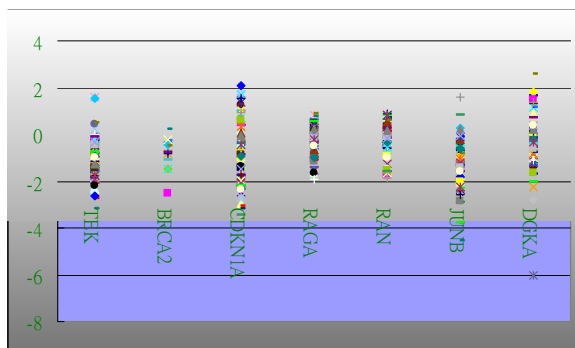
我們在 NCBI 的 PubMed 文獻中，尋找有討論關於致癌基因 (oncogene) 與抑癌基因 (tumor suppressor gene) 的文章，再經人為經驗判斷來歸類，區分為致癌基因與抑癌基因這兩類，對於定義致癌基因與抑癌基因，或許會有不同的觀點，也會有以前定義為致癌基因到最後定義為抑癌基因的可能性，所以在於資料的篩選，並非是絕對的致癌基因與抑癌基因，但是可以說是大部分都是確定了。



圖二、差異表現基因的流程圖

4 結果

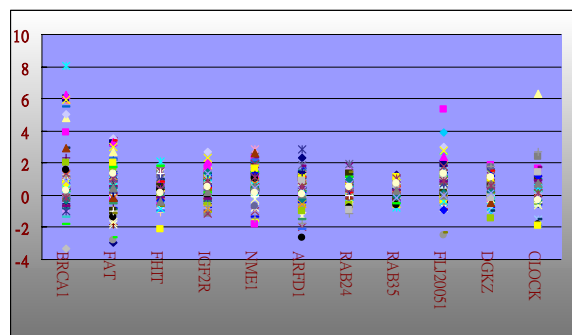
當我們在做分析時，因為只能取得同時在致癌基因與抑癌基因都有資料的檢體，每個基因最多會有 75 個差異表現的檢體樣本，且會有基因的檢體資料分佈圖，我們展示幾個抑癌基因與致癌基因在肝癌 Microarray 資料中差異性表現的分佈圖(如圖三與圖四)，基因上的每一點代表每一個檢體樣本的表现值，所以，圖三上的基因都是表現不足基因，所以其分佈的範圍很明顯的偏向負值，而圖四上的基因都為過度表現的基因，其分佈的範圍偏向正值。另外我們可以使用統計方法來將其分佈結果數據化，如表一與表二的方式，利用平均值與標準差來表示基因的分佈落於正值或負值。



圖三、抑癌基因差異表現的分佈圖

我們了解所謂的致癌基因就是因為表現過度而造

成癌症，抑癌基因則是因為表現不足而導致癌症，所以在正常組織與癌症組織的差異性比較下，如果是正值的



圖四、致癌基因差異表現的分佈圖

表現，我們可以判斷此基因是因為表現過度而造成癌症，所以可以驗證他是致癌基因，抑癌基因則反之。

表一與表二分別為致癌基因與抑癌基因的部份結果，欄位由左至右分別為 Gene Symbol、檢體樣本平均值、檢體樣本變異數、T-test 檢定、過度表現 (Over-expression Layer: OEL) 的檢體數目、表現不足 (Under-expression Layer: UEL) 的檢體數目。紅色與綠色的 Gene Symbol 是經 T-test 檢定後，具有統計上的意義之基因，表示基因在正常組織與癌症組織的分配相同，其差異性的比較是比較具有可信度。

從表一與表二中，我們可以發現大多數的基因平均值都大於 0，可是經由 T-test 檢定後，可以更進一步的篩選出可信度較高的致癌基因。在我們的研究中，我們也有針對檢體來做分類，經過分類後的檢體再進行一次差異性比較後，發現 T-test 小於 0.05 的基因表現是相當的穩定，不會有負值的變動，所以可以確信經 T-test 檢定後基因表現值小於 0.05 的基因是值得相信。而抑癌基因與致癌基因的檢定有相同的結果，只是表現值相反。紅色與綠色的 Gene Symbol 是經 T-test 檢定後，具有統計上的意義之基因，表示基因在正常組織與癌症組織的分配相同，其差異性的比較是比較具有可信度。在抑癌基因的表格中，我們可以找出幾個基因來說，例如 JunB、RAGA、BRCA2 都是目前經生物實驗後分別在文獻上發表的抑癌基因，在我們的資料庫也表示出，這些基因有表現不足的現象。可是 TP53 卻無法很明確的表示出它是屬於過度表現還是表現不足，其原因有很多，TP53 以前被認為是致癌基因，可是近幾年被發現為抑癌基因，所以在以前與現在之文獻上的討論具有兩種認定。在此研究中，我們把造成錯誤的因素放在 Microarray 資料的本身，可能是因為資料在做前置處理時，因 Microarray 實驗的誤差導致的錯誤。總而言之，利用差異性基因表現的方法，確實可以篩選出具有實驗價值的基因。

5 討論

本研究意在收集分析這些 Microarray 的資料，並運用生物資訊的方法來尋找與癌症有關的基因，就能縮小實驗的範圍，確定實驗的方向，避免一些不必要的實驗，以便協助生物學家更快地找出與癌症相關的基因。

Microarray 的實驗目前還是為大家所質疑，所實驗出來的結果也因種種的因素導致有相當的錯誤率，所以在此，我們針對差異表現基因的 Microarray 資料經過 T-test 的檢定，來減少實驗時所造成的雜訊，以提高 Microarray 資料的可靠度。可是由於此動作會造成檢體樣本的減少，而導致樣本不足，使得有些基因無法被辨識出為致癌基因或抑癌基因。對於檢體樣本的減少所造成的誤差，我們未來將針對 Microarray 資料的前置處理來做進一步的研究與分析，以確保在做差異表現基因研究時，有足夠的檢體樣本資料來分析。另外，我們也會繼續的收集各個在網路上分享的 Microarray 資料，更充實我們的資料庫，也同時希望國內有更多的 Microarray 實驗室可以分享他們的資料，以提供大家來做研究。畢竟東方人的人種與西方人的人種有所不同，所以基因的表現量也會有所不同。

Oncogene in Liver Cancer					
Gene Symbol	Mean	SD	T-Test	#Num of OEL Cases	#Num of UEL Cases
AGR2	0.651045	1.847749	0.000924	40	26
ANF32B	0.247716	0.684424	0.002032	47	27
APC	0.515107	0.635663	0.01499	18	10
ARAF1	-0.07772	0.608734	0.377815	6	6
ARHA	0.415109	0.492116	2.73E-11	49	15
ARMET	0.32193	1.10499	0.012224	45	26
BCR	0.120356	0.682627	0.120777	42	31
BUB1	1.650833	1.37886	5.58E-15	62	4
CCNA2	1.499014	1.698417	2.8E-10	59	12
CDK4	0.795081	0.805853	1.46E-12	63	11
CTBP1	0.085833	0.526258	0.154207	26	19
CTNNB1	0.433792	0.580022	3.07E-07	45	5
CTSL	-0.17262	0.692472	0.086051	48	2
CTSL	0.536085	0.95307	2.07E-06	64	0
CTSL	0.54	0.7516	0.292804	28	28
PML				19	16
RAF1				5	3
RET	0.486129	0.577609	0.176132	34	16
RPS19	0.18307	0.454111	0.006364	52	19
RPS27	0.43738	0.654559	7.35E-06	48	24
RPS3	0.283549	0.7216	0.002769	43	28
SETDB1	0.13993	0.617968	0.031084	19	2
SHC1	1.336286	0.950199	1.08E-06	33	33
SKIV2L	0.132778	0.690033	0.055185	61	12
TACC3	0.874425	0.954005	9.96E-11	48	26
TAZ	0.187144	0.987323	0.056506	34	33
TFP3	0.036249	0.900516	0.385303	58	16
TFP52	0.643905	0.817825	8.5E-08		

表一、致癌基因的肝癌差異性基因表現表

Tumor Suppression Gene in Liver Cancer					
Gene Symbol	Mean	SD	T-Test	#Num of OEL Cases	#Num of UEL Cases
BNIP3L	-0.43858	0.772753	1.74E-06	17	57
BRCA2	-0.80919	0.709547	0.005187	1	12
BTG2	-0.20438	1.190317	0.067086	37	37
CDKN1A	-0.75982	1.470456	0.000368	22	43
CDKN2A	-0.13028	0.909235	0.124133	30	44
COL18A1	-0.34945	0.998815	0.003446	28	46
CUL1	-0.02784	0.644664	0.407262	33	41
CUL3	-0.19098	0.681247	0.086051	9	13
CUL4A	0.021693	0.659364	0.407679	32	43
DAB2	-0.04172	1.040531	0.391847	24	31
DGKA	-0.32868	1.292807	0.016243	19	40
JUNB	-1.42342	1.088474	1.9E-16	4	58
MEN1	-0.34264	1.199036	0.007794	27	46
NF2	-0.05353	1.257971	0.348224	39	34
PLA2G2A	-1.71512	3.313052	1.45E-05	21	52
PTEN	-0.12759	0.792619	0.11876	38	36
RAGA	-0.277	0.623975	9.85E-05	23	48
RAN	-0.23704	0.762237	0.007525	30	44
RBM6	0.015685	0.651023	0.439804	39	34
TEK	-1.03511	0.881038	1.34E-14	7	67
TGFB1	-0.37579	0.963309	0.148383	2	5
TP53	0.020887	0.902702	0.432547	34	34

表二、抑癌基因的肝癌差異性基因表現表

參考文獻

- [1] W.C. Hahn and R.A. Weinberg, "Modelling the molecular circuitry of cancer," *Nat Rev Cancer*. 2(5), pp. 331-41, 2002.
- [2] A. Kramer, K. Neben and A.D. Ho, "Centrosome replication, genomic instability and cancer," *Leukemia*, 16(5), pp. 767-75, 2002.
- [3] S.E. Andrew, and A.C. Peters, "DNA instability and human disease," *Am J Pharmacogenomics*. 1(1), pp. 21-8, 2002.
- [4] I.M. Frayling, "Methods of molecular analysis: mutation detection in solid tumours," *Mol Pathol*. 55(2), pp. 73-9, 2002.
- [5] C. Bernstein, H. Bernstein, C.M. Payne and H. Garewal, "DNA repair/pro-apoptotic dual-role proteins in five major DNA repair pathways: fail-safe protection against carcinogenesis," *Mutat Res*. 511(2), pp. 145-78, 2002.
- [6] L.S. Kirschner, "Signaling pathways in adrenocortical cancer," *Ann N Y Acad Sci*. 968, pp. 222-39, 2002.
- [7] T. Skorski, "Oncogenic tyrosine kinases and the DNA-damage response," *Nat Rev Cancer*. 2(5), pp. 351-60, 2002.
- [8] Y. Murakami, "Functional cloning of a tumor suppressor gene, TSLC1, in human non-small cell lung cancer," *Oncogene*. 21(45), pp. 6936-48, 2002.
- [9] S.J. Xia, J.G. Pressey and F.G. Barr, "Molecular pathogenesis of rhabdomyosarcoma," *Cancer Biol Ther*. 1(2), pp. 97-104, 2002.
- [10] M. Esteller, "CpG island hypermethylation and tumor suppressor genes: a booming present, a brighter future," *Oncogene*. 21(35), pp. 5427-4, 2002.
- [11] J.P. Tsai, J.G. Chang, S.H. Shih, R.M. Chen, H.W. Hsiao, R.M. Hu, S.N. Chen, M.M. Lee, F.M. Liu, and W.L. Chan, "A Framework for Cancer-related Genes Mining over the Internet," *Proc. IEEE 3rd International Symposium on Bioinformatics and Bioengineering*, Washington DC, USA, March 10-12, 2003.
- [12] J.P. Tsai, J.G. Chang, S.H. Shih, R.M. Chen, H.W. Hsiao, R.M. Hu, S.N. Chen, M.M. Lee, W.L. Chan, P.X. Wu, W.H. Chen, S.S. Shen, F.M. Liu and Y.C. Lin, "A Bioinformatics Approach for Exploring Cancer-related Genes," *Proc. Workshop on Bioinformatics, held in conjunction with 19th Int'l Conf. on Data Engineering*, Bangalore, India, March 5-8, 2003.
- [13] Sargent TD, "Isolation of differentially expressed genes," *Methods Enzymol*. 152: 423-432, 1987.
- [14] Kishimoto TK, O'Connor K, Lee A, Roberts TM and Springer TA, "Cloning of the b subunit of the leukocyte adhesion proteins: homology to an extracellular matrix receptor defines a novel supergene family," *Cell* 48: 681-690, 1987.
- [15] Hogervorst F, Kuikman I, von dem Borne AE and Sonnenberg A, "Cloning and sequence analysis of beta-4 cDNA: an integrin subunit that contains a

- unique 118 kD cytoplasmic domain,” *EMBO J.* 9: 765-770, 1990.
- [16] Suzuki S, Huagn ZS and Tanihara H, “Cloning of an integrin b subunit exhibiting high homology with integrin b3 subunit,” *Proc. Natl. Acad. Sci. USA* 87: 5354-5358, 1990.
- [17] Liang P and Pardee AB, “Differential display of eukaryotic messenger RNA by means of the polymerase chain reaction,” *Science* 257: 967-971, 1992.
- [18] Liang P, Averboukh L and Pardee AB, “Distribution and cloning of eukaryotic mRNAs by means of differential display: refinements and optimization,” *Nucleic Acid Res.* 21: 3269-3275, 1993.
- [19] Heermann KH, Hagos Y and Thomssen R, “Liquid-phase hybridization and capture of hepatitis B virus DNA with magnetic beads and fluorescence detection of PCR product,” *J. Virol. Methods.* 50: 43-58, 1994.

以調控序列叢集分析人類啟動子區域

陳士農^{1,2} 蔡進發¹ 林俊谷² 黃志瑋¹

¹ 臺中健康暨管理學院資訊科技學系

² 臺中健康暨管理學院生物資訊學系

413 台中縣霧峰鄉柳豐路 500 號

{nung, jptsai}@thmu.edu.tw {std91242005, std91241007}@ms1.thmu.edu.tw

摘要

由於人類基因體計畫(Human Genome Project)提供了完整的人類基因體序列,使用基因體序列資訊來分析基因功能的相關研究,已經在世界各地展開。利用基因體序列資料我們能夠進行基因功能研究,基因可以分成數個結構部份,在這些部份中,對基因表現(gene expression)來說最重要的部份是啟動子(promoter)。本論文提出一個在未知序列中,預測分析啟動子的方法,本研究利用調控蛋白結合部位(regulatory protein binding site)的位置權重矩陣(position weight matrix)來統計調控蛋白對序列結合的強度與位置的相關性,再加上 G+C 含量(G+C content)與 CpG 雙核苷酸(CpG dinucleotide)的數量,利用線性回歸的方法,計算出三個線性分析函數,用於預測人類啟動子在基因序列中的位置,除此之外,還能對啟動子附近的區域進行分析,如核心啟動子(core promoter)的位置、調控蛋白結合部位(regulatory protein binding site)的位置。最後,我們的方法與其他研究比較預測的結果,證實我們的方法的確比較優秀。

關鍵字：啟動子(promoter)、轉錄因子結合部位(transcription factor binding site)、位置權重矩陣(position weight matrix)、調控蛋白結合部位(regulatory protein binding site)、RNA 聚合酶(RNA polymerase)

1. 前言

隨著人類基因體計畫的進行,有越來越多的研究人員投入生物資訊的研究行列,不管是序列預測、藥物研發、或是疾病治療等,都是重要的研究課題。就疾病來說,病症的出現可能是因為代謝途徑(metabolic pathway)發生錯誤或是基因表現的產物—蛋白質(protein)之間相互影響所導致,若要從基因表現層面來研究相關疾病的發病或症狀,可以藉由探索基因表現機制來達成此一目的。但如果一開始就直接利用生物實驗來驗證,將會花費相當大的人力與物力;若先經由生物資訊方法的研究分析,在進行生物實驗之前,將可能的變因範圍縮小,並獲取所需的相關資訊做為參考,必能減少錯誤實驗所導致的浪費,並加速研究的進行。

在公共領域(public domain)的資料庫中, mRNA 的數量遠超過啟動子的數量,這是因為 mRNA 的資料可以合併 dbEST [1]推算出 mRNA 的位置,目前人類的 EST (Expressed Sequence Tags) [2]的數量已經超過 500 萬筆,反觀啟動子必須透過生物實驗驗證—足跡法(footprinting) [3],才能確定啟動子的區域,因此,如果有一個準確率很高的預測分析程式可針對基因進行分析,將可促進對生物轉錄機制的了解。

在基因表現的過程中,從 DNA 轉錄(transcription)成 RNA,再由 RNA 轉譯(translation)

成蛋白質，每個階段都有相對應的調控機制[4]，這也是生物體內最重要的過程。然而，此過程非常複雜且難以理解，因此，若是現有的資料庫中，能夠提供啟動子的相關資訊，生物學家就可以從轉錄的階段進行研究，這將會更準確且更有效率。啟動子對轉錄的重要性在於它可以決定在轉錄的過程中，RNA 聚合酶(RNA polymerase)會以那一股為模板進行轉錄，以及轉錄的起始位置，此外，啟動子也會影響 RNA 聚合酶在 DNA 上面的結合強度，這將會影響整個轉錄的效率。

在真核生物的基因當中，有三種 RNA 聚合酶能與基因發生作用，分別是 RNA 聚合酶 I、RNA 聚合酶 II、RNA 聚合酶 III，且不同的 RNA 聚合酶會產生不同的產物，分別是 rRNA、蛋白質、tRNA。在本論文中，只有針對 RNA 聚合酶 II 相關的基因進行研究，因為在生物體內幾乎所有的生理生化反應都是透過蛋白質與蛋白質之間的作用，或是蛋白質作用於 DNA 上的影響來調節生理現象。在真核細胞典型的轉錄機制中，RNA 聚合酶 II 並不能直接找到基因的位置進行轉錄過程，必須經由轉錄因子的協助，才能順利辨識位於 DNA 上的基因位置進行轉錄。在 2003 年，Stephen T. Smale [5]說明在果蠅與人類的核心啟動子中，會出現特定的共同序列，如 BRE、TATA box、Inr、DPE 等，透過這些特殊的共同序列可以做為預測啟動子的基準，當成尋找啟動子的線索[6] [7] [8]。

在 2002 年，Jennifer [9]的研究中指出，在一組 205 筆啟動子的資料中，有 29%含有 TATA box，26%含有 DPE，14%同時含有 TATA box 與 DPE，31%是兩者都沒有。而沒有 TATA box 與 DPE 的基因，它的轉錄起始機制將是我們的研究重點，究竟是何種機制，才能使沒有 TATA box 與 DPE 的基因進行轉錄？因此，我們懷疑在不同物種間的同源基因中，是否也有不同的轉錄機制出現？或是在相同基因家族內，是否也有相同的轉錄機制？在公共領域的真核啟動子資料庫(Eukaryotic Promoter

Database, EPD) [10]中，記錄了將近 3000 筆真核生物的啟動子資料，其中人類的啟動子資料大約有 1871 筆，在這個資料庫中的轉錄起始點資料是經由生物實驗驗證所得。而在人類啟動子資料庫(Human Promoter Database, HPD) [11]中，則記錄了 2004 筆的啟動子資料，然而進行啟動子研究如果只利用這些資料來分析，這樣的數量是不夠的，因此，一個具有高準確性的啟動子預測程式，對調控機制的研究將有極大的幫助。

2. 方法

2.1 資料收集

在本研究中，收集的資料可分為兩大類，第一類為啟動子資料，第二類為非啟動子資料。啟動子資料有兩個主要的來源，其一是從真核啟動子資料庫中取得，其版本號碼為 77，擷取長度為 250bp 的人類啟動子序列，範圍為上游-199bp 到下游+50bp，總共有 1871 筆序列資料，這些序列包含了轉錄起始點(transcription start site)的位置。另一個啟動子資料來源是人類啟動子資料庫，擷取的範圍同樣是從上游-199bp 到下游+50bp 的位置，資料共有 2004 筆。非啟動子資料則取自 NCBI 的 GenBank [12]資料庫與 RefSeq [13]資料庫，序列組合版本號碼為 34。

為了避免與啟動子區域重疊，從序列資料庫中擷取正股基因 CDS (coding sequence)的 DNA 序列，選取的範圍是從轉譯起始點(translation start site)往下游+1501bp 到+1750bp 的區域。我們從人類第一號到第二十二號染色體的正股上，總共取得 11731 筆非啟動子序列，為了取得與 3875 筆啟動子序列相同數量的非啟動子序列，我們由 11731 筆非啟動子序列中，隨機選取 10 組 3875 筆的非啟動子序列做為訓練之用。最後，為了與其他研究進行比較分析，我們採用 Matthias Scherf 在 PromoterInspector [14]中使用的其中一組序列，其

編號為 AC002397，總長度為 227538bp，含有 17 個轉錄起始點。

2.2 位置權重矩陣

本研究所使用的調控蛋白結合部位的權重矩陣包含了 ap1、ccaat、cre、e2f、ere、ets、gata、lsf、mef2、myc、myf、nfl、sp1、srf、tata、tef 等 16 個[15][16][17][18][19][20][21][22]。我們先將每個位置出現的次數除以全部出現的次數以轉換成頻率，由於背景的機率是 0.25，所以將頻率再除以 0.25 以得到勝率分數(odds score)，最後，將勝率分數轉換成以 2 為底的對數值，做為最後的權重矩陣，然後再利用這些權重矩陣來計算不同位置的分數。

在 16 個權重矩陣中，寬度並不一定相同，範圍則從 8 個鹼基到 16 個鹼基。在本研究中，我們使用固定長度的方法，因為我們發現在權重矩陣中，單一核苷酸出現次數較高的共同長度大約為 8 個鹼基，也就是說每個權重矩陣中，結合強度最好的範圍是集中在中間 8 個鹼基的位置。在權重矩陣中，由於某些位置的核苷酸出現次數為 0，當轉換成對數勝率時會變成無限小的值，在這裡我們直覺地使用 -10 來取代無限小的值，原因是如果出現一次 -10 的情形，除非其他的核苷酸分數非常高，否則，這一組序列的分數成為前五大的機率是非常小的。

2.3 調控蛋白結合部位計分

我們使用一個 8bp 的滑動窗(sliding window)在 250bp 長度的序列中，利用不同的調控蛋白結合部位的權重矩陣，計算每個位置的分數，並記錄前五大的分數與位置。不同的調控蛋白結合部位會在不同的位置與不同的調控蛋白結合，只有在正確位置的調控蛋白結合部位會有較高的分數，例如，起始子(initiator)的位置在轉錄起始點的附近幾個鹼

基，如果在位置內，將會有較高的分數，如果在可能的位置外，則分數將會很低。在整個計算過程中，我們使用 16 個調控蛋白結合部位的權重矩陣，分別得到前五大分數與位置，而且前五大分數的值必須大於 0，如果沒有大於 0 則以 0 計算。因此，可以獲得兩組 80 個分數與位置的變數值，再加上 CpG 雙核苷酸與 G+C 含量比例，總共會有 162 個變數可以用來判別啟動子的位置與調控蛋白結合部位的分佈情形。

2.4 線性分析函數(Linear Analysis Function)

將分數、位置、CpG 雙核苷酸所組成的變數值分別使用線性回歸(linear regression)方法，使得兩群啟動子與非啟動子的分數到分界線的垂直距離為最小，也就是使得所有序列中的前五大調控蛋白結合部位的分數、位置、CpG 雙核苷酸的分數與這條分界線的垂直距離平方和為最小，分界線公式如下：

$$LSF(S_i) = \sum_{i=1}^{80} A_i \cdot S_i + C_s \quad (1)$$

其中， $LSF(S_i)$ 為線性分數函數， S_i 是由權重矩陣計算產生的前五大分數值， A_i 為係數， C_s 為常數。

$$LPF(P_j) = \sum_{j=1}^{80} A_j \cdot P_j + C_p \quad (2)$$

其中， $LPF(P_j)$ 為線性位置函數， P_j 是由權重矩陣計算產生的前五大分數的位置， A_j 為係數， C_p 為常數。

$$LCF(C_k) = \sum_{k=1}^2 A_k \cdot C_k + C_c \quad (3)$$

其中， C_1 為 G+C 的比例， C_2 為 CpG 雙核苷酸的數量， A_k 為係數， C_c 為常數。

為了測試上述三個線性分析函數的準確性，我們將三個線性分析函數的計算結果與其他預測程式相互比較，這些程式包含了PromoterInspector [14]、TSSG [23]、TSSW [24]、NNPP 2.1 [25]、Promoter 2.0 [26]等，所使用的測試序列為Matthias Scherf在PromoterInspector [14]中使用的其中一組序列，其編號為AC002397，總長度為227538bp，含有17個轉錄起始點。

3. 結果

3.1 訓練資料組(Training Set)與測試資料組(Testing Set)

選取真核啟動子資料庫與人類啟動子資料庫組成具有啟動子的序列資料共有 3875 筆，另外在 NCBI 的 GenBank 資料庫與 RefSeq 資料庫中，從 11731 筆資料中隨機選取 10 組 3875 筆序列做為訓練使用。在測試資料組中，由 PPR 的 4737 筆組成，長度分別為 250bp 與 3001bp，以及一組長度為 227538bp 的序列。

3.2 訓練資料組的 G+C 比例與 CpG 雙核苷酸的數量

表1. 訓練資料組的G+C比例與CpG雙核苷酸的數量

	啟動子		非啟動子
	LCF 相關	LCF 非相關	
G+C 平均值	0.68289	0.51509	0.451751
G+C 變異數	0.06913	0.09198	0.103749
CpG 平均值	26.0498	6.40941	3.683318
CpG 變異數	6.85115	4.66613	0.103667

表 1 顯示在啟動子相關的序列中，平均的 G+C 比例均大於 50%，非啟動子的 G+C 比例只有 45%。在 CpG 雙核苷酸的數量上，啟動子的 CpG 雙核苷酸平均數遠大於非啟動子的 CpG 雙核苷酸

平均數，與 LCF 函數相關的啟動子，CpG 雙核苷酸的平均數為 26.05，標準差為 6.85，與 LCF 函數非相關的啟動子，CpG 雙核苷酸的平均數為 6.41，標準差為 4.67，非啟動子相關序列的 CpG 雙核苷酸平均數為 3.68，標準差為 0.1。

3.3 線性分析函數的係數比較

表2. 兩個線性分析函數的變數、係數分析比較表

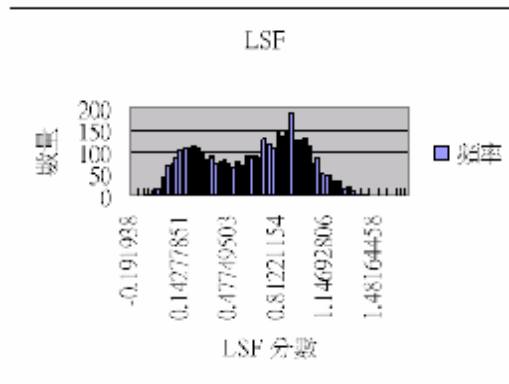
	$A_i/A_j, A_k$ 大於0的數量	$A_i/A_j, A_k$ 小於0的數量	$A_i/A_j, A_k$ 大於0的變數類型
LSF	36(12)	44(14)	cre, e2f, mef2, myc, spl, srf, tata
LPF	45(5)	38(1)	cre, e2f, ere

表 2 顯示 LSF 與 LPF 兩個線性分析函數的係數比較情形。在 LSF 中，大於 0 的係數有 36 個，其中 12 個大於 0.01，最大為 0.45759；小於 0 的係數有 44 個，其中 14 個大於 -0.01，最大為 -0.051852，影響較大的變數類型分別為 cre、e2f、mef2、myc、spl、srf、tata。在 LPF 中，大於 0 的係數有 45 個，其中 5 個大於 0.001，最大為 0.001417；小於 0 的係數有 38 個，其中 1 個大於 -0.001，最大為 -0.001278，影響較大的變數類型分別為 cre、e2f、ere。

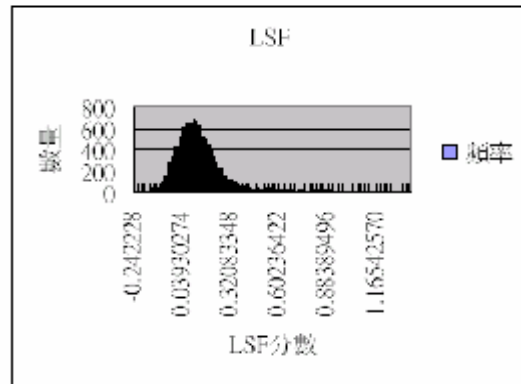
3.4 訓練資料組分析

在圖 1 的(A) (B) (C)中，啟動子資料共有 3875 筆，另外可將啟動子資料分成左右兩群，左邊的資料與 LSF、LPF、LCF 相關程度較低，右邊的資料與 LSF、LPF、LCF 相關程度較高。在圖 1 的(D) (E) (F)中，非啟動子資料共有 11731 筆，LSF、LPF、LCF 的平均值也遠小於啟動子 LSF、LPF、LCF 的平均值，兩者相距超過 2 倍的標準差，非啟動子平均值+2 倍非啟動子標準差<0.43<啟動子平均值+2 倍啟動子標準差，所以，我們取輸出值大於 0.5 的序列為啟動子序列，輸出值大於 0.5 可以降低誤判非啟動子為啟動子的機率。

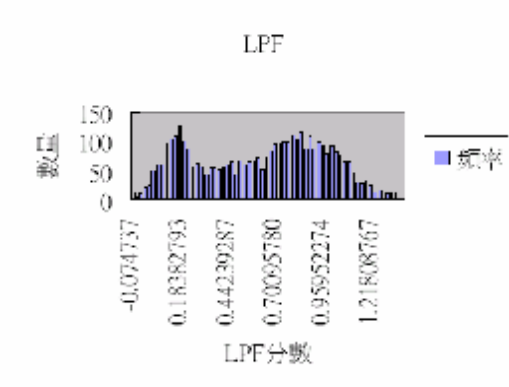
(A)



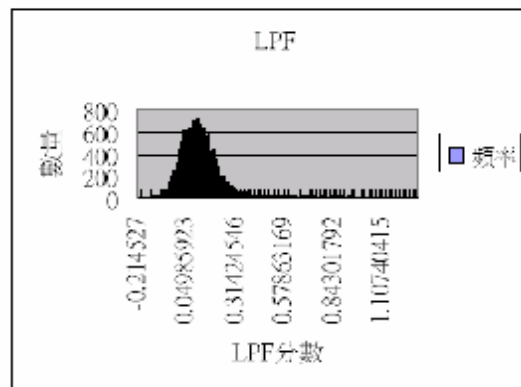
(D)



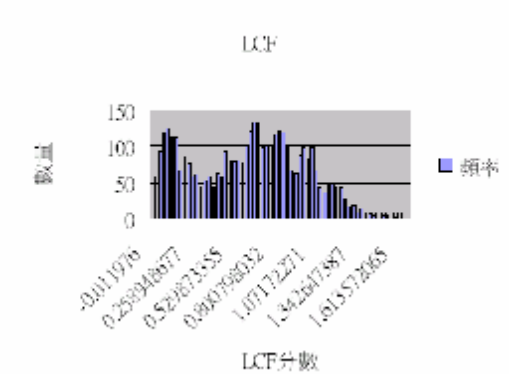
(B)



(E)



(C)



(F)

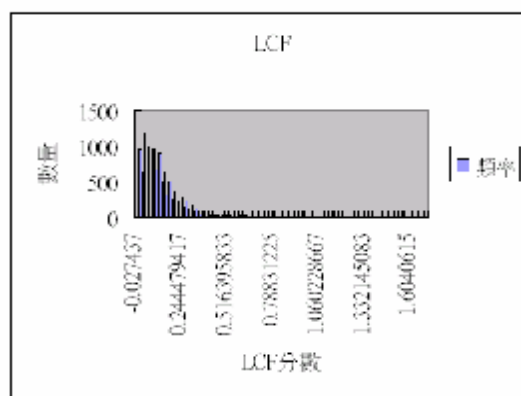


圖 1. 三個線性分析函數輸出值次數分佈圖，此例以所有訓練資料為輸入值，(A)~(C)為啟動子分佈圖，(D)~(F)為非啟動子分佈圖，(A)、(D)為 LSF，(B)、(E)為 LPF，(C)、(F)為 LCF。(A)以 0.5 為分界線，左邊區域平均值為 0.237738，變異數為 0.141387，右邊區域平均值為 0.853385，變異數為 0.190675。(B)以 0.5 為分界線，左邊區域平均值為 0.227332，變異數為 0.188471，右邊區域平均值為 0.842643，變異數為 0.188471。(C)以 0.5 為分界線，左邊區域平均值為 0.238256，變異數為 0.136892，右邊區域平均值為 0.84626，變異數為 0.205104。(D)平均值為 0.125604，變異數為 0.426452。(E)平均值為 0.129683，變異數為 0.431492。(F)平均值為 0.125935，變異數為 0.426624。

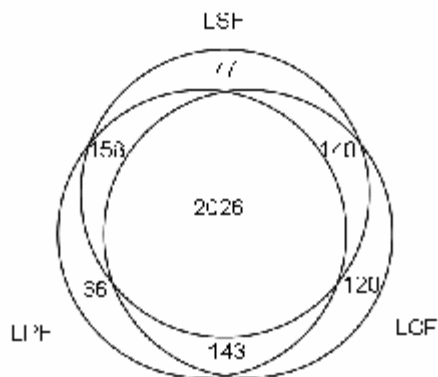


圖 2. 三個線性分析函數輸出值大於 0.5 的資料交互情形。

圖 2 顯示在訓練資料中，經由三個線性分析函數輸出值大於 0.5 的資料交互情形，LSF 共有 2401 筆，LPF 共有 2393 筆，LCF 共有 2429 筆，LSF 與 LPF 同時大於 0.5 的資料共有 2184 筆，LSF 與 LCF 同時大於 0.5 的資料共有 2166 筆，LPF 與 LCF 同時大於 0.5 的資料共有 2169 筆，LSF、LPF 與 LCF 同時大於 0.5 的資料共有 2026 筆。

3.5 預測結果分析

表 3. 預測結果比較表

序列編號	方法	TP	FP
AC0902397	TSSG	8	97
	TSSW	6	101
	NNPP 2.1	10	658
	Promoter 2.0	5	455
	PromoterInspector	5	1
	Our method	8	3

表 3 是我們的方法與其他研究的預測結果比較表，在我們的方法中，只有正股序列接受測試，反股序列並沒有加入測試，在正股序列中，共有 13 個轉錄起始點，我們預測了 11 個調控序列叢集，其中有 8 個包含了轉錄起始點。在其他研究中，PromoterInspector [14]預測了 6 個轉錄起始點，其中有 5 個正確，TSSG [23]預測了 105 個轉

錄起始點，其中有 8 個正確，TSSW [24]預測了 107 個轉錄起始點，其中有 6 個正確，NNPP 2.1 [25]預測了 668 個轉錄起始點，其中有 10 個正確，Promoter 2.0 [26]預測了 560 個轉錄起始點，其中有 5 個正確。

4. 討論

我們利用真核啟動子資料庫與人類啟動子資料庫做為訓練資料組，並沒有去除部份沒有關聯的資料，而是全部用來訓練，經過多次訓練的結果，雖然無法將所有的啟動子與非啟動子完全分成兩群，但已經能將啟動子序列分成與線性分析函數相關與非相關兩群資料，由於與線性分析函數非相關啟動子與非啟動子資料較為相似，而無法使用本研究所提出的三個線性分析函數來區分，因此，線性函數非相關的啟動子序列仍是非常值得研究的部份。

在 LCF 分析函數中，輸出值大於 0.5 的資料可以發現 CpG 雙核苷酸的數量在 15 以上，G+C 的比例則超過 50%，這個結果與其他以 CpG islands 為判別標準的研究大致相符 [27] [28] [29]。利用本研究所提出的三個線性分析函數交互確認啟動子區域以縮小預測範圍，雖然速度較慢，但是準確率卻可提高，因此，我們還是決定使用三個線性分析函數交互確認的方式做為啟動子區域的預測法則。最後，與其他研究比較的結果，證實我們的方法確實有效。

5. 結論

在我們提出的方法中，的確能藉由三個線性分析函數的交互確認來確定啟動子區域，雖然還不能確定啟動子區域的起始與結束的位置，卻已經能夠從大範圍的序列中，找到轉錄起始點的位置，與其他研究的方法比較，我們的方法也有較高的敏感度。

参考文献

- [1] Boguski MS, Lowe TM, Tolstoshev CM. (1993) dbEST - database for expressed sequence tags *Nature Genetic*, **4**, 332-333.
- [2] Adams MD, Kelley JM, Gocayne JD, Dubnick M, Polymeropoulos MH, Xiao H, Merrill CR, Wu A, Olde B, Moreno RF, et al. (1991) Complementary DNA sequencing: expressed sequence tags and human genome project. *Science*, **252**, 1651-1656.
- [3] Komura J, Ono T. (2003) Nucleosome Positioning in the Human c-FOS Promoter Analyzed by in Vivo Footprinting with Psoralen and Ionizing Radiation. *Biochemistry*, **42**, 15084-15091.
- [4] Gerhard Krauss. (2001) *Biochemistry of Signal Transduction and Regulation*, 2nd edition Wiley-VCH Verlag GmbH press, pp 1-88.
- [5] Stephen T. Smale and James T. Kadonaga. (2003) The RNA Polymerase II Core Promoter. *Annu. Rev. Biochemistry*, **72**, 449-479.
- [6] Uwe Ohler, Guo-chun Liao, Heinrich Niemann and Gerald M Rubin. (2003) Computational analysis of core promoters in the *Drosophila* genome, *Genome Biology*, **12**, 87-98.
- [7] James T. Kadonaga. (2002) The DPE, a core promoter element for transcription by RNA Polymerase II, *EXPERIMENTAL and MOLECULAR MEDICINE*, **34**, 259-264.
- [8] ALAN K. KUTACH and JAMES T. KADONAGA. (2000) The Downstream Promoter Element DPE Appears To Be as Widely Used as the TATA Box in *Drosophila* Core Promoters, *MOLECULAR AND CELLULAR BIOLOGY*, **20**, 4754-4764.
- [9] Jennifer E.F. Butler and James T. Kandonaga. (2002) The RNA polymerase II core promoter: a key component in the regulation of gene expression, *Genes & Development*, **16**, 2583-2593.
- [10] Cavin Perier R, Praz V, Junier T, Bonnard C, Bucher P. (2000) The Eukaryotic Promoter Database(EPD). *Nucleic Acid Res*, **28**, 302-303.
- [11] <http://zlab.bu.edu/~mfrith/HPD.html>
- [12] Benson DA, Karsch-Mizrachi I, Lipman DJ, Ostell J, Wheeler DL. (2003) GenBank. *Nucleic Acids Research*, **31**, 23-27.
- [13] Pruitt KD, Maglott DR. (2001) RefSeq and LocusLink: NCBI gene-centered resources. *Nucleic Acids Research*, **29**, 137-40.
- [14] Matthias Scherf, Andreas Klingenhoff and Thomas Werner. (2000) Highly Specific Localization of Promoter Regions in Large Genomic Sequences by PromoterInspector: A Motif Context Analysis Approach. *Journal of Molecular Biology*, **297**, 599-606.
- [15] Martin C. Frith, Ulla Hansen and Zhiping Weng. (2001) Detection of cis-element clusters in higher eukaryotic DNA. *Bioinformatics*, **17**, 878-889.
- [16] Bucher P. (1990) Weight matrix descriptions of four eukaryotic RNA polymerase II promoter elements derived from 502 unrelated promoter sequences. *Journal of Molecular Biology*, **212**, 563-578.
- [17] Mantovani R (1999) The molecular biology of the CCAAT-binding factor NF-Y, *Gene* **239**, 15-27.
- [18] Matys,V., Fricke,E., Geffers,R., Kel-Margoulis,O.V. et al. (2003) TRANSFAC: transcriptional regulation, from patterns to profiles. *Nucleic Acids Research*, **31**, 374-378.

- [19] Mimeault M. (2000) Structure-function studies of ETS transcription factors. *Critical Reviews in Oncogenesis*, **11**, 227-253.
- [20] O'Lone R, Frith MC, Hansen U (in preparation).
- [21] Grandori C & Eisenman RN. (1997) Myc target genes, *TIBS*, **22**, 177-181.
- [22] Wasserman WW, Fickett JW. (1998) Identification of regulatory regions which confer muscle-specific gene expression, *Journal of Molecular Biology*, **278**, 167-81.
- [23] Kel AE, Kel-Margoulis OV et al. (2001) Computer-assisted identification of cell cycle-related genes: new targets for E2F transcription factors. *Journal of Molecular Biology*, **309**, 99-120.
- [24] Solovyev, V. & Salamov. (1997) A. The gene-finder computer tools for analysis of human and model organisms genome sequences. *ISMB*, 294-302.
- [25] Reese MG. (2001). Application of a time-delay neural network to promoter annotation in the *Drosophila melanogaster* genome, *Computer Chem*, **26**, 51-56.
- [26] Kundsén, S. (1999), Promoter 2.0: for the recognition of Pol II promoter sequences. *Bioinformatics*, **15**, 356-361.
- [27] Ioshikhes I and Zhang MQ. (2000) Large-scale human promoter mapping using CpG islands. discrimination. *Nature Genetics*, **26**, 61-63.
- [28] Leonardo Marino-Ramirez, John L. Spouge, Gavin C. Kanga and David Landsman, (2004) Statistical analysis of over-represented words in human promoter sequences, *Nucleic Acids Research*, **32**, 949-958.
- [29] Loic Ponger and Dominique Mouchiroud, (2002) CpGProD : identifying CpG islands associated with transcription start sites in large genomic mammalian sequences, *Bioinformatics*, **19**, 31-633.

On the uniform edge-partition of a tree

Bang Ye Wu*

吳邦一

Shu-Te University

Hung-Lung Wang

王弘倫

National Taiwan University

Shih Ta Kuan

管世達

Shu-Te University

Kun-Mao Chao

趙坤茂

National Taiwan University

Abstract

We study the problem how uniformly one can partition the edge set of a tree with n edges into k connected components, where $k \leq n$. The objective is to minimize the ratio of the maximum to the minimum number of edges of the subgraphs in the partition. We show that, for any tree and $k \leq 4$, there exists a k -split with ratio at most two. For general k , we propose a simple algorithm that finds a k -split with ratio at most three in $O(n \log k)$ time. Experimental results on random trees are also shown.

1 Introduction

In this paper, we study the problem of splitting a tree into k parts with approximately equal number of edges in each part subject to that the edges in each part are connected. How well can one do it?

More formally, we define a k -split of a tree T as follows. Let T be a tree and $1 \leq k \leq e(T)$. A k -tuple $(T_1, T_2, \dots, T_k)$ is a k -split of T if (1) each T_i is a connected subgraph of T ; and (2) T_i and T_j are edge disjoint for $i \neq j$; and (3) the union of all the subgraphs form the whole tree T . To measure how equal a partition is, three natural objectives are usually used.

- To minimize the maximum (min-max).
- To maximize the minimum (max-min).
- To minimize the ratio of the maximum to the minimum (min-ratio).

For partitioning into two parts, all the three objectives are equivalent, but they differ when the number of parts is larger than two. For the k -split of a tree, the worst cases for both min-max and max-min objectives are trivial. However, it is much involved for the min-ratio problem. We

show that, in the algorithmic aspect, to find the optimal k -split of a tree with respect to each of the three objectives is NP-hard, even for unweighted tree. We focus on the worst case analysis of the ratio, and prove that, for any tree and $k \leq 4$, there exists a k -split with ratio at most two. For general k , we propose a simple algorithm that finds a k -split with ratio at most three in $O(n \log k)$ time. Experimental results on random trees are also shown.

Graph partition is an important problem in computer science. It finds applications in parallel computing, data storage and segmentation, and operation research. Most of the previous research was devoted to the *vertex partition*, and many variants of the problem have been defined and investigated with different objectives and constraints. Most of the problems on general graphs are NP-hard [1, 10]. For the vertex partition of a tree, polynomial time algorithms for both the min-max and the max-min objectives were developed [5, 7, 15, 17]. Becker and Perl [6] summarized their previous results with some other co-authors and showed that the tree vertex partition problem of several other objective functions can also be solved by using the shifting algorithm. An open problem in that paper is the *most uniform vertex partitioning* problem for trees, in which the objective is to minimize the difference of the maximum and the minimum weights of the vertex set in the partition. For a special case that the tree is a path, a solution was given in [16]. One can image that the problem is more difficult than the min-max or max-min problem since both the smallest and the largest parts are concerned. In this paper, we shall see that it is similar for the edge partition problem. In addition to the algorithmic aspect, some properties of the tree vertex partition have also been studied [9, 14].

The study on edge partition is helpful for the multiserver routing problem. In such a problem, we are given a network, k identical servers, a set of demand points, and a home location. The servers are required to visit some set of demand points

*Corresponding author

and return back to the home in a “fair” way such that each demand point is visited by at least one server. In [2], Averbakh and Berman showed that the problem is NP-hard even for an edge weighted tree network and $k = 2$. They gave approximation algorithms for the problem and a variant that the two servers have different home locations. The authors further developed approximation algorithms for the case of k servers and some other variants of the problem [3].

Another application of the tree edge partition is for Multiple Sequence Alignments (MSA), which is important in computational biology. To align a set of n sequences, we insert gaps into the sequences and arrange their characters into columns with n rows, one from each sequence. The MSA problem has typically been formalized as optimization problems with different objectives. The *sum-of-pairs* or SP objective for multiple alignment is to minimize the sum, over all pairs of sequences, of the pairwise distance between them in the alignment. The SP-alignment problem is NP-hard. A *progressive* approach for MSA incrementally builds the alignment by inserting sequences one at a time but the resulting SP-cost heavily depends on the inserting order and there is no performance guarantee. One way to get an alignment with worst case guarantee is to employ a tree, in which each vertex represents one sequence, and perform the progressive alignments along the tree edges [4, 11, 18, 20]. More details about MSA and algorithms for constructing such a guide tree are referred to [12, 19]. For n sequences, each with length m , the time complexity of such an approach is $O(nm^2)$, and it may be very time consuming for large n and m . Since one tree edge corresponds to performing a pairwise alignment, a k -split of the tree partitions the whole work into k parts and derives a parallel algorithm for the problem. To balance the working load, a k -split with small ratio should be applied.

The remaining sections are organized as follows. In Section 2, we define some notations, explain the computational complexity of the problem, and show some preliminary results. The worst cases of the min-ratio for $k = 3$ and $k = 4$ are discussed in Section 3. In Section 4, we show a simple algorithm for general k with ratio at most three, and present some experimental results. Finally, concluding remarks are given in Section 5.

2 Notations and Preliminaries

Let $E(T)$ denote the edge set of a tree T and $e(T)$ denote the number of edges of tree T . Throughout this paper, $n = e(T)$. An edge with endpoints u and v is denoted by (u, v) . Let T be a rooted tree and v be a vertex of T . We use T_v to denote the subtree rooted at v , i.e., the subgraph induced on v and all its descendants. Let u be a child of v . The subgraph $T_u \cup (u, v)$ is called a *branch* of v .

Definition 1: Let T be a tree, $1 \leq k \leq e(T)$, and $p \geq 1$. The ratio of a k -split $(T_1, T_2, \dots, T_k)$ of T is defined by $\frac{\max_i \{e(T_i)\}}{\min_i \{e(T_i)\}}$.

By $T = A \uplus B$, we denote that T is split into A and B , i.e., the edge sets of the two subgraphs form a partition of $E(T)$. It is also noted that A and B share a common vertex if $T = A \uplus B$. By $T = A \uplus B \uplus C$, we understand a 3-split (A, B, C) of T , in which B intersects with both A and C . It includes the case that the three subgraphs share a common vertex.

PROBLEM: Minimum Ratio k -Split

INSTANCE: Tree T and an integer $1 < k < e(T)$.

GOAL: Find a k -split of T with minimum ratio.

The min-max and the max-min k -split problem are defined similarly except that the objectives are to minimize the maximum subgraph, and to maximize the minimum subgraph respectively. We can easily show that all the three problems are NP-hard by a simple reduction to the following problem.

PROBLEM: 3-Partition

INSTANCE: A bound $B \in \mathbb{Z}^+$ and a set A of $3m$ integers a_i , $1 \leq i \leq 3m$, satisfying $B/4 < a_i < B/2$ and $\sum_i a_i = mB$.

QUESTION: Can A be partitioned into m disjoint sets A_i , $1 \leq i \leq m$, such that $\sum_{a \in A_i} a = B$ for $1 \leq i \leq m$ (note that each A_i must therefore contain exactly three elements from A)?

Given A and B as an instance of the 3-partition problem, we construct a tree T consisting of a root r and $3m$ branches Y_i , $1 \leq i \leq 3m$, incident with the root, in which Y_i is an arbitrary tree of a_i edges. It is easy to see that there exists a m -split of T with ratio one if and only if the answer of the 3-partition problem is “yes”. Since the 3-partition

problem is NP-complete in the strong sense [10], we have the following result.

Theorem 1: The Minimum Ratio k -Split problem is NP-hard.

Obviously, the reduction remains true for the min-max and the max-min objective functions.

Corollary 2: Both the Min-Max and the Max-Min k -Split problems are NP-hard.

The next lemma appeared in [13]. For our convenience, we rewrite it and give a proof for the completeness.

Lemma 3: Let T be a rooted tree. For any $1 \leq \gamma \leq e(T)$, we can split T into (T_1, T_2) at a vertex v in linear time such that $\gamma \leq e(T_1) \leq 2\gamma$, in which v is a vertex satisfying $e(T_v) \geq \gamma$ and $e(T_u) < \gamma$ for any child u of v .

Proof: In linear time, we can traverse the tree in the post order and compute the number of edges for the subtree rooted at each vertex. Such a vertex v can be easily found while traversing the tree. Assume that $B_1, B_2, \dots, B_k$ are the branches at v . If $e(T_v) = \gamma$, we have done. Otherwise, we can find $j < k$ such that $\sum_{i=1}^{j-1} e(B_i) < \gamma$ and $\sum_{i=1}^j e(B_i) \geq \gamma$. Since $e(B_j) \leq \gamma$, we have that $\sum_{i=1}^j e(B_i) \leq 2\gamma$. The union $\bigcup_{i=1}^j B_i$ is the desired subgraph. $\square$

Taking $\gamma = n/3$ in Lemma 3, we have the following result. To show that the bounds are tight, consider a tree consisting of exact three branches, each with $n/3$ edges, incident with the centroid. A centroid of a tree is a vertex such that no branch contains more than one half of the vertices if we root the tree at its centroid.

Corollary 4: For any tree T , there is a 2-split of T with ratio at most two. The numbers of the two subgraphs are at most $2n/3$ and at least $n/3$. Furthermore, such a 2-split can be found in $O(n)$ time and the bounds are tight.

The following simple result shows an upper and a lower bounds for the sizes of the subgraphs in a k -split with a limited ratio.

Lemma 5: If $(T_1, T_2, \dots, T_k)$ is a k -split of T with ratio r , then, for each subgraph T_i , $\frac{n}{r(k-1)+1} \leq e(T_i) \leq \frac{rn}{k+r-1}$.

Proof: Let x be the number of edges of the maximum component. Since the number of edges

of the minimum component is no more than the mean of the remainder, i.e., $\frac{n-x}{k-1}$,

$$x \leq \frac{r(n-x)}{k-1}.$$

Solving the inequation, we have $x \leq \frac{rn}{k+r-1}$. Similarly, let y denote the minimum number of edges. The maximum is no less than the mean of the remainder, $\frac{n-y}{k-1}$, and we have $y \geq \frac{(n-y)}{r(k-1)}$, which implies $y \geq \frac{n}{r(k-1)+1}$. $\square$

Particularly, for $r = 2$, the number of edges of each subgraph in a k -split with ratio at most two is between $\frac{n}{2k-1}$ and $\frac{2n}{k+1}$. But the converse of the lemma is obviously not true. Corollary 4 shows the worst case of $k = 2$ for the min-max, the max-min, and the min-ratio objectives. For the min-max objective, it is trivial to extend to k -split and show that $\frac{2n}{k+1}$ is a tight bound by induction. Given a tree T , by Lemma 3, we find $T = T_1 \uplus T'$ such that

$$\frac{n}{k+1} \leq e(T_1) \leq \frac{2n}{k+1}.$$

Suppose by induction hypothesis that T' can be split into $k-1$ subgraphs, each with at most $2e(T')/k$ edges. Since $e(T_1) \geq n/(k+1)$, the number of edges of each subgraph is upper bounded by

$$\frac{2(n - n/(k+1))}{k} = \frac{2n}{k+1}.$$

The tightness of the bound can be easily shown by considering an extreme case in which the tree has $k+1$ branches at the root and each has exactly $n/(k+1)$ edges. Similarly, one can easily show that $\frac{n}{2k-1}$ is the tight bound of the k -split with max-min objective. Also, it can be easily shown that the extreme case for the min-max objective (or the max-min objective) is a witness to that the min-ratio is lower bounded by two. To be a worst case, it need to be shown that any tree can be split with ratio at most two. However, such a simple induction does not work for the min-ratio objective.

In the next lemma, we show that there is a similar result for the min-ratio but with a stronger condition, and this result will be used as one of the cases for proving the bound of the min-ratio.

Lemma 6: Let $T = Y \uplus T'$ and $(T_1, T_2, \dots, T_{k-1})$ be a $(k-1)$ -split of T' with ratio at most 2. If $\frac{n}{k+1} \leq e(Y) \leq \frac{2n}{2k-1}$, then $(Y, T_1, T_2, \dots, T_{k-1})$ is a k -split of T with ratio at most two.

Proof: Let $t_{\max} = \max_i e(T_i)$, $t_{\min} = \min_i e(T_i)$, and $y = e(Y)$. It is sufficient to

show that $\frac{t_{\max}}{2} \leq y \leq 2t_{\min}$. By Lemma 5, $t_{\max} \leq 2(n-y)/k$ and $t_{\min} \geq (n-y)/(2k-3)$. Since $y \geq n/(k+1)$, we have

$$\begin{aligned} \frac{t_{\max}}{y} &\leq \frac{2(n-y)}{ky} \\ &= \frac{2n}{ky} - \frac{2}{k} \\ &\leq \frac{2(k+1)}{k} - \frac{2}{k} = 2. \end{aligned}$$

Similarly, since $y \leq 2n/(2k-1)$,

$$\begin{aligned} \frac{t_{\min}}{y} &\geq \frac{(n-y)}{y(2k-3)} \\ &= \frac{n}{y(2k-3)} - \frac{1}{2k-3} \\ &\geq \frac{2k-1}{2(2k-3)} - \frac{1}{2k-3} = 1/2. \end{aligned}$$

□

3 Worst cases of 3-splits and 4-splits

Now let us consider the 3-split of a tree. By Lemma 6, if a tree T can be split into $Y \uplus T'$ such that $\frac{e(T)}{k+1} \leq e(Y) \leq \frac{2e(T)}{2k-1}$, we can find a 3-split of T with ratio at most two. However, it is not always possible to find such a split. But it does not imply that there is no 3-split of ratio within two. In the following, we show that such a 3-split always exists for any tree. First, we establish a 3-split which will be used as a basis of our discussion for $k = 3$ and 4. In the remaining paragraphs, we shall use the following notations: Let $x = e(X)$, $y = e(Y)$, $x_i = e(X_i)$, and $y_i = e(Y_i)$ for $i = 1, 2$.

Claim 7: For any $k \geq 3$, a tree T can be split into $X \uplus P \uplus Y$ such that

$$\frac{n}{k+1} \leq x, y \leq \frac{2n}{k+1}.$$

Proof: Root T at an arbitrary vertex. By Lemma 3, we split $T = X \uplus T_1$ at a vertex u such that $\frac{n}{k+1} \leq e(X) \leq \frac{2n}{k+1}$. Then, root T_1 at u , and we can split another subgraph Y , $\frac{n}{k+1} \leq e(Y) \leq \frac{2n}{k+1}$, from T_1 at a vertex v . Note that u and v are not necessarily distinct. □

Claim 8: Let $3 \leq k \leq 4$ and X be a tree rooted at u and $\frac{2n}{2k-1} \leq x \leq \frac{2n}{k+1}$. If each branch at u has no more than $\frac{n}{k+1}$ edges, X can be split into X_1 and X_2 at u such that $x_1 \geq x_2$ and $\frac{n}{2k-1} \leq x_1 \leq \frac{2n}{2k-1}$.

Proof: First we show that a subgraph X_1 can be split from X at u such that $\frac{n}{2k-1} \leq e(X_1) \leq \frac{2n}{2k-1}$. If there exists a branch of more than or equal to $n/(2k-1)$ edges, the branch is the desired subgraph since $n/(k+1) < 2n/(2k-1)$. Otherwise, the result directly follows Lemma 3.

Second, we show that we can assume that $x_1 \geq x_2$ without loss of the generality. Suppose that $x_1 < x_2$. Since, for $k \leq 5$,

$$x_2 = x - x_1 < \frac{2n}{k+1} - \frac{n}{2k-1} < \frac{2n}{2k-1},$$

the number of edges of X_2 is also in the desired range, and we may exchange X_1 and X_2 . □

Theorem 9: For any tree T , a 3-split of T with ratio at most two can be found in $O(n)$ time.

Proof: By Claim 7, we can find $T = X \uplus P_0 \uplus Y$ such that

$$\frac{n}{4} \leq y \leq x \leq \frac{n}{2}.$$

We consider the following two cases.

- Case 1: $y \leq 2n/5$.

In this case T can be split into $Y \uplus T_1$ such that $n/4 \leq y \leq 2n/5$. By Corollary 4, there is a 2-split (P_1, P_2) of T_1 with ratio at most two. By Lemma 6, (Y, P_1, P_2) is a 3-split of T with ratio at most two.

- Case 2: $2n/5 < y \leq x \leq n/2$.

As in Claim 8, we split $X = X_1 \uplus X_2$ such that $n/5 \leq x_1 \leq 2n/5$ and $x_1 \geq x_2$, in which $x_1 = e(X_1)$ and $x_2 = e(X_2)$. It should be noted that $X_2 \cup P_0$ is connected since X_1 and X_2 are split at the vertex shared by X and P_0 . If $x_1 \geq n/4$, it is similar to Case 1. Otherwise we have $n/5 \leq x_1 < n/4$. Since $x_1 \geq x/2$ and $x \geq y$, it follows that $x_1 \geq y/2$. By $e(X_2 \cup P_0) = n - x_1 - y$, we have

$$n/4 < e(X_2 \cup P_0) < 2n/5.$$

Consequently, $(X_1, X_2 \cup P_0, Y)$ is a 3-split of T with ratio at most two.

We have show that there exists a 3-split with ratio at most 2 in both cases, and the proof is completed since the time complexity is obviously $O(n)$. □

Next, we turn to the 4-splits. We show the following result.

Theorem 10: For any tree T , there exists a 4-split of T with ratio at most two.

Proof: Similar to the proof of Theorem 9, we start at splitting T into $X \uplus P_0 \uplus Y$ as in Claim 7 such that

$$\frac{n}{5} \leq y \leq x \leq \frac{2n}{5}.$$

Case 1: $y \leq 2n/7$.

In this case T can be split into $Y \uplus T_1$ such that $n/5 \leq y \leq 2n/7$. By Theorem 9, there is a 3-split (P_1, P_2, P_3) of T_1 with ratio at most two. By Lemma 6, (Y, P_1, P_2, P_3) is a 4-split of T with ratio at most two.

Case 2: $2n/7 < y \leq x \leq 2n/5$.

By Claim 8, we split $X = X_1 \uplus X_2$ such that $x_1 \geq x_2$ and $n/7 \leq x_1 \leq 2n/7$. If $x_1 \geq n/5$, it is similar to Case 1, and therefore we assume that $n/7 \leq x_1 < n/5$. Similarly we split $Y = Y_1 \uplus Y_2$ such that $y_1 \geq y_2$ and $n/7 \leq y_1 < n/5$. Let $P = P_0 \cup X_2$, and we have $T = X_1 \uplus P \uplus Y$ as in Figure 1.(a). Remember that $2x_1 \geq y$.

By the property of a centroid, P can be split into three subgraphs P_2 , P_{1a} and P_{1b} (possibly null) at its centroid in such a way that each of the subgraphs has no more than $\lceil e(P)/2 \rceil$ edges. If there are only two branches and $e(P)$ is an odd number, we add a dummy edge incident with the centroid to simplify the proof. One may check that the correctness is not affected. Therefore we can assume that each of the three subgraphs has no more than $e(P)/2$ edges. Let P_2 be the largest and $P_1 = P_{1a} \cup P_{1b}$. We have

$$e(P_2) \leq e(P_1) \leq 2e(P_2). \quad (1)$$

Since $x_1 < n/5$ and $y \leq 2n/5$, we have

$$e(P_1) > n/5 > x_1. \quad (2)$$

Since $x_1 \geq n/7$ and $y \geq 2n/7$, we have

$$e(P_2) \leq \frac{1}{2}(n - x_1 - y) \leq \frac{2n}{7} \leq y. \quad (3)$$

By Eqs. (1)–(3) and $x_1 \leq y \leq 2x_1$, we further divide this case into the following subcases:

- $y/2 \leq e(P_2) \leq e(P_1) \leq 2x_1$.
- $e(P_1) > 2x_1$.
- $e(P_2) < y/2$.

For each case, we shall show that there exists a desired 4-split.

- Case 2.1: $y/2 \leq e(P_2) \leq e(P_1) \leq 2x_1$. In this case (X_1, Y, P_1, P_2) is a desired 4-split.

- Case 2.2: $e(P_1) > 2x_1$. we divide into two subcases.

Case 2.2.1: P_1 adjacent to X_1 . Let $Q = P_1 \cup X_1$. Split Q into Q_1 and Q_2 such that

$$\frac{e(Q)}{3} \leq e(Q_2) \leq e(Q_1) \leq \frac{2e(Q)}{3}. \quad (4)$$

We show that (P_2, Y, Q_1, Q_2) is a desired 4-split. First, since $e(P_2) \geq e(P_1)/2 > x_1 \geq y/2$, we have

$$e(P_2) \leq y \leq 2e(P_2) \quad (5)$$

Since $e(P_1) > 2x_1$,

$$e(Q_2) \geq \frac{1}{3}(e(P_1) + x_1) > x_1 \geq y/2. \quad (6)$$

Since $x_1 < e(P_2)$ and $e(P_1) \leq 2e(P_2)$,

$$e(Q_1) \leq \frac{2}{3}(e(P_1) + x_1) < 2e(P_2). \quad (7)$$

By Eqs. (4)–(7), the result follows.

Case 2.2.2: P_2 adjacent to X_1 . In this case, P_2 contains X_2 (Figure 1.(b)) since $e(P_2) > x_1 \geq x_2$. Let $P_{2a} = P_2 - X_2$ and $e(P_{1a}) \geq e(P_{1b})$. We show that $(X, P_{2a} \cup P_{1b}, P_{1a}, Y)$ is a desired 4-split.

Since $e(P_2) \geq e(P_{1a})$, $e(P_1) > 2x_1$, and $x_1 \geq x_2$, we have

$$\begin{aligned} e(P_{2a}) + e(P_{1b}) &= e(P_2) + e(P_{1b}) - x_2 \\ &\geq e(P_1) - x_2 \\ &> 2x_1 - x_2 \geq \frac{x}{2}. \end{aligned} \quad (8)$$

Since $e(P_{1a}) \geq e(P_{1b})$ and $e(P_1) > 2x_1$,

$$e(P_{1a}) > x_1 \geq \frac{x}{2}. \quad (9)$$

Since $x + y \geq 4n/7$ and $x \geq y$, Eqs. (8) and (9) also imply X is the largest subgraph, and the result follows.

- Case 2.3: $e(P_2) < y/2$. We divide into two subcases.

Case 2.3.1: P_2 adjacent to Y (Figure 1.(c)). We show that $(X_1, P_1, P_2 \cup Y_2, Y_1)$ is a desired 4-split. Since $y_2 \leq y_1$ and $e(P_2) < y/2 \leq y_1$, we have

$$e(P_2 \cup Y_2) \leq 2y_1. \quad (10)$$

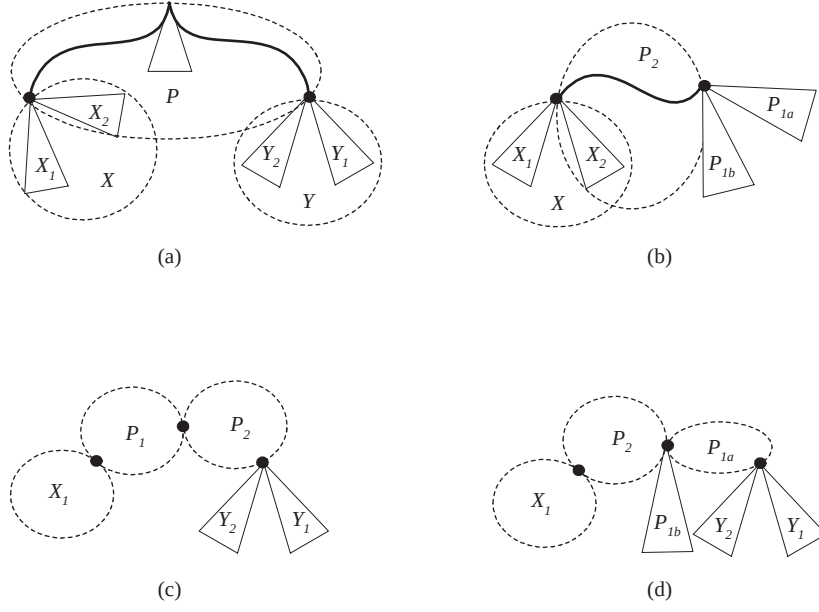


Figure 1: 4-split cases

Since $e(P_2) \geq (n - x_1 - y)/3$,

$$\begin{aligned} e(P_2) + y_2 &\geq (n - x_1 - y_1)/3 \\ &\geq (n - n/5 - n/5)/3 \\ &= n/5 \geq y_1 \end{aligned}$$

Combined with Eq. (10), we have

$$y_1 \leq e(P_2 \cup Y_2) \leq 2y_1. \quad (11)$$

By Eq. (2) and $e(P_1) \leq 2e(P_2) < y \leq 2x_1$, we have

$$x_1 \leq e(P_1) \leq 2x_1, \quad (12)$$

and

$$e(P_1) < y \leq 2y_1. \quad (13)$$

Since $e(P_2) < y/2$ and $y_2 \leq y/2$,

$$e(P_2 \cup Y_2) < y \leq 2x_1. \quad (14)$$

By Eqs. (11)–(14), the result follows.

Case 2.2.2: P_1 adjacent to Y (Figure 1.(d)). Suppose that Y is adjacent to P_{1a} , and here P_{1a} may be larger or smaller than P_{1b} . In this case, we show that $(X_1, P_2 \cup P_{1b}, P_{1a} \cup Y_2, Y_1)$ is a desired 4-split.

Since $y_2 \leq y_1$ and $e(P_{1a}) < e(P_2) < y/2 \leq y_1$, we have

$$e(P_{1a} \cup Y_2) \leq 2y_1. \quad (15)$$

Similarly,

$$e(P_{1a} \cup Y_2) \leq 2x_1. \quad (16)$$

Since $e(P_2) < y/2$ and $e(P_{1b}) \leq e(P_2)$,

$$e(P_2 \cup P_{1b}) < y \leq 2y_1. \quad (17)$$

Similarly,

$$e(P_2 \cup P_{1b}) < 2x_1. \quad (18)$$

Since $e(P_2 \cup P_{1b}) < 2y_1$, $y_1 < n/5$, and $x_1 < n/5$, we have

$$e(P_{1a} \cup Y_2) \geq n - (x_1 + y_1 + e(P_2 \cup P_{1b})) > n/5. \quad (19)$$

Similarly,

$$e(P_2 \cup P_{1b}) > n/5. \quad (20)$$

By Eqs. (19) and (20), in $(X_1, P_2 \cup P_{1b}, P_{1a} \cup Y_2, Y_1)$, the subgraph X_1 or Y_1 is the smallest and no less than a half of the maximum (by Eqs. (15) – (18)). Therefore it is a 4-split with ratio at most two. $\square$

The next corollary is directly from the above theorem.

Corollary 11: Given a tree T of n edges, a 4-split of T with ratio at most two can be found in $O(n)$ time.

4 On general k

4.1 A simple algorithm

We now propose a simple algorithm which finds a k -split of a tree. Given a tree T and an integer k , the algorithm starts at the 1-split (T) and repeatedly computes a $(i+1)$ -split from the i -split by 2-splitting the maximum subgraph. We shall show that the algorithm takes only $O(n \log k)$ time and always return a k -split with ratio at most three.

Algorithm Simple-Split

Input: A tree T and an integer $k \leq e(T)$.

Output: A k -split of T .

- 1: Initiate an empty queue Q of trees, and insert T into Q .
- 2: For $i \leftarrow 1$ to $k-1$ do
 - 2.1: Choose a tree Y in Q with maximum number of edges.
 - 2.2: Find a 2-split (Y_1, Y_2) of Y with ratio at most two.
 - 2.3: Remove Y from Q .
 - 2.4: Insert Y_1 and Y_2 into Q .
- 3: Output the k trees in Q as the k -split of T .

In the next theorem, we show the performance of the algorithm.

Theorem 12: Given a tree T with n edges and an integer $k \leq n$, the algorithm **Simple-Split** finds a k -split of T with ratio at most 3 in $O(n \log k)$ time.

Proof: Let M_i and m_i be respectively the maximum and minimum numbers of edges of trees in the queue Q at i -th iteration. We first claim that the ratio M_i/m_i is at most 3 for each i . Initially Q contains only the input tree T , and $M_1/m_1 = 1$. Suppose that $M_i/m_i \leq 3$ for some i . We shall show that $M_{i+1}/m_{i+1} \leq 3$, and then the above claim is consequently true by induction. At $(i+1)$ -th iteration, the maximum tree Y is chosen and split into Y_1 and Y_2 with ratio at most 2. Therefore, $M_{i+1} \leq M_i$, and $m_{i+1} = \min\{m_i, e(Y_1), e(Y_2)\}$. Since $\min\{e(Y_1), e(Y_2)\} \geq e(Y)/3 = M_i/3$ and $M_i/m_i \leq 3$, we have

$$\frac{M_{i+1}}{m_{i+1}} \leq \frac{M_i}{M_i/3} \leq 3.$$

Next, we turn to the time complexity. Let $f_n(i)$ be the total time complexity of executing Step 2.2 in the first i iterations. By Corollary 4, splitting

a tree of M_i edges at i -th iteration takes $O(M_i)$ time. Since the ratio M_i/m_i is at most three, by Lemma 5, we have

$$M_i \leq \frac{3n}{i+2}.$$

Therefore, for some constant c , $f_n(1) \leq cn$, and

$$f_n(i) \leq f_n(i-1) + c \frac{3n}{i+2}$$

for $i > 1$. Solving the recurrence relation, we have

$$\begin{aligned} f_n(k) &\leq c \sum_{i=1}^{k-1} \frac{3n}{i+2} \\ &\leq 3cn \sum_{i=1}^k \frac{1}{i} = 3cnH_k, \end{aligned}$$

in which H_k is the well-known k -th *harmonic number*. Since $H_k = O(\log n)$, we obtain $f_n(k) = O(n \log k)$.

For Step 2.1, 2.3, and 2.4, by simply using a data structure like *heap* to store the numbers of edges of the trees in the queue, all the operations can be done in totally $O(k \log k)$ time. Therefore the total time complexity is $O(n \log k)$. $\square$

4.2 Experimental results on random trees

To investigate the practical behavior of the algorithm **Simple-Split**, we implemented the algorithm and performed tests on random trees. Before showing the experimental results, we explain how we find the 2-split at Step 2.2. By Corollary 4, we can find a 2-split of ratio at most two by the procedure described in the proof of Lemma 3. However, although it ensures the bound of the worst ratio, the procedure does not try to find the best one. We used the following procedure to find a 2-split. For a given tree Y , we root the tree at its centroid. Initially we regard each branch as a subgraph, and then repeatedly merge the smallest two subgraphs until only two subgraphs are left. To see that the procedure always returns a 2-split (Y_1, Y_2) of ratio at most two, it is sufficient to show that the smaller subgraph Y_1 contains at least $e(Y)/3$ edges. Since the tree is rooted at its centroid, each branch contains no more than $e(Y)/2$ edges. If $e(Y_1) < e(Y)/3$, it implies that $e(Y_2) < 2e(Y)/3$ since Y_2 is either a single branch or obtained by merging two subgraphs smaller than Y_1 . But $e(Y_1) + e(Y_2) = e(Y)$, and it is a contradiction.

Table 1: The average ratios

n	100	500	1000	3000	6000	10000
$k = 2$	1.21	1.23	1.20	1.17	1.19	1.21
$k = 3$	1.85	1.84	1.85	1.87	1.89	1.85
$k = 4$	1.51	1.50	1.46	1.39	1.45	1.48
$k = 5$	1.97	1.99	2.00	2.06	1.98	1.97
$k = 10$	2.09	2.11	2.08	2.13	2.09	2.10
$k = 20$	2.23	2.26	2.21	2.22	2.17	2.17
$k = 50$	3.00	2.38	2.41	2.41	2.39	2.43
$k = 100$	1.00	2.43	2.50	2.54	2.51	2.50

Table 2: The distribution of ratios (in percentage)

(n, k)	≤ 1.4	≤ 1.6	≤ 1.8	≤ 2	≤ 2.2	≤ 2.4	≤ 2.6	≤ 2.8
(100, 4)	45.9	65.3	83.2	93.9	98.4	99.2	99.8	100
(500, 4)	48.0	73.2	88.2	96.7	98.5	99.4	100	100
(100, 10)	0.4	1.7	9.9	48.5	72.0	87.5	97.4	99.8
(500, 10)	0	1.7	9.0	40.2	68.4	86.5	95.7	99.5
(5000, 10)	0.2	1.5	9.3	35.4	68.3	86.4	95.4	99.3
(500, 20)	0	0	2.1	20.8	50.6	76.4	92.3	98.7
(5000, 20)	0	0	0.6	15.1	49.6	76.0	92.1	99.1

For different n (number of edges) and k , we recorded the ratios of the k -splits found by the algorithm. Since the program runs very fast, we do not show the execution time. For each (n, k) , hundreds of instances were tested, and the average ratios are shown in Table 1.

Since the worst cases (ratio 3) do exist, showing the worst ratios in the test is meaningless. The more instances we run, the larger the worst ratio is. Instead, we show the distributions for some typical pairs (n, k) . In Table 2, we show the percentage of the ratio in each specified range. For example, the value 65.3 in the cell of the second row and third column means that, for $n = 100$ and $k = 4$, there are 65.3% of the instances in the test such that the ratio of the obtained split is less than or equal to 1.6.

By the experimental results, we observed the following.

- For small k , the algorithm performs well, but the obtained ratios get larger and tend toward the worst case as k increasing. Observing the cases of $k = 2$, we find that the algorithm splits a tree into two parts quite evenly, and it is also the reason why the performance is good for $k = 4$ but rather bad for $k = 3$.
- As long as k is small with respect to n , the results are almost not affected by n . In Table 1, we can see that the average ratios in each

row are almost the same except for $(n, k) = (100, 50)$ and $(100, 100)$.

- The distributions approximate to the normal distribution. For each (n, k) , the standard deviation is approximately 0.27. In our test, the obtained ratios of about 70% of the instances are in the range $[\mu - \sigma, \mu + \sigma]$, in which μ is the mean and σ is the standard deviation.
- There are many instances that the algorithm obtained a ratio larger than two. In this aspect, it is significant to develop an algorithm always finding a ratio within two for general k . Even for $k = 3$ and $k = 4$, there are still about respectively 35% and 5% of the instances in our test such that the obtained ratios are larger than two. Therefore, the results for 3-splits and 4-splits in this paper is meaningful.

5 Concluding Remarks

One of the most important open problems in this line of investigation is that whether there exists a k -split with ratio at most two for general k . Our future work includes exact and approximation algorithms for finding the min-ratio k -split for general or fixed k .

References

- [1] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela and M. Pro-
tasi, *Complexity and Approximation — Combinatorial optimization problems and their approximability properties*, Springer Verlag, 1999.
- [2] I. Averbakh and O. Berman, A heuristic with worst-case analysis for minimax routing of two traveling salesmen on a tree, *Discrete Appl. Math.*, 68:17–32, 1996.
- [3] I. Averbakh and O. Berman, $(p-1)/(p+1)$ -approximate algorithms for p -traveling salesmen problem on a tree with minmax objective, *Discrete Appl. Math.*, 75:201–216, 1997.
- [4] V. Bafna, E. L. Lawler and P. Pevzner, Approximation algorithms for multiple sequence alignment, *Theoretical Computer Science*, 182:233–244, 1997.
- [5] R. Becker and Y. Perl, Shifting algorithms for tree partitioning with general weighting functions, *J. Algorithms*, 4:101–120, 1983.
- [6] R. Becker and Y. Perl, The shifting algorithm technique for the partitioning of trees, *Discrete Appl. Math.*, 62:15–34, 1995.
- [7] R. Becker, S.R. Schach and Y. Perl, A shifting algorithm for min-max tree partitioning, *J. ACM*, 29:58–67, 1982.
- [8] R. Becker, B. Simeone and Y.-I Chiang, A shifting algorithm for continuous tree partitioning, *Theor. Comput. Sci.*, 282:353–380, 2002.
- [9] G.J. Chang and F.K. Hwang, Optimality of consecutive and nested tree partitions, *Networks*, 30:75–80, 1997.
- [10] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H. Freeman and Company, San Francisco, 1979.
- [11] D. Gusfield, Efficient methods for multiple sequence alignment with guaranteed error bounds, *Bulletin of Mathematical Biology*, 55:141–154, 1993.
- [12] D. Gusfield, *Algorithms on Strings, Trees, and Sequences – Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [13] C.M. Huang, B.Y. Wu and C.B. Yang, Tree edge decomposition with an application to minimum ultrametric tree approximation, unpublished manuscript.
- [14] I. Krasikov, On a tree cutting problem of P. Ash, *Discrete Math.*, 93:51–61, 1991.
- [15] S. Kundu and J. Misra, A linear tree partitioning algorithm, *SIAM J. Comput.*, 6:151–154, 1977.
- [16] M. Lucertini, Y. Perl and B. Simeone, Most uniform path partitioning and its use in image processing, *Discrete Appl. Math.* 42:227–256, 1993.
- [17] Y. Perl and S.R. Schach, Max-min tree partitioning, *J. ACM*, 28:5–15, 1981.
- [18] P. Pevzner, Multiple alignment, communication cost, and graph matching, *SIAM J. Appl. Math.*, 52:1763–1779, 1992.
- [19] B.Y. Wu and K.-M. Chao, *Spanning Trees and Optimization Problems*, Chapman & Hall / CRC Press, 2004.
- [20] B.Y. Wu, G. Lancia, V. Bafna, K.-M. Chao, R. Ravi and C.Y. Tang, A polynomial time approximation scheme for minimum routing cost spanning trees, *SIAM J. Comput.*, 29:761–778, 2000.

The Constrained Independent Domination Problem

Yu-Wei Chang, Meng-Ti Liu, and Chang Wu Yu

Department of Computer Science and Information Engineering,
Chung Hua University, Hsin-Chu, Taiwan, R. O. C

Email: {yuwai@chu.edu.tw; m9102018@chu.edu.tw; cwyu@chu.edu.tw}

摘要

在此論文中我們定義一個新的獨立支配的圖論問題。我們稱此問題為：有限制的獨立支配問題 (the constrained independent domination problem*, 簡稱 CIDP)，我們將證明 CIDP 在一般圖 (general graphs) 中是 NP-hard 的問題，同時我們提出一個循序演算法來解 CIDP 問題。當輸入圖 (input graph) 是一個 trivial perfect graphs 的子集合時，我們設計一個多項式時間 (polynomial time) 的循序演算法。本論文的結果可應用於解決無線電網路上的資源分配問題。

關鍵字：Graphs, Dominating set, independent dominating set, wireless Networks.

1 簡介

任意給一個圖 $G(V, E)$ ，其中一個 V 的子點集合 (vertex subset) S ，如果 V 中的每個點不是屬於 S 中的一個元素，就是連接到 (adjacent to) S 中的一個元素，則此 S 被稱作為支配集合 (dominating set)。一個獨立支配集合 (independent dominating set) 是一個支配集合，而且任意在此集合中的兩點是不相連的。所謂獨立支配問題 (the independent domination problem) 就是在一個輸入圖 (input graphs) 中，回答是否可以找到 size 小於或等於 K 的獨立支配集合。如圖 1 中，當獨立支配問題中 K 等於 2 或 2 以上時， $S = \{4, 5\}$ 即為獨立支配問題的一個解。

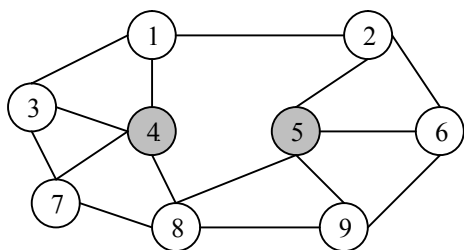


圖 1. 獨立支配問題

在此我們定義一個新的獨立支配的圖論問題。此問題需輸入三種資料：

1. 一個圖 $G(V, E)$
2. 一個點集合 V 的獨立支配切割 (independent domination partition) P ，此切割點集合 V 分成 k 個 disjoint 獨立支配集合 $P_1, P_2, \dots, P_k$ 。換而言之 $V = P_1 \cup P_2 \cup \dots \cup P_k$ ，其中針對任意 i 和 j ， $P_i \cap P_j = \emptyset$ ；而且每個 P_i 都是 G 的一個獨立支配集合 (但不一定 size 要最小)。有一個類似 (但不同) 的定義稱為 domatic partition 曾出現在 [16]。
3. 一個維度為 k 的整數向量 $R = (r_1, r_2, \dots, r_k)$ 。

我們考慮的圖論問題就是：找出一個最小的獨立支配點集合 W ，使得它的點出現在每個 P_i 的數量大於或等於 r_i 。我們稱此問題為：有限制的獨立支配問題 (the constrained independent domination problem, 簡稱為 CIDP)。當 R 為零向量 (i.e. $R = (0, 0, \dots, 0)$) 時，CIDP 便退化成一般的獨立支配問題。如圖 2 中， $P = (P_1, P_2, P_3)$ 是一個獨立支配切割，當 $R = (0, 2, 0)$ 時，此 CIDP 的其中一解為 $(4, 5, 9)$ 。注意，任意給一個 CIDP，有時不見得一定有解。

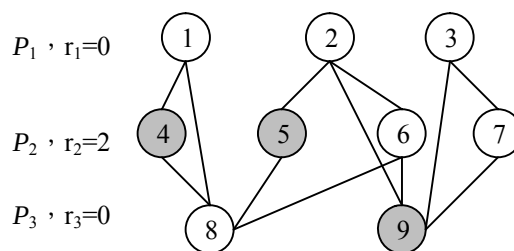


圖 2. CIDP 中的一個圖 G

我們將證明 CIDP 在一般圖 (general graphs) 為 NP-hard 的問題，同時我們提出一個循序演算法來解 CIDP 問題，而且當輸入圖 (input graph) 是一個 Trivial perfect graphs 的子集合時，我們有多項式時間 (polynomial time) 的循序演算法。我們也將在稍後解釋：如果解決了

* This work is partially supported by the National Science Council under grant number NSC91-2213-E16-009

CIDP，就可解決無線電網路上一個資源分配的問題。藉此說明此新的圖論問題是自然的、有意義的，並非虛構無用的。

本篇論文的主要架構如下：第二節將描述獨立支配問題背景知識，以及前人的相關研究。第三節描述我們對於CIDP的相關研究成果。第四節是將此CIDP應用於解決無線電網路資源配置問題。最後一節說明未來展望與工作。

2 背景

獨立支配問題已被證明為NP-complete [14]，甚至當輸入圖(input graphs)為bipartite graph [6] 或comparability graph [6] 或line graphs of graphs [31] 時，此問題依舊是NP-complete。

在1979年，Beyer、Proskurowski、Hedetniemi 和 Mitchell 提出了tree上的第一個演算法[4]。當輸入圖為chordal graphs、strongly chordal graphs 或interval graphs 時，Farber 找到了polynomial time 演算法來解決最小權重(weighted)獨立支配問題[11, 12]。Farber 和 Keil 也證明permutation graphs的最小權重獨立支配集合可在 $O(n^3)$ 時間找到[13]。後來Brandstadt 和 Kratsch 又將時間複雜度從 $O(n^2)$ 降到 $O(n^2)$ [36]。近來還有不少的學者提出在cocomparability graphs[19]、 k -polygon graphs(k 值固定)[9]、circle graphs[8]、circular-arc graphs[5]、interval graphs[1]上更快的演算法。至於獨立支配問題的平行演算法也有一些成果，當輸入圖為trapezoid graphs 時，Liang 設計了需 $O(\log^2 n)$ 時間的平行演算法[20]。至於獨立支配問題的分散式演算法，我們並未找到相關的文獻，倒是有一些支配問題(domination problem)的分散式演算法被設計出來，並且應用於隨意網路(ad hoc networks)中[25, 29]。

在圖學上，和獨立支配相似的問題還有相當多的類型：如支配(domination)問題、連接支配(total domination)問題、支配顆粒(dominating clique)問題等[15]。近年來，這些不同種類的支配集合問題受到相當大的重視，有相當多的研究成果在此問題上[15, 16]。理由是這些支配集合問題在通訊網路(communication

networks)的設計和分析上、社會科學上(social sciences)、計算理論上(computational complexity)、和演算法設計(algorithm design)上都有相當重要的應用[15, 16]。

3 CIDP 及相關演算法

首先我們將證明 CIDP 在一般圖(general graphs)中是 NP-hard 的問題。

Theorem 1.: CIDP in general graphs is NP-hard problem.

Proof. 因為當 CIDP 中 R 中所有的 r_i 皆為 0 時，CIDP 變退化成獨立支配問題(the independent domination problem, 簡稱 IDP)。所以可以說 IDP 是 CIDP 的一個子集合(subclass)。因為 IDP 在一般圖(general graphs)中已被證明為 NP-hard[14]，所以 CIDP 在一般圖中也為 NP-hard 的問題。

3.1 CIDP in general graphs

首先，我們先提出一個解決 CIDP 在一般圖(general graphs)上的演算法，首先介紹一個組合數學的技巧，這個技巧源自於科學家與保險櫃問題。假設有 N 科學家參與一個極機密的工作計畫。他們想將重要的文件鎖在一保險櫃中，使此保險櫃必須有他們中 M 位或 M 位以上在一起才能打開。試問最少需要裝多少鎖才夠？又每一位科學家最少需要配帶多少把鑰匙？根據[37]的解法，最少需要 $C(N, M-1)$ ($C(n, r)$ 符號代表 n 件相異物品的 r 組合數)個鎖和每位科學家最少需帶有 $C(N-1, M-1)$ 把鑰匙。

舉例說明如下，當 $N = 4$, $M = 3$ 時，我們可以算出每個鎖平均需要 $C(4-1, 3-1) = C(3, 2) = 3$ 把鑰匙。每個鎖都有 3 把鑰匙也就等於是 3 位科學家可以打開它，故鎖及鑰匙的分配方法就是每種 $C(4, 3)$ 組合，如圖 3 中任意有 3 位科學家即可將所有鎖打開。

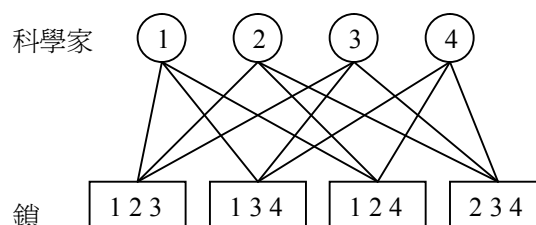


圖 3. 科學家與鎖

我們將輸入圖 G 的 CIDP，轉換成 G^* 上的 IDP 問題，其轉換過程。針對每一個 P_i 及 r_i ，令 $N=|P_i|$ 而且 $M=r_i$ ，依上述加入與鎖相同數量的點 (Vertices)，依照鑰匙分配的方式將輸入圖 G 中的點拉起新增的 edge。如圖 4，即為圖 2 G 之 G^* ， L 代表新增的點，粗線代表依照鑰匙分配方式新增的 edge。

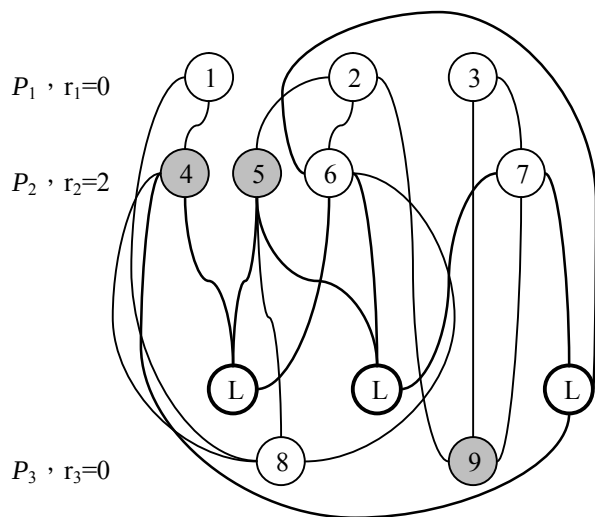


圖 4. G^*

Theorem 2.: G 上的 CIDP 可轉換成為 G^* 上的 IDP.

Proof. 當我們在找 G 中的最小獨立支配集合 W 時，可利用給每個點一個權重值 (weight)。我們將給予 G^* 中新增的點 L 較大的權重值，使得 L 將會被排除在 W 之外，同時因為利用前述科學家和保險櫃的問題，我們所新增的 edge 可保證 P_i 中必定有 r_i 個點被選入 W 中。此方法可以在一般圖中將 CIDP 退化成為 IDP。 ■

藉此我們即可利用前人對於 IDP 的演算法，即可直接用於 CIDP 之上。

3.2 CIDP in Trivial perfect graphs

當輸入圖為 trivial perfect graphs 的一個子集合時，我們設計了一個多項式時間 (polynomial time) 的循序演算法來解決 CIDP。

Trivial perfect graphs 是一種特殊的 comparability graphs (i.e. the comparability graphs of root tree posets [33, 35, 46])。也就是任意一棵樹任意兩 node 有祖先和後代的關係，則加上一個 edge，如此樹就會成為 trivial

perfect graphs。如圖 5 所示。

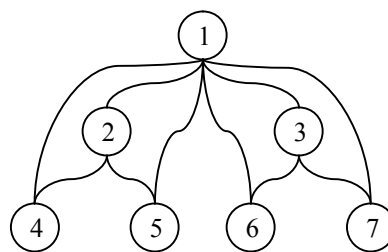


圖 5. wonderful graphs

接著我們定義了一個新的 graphs 稱為 wonderful graphs G ，如果 G 是 the comparability graphs of root full binary tree posets。圖五即是 wonderful graphs。

當輸入圖為 wonderful graphs 且 P 以 tree level 切割，則我們將有多項式時間 (polynomial time) 的循序演算法來解決 CIDP。我們發現 CIDP 中的 partition P 就是以 full binary tree 的 level 來切割，因為每個 level 中的任兩點剛好是 independent；而且每個 level 的所有點，剛好形成一個支配集合。如圖 6 即為圖 5 之 CIDP。

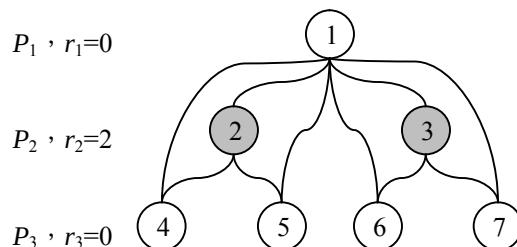


圖 6. CIDP in wonderful graphs

我們將利用深度追蹤法 (depth first search, DFS) 來解決 wonderful graphs 上的 CIDP。其概念為由 full binary tree 的根節點 (root) 開始往下尋找適當的碼，直到符合我們的條件向量則停止。滿足整數 (條件) 向量後，我們將用最少的節點去完成獨立支配集合，也就解決在 wonderful graphs 上的 CIDP 問題。詳細演算法如下所示 (為了方便說明，我們將 wonderful graphs 簡化成為一個 full binary tree)：

Algorithm W: wonderful graphs 上的 CIDP 演算法。

Step 1. 自 full binary tree 的 root 出發。
Step 2. 先選擇 root 的 left children，在此點為起始點做 DFS。

Step 3. 當走到 u 點可以符合 r_i 的條件向量時，我們則回到最近走過，而且 right

children 尚未被走過的 v 點，並以此 v 點的 right children 為起始點作 DFS。

Step 4. 當完全符合我們的條件向量時，則停止。

Step 5. 將 w_1 設為所有曾經走到符合 r_i 的條件向量的點 u 所成的集合。

Step 6. 將 w_2 設為中途經過所有點 v 的且 right children 尚未經過所成的集合。

Step 7. 將 w_3 設為 w_2 所有元素的 right children 的集合。

Step 8. $W = w_1 \cup w_3$ 即為 wonderful graphs 之 CIDP 一解。 ■

Theorem 3. W 為 wonderful graphs 上的 CIDP

Proof. W 為 wonderful graphs 上的 CIDP 需滿足四項條件分別如下所示：1. 符合 R 條件向量、2. W 為 independent set、3. W 為 dominating set 和 4. W 的 size 最少。

分別說明之，第 1 項符合 R 條件向量：因為根據演算法的 output W 是由 w_1 和 w_3 所聯集而成，而其中 w_1 部分即完全符合 R 條件向量，所以 W 必定符合 R 條件向量。

第 2 項 W 為 independent set：因為是利用 DFS 下去配置 w_1 和 w_3 ，因為 DFS 所經過的所有點為一棵 binary tree，其中 w_1 和 w_3 皆為此 binary tree 的子葉 (leaf)，所以 W 中任意兩元素，沒有直系直系血緣關係，故 W 為 independent set。

第 3 項 W 為 dominating set：因為 w_1 剛好滿足了 R 條件向量，因為 w_1 是由 DFS 的方式所配置出來，所以存在一棵 binary tree 而且子葉皆為 w_1 的元素，將 wonderful graphs 扣去此 binary tree 以及 w_1 的子孫，將會生成出 N 個子樹 (subtree)。而且 w_3 分別為這些子樹的根節點 (root)，所以 w_1 和 w_3 的聯集為 dominating set。

第 4 項 W 的 size 最少：因為 w_1 剛好滿足了 R 條件向量，但 w_1 並不是 dominating set，因為 $W - w_1 = w_3$ ，所以當 w_3 個數為最少時， W 的 size 即為最少。

而在找尋 w_3 上，我們將這個問題轉換成演算法中的找零錢問題 (coin change problem)，把樹的每一層 level 視為零錢面額，而 $W - w_1$ 所剩餘尚未滿足 dominating set 的部分視為所要兌換的金額。在找零錢問題中，如果錢幣面額為 $C_0, C_1, \dots, C^n$ 且 $C > 1$ 和 $n \geq 1$ 則可

使用貪婪法 (greedy method) 求其最佳解。在找尋 w_3 我們即是盡量使用 level 較高的點，所以 w_3 的個數也為最少。所以 W 的 size 最少。

根據上述四點，所以 W 為 wonderful graphs 上的 CIDP。 ■

Theorem 4.: Algorithm W 之時間複雜度為 $O(n+e)$ 。

Proof. 因為我們採用的是深度追蹤法 (Depth First Search, DFS)，所以在最壞的情況下為 full binary tree 時的 $O(n+e)$ 。 ■

4 應用 CIDP 於無線電網路資源管理

如何將 CIDP 應用於無線電網路上的資源管理問題，將採用下列的篇幅來說明。

4.1 OVSF code tree 之介紹

當今第三代無線電網路 UMTS/IMT-2000 運用 direct-sequence code division multiple access (DS-SS) 的編碼技術，來滿足多媒體應用時之不同傳輸速度之需求 [34]。DS-SS 透過不同正交的 (orthogonal) 編碼，可在同一媒介上同時傳輸多組不同的通訊資料。其中的 OVSF CDMA 系統 (orthogonal variable-spreading-factor CDMA system)，可以用長短不同的一個碼來指定不同的傳輸頻寬，而且所有的碼可以用一棵二元樹 (binary tree) 的資料結構來表示，此樹被稱作 OVSF code tree [30]。以圖 7 舉例，其中父節點 (parents) 和子節點 (child) 的編碼關係，如樹上括弧中的值所表示的 (其中 x 代表一個合法的 code)，即父節點的碼會重複出現在子節點的碼前半段。因此，父節點的碼和子節點的碼，如果同時使用會分不清楚彼此的訊號 [30]。

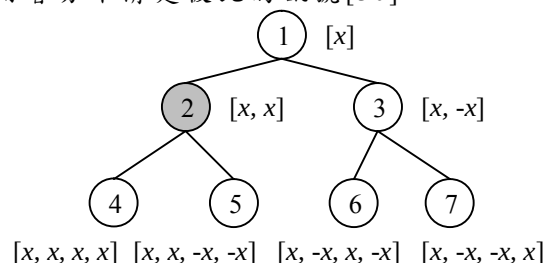


圖 7. OVSF code tree

我們更進一步的說明 (為了簡潔起見，我們將稱在此樹標號 i 的碼為「碼 i 」)，如果我們使用了圖 7 中二元樹的

碼 2，則所有碼 2 的祖先(ancestor)(i.e. 碼 1)和後代(descendant)(i.e. 碼 4, 5)都不可同時被使用。一個基本性質是：使用父節點的碼所得的頻寬，為使用子節點的碼所得的頻寬的兩倍。原因是父節點碼的長度為子節點碼的長度的兩倍。

在基地台附近的行動設備(mobile devices)，因本身對於頻寬的需求，會要求基地台給予適合的 OVSF code。因此如何選取和管理此 OVSF tree 對於無線電網路資源而言，是一件相當重要的工作。目前為止，已經有一些 heuristic 演算法來研究如何管理此 OVSF code tree，使得整個系統阻塞率(blocking rate)可降低[27]；也有一些研究關心，如何使的重新分配碼(code reassignments)的次數減少[30]。但就我們所知，並沒有使用圖論(graph theory)的技巧來解這個問題的相關論文出現。

4.2 OVSF code tree 之管理問題

首先在正式討論管理問題前，我們先說明碼阻塞(code blocking)為有一需求被系統拒絕，但實際上，系統仍然有足夠的頻寬可以容納該需求。

例如圖 8 中碼 5 及碼 6 已被使用，則此樹不可能提供碼 2 或碼 3；但如果使用碼 4 代替碼 6，則此樹尚可提供碼 3。

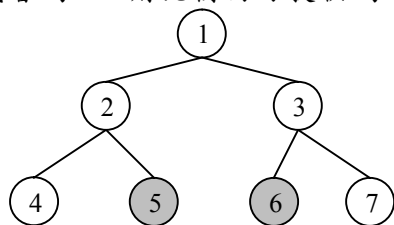


圖 8. 碼阻塞

所以在此產生了一個有趣的現象，如果這些碼分布在 tree 中的位置不佳，則會大量浪費頻寬資源。而配置碼的技巧精神就在於將不使用的碼盡量集中於少數的子樹(subtree)中。

藉由上述的現象，我們在此解釋 CIDP 和 OVSF code tree 的管理有何重要的關係。簡單的說，解決一個 OVSF code tree 上的 CIDP，就會找到一種最佳配置碼的方式(optimal code assignment)，不但符合使用者的需求，而且使得剩餘的資源會聚集到最少的 OVSF 馬上，如此可排除上述頻寬資源浪費的現象，進而而有更大的機會滿足不

同頻寬之需求，因而降低系統阻塞率(blocking rate)。

4.3 OVSF 和 CIDP 的對映關係

我們在此描述 OVSF code tree 最佳碼分配的方式和 CIDP 之間的對映(mapping)關係。

首先我們先將一個 OVSF code tree 轉換成一個 CIDP。如圖 8 或圖 7 在 OVSF code tree 上，如果任一有兩個碼會互相衝撞(此兩碼將會有祖先和後代的關係)，則加上一個 edge，如此 OVSF code tree 成為了 wonderful graphs。如圖 9 即為圖 8 的轉換。

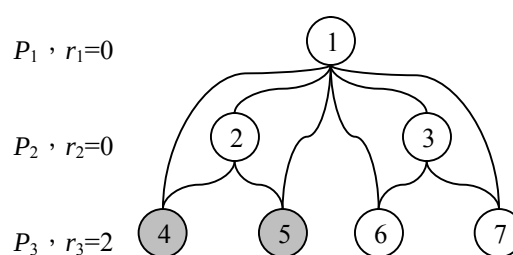


圖 9. OVSF to CIDP

而圖中 $R=(r_1, r_2, \dots, r_k)$ ， r_i 就是代表在原 OVSF code tree 上第 i level 上被配置碼的數目。如此我們說明了 OVSF code tree 轉換成一個 CIDP 的過程。

注意任意一個不衝撞的 code assignment 在 G 中都是一個獨立支配集合。而 R 就代表了配置碼之頻寬需求，符合 R 的限制而 size 最小的獨立支配集合，就是滿足原先使用的頻寬需求，而又把剩餘的頻寬集中在最少個碼的一種 code assignment。這就是我們的 CIDP。

4.4 OVSF 碼配置

本節將會說明 OVSF code tree 碼配置的方式。

因為 OVSF code tree 本身為完整二元樹(complete binary tree)，所以由 OVSF code tree 即為 wonderful graphs 之骨幹，即可使用之前我們所提的多項式時間(polynomial time)的循序演算法來配置碼。

其中 algorithm W 中的 w_1 即為配置出去的碼，而 w_3 則是成為了預留碼，當下次有需求進來時，我們即可檢查是否有預留碼可供其使用。其中也有分成兩種選擇，預設為直接使用了預留碼，這

樣的話對於 OVFS code tree 的 CIDP 完全沒有增加；當無法直接使用預留碼時，則是把預留碼的子樹(subtree)拿來當作預留的 OVFS code tree 使用，這樣做依舊會讓整個 OVFS code tree 保持著 CIDP 的狀態。

如圖 10 碼 2 已經被配置出去，碼 3 則為預留碼狀態，當有 level r_2 有需求時，將會直接使用碼 3；但需求為 level r_3 時，則會由碼 3 成為子樹的根節點，開始配置，最終會找到碼 6。

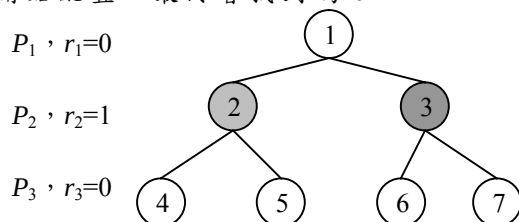


圖 10. 碼的分配

5 未來展望與工作

我們希望可以推廣 CIDP 的應用於不同的研究領域，同時我們想界定 CIDP 的困難範圍。也就是在哪些特殊圖中，此問題是屬於 NP-hard 或著是屬於 P 的集合。

因為 $\text{trivial perfect graphs} \subseteq \text{interval graphs} \subseteq \text{permutation graphs} \subseteq \text{comparability graphs} \subseteq \text{general graphs}$ [37]，在這些特殊圖的層級中，哪些屬於 P？哪些屬於 NP-hard 的範圍？

而就 CIDP 在無線電網路上的應用 OVFS code tree 來說，重配置(code reassignments)也是相當重要的課題，如何將重配置與 CIDP 也能緊密的結合？如何在重配置時又能保佑 CIDP 的特性又能減少搬移的次數？這些都是相值得探索的問題。

References

1. G.S. Adhar and S. Peng, "Parallel algorithms for finding connected, independent and total domination in interval graphs," Manuscript, 1992.
2. S. Arnborg, J. Lagergren, and D. Seese, "Easy problems for tree-decomposable graphs," *Journal of Algorithms*, vol. 12, pp. 308-340, 1991.
3. M.J. Atallah and S. R. Kosaraju, "An efficient algorithm for maxdominance with application,

- "*Algorithmica*, vol. 4, pp. 221-236, 1989.
4. T.A. Beyer, A. Proskurowski, S.T. Hedetniemi, and S. Mitchell, "Independent domination in trees," *Congressus Numerantium*, vol. 19, pp. 321-328, 1977.
5. M.S. Chang, "Efficient algorithms for the dominating problems on interval and circuit-arc graphs," *SIAM Journal on Computing*, vol. 27, no. 6, pp. 1671-1694, 1998.
6. D.G. Corneil and Y. Perl, "Clustering and domination in perfect graphs," *Discrete Applied Mathematics*, vol. 9, pp. 27-39, 1984.
7. G. Coulouris, J. Dollimore, and T. Kindberg, *Distributed Systems: Concepts and Design*, Third edition, Addison-Wesley, 2001.
8. M. Damian-Iordache and S.V. Pemmaraju, "Hardness of approximating independent domination in circle graphs," *ISAAC'99, LNCS 1741*, pp. 56-69, 1999.
9. E.S. Elmallah and L.K. Stewart, "Domination in polygon graphs," *Congr. Numer.*, vol. 77, pp. 63-76, 1990.
10. S. Even, *Graph Algorithms*, Computer Science Press, Rockville, Maryland.
11. M. Farber, "Independent domination in chordal graphs," *Operations Research Letters*, vol. 1, pp. 134-138, 1982.
12. M. Farber, "Domination, independent domination, and duality in strongly chordal graphs," *Discrete Applied Mathematics*, vol. 7, pp. 115-130, 1984.
13. M. Farber and J. M. Keil, "Domination in permutation graphs," *Journal of Algorithms*, vol. 6, pp. 309-321, 1985.
14. M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP Completeness*, W. H. Freeman and Co., San Francisco, California, 1979.
15. T.W. Haynes, S.T. Hedetniemi, and P.J. Slater, *Fundamentals of Domination in Graphs*, Number 208 in Pure and Applied Mathematics: A series of monographs and textbooks, Marcel Dekker Inc., New York, 1998.

16. T.W. Haynes, S.T. Hedetniemi, and P.J. Slater, *Domination in Graphs: Advanced Topics*, Number 209 in Pure and Applied Mathematics: A series of monographs and textbooks, Marcel Dekker Inc., New York, 1998.
17. E. Horowitz, S. Sahni, and S. Rajasekaran, *Computer Algorithms*, W. H. Freeman, New York, 1998.
18. R.W. Irving, "On approximating the minimum independent dominating set," *Information Processing Letters*, vol. 37, pp. 197-200, 1991.
19. D. Kratsch and L. Stewart, "Domination on cocomparability graphs," *SIAM J. Discrete Math.*, vol. 6, no. 3, pp. 400-417, 1993.
20. Y.D. Liang, "Parallel algorithms for the domination problems in trapezoid graphs," *Discrete Applied mathematics*, vol. 74, pp. 241-249, 1997.
21. N. Lynch, *Distributed Algorithms*, Morgan Kaufmann, 1996.
22. M. Raynal, *Distributed Algorithms and Protocols*, Wiley, 1988.
23. J.A. Telle, "Complexity of domination-type problems in graphs," *Nordic J. Comput.*, vol. 1, pp. 157-171, 1994.
24. Bevan Das and Vaduvur Bharghavan, "Routing in ad-hoc networks using minimum connected dominating Sets," *IEEE International Conference on Communications*, 1997, pp. 376-380.
25. Jie Wu and Hailan Li, "Domination and Its Applications in Ad Hoc Wireless Network with Unidirectional Links," *International Conference on Parallel Processing*, 2000, pp. 189-197.
26. A.A. Bertossi and M.A. Bonuccelli, "Code assignment for hidden terminal interference avoidance in multihop packet radio networks," *IEEE/ACM Transactions on Networking* vol. 3 4, pp. 441-449, 1995
27. Ray-Guang Cheng and Phone Lin, "OVSF Code Channel Assignment for IMT-2000," *Proceedings of 51st Vehicular Technology Conference*, 2000, vol. 3, pp. 2188-2192.
28. J.J. Garcia-Luna-Aceves and J. Raju, "Distributed assignment of codes for multihop packet-radio networks," *MILCOM 97*, 1997, vol. 1, pp. 450-454.
29. Limin Hu, "Distributed code assignments for CDMA packet radio networks," *IEEE/ACM Transactions on Networking*, vol. 16, pp. 668-677, 1993.
30. Thit Minn and Kai-Yeung Siu, "Dynamic Assignment of Orthogonal Variable-Spreading-Factor Codes in W-CDMA," *IEEE Journal on Selected Areas in Communications*, vol. 188, pp. 1429-1440, 2000.
31. M. Yannakakis and F.Garril, "Edge dominating sets in graphs," *SIAM J. Appl. Math.*, vol. 38, pp. 264-272, 1980.
32. A. Brandstadt and D. Kratsch, "On domination problems on permutation and other graphs," *Theoret. Comput. Sci.*, vol. 54, pp. 181-198, 1987.
33. A. Brandstadt, V. B. Le, and J. P. Spinrad, *Graph Classes: a survey*, SIAM Monographs on Discrete Mathematics and Applications, 1999.
34. 3GPP TS 23.107, v3.3.0, QoS Concept and Architecture, June, 2000.
35. M.C. Golumbic, "Trivial perfect graphs," *Discrete Mathematics*, vol. 24, pp. 105-107, 1978.
36. E.S. Wolk, "The comparability graph of a tree," *Proc. Amer. Math. Soc.*, vol. 16, pp. 17-20, 1965.
37. C.L.Liu, *Introduction to Combinatorial Mathematic*, New York, McGraw-Hill, 1968.

Lower Bounds to the Size of the Minimum Feedback Vertex Sets in Split-stars *

Fu-Hsing Wang Cheng-Ru Hsu

Department of Information Management, Chungyu Institute
of Technology, Keelung, Taiwan, Republic of China.

Abstract

In a graph $G = (V, E)$, a subset $F \subset V(G)$ is a feedback vertex set of G if the subgraph induced by $V(G) \setminus F$ is acyclic. In this paper, we establish a lower bound to the size of the minimum feedback vertex sets in split-stars.

1 Introduction

Given a graph G , the feedback vertex set problem is to find a subset F of vertices whose deletion results in an acyclic subgraph. The set F is called a *feedback vertex set* [12, 20]. Furthermore, if the cardinality $|F|$ is minimum among all possible feedback vertex sets, then F is called a *minimum feedback vertex set*. In particular, we use $F_m(G)$ to denote a certain minimum feedback vertex set of the graph G .

The problem of finding a minimum feedback vertex set is NP-hard for general graphs [12]. The best known approximation algorithm for this problem has approximation ratio 2 [2]. In addition, most recent research have been devoted to solving the problem for certain special graphs in polynomial time, e.g., reducible graphs [20], cocomparability graphs [16], convex bipartite graphs [16], cyclically reducible graphs [21], and interval graphs [17]. Recently, the lower and upper bounds to the size of the feedback vertex sets have been established and improved on meshes, tori, butterflies, cube connected cycles, hypercubes, and star graphs [4, 10, 18, 22].

The feedback vertex set problem has important applications to several fields, for example, deadlock prevention in operating systems. Once a deadlock has been detected, a strategy is needed

to break up the deadlock. Usually, a deadlock in a system can be described by using a wait-for graph [19]. In a wait-for graph, each vertex represents a process, and the existence of an edge (i, j) indicates that process i is waiting for process j to release a resource requested by process i . A deadlock exists in a system if and only if the corresponding wait-for graph contains a cycle. One of the best-known approaches for solving the deadlock problem can be carried out to abort as less deadlocked processes as possible in the wait-for graph. Using graph-theoretic terminology, the strategy is equivalent to finding a feedback vertex set for such a system.

We consider the problem for a particular interconnection network, namely, split-stars, which can be viewed as companion graphs of alternating group graphs. The split-stars act as bridges between the star graphs and the alternating group graphs [6]. Some nice properties of split-stars are given in [5, 6, 7, 8]. In this paper, we derive lower bounds to the size of the minimum feedback vertex sets for split-stars.

2 Preliminaries

All graphs considered in this paper are finite and simple (i.e., without loops and multiple edges). Let $G = (V, E)$ be an undirected graph with vertex set V and edge set E , where $E \subseteq V \times V$. For any set S of vertices in G , the *induced subgraph* $\langle S \rangle$ is the maximal subgraph with vertex set S . A subset $C(G)$ of V is called *cycle-free* if the subgraph of G induced by $C(G)$ has no cycles. A cycle-free set $C(G)$ is *maximum* if the cardinality $|C(G)|$ is maximum among all possible cycle-free sets. Thus, finding a minimum feedback vertex set is equivalent to finding a maximum cycle-free set. We also use $C_m(G)$ to denote

*Email address: cysir@pchome.com.tw (Fu-Hsing Wang).

a certain maximum cycle-free set of the graph G .

A *permutation* is a sequence of elements in which no element appears more than once. Let $p = [p_1, p_2, \dots, p_n]$ be a permutation, where $p_i \in \{1, 2, \dots, n\}$ for all $1 \leq i \leq n$. A *2-exchange* interchanges the first symbol p_1 with the second symbol p_2 . A *3-rotation* rotates the symbols in positions 1, 2 and 3 for some $i \in \{3, 4, \dots, n\}$. The *n -dimensional split-star S_n* is a graph whose vertices are in a one-to-one correspondence with $n!$ permutations, and two vertices u and v of S_n are connected by an edge (u, v) if and only if the permutation of v can be obtained from u by either a 2-exchange or a 3-rotation. We refer to such an edge as a *2-exchange edge* (respectively, *3-rotation edge*) if it is caused by a 2-exchange (respectively, 3-rotation). Vertices u and v are called *adjacent* if edge (u, v) exists, and u is called a *neighbor* of v . For subsets of vertices $A, B \subseteq V$, A and B are adjacent if there exists at least one edge (u, v) where $u \in A, v \in B$. The vertex set of S_n is denoted by $V(S_n)$. For simplicity, we also use V to represent $V(S_n)$ if it will not make confused.

An S_n is a recursive structure that contains n substars. Substar $S_n^i, 1 \leq i \leq n$, is a subgraph of S_n consisting of all the vertices whose rightmost symbol p_n is equal to i , and all the edges induced by these vertices. It is a simple fact that a substar is also an S_{n-1} . Let $C(S_n^i), 1 \leq i \leq n$, denote the subset of the maximum cycle-free set $C_m(S_n)$ in S_n^i , i.e., $C(S_n^i) = C_m(S_n) \cap V(S_n^i)$. Notice that $C_m(S_n) = C(S_n^1) \cup C(S_n^2) \cup \dots \cup C(S_n^n)$. The set consisting of two adjacent vertices u and v is said to be an *exclusive pair*, denoted $X(u, v)$, if $|\{u, v\} \cap C_m(S_n)| \leq 1$, and u and v are called *exclusive*. We use N_{ij} to denote the neighbors of $V(S_n^j)$ in S_n^i , i.e., $N_{ij} = \{u \mid (u, v) \in E, \text{ where } u \in V(S_n^i), v \in V(S_n^j)\}$. Figure 1 depicts an S_3 which contains 6 vertices. Figure 2 illustrates four substars (double triangles) S_4^1, S_4^2, S_4^3 and S_4^4 in S_4 , where symbols a, b, c and d indicate the connection through the same symbol. Note that S_3 in Figure 1 is isomorphic to a substar in Figure 2. We use Figure 2 as an example to illustrate the above notation. For instance, 1234 and 2134 are 2-exchange neighbors since their labels differ only in the first and the second positions, and 1234 is a 3-rotation neighbor of 2314 by means of the rotation of their symbols in positions 1, 2 and 3. (Here, for notational convenience, we write $[1, 2, 3, 4]$ as 1234.) The set represented by the circled vertices is a cycle-free set while the other vertices indicate a feedback vertex set. We will show later on that this cycle-free set is indeed maximum.

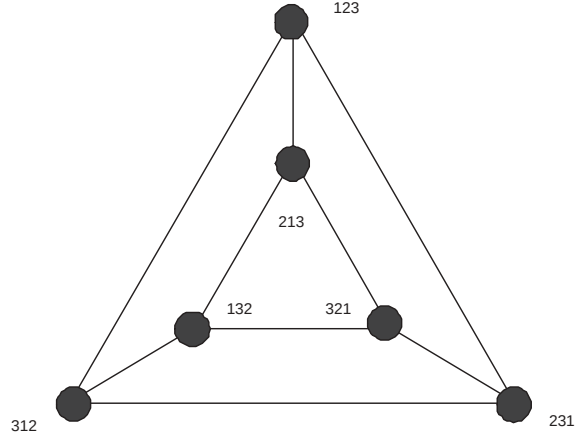


Figure 1: A 3-dimensional split-star S_3 .

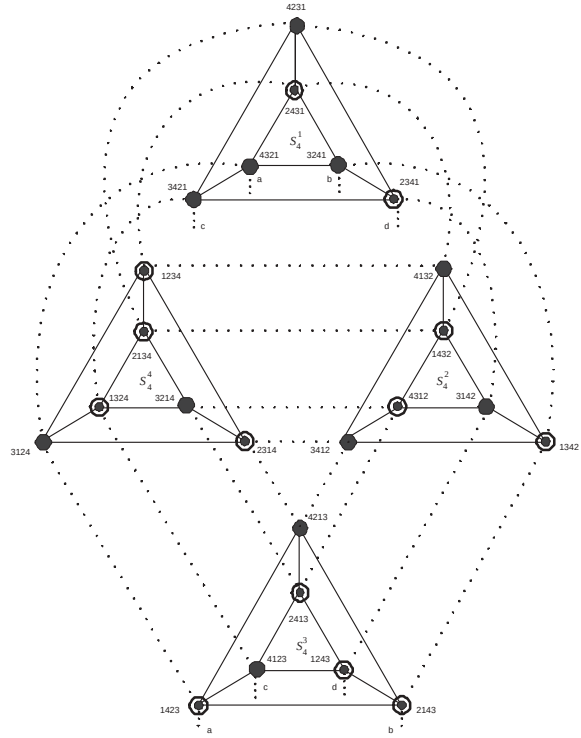


Figure 2: A 4-dimensional split-star S_4 . The set $C_m(S_4)$ is represented by the circled vertices.

3 Lower bounds

Luccio in [18] proved a lower bound for the hypercubes. In [4], Caragiannis et al. proposed a simple formula useful to prove lower bounds in any graphs.

Lemma 1 [4] *For a minimum feedback vertex set $F_m(G)$ in a graph $G = (V, E)$ with maximum degree Δ it holds that*

$$|F_m(G)| \geq \frac{|E| - |V| + 1}{\Delta - 1}.$$

Note that the n -dimensional split-star has $n!$ vertices and $\frac{(2n-3)n!}{2}$ edges and each vertex has degree $2n - 3$. By applying the above formula, a lower bound $\frac{(2n-5)n!+2}{4n-8}$ can easily be achieved for split-stars S_n . Recently, Hsu et al. tried to improve the lower bound of $|F_m(S_n)|$ by determining $F_m(S_4)$ with size at least 11 [13]. However, some flaws exist in their proof. In this paper, we show that $|F_m(S_4)| = 11$ and establish a new lower bound to the size of $F_m(S_n)$.

Lemma 2 *For a substar S_4^i of S_4 , if $|C(S_4^i)| = 4$, then $\langle C(S_4^i) \rangle$ is connected.*

Proof. It is clear that each 3-cycle contains at most two vertices in a cycle-free set. Since S_4^i has two disjoint 3-cycles and $|C(S_4^i)| = 4$, each 3-cycle contains exactly two adjacent vertices of $C(S_4^i)$. Moreover, one of the three 2-exchange edges of S_4^i connects the vertices of $C(S_4^i)$ in different 3-cycles. Thus, $\langle C(S_4^i) \rangle$ is connected.

Q. E. D.

A split-star is a vertex and edge symmetric graph [5, 6]. We now show the adjacency relation between any two substars of S_4 containing four vertices of $C_m(S_4)$.

Lemma 3 *In S_4 , if $|C(S_4^1)| = |C(S_4^2)| = |C(S_4^3)| = 4$, then $C(S_4^1)$ and $C(S_4^2)$ are adjacent.*

Proof. Suppose to the contrary that $C(S_4^1)$ and $C(S_4^2)$ are not adjacent. Since S_4^1 and S_4^2 are connected by exactly four edges and $|C(S_4^1)| = |C(S_4^2)| = 4$, $|N_{12} \cap C_m(S_4)| \geq 2$ and $|N_{21} \cap C_m(S_4)| \geq 2$. Furthermore, since S_4^1 and S_4^2 are not adjacent, $|N_{12} \cap C_m(S_4)| = |N_{21} \cap C_m(S_4)| = 2$. Without loss of generality, we let $C(S_4^1) = \{2431, 4321, 3421, 2341\}$. Then, vertices 4132 and 3142 are forbidden to be vertices of $C(S_4^2)$. Thus, $C(S_4^2) = \{1432, 4312, 3412, 1342\}$. It can be seen that each vertex of S_4^3 is adjacent to a vertex of $C(S_4^1)$ or $C(S_4^2)$. Furthermore, since $|C(S_4^3)| = 4$, by Pigeon Hole Principle, there are at least two

vertices, say u and v , of $C(S_4^3)$ adjacent to $C(S_4^1)$ or $C(S_4^2)$. Moreover, by Lemma 2, both $\langle C(S_4^1) \rangle$ and $\langle C(S_4^2) \rangle$ are connected and $(u, v) \in E$. It reveals that either $\langle \{u, v\} \cup C(S_4^1) \rangle$ or $\langle \{u, v\} \cup C(S_4^2) \rangle$ is cyclic. This contradicts that $\langle C_m(S_4) \rangle$ is acyclic.

Q. E. D.

The size of the maximum cycle-free set in S_4 is shown as follows.

Theorem 4 $|C_m(S_4)| = 13$.

Proof. Obviously, the vertex set $\{123, 312, 132, 321\}$ is a maximum cycle-free set of S_3 (See Figure 1). Therefore, each substar of S_4 has at most four vertices in $C_m(S_4)$. To conclude that $|C(S_4)| = 13$, it suffices to show that $|C_m(S_4)| \neq 14$ because a cycle-free set of order 13 (see Figure 2) indeed exists [13]. Suppose to the contrary that $|C_m(S_4)| = 14$. By Pigeon Hole Principle, there are at least two substars having four vertices in $C_m(S_4)$. By symmetry property, we let $|C(S_4^1)| = |C(S_4^2)| = 4$. Furthermore, by Lemma 2, both $\langle C(S_4^1) \rangle$ and $\langle C(S_4^2) \rangle$ are connected. There are three cases to be considered depending on the values of $|C(S_4^3)|$ and $|C(S_4^4)|$.

Case 1. $|C(S_4^3)| = 4$ and $|C(S_4^4)| = 2$.

By Lemmas 2 and 3, $C(S_4^1), C(S_4^2)$ and $C(S_4^3)$ are all connected and pairwise adjacent. So, $\langle C(S_4^1) \cup C(S_4^2) \cup C(S_4^3) \rangle$ must have cycles. Thus, this case is impossible.

Case 2. $|C(S_4^3)| = 2$ and $|C(S_4^4)| = 4$.

This proof is similar to *Case 1*.

Case 3. $|C(S_4^3)| = |C(S_4^4)| = 3$.

The subgraph induced by N_{12} is indeed a 4-cycle. Therefore, $|N_{12} \cap C_m(S_4)| \leq 3$. Furthermore, since $|V(S_4^1)| = 6$ and $|C(S_4^1)| = 4$, there are at least two vertices of N_{12} that belong to $C(S_4^1)$. So, $|N_{12} \cap C_m(S_4)| = 3$ or 2. Similarly, $|N_{21} \cap C_m(S_4)| = 3$ or 2. We first show that $|N_{12} \cap C_m(S_4)| \neq |N_{21} \cap C_m(S_4)|$. If $|N_{12} \cap C_m(S_4)| = |N_{21} \cap C_m(S_4)| = 3$, then there exist at least two vertices in $N_{12} \cap C_m(S_4)$ adjacent to $N_{21} \cap C_m(S_4)$. It contradicts that $C_m(S_4)$ is acyclic. For the case $|N_{12} \cap C_m(S_4)| = |N_{21} \cap C_m(S_4)| = 2$. Without loss of generality, we let $C(S_4^1) = \{2431, 4321, 3421, 2341\}$. Then, $T = \{4312, 3412\}$ is a subset of $C(S_4^2)$. Therefore, $C(S_4^2)$ is either equal to $A = T \cup \{4132, 3142\}$ or $B = T \cup \{1432, 1342\}$. If $C(S_4^2) = A$, then vertices 4132 and 3142 are adjacent to vertices 2431 and 2341, respectively. That is, $\langle C(S_4^1) \cup A \rangle$ has cycles. Otherwise, $C(S_4^2) = B$. Since $|N_{23} \cap C_m(S_4)| = 3$, $|N_{32} \cap C_m(S_4)| \leq 2$ and thus $|\{1423, 4123\} \cap C(S_4^3)| \geq 1$. However, vertices

1423 and 4123 are exclusive. So, $\{1423, 4123\}$ contains exactly one vertex in $C(S_4^3)$. Similarly, $\{4213, 2413\}$ also contains exactly one vertex in $C(S_4^3)$. Therefore, $\{1243, 2143\}$ must contain exactly one vertex in $C(S_4^3)$. By the same reason, sets $\{1324, 3124\}$, $\{3214, 2314\}$, and $\{1234, 2134\}$ all have exactly one vertex in $C(S_4^4)$. Two subcases are now to be considered.

Subcase 3.1. Vertex $4213 \in C(S_4^3)$.

Since vertices 4213 and 2143 (respectively, 2314) are exclusive and $|X(2143, 1243) \cap C(S_4^3)| = 1$ (respectively, $|X(2314, 3214) \cap C(S_4^4)| = 1$), $1243 \in C(S_4^3)$ (respectively, $3214 \in C(S_4^4)$). Furthermore, $1423 \in C(S_4^3)$ (respectively, $1234 \in C(S_4^4)$) because vertices 1243 and 4123 (respectively, 2314) are exclusive and $|X(4123, 1423) \cap C(S_4^3)| = 1$ (respectively, $|X(2134, 1234) \cap C(S_4^4)| = 1$). Moreover, vertex 1324 is in $C(S_4^4)$ since $3124 \notin C(S_4^4)$ due to $X(2134, 1234)$. Consequently, a cycle: $4213 \rightarrow 1423 \rightarrow 4321 \rightarrow 3421 \rightarrow 1324 \rightarrow 3214 \rightarrow 4312 \rightarrow 3412 \rightarrow 4213$ exists, a contradiction.

Subcase 3.2. Vertex $4213 \notin C(S_4^3)$.

Clearly, $2413 \in C(S_4^3)$ to ensure that $|\{2413, 4213\} \cap C(S_4^3)| = 1$. Then vertex $3214 \notin C(S_4^4)$ for their exclusivity respect to 4312. Furthermore, vertex 2314 $\in C(S_4^4)$ to ensure that $|\{3214, 2314\} \cap C(S_4^4)| = 1$.

Subcase 3.2.1. Vertex $4123 \in C(S_4^3)$.

Obviously, vertex 1324 $\notin C(S_4^3)$ due to $X(1324, 4123)$ respect to 3421. To make sure that $|\{1324, 3124\} \cap C(S_4^4)| = 1$, vertex 3124 must be in $C(S_4^4)$. In such a situation, a cycle: $4123 \rightarrow 2413 \rightarrow 4312 \rightarrow 3412 \rightarrow 2314 \rightarrow 3124 \rightarrow 4321 \rightarrow 3421 \rightarrow 4123$ exists, a contradiction.

Subcase 3.2.2. Vertex $4123 \notin C(S_4^3)$.

Clearly, $1423 \in C(S_4^3)$. Consider $X(1423, 3124)$, we get $3124 \notin C(S_4^4)$. So, 1324 is another vertex in $C(S_4^4)$. Finally, there remains to discuss exclusive pairs $X(1234, 2134)$ and $X(1243, 2143)$. Clearly, the subgraphs induced by $A = \{1324, 1423\} \cup C(S_4^1)$ and $B = \{2413, 2314\} \cup C(S_4^2)$ are connected. Moreover, it is easy to see that each vertex $u \in X(1234, 2134)$ and $v \in X(1243, 2143)$ are adjacent to both A and B . This implies that $\langle \{u, v\} \cup A \cup B \rangle$ has cycles.

We conclude that $|N_{12} \cap C_m(S_4)| \neq |N_{21} \cap C_m(S_4)|$. By symmetry property, we are now at a position to consider only the case $|N_{12} \cap C_m(S_4)| = 3$ and $|N_{21} \cap C_m(S_4)| = 2$ to complete the proof. At first, we want to explore the vertices of $C(S_4^2)$ that is helpful to further decide the vertices of $C(S_4^3)$ and $C(S_4^4)$. Without loss of generality, we let $C(S_4^1) = \{2431, 3241, 2341, 3241\}$. Since

$|N_{21} \cap C_m(S_4)| = 2$, $\{4312, 3412\} \subseteq C(S_4^2)$ and $1432 \in C(S_4^2)$. Since vertices 1432 and 4312 are adjacent and belong to $C(S_4^2)$, their common neighbor 3142 cannot be in $C(S_4^2)$; otherwise these three vertices induce a 3-cycle in $\langle C(S_4^2) \rangle$. In addition, 4132 cannot be a vertex of $C(S_4^2)$; otherwise 4132, 1432, 4312 and 3412 induce a 4-cycle in $\langle C(S_4^2) \rangle$. Therefore, $1342 \in C(S_4^2)$. Then, we proceed to explore the vertices of $C(S_4^3)$. Since vertices 3241 and 1342 are adjacent and in $C_m(S_4)$, their common neighbor 2143 cannot be in $C(S_4^3)$. Furthermore, vertices 4123 and 1243 are exclusive because both of them are adjacent to the vertices in $C(S_4^1)$. Similarly, $X(4213, 2413)$ exists. Then, since $|C(S_4^3)| = 3$, vertex $1423 \in C(S_4^3)$. Moreover, 4213 is also in $C(S_4^3)$; otherwise, 2413 and some vertex of $X(4123, 1243)$ are in S_4^3 that contradicts the existence of exclusive pairs $X(2413, 4123)$ and $X(2413, 1243)$. Thus, $\{1423, 4213\} \subseteq C(S_4^3)$. Now, we determine the members of $C(S_4^4)$. Since vertices 3412 and 4213 are adjacent and belong to $C_m(S_4)$, their common neighbor 2314 cannot be in $C(S_4^4)$. Since $|C(S_4^3)| = 3$ and the existence of $X(2134, 3214)$ and $(1234, 1324)$, vertex $1423 \in C(S_4^3)$. Consider vertex 2134. It can be seen that 2134 and each neighbor in S_4^4 form exclusive pairs. This implies that $C(S_4^4) = \{2134, 3124\}$ which contradicts that $|C(S_4^4)| = 3$. Consequently, $2134 \notin C(S_4^4)$. Since the existence of $X(1324, 3214)$ and $X(1234, 3124)$, $|C(S_4^4)| \leq 2$, a contradiction.

Q. E. D.

Obviously, $|F_m(S_3)| = 2$. By Theorem 4, the size of the minimum feedback vertex of S_4 is 11. Note that the n -star has $n!$ vertices and hence a lower bound $\frac{11n!}{24}$ for $n \geq 4$ is immediately obtained. It is clear that this new lower bound improves the previous one when $4 \leq n \leq 7$. Finally, We conclude the result as follows.

Corollary 5

$$|F_m(S_n)| \geq \begin{cases} \frac{11n!}{24}, & \text{if } 4 \leq n \leq 7, \\ \frac{(2n-5)n!+2}{4n-8}, & \text{if } n \geq 8. \end{cases}$$

References

- [1] S.B. Akers and B. Kirshnamurthy, A group-theoretic model for symmetric interconnection networks, *IEEE Transactions on Computers*, Vol. **38** No. 4, pp.555–565, 1989.
- [2] V. Bafna, P. Berman, and T. Fujito, A 2-approximation algorithm for the undirected

- feedback vertex set problem, *SIAM Journal on Discrete Mathematics*, Vol. **12** No. 3, pp.289–297, 1999.
- [3] M. Cai, X. Deng, and W. Zang, An approximation algorithm for feedback vertex sets in tournaments, *SIAM Journal on Computing*, Vol. **30** No. 6, pp.1993–2007, 2001.
- [4] I. Caragiannis, C. Kaklamanis, and P. Kanellopoulos, New bounds on the size of the minimum feedback vertex set in butterflies, *Information Processing Letters*, Vol. **83** No. 5, pp.275–280, 2002.
- [5] Eddie Cheng and Marc J. Lipman, Increasing the connectivity of split-stars, *Congressus Numerantium*, Vol. **146**, pp.97–111, 2000.
- [6] Eddie Cheng, Marc J. Lipman and Hyungju Park, Super connectivity of star graphs and split-stars, *Ars Combinatoria*, Vol. **52**, pp.107–116, 2001.
- [7] Eddie Cheng and Marc J. Lipman, Disjoint paths in split-stars, *Congressus Numerantium*, Vol. **139**, pp.21–32, 1999.
- [8] Eddie Cheng and Marc J. Lipman, Fault tolerant routing in split-stars and alternating group graphs, *Congressus Numerantium*, Vol. **137**, pp.47–63, 1999.
- [9] G. Even, J. Naor, B. Schieber, and M. Sudan, Approximating minimum feedback sets and multicuts in directed graphs, *Algorithmica*, Vol. **20** No. 2, pp.151–174, 1998.
- [10] R. Focardi, F.L. Luccio, and D. Peleg, Feedback vertex set in hypercubes, *Information Processing Letters*, Vol. **76** No. 1-2, pp.1–5, 2000.
- [11] P. Festa, P.M. Pardalos, and M.G.C. Resende, Feedback set problems, in: *Handbook of Combinatorial Optimization*, Supplement Vol. **A**, Kluwer Academic Publishers, pp.209–258, 1999.
- [12] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-completeness*, Freeman, San Francisco, 1979.
- [13] C.J. Hsu and I.J. Lin, Feedback vertex set in split-stars and alternating groups, in: *Proceedings of the 2002 ICS: Workshop on Algorithms and Computational Molecular Biology*, Hualien, Taiwan, 2002.
- [14] M.X. Goemans and D.P. Williamson, Primal-Dual approximation algorithms for feedback problems in planar graphs, in: *Lecture Notes in Computer Science 1084*, Springer-Verlag, pp.147–161, 1996.
- [15] J. B. Jensen and G. Gutin, *Digraphs: Theory, Algorithms and Applications*, Springer-Verlag, London, 2001.
- [16] Y.D. Liang and M.S. Chang, Minimum feedback vertex set in cocomparability and convex Bipartite graphs, *Acta Informatica*, Vol. **34** No. 5, pp.337–346, 1997.
- [17] C.L. Lu and C.Y. Tang, A linear-time algorithm for the weighted feedback vertex problem on interval graphs, *Information Processing Letters*, Vol. **61** No. 2, pp.107–111, 1997.
- [18] Flaminia L. Luccio, Almost exact minimum feedback vertex set in meshes and butterflies, *Information Processing Letters*, Vol. **66** No. 2, pp.59–64, 1998.
- [19] A. Silberschatz, P.B. Galvin, and G. Gagne, *Operating Systems Concepts*, Sixth Edition, John Wiley and Sons, Inc., New Jersey, 2003.
- [20] A. Shamir, A linear time algorithm for finding minimum reducible graphs, *SIAM Journal on Computing*, Vol. **8** No. 4, pp.645–655, 1979.
- [21] C. Wang, E.L. Lloyd, and M.L. Sofra, Feedback vertex sets and cyclically reducible graphs, *Journal of the ACM*, Vol. **32** No. 2, pp.296–313, 1985.
- [22] F.H. Wang, Y.L. Wang and J.M. Chang, Feedback vertex sets in star graphs, *Information Processing Letters*, Vol. **89** No. 4, pp.203–208, 2004.

Generalized pancake graphs 的漢米爾頓容錯性質

賴盈志，洪春男

大葉大學資訊工程學系

E-mail(r9106032@mail.dyu.edu.tw)

摘要

在本篇論文中，我們研究 n -dimensional 2-sided pancake graphs ($2P_n$) 容錯性質。令 $n \geq 3$ ， $F \subseteq (V(2P_n) \cup E(2P_n))$ ，當 $|F| \leq (n-2)$ ， $2P_n - F$ 為漢米爾頓圖形 (Hamiltonian graphs)，而當 $|F| \leq (n-3)$ 時， $2P_n - F$ 為漢米爾頓連通圖 (Hamiltonian connected graphs)。

關鍵字：pancake graphs, fault-tolerant embedding, Hamiltonian path, Hamiltonian cycle, Hamiltonian connected graph.

1. 序論

容錯性質是連結網路上很重要的主題，特別當連結網路上點與邊的量越大，發生錯誤的機率會提高很多，容錯性質就更形重要。

許多研究者已對特定連結網路的容錯性質加以探討，在 [2] 中謝孫源等教授探討當壞掉一定數目的點與邊後，Arrangement graphs 仍然保有漢米爾頓迴路；在 [3] 裡探討到 pancake graphs 的容錯性質，其中發現當錯誤的邊或點的個數不超過 $n-3$ 時，仍然為漢米爾頓圖形，而當錯誤的邊或點的個數不超過 $n-4$ 時，仍然為漢米爾頓連通圖。Justan 等作者在 [4] 提出 pancake graphs 的一般化圖形 n -dimensional m -sided pancake graphs，而我們在本篇論文中探討其中 n -dimensional 2-sided pancake graphs 的容錯性質。 n -dimensional 2-sided pancake graphs 每個點的分支度為 n ，我們證明出當錯誤的邊或點的個數不超過 $n-2$ 時，仍然為漢米爾頓圖形，而當錯誤的邊或點的個數不超過 $n-3$ 時，仍然為漢米爾頓連通圖。

2. 基本定義

令 $G = (V, E)$ 為一圖形， V 為點集合、 E 為邊集合，也就是 $E = \{(u, v) \mid u, v \in V\}$ 。令 $x \in$

V ， $\deg_G(x)$ 代表在 G 中 x 的分支度 (degree)。

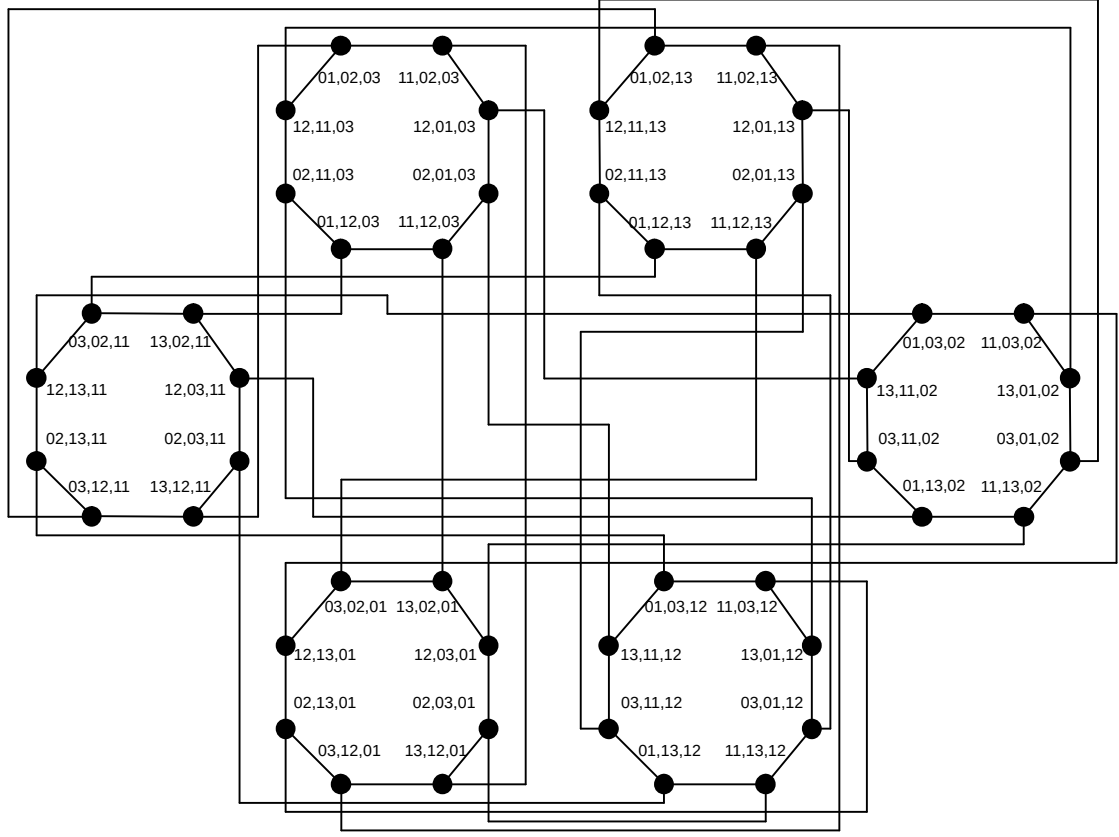
當 $(u, v) \in E$ 時，即代表 u 和 v 兩點為相鄰 (adjacent)。用 $\langle v_0, v_1, v_2, \dots, v_k \rangle$ 表示一條由 v_0 到 v_k 的路徑 (path)，其中每一對 $(v_i, v_{i+1}) \in E$ 。而路徑 Q 的長度是指 Q 上邊的個數，我們可把路徑 $\langle v_0, v_1, v_2, \dots, v_k \rangle$ 改寫成 $\langle v_0, Q_1, v_i, v_{i+1}, \dots, v_j, Q_2, v_t, \dots, v_k \rangle$ ，其中 Q_1 為 $\langle v_0, v_1, v_2, \dots, v_i \rangle$ ，而 Q_2 為 $\langle v_j, v_{j+1}, v_{j+2}, \dots, v_t \rangle$ 。當路徑的起點與終點為同一點時，此路徑即稱為迴路 (cycle)。我們使用 $d(u, v)$ 代表 u 與 v 之間的距離，也就是其最短路徑的長度。

經過 V 上所有的點，並且所有的點與邊都不重複的路徑稱為漢米爾頓路徑 (Hamiltonian path)；恰好經過每一點一次的迴路稱為漢米爾頓迴路 (Hamiltonian cycle)；有漢米爾頓迴路的圖形稱為漢米爾頓圖形 (Hamiltonian graph)；若是在圖形中的任兩點間都有漢米爾頓路徑，則此圖形稱為漢米爾頓連通圖 (Hamiltonian connected graph)。一個圖形壞掉的點數與邊數總合為 k ，仍然含有漢米爾頓迴路的話，則此圖形稱為 k -漢米爾頓圖形 (k -Hamiltonian graph)。一個圖形壞掉的點數與邊數總合為 k ，仍然為漢米爾頓連通圖的話，則此圖形稱為 k -漢米爾頓連通圖 (k -Hamiltonian connected graph)。

在本篇論文中，使用 $\langle n \rangle$ 來代表集合 $\{1, 2, \dots, n\}$ ，以 $2P_n$ 來代表 n -dimensional 2-sided pancake graphs，其點集合的規則為： $V(2P_n) = \{((k_1, u_1), (k_2, u_2), \dots, (k_n, u_n)) \mid u_i \in \langle n \rangle, k_j \in \{0, 1\}, 1 \leq i, j \leq n, \text{ 當 } r \neq s \text{ 時, } u_r \neq u_s\}$

而邊的定義如下：

當 $1 \leq i \leq n$ ，點 u 與點 v 相連， $u = ((k_1, u_1), (k_2, u_2), \dots, (k_i, u_i), (k_{i+1}, u_{i+1}), \dots, (k_n, u_n))$ ， $v = (((k_i + 1)_{\text{mod } 2}, u_i), ((k_{i-1} + 1)_{\text{mod } 2}, u_{i-1}), \dots, ((k_1 + 1)_{\text{mod } 2}, u_1), (k_{i+1}, u_{i+1}), \dots, (k_n, u_n))$ ，我們稱此邊為 i -dimensional 的邊， $2P_3$ 的圖形如圖一所示。



圖一： $2P_3$ 的圖形

3. n -dimensional 2-sided pancake graphs 的重要性質

令 $u = ((k_1, u_1), (k_2, u_2), \dots, (k_i, u_i), (k_{i+1}, u_{i+1}), \dots, (k_n, u_n))$ 為 $2P_n$ 的任意點，且令 $(u)_i = (k_i, u_i)$ 為 u 的第 i 個元素，其中 $1 \leq i \leq n$ 。根據 $2P_n$ 的定義， u 恰有一個以 i -dimensional 的邊相連的鄰點，稱為 i -neighbor，我們用 $i(u)$ 來表示，可簡單推論到 $i(i(u)) = u$ 。

在 $2P_n$ 中擁有相同 (k_n, u_n) 的點所形成的子圖為一個 $2P_{n-1}$ 的圖形，我們用 $2P_{n-1}(k_n, u_n)$ 代表這個子圖，因此一個 $2P_n$ 的圖形是由 $2n$ 個 $2P_{n-1}$ 的圖形所組成，分別為 $2P_{n-1}(0, 1), 2P_{n-1}(0, 2), \dots, 2P_{n-1}(0, n), 2P_{n-1}(1, 1), 2P_{n-1}(1, 2), \dots, 2P_{n-1}(1, n)$ 。

我們用 $E^{(a,i),(b,j)}$ 來代表 $2P_{n-1}(a, i)$ 跟 $2P_{n-1}(b, j)$ 之間的邊集合， $a, b \in \{0, 1\}$ ， $i, j \in \langle n \rangle$ ， $i \neq j$ 。在證明 $2P_n$ 的容錯性質時，我們需要用到以下幾個輔助定理。

輔助定理 1: $|E^{(a,i),(b,j)}| = (n-2)! \cdot 2^{n-2}$ ， $a, b \in \{0, 1\}$ ， $i, j \in \langle n \rangle$ ， $i \neq j$ 。

證明: $2P_{n-1}(a, i)$ 與 $2P_{n-1}(b, j)$ 中相連的邊數，相當於在 $2P_{n-1}(a, i)$ 中找出 $((b+1) \bmod 2, j)$

$j), \dots, (a, i)$ 的點數，此點數與 $2P_{n-2}$ 相同，根據 [4] 得知 $2P_{n-2}$ 的點數為 $(n-2)! \cdot 2^{n-2}$ ，因此 $|E^{(a,i),(b,j)}| = (n-2)! \cdot 2^{n-2}$ 。

輔助定理 2: 令 u, v 為 $2P_n$ 中的相異點， $(u)_n = (v)_n$ ， $d(u, v) \in 2$ ，我們可得知 $2P_{n-1}((n(u))_n) \neq 2P_{n-1}((n(v))_n)$ 。

證明: 假設 $u \neq v$ ， $2P_{n-1}((n(u))_n) = 2P_{n-1}((n(v))_n)$ ，即代表 $(u)_1 = (v)_1$ ，令 $u = ((a_1, i_1), (a_2, i_2), \dots, (a_k, i_k), \dots, (a_n, i_n))$ ， $a_j \in \{0, 1\}$ ， $i_j \in \langle n \rangle$ ，與 u 相鄰的點為 $((a_k+1) \bmod 2, i_k), ((a_{k-1}+1) \bmod 2, i_{k-1}), \dots, ((a_1+1) \bmod 2, i_1), \dots, (a_n, i_n)$ ，因為 $d(u, v) \leq 2$ ，所以只能使 $(v)_1 = (((a_1+1) \bmod 2 + 1) \bmod 2, i_1) = (a_1, i_1) = (u)_1$ ，因此 $u = v$ ，與假設發生矛盾，得證 $2P_{n-1}((n(u))_n) \neq 2P_{n-1}((n(v))_n)$ 。

輔助定理 3: 當 $n \geq 4$ ，在 $2P_n$ 中任選出五個以上的 $2P_{n-1}$ 子圖，選出的 $2P_{n-1}$ 子圖要為 $(n-4)$ -漢米爾頓連通圖，而這些子圖所形成的圖形中，壞掉點或邊的個數不超過 $n-2$ ，任兩個不同子圖中兩點，必然存在漢米爾頓路徑。

證明: 根據 [1] 中的定理 6.2.14 得知，令 x 為總點數， u, v 為任兩個不相連的點，當 $\deg_G(u) + \deg_G(v) \geq x+1$ 成立時，則此圖形為漢米爾頓連通圖。我們把 $2P_n$ 中的 $2P_{n-1}$ 子圖當成一個點，令有 k 個 $2P_{n-1}$ 子圖，根據定義得

知每個子圖最多只會有一個子圖無法相連，所以每個子圖的分支度為 $k - 2$ ，因此之前的判斷式為 $(k - 2) + (k - 2) \geq k + 1$ ，可得知當 $k \geq 5$ ，將 $2P_{n-1}$ 子圖當成一點之後，這些點所形成的圖形為漢米爾頓連通圖。

我們得知當 $n \geq 4$ ， $2P_{n-1}$ 是 $(n - 4)$ -漢米爾頓連通圖，因此 $2P_{n-1}$ 子圖中對任兩點間必存在漢米爾頓路徑，由於 $2P_{n-1}$ 子圖所形成的圖形為漢米爾頓連通圖，而且有邊相連的子圖間至少存在 6 條無錯誤的邊，所以任兩個不同子圖中的兩點可藉由各個 $2P_{n-1}$ 子圖所形成的漢米爾頓路徑相連接，因此任兩個不同子圖中的兩點，必然存在漢米爾頓路徑。

輔助定理 4： $2P_n$ 為點對稱圖形。

證明：對任兩點都存在轉換函數 F ，使得 $2P_n$ 中的任意相連兩點，使用 F 轉換後亦會相連，反之亦然，則 $2P_n$ 為點對稱圖形。

轉換函數： $F(k_i, a_i) = (F_1(k_i, a_i), F_2(k_i, a_i))$ ，
 $F((k_i + 1)_{\text{mod } 2}, a_i) = (F_1((k_i, a_i) + 1)_{\text{mod } 2}, F_2(k_i, a_i))$ ， $1 \leq i \leq n$ 。

對任兩相鄰點 $((l_1, b_1), (l_2, b_2), \dots, (l_i, b_i), (l_{i+1}, b_{i+1}), \dots, (l_n, b_n))$ 與 $((l_{i+1} + 1)_{\text{mod } 2}, b_i), ((l_{i-1} + 1)_{\text{mod } 2}, b_{i-1}), \dots, ((l_1 + 1)_{\text{mod } 2}, b_1), (l_{i+1}, b_{i+1}), \dots, (l_n, b_n))$ 來說，經過轉換後的兩點為 $((F_1(l_1, b_1), F_2(l_1, b_1)), (F_1(l_2, b_2), F_2(l_2, b_2)), \dots, (F_1(l_i, b_i), F_2(l_i, b_i)), (F_1(l_{i+1}, b_{i+1}), F_2(l_{i+1}, b_{i+1})), \dots, (F_1(l_n, b_n), F_2(l_n, b_n)))$ 與 $((F_1((l_i, b_i) + 1)_{\text{mod } 2}, F_2(l_i, b_i)), (F_1((l_{i-1}, b_{i-1}) + 1)_{\text{mod } 2}, F_2(l_{i-1}, b_{i-1})), \dots, (F_1((l_1, b_1) + 1)_{\text{mod } 2}, F_2(l_1, b_1)), (F_1((l_{i+1}, b_{i+1}), F_2(l_{i+1}, b_{i+1})), \dots, (F_1(l_n, b_n), F_2(l_n, b_n)))$ ；根據 $2P_n$ 的點連接定義得知此兩點亦相連。反之，將前面的順序倒過來，即可得到轉換後相連的兩點，在轉換前也必然相連。因此 $2P_n$ 為點對稱圖形。

4. n -dimensional 2-sided pancake graphs 是 $(n - 2)$ -漢米爾頓圖形和 $(n - 3)$ -漢米爾頓連通圖

在此節中我們要用數學歸納法來證明 $2P_n$ 是 $(n - 2)$ -漢米爾頓圖形和 $(n - 3)$ -漢米爾頓連通圖，輔助定理 5 是數學歸納法裡的 base case，而輔助定理 6、7 為歸納法推論的部分。

輔助定理 5： $2P_3$ 是 1-漢米爾頓圖形和漢米爾頓連通圖。

證明：令 $\{f\} \subseteq (V(2P_3) \cup E(2P_3))$ ，我們需證明 $2P_3 - \{f\}$ 是漢米爾頓圖形。由輔助定理 4 得知， $2P_3$ 是點對稱圖形，因此我們只需證明，當 $f = (01, 02, 03)$ ， $f \in \{((01, 02, 03), (11, 02, 03)), ((01, 02, 03), (12, 11, 03)), ((01, 02, 03), (13, 12, 11))\}$ ， $2P_3 - \{f\}$ 仍有漢米爾頓迴路，附錄中的表 1 列出 $2P_3 - \{f\}$ 後仍存在的漢米爾頓迴路，因此 $2P_3$ 為 1-漢米爾頓圖形。

為了證明 $2P_3$ 是漢米爾頓連通圖，我們找出任兩點 u, v 都存在漢米爾頓路徑。由於 $2P_3$ 是點對稱圖形，只需討論 $u = (01, 02, 03)$ ， v 為與 u 相異的任意點。在附錄中的表 2 列出所有漢米爾頓路徑，因此 $2P_3$ 為漢米爾頓連通圖。

輔助定理 6：當 $n \geq 4$ 時，假設 $2P_{n-1}$ 是 $(n - 3)$ -漢米爾頓圖形和 $(n - 4)$ -漢米爾頓連通圖，則 $2P_n$ 是 $(n - 2)$ -漢米爾頓圖形。

證明：令 $F \subseteq (V(2P_n) \cup E(2P_n))$ 且 $|F| = n - 2$ ，當 $n \geq 4$ ，對於在相異 $i \neq j \in \langle n \rangle$ 的 $2P_{n-1}(a, i), 2P_{n-1}(b, j)$ 來說，其中 $a, b \in \{0, 1\}$ ， $|E^{(a,i),(b,j)} - F| \geq (n - 2)! * 2^{n-2} - (n - 2) \geq 6$ 。因此 $2P_{n-1}(a, i), 2P_{n-1}(b, j)$ 之間至少有 6 條沒有壞掉的邊。

假設 $|F \cap 2P_{n-1}(a, i)|$ 的個數為所有子圖中最大的。

分類 1： $|F \cap 2P_{n-1}(a, i)| = n - 2$ ，其中 $a \in \{0, 1\}$ ， $i \in \langle n \rangle$ 。

令 $f \in F$ ，根據假設得知，在 $2P_{n-1}(a, i) - (F - \{f\})$ 中存在漢米爾頓迴路 Q ，我們把 Q 寫成 $\langle u, Q_1, v, f, u \rangle$ 。由於 $d(u, v) \leq 2$ ，根據輔助定理 2， $2P_{n-1}((n(u))_n) \neq 2P_{n-1}((n(v))_n)$ 。然後根據輔助定理 3， $n(u)$ 與 $n(v)$ 之間存在 $2P_n - 2P_{n-1}(a, i)$ 的漢米爾頓路徑 Q_2 。所以在 $2P_n - F$ 中存在 $\langle u, n(u), Q_2, n(v), v, Q_1, u \rangle$ 所形成的漢米爾頓迴路。

分類 2： $|F \cap 2P_{n-1}(a, i)| = n - 3$ ，而 $|F \cap 2P_{n-1}(b, j)| \leq 1$ ， $a, b \in \{0, 1\}$ ， $i, j \in \langle n \rangle$ 。

令 $\forall 2P_{n-1}(c, k) \neq 2P_{n-1}(a, i)$ 且 $2P_{n-1}(c, k) \neq 2P_{n-1}(b, j)$ ， $|F \cap 2P_{n-1}(c, k)| = 0$ ，其中 $c \in \{0, 1\}$ ， $k \in \langle n \rangle$ 。由於 $|E^{(a,i),(c,k)} - F| \geq (n - 2)! * 2^{n-2} - (n - 2)$ ，所以在 $E^{(a,i),(c,k)}$ 中存在無錯誤的邊 (u, v) 其中 $u \in V(2P_{n-1}(a, i))$ ， $v \in V(2P_{n-1}(c, k))$ 。而在 $E^{(b,j),(c,k)}$ 中亦存在無錯誤的邊 (x, y) 其中 $x \in V(2P_{n-1}(b, j))$ ， $y \in V(2P_{n-1}(c, k))$ ，其中 $v \neq y$ 。根據假設得知，在 $2P_{n-1}(a, i) - F$ 中存在漢米爾頓迴路 C_1 ，在 $2P_{n-1}(b, j) - F$ 中存在漢米爾頓迴路 C_2 ，且在 $2P_{n-1}(c, k)$ 中對於 v, y 之間存在漢米爾頓路徑 Q_3 。

我們把 C_1 寫成 $\langle u, z_1, Q_1, z_2, u \rangle$ ， C_2 寫成 $\langle x, z_3, Q_2, z_4, x \rangle$ 。由於 $d(z_1, z_2) \leq 2, d(z_3, z_4) \leq 2$ ，根據輔助定理 2，可得知 $2P_{n-1}((n(z_1))_n) \neq 2P_{n-1}((n(z_2))_n), 2P_{n-1}((n(z_3))_n) \neq 2P_{n-1}((n(z_4))_n)$ ，所以我們就可從 z_1 跟 z_2 中選一個出來，令選出的是 z_1 ，而從 z_3 跟 z_4 中也選出一個，令選出的是 z_3 ，使得 $2P_{n-1}((n(z_1))_n) \neq 2P_{n-1}((n(z_3))_n)$ ，並且 $(z_1, n(z_1))$ 跟 $(z_3, n(z_3))$ 也是無錯誤的邊。根

據**輔助定理 3** 得知, $n(z_1), n(z_3)$ 之間存在 $2P_n - 2P_{n-1}(a, i) - 2P_{n-1}(b, j) - 2P_{n-1}(c, k)$ 的漢米爾頓路徑 Q_4 。所以對於 $2P_n - F$ 來說, $\langle u, v, Q_3, y, x, z_4, Q_2, z_3, n(z_3), Q_4, n(z_1), z_1, Q_1, z_2, u \rangle$ 形成了漢米爾頓迴路。

分類 3: $|F \cap 2P_{n-1}(a, i)| \leq n - 4, a \in \{0, 1\}, i \in \langle n \rangle$ 。

令 $(u, v) \in E^{(a,i),(b,j)}$, 且 $u \in V(2P_{n-1}(a, i)), v \in V(2P_{n-1}(b, j))$, $(u, v) \notin F$, 由於每個 $2P_{n-1}$ 子圖錯誤的個數不超過 $n - 4$, 因此每個 $2P_{n-1}$ 子圖都為漢米爾頓連通圖。根據**輔助定理 3**, 在 $2P_n - F$ 中對於 u, v 之間存在漢米爾頓路徑 Q_1 , 而在 $2P_n - F$ 中 $\langle u, Q_1, v, u \rangle$ 則形成漢米爾頓迴路。

輔助定理 7: 當 $n \geq 4$ 時, 假設 $2P_{n-1}$ 是 $(n - 3)$ -漢米爾頓圖形和 $(n - 4)$ -漢米爾頓連通圖, 則 $2P_n$ 是 $(n - 3)$ -漢米爾頓連通圖。

證明: 令 $F \subseteq (V(2P_n) \cup E(2P_n))$ 且 $|F| = n - 3$ 。令 s, t 為 $2P_n - F$ 中任意的兩點, 我們的目標就是在 $2P_n - F$ 中找出以 s, t 為端點的漢米爾頓路徑。當 $n \geq 4$, 對於 $i \neq j \in \langle n \rangle$ 中的 $2P_{n-1}(a, i), 2P_{n-1}(b, j)$ 來說, 其中 $a, b \in \{0, 1\}$, $|E^{(a,i),(b,j)} - F| \geq (n - 2)! * 2^{n-2} - (n - 3) \geq 7$ 。因此在 $2P_{n-1}(a, i), 2P_{n-1}(b, j)$ 中至少有 7 條無錯誤的邊。

假設 $|F \cap 2P_{n-1}(a, i)|$ 的個數為所有子圖中最多的。

分類 1: $|F \cap 2P_{n-1}(a, i)| = n - 3, a \in \{0, 1\}, i \in \langle n \rangle$ 。

子分類 1.1: $s, t \in V(2P_{n-1}(a, i))$ 。

令 $f \in F$, 根據假設, 在 $2P_{n-1}(a, i) - (F - \{f\})$ 中對於 s, t 之間存在漢米爾頓迴路 Q , 可將 Q 寫成 $\langle s, Q_1, x, f, y, Q_2, t \rangle$, 其中 $d(x, y) \leq 2$, 根據**輔助定理 2**, $2P_{n-1}((n(x))_n) \neq 2P_{n-1}((n(y))_n)$ 。由**輔助定理 3** 可知, $n(x)$ 與 $n(y)$ 之間存在 $2P_n - 2P_{n-1}(a, i)$ 的漢米爾頓路徑 Q_3 。所以在 $2P_n - F$ 中對於 s, t 之間存在漢米爾頓路徑 $\langle s, Q_1, x, n(x), Q_3, n(y), y, Q_2, t \rangle$ 。

子分類 1.2: $s \in V(2P_{n-1}(a, i)), t \in V(2P_{n-1}(b, j)), 2P_{n-1}(a, i) \neq 2P_{n-1}(b, j)$, 其中 $a, b \in \{0, 1\}, i, j \in \langle n \rangle$ 。

根據假設, 在 $2P_{n-1}(a, i) - F$ 中存在漢米爾頓迴路 C_1 。 $C_1 = \langle s, u, Q_1, v, s \rangle$ 。由於 $d(u, v) \leq 2$, 根據**輔助定理 2** 得知 $2P_{n-1}((n(u))_n) \neq 2P_{n-1}((n(v))_n)$ 。我們選擇 u 或 v (假設選出) u 使得 $2P_{n-1}((n(u))_n) \neq 2P_{n-1}(b, j)$ 。根據**輔助定理 3** 得知, $n(u)$,

t 之間存在 $2P_n - 2P_{n-1}(a, i)$ 的漢米爾頓路徑 Q_2 。所以在 $2P_n - F$ 中對於 s, t 之間存在漢米爾頓路徑 $\langle s, v, Q_1, u, n(u), Q_2, t \rangle$ 。

子分類 1.3: $s, t \in V(2P_{n-1}(b, j)), 2P_{n-1}(b, j) \neq 2P_{n-1}(a, i), a, b \in \{0, 1\}, i, j \in \langle n \rangle$ 。

令 $2P_{n-1}(c, k) \neq 2P_{n-1}(a, i)$ 且 $2P_{n-1}(c, k) \neq 2P_{n-1}(b, j)$, 由於在 $E^{(a,i),(c,k)}$ 中至少有 7 條無錯誤的邊, 所以在 $E^{(a,i),(c,k)}$ 中會存在一條無錯誤的邊 (u, v) 使得 $u \in V(2P_{n-1}(a, i)), v \in V(2P_{n-1}(c, k))$, 而在 $E^{(c,k),(b,j)}$ 中亦存在一條無錯誤的邊 (x, y) 使得 $x \in V(2P_{n-1}(c, k)), y \in V(2P_{n-1}(b, j))$, 並且 $x \neq v$ 。根據假設在 $2P_{n-1}(a, i) - F$ 中存在漢米爾頓迴路 C_1 , 並且在 $2P_{n-1}(b, j)$ 中對於 s, t 之間存在漢米爾頓路徑 Q_1 , 而在 $2P_{n-1}(c, k)$ 中對於 x, v 之間存在漢米爾頓路徑 Q_2 。 $Q_1 = \langle s, Q_3, y, z_1, Q_4, t \rangle, C_1 = \langle u, z_2, Q_5, z_3, u \rangle$, 由於 $d(z_2, z_3) \leq 2$, 根據**輔助定理 2** 得知 $2P_{n-1}((n(z_2))_n) \neq 2P_{n-1}((n(z_3))_n)$ 。所以我們可選擇 z_2 或 z_3 , 假設選出 z_2 使得 $2P_{n-1}((n(z_2))_n) \neq 2P_{n-1}((n(z_1))_n)$ 。根據**輔助定理 3** 得知, $n(z_1)$ 與 $n(z_2)$ 之間存在 $2P_n - 2P_{n-1}(a, i) - 2P_{n-1}(b, j) - 2P_{n-1}(c, k)$ 的漢米爾頓路徑 Q_7 。所以在 $2P_n - F$ 中對於 s, t 之間存在其漢米爾頓路徑 $\langle s, Q_3, y, x, Q_2, v, u, z_3, Q_5, z_2, n(z_2), Q_7, n(z_1), z_1, Q_4, t \rangle$ 。

子分類 1.4: $s \in V(2P_{n-1}(b, j)), t \in V(2P_{n-1}(c, k)), 2P_{n-1}(a, i) \neq 2P_{n-1}(b, j), 2P_{n-1}(a, i) \neq 2P_{n-1}(c, k), 2P_{n-1}(b, j) \neq 2P_{n-1}(c, k), a, b, c \in \{0, 1\}, i, j, k \in \langle n \rangle$ 。

由於在 $E^{(b,j),(a,i)}$ 中至少有 7 條無錯誤的邊, 所以 $E^{(b,j),(a,i)}$ 會存在一條無錯誤的邊 (u, v) 使得 $u \in V(2P_{n-1}(b, j)), v \in V(2P_{n-1}(a, i)), u \neq s$ 。根據假設在 $2P_{n-1}(a, i) - F$ 中存在漢米爾頓迴路 C_1 , 並且在 $2P_{n-1}(b, j)$ 中對於 s, u 之間存在漢米爾頓路徑 Q_1 。我們把 C_1 寫成 $\langle v, x, Q_2, y, v \rangle$ 。由於 $d(x, y) \leq 2$, 根據**輔助定理 2** 得知 $2P_{n-1}((n(x))_n) \neq 2P_{n-1}((n(y))_n)$ 。所以我們可選擇 x 或 y , 假設選出 x 使得 $2P_{n-1}((n(x))_n) \neq 2P_{n-1}(c, k)$ 。根據**輔助定理 3** 得知, $n(x), t$ 之間存在 $2P_n - 2P_{n-1}(a, i) - 2P_{n-1}(b, j)$ 的漢米爾頓路徑 Q_3 。所以在 $2P_n - F$ 中對於 s, t 之間存在其漢米爾頓路徑 $\langle s, Q_1, u, v, y, Q_2, x, n(x), Q_3, t \rangle$ 。

分類 2: $|F \cap 2P_{n-1}(a, i)| \leq n - 4$ 。也就是每個 $2P_{n-1}$ 子圖都為漢米爾頓連通圖。

子分類 2.1: $2P_{n-1}((s)_n) = 2P_{n-1}((t)_n)$ 。

根據假設，在 $2P_{n-1}((s)_n) - F$ 中對於 s, t 存在漢米爾頓路徑 Q_1 。令 (x, y) 為 Q_1 上的一條邊，且滿足 $(x, n(x))$ 和 $(y, n(y))$ 亦為無錯誤的邊。我們把 Q_1 寫成 $\langle s, Q_2, x, y, Q_3, t \rangle$ 。由於 $d(x, y) = 1$ ，根據**輔助定理 2**得知 $2P_{n-1}((n(x))_n) \neq 2P_{n-1}((n(y))_n)$ 。根據**輔助定理 3**得知， $n(x)$ 與 $n(y)$ 之間存在 $2P_n - 2P_{n-1}((s)_n)$ 的漢米爾頓路徑 Q_4 。所以 s, t 之間存在漢米爾頓路徑 $\langle s, Q_2, x, n(x), Q_4, n(y), y, Q_3, t \rangle$ 。

子分類 2.2: $2P_{n-1}((s)_n) \neq 2P_{n-1}((t)_n)$ 。

由**輔助定理 3**得知， $2P_n - F$ 中 s, t 之間存在漢米爾頓路徑。

由**輔助定理 5, 6, 7**可得到下面的**定理一**。

定理一: 當 $n \geq 3$ ， $2P_n$ 是 $(n-2)$ -漢米爾頓圖形和 $(n-3)$ -漢米爾頓連通圖。

5. 結論

在這篇論文裡，我們研究 n -dimensional 2-sided pancake graphs ($2P_n$) 漢米爾頓容錯性質，證明出 $2P_n$ 是 $(n-2)$ -漢米爾頓圖形與 $(n-3)$ -漢米爾頓連通圖的特性。未來的目標是討論 n -dimensional m -sided pancake graphs (mP_n) 的漢米爾頓容錯性質，相信很快就會有成果了。

參考文獻

- [1]. R.Balakrishnan, K.Ranganathan, "A TEXTBOOK OF GRAPH THEORY", ISBN 0-387-98859-9 Springer-Verlag New York Berlin Heidelberg SPIN 10727816, 1991
- [2]. S. Y. Hsieh, G. H. Chen, and C. W. Ho, "Fault-Free Hamiltonian Cycles in Faulty Arrangement Graphs", *IEEE Transactions on Parallel and Distributed Systems*, vol. 10, no. 3, pp. 223-237, 1999.
- [3]. Chun-Nan Hung, Hong-Chun Hsu, Kao-Yung Liang, Lih-Hsing Hsu, "Ring embedding in faulty pancake graphs", *Information Processing Letters* 86 (2003) 271-275.
- [4]. Marissa P. Justan, Felix P. Muga II, Ivan Hal Sudborough, "On the Generalization of the Pancake Network", *Proceedings of*

(03,01,02), (11,13,02), (01,13,02), (03,11,02), (13,11,02), (01,03,02), (11,03,02), (12,13,01), (02,13,01),
 (11,03,12), (13,01,12), (03,01,12), (11,13,12), (01,13,12), (03,11,12), (13,11,12), (01,03,12), (02,13,11),
 (12,13,11), (03,02,11), (13,02,11), (12,03,11), (02,03,11), (13,12,11), (03,12,11), (01,02,13), (12,11,13),
 (02,11,13), (01,12,13), (11,12,13), (03,02,01), (13,02,01), (12,03,01), (02,03,01), (13,12,01), (03,12,01),
 (11,02,13), (12,01,13), (02,01,13).

(01,02,03), (11,02,03), (12,01,03), (02,01,03), (11,12,03), (01,12,03), (02,11,03), (12,11,03), (13,01,02),
 (11,03,02), (01,03,02), (13,11,02), (03,11,02), (01,13,02), (12,03,11), (13,02,11), (03,02,11), (12,13,11),
 (02,13,11), (01,03,12), (13,11,12), (03,11,12), (01,13,12), (02,03,11), (13,12,11), (03,12,11), (01,02,13),
 (11,02,13), (03,12,01), (13,12,01), (02,03,01), (11,13,12), (03,01,12), (13,01,12), (11,03,12), (02,13,01),
 (12,13,01), (03,02,01), (13,02,01), (12,03,01), (11,13,02), (03,01,02), (12,11,13), (02,11,13), (01,12,13),
 (11,12,13), (02,01,13), (12,01,13).

On the Connectivity of a Graph and its Complement Graph

Jia-De Lin¹, Yue-Li Wang¹, Jou-Ming Chang² and Hung-Chang Chan³

¹Department of Information Management, National Taiwan University
of Science and Technology, Taipei, Taiwan, R.O.C.

²Department of Information Management, National Taipei College
of Business, Taipei, Taiwan, R.O.C

³Department of Computer Science and Information Engineering,
National Taiwan University of Science and Technology, Taipei, Taiwan, R.O.C.

Abstract

In this paper, we study the connectivity of the complement of a graph. We first establish a simple formula to compute the connectivity of a graph that comes from the join of graphs. Based on this formula, we give an upper and a lower bound to the connectivity of the complement of a graph.

Keyword: connectivity, complement, join, vertex of attachment.

1 Introduction

All graphs considered in this paper are finite and simple (i.e., without loops and multiple edges). We denote the vertex set and edge set of a graph G by $V(G)$ and $E(G)$, respectively. The *order* of G is defined to be $n(G) = |V(G)|$. The *complement* of G is the graph denoted by $\overline{G}$ such that $V(\overline{G}) = V(G)$ and $E(\overline{G}) = \{uv : uv \notin E(G)\}$. For a subset $S \subset V(G)$, the subgraph of G induced by S is denoted by $G[S]$. Also, we use $G - S$ to denote the graph $G[V \setminus S]$ where $V \setminus S$ is the set $\{v \in V \mid v \notin S\}$. Similarly, for $F \subset E(G)$ we denote the graph with vertex set $V(G)$ and edge set $E(G) \setminus F$ by $G - F$. If S or F is a singleton x , we use the notation $G - x$ instead of $G - \{x\}$. If H is a subgraph of G , then for simplicity, we sometimes write H to mean $V(H)$.

A graph G is *connected* if there is at least one path between any two vertices of G , otherwise G is *disconnected*. The *vertex-connectivity* (or *connectivity* for simplicity) of a graph G , denoted by

$\kappa(G)$, is the minimum number of vertices whose removal from G results in a disconnected or trivial graph, i.e., a graph has only one vertex. Thus, $\kappa(K_n) = n - 1$, where K_n is a complete graph with n vertices. A graph G is *k-connected* if $\kappa(G) = k$, or equivalently, it has at least $k + 1$ vertices and for every pair of nonadjacent vertices u and v there are at least k internally-vertex-disjoint paths joining u and v in G .

In [1], Ando defined that a graph G is *critical connected* if $\kappa(G - v) = \kappa(G) - 1$ for each vertex $v \in V(G)$ and *(k, k')-critical connected* if $\kappa(G - v) = k - 1$ or $\kappa(\overline{G} - v) = k' - 1$ for each vertex $v \in V(G)$, where $k = \kappa(G)$ and $k' = \kappa(\overline{G})$. As a result, Ando obtained an upper bound of the number of vertices for *(k, k')-critical connected* graphs using the terms k and k' . Ando and Kaneko [2] defined that a graph G is *bi-3-connected* if both G and $\overline{G}$ are 3-connected. As a consequence, they proved that every bi-3-connected graph of order at least 22 has a two-vertex set called *bi-contractible pair* whose contraction results still in a bi-3-connected graph. Recently, Ito and Yokoyama [4] showed that $\kappa(\overline{G})$ can be computed in linear time if $\kappa(\overline{G}) \leq 3$. In this paper, we shall propose an upper bound and a lower bound of $\kappa(\overline{G})$ related to $\kappa(G)$.

The rest of this paper is organized as follows. In Section 2, we give some definitions and notation. Furthermore, some prior results are introduced. In Section 3, we discuss the connectivity of a graph which is obtained from the join of graphs. In Section 4, we give an upper and a lower bound to the connectivity of the complement of a graph. Finally, some concluding remarks are given in Section 5.

2 Preliminaries

A subgraph H of a graph G is called a *component* if H is a maximal connected subgraph of G . A *cutset* of G is a subset of vertices whose removal either increases the number of components or increases the number of isolated vertices in G . A cutset with the minimum size is called a *minimum cutset*. Using this modification of the standard definition permits one to compute $\kappa(G)$ by finding the size of a minimum cutset for any connected graph G . Specifically, a cutset with only one vertex is called a *cut vertex*.

Let G_1 and G_2 be two vertex-disjoint graphs. Then the *join*, $G_1 \vee G_2$, of G_1 and G_2 is the graph with vertex set $V(G_1) \cup V(G_2)$ and edge set $E(G_1) \cup E(G_2) \cup \{uv : u \in V(G_1) \text{ and } v \in V(G_2)\}$. Obviously, it follows from the definition that $G_1 \vee G_2$ is connected. For convenience, we write $\bigvee_{i=1}^k G_i = G_1 \vee G_2 \vee \dots \vee G_k$. We use an example to depict the concept of the join of two graphs. Figure 1(a) shows that two separated graphs G_1 and G_2 , and Figure 1(b) shows the graph $G_1 \vee G_2$. Note that both G_1 and G_2 are subgraphs of $G_1 \vee G_2$ induced by $V(G_1)$ and $V(G_2)$, respectively.

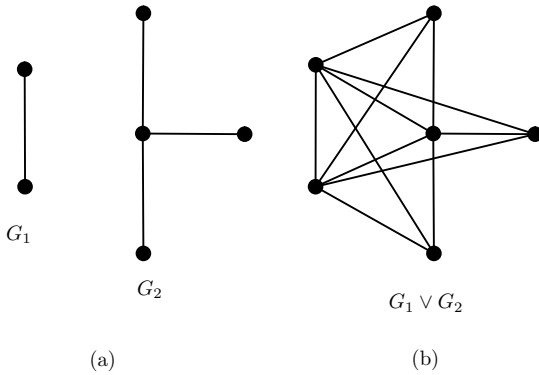


Figure 1: Two graphs and their join.

For a subgraph H of G , the *attachment* of H in G , denoted by $A_G(H)$, is the maximal subset of $V(H)$ such that every vertex in the set is adjacent to at least one vertex of $G - H$. Thus, $A_G(G) = A_G(\emptyset) = 0$. For two nonempty subsets $S_1, S_2 \subset V(G)$, the set containing all edges of G with one end vertex in S_1 and the other end vertex in S_2 is denoted by $E_G[S_1, S_2]$. When no ambiguity arises, we drop the index G in the above notations.

Boland et al. in [3] studied the *inclusive connectivity* (*i-connectivity* for short) of graphs, which is

a variant pertaining to the connectivity of graphs. Let G be a graph and $v \in V(G)$. The *vertex i-connectivity* of v , denoted by $\kappa_i(v, G)$, is the cardinality of a minimum vertex set $T \subset V(G)$ whose removal results in a subgraph containing v as a cut vertex. In particular, $\kappa_i(v, G) = 0$ if v is a cut vertex of G , and $\kappa_i(v, G) = n(G) - 1$ if T does not exist. As a result, Boland et al. provided a way to find the vertex *i-connectivity* of the join of two graphs.

Theorem 1 [3] *Let G and H be two arbitrary connected graphs and $v \in V(G)$. Then*

$$\kappa_i(v, G \vee H) = \min\{\kappa_i(v, G) + n(H), \kappa(H) + n(G) - 1\}.$$

For example, Figure 2 shows a graph $C_4 \vee K_2$, where C_4 contains a vertex v . From Theorem 1, we are easy to verify that $\kappa_i(v, C_4) = 1$, $n(K_2) = 2$, $\kappa(K_2) = 1$, and $n(C_4) = 4$. Thus $\kappa_i(v, C_4 \vee K_2) = 3$. In this figure, the part of shadowed vertices is the minimum vertex set T such that v becomes a cut vertex if T is removed.

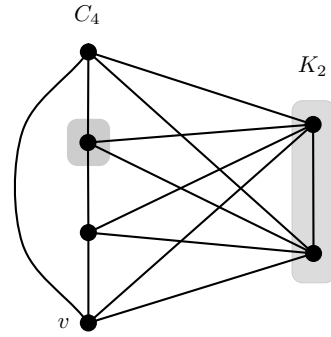


Figure 2: $\kappa_i(v, C_4 \vee K_2) = 3$ where $v \in V(G)$.

3 The connectivity of the join of graphs

In this section, we discuss the connectivity of the join of graphs. The following lemma is similar to Theorem 1 that gives a formula to compute the connectivity of the join of two graphs.

Lemma 2 *Let G_1 and G_2 be two arbitrary graphs that each has a minimum cut set C_i for $i = 1, 2$, respectively. Then*

$$\kappa(G_1 \vee G_2) = \min\{n(G_1) + \kappa(G_2), n(G_2) + \kappa(G_1)\}.$$

In particular, if $\kappa(G_1 \vee G_2) = n(G_i) + \kappa(G_j)$ where $i, j \in \{1, 2\}$ and $i \neq j$, then $V(G_i) \cup C_j$ is a minimum cut set of $G_1 \vee G_2$.

Proof. If both G_1 and G_2 are complete graphs, then so is $G_1 \vee G_2$. In this case, it is clear that $\kappa(G_1 \vee G_2) = n(G_1) + n(G_2) - 1$ and the lemma holds. We now suppose that at least one of G_1 and G_2 is not a complete graph and let C be a minimum cut set in $G_1 \vee G_2$. Then $\kappa(G_1 \vee G_2) = |C|$. Since in $G_1 \vee G_2$ every vertex of G_1 is adjacent to every vertex of G_2 , if two vertices u and v are separated in $(G_1 \vee G_2) - C$, then both u and v must come from G_1 or G_2 . Thus, either $V((G_1 \vee G_2) - C) \subset V(G_1)$ or $V((G_1 \vee G_2) - C) \subset V(G_2)$, which is dependent on the size of C .

Without loss of generality, we assume $V((G_1 \vee G_2) - C) \subset V(G_1)$. Clearly, $V(G_2) \subset C$. Since $G_1 \vee G_2$ is not a complete graph, $(G_1 \vee G_2) - C$ has at least two components that each contains only the vertices from G_1 . This implies that C must contain a cut set of G_1 . Thus, $|C| \geq n(G_2) + \kappa(G_1)$. Since C is a minimum cut set in $G_1 \vee G_2$, it must consist of exactly $V(G_2)$ and a minimum cut set of G_1 . In particular, we can choose $C = V(G_2) \cup C_1$. $\square$

We now use an example to illustrate Lemma 2. Two graphs G_1 and G_2 are shown in Figure 2(a), and $G_1 \vee G_2$ is shown in Figure 2(b). Since $n(G_1) = 3$, $\kappa(G_1) = 1$, $n(G_2) = 5$, and $\kappa(G_2) = 2$, by Lemma 2, $\kappa(G_1 \vee G_2) = \min\{5 + 3, 3 + 2\} = 5$. The shadowed part in each graph indicates the minimum cutset of G_1 , G_2 , and $G_1 \vee G_2$, respectively.

According to Lemma 2, the following extension can easily be proved by induction.

Theorem 3 Let $G_1, G_2, \dots, G_n$ be distinct graphs that each has a minimum cut set C_i for $i = 1, 2, \dots, n$. Then

$$\kappa\left(\bigvee_{i=1}^n G_i\right) = \min_{1 \leq j \leq n} \left\{ \kappa(G_j) + \sum_{\substack{1 \leq i \leq n \\ i \neq j}} n(G_i) \right\}$$

In particular, if $\kappa(\bigvee_{i=1}^n G_i) = \kappa(G_j) + \sum_{\substack{1 \leq i \leq n \\ i \neq j}} n(G_i)$ for some $j \in \{1, \dots, n\}$, then $(\bigcup_{\substack{1 \leq i \leq n \\ i \neq j}} V(G_i)) \cup C_j$ is a minimum cut set of $\bigvee_{i=1}^n G_i$.

Proof. We prove this theorem by induction on n . For $n = 2$, the result is shown in Lemma 2. Assume that this theorem holds for any k graphs $G_1, G_2, \dots, G_k$, where $k \geq 2$. We now consider

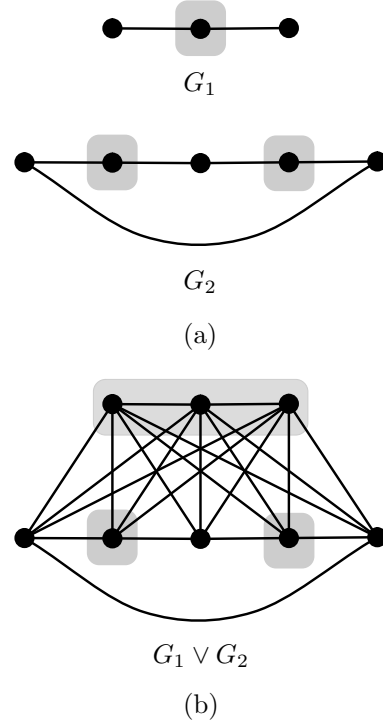


Figure 3: The minimum cutsets of G_1 , G_2 and $G_1 \vee G_2$.

$k + 1$ graphs $G_1, G_2, \dots, G_{k+1}$ as follows.

$$\begin{aligned} & \kappa\left(\bigvee_{i=1}^{k+1} G_i\right) \\ &= \kappa\left(\left(\bigvee_{i=1}^k G_i\right) \vee G_{k+1}\right) \\ &= \min\left\{n\left(\bigvee_{i=1}^k G_i\right) + \kappa(G_{k+1}), n(G_{k+1}) + \kappa\left(\bigvee_{i=1}^k G_i\right)\right\} \\ &= \min\left\{\sum_{i=1}^k n(G_i) + \kappa(G_{k+1}), n(G_{k+1}) + \min_{1 \leq j \leq k} \left\{ \kappa(G_j) + \sum_{\substack{1 \leq i \leq k \\ i \neq j}} n(G_i) \right\}\right\} \\ &= \min\left\{\sum_{i=1}^{k+1} n(G_i) + \kappa(G_{k+1}) - n(G_{k+1}), \sum_{i=1}^{k+1} n(G_i) + \min_{1 \leq j \leq k} \left\{ \kappa(G_j) - n(G_j) \right\}\right\} \\ &= \sum_{i=1}^{k+1} n(G_i) + \min\left\{\kappa(G_{k+1}) - n(G_{k+1}), \min_{1 \leq j \leq k} \left\{ \kappa(G_j) - n(G_j) \right\}\right\} \end{aligned}$$

$$\begin{aligned}
&= \sum_{i=1}^{k+1} n(G_i) + \min_{1 \leq j \leq k+1} \{\kappa(G_j) - n(G_j)\} \\
&= \min_{1 \leq j \leq k+1} \{\kappa(G_j) + \sum_{\substack{1 \leq i \leq k+1 \\ i \neq j}} n(G_i)\}
\end{aligned}$$

This shows that the connectivity of $\bigvee_{i=1}^n G_i$ is $\kappa(G_j) + \sum_{\substack{1 \leq i \leq n \\ i \neq j}} n(G_i)$ for some $j \in \{1, \dots, n\}$.

In this case, to find out a minimum cutset C of $\bigvee_{i=1}^{k+1} G_n$, we can choose $C = (\bigcup_{\substack{1 \leq i \leq n \\ i \neq j}} V(G_i)) \cup C_j$.

This completes the proof. $\square$

Corollary 4 *If G is a disconnected graph with k components, then $\kappa(\overline{G}) \geq k - 1$.*

Proof. Let $H_1, H_2, \dots, H_k$ be the k components of G . Clearly, $\overline{G} = \bigvee_{i=1}^k \overline{H_i}$ is connected. By Theorem 3 and the fact that $\kappa(\overline{H_i}) \geq 0$ and $n(\overline{H_i}) \geq 1$ for every $i = 1, \dots, k$, we have $\kappa(\overline{G}) = \kappa(\bigvee_{i=1}^k \overline{H_i}) = \min_{1 \leq j \leq k} \{\kappa(\overline{H_j}) + \sum_{\substack{1 \leq i \leq k \\ i \neq j}} n(\overline{H_i})\} \geq k - 1$. $\square$

4 The connectivity of the complement of a Graph

Throughout the rest, we consider that G is a nontrivial, noncomplete, connected graph with a cutset C , and suppose that there are $k \geq 2$ components in $G - C$, say H_i for $i = 1, \dots, k$. Clearly, $\overline{G - C} = \overline{H_1} \vee \overline{H_2} \vee \dots \vee \overline{H_k}$ and the attachment $A_G(H_i)$, $i = 1, \dots, k$, are nonempty sets. For simplicity, we write A_i to mean $A_G(H_i)$ and let $D_i = V(H_i - A_i)$.

With respect to the cutset C of G , we first define a special graph $\overline{G_C}$. Then, we will show that $\kappa(\overline{G_C})$ is a lower bound of the connectivity of $\overline{G}$ and it can be computed by a simple way. Formally, we define as follows:

$$\overline{G_C} = \overline{G} - E_{\overline{G}}[C, \bigcup_{i=1}^k A_i] - (\bigcup_{i=1}^k E_{\overline{G}}[A_i, D_i]) \quad (1)$$

Note that $\overline{G_C}$ is a spanning subgraph of $\overline{G}$. For example, a visual illustration of a graph G with a cutset C is shown in Figure 4(a), where $G - C$ contains three components H_1, H_2, H_3 and the edge sets $E[C, \bigcup_{i=1}^3 A_i]$ and $E[A_i, D_i]$ for $1 \leq i \leq 3$ are drawn by dotted lines. According to the definition (1), another visual illustration of the graph $\overline{G_C}$ is shown in Figure 4(b), where the edge sets $E_{\overline{G_C}}[\overline{H_i}, \overline{H_j}] = \{xy : x \in V(H_i), y \in V(H_j)\}$ for

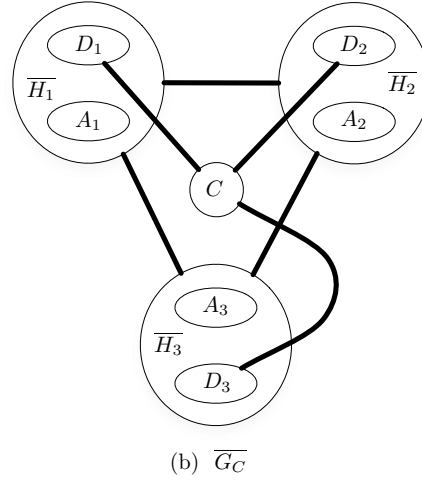
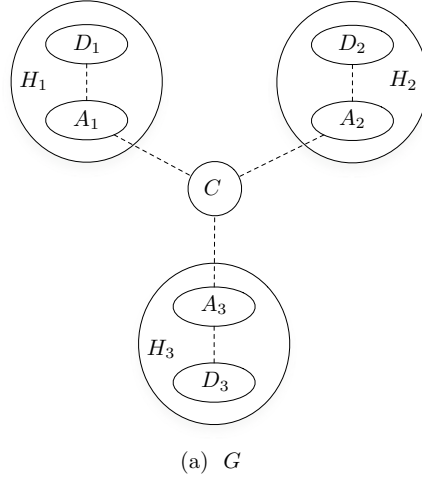


Figure 4: (a) A visualization of a graph G with a cutset C and the three components of $G - C$; (b) The graph $\overline{G_C}$.

$i \neq j$ and $E_{\overline{G_C}}[C, D_i] = \{cd : c \in C, d \in D_i\}$ for $1 \leq i \leq 3$ are drawn by solid lines.

Lemmas 5 and 6 are used to achieve $\kappa(\overline{G_C})$.

Lemma 5 *Let G be a graph and H, K be two disjoint connected induced subgraphs of G . Suppose that K has a minimum cutset C_K . Then, the following statements are true:*

- (1) *If $\kappa(G[K \cup H]) > \kappa(K)$, then $E[S, H] \neq \emptyset$, where S is any component of $K - C_K$.*
- (2) *If $\kappa(G[K \cup H]) \leq \kappa(K)$, then a minimum cutset of $G[K \cup H]$ is either C_K or a subset of $A_{G[K \cup H]}(K) \cup V(H)$.*

Proof. To prove statement (1), we let S be any component of $K - C_K$. Since $\kappa(G[K \cup H]) > \kappa(K)$, $G[K \cup H] - C_K$ is connected and it contains S as an induced subgraph. In this case, since there exist no edges joining vertices between any two components of $K - C_K$, S must be directly connected to H . Thus $E[S, H] \neq \emptyset$.

We prove statement (2) by contradiction. Suppose to the contrary that $\kappa(G[K \cup H]) \leq \kappa(K)$ and there is a minimum cutset C of $G[K \cup H]$ which is not C_K or a subset of $A_{G[K \cup H]}(K) \cup V(H)$. Let

$$X = C - (A_{G[K \cup H]}(K) \cup V(H))$$

and

$$Y = C - X.$$

Clearly, $X \neq \emptyset$ and Y is not a cutset of $G[K \cup H]$. Thus $G[K \cup H] - Y$ is connected. Let

$$Z = A_{G[K \cup H]}(K) - Y.$$

Since $A_{G[K \cup H]}(K)$ is a cutset of $G[K \cup H]$, Y cannot contain all the vertices of $A_{G[K \cup H]}(K)$. This implies that Z is nonempty. By the fact $C = X \cup Y$, X is a cutset of $G[K \cup H] - Y$. Moreover, we can find that the vertices of $(H - Y) \cup Z$ must be contained in one of the components of $G[K \cup H] - Y - X$. Therefore, X is also a cutset of $K - Y$. Now, if $K[C]$ is a cutset of K and $K[C] \neq C_K$, then $n(K[C]) > \kappa(K)$. However, by assumption $\kappa(K) \geq \kappa(G[K \cup H]) = n(C) \geq n(K[C])$. It is a contradiction. $\square$

Lemma 6 For any graph G with a cutset C ,

$$\kappa(\overline{G_C}) = \min\left\{\sum_{i=1}^k n(D_i), \min_{1 \leq i \leq k} \{n(G - C - H_i)\}\right\}$$

Proof. Let $H = \overline{G - C} = \bigvee_{i=1}^k \overline{H_i}$ and C_H be a minimum cutset of H . By Theorem 3, all components of $H - C_H$ must be contained in $\overline{H_j}$ for a certain $j \in \{1, \dots, k\}$. In particular, there is a component of $H - C_H$, say S , that contains only the vertices of A_j since by definition $\kappa(\overline{H_j}) = 0$ and $E_{\overline{G_C}}[A_j, D_j] = \emptyset$.

Obviously, $\overline{G_C} = \overline{G_C}[V(H) \cup C]$. We now claim that $\kappa(\overline{G_C}) \leq \kappa(H)$. Suppose not, by Lemma 5 we know that every component of $H - C_H$ is directly connected to C . Thus $E_{\overline{G_C}}[S, C] \neq \emptyset$. However, this is impossible since S contains only the vertices of A_j and by definition there exist no edges between A_j and C in $\overline{G_C}$. Since $\kappa(\overline{G_C}) \leq \kappa(H)$,

again by Lemma 5, a minimum cutset of $\overline{G_C}$ is either C_H or a subset of $A_{\overline{G_C}}(H) \cup C$. Thus we consider the following two cases.

Case 1: C_H is a minimum cutset of $\overline{G_C}$. In this case $\kappa(\overline{G_C}) = \kappa(H) = |C_H|$. By Theorem 3, we can compute the connectivity of $\overline{G_C}$ as follows.

$$\begin{aligned} \kappa(H) &= \kappa(\bigvee_{i=1}^k \overline{H_i}) \\ &= \min_{1 \leq j \leq k} \{\kappa(\overline{H_j}) + \sum_{\substack{1 \leq i \leq k \\ i \neq j}} n(\overline{H_i})\} \\ &= \min_{1 \leq i \leq k} \{n(G - C - H_i)\} \end{aligned} \quad (2)$$

The last equality holds due to the fact that $\kappa(\overline{H_j}) = 0$ and $n(H) - n(\overline{H_i}) = n(G - C - H_i)$.

Case 2: There is a subset of $A_{\overline{G_C}}(H) \cup C$, say T , which forms a minimum cutset of $\overline{G_C}$. Since each vertex of C is adjacent to each vertex of D_i in $\overline{G_C}$ for all $i = 1, \dots, k$, either $C \subseteq T$ or $(\bigcup_{i=1}^k D_i) \subseteq T$. For the former case, since $T - C$ is a cutset of H , it follows from Case 1 that $|T| \geq |C| + \kappa(H) = |C \cup C_H| > \kappa(\overline{G_C})$, a contradiction. Contrastively, since $E_{\overline{G_C}}[\bigcup_{i=1}^k A_i, C] = \emptyset$ and T is a minimum cutset of $\overline{G_C}$, the latter case implies that $T = \bigcup_{i=1}^k D_i$ and $\kappa(\overline{G_C}) = |T| = \sum_{i=1}^k n(D_i)$. $\square$

As a consequence, we have the following result.

Theorem 7 Let G be an arbitrary graph with a minimum cutset C and $\overline{G_C}$ be defined as in (1). Then

$$\kappa(\overline{G_C}) \leq \kappa(\overline{G}) \leq \kappa(\overline{G - C}) + \kappa(G)$$

where,

$$\kappa(\overline{G - C}) = \min_{1 \leq i \leq k} \{n(G - C - H_i)\}.$$

Proof. The lower bound of $\kappa(\overline{G})$ follows from Lemma 6 and the fact that $\overline{G_C}$ is a spanning subgraph of $\overline{G}$. Similarly, since $\overline{G}$ is a spanning subgraph of $\overline{G - C} \vee \overline{C}$, $\kappa(\overline{G}) \leq \kappa(\overline{G - C} \vee \overline{C})$. By Theorem 3,

$$\begin{aligned} \kappa(\overline{G}) &\leq \min\{\kappa(\overline{G - C}) + n(\overline{C}), \kappa(\overline{C}) + n(\overline{G - C})\} \\ &\leq \kappa(\overline{G - C}) + n(C) \\ &= \kappa(\overline{G - C}) + \kappa(G). \end{aligned}$$

Moreover, $\kappa(\overline{G - C})$ can be computed as in (2). $\square$

5 Concluding remarks

In this paper, we derive an upper bound and a lower bound of the connectivity of the complement of a graph G , i.e., $\kappa(\overline{G-C}) \leq \kappa(\overline{G}) \leq \kappa(G-C) + \kappa(G)$, where $\kappa(\overline{G-C})$ and $\kappa(G-C)$ can be computed by a simple way. We are now trying to find the class of graphs so that the equality holds in the above inequality.

References

- [1] K. Ando, An upper bound for orders of certain $(k, \bar{k})$ -connected graphs, *Discrete Mathematics* 135 (1994) 371–375.
- [2] K. Ando and A. Kaneko, A remark on the connectivity of the complement of a 3-connected graph, *Discrete Mathematics* 151 (1996) 39–47.
- [3] J.W. Boland, L.M. Lawson and R.D. Ringeisen, i -connectivity of the join of two Graphs, *Congressus Numerantium* 109 (1995) 81–87.
- [4] H. Ito and M. Yokoyama, Linear time algorithms for graph search and connectivity determination on complement graphs, *Information Processing Letters* 66 (1998) 209–213.

24皇后問題共有227,514,172,009,468個解

謝育平^{a*}, 項潔^{a†}, 黃光璿^b, 許德標^c

^a國立台灣大學 資訊工程學系

^b國立暨南大學 資訊工程學系

^cDepartment of Computer and Information Science, Fordham University.

arping@turing.csie.ntu.edu.tw

hsiang@csie.ntu.edu.tw

shieng@ncnu.edu.tw

hsu@trill.cis.fordham.edu

Abstract

n -Queen 問題已經被討論至少 150 年, 產出至少 88 篇相關的論文, 衍生出很多的變形問題並激發出各式不同的演算法。其中最受人矚目的還是 n -Queen 原始問題的存在性問題及計數性問題。本文嘗試有別於以往的演算法, 使用 Divide and Conquer 的概念來處理計數性問題, 並計算出 24-Queen 的解個數。

1 Introduction

n -Queen 問題[1, 3, 13] 已經被討論至少 150 年, 產出至少 88 篇相關的論文[11], 衍生出很多的變形問題[5, 7], [20](Queen) (在[20] 的網站中, 選取 “Word” 並查詢 “Queen”, 會出現 52 個有關 n -Queen 問題的數字數列), 其中最受人矚目的還是 n -Queen 問題本身的解存在性問題[1, 2] 及解計數性問題[17, 5, 7], [20](A000170) (在[20] 的網站中選取 “Sequence number”, 查詢 “A000170”, 即可得到 n 從 1 到 23 的 n -Queen 問題解的個數)。另外兩個跟 n -Queen 問題有極大關係的是 Toroidal(Modular) n -Queen Problem [20](A007705) 及 Toroidal-Semi n -Queen Problem[20](A006717)。

我們最早開始這方面的興趣是起於研究 complete mapping [12, 6, 15, 8]。而 Complete Mapping 跟 n -Queen Problem 正有著密不可分的關係。一個 cyclic group $(Z_n, +)$ 的 Complete Mapping 就是一個 Toroidal-Semi n -Queen Problem 的解, 但是 Complete Mapping 多了一個條件就是第一個 Queen 必須放在第一個位置($f(0) = 0$), 所以 Toroidal-Semi n -Queen Problem 的解個數剛好是 Complete Mapping 個數的 n 倍。而一個 Z_n group 的 Strong Com-

plete Mapping [9] 就是一個 Toroidal n -Queen Problem 的解, 但是多了一個條件就是第一個 Queen 必須放在第一個位置, 所以 Toroidal n -Queen Problem 的解個數也剛好是 Strong Complete Mapping 個數的 n 倍。我們將計算 Complete Mapping 個數的技巧[19, 7] 應用在計算 24-Queen 問題上, 並加上一些 n -Queen 問題特有的技術, 嘗試使用 Divide and Conquer 的概念來處理, 並解決 24-Queen 的計數性問題。

2 n -Queen問題

n -queen 計數問題: 給定一個 n 乘 n 的棋盤, 有多少種方法可以將 n 個皇后放進棋盤中, 使得每一行、每一列、每一個 (左上右下) 斜線和 (右上左下) 斜線中, 至多一個皇后。我們將一個解答 (擺設的方法) 用一個對射函數 (1-1, onto) $f: Z_n \rightarrow Z_n$ 紀錄之。下圖表示了 8-Queen 問題的一個解。

f	0	1	2	3	4	5	6	7	f	$f - id$	$f + id$
0		Q							1	1	1
1	\							Q	7	8	6
2		\				Q		/	5	7	3
3	Q		\				/		0	3	-3
4			Q	\		/			2	6	-2
5	—	—	—	—	Q	—	—	—	4	9	-1
6				/		\	Q		6	12	0
7			/	Q			\		3	10	-4

Figure 1: 8-Queen 問題的一個解。

所以我們用更數學的方式來描述 n -Queen 問題: 一個 n -Queen 問題的解是一個從 Z_n 對到 Z_n 的函數

*感謝國科會計畫 NSC 92-2811-E-002-027 的支持

†感謝國科會計畫 NSC 92-2213-E-002-005 的支持

$f(x)$ 使得

$$\begin{aligned}\forall i \neq j \in Z_n \quad & f(i) \neq f(j), \\ \forall i \neq j \in Z_n \quad & f(i) - i \neq f(j) - j, \\ \forall i \neq j \in Z_n \quad & f(i) + i \neq f(j) + j\end{aligned}$$

(此處的加減是指整數下的加法)。而 n -Queen 的計數問題則是探討有多少個解。我們用 $Q(n)$ 來表示解的個數。我們很容易地可以發現將原來的解隨著棋盤旋轉 90° 後也會是一個 n -Queen 問題的解。我們稱這樣的一個動作 (旋轉 90°) 為一個對稱運算子。定義如下: π 是作用在解答上的一個對稱運算子 (symmetry operator): 如果 f 是一個解則 $\pi(f)$ 也是一個解, 反之亦然。對於 n -Queen 問題, 有八個很自然的對稱運算子由兩個基本運算子所延展[14, 4]。

1. 一個是逆時鐘旋轉 90° :
 $R_{90^\circ}(f)(x) = (n - 1 - f)^{-1}(x)$,
2. 另一個是依主對角線 (左上右下) 翻轉:
 $R(f)(x) = f^{-1}(x)$ 。

根據 Integer Sequence[20] 上記載, 及本文的結果, 我們將 n -Queen 解的個數列表如下。

n	$Q(n)$
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
9	352
10	724
11	2,680
12	14,200
13	73,712
14	365,596
15	2,279,184
16	14,772,512
17	95,815,104
18	666,090,624
19	4,968,057,848
20	39,029,188,884
21	314,666,222,712
22	2,691,008,701,644
23	24,233,937,684,440
24	227,514,172,009,468

在本文之前, n -Queen 計數問題, 最好的結果為 Sylvain Pion 及 Joel-Yann Fourre [16] 所計算的 $Q(23)$ 。

3 計算 $Q(24)$

對於計算 $Q(24)$ 來說, 我們主要採取 Partition Strategies[18] 為主, symmetry cutting strategies[10] 為輔。首先我們觀察以下的定義及定理, 我們依照原本 n -Queen Problem 解的定義找出其若且唯若的九個條件, 並將這九個條件應用在程式寫作上。

定義 1. 若 $f: Z_n \rightarrow Z_n$ 滿足以下三個條件, 則我們稱 f 為 n -Queen 問題的一個解。

1. $\forall i \neq j \in Z_n \quad f(i) \neq f(j)$,
2. $\forall i \neq j \in Z_n \quad f(i) - i \neq f(j) - j$ 和
3. $\forall i \neq j \in Z_n \quad f(i) + i \neq f(j) + j$ 。

此處的加減法是整數的加減法 $(Z, +)$, 並非是同餘群下的加減法 $(Z_n, +)$ 。

定理 1. 假設 U 是 Z_n 的一個子集, 取 $L = Z_n \setminus U$ 為 Z_n 對 U 的差集。則 $f: Z_n \rightarrow Z_n$ 是 n -Queen 問題的一個解若且唯若 f 滿足以下九個條件。

1. $\forall i \neq j \in U \quad f(i) \neq f(j)$
2. $\forall i \neq j \in U \quad f(i) - i \neq f(j) - j$
3. $\forall i \neq j \in U \quad f(i) + i \neq f(j) + j$
4. $\forall i \neq j \in L \quad f(i) \neq f(j)$
5. $\forall i \neq j \in L \quad f(i) - i \neq f(j) - j$
6. $\forall i \neq j \in L \quad f(i) + i \neq f(j) + j$
7. $f(U) \cap f(L) = \emptyset$
8. $(f - id)(U) \cap (f - id)(L) = \emptyset$
9. $(f + id)(U) \cap (f + id)(L) = \emptyset$

其中 $f(S) = \{f(x) \mid x \in S\}$, $(f - id)(S) = \{f(x) - x \mid x \in S\}$, $(f + id)(S) = \{f(x) + x \mid x \in S\}$ 。

3.1 基本架構

所以在計算 $Q(24)$ 時, 我們先將 Z_{24} 拆成兩部分, $U = \{0, 1, 2, \dots, 11\}$, $L = \{12, 13, 14, \dots, 23\}$, 對於一個 24-Queen 解 $f(x)$ 來說, $f(U) = \{f(i) \mid i \in U\}$ 是固定的, 而且 $|f(U)| = 12$, 所以我們依照 $f(U)$ 的不同, 將所有的解分成 C_{12}^{24} 個 partition slots, 作為分割計算的方法。在 $f(U)$ 決定後, 我們使用條件 4 及條件 7, $f(L)$ 也會被唯一決定。所以我們的演算法架構如下:

令 number=0 來記載 Q(24) 的值
 令 $U = \{0, 1, 2, \dots, 11\}$, 而 $L = Z_n \setminus U$
 For each partition slot $A \subset Z_n, |A| = 12$

1. 令 $f(U) = A, f(L) = Z_n \setminus A$.
此時條件 7 自然滿足。
2. 產生所有的部分解 $f : U \rightarrow f(U)$
使得條件 1, 2, 3 都成立並紀錄所有的 U-record:
 $((f - id)(U), (f + id)(U))$ 。
3. 產生所有的部分解 $f : L \rightarrow f(L)$
使得條件 4, 5, 6 都成立並紀錄所有的 L-record:
 $((f - id)(L), (f + id)(L))$ 。
4. 將所有的 U-record 對所有的 L-record
進行比對, 檢查條件 8,9 是否成立,
若成立則將 number 加 1。

End

3.2 使用對稱運算子加速計算

此時我們引用 symmetry cutting strategies, 使用對稱運算子來加速計算。因為我們使用 partition strategies 為主的關係導致並非八個對稱運算子都可以拿來加速計算, 只有上下翻轉 ($R_{90^\circ} \circ R$) 和左右翻轉 ($R_{270^\circ} \circ R$) 所延展的四個對稱運算子可供使用。也就是說在計算上, 將大致有四倍的加速。這裡我們強調一下, 如何運用 symmetry cutting strategies 會因 n 的奇偶性而有所不同, 所以以下的整個演算法架構是針對偶數 n 而設計的。

定義 2. 假設 A, B 為 Z_n 的兩個子集, 而 h 為 Z_n 的一個元素。

1. $h - A = \{h - y \mid y \in A\}$.
2. $A < B$ if there exists an $x \in A \setminus B$ such that $x < y$ for all $y \in B \setminus A$.

引理 1. 令集合 ordering $<$ 的定義如上, 則對於所有 Z_n 的子集 A, B , 則 $A = B, A < B, B < A$ 恰一成立。

定理 2. 假設 $n = 2k, f : Z_n \rightarrow Z_n$ 為 n -Queen 問題的一個解。令 $U = \{0, 1, 2, \dots, k-1\}, L = Z_n \setminus U = \{k, k+1, \dots, 2k-1\}$ 。對於 n -Queen 問題的上下翻轉運算子 $\pi_1 = R_{90^\circ} \circ R$, 和左右翻轉運算子 $\pi_2 = R_{90^\circ} \circ R$, 以下性質成立。令 $A = f(U)$ 則

1. $\pi_1(f)(U) = Z_n \setminus A$.
2. $\pi_2(f)(U) = (n-1) - A$.
3. $\pi_2 \circ \pi_1(f)(U) = \pi_1 \circ \pi_2(f)(U) = (n-1) - (Z_n \setminus A)$.
4. $\min\{A, Z_n \setminus A, (n-1) - A, (n-1) - (Z_n \setminus A)\}$ 存在且唯一。

令 number=0 來記載 Q(24) 的值
 令 $U = \{0, 1, 2, \dots, 11\}$, 而 $L = Z_n \setminus U$
 For each partition slot $A \subset Z_n, |A| = 12$

- a. 令 factor =
 $|\{A, (n-1) - A, Z_n \setminus A, (n-1) - Z_n \setminus A\}|$
- b. 如果 $A \neq$
 $\min\{A, (n-1) - A, Z_n \setminus A, (n-1) - Z_n \setminus A\}$
則繼續下一個 partition slot.
1. 令 $f(U) = A, f(L) = Z_n \setminus A$.
此時條件 7 自然滿足。
2. 產生所有的部分解 $f : U \rightarrow f(U)$
使得條件 1, 2, 3 都成立並紀錄所有的 U-record:
 $((f - id)(U), (f + id)(U))$ 。
3. 產生所有的部分解 $f : L \rightarrow f(L)$
使得條件 4, 5, 6 都成立並紀錄所有的 L-record:
 $((f - id)(L), (f + id)(L))$ 。
4. 將所有的 U-record 對所有的 L-record
進行比對, 檢查條件 8,9 是否成立,
若成立則將 number 加 factor。

End

3.3 使用 bit tree 加速計算

在上述的所有步驟中以步驟 4 為時間的瓶頸, 所以我們將步驟 4 做些改善。在步驟 2 或 3 中, 因為 $f - id$ 的值域為 $[-23, 23]$ 而 $f + id$ 的值域為 $[0, 46]$, 所以紀錄 U-record 或是 L-record 須要 94 bits。

令 $x \in U, y \in L$, 則 $0 \leq x \leq 11, 12 \leq y \leq 23$, 所以 $-11 \leq f(x) - x \leq 23, -23 \leq f(y) - y \leq 11$, 所以要計算 $(f - id)(U), (f - id)(L)$ 是否互斥只需要檢查 $(f - id)(U) \cap [-11, 11]$ 和 $(f - id)(L) \cap [-11, 11]$ 是否互斥即可。故我們使用 23 bits 來記載 $(f - id)$ 的部分。同理 $0 \leq f(x) + x \leq 34, 12 \leq f(y) + y \leq 46$, 所以要計算 $(f + id)(U), (f + id)(L)$ 是否互斥只需要檢查 $(f + id)(U) \cap [12, 34]$ 和 $(f + id)(L) \cap [12, 34]$ 是否互斥即可。故我們也只須使用 23 bits 來記載 $(f + id)$ 的部分。所以我們不需要記載完整的 record 資訊, 我們只需使用 46 bits 來紀錄 U-record 和 L-record。

接下來使用 46 bits 檢查條件 8, 9 是整個演算法的瓶頸, 是最耗時的一段程式區碼。令 US 分別為步驟 2 所產生的所有 U-record 所形成的 multi-set。令 LS 為步驟 2 所產生的所有 L-record 所成的 multi-set。此時若我們將 US 中的每個 record 拿來跟 LS 中的每個 record 做比對來檢查條件 8, 9, 此時所需要的比對為 $|US| \times |LS|$ 次, 所以如果可以在這裡取得突破則可以大大加速計算。

接下來我們將 US 中的 record 依照 bit0 分兩區 US_0, US_1 , 每一區再依照 bit1 再分兩區成 $US_{00}, US_{01}, US_{10}, US_{11}$ 。如此建構成一棵樹來記載 US, 每個 leaf node 都表示一個 record, 而每個 internal node 都表示一群 records。同理將 LS 用一棵樹來表達。此時紀錄 record 的 bit 數及其順序對建樹所使用的記憶體及條件 8, 9 的檢查時間有很大的影響, 這也就是為什麼我們要將 bit 數從 94 降為 46。

然後當我們在比對 record 來檢查條件 8, 9 時, 就變成了一個遞迴的過程。在這個過程中, 我們得到了一些好處。

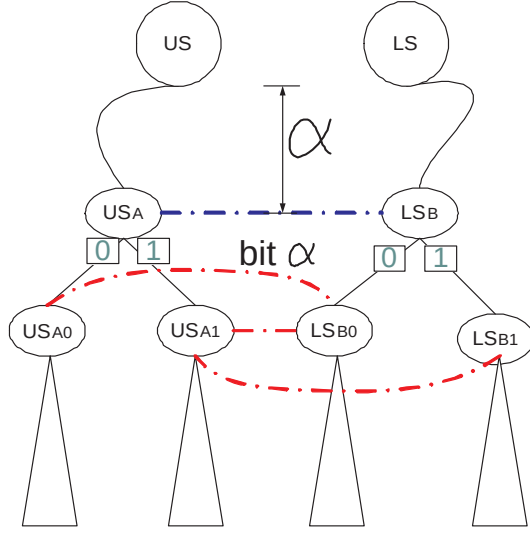


Figure 2: Record比對狀況。

令 $A, B \in \{0, 1\}^*$ 為從根節點到某節點的路徑且 A, B 長度皆為 α , 則 US_A, LS_B 分別代表 US 及 LS 樹的某個節點, 也就是分別代表一群 U-record 及 L-record。此時因為條件 8, 9 要滿足, 蘊含 U-record 和 L-record 在 $\text{bit}\alpha$ 不可皆為 1。所以當我們要比對這兩群 (US_A, US_B) 滿不滿足條件 8, 9 時, 只需比對 $(US_{A0}, LS_{B1}), (US_{A1}, LS_{B0}), (US_{A0}, LS_{B0})$ 而不需比對 (US_{A1}, LS_{B1}) 所以省下了很多寶貴的時間。

我們假設每個 US 或 LS 的 record 記載的 46 bits 資訊中, 每個 bit 被設 1 的機率大致相同, 在此假設下, 我們約略估算每個 bit 被設 1 的機率為 $(12 + 12)/(47 + 47)$ 約為 0.25, 這表示當我們在比對 (US_A, LS_B) 時, US_{A1} 和 US_{A0} 的元素個數比約為 $0.25 : 0.75 = 1 : 3$, 同樣地, LS_{B1} 和 LS_{B0} 的元素個數比約為 $1 : 3$ 。所以當我們在比對 (US_A, LS_B) 時我們節省了比對 (US_{A1}, LS_{B1}) 的時間, 所以我們可以省下約 $\frac{1}{4} \times \frac{1}{4} = \frac{1}{16}$ 的時間。由於這個演算法是遞迴地在處理, 所以省下的時間是 $\frac{1}{16} \times (1 + \frac{15}{16} + (\frac{15}{16})^2 + \dots + (\frac{15}{16})^{45}) = 0.95$, 則所花的時間就是 $1 - 0.95 = 0.05$, 幾乎加速 20 倍。我們這裡使用了每個 bit 被 set 1 的機率大致相同的假設, 事實上, 我們可以將設 1 頻率越高的 bit (棋盤中越中間的部位) 排越前面, 然後再進行建樹的工作, 此項安排將會再加速本演算法。

在實驗上, 下表列出平均每個 partition slot 在步驟 4 中所花的時間, 其中區分為使用或未使用 bit-tree 技術的時間 (單位為 megacycles), 我們並列出兩個時間的比與 $(\frac{15}{16})^{2n-2}$ 數值的比較。值得觀察的是, 使用 bit-tree 技術的時間與未使用該技術的時間比, 果然從 $n = 6$ 的 2.00 降到 $n = 20$ 的 0.15, 雖然 $(\frac{15}{16})^{(2n-2)}$ 是我們理論上期待的數值, 但其差距亦隨著 n 的增大而縮小, 所以

猜測 $n = 24$ 的所花時間比會小於 $(\frac{15}{16})^{(2n-2)}$ 。

n	使用	未使用	使用/未使用	$(\frac{15}{16})^{2n-2}$
6	1.00E-03	5.00E-04	2.00	0.52
8	1.00E-03	4.29E-04	2.33	0.41
10	2.46E-03	1.27E-03	1.94	0.31
12	1.47E-02	7.36E-03	2.00	0.24
14	1.23E-01	9.12E-02	1.35	0.19
16	1.31E+00	1.93E+00	0.68	0.14
18	1.84E+01	5.50E+01	0.33	0.11
20	2.93E+02	2.01E+03	0.15	0.09
22	5.31E+03	未算出	未算出	0.07
24	1.11E+05	未算出	未算出	0.05

4 實驗結果與討論

我們使用平行計算將 $Q(24)$ 問題依 $C_{12}^{24} = 2,704,156$ partition slots 分成 1000 jobs, 然後以 job 為單位分送各台 PC 計算。在各台 PC 中, 我們將計算程式的執行優先序設至最低, 以免影響其他同仁的工作, 雖然如此, 每台機器大抵都有 90% 以上的時間是在執行 24-Queen 的計算。我們的演算法在每台機器使用了 800 Mega Bytes 記憶體, 成功地在 322,730,315,531.47 mega cycles (約 1867 個 P4-2.0GHz PC 工作天) 算出 $Q(24) = 227,514,172,009,468$ 。

Figure 3 是 20-Queen, 22-Queen, 24-Queen 問題中, 我們演算法的執行各 job 的時間圖表。在三張圖表中, 大抵 job id 比較低的, 執行時間比較久, 這是因為使用 symmetry cutting 策略的關係, 使得後面有很多 partition slot 不需要計算的原因。除了 24-Queen 問題有些瑕疵之外, 三張圖除了時間規模之外, 圖形走勢非常地相似。24-Queen 問題中的幾個超出預期執行時間的 job 可能肇因於某 PC 同時間有其他同仁在大量使用 CPU 時間或是該電腦受病毒影響拖累 CPU 使用效能。

Pion 利用 Backtracing 的方法, 從 $i = 0$ 到 $i = n - 1$ 逐一設定 $f(i)$ 的值, 然後用 “bitwise and” 和 “bitwise shift” 的方法來檢驗 $\forall i \neq j \in Z_n f(i) \neq f(j)$, $\forall i \neq j \in Z_n f(i) - i \neq f(j) - j$, $\forall i \neq j \in Z_n f(i) + i \neq f(j) + j$, 是一個標準且直覺的做法, 但是因為 Pion 的巧思設計 bitwise operator 的運作使得計算速度大幅提升。Pion 並且利用平行計算算出 $Q(23)$ 。Ken'ichiro Takahashi[21] (感謝 Patrick René Guillemin 告知) 利用 Pion 的方法加上使用 8 個 Symmetry operator 來做 symmetry cutting 使得速度再次提升。

我們重新製作 Pion 的方法進行實驗合併 Takahashi 在網頁上的數據整理如下, 所需時間是以 2.0GHz PC 為依據, 其中時間數據格式為 (day hour:minute:second)。數據中值得觀察的是 Pion's Algorithm 所需時間與 Takahashi's Algorithm 所需時間大致成正比, 但是 Partition Strategy 所需時間與 Takahashi's Algorithm 所需時間的比則從 $n = 16$ 的 5 倍下降到 $n = 18$ 的 3 倍多再降到 $n = 20$ 的 2 倍多, 這意味著 Partition Strategies 在 $n = 22$ 或

$n = 24$ 時將取得稍微的領先。

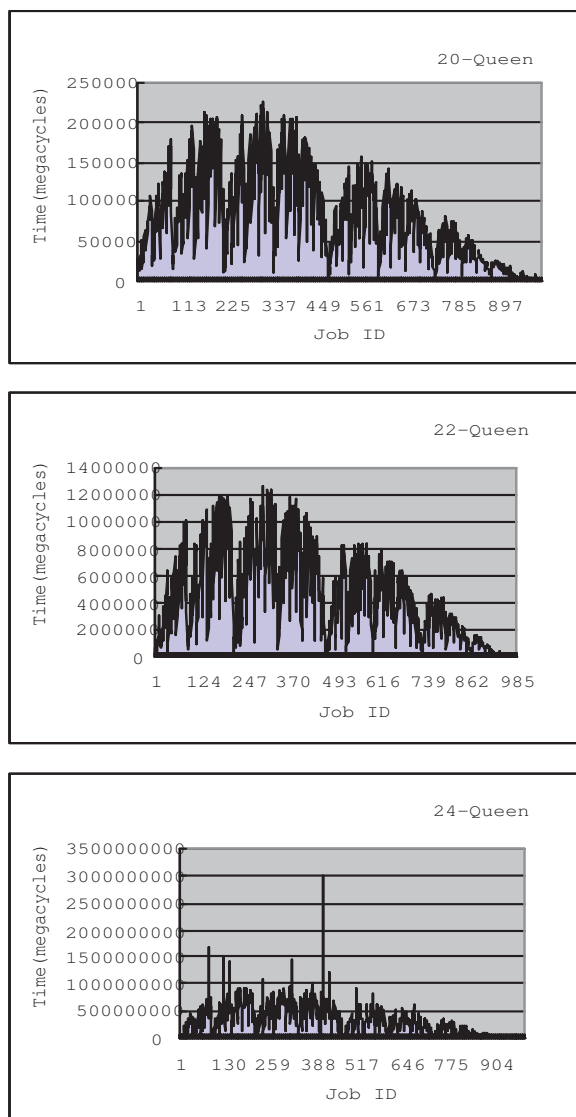


Figure 3: 計算 20, 22, 24-Queen 問題中各 job 的時間。

n	Pion 的演算法 所需時間	Takahashi 的演算法 所需時間	本演算法 所需時間	建樹 記憶體 (KB)
14	0 00:00:01	0 00:00:00	0 00:00:01	92
15	0 00:00:08	0 00:00:01		
16	0 00:00:57	0 00:00:06	0 00:00:30	523
17	0 00:06:21	0 00:00:41		
18	0 00:48:12	0 00:04:56	0 00:16:05	2603
19	0 06:03:39	0 00:37:50		
20	2 01:38:23	0 05:03:35	0 11:09:44	15656
21		1 18:40:00		
22			25.75天	104207
23				
24			1867天	795420

References

- [1] B. ABRAMSON AND M. YUNG, *Divide and conquer under global constraints: A solution to the n -queens problem*, Journal of Parallel and Distributed Computing, 6 (1989), pp. 649–662.
- [2] W. AHRENS, *Mathematische unterhaltungen und spiele*, B. G. Teubner, Leipzig, (1910).
- [3] W. W. R. BALL, *Mathematical recreations and essays*, MacMillan and Co., (1926), p. 113.
- [4] P. CULL AND R. PANDY, *Isomorphism and the n -queens problem*, SIGCSE Bulletin, (1994), pp. 26:29–36.
- [5] C. ERBAS, S. SARKESHIK, AND M. M. TANIK, *Different perspectives of the n -queens problem*, in Proceedings of the ACM 1992 Computer Science Conference, 1992.
- [6] A. B. EVANS, *Orthomorphism graphs of groups*, vol. 1535 of Lecture Notes in Mathematics, Springer-Verlag, 1991.
- [7] J. HSIANG, Y.-P. SHIEH, AND Y.-C. CHEN, *The cyclic complete mappings counting problems*, in Problems and Problem Sets for ATP, volume 02-10 of DIKU technical reports, G. Sutcliffe, J. Pelletier, and C. Sutner, eds., July 31 - Aug 1 2002.
- [8] D. F. HSU, *Orthomorphisms and near orthomorphisms*, in Graph Theory, Combinatorics, and Applications (Y. Alavi eds.), John Wiley and Sons, Inc., 1991, pp. 667–679.
- [9] D. F. HSU AND A. D. KEEDWELL, *Generalized complete mappings, neofields, sequenceable groups and block designs II*, Pacific J. of Math., 117 (1985), pp. 291–312.
- [10] G.-S. HUANG, *Search Reduction Techniques and Applications to Problems in Combinatorics*, PhD thesis, National Taiwan University, 1999.
- [11] W. KOSTERS, *N-Queens* <http://www.liacs.nl/home/kosters/nqueens.html>.
- [12] H. B. MANN, *The construction of orthogonal Latin squares*, Ann. Math. Statistics, 13 (1942), pp. 418–423.

- [13] F. NAUCK, *Schach*, Illustrierter Zeitung, 361 (1850), p. 352.
- [14] A. T. OLSON, *The eight queens problem*, Journal of Computers in Mathematics and Science Teaching, (1993), p. 12:93.
- [15] L. J. PAIGE, *A note on finite abelian groups*, Bull. Amer. Math. Soc., 53 (1947), pp. 590–593.
- [16] S. PION AND J.-Y. FOURRE, *N-Queen Program* <http://www-sop.inria.fr/prisme/personnel/pion/progs/reines.tgz>.
- [17] I. RIVIN, I. VARDI, AND P. ZIMMERMANN, *The n-queens problem*, vol. 101 of Amer. Math. Monthly, 1994, pp. 629–639.
- [18] Y.-P. SHIEH, *Partition Strategies for #P-Complete Problems with Applications to Enumerative Combinatorics*, PhD thesis, National Taiwan University, 2001.
- [19] Y.-P. SHIEH, J. HSIANG, AND D. F. HSU, *On the enumeration of abelian K-complete mappings*, vol. 144 of Congressus Numerantium, 2000, pp. 67–88.
- [20] N. J. A. SLOANE, *Integer Sequence* <http://www.research.att.com/~njas/sequences/>.
- [21] K. TAKAHASHI, *N-Queen Problem* <http://www.ic-net.or.jp/home/takaken/e/queen/index.html>, 2003.

GLY Conjecture on Counting number of integral points in a general n -dimensional tetrahedra

Ke-Pao Lin*

Department of Information Management
Chang Gung Institute of Technology
261 Wen-Hwa 1 Road
Kwei-Shan, Tao-Yuan
Taiwan 333-03, R.O.C
kplin@mail.cgit.edu.tw

Stephen S.-T. Yau[†]Department of Mathematics, Statistics
and Computer Science (M/C 249)
University of Illinois at Chicago
851 South Morgan Street
Chicago, IL 60607-7045, U.S.A.
yau@uic.edu

Abstract

Recently there has been tremendous interest in counting the number of integral points in n -dimensional tetrahedra with non-integral vertex due to its applications in primality testing and factoring in number theory and in singularities theory. The purpose of this paper is to summarize our previous works. A conjecture on sharp upper estimate of the number of integral points in n -dimensional tetrahedra with non-integral vertex which is called GLY conjecture. We show that this

conjecture is true for low dimensional cases as well as in the case of homogeneous n -dimensional tetrahedra.

1 Introduction

The general problem of counting the number $Q_n = Q(a_1, \dots, a_n)$ of nonnegative integral points satisfying

*Research carried out with support of National Science Council Grant NSC 92-2115-M-255-001.

[†]Research supported in part by NSA.

$$\frac{x_1}{a_1} + \frac{x_2}{a_2} + \cdots + \frac{x_n}{a_n} \leq 1 \quad (1.1)$$

where $a_1, a_2, \dots, a_n$ are positive integers, has been a challenging problem for many years. In 1951, Mordell [Mo] gave a formula for Q_3 , expressed in terms of three Dedekind sums, in the case that a_1, a_2, a_3 are pairwise relatively prime. In 1993, Pommersheim [Po], using toric varieties, gave a formula for Q_3 for arbitrary positive integers a_1, a_2 and a_3 . More generally, let Δ be a polytope of dimension n in the lattice $\mathbf{Z}^n$. Denote $\ell_\Delta(k)$ the number of lattice points in Δ dilated by a factor k , where k is an integer:

$$\ell_\Delta(k) = \sharp(k\Delta \cap \mathbf{Z}^n), \quad k \in \mathbf{Z}_+ \quad (1.2)$$

Ehrhart proved that $\ell_\Delta(k)$ is a polynomial in k of degree n .

$$\ell_\Delta(k) = b_n k^n + b_{n-1} k^{n-1} + \cdots + b_0 \quad (1.3)$$

where b_n = volume of Δ , b_{n-1} = half the sum of the volumes of $(n-1)$ -dimensional faces of Δ (measured with respect to the $(n-1)$ -dimensional lattice in the $(n-1)$ -plane), $b_0=1$. In 1993, Kantor and Khovanskii [Ka-Kh] succeeded in computing b_{n-2} . In fact they gave a general formula for the number of integral points in any integral polytope in $\mathbf{R}^4$. In 1994 Cappell and Shaneson [Ca-Sh] have announced a fantastic result with which they can compute all the coefficients b_i in (1.3) explicitly. Unfortunately, in all the above mentioned results, they need to assume the vertex points of Δ are in $\mathbf{Z}^n$.

However, for the sake of applications in number theory and geometry, we are interested in the problem of estimating the number $P_n = P(a_1, \dots, a_n)$ of positive integral points satisfying (1.1), where $a_1, a_2, \dots, a_n$ are positive real numbers. Of course one can deduce an estimate for Q_n once an estimate for P_n is known and vice versa [Li-Ya 2]. In fact, let $a = \frac{1}{a_1} + \cdots + \frac{1}{a_n}$. It is easy to show that

$$P(a_1, \dots, a_n) = Q(a_1(1-a), \dots, a_n(1-a)) = Q(b_1, \dots, b_n). \quad (1.4)$$

The novelty in our problem is that we count the lattice points in a polytope whose vertices are not necessarily integer points (or even rational points). There are at least two reasons to consider this problem.

Firstly, we are told by Professor Granville [Gr] that this is an extremely important question in number theory; it would have many applications to current problems in analytic number theory, primality testing and in factoring. Given a set $\mathcal{P}$ of primes $p_1 < p_2 < \cdots < p_n \leq y$. Number theorists are interested in counting the number of integers $m \leq x$ where $m = p_1^{\ell_1} p_2^{\ell_2} \cdots p_n^{\ell_n}$ is composed only of primes from $\mathcal{P}$, and $x = y^u$, for u not too large (for all $u > 2$ would be nice, but for all $u > \log y$ would still be interesting). Thus they wish to count the number of $(\ell_1, \dots, \ell_n) \in (\mathbf{Z}_+ \cup \{0\})^n$ such that

$$\frac{\ell_1}{a_1} + \frac{\ell_2}{a_2} + \cdots + \frac{\ell_n}{a_n} \leq 1 \quad \text{where} \quad a_i = \frac{\log x}{\log p_i} \geq u$$

Observe that a_i 's are real numbers. This is, of course, the problem Q_n that we consider above. Perhaps the best reference for the application is Carl Pomerance's ICM 1994 lecture at Zürich [Pom 1], and his lecture notes [Pom 2].

Secondly, the problem has an interesting application in geometry and singularity theory. Let $f : (\mathbf{C}^n, 0) \rightarrow (\mathbf{C}, 0)$ be the germ of a complex analytic function with isolated critical point at the origin. Let M be a resolution of $V = \{(z_1, \dots, z_n) \in \mathbf{C}^n : f(z_1, \dots, z_n) = 0\}$. The Milnor number of the singularity $(V, 0)$ is $\mu = \dim \mathbf{C}\{z_1, z_2, \dots, z_n\} / (f_{z_1}, f_{z_2}, \dots, f_{z_n})$. The geometric genus p_g of the singularity $(V, 0)$ is $\dim H^{n-2}(M, \mathcal{O})$, which is an important invariant of the singularity. In 1978 Durfee [Du] made the following conjecture.

Durfee Conjecture $n! p_g \leq \mu$ with equality only when $\mu = 0$.

The connection between Durfee conjecture and the proposed problem is as follows. A polynomial $f(z_1, \dots, z_n)$ is weighted homogeneous of type $(w_1, \dots, w_n)$, where $w_1, \dots, w_n$ are fixed positive rational numbers, if f can be expressed as a linear combination of monomials $z_1^{i_1} \dots z_n^{i_n}$ for which $i_1/w_1 + \dots + i_n/w_n = 1$. If $f(z_1, \dots, z_n)$ is a weighted homogeneous polynomial of type $(a_1, \dots, a_n)$ with an isolated singularity at the origin, then Milnor and Orlik [Mi-Or] proved that $\mu = (a_1 - 1)(a_2 - 1) \dots (a_n - 1)$. On the other hand, Merle and Teissier [Me-Te] showed that p_g is exactly the number P_n of positive integral points satisfying (1.1). Thus Durfee conjecture provides us a guidance for the upper estimate of P_n . Unfortunately Durfee conjecture is not sharp, see for example [Xu-Ya 2]. In 1995, the second author has made the following conjecture.

Conjecture Let $f : (\mathbf{C}^n, 0) \rightarrow (\mathbf{C}, 0)$ be a weighted homogeneous polynomial with an isolated critical point at the origin. Then

$$\mu - h(\nu) \geq n! p_g$$

and equality holds if and only if f is a homogeneous polynomial. Here $h(\nu)$ is a polynomial function on the multiplicity ν with the properties $h(\nu) \geq 0$ and $h(\nu) = 0$ if and only if $\nu = 1$.

Counting the number of integral points in an n -dimensional tetrahedra with non-integral vertices is a classical subject which has attracted a lot of famous mathematicians. For $n = 2$, Hardy and Littlewood [Ha-Li 2] [Ha-Li 3] wrote two famous papers on the lattice points of a right-angled triangle because of its relations to their International Congress of Mathematics lecture [Ha-Li 1] in 1912 on Diophantine approximation. A general result was obtained by D.C. Spencer [Sp 1] [Sp 2] in 1942 via complex function-theoretic methods, as follows. Write $a_i = x/\rho_i$, $1 \leq i \leq n$. Then

$$Q(a_1, \dots, a_n) = \frac{1}{n!} a_1 \dots a_n + \frac{1}{2(n-1)!} \left(\frac{a_n}{a_1} + \dots + \frac{a_n}{a_{n-1}} + 1 \right) (a_1 \dots a_{n-1}) + o(x^{n-1}).$$

In 1975, Beukers [Be] used an elementary method to reprove the above formula. Unlike the

sharp upper estimate in our Main Theorem, the above formula of Hardy-Littlewood-Spencer is only asymptotic.

Rosser [Ro] in 1939 obtained a lower bound for $Q(a_1, \dots, a_n)$ as a polynomial $R(a_1, \dots, a_n)$ which, when extended to the general tetrahedron, may be written

$$R(a_1, \dots, a_n) = \frac{1}{n! \rho_1 \rho_2 \dots \rho_n} \left(x^n + \frac{n}{2} \sigma'_1 x^{n-1} + \frac{n(n-1)}{2^2} \sigma'_2 x^{n-2} + \dots + \frac{n!}{2^{n-1}} \sigma'_{n-1} x \right)$$

where σ'_k is the sum of the products k at a time of $\rho_2, \rho_3, \dots, \rho_n$. Lehmer [Le] constructed two polynomials $\ell(a_1, \dots, a_n)$ and $L(a_1, \dots, a_n)$ which approximate $Q(a_1, \dots, a_n)$ from below and above respectively. $\ell(a_1, \dots, a_n)$ and $L(a_1, \dots, a_n)$ are polynomials of degree n in x with coefficients depending on $\rho_1, \dots, \rho_n$. The first two coefficients of $\ell(a_1, \dots, a_n)$ are seen to agree with those of $R(a_1, \dots, a_n)$. For $n > 2$ and x large enough, Lehmer asserted that

$$R(a_1, \dots, a_n) < \ell(a_1, \dots, a_n) < Q(a_1, \dots, a_n).$$

Let $(\rho_1, \dots, \rho_5)$ be $(\log_{10} 2, \log_{10} 3, \log_{10} 5, \log_{10} 5, \log_{10} 7, \log_{10} 11)$. A.E. Western has prepared extensive tables of $Q(a_1, a_2, a_3, a_4, a_5)$. From these tables, he has constructed an approximating polynomial $w(a_1, a_2, \dots, a_n)$ by applying the method of least squares. Unfortunately, all these polynomials are far from being sharp. In particular, they are not useful for the geometric applications mentioned above.

If $f(z)$ is a weighted homogeneous polynomial of type $(w_1, \dots, w_n)$, then the Milnor number μ is given by $\mu = (w_1 - 1)(w_2 - 1) \dots (w_n - 1)$. Therefore Durfee Conjecture is a special case of the following conjecture.

Conjecture 1.1 *Let $a_1, \dots, a_n$ be positive real numbers greater than or equal to two. Then*

$$n! P_{(a_1, \dots, a_n)} \leq (a_1 - 1)(a_2 - 1) \dots (a_n - 1). \quad (1.5)$$

The estimate in the above conjecture is sharper than the following polynomial estimate (1.7) provided by number theorist. Attach a unit cube to the right and above each lattice point of $\Delta(a_1, \dots, a_n)$. Then

$$\begin{aligned} Q_{(a_1, \dots, a_n)} &= \Sigma \text{ volume of the unit cube attached to each lattice point} \\ &\leq \text{volume of } \left\{ (x_1, \dots, x_n) \in \mathbf{R}_+^n : \sum_{i=1}^n \frac{x_i - 1}{a_i} \leq 1 \right\} \\ &= \frac{1}{n!} \left(\prod_{i=1}^n a_i \right) \left(1 + \sum_{j=1}^n \frac{1}{a_j} \right)^n \end{aligned} \quad (1.6)$$

Hence by (1.4) and (1.5), we have

$$\begin{aligned} P_{(a_1, \dots, a_n)} &= Q_{(b_1, \dots, b_n)} \\ &\leq \frac{1}{n!} \left(\prod_{i=1}^n b_i \right) \left(1 + \sum_{j=1}^n \frac{1}{b_j} \right)^n \\ &= \frac{1}{n!} \prod_{i=1}^n a_i. \end{aligned} \quad (1.7)$$

The estimate of $P_{(a_1, \dots, a_n)}$ given by (1.5) is nice. However it is not sharp enough to provide a solution of the following thirty year old problem in singularities theory.

Problem. Let $f : (\mathbb{C}^n, 0) \longrightarrow (\mathbb{C}, 0)$ be a complex analytic function with isolated critical point at the origin. Find an intrinsic characterization for f to be a homogeneous polynomial.

In order to solve the above problem, we need a sharp upper polynomial estimate of $P_{(a_1, \dots, a_n)}$ such that equality holds if and only if $a_1 = a_2 = \dots = a_n = \text{integer}$. The intrinsic characterization of homogeneous polynomial problem was solved for $n = 3$ by Xu and Yau [Xu-Ya1] and for $n = 4$ by Lin and Yau [Li-Ya1]. The purpose of this paper is to formulate a conjectural sharp polynomial upper estimate for $P_{(a_1, \dots, a_n)}$ such that equality holds if and only if $a_1 = a_2 = \dots = a_n = \text{integer}$. In section 2, we show that our conjecture is true for $n = 3, 4$ and 5. In section 3, we give an exact formula for $P_{(a_1, \dots, a_n)}$ for homogeneous n -dimensional tetrahedra (i.e. $a_1 = \dots = a_n$). We show that our conjecture is also true in this case.

2 Sharp polynomial upper estimate

Before we formulate our sharp polynomial upper estimate conjecture, it is convenient to introduce some notations. Let $a, a_1, \dots, a_n$ be positive real numbers greater than or equal to $n - 1$. We shall denote

$$S_k^{n-1} = \sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n-1} i_1 i_2 \dots i_k, \quad S_0^{n-1} = 1, \quad S_{n-1}^{n-1} = 1 \cdot 2 \dots (n-1) \quad (2.1)$$

where $i_1, i_2, \dots, i_k$ are integers. Then

$$\begin{aligned} a(a-1)(a-2) \dots (a-(n-1)) &= a^n - \left(\sum_{i_1=1}^{n-1} i_1 \right) a^{n-1} + (-1)^2 \sum_{1 \leq i_1 < i_2 \leq n-1} i_1 i_2 a^{n-2} \\ &\quad + \dots + (-1)^k \sum_{1 \leq i_1 < \dots < i_k \leq n-1} i_1 i_2 \dots i_k a^{n-k} + \dots + (-1)^{n-1} \left(\prod_{i=1}^{n-1} i \right) a \\ &= a^n + (-1) S_1^{n-1} a^{n-1} + (-1)^2 S_2^{n-1} a^{n-2} + \dots + (-1)^k S_k^{n-1} a^{n-k} + \dots + (-1)^{n-1} S_{n-1}^{n-1} a. \end{aligned} \quad (2.2)$$

We shall denote

$$A_{n-k}^n = \left(\prod_{i=1}^n a_i \right) \sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n} \frac{1}{a_{i_1} a_{i_2} \dots a_{i_k}} \quad (2.3)$$

$$A_n^n = \prod_{i=1}^n a_i, \quad A_0^n = 1. \quad (2.4)$$

Observe that A_{n-k}^n is a polynomial in $a_1, \dots, a_n$ of degree $n - k$.

The following is our conjecture on sharp polynomial estimate for $P_{(a_1, \dots, a_n)}$.

Main Conjecture. Let $P_{(a_1, \dots, a_n)} = \# \left\{ (x_1, \dots, x_n) \in \mathbb{Z}_+^n : \frac{x_1}{a_1} + \dots + \frac{x_n}{a_n} \leq 1 \right\}$, where $a_1 \geq a_2 \geq \dots \geq a_n \geq n-1$, $n \geq 3$. Then

$$\begin{aligned}
n!P_{(a_1, \dots, a_n)} &\leq \prod_{i=1}^n a_i + (-1) \frac{S_1^{n-1}}{n} \left(\prod_{i=1}^n a_i \right) \left(\sum_{k=1}^n \frac{1}{a_k} \right) + (-1)^2 \frac{S_2^{n-1}}{\binom{n-1}{1}} \left(\prod_{i=1}^{n-1} a_i \right) \left(\sum_{k=1}^{n-1} \frac{1}{a_k} \right) \\
&\quad + (-1)^3 \frac{S_3^{n-1}}{\binom{n-1}{2}} \left(\prod_{i=1}^{n-1} a_i \right) \left(\sum_{1 \leq i_1 < i_2 \leq n-1} \frac{1}{a_{i_1} a_{i_2}} \right) \\
&\quad + (-1)^4 \frac{S_4^{n-1}}{\binom{n-1}{3}} \left(\prod_{i=1}^{n-1} a_i \right) \left(\sum_{1 \leq i_1 < i_2 < i_3 \leq n-1} \frac{1}{a_{i_1} a_{i_2} a_{i_3}} \right) \\
&\quad + \dots + \\
&\quad + (-1)^{k+1} \frac{S_{k+1}^{n-1}}{\binom{n-1}{k}} \left(\prod_{i=1}^{n-1} a_i \right) \left(\sum_{1 \leq i_1 < \dots < i_k \leq n-1} \frac{1}{a_{i_1} a_{i_2} \dots a_{i_k}} \right) \\
&\quad + \dots + \\
&\quad + (-1)^{n-1} \frac{S_{n-1}^{n-1}}{\binom{n-1}{n-2}} (a_1 + a_2 + \dots + a_{n-1}) \\
&= A_n^n + (-1) \frac{S_1^{n-1}}{n} A_{n-1}^n + (-1)^2 \frac{S_2^{n-1}}{\binom{n-1}{1}} A_{n-2}^{n-1} + (-1)^3 \frac{S_3^{n-1}}{\binom{n-1}{2}} A_{n-3}^{n-1} \\
&\quad + (-1)^4 \frac{S_4^{n-1}}{\binom{n-1}{3}} A_{n-4}^{n-1} + \dots + (-1)^{k+1} \frac{S_{k+1}^{n-1}}{\binom{n-1}{k}} A_{n-k-1}^{n-1} + \dots + (-1)^{n-1} \frac{S_{n-1}^{n-1}}{\binom{n-1}{n-2}} A_1^{n-1} \quad (2.5)
\end{aligned}$$

and the equality holds if and only if $a_1 = a_2 = \dots = a_n = \text{integer}$.

For $n = 3$, the Main Conjecture asserts that for $a_1 \geq a_2 \geq a_3 \geq 2$,

$$\begin{aligned}
3!P_{(a_1, a_2, a_3)} &\leq A_3^3 - \frac{S_1^2}{3} A_2^3 + \frac{S_2^2}{\binom{2}{1}} A_1^2 \\
&= a_1 a_2 a_3 - \frac{1+2}{3} a_1 a_2 a_3 \left(\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} \right) + \frac{2}{2} a_1 a_2 \left(\frac{1}{a_1} + \frac{1}{a_2} \right) \\
&= a_1 a_2 a_3 - (a_1 a_2 + a_1 a_3 + a_2 a_3) + (a_1 + a_2)
\end{aligned}$$

with equality if and only if $a_1 = a_2 = a_3 = \text{integer}$. This is the main result proved by Xu and Yau in their paper [Xu-Ya1].

For $n = 4$, the Main Conjecture asserts that for $a_1 \geq a_2 \geq a_3 \geq a_4 \geq 3$,

$$4!P_{(a_1, a_2, a_3, a_4)} \leq A_4^4 + (-1) \frac{S_1^3}{4} A_3^4 + (-1)^2 \frac{S_2^3}{\binom{3}{1}} A_2^3 + (-1)^3 \frac{S_3^3}{\binom{3}{2}} A_1^3$$

$$\begin{aligned}
&= a_1 a_2 a_3 a_4 - \frac{1+2+3}{4} a_1 a_2 a_3 a_4 \left(\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \frac{1}{a_4} \right) \\
&\quad + \frac{1 \cdot 2 + 1 \cdot 3 + 2 \cdot 3}{3} a_1 a_2 a_3 \left(\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} \right) \\
&\quad - \frac{1 \cdot 2 \cdot 3}{3} a_1 a_2 a_3 \left(\frac{1}{a_1 a_2} + \frac{1}{a_1 a_3} + \frac{1}{a_2 a_3} \right) \\
&= a_1 a_2 a_3 a_4 - \frac{3}{2} (a_1 a_2 a_3 + a_1 a_2 a_4 + a_1 a_3 a_4 + a_2 a_3 a_4) \\
&\quad + \frac{11}{3} (a_1 a_2 + a_1 a_3 + a_2 a_3) - 2(a_1 + a_2 + a_3)
\end{aligned}$$

with equality if and only if $a_1 = a_2 = a_3 = a_4 = \text{integer}$. Xu and Yau [Xu-Ya3] proved that if $a_1 \geq a_2 \geq a_3 \geq a_4 \geq 2$ and $P_{(a_1, a_2, a_3, a_4)} > 0$, then the above result is true. Lin and Yau [Li-Ya1] proved that the condition $P_{(a_1, a_2, a_3, a_4)} > 0$ can be removed if we assume $a_1 \geq a_2 \geq a_3 \geq a_4 \geq 3$. Therefore our Main Conjecture is true for $n = 4$.

For $n = 5$, the Main Conjecture asserts that for $a_1 \geq a_2 \geq a_3 \geq a_4 \geq a_5 \geq 4$.

$$\begin{aligned}
&5!P_{(a_1, a_2, a_3, a_4, a_5)} \leq A_5^5 + (-1) \frac{S_1^4}{5} A_4^5 + \frac{S_2^4}{\binom{4}{1}} A_3^4 - \frac{S_3^4}{\binom{4}{2}} A_2^4 + \frac{S_4^4}{\binom{4}{3}} A_1^4 \\
&= a_1 a_2 a_3 a_4 a_5 - \frac{1+2+3+4}{5} a_1 a_2 a_3 a_4 a_5 \left(\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \frac{1}{a_4} + \frac{1}{a_5} \right) \\
&\quad + \frac{(1 \cdot 2 + 1 \cdot 3 + 1 \cdot 4 + 2 \cdot 3 + 2 \cdot 4 + 3 \cdot 4)}{4} a_1 a_2 a_3 a_4 \left(\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \frac{1}{a_4} \right) \\
&\quad - \frac{(1 \cdot 2 \cdot 3 + 1 \cdot 2 \cdot 4 + 1 \cdot 3 \cdot 4 + 2 \cdot 3 \cdot 4)}{\binom{4}{2}} a_1 a_2 a_3 a_4 \\
&\quad \quad \left(\frac{1}{a_1 a_2} + \frac{1}{a_1 a_3} + \frac{1}{a_1 a_4} + \frac{1}{a_2 a_3} + \frac{1}{a_2 a_4} + \frac{1}{a_3 a_4} \right) \\
&\quad + \frac{1 \cdot 2 \cdot 3 \cdot 4}{4} a_1 a_2 a_3 a_4 \left(\frac{1}{a_1 a_2 a_3} + \frac{1}{a_1 a_2 a_4} + \frac{1}{a_2 a_3 a_4} \right) \\
&= a_1 a_2 a_3 a_4 a_5 - 2(a_1 a_2 a_3 a_4 + a_1 a_2 a_3 a_5 + a_1 a_2 a_4 a_5 + a_1 a_3 a_4 a_5 + a_2 a_3 a_4 a_5) \\
&\quad + \frac{35}{4} (a_1 a_2 a_3 + a_1 a_2 a_4 + a_1 a_3 a_4 + a_2 a_3 a_4) \\
&\quad - \frac{25}{3} (a_1 a_2 + a_1 a_3 + a_1 a_4 + a_2 a_3 + a_2 a_4 + a_3 a_4) \\
&\quad + 6(a_1 + a_2 + a_3 + a_4)
\end{aligned}$$

with equality if and only if $a_1 = a_2 = a_3 = a_4 = a_5 = \text{integer}$. This is the main result proved by Lin and Yau in their paper [Li-Ya3].

3 Number of positive integral points in a homogeneous n -dimensional tetrahedra

In this section we shall prove that our Main Conjecture in section 2 holds for homogeneous n -dimensional tetrahedra $\Delta(a, a, \dots, a)$. We first start with an elementary summation by parts lemma.

Lemma 3.1 *Let $\{u_k\}, \{v_k\}, k = 1, \dots, n$ be two sequence of numbers. Then*

$$\sum_{k=1}^n u_k v_k = u_n \sum_{k=1}^n v_k - \sum_{r=2}^n \left[(u_r - u_{r-1}) \sum_{k=1}^{r-1} v_k \right]. \quad (3.1)$$

PROOF: We shall prove (3.1) by induction. (3.1) is obviously true for $n = 2$. Assume that (3.1) holds for $n - 1$. Then

$$\begin{aligned} \sum_{k=1}^n u_k v_k &= \sum_{k=1}^{n-1} u_k v_k + u_n v_n \\ &= u_{n-1} \sum_{k=1}^{n-1} v_k - \sum_{r=2}^{n-1} \left[(u_r - u_{r-1}) \sum_{k=1}^{r-1} v_k \right] + u_n v_n \\ &= u_n \sum_{k=1}^n v_k - u_n \sum_{k=1}^{n-1} v_k + u_{n-1} \sum_{k=1}^{n-1} v_k - \sum_{r=2}^{n-1} \left[(u_r - u_{r-1}) \sum_{k=1}^{r-1} v_k \right] \\ &= u_n \sum_{k=1}^n v_k - (u_n - u_{n-1}) \sum_{k=1}^{n-1} v_k - \sum_{r=2}^{n-1} \left[(u_r - u_{r-1}) \sum_{k=1}^{r-1} v_k \right] \\ &= u_n \sum_{k=1}^n v_k - \sum_{r=2}^n \left[(u_r - u_{r-1}) \sum_{k=1}^{r-1} v_k \right] \end{aligned}$$

Q.E.D.

Theorem 3.1 *Let $n \geq 2$ be a positive integer and $a \geq n$. Let P_n be the number of positive integral solution of $\frac{x_1}{a} + \frac{x_2}{a} + \dots + \frac{x_n}{a} \leq 1$, i.e. $P_n = \#\{(x_1, x_2, \dots, x_n) \in \mathbb{Z}_+^n : \frac{x_1}{a} + \frac{x_2}{a} + \dots + \frac{x_n}{a} \leq 1\}$. Let $[a]$ be the greatest integer less than or equal to a . Then*

$$\begin{aligned} n!P_n &= [a]([a] - 1)([a] - 2) \cdots ([a] - n + 1) \\ &\leq a(a - 1)(a - 2) \cdots (a - n + 1) \end{aligned} \quad (3.2)$$

with equality if and only if a is an integer.

PROOF: For $n = 2$, we have $\frac{x_1}{a} + \frac{x_2}{a} \leq 1$. From the level $x_2 = 1$ to the level $x_2 = [a] - 1$, we have the following positive integral points:

$$\begin{aligned} x_2 = 1 & : (1, 1), (2, 1), \dots, ([a] - 1, 1) \\ x_2 = 2 & : (1, 2), (2, 2), \dots, ([a] - 2, 2) \\ & \vdots \\ x_2 = [a] - 1 & : (1, [a] - 1). \end{aligned}$$

Hence $P_2 = 1 + 2 + \dots + ([a] - 1) = \frac{[a]([a]-1)}{2}$, i.e. $2!P_2 = [a]([a] - 1)$.

Now assume that (3.2) holds for any integer n . Consider a homogeneous $(n+1)$ -dimensional tetrahedra $\frac{x_1}{a} + \frac{x_2}{a} + \dots + \frac{x_{n+1}}{a} \leq 1$.

From the level $x_{n+1} = 1$ to the level $x_{n+1} = [a] - n$, we have the following homogeneous n -dimensional tetrahedra:

$$\begin{aligned} x_{n+1} = 1 & : \frac{x_1}{a-1} + \frac{x_2}{a-1} + \dots + \frac{x_n}{a-1} \leq 1 \\ x_{n+1} = 2 & : \frac{x_1}{a-2} + \frac{x_2}{a-2} + \dots + \frac{x_n}{a-2} \leq 1 \\ & \vdots \\ x_{n+1} = [a] - n & : \frac{x_1}{a-[a]+n} + \frac{x_2}{a-[a]+n} + \dots + \frac{x_n}{a-[a]+n} \leq 1. \end{aligned}$$

Hence by induction hypothesis we have

$$\begin{aligned} n!P_{n+1} &= \sum_{k=n}^{[a]-1} k(k-1)\dots(k-(n-1)) \\ &= \sum_{k=1}^{[a]-1} k(k-1)\dots(k-(n-1)). \end{aligned}$$

Now we apply Lemma 3.1 with $u_k = (k-1)(k-2)\dots(k-(n-1))$, $v_k = k$. Then $\sum_{k=1}^{r-1} v_k =$

$$\sum_{k=1}^{r-1} k = \frac{r(r-1)}{2}$$

$$\begin{aligned} u_r - u_{r-1} &= (r-1)(r-2)\dots(r-(n-1)) - (r-2)(r-3)\dots(r-n) \\ &= (r-2)(r-3)\dots(r-n+1)((r-1) - (r-n)) \\ &= (n-1)(r-2)(r-3)\dots(r-n+1). \end{aligned}$$

From (3.1), we have

$$\sum_{k=1}^{[a]-1} k(k-1)\dots(k-n+1)$$

$$\begin{aligned}
&= u_{[a]-1} \frac{[a]([a]-1)}{2} - \sum_{r=2}^{[a]-1} (n-1)(r-2)(r-3) \dots (r-n+1) \frac{r(r-1)}{2} \\
&= ([a]-2)([a]-3) \dots ([a]-n) \frac{[a]([a]-1)}{2} - \frac{n-1}{2} \sum_{k=1}^{[a]-1} k(k-1)(k-2) \dots (k-n+1).
\end{aligned}$$

Hence we have

$$\begin{aligned}
&\left(1 + \frac{n-1}{2}\right) \sum_{k=1}^{[a]-1} k(k-1)(k-2) \dots (k-n+1) \\
&= \frac{1}{2} [a]([a]-1)([a]-2) \dots ([a]-n),
\end{aligned}$$

which implies

$$(n+1) \sum_{k=1}^{[a]-1} k(k-1)(k-2) \dots (k-n+1) = [a]([a]-1)([a]-2) \dots ([a]-n).$$

Therefore we have

$$\begin{aligned}
n!P_{n+1} &= \sum_{k=1}^{[a]-1} k(k-1)(k-2) \dots (k-n+1) \\
&= \frac{1}{n+1} [a]([a]-1)([a]-2) \dots ([a]-n)
\end{aligned}$$

i.e. $(n+1)!P_{n+1} = [a]([a]-1)([a]-2) \dots ([a]-n)$. *Q.E.D.*

Corollary 3.1 *The Main Conjecture in section 2 is true for n -dimensional homogeneous tetrahedra $\Delta(a, a, \dots, a)$.*

PROOF: Set $a_1 = a_2 = \dots = a_n = a$ in the Main Conjecture. We get

$$\begin{aligned}
n!P_{(a, \dots, a)} &\leq a^n + (-1)S_1^{n-1}a^{n-1} + (-1)^2S_2^{n-1}a^{n-2} + (-1)^3S_3^{n-1}a^{n-3} \\
&\quad + \dots + (-1)^{k+1}S_{k+1}^{n-1}a^{n-k-1} + \dots + (-1)^{n-1}S_{n-1}^{n-1}a \\
&= a(a-1)(a-2) \dots (a-(n-1)) \quad \text{by (2.2)}
\end{aligned}$$

This is exactly Theorem 3.1. *Q.E.D.*

References

[Br-Vel] Brion, M. and Vergne, M., An equivariant Riemann-Roch theorem for complete, simplicial toric varieties, J. Reine Angew. Math. 482 (1997), 67-92.

- [Br-Ve2] Brion, M. and Vergne, M., Lattice points in simple polytopes, *Journal of American Mathematical Society* 10 (1997), 371-392.
- [Ca-Sh] Cappell, S.E. and Shaneson, J.L., Genera of algebraic varieties and counting lattice points, *Bulletin of American Mathematical Society* 30 (1994), 62-69.
- [Di-Ro] Diaz, R. and Robbin, S., The Ehrhart polynomial of a lattice polytope, *Annals of Mathematics* 135 (1997), 503-518.
- [Du] Durfee, A.H., The signature of smoothings of complex surface singularities, *Math. Ann.* 232 (1978), 85-98.
- [Eh] Ehrhart, E., Sur un probleme de geometrie diophantienne lineaire II, *J. Reine Angew Math.* 227 (1967), 25-49.
- [Ha-Li1] Hardy, G.H. and Littlewood, J.E., Some problems of diophantine approximation, *Proc. 5th Int. Congress of Mathematics* (1912), 223-229.
- [Ha-Li2] Hardy, G.H. and Littlewood, J.E., The lattice points of a right-angled triangle, *Proc. London Math. Soc.* (2), 20 (1921), 15-36.
- [Ha-Li3] Hardy, G.H. and Littlewood, J.E., The lattice points of a right-angled triangle (second memoir), *Hamburg Math. Abhandlungen*, 1 (1922), 212-49.
- [Ha-Li4] Hardy, G.H. and Littlewood, J.E., A series of cosecants, *Bull. Calcutta math. Soc.* 20 (1930), 251-66.
- [Hi-Za] Hirzebruch, F. and Zagier, D., *The Atiyah-Singer Index Theorem and Elementary Number Theory*, Publish or Perish, Inc., Boston, MA, 1974.
- [Ka-Kh] J.M. Kanter and A. Khovanskii, Une application du Théorème de Riemann-Roch combinatoire au polynôme d'Ehrhart des polytopes entier de $\mathbb{R}^d$, *C.R. Acad. Sci. Paris* 317 (I) (1993), 501-507.
- [Li-Ya1] Lin, K.-P. and Yau, S.S.-T., Sharp upper estimate of geometric genus in terms of Milnor number and multiplicity with application on coordinate free characterization of 3-dimensional homogeneous singularities, (preprint).
- [Li-Ya2] Lin, K.-P. and Yau, S.S.-T., Analysis of sharp polynomial upper estimate of number of positive integral points in 4-dimensional tetrahedra, *J. reine angew. Math.* 547 (2002), 191-205.
- [Li-Ya3] Lin, K.-P. and Yau, S.S.-T., A sharp upper estimate of the number of integral points in 5-dimensional tetrahedra, *Journal of Number Theory*, 93 (2002), 207-234.
- [Me-Te] Merle, M. and Teissier, B., Conditions d'adjonction d'après Du Val, *Séminaire sur les singularités des surfaces* (center de Math. de l'Ecole Polytechnique, 1976-1977), *Lecture notes in Math*, vol. 777, Springer, Berlin, 1980, 229-245.

- [Mo1] Mordell, L.J., Lattice points in a tetrahedron and Dedekind sums, J. Indian Math. 15 (1951), 41-46.
- [Mor] Morelli, R., Pick's theorem and the Todd class of tori variety, Advances in Mathematics 100 (1993), 183-231.
- [Po] Pommersheim, J., Toric varieties, lattice points and Dedekind sums, Math. Ann. 295 (1993), 1-24.
- [Sp1] Spencer, D.C., On a Hardy-Littlewood problem of Diophantine approximation, Proc. Cambridge Phil. Soc., XXXV (1939), 527-547.
- [Sp2] Spencer, D.C., The lattice points of tetrahedron, Journal of Mathematics and Physics, vol. XXI, no. 3, Oct., (1942), 189-197.
- [Xu-Ya1] Xu, Y.-J. and Yau, S.S.-T., Sharp estimate of number of integral points in a tetrahedron, J. Reine Angew. Math. 423 (1992), 199-219.
- [Xu-Ya2] Xu, Y.-J. and Yau, S.S.-T., Durfee conjecture and coordinate free characterization of homogeneous singularities, J. Diff. Geom. 37 (1993), 375-396.
- [Xu-Ya3] Xu, Y.-J. and Yau, S.S.-T., Sharp estimate of numbers of integral points in a 4-dimensional tetrahedron, J. Reine Angew. Math. 473 (1996), 1-23.

求出正整數所有因數分解形式之演算法

黃德豐

龍華科技大學電機工程學系

dfhuang@mail.lhu.edu.tw

摘要

本文提出一種可求出給定正整數所有因數分解形式的演算法，其關鍵過程等價於可重複選取之組合問題，在引進相關之定義後，提出一可實作之演算法及實例說明。

關鍵字：因數分解，演算法

1. 介紹

此處所提出的是一個相當簡單的問題，應該是國中階段就會碰見的問題，就是當給定一個正整數時，如何求出其所有的因數組合。當然此處不需考慮質數之情形，雖然討論質數的文獻遠比討論合成數的數量多。當其數值不大時，要找出所有的因數組合應該不難，但是在給定任意整數後，若要求出所有因數相乘形式，勢必需要具體之演算法，因此本文提出的是可由程式實作之演算法以達到上述目的。

第 2 節為基本之問題陳述，第 3 節為等價之組合問題，第 4 節為第 3 節之組合概念之嚴謹數學表示形式，第 5 節為演算法，第 6 節為一實例。

2. 問題觀察

由算術基本定理可知，任意正整數可改寫為正質數的乘積，而且不考慮其組合位置知其唯一質數分解表示式如下（例[1]）

$$N = p_1^{d_1} \cdot p_2^{d_2} \cdots p_r^{d_r} \quad (1)$$

其中 p_i ($1 \leq i \leq r$) 為質數且不失一般性假設 $p_1 > p_2 > \cdots > p_r$ 為 N 的質因數。再由另一定理

若 $a = p_1^{a(1)} p_2^{a(2)} \cdots p_r^{a(r)}$ 且 $b | a$ ，則

$$b = p_1^{b(1)} p_2^{b(2)} \cdots p_r^{b(r)}, \quad 0 \leq b(i) \leq a(i),$$

$$1 \leq i \leq r。$$

由於具有 (1) 分解形式的任意給定正數 N 之因數可寫為

$$N_i = p_1^{e_{1,i}} \cdot p_2^{e_{2,i}} \cdots p_r^{e_{r,i}}。$$

因此，當此正整數寫為 k 個因數的乘積時，例如

$$N = N_1 N_2 \cdots N_k \quad (2)$$

顯然可知（例[1]）

$$N_1 = p_1^{e_{1,1}} \cdot p_2^{e_{1,2}} \cdots p_r^{e_{1,r}} \quad (3a)$$

$$N_2 = p_1^{e_{2,1}} \cdot p_2^{e_{2,2}} \cdots p_r^{e_{2,r}},$$

$\cdots,$

$$N_k = p_1^{e_{k,1}} \cdot p_2^{e_{k,2}} \cdots p_r^{e_{k,r}}$$

對於每個因數 N_i 中之質因數次方與 N 中之質因數次方而言，具有 ($1 \leq i \leq k, 1 \leq j \leq r$)

$$0 \leq e_{i,j} \leq d_j \quad (3b)$$

而且

$$\sum_{j=1}^r e_{i,j} = d_i \quad (3c)$$

的關係。

3. 組合之等價思考方式

由於正整數之因數可由 N 本身及寫成 D 個因數的乘積，因此吾人要求出這些因數，亦即必須由一個因數求到 D 個因數的乘積。換個角度來看，當把所有質因數看成一個標上數字的球，原來的質因數分解形式 (1) 可以看做裝著 D 個球的箱子，每個球標明一個質因數之數值，因此此

箱中裝有 d_1 個標著 p_1 的球， d_2 個標著 p_2 的球， $\dots$ ， d_r 個標著 p_r 的球，共有 D 個球。要找出 N 為 k 個因數的乘積時，相當於將這 D 個標上因數數字的球放進 k 個箱子中，每一個箱子中所有球上數字的乘積即為形成 k 個因數的乘積中的，例如對於其中任何一個因數，例如

$$N_i = p_1^{e_{i,1}} \cdot p_2^{e_{i,2}} \cdots p_r^{e_{i,r}} \text{ 而言，要找出其因數相}$$

當於選出 $e_{i,1}$ 個標著 p_1 的球， $e_{i,2}$ 個標著 p_2 的

球， $\dots$ ， $e_{i,r}$ 個標著 p_r 的球放進第 i 個箱子，因

此若要找出所有形如 (2) 中之 k 個因數，相當於將全部共 D 個標著因數數字的球依次放進 1 個箱子 (即 $N = p_1^{d_1} \cdot p_2^{d_2} \cdots p_r^{d_r}$)，2 個箱子，一直到放進 D 個箱子 (即 $N = p_1^{d_1} \cdot p_2^{d_2} \cdots p_r^{d_r}$ 的情形) 為止。

但當有 k 個箱子時，每個箱子該放幾個球？若不考慮組合的順序時，假設不失一般性的情況下，即表示由大到小，只考慮每個箱子該放幾個球，由 (2) 所有指數和可知道，共有 D 個球要放進 k 個箱子，假設第 1 個箱子放 q_1 個球，第 2 個箱子放 q_2 個球， $\dots$ 第 k 個箱子放 q_k 個球；亦即第 i 個箱子放 q_i 個球時，必須將所有球放進 k 個箱子，所以 $D = q_1 + q_2 + \cdots + q_k$ 。

亦即當將 D 寫為 k 個正整數的和 $D = q_1 + q_2 + \cdots + q_k$ 形式時，要決定這 k 個箱子每個箱子放的球數，必須找出將 D 分解為 k 個正數和的方式；換成數學的說法，這些分配給各箱子個數的方法即為決定出給定一正數的數值分割 (numerical partition)，所謂 k -數值分割表示將一正整數分解為 k 個正整數之和；其方法可在一般討論組合學演算法的書中見到 (例如[2])。將 D 分解為 k 個正數和的數目的公式可在近代討論離散數學的書中見到 (例如[1])。

4 嚴謹形式

吾人必須引進一個與集合概念類似但允許元素重複的集合稱為複集合 (multi-set) 的定義[3] 以幫助符號的使用。由此表示式可知道，對於任

何質因數分解的整數 $N = p_1^{d_1} \cdot p_2^{d_2} \cdots p_r^{d_r}$ 而言，其對應之複集合為

$$F_N^D = \{d_1 \cdot p_1, d_2 \cdot p_2, \dots, d_r \cdot p_r\}_M$$

$$= \left\{ \underbrace{p_1, p_1, \dots, p_1}_{d_1}, \underbrace{p_2, p_2, \dots, p_2}_{d_2}, \dots, \underbrace{p_r, p_r, \dots, p_r}_{d_r} \right\}_M$$

，其中上標 D 表示該複集合的元素個數而足標 N 表示被因數分解的整數， d_i 為質因數 p_i 的指數且 $D = d_1 + d_2 + \cdots + d_r$ ；換言之，

$$M = \prod_{p \in F_M^D} p。$$

對於任何可表示成如 (2) 的因數分解形式時之對應複集合為 $F_N^k = \{N_1, N_2, \dots, N_k\}_M$ ，而且每一因數 N_i 具 (3a) - (3c) 的關係時之對應複集合分別為

$$F_{N_1}^{q_1} = \{e_{1,1} \cdot p_1, e_{1,2} \cdot p_2, \dots, e_{1,r} \cdot p_r\}_M，$$

$$F_{N_2}^{q_2} = \{e_{2,1} \cdot p_1, e_{2,2} \cdot p_2, \dots, e_{2,r} \cdot p_r\}_M，$$

$\dots$ ，

$$F_{N_k}^{q_k} = \{e_{k,1} \cdot p_1, e_{k,2} \cdot p_2, \dots, e_{k,r} \cdot p_r\}_M。$$

現以 $F_N^{(q_1, q_2, \dots, q_k)} = (F_{N_1}^{q_1}, F_{N_2}^{q_2}, \dots, F_{N_k}^{q_k})$ 序列

表示第 i 個因數 N_i 之值為由 F_N^D 中選出的 q_i 個

質因數相乘而得，即 $N_i = \prod_{p \in F_{N_i}^{q_i}} p$ ，而且將這 q_i 個

質因數放進第 i 個複集合 $F_{N_i}^{q_i}$ 。雖然

$F_N^{(q_1, q_2, \dots, q_k)} = (F_{N_1}^{q_1}, F_{N_2}^{q_2}, \dots, F_{N_k}^{q_k})$ 表示一由 k 個

複集合所形成之序列，但因其意義為此 k 個因數的複集合而形成的乘積 N ，即

$$N = N_1 N_2 \cdots N_k$$

$$= \left(\prod_{p \in F_{N_1}^{q_1}} p \right) \left(\prod_{p \in F_{N_2}^{q_2}} p \right) \cdots \left(\prod_{p \in F_{N_k}^{q_k}} p \right),$$

當不會發生混淆時此兩種符號將交互使用。由以上符號及觀念，(1) 之唯一質因數分解可寫為

$$F_N^{(1,1,\dots,1)} =$$

$$\left(F_{p_1}^1, \dots, F_{p_1}^1, F_{p_2}^1, \dots, F_{p_2}^1, \dots, F_{p_r}^1, \dots, F_{p_r}^1 \right)^\circ$$

事實上，上式表示 $\mathbf{q} = (q_1, q_2, \dots, q_k)$ 為 D 分解為 k 個值 $q_1, q_2, \dots, q_k$ 之數值分割，當

$$D = q_1 + q_2 + \cdots + q_k$$

時，除了最極端的兩種情形外（ $k=1$ 及 $k=D$ ），通常不只一種分割方法，其分割的數目可由生成函數導出（例 [1]）。若定義

$$\mathfrak{F}_N^{(k)} = \left\{ F_N^{(q_1, q_2, \dots, q_k)} \mid 0 \leq q_i \leq d_i, \sum_{j=1}^k q_j = D \right\}_M$$

表示將 N 寫為由 k 個因數相乘所形成的複集合，欲找出所有因數相乘形式之對應複集合即表示必須找出 $\mathfrak{F}_N^{(1)}$ ， $\mathfrak{F}_N^{(2)}$ ， $\dots$ ， $\mathfrak{F}_N^{(D)}$ 。

至於如何找出每一個 $\mathfrak{F}_N^{(k)}$ ，首先由 F_N^D 中選出 q_1 個質因數以形成第一個因數 N_1 所對應的複集合 $F_{N_1}^{q_1}$ ，此時所有未被選出之質因數形成複集合 $F_N^D \setminus_M F_{N_1}^{q_1}$ ，其中 $\setminus_M$ 表示複集合減法，對於兩複集合形如

$$A = \{a_i \cdot p_i \mid a_i \in N\}$$

及

$$B = \{b_i \cdot p_i \mid b_i \in N\}$$

時，其意義為

$$A \setminus_M B = \{(a_i - b_i) p_i \mid p_i \in A\}_M,$$

此時

$$F_N^D \setminus_M F_{N_1}^{q_1} =$$

$$\{(d_1 - e_{1,1}) p_1, (d_2 - e_{2,1}) p_2, \dots, (d_r - e_{r,1}) p_r\}_M$$

由於 $F_{N_1}^{q_1}$ 為 F_N^D 之複子集合（multi-subset），第

二個因數 N_2 所對應的複集合 $F_{N_2}^{q_2}$ 由 $F_N^D \setminus_M F_{N_1}^{q_1}$

選出 q_2 個元素形成；同理，依此類推可以得到由 $N_2, N_3, \dots, N_k$ 所有的因數分解之對應複子

集合 $F_{N_3}^{q_3}, \dots, F_{N_k}^{q_k}$ ，或此組因數複集合之等價

形式， $F_N^{(q_1, q_2, \dots, q_k)} = (F_{N_1}^{q_1}, F_{N_2}^{q_2}, \dots, F_{N_k}^{q_k})$ 。

5. 演算法

1. 質因數分解 $N = p_1^{d_1} \cdot p_2^{d_2} \cdots p_r^{d_r}$ 。

2. 令 $D = d_1 + d_2 + \cdots + d_r$ 。

3. for $k = 1$ to D

A. 對 D 作 k -數值分割，

B. 求出

$$\mathfrak{F}_N^{(k)} = \left\{ F_N^{(q_1, q_2, \dots, q_k)} \mid 0 \leq q_i \leq d_i, \sum_{j=1}^k q_j = D \right\}_M$$

endfor

4. 將所有 $\mathfrak{F}_N^{(k)}$ （ $1 \leq k \leq D$ ）中的每一元素相乘即得到該因數分解之組合。

6. 實例說明

以求 36 之所有因數相乘形式為例：

1. 質因數分解 $36 = 2 \cdot 2 \cdot 3 \cdot 3$ 。

2. $D = 4$ 。而 4 的數值分割為

$$4 = \overset{(k=2)}{3+1} = \overset{(k=2)}{2+2} = \overset{(k=3)}{2+1+1} = \overset{(k=4)}{1+1+1+1}$$

共有 4 種。

$$3. \quad k=1, \quad \mathfrak{F}_{36}^{(1)} = \left\{ \{2, 2, 3, 3\}_M \right\}_M$$

$$k=2,$$

$$\begin{aligned} \mathfrak{F}_{36}^{(2)} &= \left\{ F_{36}^{(3,1)}, F_{36}^{(2,2)} \right\}_M \\ &= \left\{ \underbrace{\{12, 3\}_M, \{18, 2\}_M}_{(3,1)}, \underbrace{\{6, 6\}_M, \{4, 9\}_M}_{(2,2)} \right\}_M \end{aligned}$$

$$k=3,$$

$$\begin{aligned} \mathfrak{F}_{36}^{(3)} &= \left\{ F_{36}^{(2,1,1)} \right\}_M \\ &= \left\{ \underbrace{\{2 \times 2, 3, 3\}_M, \{2 \times 3, 3, 2\}_M, \{3 \times 3, 2, 2\}_M}_{(2,1,1)} \right\}_M \end{aligned}$$

4. 36 所有的因數相乘形式

$$\begin{aligned} &36 \\ &= 36 \\ &= 13 \times 3 = 18 \times 2 = 6 \times 6 = 4 \times 9 \\ &= 4 \times 3 \times 3 = 6 \times 3 \times 2 = 9 \times 2 \times 2 \\ &= 2 \times 2 \times 3 \times 3 \end{aligned}$$

7. 結論

本文引進複集合的概念，發展出可求出給定正整數所有因數相乘形式的演算法，由於可以用程式語言實作，目前已以此演算法得到由 2 到 10000 之整數的因數相乘形式的資料庫，可從事關於因數數量（非僅質數個數）的研究。

8. 參考資料

- [1] J. A. Anderson, "Discrete mathematics with combinatorics", Prentice-Hall, 2001.
- [2] Albert Nijenhuis and Herbert S. Wilf, "Combinatorial algorithms", New York, Academic Press, 1975.
- [3] R. P. Bogart, "Introductory combinatorics", 3rd ed., Academic Press, 2000.

植基於超增序列的單伺服器私密資訊擷取機制

陳俊華^{1&2}

¹ 中興大學資訊科學所 台中市國光路 250 號

² 建國技術學院電子系 彰化市介壽北路 1 號

godsons@ckit.edu.tw

摘要

在電子商務(e-commerce)的環境中要保護使用者的隱私權,使伺服器也無法得知使用者所擷取的資訊為何,一般認為這是不可能的。直到私密的資訊擷取(Private Information Retrieval: PIR)的問題被提出並加以解決。一個私密的資訊擷取機制允許使用者從線上伺服器的資料庫中擷取資料項,但卻使伺服器無法得知使用者查詢的是何項資料。PIR 自 1995 年由 Chor 等人提出後受到相當的重視,後續也有不少相關的研究。

在本論文中,我們提出一個植基於超增序列(Superincreasing Sequence)的單伺服器的私密資訊擷取(PIR)機制,其安全性為資訊理論上的安全。相對於之前其它的 PIR 機制,我們所提出的機制較簡單且更適用於電子商務的環境中。另外,本篇論文也包含安全性的證明(provable secure)及與其他 PIR 機制的比較。

關鍵詞：電子商務,使用者隱私權保護,私密的資訊擷取,超增序列

一、導論

在電子商務中線上查詢、擷取有價資訊的環境中,有許多的情況使用者(或顧客)的隱私性需要被保護;換句話說,當使用者線上查詢付費資訊時,會希望資料庫伺服器(Database Server)並不知道使用者所查詢的是哪幾筆資訊;以下茲就幾例加以說明之[12]:

(一) 專利資料庫:

如果專利資料庫的伺服器知道,線上使用者查詢的意欲為何,這將導致許多的問題。我們想像若有一個科學家發現了一個重要的公式,例如「 $H_2 + O_2 \Rightarrow H_2O$ 」,很自然的他想去申請專利(這發現可用於沙漠中造水等許多有市場價值的應用)。但首先他需要去線上專利資料庫查詢,是否已存在這樣的專利或類似的專利。但此時該專利資料庫伺服器管理者,已於該科學家查詢時得知:該科學家有將「 $H_2 + O_2 \Rightarrow H_2O$ 」申請為專利的意圖,這立刻給該伺服器管理者有以下想法:(i)知道該科學家正在作這方面的研究(ii)「 $H_2 + O_2 \Rightarrow H_2O$ 」可能是新的發明,為什麼不能比該科學家早一步申請該專利。以上兩項想法都是非常有價值的情報,該伺服器管理者,大可待價而沽、大發利市了。由此例我們知道,使用者查詢的隱私權應當被保護,以免產生以上不合理的現象。

(二) 製藥資料庫:

製藥公司為開發新藥必需去搜集關於藥品的基本成份和它們的特性,通常會有專門的公司提供製藥的資料庫以供查詢。合成一種新藥的過程需要從該資料庫取得該數種基本成份的資訊。為了讓製造新藥的計畫不致外洩,製藥公司往往購買下整個製藥的資料,如果這種線上資料庫的查詢能作到使用者隱私權保護(只知該使用者上線查詢,卻不知使用者所查詢的是哪幾筆資料),則可省去購買整個資料庫的大筆費用,只需購買所需要的幾種基本成份的線上資訊即可。

(三) 線上有價數位商品：

有些數位商品如電子書、音樂檔(例：MP3)、影像檔及視訊影片檔等，使用者直接可由線上查詢、擷取。可能會有不少的使用者會希望在線上購買這些數位產品時保持其隱私性。

有一家最大型的線上多媒體公司宣稱其擁有百萬客戶的資料及其購買嗜好，這也被當成該公司的重要資產。這可能成為一個商場的交易，客戶的資料及其購買嗜好會在未經使用者允許的情形下賣給其它公司[12](由於公司破產或老闆結束營業)；這是不太合理的現象，但也更凸顯出電子商務環境中使用使用者隱私權保護機制的重要性。於下一段落中我們將介紹最基本的雙伺服器私密資訊擷取機制，以便使讀者了解如何做到使用者隱私權保護。

二、基本雙伺服器私密資訊擷取機制

本段落詳細說明先驅者 Chor 等人於[1][2]中所提出之最基本私密資訊擷取機制—雙伺服器私密資訊擷取機制(two-server PIR scheme)，藉著向兩個伺服器作查詢而隱藏了真正查詢的意圖，而藉著重建兩個伺服器所傳回的結果而得到真正使用者所欲查詢的結果。於本段落中我們整理出 k 伺服器 PIR 機制的相關觀念、定義，並以實例說明雙伺服器(k=2)私密資訊擷取機制；最後我們簡單證明該機制的正確性。

(一) 預備觀念：

- n** 索引 $i \in [n] \equiv \{1, 2, 3, \dots, n\}$ (i 是使用者有興趣的位元， n 是資料庫的位元數)
- n** $X \equiv x_1 \dots x_n$ 一個包含於 $\{0, 1\}^n$ 的字串，代表一資料庫
- n** 隨機輸入 r 其長度為 L_{rnd}
- n** 使用者向 k 個伺服器產生 k 個長度為 L_q 的查詢： $Q_1(i, r), \dots, Q_k(i, r)$
- n** k 個伺服器根據於資料庫 X 的全部內容，回應長度為 L_a 的 $A_1, \dots, A_k$

- n** 使用者依據 $A_1, \dots, A_k$ 的值，以重建函數 R 重建有興趣的位元 x_i

(二) k 伺服器 PIR 機制的正式定義：

定義：若資料庫長度為 n ，一個 k 伺服器 PIR 機制(**k-server PIR Scheme**)包含：

- n** k 個查詢函數 $Q_1, \dots, Q_k$ ：
 $[n] \times \{0, 1\}^{L_{rnd}} \rightarrow \{0, 1\}^{L_q}$
- n** k 個回應函數， $A_1, \dots, A_k$ ：
 $\{0, 1\}^n \times \{0, 1\}^{L_q} \rightarrow \{0, 1\}^{L_a}$
- n** 一個重建函數 R ：
 $[n] \times \{0, 1\}^n \times \{0, 1\}^{L_{rnd}} \times (\{0, 1\}^{L_a})^k \rightarrow \{0, 1\}$

以上函數需滿足正確性及隱私性

正確性代表經過使用者重建函數的計算後，確實得到使用者有興趣的位元值 x_i

隱私性代表經過以上程序處理後，對於資料庫伺服器來說，使用者有興趣的位元是 x_i 或是 x_j ($i \neq j$)，其機率是完全一樣的(也就是能保護使用者查詢的隱私權)

正確性：For every $x \in \{0, 1\}^n$,
 $i \in [n]$, and $r \in \{0, 1\}^{L_{rnd}} \rightarrow$
 $R(i, r, A_1(X, Q_1(i, r)), \dots, A_k(X, Q_k(i, r))) = x_i$

隱私性：For every $i, j \in [n]$,
 $s \in [k]$, and $q \in \{0, 1\}^{L_q} \rightarrow$
 $\Pr(Q_s(i, r) = q) = \Pr(Q_s(j, r) = q)$

其中隨機輸入 $r \in \{0, 1\}^{L_{rnd}}$ 是以均勻分佈的機率隨機選取

(三) 會用到的表示法：

- n** U ：使用者
- n** $SRV_1, \dots, SRV_k$ ： k 個伺服器
- n** $X \equiv x_1 \dots x_n$ 一個包含於 $\{0, 1\}^n$ 的字串，代表 k 個伺服器上相同的資料庫
- n** i ：代表使用者有興趣的位元索引
- n** $[m] \equiv \{1, 2, \dots, m\}$
- n** 對於一個集合 S 及一個元素 a
 $S \oplus a \equiv S \setminus \{a\} \quad \text{if } a \notin S$
 $\equiv S \cup \{a\} \quad \text{if } a \in S$

裡不再贅述。

雙伺服器 PIR 機制的執行步驟如下：

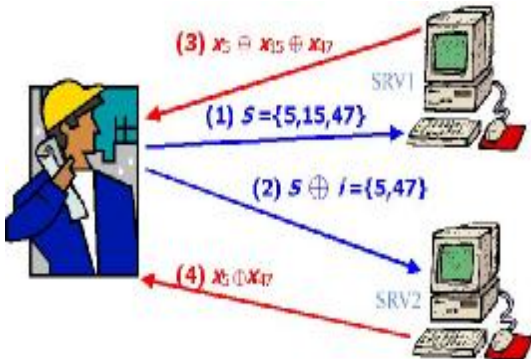
1. 使用者隨機選取一個集合 $S \subseteq [n]$
2. 使用者將 S 送給 SRV1 並且將 $S \oplus i$ 送給 SRV2
3. 每一個伺服器回應一個位元，其值是由 Exclusive OR 所有位元值屬於 S 或屬於 $S \oplus i$ (SRV1 回應 $\bigoplus_{j \in S} x_j$; SRV2 回應 $\bigoplus_{j \in S \oplus i} x_j$)
4. 使用者 Exclusive OR SRV1 回應及 SRV2 回應的值則可萃取出他想查詢的位元值 x_i

雙伺服器 PIR 機制的舉例及簡單證明如下：

使用者隨機選取 $S = \{5, 15, 47\}$

(假定 $n=10000$, index $i = 15$,

$S \oplus i = \{5, 47\}$)



使用者計算想得到的位元 x_i

$= (\text{SRV1 的回應}) \oplus (\text{SRV2 的回應})$

$$= (x_5 \oplus x_{15} \oplus x_{47}) \oplus (x_5 \oplus x_{47})$$

$$= (x_5 \oplus x_5) \oplus (x_{47} \oplus x_{47}) \oplus x_{15}$$

$$= 0 \oplus 0 \oplus x_{15}$$

$$= x_{15}$$

$$(\oplus x \oplus y = y \oplus x)$$

$$x \oplus (y \oplus z) = (x \oplus y) \oplus z = x \oplus y \oplus z$$

$$x_i \oplus 0 = 0 \oplus x_i = x_i$$

以上所討論的機制是雙伺服器私密資訊擷取機制；至於多伺服器($k \geq 2$)的 PIR 機制可參考[1][2]中詳細的闡釋、證明，這

三、 相關研究回顧

一種衡量 PIR 機制的方法為萃取一個位元所需的傳輸複雜度(communication complexity)，Chor 等人已於[2]中證明：單伺服器 PIR 機制其所需的傳輸複雜度為 $O(n)$ ：相當於 download 整個資料庫。雖然可藉著 k 伺服器的 PIR 機制($k \geq 2$)來降低傳輸複雜度，從原來只有一個伺服器要達到 PIR 的 $O(n)$ ，降至 $O(n^{1/k})$ 。但當 database 的 size 非常大時這仍有不小的 overhead；因此有些關於 PIR 的後續研究上[4-7]，都著眼於如何降低其複雜度。

Andris 於[4]證明 k 伺服器的 PIR 機制($k \geq 2$)其傳輸複雜度降為 $O(n^{1/(2k-1)})$ 。Yael Gertner 等人於[7]中提出一個新的 PIR 機制(scheme)，除了原來的資料庫伺服器外，使用多個輔助的伺服器，仍能保持原先 PIR 機制的理論安全性(Information Theoretic Security)；且使用者對資料庫伺服器線上查詢時其傳輸複雜度只需 $O(1)$ ，此機制的優點為不需複製相同資料庫到多個伺服器上，減少了分散式資料庫系統管理上的負擔。但其缺點是此機制所用的伺服器數目是原先 PIR 機制[1][2]的兩倍；且這些輔助伺服器在建置(set up)階段，計算及傳輸所需的複雜度仍高達 $O(n)$ 。

Chor 於[9]中提出 Computational PIR(CPIR)的觀念，降低其在安全性的要求為計算上的安全(Computational Security)，以別於之前的 PIR 均為資訊理論上的安全 (Information Theoretic PIR)。Chor 等人提出兩個資料庫伺服器的 Computational PIR(CPIR)機制，其傳輸複雜度(Communication Complexity)為 $O(n^\epsilon)$ ，對任何的 $\epsilon > 0$ 。Cachin 於[5]中提出改進 CPIR 的機制使其傳輸複雜度降為 $O(k)$ *

$O(\log n)$ ，其中 k 為所選定的機密值；此篇論文採用一個新的安全性假定 Φ -Hiding Assumption，有許多理論和證明的的工作，顯得其機制較複雜、較理論；且其每次查詢一個位元的資料，資料庫伺服器都至少要作 n 個乘冪(exp)及 n 個除餘(mod)的運算，才能將回應給使用者的值計算出來，資料庫伺服器的負擔相當重。我們認為在電子商務的環境中，伺服器能快速回應使用者查詢的重要性，不下於較低的傳輸複雜度。

在多伺服器的機制中(無論是 PIR 或 CPIR)，都有一個安全性的問題亦即伺服器共謀的問題，不論在 PIR 或 CPIR 的機制中若所有伺服器共謀，在收到使用者對 k 個伺服器的查詢資訊後，將此 k 個訊息比對後可得到使用者想查的是資料庫哪一筆資訊，這代表所有伺服器共謀將破解 PIR 或 CPIR 保護使用者隱私權的安全機制。因此 Chor 等人在[2]中，乾脆將伺服器不能共謀作為此安全機制的先決條件。在多伺服器的機制中還有一個問題，因為最後使用者所要得到的資訊，是由所有伺服器的回應計算而得，因此若有伺服器回應的訊息若是錯誤的或無法回應，將造成使用者最後查詢結果的錯誤。有以下兩種情況將產生伺服器回應的問題：(1)惡意的伺服器，故意回應錯誤的訊息。(2)因伺服器當機、網路連線中斷或擁塞造成伺服器無法回應訊息。因此 Beimel 等人於[3]中提出強韌的 PIR 機制(Robust PIR Scheme)以解決以上問題。此篇論文提出一個 k -out-of- L PIR 機制，亦即只要在 L 個伺服器中只要有 k 個伺服器能回應訊息且回應的訊息是正確的，就足以正確的計算出使用者所欲查詢、擷取的資訊。此篇論文所提出的機制雖然是強韌的(Robust)，但其付出的代價就是要用到更多的伺服器($L > k$)。Yang 等人於[10][11]中針對相同的問題提出具容錯能力的(fault tolerance) PIR 機制，並在實際的多伺服器資料庫($k=2\sim 11$)

中作一些簡單的實驗。

根據以上相關的研究顯示，PIR 的研究確實對電子商務環境中使用隱私權的保護有相當關鍵的突破。但從以上研究的分析中我們發現多伺服器的 PIR 機制其伺服器管理的負擔極大，且容易有安全上的漏洞，這會使得 PIR 機制在實務上變得較不可行。因此，我們於本篇論文中提出一個實務上較可行的單伺服器私密資訊擷取(PIR)機制。

四、我們所提出的單伺服器私密資訊擷取機制

(一) 預備觀念

- n $i \in [n] \equiv \{1, 2, 3, \dots, n\}$ (i 是使用者有興趣的位元編號， n 是資料庫的位元數)
- n $X \equiv x_1 \dots x_n$ 一個包含於 $\{0, 1\}^n$ 的字串，代表一資料庫
- n 使用者以 $Q(i)$ 產生查詢並送給資料庫伺服器
- n 資料庫伺服器根據於資料庫 X 的全部內容，以 $A(X, Q(i))$ 產生回應給使用者
- n 使用者以 $R(i, Q(i), A(X, Q(i)))$ 重建有興趣的位元值 x_i
- n 迷袋問題(Knapsack Problem)：指在給定的自然序列 $B = (b_1, b_2, \dots, b_n)$ 及一數 S 是否存在一子序列 $B' \subseteq B$ ，其中 $B' = (b_1', \dots, b_m')$ ，使得 $\sum_{i=1}^m b_i' = S$ 。這個問題已被證實是一個 NP-Complete 問題[8]。但並非所有的迷袋問題都是 NP-Complete 問題，在 B 為某些特定的序列的情況下(如超增序列)，迷袋問題成為易解的問題。
- n 超增序列(Superincreasing Sequence)
遞增序列 $B = (b_1, b_2, \dots, b_n)$ ，如果滿足下面的條件，就稱 B 為超增序列：
 $b_j > \sum_{i=1}^{j-1} b_i, j > 1$ 。若 B 為滿足上述定義的序列，且向量 $X = (x_1, \dots, x_n), x_i \in \{0, 1\}$ ，使得

$S = X \times B^T = \sum_{i=1}^n b_i x_i$ ，則由下列的演算法，在已知 S 及 B 的情況下很容易求出 X 。

n 超增序列演算法：

假定 $M = (m_1, m_2, \dots, m_n)$ ， $m_i \in \{0, 1\}$

For $i := n$ downto 1 do

Begin

If $S \geq b_i$ then $m_i := 1$

else $m_i := 0$;

$S := S - (b_i \times m_i)$;

End

If $S = 0$ then $X := M$

else “no solution exists”

(二) 單伺服器私密資訊擷取機制的定義

單伺服器的私密資訊擷取機制，對於大小為 n 個位元的資料庫包含一個查詢函數 $Q(i)$ ，一個回應函數 $A(Q(i))$ ，和一個重建函數

$R(i, Q(i), A(X, Q(i)))$

這些函數 (Q, A, R) 必須滿足以下特性：

正確性： For every $X \in \{0, 1\}^n$,

$i \in [n] \Rightarrow$

$R(i, Q(i), A(X, Q(i))) = x_i$

隱私性： For every $i, j \in [n] \Rightarrow$

$\Pr(Q(i) = q) = \Pr(Q(j) = q)$

(三) 機制的描述

1. 前置處理：

(1) 使用者隨機選取一個超增序列 $B =$

$(b_1, b_2, \dots, b_n)$ ，及兩整數 W, M 滿足：

$M > \sum_{i=1}^n b_i$ 且 $\gcd(W, M) = 1$

(2) 使用者將超增序列轉換成擬亂序列

$A = (a_1, a_2, \dots, a_n)$ ，其中 a_i 滿足

$a_i = (b_i \times W) \bmod M$

(3) 使用者將扮演類似公開金鑰的角色的

序列 A 連同使用者 ID 上傳給資料庫伺服器，以作為一段時期保護使用者隱私權之用。

2. 查詢時的處理：

(1) 使用者向資料庫伺服器傳遞使用者

ID，以表達欲查詢的意願；換句話說

$Q(i)$ 就是使用者 ID

(2) 資料庫伺服器計算 $A(X, Q(i))$ 的結果

並將其值傳回給使用者

(a) 伺服器依據使用者 ID 找出對應的

序列 $A = (a_1, a_2, \dots, a_n)$

(b) 計算 $S' = \sum_{i=1}^n a_i x_i$

(x_i 代表資料庫 X 編號 i 的位元值)

(c) $A(X, Q(i)) \leftarrow S'$

3. 使用者藉著計算 $R(i, Q(i), A(X, Q(i)))$ 的

值，可萃取出所欲查詢的位元值 x_i

(1) 根據 $A(X, Q(i)) : S'$ 及前置處理選取的

W, M 來計算 $S : \sum_{i=1}^n b_i x_i$

$S = (S' \times W^{-1}) \bmod M$

(2) 根據 i 值及前面所述的超增序列演算

法可萃取出 x_i 的值

雖然我們所提出的機制每次查詢可得到一個位元的值，像其他的 PIR 機制一樣；但 Chor 等人已證明[1]這是可以推廣至一個區塊(亦即一筆紀錄)的。

五、 機制正確性及安全性的證明

定理：我們所提出的私密資訊擷取機制是單伺服器的私密資訊擷取機制；並且在保護使用者的隱私權上具有資訊理論上的安全性。

證明：

1. 符合單伺服器 PIR 機制的定義形式：

在我們的機制中我們擁有一個查詢函數 $Q(i)$ ，一個回應函數 $A(Q(i))$ 和一個重建函數 $R(i, Q(i), A(X, Q(i)))$ ；因而符合單伺服器 PIR 機制的定義形式。

2. 滿足正確性：

因為使用者是以收到的 S' 來乘以 W^{-1} 並 $\bmod M$ 來求得超增序列 S

$$\begin{aligned}
&= (S' \times W^{-1}) \bmod M \\
&= (\sum_{i=1}^n b_i x_i W W^{-1}) \bmod M \\
&\quad (\because a_i = (b_i \times W) \bmod M) \\
&= (\sum_{i=1}^n b_i x_i) \bmod M \\
&= S
\end{aligned}$$

我們再依據超增序列的特性及其演算
我們可萃取出正確的 x_i 位元值。

3. 滿足隱私性：(資訊理論上的安全性)

由於 $Q(i)$ 的值為使用者 ID 值，因此
 $Q(i)$ 的值與 i 的值是完全不相關。

$$\therefore \text{For every } i, j \in [n] \quad \mathbb{E}$$

$$\Pr(Q(i) = q) = \Pr(Q(j) = q)$$

並且資料庫伺服器僅從使用者得到與
 i 的值完全無關的使用者 ID 值，之後
判斷資料庫的每一個位元值是否為 1
以決定是否將 a_i 加總至 S' ，亦即資
料庫中的每個位元均以完全相同的方式
對待，所以完全不可能洩漏出 i 的
值；因此在保護使用者隱私權上具有
資訊理論上的安全性。

綜合以上證明，我們可說我們的私密資訊擷
取機制是具有資訊理論上的安全性單伺服
器私密資訊擷取機制；以上定理得證。

六、 討論

誠如我們於段落三相關研究回顧中所提
及的，在 k 伺服器的 PIR 機制中對於伺服器
的管理有極大的負擔，既要保持伺服器上資
料庫的一致性，更要避免其共謀；因此在電
子商務的環境中可行性並不高。我們所提出
的機制為單伺服器 PIR 機制，且我們的機制
於每次使用者查詢時資料庫伺服器只需作 n 次
加法運算比[5](需作 n 次乘冪及 n 次除餘)的
機制簡單許多，因此可行性較高；我們將相
關的 PIR 機制的比較整理如下表。

表 一：本論文的單伺服器 PIR 機制
與其它 PIR 機制的比較

比較 機制	安全性	使用單伺 服器或 多伺服器	實際 可行性
[1,2]中的 PIR 機制	資訊理論上 的安全性	k 伺服器	低
[3]中的 Robust PIR 機制	資訊理論上 的安全性	L 伺服器 ($L > k$)	低
[5]中的 CPIR 機制	計算上的 安全性	單伺服器	中
植基於超增 序列的 單伺 服器機制	資訊理論上 的安全性	單伺服器	高

七、 結論

在本篇論文中我們提出在電子商務環
境中較具有可行性之植基於超增序列的單
伺服器的私密資訊擷取機制。此機制避免
了管理多伺服器的龐大負擔(包含了使多
個資料庫一致及避免多伺服器的共謀);且
具有資訊理論上的安全性。

參考文獻

- [1] Benny Chor, Oded Goldreich, Eyal Kushilevitz and Madhu Sudan. Private Information Retrieval. *Proc. of the 36th IEEE Symposium on Foundations of Computer Science (FOCS)36th*, pp. 41-50, 1995.
- [2] Benny Chor, Oded Goldreich, Eyal Kushilevitz and Madhu Sudan. Private Information Retrieval. *Journal of ACM*, 45:965-981, 1998.
- [3] A. Beimel and Y. Stahl. Robust Information-Theoretic Private Information Retrieval. *Proc. of the 3rd Conference on Security in Communication Networks*, vol. 2576 of

- Lecture Notes in Computer Science*, pp. 326-341, 2002.
- [4] Andris Ambainis. Upper bound on the communication complexity of private information retrieval. *Proc. of 24th ICALP 97, Springer, Lecture Notes in Computer Science*, vol. 1256, pp. 401-407, 1997.
 - [5] Christian Cachin, Silvio Micali, and Markus Stadler. Computationally Private Information Retrieval With poly-logarithmic Communication. *Eurocrypt 99, Vol. 1592 of Lecture Notes in Computer Science*, pp. 402-414, 1999.
 - [6] A. Beimel, Y. Ishai, and T. Malkin. Reducing the Servers Computation in Private Information Retrieval: PIR with Preprocessing. *CRYPTO 2000, vol. 1880 of Lecture Notes in Computer Science*, pp. 56-74, 2000. Journal version to appear in *Journal of Cryptology* [Updated June 2003].
 - [7] Yael Gertner, Shafi Goldwasser, and Tal Malkin. A Random Server Model for PIR. *RANDOM 98, vol. 1518 of Lecture Notes in Computer Science*, pp. 200-217, 1998.
 - [8] M. R. Garey and D. S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. *W.H. Freeman Co., San Francisco*, 1976.
 - [9] B. Chor and N. Gilboa. Computationally private information retrieval. *Proc. of 29th STOC*, 1997.
 - [10] Erica Y. Yang, Jie Xu and Keith H. Bennett. Private Information Retrieval in the Presence of Malicious Failures. *Proceedings of the 26 th IEEE Annual International Computer Software and Applications Conference (COMPSAC '02)*, 0730-3157/02.
 - [11] Erica Y. Yang, Jie Xu and Keith H. Bennett. A Fault-Tolerant Approach to Secure Private Retrieval. *IEEE 2002*, 1060-9857/02, pp. 12~21.
 - [12] Dmitri Asonov. Private Information Retrieval - An overview and current trends. *Proceedings of the ECDPvA Workshop, Informatik 2001, Vienna, Austria, September 2001*.

An Efficient Parallel-Pipelined Algorithm and Architecture for Computation of 2-D DCT on Two Successive Processors

Tze-Yun Sung Yi-Hsun Sung Chih-Sin Chen

Electrical Engineering Department

Chung Hua University

Hsin-chu, Taiwan 300-12

Tel : 886-3-579-0902

Fax: 886-3-578-1150

E-mail : bobsung@btcx.com.tw, tom@btcx.com.tw, sin@btcx.com.tw

Abstract

2-D Discrete Cosine Transform (DCT) applies on image compression and saves more memories. In this paper, we use row-column decomposition method, and propose a parallel-pipelined architecture to implement a 8×8 DCT processor. This processor involves two 8-point DCT processors, two SRAMs (64 words), twelve multiplexers, sixteen registers, two timing controllers and counters. The complex arithmetic unit (AU) of DCT processor is designed by CORDIC arithmetic. This processor saves hardware and achieves high performance. The chip of 8×8 DCT processor is fabricated in a $0.25 \mu\text{m}$ CMOS. All control signals are generated internally on-chip. The chip has been implemented and applied on image compression successfully.

Keywords: DCT, row-column decomposition method, parallel-pipelined, CORDIC arithmetic, image compression.

1. Introduction

For the application of image data compression, a transform encoding technique is more suitable than that of a linear prediction coding (LPC). It is clear

that the Walsh-Hadamard transform is the simplest one for implementation. The kernel matrix of the Walsh-Hadamard transform involves only additions and subtractions [1]. However, the Walsh-Hadamard transform is definitely not the best transform encoder due to the absence of symmetry property of the transform kernel. Currently, the Discrete Cosine Transform (DCT) becomes the best transform encoder because of that the transform kernel of the DCT involves both the periodicity and symmetry properties, and therefore the DCT can perform very close to the theoretically optimal Karhunen-Loeve transform [2]. Conventionally, a $2N$ -point DCT or the double size Fast Fourier Transform (FFT) computation algorithm was employed to the implementation of DCT involving complex arithmetic [1], where the processor units as well as the computation time were $2N$ and $3N$, respectively required for the implementation of DFT algorithm, and were $2\log 2N$ and $O(\log 2N + 1)$, respectively required for the implementation of FFT algorithm. Some fast computation algorithms were proposed and discussed [3]-[11], and the VLSI chips implementations of some DCT concurrent architectures were accomplished for real-time applications [12]-[14].

In this paper, a parallel-pipelined computation

approach for the DCT implementation will be investigated. In section II, basic considerations for 2-D DCT algorithm will be described. In section III, both the parallel-pipelined 1-D and 2-D DCT architectures are presented. In section IV, the 8×8 2-D DCT architectures are developed, the chip of 8×8 2-D DCT processor is implemented, and comparisons of different architectures and chips will also be discussed. Finally, conclusions will be given.

2. 2-D Discrete Cosine Transform (DCT) Algorithm

The N -point 1-D DCT is defined as

$$Y(m) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \sqrt{2} K_m \cos\left[\frac{(2n+1)m\pi}{2N}\right] \cdot x(n), \quad (1)$$

where $m = 0, \dots, N-1$,

$$K_m = \frac{1}{\sqrt{2}}, \text{ where } m = 0,$$

$$1, \text{ where } m > 0.$$

The $M \times N$ -point DCT is defined as

$$Z(u, v) = \frac{2 \cdot c(u)c(v)}{\sqrt{M \cdot N}} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x(m, n) \cdot \cos\left[\frac{(2m+1)u\pi}{2M}\right] \cdot \cos\left[\frac{(2n+1)v\pi}{2N}\right], \quad (2)$$

where $u = 0, \dots, M-1, v = 0, \dots, N-1$,

$$c(k) = \frac{1}{\sqrt{2}}, \text{ where } k = 0,$$

$$1, \text{ where } k > 0.$$

According to eq. (2), we have

$$Z(u, v) = \frac{1}{\sqrt{M}} \sum_{m=0}^{M-1} \sqrt{2} c(u) \cdot \cos\left[\frac{(2m+1)u\pi}{2M}\right] \cdot \left\{ \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \sqrt{2} c(v) \cdot \cos\left[\frac{(2n+1)v\pi}{2N}\right] \cdot x(m, n) \right\}, \quad (3)$$

Where $u = 0, \dots, M-1, v = 0, \dots, N-1$,

$$c(k) = \frac{1}{\sqrt{2}}, \text{ where } k = 0,$$

$$1, \text{ where } k > 0.$$

We use row-column decomposition method, and then 8-point 1-D DCT can be represented as

$$\begin{bmatrix} Y(0) \\ Y(1) \\ Y(2) \\ Y(3) \\ Y(4) \\ Y(5) \\ Y(6) \\ Y(7) \end{bmatrix} = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ a & c & d & f & -f & -d & -c & -a \\ b & e & -e & -b & -b & -e & e & b \\ c & -f & -a & -d & d & a & f & -c \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ d & -a & f & c & -c & -f & a & -d \\ e & -b & b & -e & -e & b & -b & e \\ f & -d & c & -a & a & -c & d & -f \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \\ x(7) \end{bmatrix}, \quad (4)$$

where

$$a = \sqrt{2} \cos\left(\frac{\pi}{16}\right), \quad b = \sqrt{2} \cos\left(\frac{\pi}{8}\right), \quad c = \sqrt{2} \cos\left(\frac{3\pi}{16}\right),$$

$$d = \sqrt{2} \cos\left(\frac{5\pi}{16}\right), \quad e = \sqrt{2} \cos\left(\frac{3\pi}{8}\right), \quad f = \sqrt{2} \cos\left(\frac{7\pi}{16}\right).$$

Let

$$\mathbf{Y} = [Y(0) \ Y(1) \ Y(2) \ Y(3) \ Y(4) \ Y(5) \ Y(6) \ Y(7)]$$

and

$$\mathbf{X} = [x(0) \ x(1) \ x(2) \ x(3) \ x(4) \ x(5) \ x(6) \ x(7)],$$

and eq. (4) can be represented as

$$\mathbf{Y} = \frac{1}{\sqrt{8}} \mathbf{C} \mathbf{X}^t = \mathbf{T} \mathbf{X}^t, \text{ where } \frac{1}{\sqrt{8}} \mathbf{C} = \mathbf{T},$$

$$\mathbf{C} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ a & c & d & f & -f & -d & -c & -a \\ b & e & -e & -b & -b & -e & e & b \\ c & -f & -a & -d & d & a & f & -c \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ d & -a & f & c & -c & -f & a & -d \\ e & -b & b & -e & -e & b & -b & e \\ f & -d & c & -a & a & -c & d & -f \end{bmatrix}.$$

According to eq. (3), we have

$$\mathbf{Z} = \mathbf{T} \mathbf{X} \mathbf{T}^t = \mathbf{T} (\mathbf{T} \mathbf{X}^t)^t = \mathbf{T} \mathbf{Y}^t. \quad (5)$$

Hence,

$$2\text{-D DCT}(\mathbf{X}) = 1\text{-D DCT}((1\text{-D DCT}(\mathbf{X}))^t).$$

3. A Parallel-Pipelined 2-D DCT

Architecture

According to eq. (4), the N -point 1-D DCT needs N -time unit, and we have

$$Y_n(m) = Y_{n-1}(m) + \mu_{m,n-1} \cdot x(n-1),$$

where μ are coefficients in transform matrix (T), n is an n -th time clock, and $Y_0(m) = 0$.

We use eq. (5) and propose the parallel-pipelined $N \times N$ 2-D DCT architecture which is shown in Fig. 1. This architecture possesses two-transposition memory, two-1-D N -point DCT processor, de-multiplexer, multiplexer, system controller and interface. N -point 1-D DCT input-processor writes results to transposition memory-1 and transposition memory-2 alternately. N -point 1-D DCT output-processor reads data from transposition memory-1 and transposition memory-2 alternately, and output the results. The time/space due to parallel-pipelined $N \times N$ 2-D DCT architecture is shown in Fig. 2.

4.Implementation of 2-D Parallel-Pipelined DCT Processor

• 1-D DCT Processor Implementation

At first, the 8-point 1-D DCT processor would be introduced; the 1-D DCT processor has complex arithmetic units (AUs), which can be accomplished by CORDIC arithmetic [15], [16]. The 1-D 8-point DCT processor is shown in Fig. 3.

The T -transform matrix of 16-point DCT is represented as

$$\mathbf{T} = \frac{1}{\sqrt{16}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ g & h & i & j & k & l & m & n & -n & -m & -l & -k & -j & -i & -h & -g \\ a & c & d & f & -f & -d & -c & -a & -a & -c & -d & -f & f & d & c & a \\ h & k & n & -l & -i & -g & -j & -m & m & j & g & i & l & -n & -k & -h \\ b & e & -e & -b & -b & -e & e & b & b & e & -e & -b & -b & -e & e & b \\ i & n & -j & -h & -m & k & g & l & -l & -g & -k & m & h & j & -n & -i \\ c & -f & -a & -d & d & a & f & -c & -c & f & a & d & -d & -a & -f & c \\ j & -l & -h & n & g & m & -i & -k & k & i & -m & -g & -n & h & l & -j \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ k & -i & -m & g & -n & -h & l & j & -j & -l & h & n & -g & m & i & -k \\ d & -a & f & c & -c & -f & a & -d & -d & a & -f & -c & c & f & -a & d \\ l & -g & k & m & -h & j & n & -i & i & -n & -j & h & -m & -k & g & -l \\ e & -b & b & -e & -e & b & -b & e & e & -b & b & -e & -e & b & -b & e \\ m & -j & g & -i & l & n & -k & h & -h & k & -n & -l & i & -g & j & m \\ f & -d & c & -a & a & -c & d & -f & -f & d & -c & a & -a & c & -d & f \\ n & -m & l & -k & j & -i & h & -g & g & -h & i & -j & k & -l & m & -n \end{bmatrix}, \quad (6)$$

where

$$\begin{aligned} g &= \sqrt{2} \cos\left(\frac{\pi}{32}\right), h = \sqrt{2} \cos\left(\frac{3\pi}{32}\right), i = \sqrt{2} \cos\left(\frac{5\pi}{32}\right), \\ j &= \sqrt{2} \cos\left(\frac{7\pi}{32}\right), k = \sqrt{2} \cos\left(\frac{9\pi}{32}\right), l = \sqrt{2} \cos\left(\frac{11\pi}{32}\right), \\ m &= \sqrt{2} \cos\left(\frac{13\pi}{32}\right), n = \sqrt{2} \cos\left(\frac{15\pi}{32}\right). \end{aligned}$$

The 16-point DCT processor is also exploited by cascading an expanding module, which is shown in Fig. 4. This architecture has high-regularity and scalability.

• Chip Implementation of 8×8 2-D DCT Processor

8×8 2-D DCT processor involves two 1-D DCT processors, timing controller/counter, multiplexers and two-SRAM (64 words). The architecture of 8×8 2-D DCT processor is shown in Fig. 4. The chip layout of 8×8 2-D DCT processor is shown in Fig. 5. The chip is fabricated in a 0.25 μ m CMOS, double-metal technology. All control signals are generated internally on-chip [17].

• Comparisons of Different 8×8 2-D DCT Architectures and Chips

Some high-performance implementations employ highly parallel approaches with high arithmetic cost. In this work, we focus on parallel-pipelined dedicated 2-D DCT architectures with a throughput rate of at least one output sample per cycle, targeted for video compression. In Table I, we select some typical proposed architecture and make the comparison [8]-[10]. In Table II, we list features of other typical DCT/IDCT chips. They still incurs large gate count requirement [11]-[14].

5. Conclusions

Based on the symmetry property considerations and row-column decomposition method, a 1-D DCT

computation algorithm has been proposed implemented into VLSI parallel-pipelined architecture. In this 1-D DCT processor, the CORDIC arithmetic were used to for the implementation of the DCT AU so as to perform complex multiply function, and there are N -AU required for computation of a frame of N -point data. Further, a 2-D DCT processor has been proposed in accordance with the 2-D DCT computation decomposed into two successive 1-D DCT kernels. These architectures avoid the complex shuffle implementations, are of trade-off designs and controlled by MCU, hence they are more scalable, flexible and efficient, and therefore very suitable for VLSI implementation.

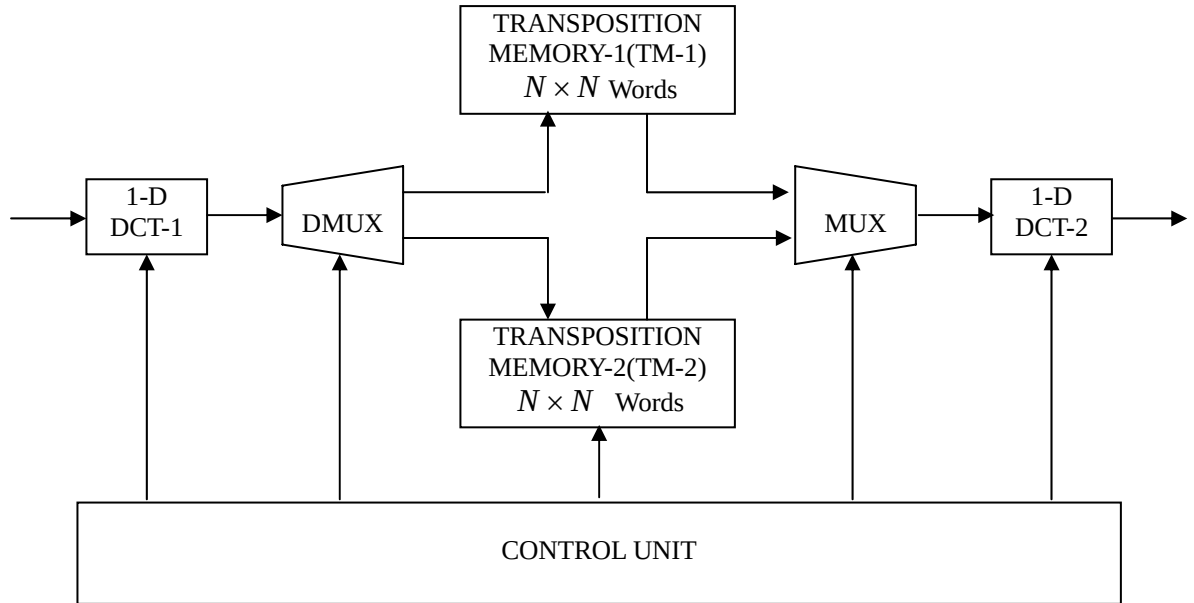
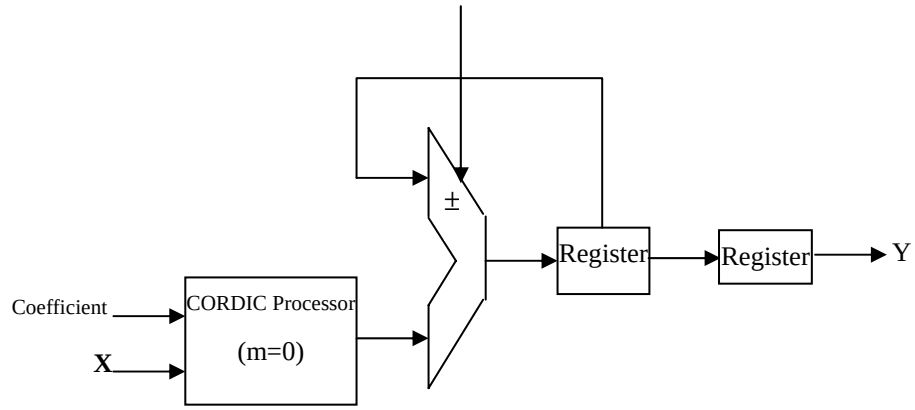


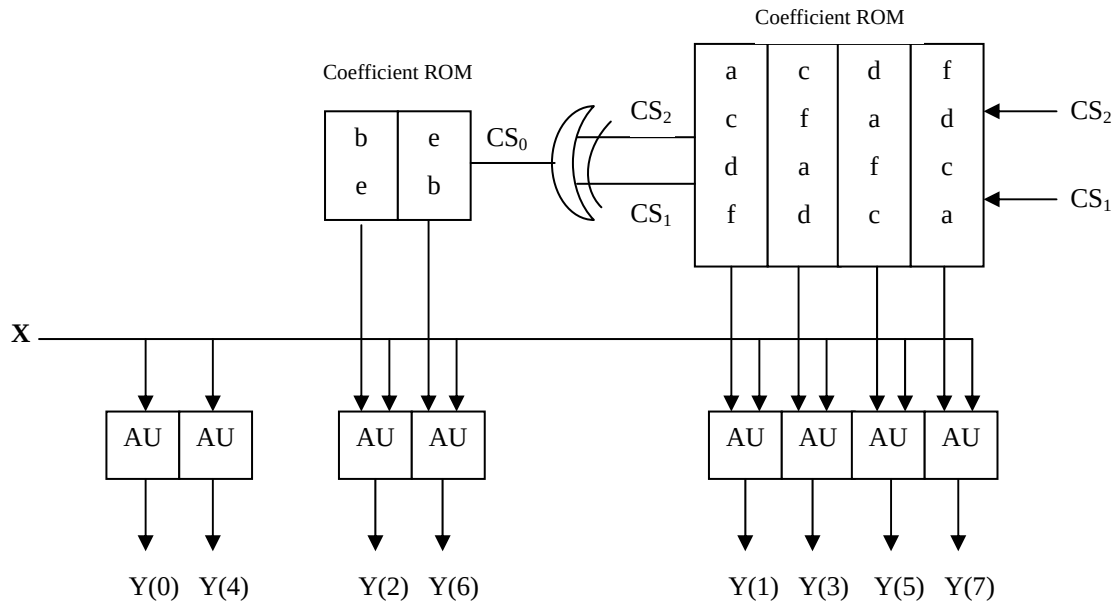
Fig.1 Parallel-pipelined $N \times N$ 2-D DCT architecture

Time Unit (t)	1-D DCT-1(P1)	TM-1	TM-2	1-D DCT-2(P2)
$t=1 \sim N$	active	P1 writes	---	---
$t=N+1 \sim 2N$	active	P2 reads	P1 writes	active
$t=2N+1 \sim 3N$	active	P1 writes	P2 reads	active
$t=3N+1 \sim 4N$	active	P2 reads	P1 writes	active
$t=4N+1 \sim 5N$	active	P1 writes	P2 reads	active

Fig. 2 Time/space due to parallel-pipelined $N \times N$ 2-D DCT architecture



(a) Complex arithmetic unit (AU) of DCT processor(X : complex number)



(b) 8-point 1-D DCT processor (X : complex number)

t	CS_2	CS_1	CS_0
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0
5	1	1	0
6	1	0	1
7	0	1	1
8	0	0	0

Fig. 3 8-point 1-D DCT Processor and sequence of CS_n

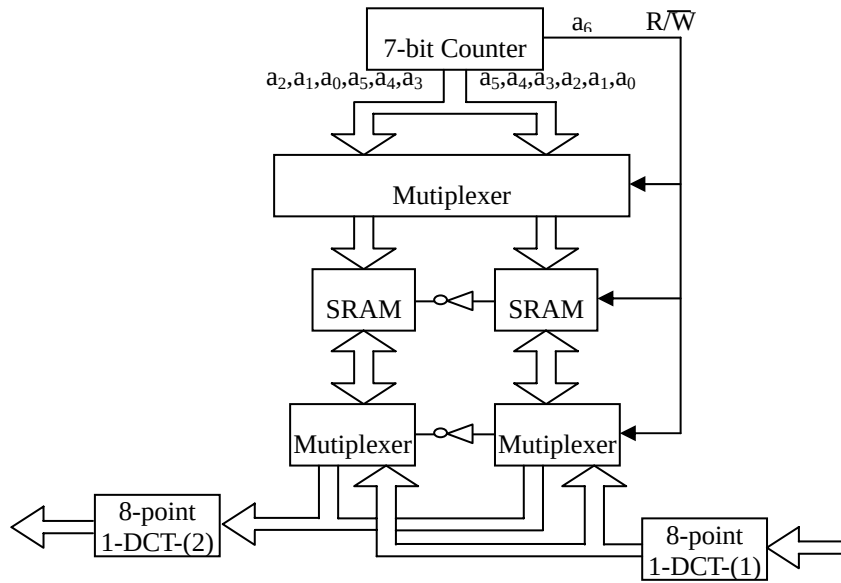


Fig. 4 The 8×8 2-D DCT processor (SRAM: 64 words)

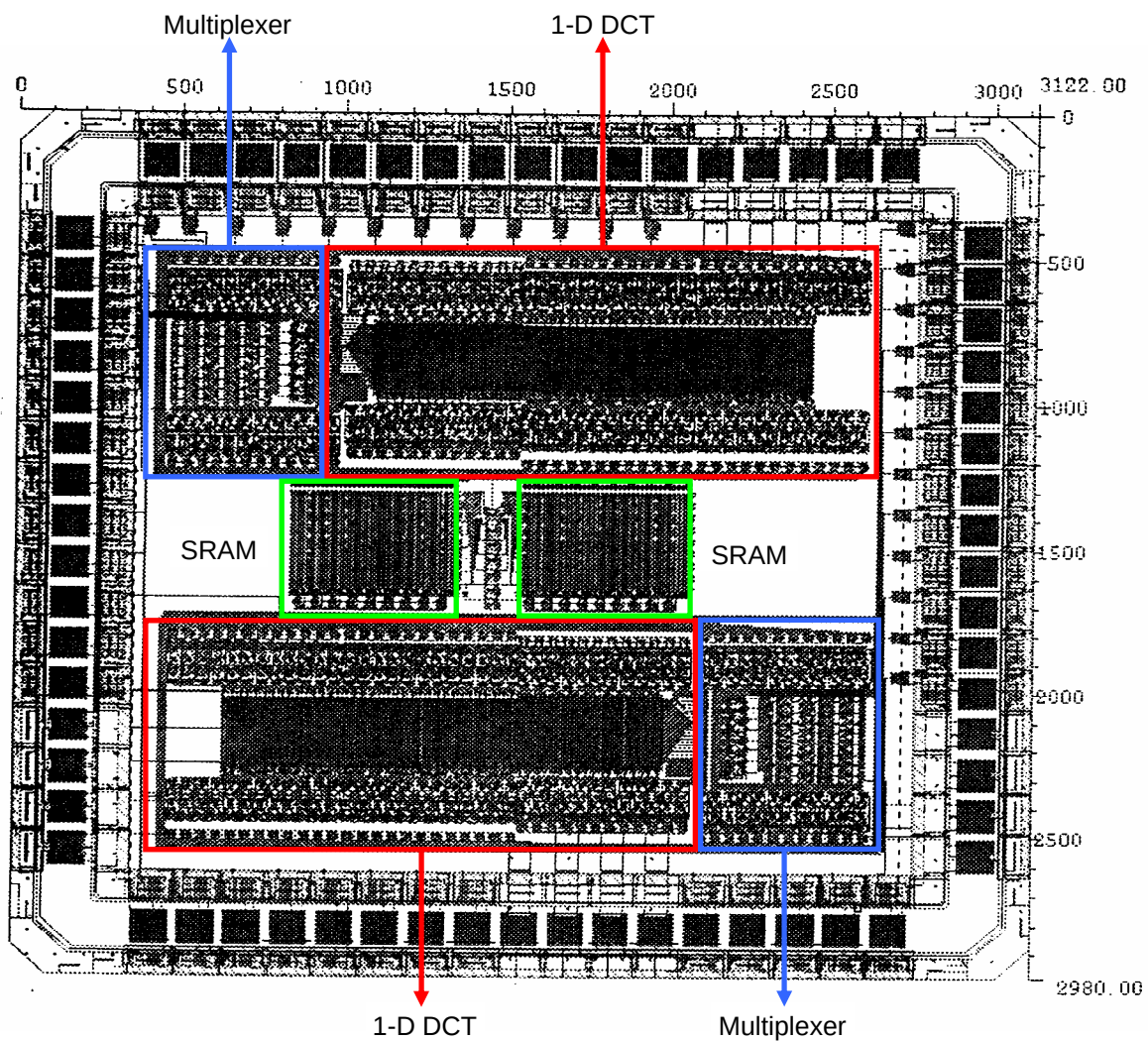


Fig. 5 The chip layout of 8×8 2-D DCT processor

8×8 2-D DCT	Lee [8]	Chang [9]	Hsiao [10]	Sung
Real-multipliers	28	64	-	-
Real-adders	134	88	-	96
Complex-multipliers	-	-	3	-
Complex-adders	-	-	9	-
Memory (Words)	~384	~200	~370	~168
Hardware complexity (AUs)	$O(N\log N)$	$O(N^2)$	$O(\log N)$	$O(2N)$
Throughput (outputs/cycle)	16	8	2	8
Scalability	poor	good	good	better

Table I Comparison of different 8×8 2-D DCT architectures

8×8 DCT	Srinivasan[11]	Kuroda[12]	Rambaldi[13]	Chen[14]	Sung
Approach	Row-column method	Row-column method	Row-column method	Direct method	Row-column decomposition method
Algorithm	Time-recursive	Variable threshold voltage scheme	14-multiplier Gate-array	Folding strategy	Folding-frame strategy
Throughput	1 pixel/cycle	1 pixel/cycle	1 pixel/cycle	1 pixel/cycle	1 pixel/cycle
Latency	72 clocks	112 clocks	80 clocks	86 clocks	64 clocks

Table II Comparison of different 8×8 2-D DCT chips

References

- [1] Elliott, D. F., Kao K. R., "Fast Transforms Algorithms, Analysis, Applications," Chapter 8, Walsh-Hadamard Transform, Prentice-Hall, pp. 301-303, 1982.
- [2] Clarke, R. J., "Relation between the Karhunen Loeve and Cosine Transform," *IEEE Proceedings*, Part F, Vol. 128, No. 6, pp. 359-360, Nov. 1981.
- [3] Narasimha, M. J., Peterson, A. M., "On the Computation of the Discrete Cosine Transform," *IEEE Transactions on Communications*, Vol. 26, No. 6, pp. 934-936, June 1978.
- [4] Haralick, R. M., "A Storage Way to Implement the Discrete Cosine Transform," *IEEE Transactions on Computers*, pp. 764-765, July 1976.
- [5] Chen, W. H., Smith, C. H., Fralick, S. C., "Fast Computational Algorithm for the Discrete Cosine Transform," *IEEE Transactions on Communications*, Vol. 25, No. 9, pp. 1004-1009, Sept. 1977.
- [6] Sung, T. Y., "VLSI Parallel and Distributed Computation Algorithms for DCT Processors", *Proceedings IEEE International Phoenix Conference on Computer and Communications*, Scottsdale, Arizona, USA. pp.121-125, March 1990.
- [7] Sung, T. Y., "Novel Parallel VLSI Architectures for Discrete Cosine Transforms," *Proceedings IEEE International Conference on Acoustics, Speech and Signal Processing*, Albuquerque,

- New Mexico, USA, pp.998-1001, April 1990.
- [8] Lee, Y. P., Chen, T. H., Chen, L. G., Ku, C. W., "A Cost-Effective Architecture for 8×8 two-dimensional DCT/IDCT Using Direct Method," *IEEE Transactions on Circuits Systems for Video Technology*, Vol. 7, No. 1, pp. 459-467, June 1997.
- [9] Chang, Y. T., Wang, C. L., "New Systolic Array Implementation of the 2-D Discrete Cosine Transform and Its Inverse," *IEEE Transactions on Circuits Systems for Video Technology*, Vol. 5, No. 1, pp. 150-157, April 1995.
- [10] Hsiao, S. F., Shiue, W. R., "A New Hardware-Efficient Algorithm and Architecture for Computation of 2-D DCTs on a Linear Array," *IEEE Transactions on Circuits and Systems for Video Technology*, Vol. 11, pp.1149-1159, Nov. 2001.
- [11] Srinivasan, V., Liu, K. J. R., "VLSI Design of High-Speed Time-Recursive 2-D DCT/IDCT Processor for Video Applications," *IEEE Transactions on Circuits Systems for Video Technology*, Vol. 6, No. 1, pp. 87-96, Feb. 1996.
- [12] Kuroda T., et al., "A 0.9-V, 150-MHz, 10-mW, 4mm^2 , 2-D Discrete Cosine Transform Core Processor with Variable Threshold-Voltage(VT) Scheme," *IEEE Journal of Solid-States Circuits*, Vol. 31, No. 11, pp.1770-1778, Nov. 1996.
- [13] Rambaldi, R., Uguzzoni, A., Guerrieri, R., "A $35\text{ }\mu\text{W}$ 1.1 V Gate Array 8×8 IDCT Processor for Video-Telephony," *Proceedings IEEE International Conference on Acoustics, Speech and Signal Processing*, pp.2993-2996, 1998.
- [14] Chen, T. H., "A Cost-Effective 8×8 2-D IDCT Core Processor with Folded Architecture," *IEEE Transactions on Consumer Electronics*, Vol. 45, No.2, pp.333-339, May 1999.
- [15] Sung, T. Y., Hu, Y. H., Yu, H. J., "Doubly Pipelined CORDIC Array for Digital Signal Processing Algorithms," *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, Vol. 2, pp.1169-1172, Apr. 1986.
- [16] Sung, T. Y., Sung, Y. H., "Numerical Accuracy and Hardware Trade-offs for CORDIC Arithmetic for DSP Applications," *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.3, 2004.
- [17] Sung, T. Y., Chen, C. S., "Design and Implementation of a 1024-Point FFT Processor Using Memory Interleaving," *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.3, 2004.

An Efficient Parallel Algorithm for Linear Programming Problems

Ming-Chang Lee

Department of Information Management, Fooyin University,

Tai-Liao Hsiang Kaohsiung Hssien, Taiwan, ROC

Sc114@mail.fy.edu.tw

Abstract

This study developed a parallel algorithm to efficiently solve linear programming models. The proposed algorithm utilizes the Dantzig-Wolfe Decomposition Principle and can be easily implemented in a general distributed computing environment. The analytical performance of the new algorithm, including the speedup upper bound and lower bound limits, was derived. Numerical experiments are also provided in order to verify the complexity of the proposed algorithm. The empirical results demonstrate that the speedup of this parallel algorithm approaches linearity, which means that it can take full advantage of the distributed computing power as the size of the problem increases.

Keywords: linear programming, parallel algorithm, speedup

1. Introduction

Linear programming (LP) involves a sequence of steps that will lead to the most effective way to allocate scarce resources among competing activities.

LP is widely used in a number of areas to help managers make decisions, such as assigning jobs to machines, mixing ingredients for a product, determining a distribution system and other situations. An LP model consists of an objective function to be optimized and mathematical statements of the constraints. Given that many LP models represent large and complex physical systems, a typical medium-sized LP model might have 20,000 variables and 5,000 constraints [1]. The required computing resources for solving a modest LP application are therefore huge.

The availability of cost-effective parallel computers has shown the potential of distributed computing power for many large-scale mathematical programming problems. Some previous studies have developed interesting results in this area (e.g., [2,3]). Robert et al. [4] use a branch- and – bound approach in solving mixed integer programming problems, which use MIP package software (MPSX v2). When the branch – and -

bound search is underway there will be many active nodes, each one of which involves the solution of an LP. These LPs are independent and can be solved in parallel if multiple processors are available. Viswanathkumar and Srinivasan [5] used a branch and bound algorithm to minimize completion time variance on a single processor.

However, exploiting parallelism with a mathematical programming algorithm is not always easy due to the communication complexity between processors, which often becomes a bottleneck during the execution process. Despite many software tools developed for the distributed computing environment, to convert a conventional application into a parallel application remains very difficult. For instance, many Operations Research textbooks (Hiller, Lieberman, and Lieberman [6]) have introduced the simplex method, which is an algebraic procedure for solving linear programming problems. The simplex method improves the feasible solution in an orderly manner by performing a series of elementary row operations until the optimality is achieved. To execute the simplex method in a parallel mode is apparently difficult due to its sequential nature.

In this study, we developed a parallel algorithm, based on the Dantzig-Wolfe decomposition principle (DWDP), to solve linear programming and other block-type optimization

problems [7]. Although Dantzig and Wolfe introduced the decomposition principle in the early sixties, it is still widely adopted to cope with large-scale optimization problems. Given that larger and more complex mathematical models have become commonplace (Chinneck [8]), the importance of the DWDP is well recognized by researchers. Using both analytical studies and numerical analysis, we show that the proposed parallel algorithm can be executed efficiently in a general distributed computing environment.

This paper is organized as follows. The next section develops the parallel linear programming model. We explore the analytical performance of the proposed algorithm in Section 3. Computational results are shown in section 4 where the new algorithm is implemented in a portable parallel-computing environment. Finally, concluding remarks are presented in Section 5.

2 Description of the algorithm

Consider a linear programming problem that can be expressed in the following form.

$$\begin{aligned} \text{Minimize } & \mathbf{c}^T \mathbf{x} \\ \text{Subject to: } & \mathbf{Ax} = \mathbf{b} \end{aligned} \quad (1)$$

$$\mathbf{x} \geq \mathbf{0}$$

where $\mathbf{A}$ is a matrix of order m by k , $\mathbf{c}$ and $\mathbf{x}$ are both k -dimensional vectors and $\mathbf{b}$ is an m -dimensional vector with each component nonnegative. It is

observed that the $\mathbf{A}$ matrix in many large linear programming problems usually has a special block-angular structure, namely:

$$\mathbf{A} = \begin{bmatrix} \mathbf{L}_1 & \mathbf{L}_2 & \dots & \mathbf{L}_n \\ \mathbf{A}_1 & & & \\ & \mathbf{A}_2 & & \\ & & \ddots & \\ & & & \mathbf{A}_n \\ & & & & \mathbf{x}_i^3 \mathbf{0} \end{bmatrix} \quad (2)$$

Where all $\mathbf{A}_i$ in the technology matrix $\mathbf{A}$ are independent blocks linked by coupling-equation matrices $\mathbf{L}_i$. As the angular-structure appears, the decomposition principle is substantiated by forming an equivalent master program (defined below) and several subproblems which correspond to each sub-matrix $\mathbf{A}_i$. The solution procedure for (1) involves iterations between a set of independent subproblems where their objective functions are formed using parameters derived from the master program.

Suppose each $\mathbf{A}_i$ has m_i rows and k_i columns and each $\mathbf{L}_i$ is an $m_0 \times k_i$ matrix, for $i = 1, 2, \dots, n$. By partitioning vectors $\mathbf{b}$, $\mathbf{x}$ and $\mathbf{c}$ into sizes corresponding to each $\mathbf{A}_i$, problem (1) can be rewritten as follows.

Minimize

$$\sum_{i=1}^n \mathbf{c}_i^T \mathbf{x}_i \quad (2)$$

$$\text{Subject to: } \sum_{i=1}^n \mathbf{L}_i \mathbf{x}_i = \mathbf{b}_0$$

$$\mathbf{x}_i \in \Omega_i$$

where $\Omega_i = \{ \mathbf{A}_i \mathbf{x}_i = \mathbf{b}_i, \mathbf{x}_i \geq \mathbf{0} \}$ for $i = 1, 2, \dots, n$. Apparently, the set Ω_i is convex and mutually independent.

We can then define the subproblem i , for $i = 1, 2, \dots, n$, as:

$$\text{Minimize } (\mathbf{c}_i^T - \mathbf{l}_0^T \mathbf{L}_i) \mathbf{x}_i \quad (3)$$

$$\text{Subject to: } \mathbf{A}_i \mathbf{x}_i = \mathbf{b}_i,$$

where $\mathbf{l}_0^T$ is the vector denoting the simplex multipliers corresponding to the constraint $\sum_{i=1}^n \mathbf{L}_i \mathbf{x}_i = \mathbf{b}_0$.

In contrast to the subproblem (3), problem (2) is called the master program. Based on the property of convexity of (2) and (3), which implies that all solutions can be written as a linear combination of their vertices, a two-level algorithm for the solution of the linear programming problem can be developed. The master program is on the first level in searching for the coefficients of the linear combination and the subproblem (3) is on the second level for solving the possible optimal vertices. Details of this two-level algorithm, which applies the Dantzig-Wolfe decomposition principle, can be found in Martinson and Tind [9].

Assume that there exists a distributed computing environment (DCE) in which the processing units are independent machines, connected by a

network, and a centralized processor (or the master processor) serves as the coordinator. Such an environment has proven to be a viable approach to provide concurrent computing power at reasonable costs [10]. The design of an algorithm on DCE requires tight load balancing in order to reduce the communication overhead and obtain good performance [11]. A parallel two-level algorithm for linear programming problems that can be implemented in a general DCE is described below.

Algorithm 1. Parallel LP method

- Step 1 Initiate the distributed computing environment by creating n processes in the network and assign one of these processes as the master process to coordinate the computing tasks.
- Step 2 Let basis matrix $\mathbf{B} = \mathbf{I}$, be an identity matrix.
- Step 3 The master process solves the current basic solution $\mathbf{X}_B$, and finds the simplex multipliers $\mathbf{l}^T = \mathbf{B}^{-1} \mathbf{c}_B^T$ where $\mathbf{l}^T = (\mathbf{l}_0^T, \bar{\mathbf{l}})$ and $\bar{\mathbf{l}} = (\bar{l}_1, \bar{l}_2, \dots, \bar{l}_n)$.
- Step 4 The master process broadcasts necessary data to each child-process and assigns the i th

child-process to solve the i th subproblem (as denoted in equation (3)) where each child-process calculates

$$r_i^* = (\mathbf{c}_i^T - \mathbf{l}_0^T$$

$$\mathbf{L}_i) \mathbf{x}_i^* - \bar{l}_i$$

for $i = 1, 2, \dots, n$.

- Step 5 Once the i th child-process solves the i th subproblem, it sends r_i^* and $\mathbf{x}_i^*$ to the master process. After all of the processes return their solutions, if all $r_i^* \geq 0$, then the algorithm terminates. Otherwise, go to Step 6.
- Step 6 The master process determines which column the basis is entered by selecting the minimum value r_i^* of the subproblems. Let $\begin{pmatrix} \mathbf{L}_i \mathbf{x}_i^* \\ \mathbf{e}_i \end{pmatrix}$ be the column that will enter the current basis $\mathbf{B}$, where $\mathbf{e}_i$ is a unit vector.
- Step 7 The master process updates $\mathbf{B}^{-1}$ and go to Step 3.
- In the presented algorithm, Step 1

declares the distributed computing environment and creates n child processes. Step 2 assigns the initial basic feasible solution for the master problem. Each child-process uses the simplex multipliers 1^T , found in Step 3, to solve the i th subproblem in Step 4. Note that this step is the most time consuming part in a sequential algorithm because n linear programming models must be solved. The master process collects the solutions obtained from each subproblem, determines the optimal solution $\mathbf{x}_i^*$ and the associated optimal objective value r_i^* , and checks the optimal condition in Step 5. If the condition is satisfied, $\mathbf{x}_i^*$ is the extreme point of Ω_i and the optimum is found. Step 6 constructs the corresponding vector that will enter the basis of the master program if the terminating condition is not satisfied. The solutions $\mathbf{x}_i^*$ for the i th subproblems are then sent to the master program, which combines these inputs to update the basic solution matrix and determines a new 1^T . The result is again sent to each child-process and the iteration proceeds until an optimality test is satisfied.

3 Analytical performance

To evaluate the performance of the proposed algorithm stated in the previous section, first we will investigate

to performance complexity. The presented algorithm could be implemented into a general distributed computing environment consisting of a network of heterogeneous computers.

Assume that the parallel algorithm uses a cluster of p workstations connected in a DCE and terminated in time T_p . Let T_s be the best possible time required to solve the same problem using a sequential (uni-processor) algorithm. The ratio

$$S_p = T_s / T_p \quad (4)$$

is called the algorithm speedup [12]. Speedup is one of the most common indicators for measuring the efficiency of a parallel algorithm. For simplicity, assume that there are enough processors to execute the n subproblem in parallel. If t is the execution time of the inherently sequential proportion of the algorithm, and α is the remaining proportion that could be performed in parallel (the execution time is t/n for a system with n processor) then $T_s = t(\alpha + \beta)$, and $T_p = t(\alpha + \frac{\beta}{n})$. Therefore, the speedup could be approximated by

$$S_p \cong \frac{a+b}{a+\frac{b}{n}} \quad (5)$$

When α is a proportion of β , that is, $\alpha = w\beta$, then

$$S \cong \frac{nw+n}{nw+1} \quad (6)$$

We can further derive the speedup upper

格式化

格式化

格式化

bound and lower bound limits as follows.

$$\text{Upper bound of } S_p = \lim_{w \rightarrow 0} S_p = n \quad (7)$$

$$\text{Lower bound of } S_p = \lim_{w \rightarrow \infty} S_p = 1 \quad (8)$$

Now, when the number of subproblems, n , is greater than the number of processors, p , that are available, and assume that $n = (p-1)q$, the speedup could be approximated by:

$$S_p \cong \frac{nw + n}{nw + 1} = \frac{(p-1)qw + (p-1)q}{(p-1)qw + 1} < \frac{(p-1)qw + (p-1)q}{(p-1)qw + q} = \frac{(p-1)w + (p-1)}{(p-1)w + 1} \quad (9)$$

The speedup upper bound and lower bound limits can now be derived as follows.

$$\text{Upper bound of } S_p = \lim_{w \rightarrow 0} S_p = p-1 \quad (10)$$

$$\text{Lowerbound of } S_p = \lim_{w \rightarrow \infty} S_p = 1 \quad (11)$$

From the above analysis, it is apparent that the ratio w is critical to the speedup of the parallel algorithm. In the proposed algorithm, since Step 4 is the most computationally intensive part, which is required to solve n linear programming models, the ratio w is therefore very small and the speedup should be approximate to the upper bound limit. That is, the speedup would approach n when there are

enough processors in the DCE.

4. Numerical experiments

In this section we present the numerical experiments results in order to justify the performance of the proposed algorithm. The algorithm presented was implemented in a cluster of distributed network workstations, which were connected via an optical fiber link. We used the Parallel Virtual Machine (PVM) library routines to develop the message-passing environment for distributed computing. PVM enables a collection of heterogeneous computer systems to be viewed as a single parallel virtual machine and has been widely adopted by researchers [13]. The algorithm code was programmed in FORTRAN/77.

The method for numerical experiments in this study was similar to the one proposed by Pisinger [8], where the number of model constraints ranged from 20 to 50 to 120. For each problem type, five sets of models, generated using different random-number generator seeds, were solved. The average CPU time (five replications for each instance) required to solve each type of test problem with respect to the different numbers of processors used during experiments is plotted in Figure 1. The speedup earned in the numerical experiments as well as the mean speedup of the proposed algorithm are calculated and plotted in Figure 2.

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

格式化

(Insert Figure 1 and Figure 2 about here)

It is clear that the numerical performance of the proposed algorithm is impressive based on the experimental results demonstrated in Figure 1. When solving a modest problem sized problem (120 constraints), the CPU time saved in a DCE could be more than six fold. Given that a distributed computing environment is very common in many computer centers and PVM is a shareware, the presented algorithm is useful to many researchers in solving the linear programming problems.

The best speedup obtained in the numerical experiments was 6.36 for the model with 120 constraints. In general, the speedups earned in the experiments were close to the mean speedups as expected (Figure 2). It is also interesting to note that near linear speedup was achieved, which means that the proposed algorithm can take full advantage of the distributed computing power as the size of the problem increases.

5. Conclusions

Distributed computing on clusters of workstations is attractive and cost-effective to researchers for evolving processing and networking technologies. This study developed a parallel linear programming algorithm and evaluated its performance on a DCE. The numerical results show that the speedup of the proposed algorithm approaches

linearity, which is consist with its analytical performance. We conclude that the presented algorithm is efficient and becomes a useful reference solution model for LP applications, especially for large-scale problems.

The proposed algorithm was implemented on a PVM system (a portable distributed computing environment). This software is free to the public and has been installed on many networked computing platforms. We are currently working on porting our computer codes into other computing environments and testing the algorithm on a wider variety of test problems. It is safe to state that further study on the development of some mathematical programming algorithms that could also take advantage of distributed computing power requires greater research efforts.

Reference

1. Forrest J. J. H., Tomlin J. A. implementing the simplex method for the optimization subroutine library, IBM Systems Journal 1992; 31(1):11-25.
2. Lyu J., Gunasekaran A., Kachitvichyanukul V. Towards a portable and efficient environment for parallel computing, International Journal of Systems Science 1995; 26(6): 1333-1341.
3. Ashford R. W., Connard P., Daniel R. Experiments in solving mixed integer programming problems on small array of transputers, Journal

- of the Operational Research Society 1992; 43(5): 61-72.
4. Viswanathkumar G., Srinivasan G. A bound and bound algorithm to minimize completion time variance on a single processor 2003; 30: 1135-1150.
5. Romeijn H. E., Smith R. L. Parallel algorithms for solving aggregated shortest-path problems, Computers and Operations Research 1999; 26: 941-953.
6. Hiller F. S., Lieberman G. J., Lieberman G. Introduction to Operations Research 1995, New York: McGraw-Hill.
7. Dantzig G. B., Wolfe P. The decomposition principle for linear programs, Operations Research 1960; 8: 101-111.
8. Chinneck J. W. Computer codes for the analysis of infeasible linear programs, Journal of the Operational Research Society 1996; 47: 61-72.
9. Martinson R. K., Tind J. An interior point method in Dantzig-Wolfe decomposition, Computers and Operations Research 1999; 26: 1195-1216.
10. Sunderam V. S. Heterogeneous network computing: the next generation, Parallel Computing 1997; 23: 121-135.
11. Sekharan C. N., Goel V., Sridhar R. Load balancing methods for ray tracing and binary tree computing using PVM, Parallel Computing 1995; 21: 1963-1978.
12. Quinn M. J. Parallel Computing: Theory and Practice 1994, McGraw Hill, N.Y.
13. Sunderam V. S., Geist G. A., Dongarra J., Manchek R. The PVM concurrent computing system: evolution, experiences, and trends, Parallel Computing 1994; 20: 531-545.
14. Pisinger D. An expanding-core algorithm for the exact 0-1 Knapsack problem, European Journal of Operational Research 1995; 87: 175-187.

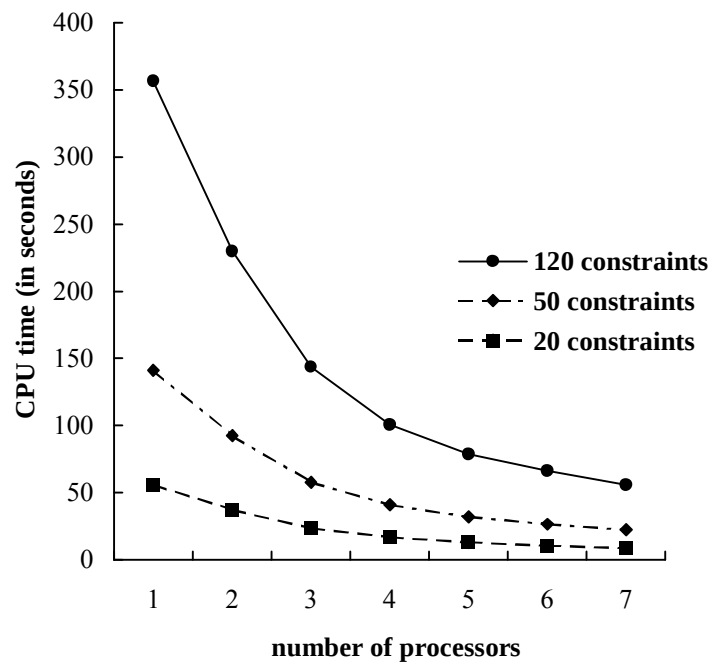


Figure 1. CPU time (in seconds) versus number of processors

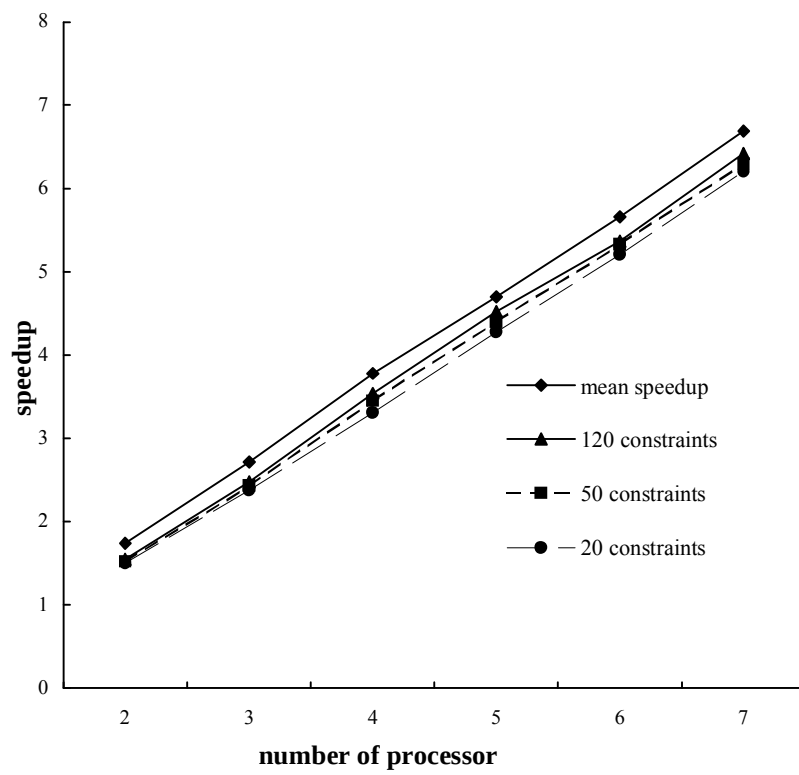


Figure 2. Mean speedup and speedup versus number of processors

A Parallel-Pipelined Constant Geometry Algorithm (PCGA) for Computation of FFT on a Special Processor

Tze-Yun Sung

Electrical Engineering Department

Chung Hua University

Hisn-chu, Taiwan 300-12

Tel : 886-3-579-0902 Fax: 886-3-578-1150

E-mail : bobsung@btcx.com.tw

Abstract

In digital signal processing applications, we usually use DFT (discrete Fourier transform) to transfer signal from time-domain to frequency-domain. In past decades, there were many papers discussing and exploiting FFT (fast Fourier transform) architectures for hardware implementation.

In this paper, we proposed a novel VLSI FFT processor based on parallel-pipelined constant geometry algorithm (PCGA). This processor provides constant geometry for each stage of FFT, data reordering is not needed any more. Besides, the architecture of PCGA FFT processor is simple and regular, and the PCGA FFT processor always needs two-set of data RAM which acts as data storages for reading and writing data alternately.

Especially, the PCGA FFT processor increases throughput with increasing input points, and sufficiently exploits the advantage of parallel-pipelined computation for digital signal processing applications. Hence, comparing to conventional FFT processors, the PCGA FFT processor is more suitable and efficient for processing large number-point FFT. A novel implementation of the FFT processor should be

achieved.

Keywords: parallel-pipelined constant geometry algorithm (PCGA), FFT, CORDIC, VLSI.

1. Introduction

Discrete Fourier transform (DFT) is a very important tool for analyze digital signal running on digital computers [1]. The technique of fast Fourier transform (FFT) is widely applied on many application areas [2]; such as power spectrum estimation, filter simulation, radar systems, sonar systems, medical instruments, measurement instruments, and etc [3], [4], [5], [6], [7], [8], [9].

FFT is a fast computation algorithm proposed by Cooley and Tukey in 1965. FFT improves computation complexity from $O(N^2)$ to $O(N \log_2 N)$ and realizes DFT in real time applications of digital signal processing. It makes DFT algorithm perfect and practical.

In past decades, many papers proposed some FFT architectures and processors based on the fast algorithm proposed by Cooley and Tukey [10], [11], [12], [13], [14], [15], [16], [17]; nevertheless the

performance is not satisfied. Especially, when the number of input points is increased, the shuffle structure becomes more complex and operations of data reordering are also increased. Besides, the throughput and transformation time are also not satisfied, the system complexity $O(AT^2)$ becomes larger, when the number of input points is increased.

According to disadvantages of previous researches, we propose parallel-pipelined constant geometry FFT algorithm (PCGA). This algorithm improves computation complexity and hardware cost obviously.

2. Review of FFT Algorithms

• Constant Geometry FFT Algorithms

According to the fast algorithm proposed by Cooley and Tukey [10], $X(k) = DFT(x(n)) = \sum_{n=0}^{N-1} x(n)W_N^{nk}$, (1) where $k=0, 1, 2, \dots, N-1$.

Based on radix-2, from eq. (1), we obtain

$$\begin{aligned} X(k) &= \sum_{r=0}^{(N/2)-1} x(2r)W_N^{k(2r)} + \sum_{r=0}^{(N/2)-1} x(2r+1)W_N^{k(2r+1)} \\ &= \sum_{r=0}^{(N/2)-1} x(2r)W_{N/2}^{kr} + W_N^k \sum_{r=0}^{(N/2)-1} x(2r+1)W_{N/2}^{kr}, \end{aligned} \quad (2)$$

where $W_{N/2} = e^{-j\left(\frac{2\pi}{N/2}\right)}$.

$$\text{Let } A(k) = \sum_{r=0}^{(N/2)-1} x(2r)W_{N/2}^{kr}, \quad (3)$$

$$B(k) = \sum_{r=0}^{(N/2)-1} x(2r+1)W_{N/2}^{kr}, \quad (4)$$

where $k=0, 1, 2, \dots, N-1$.

$$X(k) \text{ is represented as } X(k) = A(k) + W_N^k B(k), \quad (5)$$

where $k=0, 1, 2, \dots, N-1$.

We may arrange sequence of input and output data

arbitrarily, and achieve the desired architecture; nevertheless we get used to input in normal order. Hence, we rearrange the flow graph of FFT to achieve the flow graph of constant geometry FFT computation with input in normal order which is shown in Fig. 2.1 [10].

In fact, the characteristic of flow graph of constant geometry FFT computation possesses constant geometry for each stage. The data of odd and even sequence are computed independently. Constant geometry structure reduces hardware complexity [18]. This characteristic is very suitable for parallel processing and VLSI implementation.

• Butterfly Computation

We observe eqs. (3) and (4), and Fig. 2.1, and find the basic computation. The basic operation is called “butterfly computation” which is shown in Fig. 2.2. In fact, the flow graphs of FFT are composed of many butterfly computations. The basic computation is represented as

$$X_{m+1}(p) = X_m(p) + W_N^r X_m(q), \quad (6)$$

$$X_{m+1}(q) = X_m(p) - W_N^r X_m(q), \quad (7)$$

where $0 \leq r \leq (N/2-1)$, $0 \leq p \leq (N/2-1)$, $(N/2-1) < q \leq (N-1)$, $q = p + N/2$, r , p and q are integers.

The butterfly computation can be performed by CORDIC arithmetic [18], which will be discussed in PCGA FFT processor.

3. PCGA FFT Processor

• Butterfly Processor for PCGA FFT Processor

Hardware implementation of CORDIC needs only shifters and adders, the simple hardware is very attractive. CORDIC is applied in signal processing widely today, we use CORDIC arithmetic to design butterfly processor.

In circular coordinate system of CORDIC (Coordinate Rotation Digital Computer), rotation mode can be represented as [18]

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = K_c \begin{bmatrix} \cos z_0 & \sin z_0 \\ -\sin z_0 & \cos z_0 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad (8)$$

where $[x_0 \ y_0]$ is a input vector, z_0 is a rotation angle, K_c is a scale factor, and $[x_n \ y_n]$ is a output vector.

Because K_c is a constant, the scaling can be pre-processed or processed in parallel [19], [20]. The modified circular rotation can be embedded into butterfly computation as [19]

$$\begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix} = \begin{bmatrix} \cos 2\pi/N & \sin 2\pi/N \\ -\sin 2\pi/N & \cos 2\pi/N \end{bmatrix} \begin{bmatrix} \text{Re}[X_m(q)] \\ \text{Im}[X_m(q)] \end{bmatrix}. \quad (9)$$

The butterfly computation is represented as

$$\begin{bmatrix} \text{Re}[X_{m+1}(p)] \\ \text{Im}[X_{m+1}(p)] \end{bmatrix} = \begin{bmatrix} \text{Re}[X_m(p)] \\ \text{Im}[X_m(p)] \end{bmatrix} + \begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix}, \quad (10)$$

$$\begin{bmatrix} \text{Re}[X_{m+1}(q)] \\ \text{Im}[X_{m+1}(q)] \end{bmatrix} = \begin{bmatrix} \text{Re}[X_m(p)] \\ \text{Im}[X_m(p)] \end{bmatrix} - \begin{bmatrix} \text{Re}[X'_m(q)] \\ \text{Im}[X'_m(q)] \end{bmatrix}. \quad (11)$$

Fig 3.1 shows the modified CORDIC arithmetic unit and butterfly processor.

The butterfly processor [18], which is composed of a modified CORDIC arithmetic unit, two-adder and two-subtractor, performs function of butterfly computation and has multiplierless hardware. Hence, the butterfly processor has high performance and small area. These characteristics are very suitable for VLSI implementation [18], [19].

• Data RAMs for PCGA FFT Processor Using Memory Interleaving

The PCGA FFT processor possesses characteristics of parallel-pipelined processing and constant geometry for each stage. These characteristics can achieve fast transform and VLSI parallel-pipelined architecture. Due to Fig. 2.1, we find that the points of odd sequence and the points of even sequence are computed independently. According to the characteristics, we design a special structure of data RAM for FFT computation. The data RAM for 16-point PCGA FFT processor is shown in Fig. 3.2. The data RAM has input-data-buses and output-data-buses separately. In fact, twin data RAMs are read and written data alternately in the PCGA FFT processor. The data from one data RAM is sent to butterfly processors to process data, and the results are stored in the other data RAM. We use memory interleaving to arrange new data sequence.

The data RAM has four parts, and each part has dual data ports which separate input data port and output data port. “EVEN-HIGH” and “ODD-HIGH” of the data RAM are stored $X_m(q)$ ’s, and “EVEN-LOW” and “ODD-LOW” of the data RAM are stored $X_m(p)$ ’s.

According to number of butterfly processors, the data RAM has the same number of set of output data ports. In Fig. 3.2, one set of output data form “EVEN-HIGH” and “EVEN-LOW” of the data RAM, and the other set of data form “ODD-HIGH” and “ODD-LOW” of the data RAM. Two-set of data are sent to two butterfly processors simultaneously, and are computed in parallel.

• 16-Point PCGA FFT Processor

In the 16-point PCGA FFT processor, two butterfly processors compute data in parallel, and two-set of data RAM are read and written data alternately. The

16-point PCGA FFT processor and the timing diagram of reading and writing of butterfly processors are shown in Fig. 3.3.

In Fig. 3.3, we find that the data is read from *DATA RAM-I* with data sequence of 0, 8, 4, 12 and 1, 9, 5, 13, and the results are written into *DATA RAM-II* by data sequence of 0, 1, 8, 9, and 2, 3, during t_1 . Similarly, the data are read from *DATA RAM-I* by data sequence of 2, 10, 6, 14, and 3, 11, 7, 15, and the results are written into *DATA RAM-II* by data sequence of 4, 5, 12, 13, and 6, 7, during t_2 . Alternately, during t_3 and t_4 , the data are read from *DATA RAM-II* and written into *DATA RAM-I*.

According to above the sequence of reading and writing from data RAMs, we describe the timing diagram of 16-point PCGA FFT processor which is shown in Fig. 3.4. The PCGA FFT processor needs 8 time units to finish the computation of 16-point FFT.

• **Programmable 1024-Point PCGA FFT Processor and Multiprocessor System**

The programmable 1024-point PCGA FFT processor is designed and evaluated on Xilinx XC2V3000 FPGA. This processor can process 1024-point FFT, and includes 2K words data RAM, 0.5K words twiddle-factor ROM for butterfly processors, ten butterfly processors, digital serial peripheral interfaces (DSPIs) for interfacing to digital signal source and master processor, address decoders for memory interleaving, and controller for managing system operation. The master processor can input commands to the programmable PCGA FFT processor by number of point ($2^n \leq 1024$), and a “single/ pipelined” control. Some programmable PCGA FFT processors can be cascaded to perform a multiprocessor system controlled by “single/pipelined”. The multiprocessor system has a

parallel-pipelined processing architecture to increase system throughput. The multiprocessor system for 16-point FFT computation is shown in Fig. 3.5. This multiprocessor system can process 16-point FFT computation in parallel and pipeline. The latency time is eight time units, but throughput is improved to produce 16-point data for each two time units.

4. Discussion and Conclusion

The PCGA FFT processor always needs two sets of data RAM which acts as data storages for reading and writing data alternately. This architecture provides constant geometry for each stage of FFT, data reordering for each stage is not needed any more. So, the hardware implementation of PCGA FFT processor is simpler than previous designs [12], [13], [14], [15], [17], and the performance of PCGA FFT processor is better than many previous designs [12], [14], [17].

The hardware code is written in Verilog running on SUN Blade 1000 workstation under the Modelsim simulator and emulated on Xilinx XC2V3000 FPGA [21]. It is designed to evaluate the architecture and provide an embedded core for digital signal processing applications [22]. Hence, we have a table related to number of points and butterfly processors, memory size and processing time. This table is shown in Fig. 4.1.

We also analyze and simulate a pipeline FFT processor which was proposed by S. I. Sayegh [17]. Sayegh proposed a pipeline processor for mixed-size FFT's, the design of FFT processor is very practical and suitable for VLSI implementation. But Sayegh's architecture needs more complex data switching for each stage, more butterfly processors and more computation time. The table related to butterfly

processors, memories and computation time is also shown in Table 4.1.

The comparison of computation time is shown in Fig. 4.1. Besides, we also analyze and write an AT^2-N diagram which is shown in Fig. 4.2, and find that the PCGA FFT processor improves the disadvantage of Sayegh's FFT processor obviously.

Table 4.2 compares the hardware requirement for different FFT processors. A parallel complex multiplier needs four multipliers at least, but a CORDIC arithmetic unit performs complex multiplication and needs no multiplier. The PCGA FFT processor achieves a significant savings of memory [23], [24], and requires less number of arithmetic units and hardware [23], [24], [25].

Table 4.3 shows that the comparison of hardware utilization and computational efficiency. The PCGA FFT processor is higher than the implementations of Hui's [25], Chang's [24] and Bi's [13].

Especially, the PCGA FFT processor increases throughput with increasing input points, and sufficiently exploits its advantage of parallel-pipelined computation for digital signal processing applications. Hence, comparing to the conventional pipelined FFT processor architecture [12], [13], [14], [15], [17], [23], [24], [25], the PCGA FFT processor is more suitable and efficient for processing large number-point FFT computation, and achieves high performance and small area. In past decade, the research of computation error and accuracy analysis of FFT is rarely; we analyze the computation error of core arithmetic and propose the model of fixed point computation error. The design reference for the PCGA FFT processor is also proposed. The numerical accuracy of the PCGA FFT processor has been analyzed, and the computation error would be predicted. The hardware would also be saved. A novel implementation of the FFT processor should be achieved.

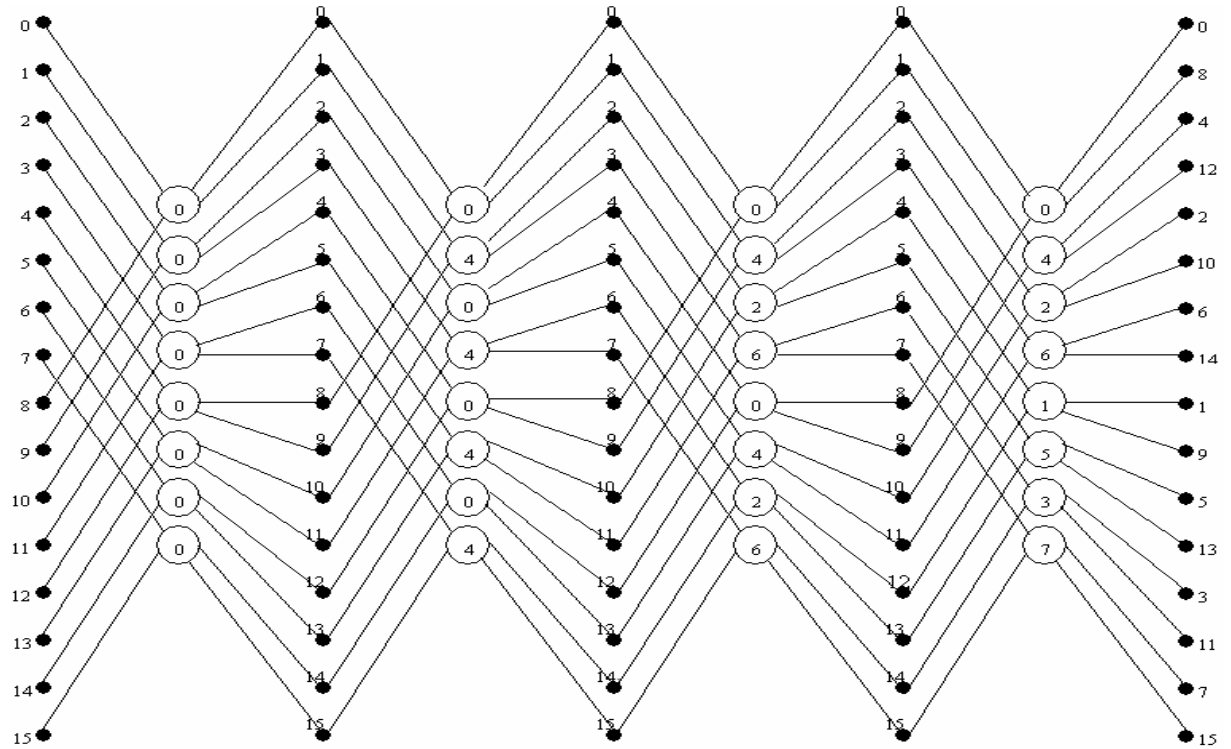


Fig. 2.1 The flow graph of constant geometry FFT computation with input in normal order.

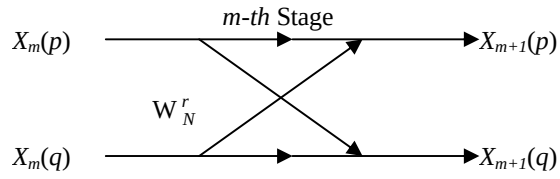
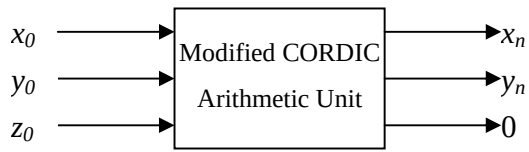
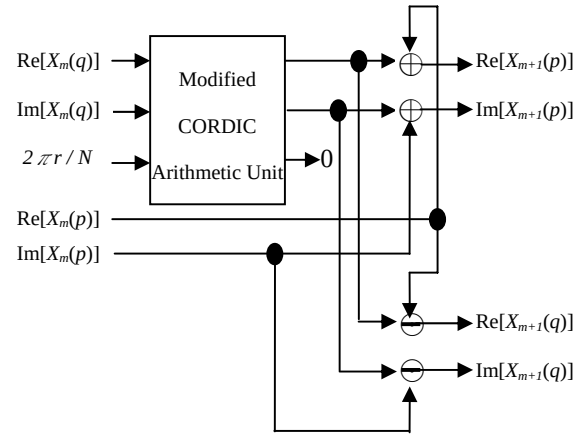


Fig. 2.2 The flow graph of butterfly computation



(a) The modified CORDIC arithmetic unit



(b) The butterfly processor

Fig. 3.1 The modified CORDIC arithmetic unit and the butterfly processor

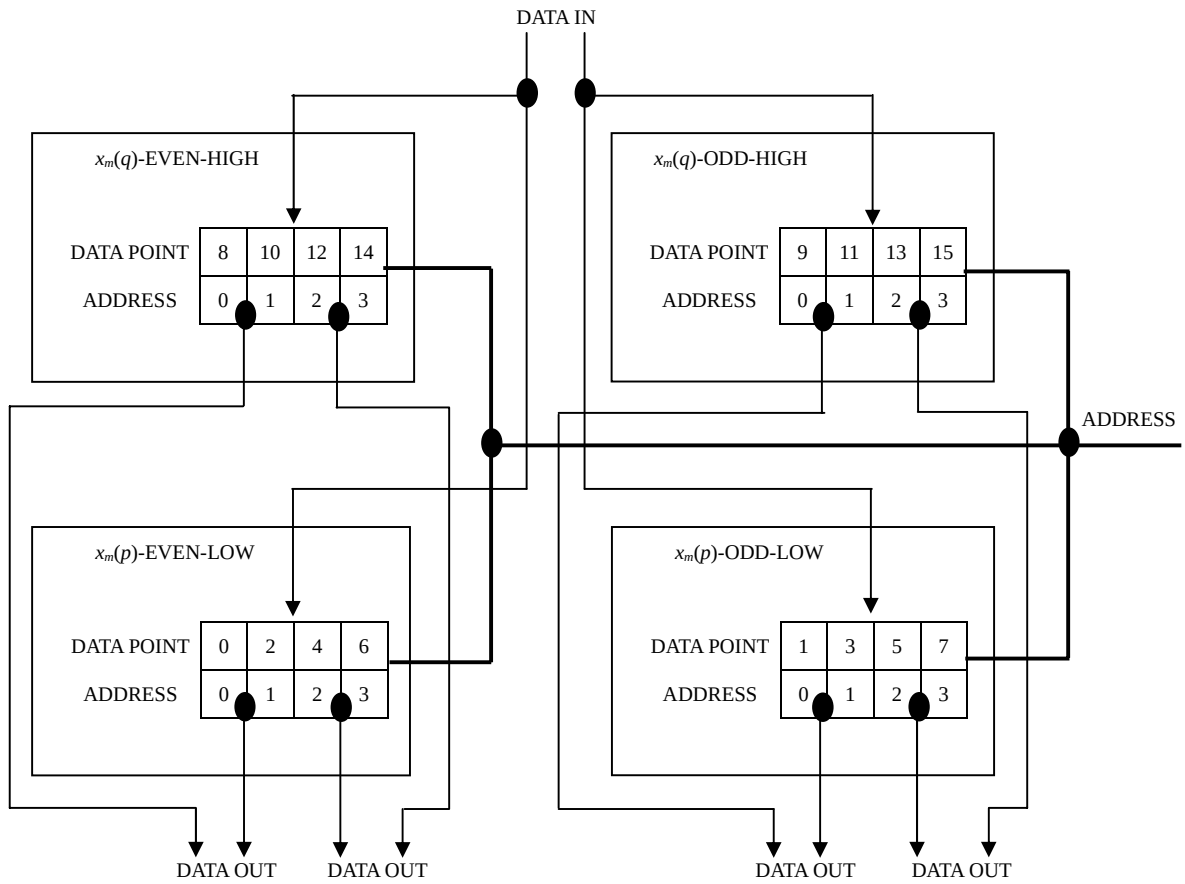
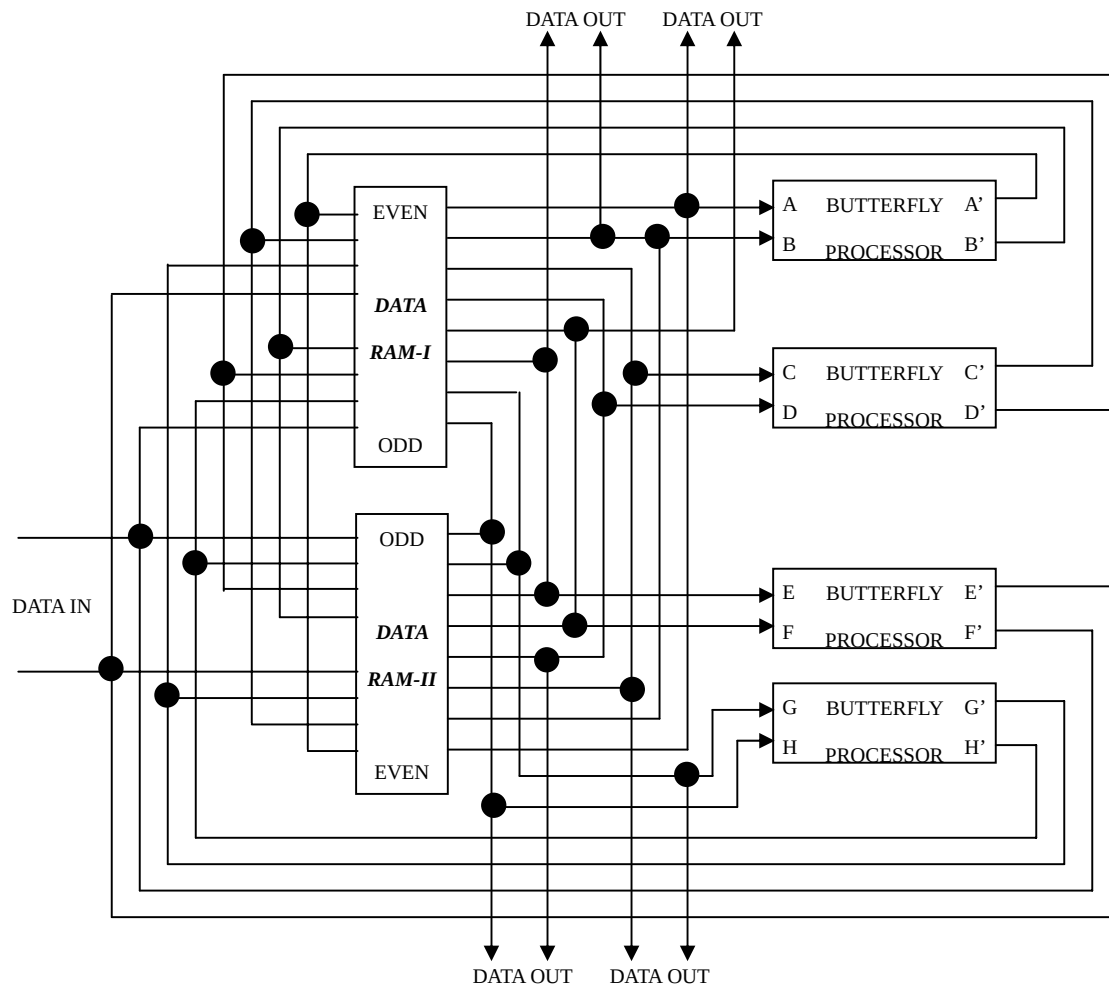


Fig. 3.2 The data RAM for the 16-point PCGA FFT processor



(a) The 16-point PCGA FFT processor

Reading Mode:

Timing Sequence	A	B	C	D	E	F	G	H
t_1	0	8	4	12	1	9	5	13
t_2	2	10	6	14	3	11	7	15

Writing Mode:

Timing Sequence	A'	B'	C'	D'	E'	F'	G'	H'
t_1	0	1	8	9	2	3	10	11
t_2	4	5	12	13	6	7	14	15

(b) The timing diagram of reading and writing butterfly processors

Fig. 3.3 The 16-point PCGA FFT processor and the timing diagram of reading and writing of butterfly processors

t	Data RAM- I	Data RAM- II	Output
t_1	10 14 11 15 2 6 3 7	8 10 9 11 0 2 1 3	----
t_2	----	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----
t_3	8 10 9 11 0 2 1 3	10 14 11 15 2 6 3 7	----
t_4	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----	----

t_5	10 14 11 15 2 6 3 7	8 10 9 11 0 2 1 3	----
t_6	----	8 10 12 14 9 11 13 15 0 2 4 6 1 3 5 7	----
t_7	----	10 14 11 15 2 6 3 7	$X_0 X_8$ $X_1 X_9$ $X_4 X_{12}$ $X_5 X_{13}$
t_8	----	-----	$X_2 X_{10}$ $X_3 X_{11}$ $X_6 X_{14}$ $X_7 X_{15}$

Fig. 3.4 The timing diagram of 16-point PCGA FFT processor

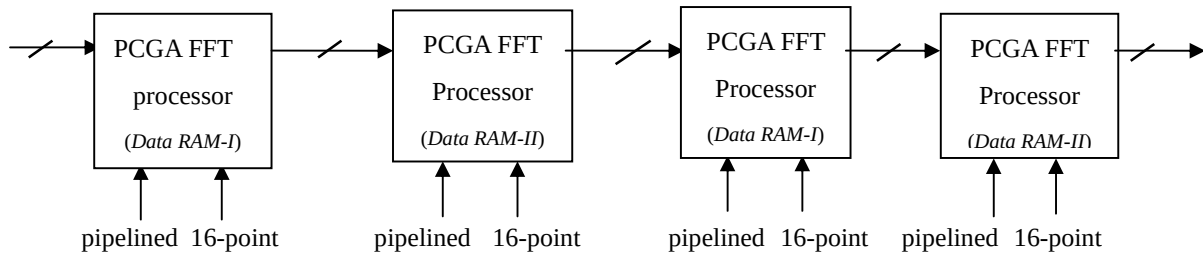


Fig. 3.5 The multiprocessor system for 16-point FFT computation

No. of Point (N)	No. of Butterfly Processor (A)		Memory Size (Words)		Computation Time (T)	
	PCGA	Sayegh[17]	PCGA	Sayegh[17]	PCGA	Sayegh[17]
8	2	3	16	12	6	10
16	4	4	32	28	8	22
32	4	5	64	60	20	46
64	6	6	128	124	36	94
128	6	7	256	252	76	190
256	8	8	512	508	128	382
512	8	9	1024	1020	288	766
1024	10	10	2048	2044	520	1534

Table 4.1 Comparison of PCGA FFT Processor and Sayegh's processor

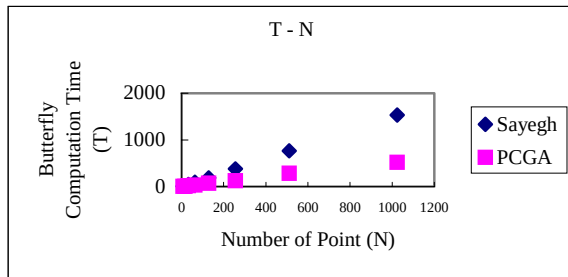


Fig. 4.1 The comparison of computation time

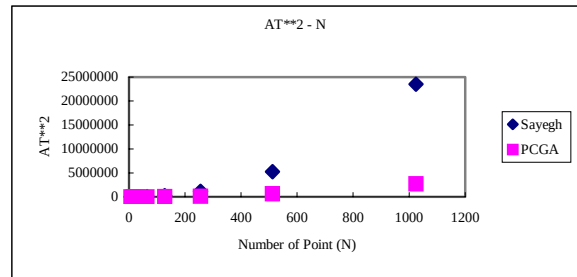


Fig. 4.2 The comparison of system complexities

No. of Points	No. of CORDIC*	No. of Complex Multipliers			Data RAM Size			
	PCGA	Chang[24]	Hui[25]	Swatzlander[23]	PCGA	Chang[24]	Hui[25]	Swatzlander[23]
16	4	3	3	3	32	20	48	48
64	6	6	6	6	128	76	160	160
256	8	9	9	9	512	302	640	640
1024	10	12	12	12	2048	1210	2560	2560

No. of Points	Twiddle-factor ROM Size			
	PCGA	Chang[24]	Hui[25]	Swatzlander[23]
16	8	8	16	16
64	32	32	64	64
256	128	128	256	256
1024	512	512	1024	1024

Table 4.2 The comparison of FFT processors (*CORDIC arithmetic unit: multiplierless)

Computational Efficiency of Arithmetic	PCGA	Bi[13]	Hui[25]	Chang[24]
Addition	100%	50%	100%	~100%
Multiplication	-	75%	100%	100%
CORDIC*	100%	-	-	-

Note: *CORDIC arithmetic operation for PCGA.

Table 4.3 The comparison of hardware utilization and computational efficiency

References

- [1] W. T. Cochran, "What is the Fast Fourier Transform?" *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-15, pp. 45-55, June 1967.
- [2] J. W. Cooley, P. Lewis, P. Welch, "Application of the Fast Fourier Transform to Computation of Fourier Integrals, Fourier Series, and Convolution Integrals," *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-15, pp.79-84, June 1967.
- [3] J. F. Kaiser, "The Digital Filter and Speech Communications," *IEEE Trans. Audio Electroacoustics*, Vol. AU-16, pp.180-183, June 1968.
- [4] H. C. Andrews, K. L. Caspari, "A Generalized Technique for Spectral Analysis," *IEEE Trans. on Comput.*, Vol. C-19, pp.16-25, Jan. 1970.
- [5] S. M. Kay, S. L. Marple, "Spectrum analysis-A modern perspective," *Proc. IEEE*, Vol. 69, pp. 1380-1419, Nov. 1981.
- [6] E. A. Robison, "The Special Issue on Spectral Estimation," *Proc. IEEE*, Vol. 70, pp. 883-1125, Sept. 1982.
- [7] M. N. S. Swamy, K. S. Thyagarajar, "Frequency Transformation of Digital Filters," *Proc. IEEE*, Vol. 65, pp. 165, Jan. 1977.
- [8] A. G. Constantinides, "Spectral Transformation for Digital Filters," *Proc. IEEE*, Vol. 117, pp. 1585-1590, Aug. 1970.
- [9] Alan V. Oppenheim, Ronald W. Schaffer, *Discrete-time Signal Processing*, Second Edition, Prentice-Hall, Inc. Chapter 9, pp.629-692, 1999.
- [10] J. W. Cooley, J. W. Tukey, "An Algorithm for the

- Machine Calculation of Complex Fourier Series,” *Math. Comput.*, Vol. 19, pp. 297-301, 1965.
- [11] H. L. Groginsky, G. A. Works, “A Pipeline Fast Fourier Transform,” *IEEE Trans. on Comput.*, Vol. C-19, No. 11, pp.1015-1019, Nov. 1970.
- [12] E. H. Wold, A. M. Despain, “Pipeline and Parallel-pipeline FFT Processor for VLSI Implementations,” *IEEE Trans. on Comput.*, Vol. C-33, No. 5, pp.414-426, May 1984.
- [13] G. Bi, E. V. Jones, “A Pipelined FFT Processor for Word-sequential Data,” *IEEE Trans. on Acoustics, Speech, and Signal Processing*, Vol. 37, No. 12, Dec. 1989.
- [14] B. Gold, T. Bially, “Parallelism in Fast Fourier Transform Hardware,” *IEEE Trans. on Audio and Electroacoustics*, Vol. AU-21, pp. 5-16, Feb. 1973.
- [15] J. M. Glass, “An Efficient Method for Improving Reliability of a Pipeline FFT,” *IEEE Trans. on Comput.*, Vol. C-29, No. 11, pp. 1017-1020, Nov. 1980.
- [16] A. M. Despain, “Very Fast Fourier transform Algorithms Hardware for Implementation,” *IEEE Trans. on Comput.*, Vol. C-28, pp. 333-341, May 1979.
- [17] Soheil I. Sayegh, “A Pipeline Processor for Mixed-size FFT’s,” *IEEE Trans. on Signal Processing*, pp. 1892-1990, Aug. 1992.
- [18] T. Y. Sung, Y. H. Hu, H. J. Yu, “Doubly Pipelined CORDIC Array for Digital Signal Processing Algorithms,” *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, Vol. 2, pp.1169-1172, Apr. 1986.
- [19] T. Y. Sung, Y. H. Sung, “Smart Design of CORDIC Processor for Signal Processing,” *Chung Hua Journal of Science and Engineering, Special Issue on Electrical and Computer Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.2, 2004.
- [20] T. Y. Sung, Y. H. Sung, “Numerical Accuracy and Hardware Trade-offs for CORDIC Arithmetic for DSP Applications,” *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.1, 2004.
- [21] T. Y. Sung, C. S. Chen, “A Parallel-Pipeline FFT Processor Using Memory Interleaving,” *Chung Hua Journal of Science and Engineering*, School of Engineering, Chung Hua University, Hsin-chu, Taiwan, Vol.1, 2004.
- [22] T. Y. Sung, Y. H. Sung, “SoC Implementation of A Programmable High-Performance FFT Processor,” *Technical Report (BTCX-SoC-031133)*, BTCX Electronic Corp., Science Based Industrial Park, Hsin-chu, Taiwan, Dec. 2003.
- [23] E. E. Swartzlander, W. K. W. Young, S. J. Joseph, “A radix-4 delay Commutator for Fast Fourier Transform Processor Implementation,” *IEEE J. Solid-State Circuits*, Vol. SC-19, pp.702-709, Oct. 1984.
- [24] Y. N. Chang, K. K. Parhi, “An Efficient Pipelined FFT Architecture,” *IEEE Transactions on Circuits and Systems-II: Analog and Digital Signal Processing*, Vol. 50, No.6, pp.332-325, June 2003.
- [25] C. C. W. Hui, T. J. Ding, J. V. McCanny, “A 64-point Fourier Transform Chip for Video Motion Compensation Using Phase Correction,” *IEEE Journal of Solid-State Circuits*, Vol.31, pp. 1751-1761, Nov. 1996.

On arbitrary binary trees of hypercubes

Shin-Feng Tsai

Department of Information Management
Southern Taiwan University of
Technology
n9090029@webmail.stut.edu.tw

Hui-Ling Huang

Department of Information Management
Southern Taiwan University of
Technology
hlhuang@mail.stut.edu.tw

摘要

嵌入樹 (tree) 於超立方體 (hypercube) 的問題在過去二十年中一直引起許多研究者的關注。超立方體是一個最受注意和討論的連結網路結構，而且被實際建構成為真正可用的平行電腦，這是因為它有著相當好的性質，如模組化、對稱性(symmetry)、規則性(regularity)、較小的直徑(diameter)和容錯(fault-tolerant)能力等等。而樹更是重要的結構，在二元樹上已被研究出很多相當有效率的演算法，可以解決各種應用問題。為了將現存某一網路上的演算法能夠輕易的轉移使用在另一網路上，嵌入(embedding)的問題就因應而生。嵌入在多處理機連結網路上是一重要的研究主題，假如我們能將某一網路嵌入另一網路，即代表在此網路上所設計的演算法都可很容易地轉移至另一網路上。本篇論文改進之前Chen與Stallmann的研究成果，我們提出一個新的遞迴演算法將任意二元樹嵌入超立方體結構中，在同樣的條件之下我們的嵌入演算法其膨脹(dilation)係數必定小於等於Chen與Stallmann的方法。

關鍵詞： 樹(tree)、超立方體(hypercube)、嵌入(embedding)、前序追蹤(preorder traversal)、格雷碼(Gray code)、膨脹係數(dilation)

一、簡介

近幾年來由於 VLSI 技術的進步，有許多使用多處理機的連結網路被提出來用以解決複雜的平行計算，如 NCUBE/ten、Cosmic Cube 和 Intel iPSC 系列等超立方體(hypercube)結構是較具代表性且已製造出來作為商業用途的多處理機系統[12, 13, 19, 20]，這些系統甚至可連結數千或數百個多處理器。由於超立方體有許多相當好的性質，因此在超立方體上已有相當多的研究成果，如廣播(broadcasting)、排序(sorting)、工作配置(job

allocation)、排程(scheduling)和其他各種應用問題[6, 7, 18, 23]。

由於近年來在其他各類的連結網路上，已設計出許多的平行演算法去解各式各樣的問題。為了將現存在某一網路上的演算法能夠輕易的轉移使用在另一網路上，嵌入(embedding)的問題就因應而生。我們將連結網路視為一種圖形結構，將處理器視為節點(node)，處理器和處理器之間的連線視為邊(edge)。在圖論中將客圖(guest graph)嵌入主圖(host graph)意指在主圖中找出一個子圖與客圖同構(isomorphic)。圖形嵌入在多處理機連結網路上是一重要的研究主題。

在樹結構上有著許多相當有效率的演算法，如排序及 divide and conquer 演算法等，尤其在資料庫上的一些演算法更可以在以樹為主的資料結構上得到相當好的執行效率。因此如何將樹結構嵌入超立方體中，使得這些在樹結構上相當有效率的演算法可以很容易地轉移至實際的超立方體平行電腦上是一個相當重要的問題。

近年來對於如何將各種樹結構嵌入超立方體已有很多研究成果，包括二元樹[2, 10, 14]、X-tree[9]、tree machines [1]、網狀樹[11, 16]、延伸樹及平衡延伸樹[13]等等。其中又以二元樹有最多研究成果，例如完全二元樹[15]、不完全二元樹[22]、任意二元樹[3-5, 21, 24]、full-ringed二元樹[15]等。

本篇論文主要是改進之前Chen與Stallmann的研究成果[8]，我們提出一個新的演算法將任意二元樹嵌入超立方體結構中，其膨脹(dilation)係數必定小於等於Chen與Stallmann的方法，並且擴張係數為1、平均邊膨脹係數不超過2。

本篇論文其餘各節的內容如下：第二節說明符號與定義，在第三節中我們將說明嵌入任意二元樹於超立方體中的演算法，並在第四節做總結。

二、符號與定義

首先我們定義超立方體結構， n 維超立方體 Q_n 有 2^n 個節點，每個節點以 n 個位元表示。任

兩個節點 u 和 v ，若其節點表示式只有一個位元不同，則此兩點之間有一個邊(edge)相連，我們以 $\{u, v\}$ 表示兩節點的邊。

我們定義任意二元樹 T_n 有 2^n 個節點，樹 T_n 的根節點為 $r(T_n)$ 。任一節點 v 至多有兩個子樹分別是 $L(v)$ 和 $R(v)$ ，在不失一般性的前提下，我們將較少節點數的子樹視作左子樹 $L(v)$ ，另一個子樹則為右子樹 $R(v)$ 。 $|T_n|$ 代表樹 T_n 的節點數。

我們定義任意兩個節點 u 和 v 其 hamming distance 為 $d(u, v)$ 。我們的嵌入方法將運用到格雷碼(Gray code)，任意連續兩個格雷碼的 hamming distance 為 1。我們定義 k 位元的格雷碼序列為 G_k ，它可由以下的遞迴函數建構而成：

$$\begin{cases} G_1 = (0, 1) \\ G_k = (0G_{k-1}, 1G_{k-1}^R) \end{cases}$$

其中 G_{k-1}^R 為 G_{k-1} 的逆向運算子。

圖一所示為 4 個位元所形成的 Gray code G_4 。我們定義其中第 i 個 Gray code 為 $\text{graycode}(i)$ 。所以圖一中 $\text{graycode}(3) = 0011$ ， $\text{graycode}(7) = 0101$ 。

NUMBER	GRAY CODE
1	0000
2	0001
3	0011
4	0010
5	0110
6	0111
7	0101
8	0100
9	1100
10	1101
11	1111
12	1110
13	1010
14	1011
15	1001
16	1000

圖一 . 4 位元的格雷碼 G_4

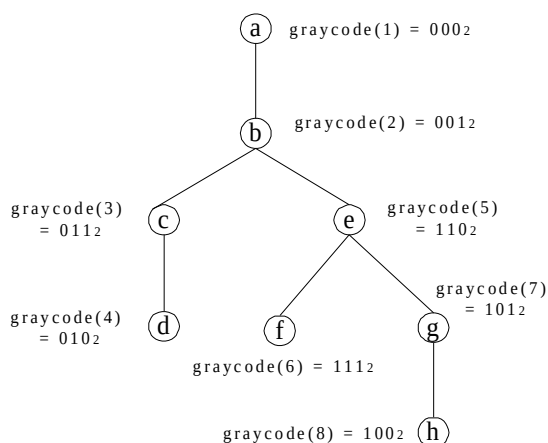
在嵌入的過程中，通常我們使用 4 個成本參數來評估嵌入的結果[15, 17]，它們分別是負載(load)、擴張(expansion)、膨脹(dilation)和擁塞

(congestion)。我們定義客圖 G 嵌入主圖 H 的負載是最多 G 上的節點映射至 H 上的單一節點之數量。擴張是 H 的節點數目除以 G 的節點數目。邊膨脹是 G 的一個邊映射至 H 上的路徑之長度，我們定義整個嵌入的膨脹值為 G 所有的邊膨脹值取最大值。邊擁塞則是 G 上的邊共用 H 上的單一邊之數量，整個嵌入的擁塞值是所有的邊擁塞值取最大值。負載可用以衡量工作的處理時間(the processing time of tasks)，擴張可用以衡量處理機的有效使用率(processor utilization)，邊膨脹可用以測量通訊的效率(communication performance)，而邊擁塞可用以測量訊息的等候延遲(the queueing delay of messages)。一個好的嵌入結果，是需要這 4 個參數能盡可能的小，但是在這 4 個參數之中，有些彼此間有相互消長(tradeoff)的關係，故在本篇論文中，我們固定住負載及擴張值等於 1，來降低平均膨脹係數。

三、嵌入二元樹演算法

之前 Chen 與 Stallmann 提出一個嵌入任意二元樹於超立方體的先序格雷碼演算法(Preorder-Gray-code Algorithm) [8]。此演算法固定由較小的左子樹開始追蹤，依照節點追蹤的順序給予格雷碼，圖二為應用此先序演算法將二元樹 T_3 嵌入三維的超立方體 Q_3 的結果。此演算法的追蹤對映有一個特點，就是比較小的子樹都會先被追蹤，我們再將此追蹤的順序編號轉成格雷碼並對映到超立方體上的節點。因為格雷碼具有相鄰兩碼其漢明距離(Hamming distance)為 1 的特性，因此任一節點與其左兒子節點的邊膨脹均為 1，但追蹤較右子樹的時候通常會造成較大的邊膨脹，如圖二的邊 $\{b, e\}$ 其邊膨脹係數是 $d(\text{graycode}(2), \text{graycode}(5)) = 3$ 。而邊 $\{e, g\}$ 的邊膨脹係數是 $d(\text{graycode}(2), \text{graycode}(5)) = 2$ 。

由於觀察到這個現象，因此我們提出一個新的演算法希望能降低原始前序嵌入法所造成右子樹膨脹係數過大的問題。當追蹤到根節點的右子樹的時候並不是直接用前序的方法，而是先比較前序追蹤與另一種中序追蹤，對於根節點與其右子樹的邊(edge)所造成的邊膨脹係數，再採用較小邊膨脹的追蹤方法，因此每回在追蹤右子樹時有可能採用前序追蹤，也有可能改為中序追蹤，但我們若採用中序追蹤，雖然可以改善根節與其右子樹的邊膨脹，但其右子樹內部的邊膨脹值可能反而會增加，為了要維持右子樹內部任一節點與其左兒子節點的邊膨脹係數保持為 1，我們採用反向前序追蹤的技巧如此將減少根節點與其右子樹這個邊的邊膨脹。



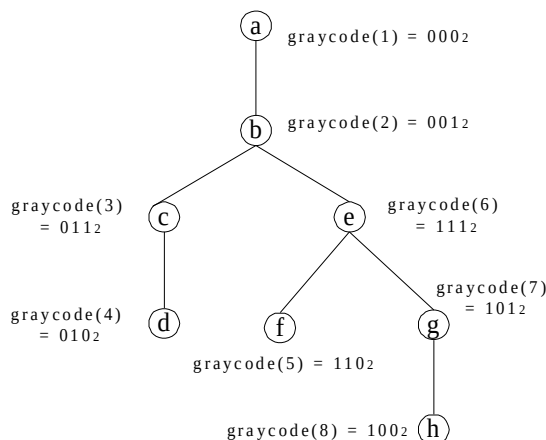
圖二 . Preorder-Gray-code 嵌入演算法

附錄為我們所提出的演算法，它增進強化了[8]的研究結果。此演算法 $PIG(T, i, j)$ 是將 $graycode(i)$ 到 $graycode(j)$ 依序指派到樹 T 中，且此樹所有節點的數目為 $j-i+1, i \leq j$ 。例如當二元樹 T_3 嵌入到超立方體 Q_3 時我們呼叫 $PIG(T_3, 1, 8)$ 。此演算法以根節點 v 為中心，首先追蹤根節點，之後比較其兩個子樹節點數目的多寡，令較小子樹為 $L(v)$ ，再先向左子樹 $L(v)$ 追蹤。我們演算法最主要的不同點在於追蹤右子樹的部份，在追蹤右子樹時，我們會比較直接用前序追蹤或是採用中序追蹤兩種不同的追蹤法所得到的膨脹係數，我們在追蹤右子樹時將採用比較過後膨脹係數較低的追蹤法。如此可以保證根節點與其左子樹的膨脹係數為 1，而根節點與其右子樹的膨脹係數值必定小於等於原始[8]演算法所得的值。

中序追蹤法 $Inorder(T, p, q)$ 將 $graycode(p)$ 到 $graycode(q)$ 依序指派到樹 T 中， $p \leq q$ 。但它並不是單純的中序追蹤，我們令 v 為 T 的根節點，即 $v = r(T)$ 。首先呼叫 $Inverse$ 程序去追蹤根節點 v 的左子樹，將 $graycode(p)$ 到 $graycode(p+|L(v)|-1)$ 指派給 $L(v)$ ，之後追蹤根節點 v ，將 $graycode(p+|L(v)|)$ 指派給節點 v ，最後呼叫 PIG 程序追蹤 v 的右子樹，將 $graycode(p+|L(v)|+1)$ 到 $graycode(q)$ 指派給 $R(v)$ 。而 $Inverse(T, y, x)$ 將 $graycode(y)$ 到 $graycode(x)$ 依序指派到樹 T 中， $y \geq x$ 。它是一種反向追蹤的程序，我們也可以把它看成是一種倒著順序的前序追蹤，即編號由最大一個編號開始，由子樹的根節點向下遞減指派，如圖三我們呼叫 $Inorder(R(b), 5, 8)$ ，則 $f(e) = graycode(6)$ ，之後呼叫 $Inverse(L(e), 5, 5)$ 則 $f(f) = graycode(5)$ 。 $graycode(M)$ 為一個函數，它傳回第 M 個格雷碼。此論文的演算法重點是 $Inorder(T, p, q)$ 追蹤程序，如果用 chen[8] 的方法， $\{b, e\}$ 的邊膨脹係數值是 3， $\{e, g\}$ 的邊膨脹係數是 2，而執行我們的

嵌入演算法將使膨脹係數降為 2，且 $\{e, g\}$ 的邊膨脹係數值減為 1。因此，執行我們的演算法不但減少 $\{b, e\}$ 的邊膨脹也同時減少 $\{e, g\}$ 的邊膨脹，而讓平均膨脹係數更小。

但若是採用 Inorder 及反向前序追蹤法，雖然可以保證此根節點與其右子樹的膨脹係數小於原始 Preorder-Gray-code 演算法的膨脹係數，但針對 $R(v)$ 後續的追蹤卻有可能發生其膨脹係數值較原始演算法為大的情形；即我們改善了較上階層節點對其右子樹的膨脹係數，但卻可能造成在較下階層節點與其右子樹節點的膨脹係數值變差。因此為了避免這樣的情況發生，我們必須遞迴地檢視比較嵌入右子樹的部份，在採反向前序追蹤程序 $Inverse(T, y, x)$ 時，追蹤右子樹的部份會再次比較反向前序追蹤或中序追蹤，相同的我們將採用膨脹係數較低的追蹤法，一旦採用中序追蹤法時將呼叫 $RInorder$ 程序，它與 Inorder 程序類似但因在反向追蹤程序中參數呼叫的方向性不同，因此無法直接用 Inorder 程序，遞迴地檢視比較右子樹嵌入將可改善進入反向前序追蹤時可能造成右子樹膨脹係數比原前序追蹤還大的情形。



圖三 . 利用我們的演算法將 T_3 嵌入到 Q_3

定理 1. 利用我們的演算法嵌入樹於超立方體其擴張係數為 1 且平均邊膨脹係數不會超過 2。

證明：令 T 為以 v 為根節點的二元樹， $|T| = n, l = |L(v)|, r = |R(v)|$ ，所以 $n = l + r + 1$ 。由格雷碼的特性[3]我們知道邊 $\{v, L(v)\}$ 邊膨脹永遠是 1，而邊 $\{v, R(v)\}$ 的邊膨脹會是小於或等於 $\lceil \log(l+1)+1 \rceil$ 。在先序中因為較小子樹總是先標

記，所以我們假設 $l \leq (n-1)/2$ 和 $r \geq (n-1)/2$ 。

我們定義 $D(n)$ 代表 n 個節點的樹 T 嵌入超立方體的所有邊膨脹係數總和，則

$$\begin{cases} D(1) = 0, \\ D(n) \leq D(r) + 1, \text{ if } l = 0 \\ D(n) \leq D(l) + D(r) + 1 + \lceil \log(l+1) + 1 \rceil, \text{ if } l \neq 0 \end{cases}$$

$$\begin{aligned} D(n) &\leq D(l) + D(r) + 1 + (\lceil \log(l+1) \rceil + 1) \\ &\leq 2l - \lceil \log(l+1) \rceil - 1 + 2r - \lceil \log(r+1) \rceil \\ &\quad - 1 + \lceil \log(l+1) \rceil + 2 \\ &= 2(l+r) - \lceil \log(r+1) \rceil \\ &\leq 2n - \left\lceil \log\left(\frac{n-1}{2} + 1\right) \right\rceil - 2 \\ &= 2n - \left\lceil \log\left(\frac{n+1}{2}\right) \right\rceil - 2 \\ &= 2n - \lceil \log(n+1) \rceil - 1 \end{aligned}$$

$$\text{則平均膨脹係數為 } \leq \left(\frac{2n - \lceil \log(n+1) - 1 \rceil}{n-1} \right) = 2$$

Q. E. D.

此平均膨脹係數的上限值雖然與[8]所提出的演算法相同，但因為我們的演算法額外改善每一倍節點與其右子樹的邊膨脹係數值，因此一般所得到的平均膨脹係數會遠比上限值小。

四、總結

針對樹狀結構已有很多演算法去解決各式各樣的問題。如何將樹狀結構嵌入超立方體中，使得這些在樹結構上相當有效率的演算法可以很容易地轉移至實際的超立方體平行電腦上是一個相當重要的問題。

本篇論文針對任意二元樹 T_n 嵌入到超方體 Q_n 提出了一個演算法，由 T_n 嵌入到 Q_n 在固定擴張及負載係數為 1 的情況下獲得比[8]更好的膨脹係數值。我們希望未來可以找到一個最佳膨脹係數的嵌入演算法。雖然目前我們無法證明此演算法為最佳的嵌入演算法，但我們相信此演算法已大幅降低膨脹係數，並且有可能再加以改善而得

到最佳的嵌入演算法，這將是我們未來的首要研究目標。

五、參考文獻

- [1] J. L. Bentley, and H. T. Kung, "A tree machine for searching problems," *Proceedings of International Conference on Parallel Processing*, pp. 257-266, 1979.
- [2] S. N. Bhatt, and I. C. F. Ipsen, "How to embed trees in hypercubes," *Yale University Res. Rep.*, RR-443, Dec. 1985.
- [3] S. N. Bhatt, F. R. K. Chung, F. T. Leighton, and A. L. Rosenberg, "Efficient embedding of trees in hypercubes," *SIAM J. Comput.*, vol. 21, pp. 151-162, Feb. 1992.
- [4] S. N. Bhatt, F. Chung, T. Leighton, and Rosenberg, "A. Optimal simulation of tree machines," *Proceedings of the 1986 27th Annual Symposium of Foundations of Computer Science*, pp. 274-282.
- [5] L. Bhuyan, and D. P. Agrawal, "Generalized hypercubes and hyperbus structures for a computer network," *IEEE Transactions on Computers*, C-33, pp. 323-333, 1984.
- [6] J. Blazewicz, and M. Drozdowski, "Scheduling divisible jobs on hypercubes," *Parallel Computing*, vol. 21, pp. 1945-1956, 1995.
- [7] Y. W. Chen, and K. L. Chung, "Improved fault-tolerant sorting algorithm in hypercubes," *Theoretical Computer Science*, vol. 255, pp. 649-658, 2001.
- [8] W. Chen, and M. F. M. Stallmann, "On Embedding Binary Trees into Hypercubes," *Journal of Parallel and Distributed Computing*, vol. 24, pp. 132-138, 1995.
- [9] A. M. Despain, and D. A. Patterson, "X-tree: A tree structured multiprocessor computer architecture," *Proceedings of the 5th Annual Symposium on Computer Architecture*, pp. 144-151, 1978.
- [10] S. R. Desphance, and R. Jenevein, "Scalability of binary trees on a hypercube," *Proceedings of International Conference on Parallel Processing*, pp. 661-668, 1986.
- [11] K. Efe, "Embedding mesh of trees in the hypercube," *Journal of Parallel and Distributed Computing*, vol. 11, pp. 222-230, 1991.
- [12] J. P. Hayes, T. N. Mudge, Q. F. Stout, S. Colley and J. Palmer, "A Microprocessor-based hypercube supercomputer," *IEEE Micro*,

- vol. 6, pp. 6-17, 1986.
- [13] C. T. Ho, and S. L. Johnsson, "Spanning balanced trees in Boolean cubes," *SIAM J. Sci. Statist. Comput.*, vol.10-4, pp. 607-630, July 1989.
- [14] E. Horowitz, and A. Zorat, "The binary tree as an interconnection network : Applications to multiprocessor systems and VLSI," *IEEE Transactions on Computers*, vol. 30, pp. 247-253, 1981.
- [15] F. T. Leighton, *Introduction to Parallel Algorithms and Architectures: Arrays, Trees and Hypercubes*, Morgan Kaufmann, San Mateo CA, 1992.
- [16] F. T. Leighton, "Parallel computing using mesh of trees," *Proceedings of the Workshop on Graph-Theoretic Concepts in Computer Science*, pp. 200-218, 1983.
- [17] I. L. Leiss, and H. N. Reddy, "Embedding complete binary trees into hypercubes," *Inform. Process. Lett.*, vol. 38, pp. 197-199, May 1991.
- [18] P. Mohapatra, "Dynamic real-time task scheduling on hypercubes," *Journal of Parallel and Distributed Computing*, vol. 46, pp. 91-100, 1997.
- [19] D. A. Read and R. M. Fujimoto, *Multi-computer Network: Message-Based Parallel Processing*, MIT Press, Cambridge MA, 1987.
- [20] Y. Saad and M. H. Schultz, "Topological properties of hypercubes," *IEEE Transactions on Computers*, vol. C-37, pp. 867-872, 1988.
- [21] A. Wagner, "Embedding arbitrary binary trees in a hypercube," *Journal of Parallel and Distributed Computing*, vol. 7, pp. 503-520, 1989.
- [22] A. Y. Wu, "Embedding of tree networks into hypercubes," *Journal of Parallel and Distributed Computing*, vol. 2, pp. 238-249, 1985.
- [23] J. Wu, "Optimal broadcasting in hypercubes with link faults using limited global information," *Journal of Systems Architecture*, vol. 42, pp. 367-380, 1996.
- [24] J. Wu, E. B. Fernandez, and Y. Luo, "Embedding of Binomial Trees in Hypercubes with Link Faults," *Journal of Parallel and Distributed Computing*, vol. 54, pp. 59-74, 1998.

附錄：

Procedure PIG(T, i, j)

```

/* 將 graycode(i)到 graycode(j)指派給樹 T,
i≤j */
{
    v = r(T);
    f(v) = graycode(i);
    /* 將 graycode(i)指派給節點 v */
    if i = j then return
    else
        i = i + 1

    Let the children of v be L(v) and R(v) with
        |L(v)| ≤ |R(v)|;
    if |L(v)| = 0 then L(v) = R(v);
    PIG(L(v), i, i + |L(v)| - 1);
    if d(f(v), graycode(|L(v)| + 2)) ≤
        d(f(v), graycode(|L(v)| + |L(R(v))| + 2))
    then PIG(R(v), i + |L(v)|, j);
    else
        j = i + |L(v)| + |L(R(v))|;
        f(v) = graycode(j);
        j = j - 1;
        Inorder(R(v), i + |L(v)|, j);
}

Procedure Inorder( $T, p, q$ ) /*  $p \leq q$  */
{
    v = r(T);
    i = p + |L(v)|;
    Inverse(L(v), i - 1, p);
    f(v) = graycode(i);
    PIG(R(v), i + 1, q);
}

Procedure Inverse( $T, y, x$ ) /*  $x \leq y$  */
{
    v = r(T);
    f(v) = graycode(y);
    if y = x then return
    else
        y = y - 1

    Let the children of v be L(v) and R(v) with
        |L(v)| ≤ |R(v)|;
    if |L(v)| = 0 then L(v) = R(v);
    Inverse(L(v), y, y - |L(v)| + 1);
    if d(f(v), graycode(|y - |L(v)||)) ≤
        d(f(v), graycode(|y - |L(v)| + |L(R(v))|))
    then Inverse(R(v), y - |L(v)|, x);
    else
        x = y - |L(v)| - |L(R(v))|;
        f(v) = graycode(x);
        x = x + 1;
        RInorder(R(v), x, y - |L(v)|);
}

```

```

Procedure RInorder ( $T, a, b$ ) /*  $a \leq b$  */
{
   $v = r(T)$ ;
   $i = b - |L(v)|$ ;
  Inverse( $R(v), i+1, a$ );
   $f(v) = \text{graycode}(i)$ ;
  PIG( $L(v), i+1, i+|L(v)|$ );
}

```

Ideal and Perfect Secret Sharing for Fault-Tolerant Distributed Access Control of Secure Data via Nondominated Bicoterie

Jehn-Ruey Jiang
Department of Information Management
Hsuan Chuang University
Hsin-Chu, 300, Taiwan
jrjiang@hcu.edu.tw

Abstract

A perfect secret sharing scheme permits a secret to be shared among participants so that only qualified subsets of participants can reconstruct the secret, but any unqualified subset has absolutely no information about the secret. In this paper, we first show that the general secret sharing scheme with a coterie access structure is perfect if the coterie is nondominated (ND). We then propose a perfect secret sharing scheme with an ND bicoterie access structure for the access control of secure data. The proposed scheme has the merits of having two types of quorums: one type to grant and the other to revoke the access of the secure data. This provides us more flexibility for the distributed access control of the secure data. Since there are a large number of different quorums consisting of few participants, the proposed scheme is fault-tolerant in the sense that it still works even when some participants are not available. We also propose an ideal and perfect secret sharing scheme with an ND grid bicoterie access structure. The proposed scheme is optimal (ideal) in the sense that its shares have the same size as the secret.

Keywords: bicoterie, coterie nondomination, distributed access control, data security, fault-tolerance, secret sharing

1 Introduction

Secret sharing has received considerable attention since it was first introduced by Blakley [4] and Shamir [25]. It deals with the problem of how to securely share a secret among a group of participants and has many applications, such as secure data access control [21], secure multiparty computation [2], visual cryptography [20] and network auction [19], and so on. In a secret sharing scheme, a secret is divided into pieces, each of which is called a *share* and is held by a participant. A secret sharing scheme must ensure that only a qualified subset of participants can reconstruct the secret. If an unqualified subset of

participants cannot obtain any information of the secret from their shares, then the secret sharing scheme is called *perfect*. The *information rate* of a secret sharing scheme is defined to be the ratio of the secret size to the largest share size. It has been shown in [10] that any perfect secret sharing scheme must have information rate ≤ 1 . A perfect secret sharing scheme is said to be *ideal* if it has information rate of 1.

The collection of qualified subsets of participants is called an *access structure* [3, 13]. Many secret sharing schemes with different access structures have been proposed in the literature [3, 5-9, 13, 23, 26-27]. In [2, 17, 21], secret sharing schemes with *coterie* access structures are proposed. A coterie [12] is a collection of subsets of participants, where each subset is called a *quorum* and any pair of quorums must intersect. The coterie is extensively studied in the literature of distributed systems because it is very useful for solving the distributed mutual exclusion [1, 12, 14, 16, 18, 24] and replica control [15-16] problems, etc. In [21], secret sharing schemes with coterie access structures are further applied to granting and revoking the access of secure data, such as decryption keys and signatures, etc. By distributing shares of secure data over wide-spread servers, shares from a complete quorum of servers can restore the secure data. And after access authorization is revoked, an unauthorized user will never be able to restore the data even if some servers still consider him/her authorized.

A general secret sharing scheme with ordinary coterie access structure was proposed in [21]. However, as we will show later, this scheme is not always perfect. In this paper, to make the work in [21] more complete, we show that the general secret sharing scheme with the coterie access structure is perfect if the coterie is *nondominated* (ND). The ND coterie [12, 14, 22] is an important type of coterie, which we will introduce later.

Specific secret sharing schemes with many types of coterie access structures was also proposed in [2, 17, 21]. Those access structures are constructed on the basis of nondominated *wr-coterie* [17], the crumbling wall coterie [24],

the tree coterie [1], and the finite projective plane coterie [18], etc. All the abovementioned coterie are ND, so they all render the secret sharing scheme perfect. However, only the last two coterie render the secret sharing scheme *ideal*. A secret sharing scheme is called ideal if the size of shares is the same as that of the secret. It has been shown in [10] that all perfect secret sharing scheme must have share size as large as the secret size (i.e., information rate ≤ 1). Certainly, small shares are preferred because the security of the system degrades as the amount of share increases and small shares can be distributed and stored more efficiently.

In this paper, we first show that the general secret sharing scheme [21] with the coterie access structure is perfect if the coterie is ND. We then present a perfect secret sharing scheme with ND bicoterie access structures for the access control of secure data. A bicoterie $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$ is an ordered pair of collections of sets, where each set is called a quorum and any quorum in $\mathcal{P}$ intersects any quorum in $\mathcal{Q}$. The secret sharing scheme with bicoterie access structures has the merits of having two types of quorums: one type to grant and the other to revoke the access of the secure data. This provides us more flexibility for the distributed access control of the secure data. Since there are a large number of different quorums consisting of few participants, the proposed scheme is fault-tolerant in the sense that it still works even when some participants are not available. At last, we propose an ideal and perfect secret sharing scheme with a specific access structure, which is constructed on the basis of an ND bicoterie — the grid bicoterie.

The rest of this paper is organized as follows. In Section 2, we elaborate some preliminaries about secret sharing schemes with the coterie and the bicoterie access structures. In Section 3, we introduce some theorems about perfect secret sharing schemes with the coterie and the bicoterie access structures. In Section 4, we propose an ideal and perfect secret sharing scheme with ND grid bicoterie access structure. Finally, we give some concluding remarks in Section 5.

2 Preliminaries

Let $U=\{u_1, \dots, u_n\}$ be the universal set of all participants $u_1, \dots, u_n$. Below, we introduce the definitions of coterie, bicoterie and the nondominated (ND) concept of them. We also introduce the preliminaries about secret sharing schemes with access structures constructed on the basis of coterie and bicoterie.

Definition 1. Coterie

A coterie [12] $\mathcal{C}$ is a collection of subsets of U , where each subset is called a quorum satisfying the following properties:

Intersection Property: $\forall G, H \in \mathcal{C}: G \cap H \neq \emptyset$;

Minimality Property: $\forall G, H \in \mathcal{C}: G \not\subseteq H$.

Definition 2. Bicoterie

A bicoterie [11] $\mathcal{B}$ is an ordered pair $(\mathcal{P}, \mathcal{Q})$, where $\mathcal{P}$ and $\mathcal{Q}$ are collections of subsets (called quorums) of U . A bicoterie $\mathcal{B}$ should satisfy the following properties:

Intersection Property: $\forall G \in \mathcal{P}, H \in \mathcal{Q}: G \cap H \neq \emptyset$;

Minimality Property: $[\forall G, H \in \mathcal{P}: G \not\subseteq H]$ and $[\forall G, H \in \mathcal{Q}: G \not\subseteq H]$.

For example, let $\mathcal{P}=\{\{u_1, u_2, u_3\}, \{u_1, u_2, u_4\}, \{u_3, u_4\}\}$ and $\mathcal{Q}=\{\{u_1, u_3\}, \{u_1, u_4\}, \{u_2, u_3\}, \{u_2, u_4\}, \{u_3, u_4\}\}$. Then, $(\mathcal{P}, \mathcal{Q})$ is a bicoterie since it satisfies the intersection and the minimality properties.

Definition 3. Domination of Coterie

Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be two coterie. We say that $\mathcal{C}_2$ dominates $\mathcal{C}_1$ if and only if

- (1) $\mathcal{C}_1 \neq \mathcal{C}_2$;
- (2) $\forall G \in \mathcal{C}_1, \exists H \in \mathcal{C}_2: H \subseteq G$.

Definition 4. Domination of Bicoterie

Let $\mathcal{B}_1=(\mathcal{P}_1, \mathcal{Q}_1)$ and $\mathcal{B}_2=(\mathcal{P}_2, \mathcal{Q}_2)$ be two bicoterie. We say $\mathcal{B}_2$ dominates $\mathcal{B}_1$ if and only if

- (1) $\mathcal{B}_1 \neq \mathcal{B}_2$, i.e., $\mathcal{P}_1 \neq \mathcal{P}_2$ and/or $\mathcal{Q}_1 \neq \mathcal{Q}_2$;
- (2) $\forall G \in \mathcal{P}_1, \exists H \in \mathcal{P}_2: H \subseteq G$;
- (3) $\forall G \in \mathcal{Q}_1, \exists H \in \mathcal{Q}_2: H \subseteq G$.

For example, let $\mathcal{P}_1=\{\{u_1, u_2, u_3\}, \{u_1, u_2, u_4\}, \{u_1, u_3, u_4\}, \{u_2, u_3, u_4\}\}$, $\mathcal{Q}_1=\{\{u_1, u_3\}, \{u_1, u_4\}, \{u_2, u_3\}, \{u_2, u_4\}\}$, $\mathcal{P}_2=\{\{u_1, u_2, u_3\}, \{u_1, u_2, u_4\}, \{u_3, u_4\}\}$, and $\mathcal{Q}_2=\{\{u_1, u_3\}, \{u_1, u_4\}, \{u_2, u_3\}, \{u_2, u_4\}, \{u_3, u_4\}\}$. Then, $\mathcal{B}_1=(\mathcal{P}_1, \mathcal{Q}_1)$ and $\mathcal{B}_2=(\mathcal{P}_2, \mathcal{Q}_2)$ are bicoterie and $\mathcal{B}_2$ dominates $\mathcal{B}_1$.

Definition 5. Nondomination of Coterie and Bicoterie

A coterie (bicoterie) is *nondominated* (ND) if and only if no other coterie (bicoterie) can dominate it.

Definition 6. Access Structures

An access structure Γ is the collection of subsets of U , which satisfying the monotone increasing property, that is, if $G \in \Gamma$ and $G \subseteq H$, then we have $H \in \Gamma$.

Definition 7. Coterie Access Structures

Let $\mathcal{C}$ be a coterie. The coterie access structure based on $\mathcal{C}$, which is denoted with $\Gamma(\mathcal{C})$, is defined as the collection of all supersets of quorums in $\mathcal{C}$. That is, $\Gamma(\mathcal{C}) = \{H \mid H \supseteq G, G \in \mathcal{C}\}$.

Definition 8. Bicoterie Access Structures

Let $\mathcal{B} = (\mathcal{P}, \mathcal{Q})$ be a bicoterie. The bicoterie access structure based on $\mathcal{B}$, which is denoted with $\Gamma(\mathcal{B})$, is defined as the collection of all supersets of quorums in $\mathcal{P}$. That is, $\Gamma(\mathcal{B}) = \{H \mid H \supseteq G, G \in \mathcal{P}\}$. Note that $\Gamma(\mathcal{B}) = \Gamma(\mathcal{P})$ and we will use this fact in the theorem proofs in the next section.

Definition 9. Secret Sharing Schemes

Let S be the domain of all possible secrets. A secret sharing scheme is a mapping $\Pi: S \times E \rightarrow S_1 \times \dots \times S_n$, where E is the domain of random numbers for splitting the secret, and S_i is the domain of shares distributed to participant u_i , where u_i belongs to $U = \{u_1, \dots, u_n\}$, the universal set of all participants. We say that Π is a secret sharing scheme with access structure Γ if the following reconstruction property is satisfied:

(S1) Reconstruction Property:

Let $H = \{u_{\sigma(1)}, \dots, u_{\sigma(k)}\}$ be any qualified subset of participants in Γ , where $\sigma(1), \dots, \sigma(k)$ are distinct and $1 \leq \sigma(1), \dots, \sigma(k) \leq n$. Associated with H , there is a function $F_H: S_{\sigma(1)} \times \dots \times S_{\sigma(k)} \rightarrow S$, such that for every pair $(s, \varepsilon) \in S \times E$, if $\Pi(s, \varepsilon) = (s_1, \dots, s_n)$ then $F_H(s_{\sigma(1)}, \dots, s_{\sigma(k)}) = s$. That is, the secret can be reconstructed by shares from any subset of Γ .

Further, Π is said to be *perfect* if the following property is satisfied:

(S2) Perfect Property:

Let $H \notin \Gamma$ be an unqualified subset of participants. And let a and $b \in S$ be any two secrets. For every possible collection of shares $\{s_i \mid u_i \in H\}$, we have $\text{prob}(a \mid \{s_i \mid u_i \in H\}) = \text{prob}(b \mid \{s_i \mid u_i \in H\})$. That is, from the set of shares of unqualified subset of participants, the probability of deriving the secret to be a , and the probability of deriving the secret to be b is equal, which implies that no information of the secret is revealed.

3 Secret Sharing Schemes with Coterie and Bicoterie Access Structures

In this section, we first show that the general secret sharing scheme with coterie access structure $\Gamma(\mathcal{C})$ is perfect if $\mathcal{C}$ is ND to make the work in [21] more complete. We then propose a perfect secret sharing scheme with ND bicoterie access structures.

3.1 The Secret Sharing Scheme with Coterie Access Structures

In [21], a general secret sharing scheme with coterie access structure $\Gamma(\mathcal{C})$ was proposed. This scheme can be used to distribute access control of the secure data so that after access authorization is revoked, an unauthorized user will never be able to restore the data even if some servers still consider him/her authorized. Since there are a large number of different quorums consisting of few participants, the proposed scheme is fault-tolerant in the sense that it still works even when some participants are not available. However, the secret sharing scheme mentioned is not perfect in the sense that even an unauthorized subset of participants may reconstruct the secret if $\mathcal{C}$ is dominated. Below, we show that the secret sharing scheme with coterie access structure $\Gamma(\mathcal{C})$ is perfect if $\mathcal{C}$ is ND.

We first describe the secret sharing scheme Π proposed in [21] as follows. Given a coterie $\mathcal{C} = \{G_1, \dots, G_m\}$ and a secret $v \in S$. We can split v into $v_1, \dots, v_m$ randomly according to a random number $\varepsilon \in E$ such that $v = v_1 + \dots + v_m$. We call each of $v_1, \dots, v_m$ the *partition* of v . Each v_j , $1 \leq j \leq m$, is associated with a quorum $G_j \in \mathcal{C}$. The shares $s_i(v) \in S_i$ of secret v distributed to participant u_i is defined as the set of all partitions associated with quorums that contain u_i . More precisely, $s_i(v) = \{v_j \mid u_i \in G_j, 1 \leq j \leq m\}$ for $1 \leq i \leq n$. In [21], the mapping $\Pi(v, \varepsilon) = (s_1(v), \dots, s_n(v))$ of $S \times E \rightarrow S_1 \times \dots \times S_n$ is shown to be a secret sharing scheme with the coterie access structure $\Gamma(\mathcal{C})$. Below, we further show that Π is perfect if $\mathcal{C}$ is ND.

We first introduce a theorem in [12], which is concerned with the properties of nondominated coterie.

Theorem 1. Let $\mathcal{C}$ be a coterie under the universal set U . $\mathcal{C}$ is dominated if and only if there exists a subset H of U such that

- 1) $\forall G \in \mathcal{C}: G \not\subseteq H$.
- 2) $\forall G \in \mathcal{C}: G \cap H \neq \emptyset$.

Based on theorem 1, we prove theorem 2 below. Before proceeding to theorem 2, we remind the reader to note the following facts: $[G \cap H = \emptyset \text{ iff } G \subseteq \overline{H}]$ and $[G \cap H = \emptyset \text{ iff } H \subseteq \overline{G}]$, or equivalently, $[G \cap H \neq \emptyset \text{ iff } G \not\subseteq \overline{H}]$ and $[G \cap H \neq \emptyset \text{ iff } H \not\subseteq \overline{G}]$.

Theorem 2. Let $\mathcal{C}$ be a coterie under the universal set U , and let $\Gamma(\mathcal{C})$ be the access structure based on $\mathcal{C}$. It follows that $\mathcal{C}$ is dominated if and only if there exists a subset H of U such that $H \notin \Gamma(\mathcal{C})$ and $\overline{H} \notin \Gamma(\mathcal{C})$.

Proof:

(if part) $H \notin \Gamma(\mathcal{C})$ implies $\forall G \in \mathcal{C}: G \not\subseteq H$. And $\overline{H} \notin \Gamma(\mathcal{C})$ implies $\forall G \in \mathcal{C}: G \not\subseteq \overline{H}$, which in turn implies $\forall G \in \mathcal{C}: G \cap H \neq \emptyset$. By theorem 1, we have that $\mathcal{C}$ is dominated.

(only if part) By theorem 1, if $\mathcal{C}$ is dominated, then there exists a subset H of U such that $\forall G \in \mathcal{C}: G \not\subseteq H$, which implies $H \notin \Gamma(\mathcal{C})$. By theorem 1, if $\mathcal{C}$ is dominated, then $\forall G \in \mathcal{C}: G \cap H \neq \emptyset$, which implies $\forall G \in \mathcal{C}: G \not\subseteq \overline{H}$; i.e., $\overline{H} \notin \Gamma(\mathcal{C})$. $\square$

Below, we introduce the following corollaries, which are direct consequences of theorem 2.

Corollary 1. Let $\mathcal{C}$ be a coterie under the universal set U , and let $\Gamma(\mathcal{C})$ be the access structure based on $\mathcal{C}$. $\mathcal{C}$ is ND if and only $G \in \Gamma(\mathcal{C})$ or $\overline{G} \in \Gamma(\mathcal{C})$ for every subset G of U .

Corollary 2. Let $\mathcal{C}$ be a coterie under the universal set U , and let $\Gamma(\mathcal{C})$ be the access structure based on $\mathcal{C}$. If $\mathcal{C}$ is ND then $[G \in \Gamma(\mathcal{C}) \text{ implies } \overline{G} \notin \Gamma(\mathcal{C})]$, $[G \in \Gamma(\mathcal{C}) \text{ implies } \overline{G} \notin \Gamma(\mathcal{C})]$, $[G \notin \Gamma(\mathcal{C}) \text{ implies } \overline{G} \in \Gamma(\mathcal{C})]$, and $[G \notin \Gamma(\mathcal{C}) \text{ implies } \overline{G} \in \Gamma(\mathcal{C})]$.

We can now prove that Π is a perfect secret sharing scheme with the coterie access structure $\Gamma(\mathcal{C})$ if $\mathcal{C}$ is ND.

Theorem 3. Let Π be a mapping as described earlier. If $\mathcal{C}$ is ND, then Π is a perfect secret sharing scheme with the coterie access structure $\Gamma(\mathcal{C})$.

Proof :

The reconstruction property of Π has been shown in [21], below we show its perfect property:

Let $H = \{u_{\sigma(1)}, \dots, u_{\sigma(k)}\} \notin \Gamma(\mathcal{C})$, where $\sigma(1), \dots, \sigma(k)$ are distinct and $1 \leq \sigma(1), \dots, \sigma(k) \leq n$. Because $\mathcal{C}$ is ND, we have $\overline{H} \in \Gamma(\mathcal{C})$ by corollary 2. It follows that there exists a quorum $G_j \in \mathcal{C}$ such that $G_j \subseteq \overline{H}$, which in turn implies $G_j \cap H = \emptyset$. Only the members of G_j own the partition v_j . So, no participant in H obtains v_j .

Let a and b be two different secrets. And let $\Pi(a, \mathcal{E}^a) = (s_1^a, \dots, s_n^a)$ be a secret split of a according to $\mathcal{E}^a$ such that $F_H(s_{\sigma(1)}^a, \dots, s_{\sigma(k)}^a) = a$. We can form another secret split $\Pi(b, \mathcal{E}^b) = (s_1^b, \dots, s_n^b)$ such that $F_H(s_{\sigma(1)}^b, \dots, s_{\sigma(k)}^b) = b$. The secret split $(s_1^b, \dots, s_n^b)$ can be derived from $(s_1^a, \dots, s_n^a)$ by substituting the partition v_j with $v_j + (a - b)$. Since $G_j \cap H = \emptyset$ and the substitution of v_j only affect the participants in G_j , the projection of $(s_1^a, \dots, s_n^a)$ and $(s_1^b, \dots, s_n^b)$ on participants of H are identical. So, we have

$prob(a | \{s_i(v) | u_i \in H\}) = prob(b | \{s_i(v) | u_i \in H\})$. That means that the shares from participants in H can not reveal any information of the secret value v .

$\square$

3.2 The secret sharing scheme with ND bicoterie access structures

Given a bicoterie $\mathcal{B} = (\mathcal{P}, \mathcal{Q})$ and a secret value $v \in S$, where $\mathcal{P} = \{H_1, \dots, H_l\}$ and $\mathcal{Q} = \{G_1, \dots, G_m\}$. We can split v into $v_1, \dots, v_m$ randomly according to a random number $\varepsilon \in E$, such that $v = v_1 + \dots + v_m$. We call each of $v_1, \dots, v_m$ the *partition* of v . Each v_j , $1 \leq j \leq m$, is associated with a quorum $G_j \in \mathcal{Q}$. The shares $s_i(v) \in S_i$ of secret v distributed to participant u_i is defined as the set of all partitions associated with $\mathcal{Q}$'s quorums that contain u_i . More precisely, $s_i(v) \equiv \{v_j | u_i \in G_j, 1 \leq j \leq m\}$ for $1 \leq i \leq n$. Below, we show that $\Sigma(v, \varepsilon) = (s_1(v), \dots, s_n(v))$ of $S \times E \rightarrow S_1 \times \dots \times S_n$ is a perfect secret sharing scheme with the bicoterie access structure $\Gamma(\mathcal{B})$ if $\mathcal{B}$ is ND.

Below, we introduce a theorem [22] about ND bicoterie.

Theorem 4. Let $\mathcal{B} = (\mathcal{P}, \mathcal{Q})$ be a bicoterie under the universal set U . $\mathcal{B}$ is dominated if and only if there exists a subset H of U such that

- 1). $\forall G \in \mathcal{P}: G \not\subseteq H$
- 2). $\forall G \in \mathcal{Q}: G \cap H \neq \emptyset$.

On the basis of theorem 4, we prove the following theorem.

Theorem 5. Let $\mathcal{B} = (\mathcal{P}, \mathcal{Q})$ be a bicoterie under the universal set U . Also let $\Gamma(\mathcal{P})$ and $\Gamma(\mathcal{Q})$ be the access structures based on $\mathcal{P}$ and $\mathcal{Q}$, respectively. It follows that $\mathcal{B}$ is dominated if and only if there exists a subset H of U such that $H \notin \Gamma(\mathcal{P})$ and $\overline{H} \notin \Gamma(\mathcal{Q})$.

Proof:

(if part) $H \notin \Gamma(\mathcal{P})$ implies $\forall G \in \mathcal{P}: G \not\subseteq H$. And $\overline{H} \notin \Gamma(\mathcal{Q})$ implies $\forall G \in \mathcal{Q}: G \not\subseteq \overline{H}$, which in turn implies $\forall G \in \mathcal{Q}: G \cap H \neq \emptyset$. By theorem 4, we have that $\mathcal{B}$ is dominated.

(only if part) By theorem 4, if $\mathcal{B}$ is dominated, then there exists a subset H of U such that $\forall G \in \mathcal{P}: G \not\subseteq H$, which implies $H \notin \Gamma(\mathcal{P})$. By theorem 4, we also have $\forall G \in \mathcal{Q}: G \cap H \neq \emptyset$, which implies $\forall G \in \mathcal{Q}: G \not\subseteq \overline{H}$; i.e., $\overline{H} \notin \Gamma(\mathcal{Q})$. $\square$

Below, we introduce the following corollaries, which are direct consequences of theorem 5.

Corollary 3. Let $\mathcal{B} = (\mathcal{P}, \mathcal{Q})$ be a bicoterie under the universal set U . Also let $\Gamma(\mathcal{P})$ and $\Gamma(\mathcal{Q})$ be the access structures based on $\mathcal{P}$ and $\mathcal{Q}$, respectively.

$\mathcal{B}$ is ND if and only if $G \in \Gamma(\mathcal{P})$ or $\overline{G} \in \Gamma(\mathcal{Q})$ for every subset G of U .

Corollary 4. Let $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$ be a bicoterie under the universal set U . Also let $\Gamma(\mathcal{P})$ and $\Gamma(\mathcal{Q})$ be the access structures based on $\mathcal{P}$ and $\mathcal{Q}$, respectively. If $\mathcal{B}$ is ND then $[G \in \Gamma(\mathcal{P}) \text{ implies } \overline{G} \notin \Gamma(\mathcal{Q})]$, $[G \in \Gamma(\mathcal{P}) \text{ implies } \overline{G} \notin \Gamma(\mathcal{Q})]$, $[G \notin \Gamma(\mathcal{P}) \text{ implies } \overline{G} \in \Gamma(\mathcal{Q})]$, and $[G \notin \Gamma(\mathcal{P}) \text{ implies } \overline{G} \in \Gamma(\mathcal{Q})]$.

We can now prove that Σ is a perfect secret sharing scheme with the bicoterie access structure $\Gamma(\mathcal{B})$ if $\mathcal{B}$ is ND.

Theorem 6. Let Σ be a mapping as described earlier. If $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$ is an ND bicoterie, then Σ is a perfect secret sharing scheme with the bicoterie access structure $\Gamma(\mathcal{B})$.

Proof :

(1) Reconstruction Property:

Let $H \in \Gamma(\mathcal{B}) = \Gamma(\mathcal{P})$. According to the intersection property of $\mathcal{B}$, we have that H intersects with every quorum in $\mathcal{Q}$. So, the participants in H can collect all partitions $v_1, \dots, v_m$ of v and reconstruct the secret value v by $v = v_1 + \dots + v_m$, where m is the cardinality of $\mathcal{Q}$.

(2) Perfect Property:

Let $H = \{u_{\sigma(1)}, \dots, u_{\sigma(k)}\} \in \Gamma(\mathcal{B}) = \Gamma(\mathcal{P})$, where $\sigma(1), \dots, \sigma(k)$ are distinct and $1 \leq \sigma(1), \dots, \sigma(k) \leq n$. Because $\mathcal{B}$ is ND, we have $\overline{H} \in \Gamma(\mathcal{Q})$ by corollary 4. It follows that there exists a quorum $G_j \in \mathcal{Q}$ such that $G_j \subseteq \overline{H}$, which in turn implies $G_j \cap H = \emptyset$. Only the members of G_j own the partition v_j . So, no participant in H obtains v_j .

Let a and b be two different secrets. And let $\Sigma(a, \mathcal{E}^a) = (s_1^a, \dots, s_n^a)$ be a secret split of a according to $\mathcal{E}^a$ such that $F_H(s_{\sigma(1)}^a, \dots, s_{\sigma(k)}^a) = a$. We can form another secret split $\Sigma(b, \mathcal{E}^b) = (s_1^b, \dots, s_n^b)$ such that $F_H(s_{\sigma(1)}^b, \dots, s_{\sigma(k)}^b) = b$. The secret split $(s_1^b, \dots, s_n^b)$ can be derived from $(s_1^a, \dots, s_n^a)$ by substituting the partition v_j with $v_j + (a - b)$. Since $G_j \cap H = \emptyset$ and the substitution of v_j only affect the participants in G_j , the projection of $(s_1^a, \dots, s_n^a)$ and $(s_1^b, \dots, s_n^b)$ on participants of H are identical. So, we have $\text{prob}(a | \{s_i(v) | u_i \in H\}) = \text{prob}(b | \{s_i(v) | u_i \in H\})$. That means that the shares from participants in H can not reveal any information of the secret value v . $\square$

4 An ideal and perfect secret sharing scheme with grid bicoterie access structures

In this section, we show an ideal and perfect secret sharing scheme with the grid bicoterie access structure. Below, we first introduce the construction of grid bicoterie.

Given an universal set U of participants $u_1, \dots, u_n$, we organize participants into an h -row, w -column grid, where $n = hw$ (h stands for the height of grid, and w , the width). Note that for the case of $n \neq hw$, we can just rule out the grid points that are not associated with any participant. Figure 1 is the illustration of organizing 12 participants into a 3-row, 4-column grid.

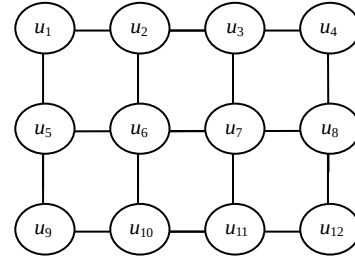


Figure 1. The illustration of organizing 12 participants into a 3-row, 4-column grid.

With the aid of the grid structure, the *grid bicoterie* $\mathcal{B}$ is defined to be an ordered pair $(\mathcal{P}, \mathcal{Q})$, where $\mathcal{Q}$ is a collection of all columns, and $\mathcal{P}$ is a collection of sets, each containing one participant from every column. Taking the grid structure of figure 1 for example, $\{u_1, u_5, u_9\}$ is a member of $\mathcal{Q}$, and $\{u_1, u_6, u_7, u_{12}\}$ is a member of $\mathcal{P}$.

In addition to the grid bicoterie, there are also ways to construct other bicoterie, which are called *extended grid bicoterie* in the following context. Below, we present two types of extended grid bicoterie:

(1) Type-1 Extended Grid Bicoterie $\mathcal{B}_1 = (\mathcal{P}_1, \mathcal{Q}_1)$: $\mathcal{Q}_1$ is the collection of sets, each containing r , $r \leq h$, participants from any column; $\mathcal{P}_1$ is the collection of sets, each containing $h - r + 1$ participants from every column.

Taking the grid structure in figure 1 for examples, for $r=2$, $\{u_2, u_6\}$ is a quorum of $\mathcal{Q}_1$ and $\{u_1, u_2, u_4, u_6, u_7, u_8, u_9, u_{11}\}$ is a quorum of $\mathcal{P}_1$. For $r=3$, $\{u_1, u_5, u_9\}$ is a quorum of $\mathcal{Q}_1$ and $\{u_1, u_6, u_7, u_{12}\}$ is a quorum of $\mathcal{P}_1$. Note that when $r=h$, a type-1 extended grid bicoterie is actually a grid bicoterie.

(2) Type-2 Extended Grid Bicoterie $\mathcal{B}_2 = (\mathcal{P}_2, \mathcal{Q}_2)$: $\mathcal{Q}_2$ is the collection of sets, each containing all participants from c , $c < w$, full columns; $\mathcal{P}_2$ is the collection of sets, each containing one participant

from $w-c+1$ columns.

Taking the grid structure in figure 1 for examples, for $c=2$, $\{u_2, u_4, u_6, u_8, u_{10}, u_{12}\}$ is a quorum of $\mathcal{Q}_2$ and $\{u_1, u_2, u_3\}$ is a quorum of $\mathcal{P}_2$. For $c=1$, $\{u_1, u_5, u_9\}$ is a quorum of $\mathcal{Q}_2$ and $\{u_1, u_6, u_7, u_{12}\}$ is a quorum of $\mathcal{P}_2$. Note that when $c=1$, a type-2 extended grid bicoterie is actually a grid bicoterie.

It is very easy to see $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$, $\mathcal{B}_1=(\mathcal{P}_1, \mathcal{Q}_1)$ and $\mathcal{B}_2=(\mathcal{P}_2, \mathcal{Q}_2)$ are all bicoterie since they all satisfy the minimality property and every member of $\mathcal{P}$, $\mathcal{P}_1$ and $\mathcal{P}_2$ intersects a member of $\mathcal{Q}$, $\mathcal{Q}_1$ and $\mathcal{Q}_2$, respectively. Below, we introduce a theorem proposed in [22], which is useful to prove that the grid and the extended grid bicoterie are ND. Before proceeding to this theorem, we first define some terms about bicoterie:

Let $\mathcal{Q}$ be a collection of subsets of U . A *transversal* of $\mathcal{Q}$ is a subset of U that intersects with all members of $\mathcal{Q}$. That is, if $H \subseteq U$ is a transversal of $\mathcal{Q}$, then $\forall G \in \mathcal{Q}: H \cap G \neq \emptyset$. The set of all minimal transversals of $\mathcal{Q}$ is denoted by $\text{Tr}(\mathcal{Q})$.

Theorem 7. Let $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$ be a bicoterie. Then, $\mathcal{B}$ is ND if and only if $\mathcal{P}$ is $\text{Tr}(\mathcal{Q})$.

On the basis of theorem 7, the reader can easily check that the grid and the extended grid bicoterie are ND (the proofs are trivial and omitted). Thus, they can all be applied to implement perfect secret sharing schemes. Below, we further show that the grid bicoterie can be applied to implement ideal and perfect secret sharing schemes.

Theorem 8. Let $\mathcal{B}=(\mathcal{P}, \mathcal{Q})$ be a grid bicoterie and let Σ be a function as defined earlier. Then, Σ is an ideal and perfect secret sharing scheme with the bicoterie access structure $\Gamma(\mathcal{B})$.

Proof:

Since the grid bicoterie is ND, Σ is a perfect secret sharing scheme with the bicoterie access structure $\Gamma(\mathcal{B})$ by theorem 6. Let u be a participant. Because u belongs to exactly one quorum G of $\mathcal{Q}$, the share of u contains only the partition associated with G , which is of the same size (the same domain) as the secret value. $\square$

5 Conclusion

In this paper, we have shown that the general secret sharing scheme proposed in [21] with the coterie access structure $\Gamma(\mathcal{C})$ is perfect if $\mathcal{C}$ is ND. We have also presented a secret sharing scheme with the bicoterie access structure for the access control of secure data. We have shown that the

grid bicoterie and the extended grid bicoterie can make the proposed secret sharing scheme perfect. We have also shown that the grid bicoterie further makes the proposed secret sharing scheme ideal. As we have mentioned, the secret sharing scheme with bicoterie access structures has the merits of having two types of quorums: one type to grant and the other to revoke the access of the secure data. This provides us more flexibility for the access control of the secure data. Since there are a large number of different quorums consisting of few participants, the proposed scheme is fault-tolerant in the sense that it still works even when some participants are not available. By grid bicoterie and extended grid bicoterie, we have even more flexibility on secure data access control in the sense that we can adjust h and w (the number of rows and columns of the grid) according to security requirements. For example, we can adjust w to be larger so that more pieces of shares are needed to reconstruct the secret value, and meanwhile, fewer servers can revoke the access authorization. In the future, we plan to investigate the effects of choosing different h and w and to investigate possible applications of secret sharing schemes with bicoterie access structures.

References

- [1] D. Agrawal and A. El-Abbadi, "An efficient and fault-tolerant solution for distributed mutual exclusion," *ACM Trans. Comp. Sys.*, vol. 9, no. 1, pp. 1- 20, 1991.
- [2] D. Beaver and A. Wool, "Quorum-based secure multi-party computation," in *Proc. of Eurocrypt'98*, LNCS 1403, pp. 375-390, 1998.
- [3] J. Benaloh and J. Leichter, "Generalized secret sharing and monotone functions," in *Proc. of Crypto'88*, LNCS 403, pp. 27-36. Springer-Verlag, 1988.
- [4] G. R. Blakely, "Safeguarding cryptographic keys," in *Proc. AFIPS*, pp. 313-317, 1979.
- [5] C. Blundo, A. De Santis, D. R. Stinson, and U. Vaccaro, "Graph decomposition and secret sharing schemes," *J. Cryptology*, vol. 8, pp. 39-64, 1995.
- [6] C. Blundo, A. De Santis, L. Gargano and U. Vaccaro, "On the information rate of secret sharing schemes," *Theoretical Computer Science*, vol. 154, no. 2, pp. 283-306, 1996.
- [7] E. F. Brickell, "Some ideal secret sharing schemes," *J. Combin. Math. And Combin. Comput.*, vol. 9, pp.105-113, 1989.

- [8] E. F. Brickell and D. M. Davenport, "On the classification of ideal secret sharing scheme," *Journal of Cryptology*, vol. 4, pp. 123-134, 1991.
- [9] E. F. Brickell and D. R. Stinson, "Some improved bounds on the information rate of perfect secret sharing schemes," *Journal of Cryptology*, vol. 5, pp. 153-166, 1992.
- [10] R. M. Capocelli, A. De Santis, L. Gargano, and U. Vaccaro, "On the size of shares for secret sharing schemes," *Journal of Cryptology*, vol. 6, pp. 157-169, 1993.
- [11] A. Fu, "Enhancing concurrency and availability for database systems," *Ph. D. Dissertation*, Simon Fraser Univ., 1990.
- [12] H. Garcia-Molina and D. Barbara, "How to assign votes in a distributed system," *Journal of the ACM*, vol. 32, no. 4, pp. 841-860, 1985.
- [13] M. Ito, A. Saito, and T. Nishizeki, "Secret sharing schemes realizing general access structure," in *Proc. IEEE Global Telecommunication Conf. (Globecomm 87)*, pp. 99-102, 1987.
- [14] J.-R. Jiang, "Quorum structures for fault-tolerant distributed mutual exclusion," *Ph.D. Dissertation*, Tsing-Hua University, Taiwan, 1995.
- [15] J.-R. Jiang, "The column protocol: a high availability and low message cost solution for managing replicated data," *International Journal of Information Systems*, vol. 20, no. 8, pp. 687-696, 1995.
- [16] J.-R. Jiang, "A framework for fault-tolerant distributed mutual exclusion and replica control using grid structures," in *Proc. Of the 8th International Conference on Parallel and Distributed Systems*, pp.311-315, Chicago, IL, 1996.
- [17] J.-R. Jiang, "Secret sharing via nondominated write-read coterie," in *Proc. of 13th International Conference on Parallel and Distributed Computing and Systems*, pp. 561-565, Aug. 2001.
- [18] M. Meakawa, "A $\sqrt{N}$ algorithm for mutual exclusion in decentralized system," *ACM Trans. Comp. Sys.*, vol. 3, no. 2, pp.145-159, 1985.
- [19] A. Menezes, P. Oorschot and S. Vanstone, "Secret sharing," book section of *Handbook of Applied Cryptography*, pp. 524-540, CRC Press, 1996.
- [20] M. Naor and A. Shamir, "Visual cryptography," in *Proc. of Eurocrypt'94*, pp. 1-12, 1994.
- [21] M. Naor and A. Wool, "Access control and signature via quorum secret sharing," *IEEE Trans. Parall. and Distrib. Syst.*, vol. 9, no. 9, pp. 909-922, 1998.
- [22] M. L. Neilsen, "Quorum structures in distributed systems," *Ph. D. Dissertation*, Kansas State University, 1992.
- [23] C. Padro, G. Saez, "Secret sharing schemes with bipartite access structure," *IEEE Trans. on Information Theory*, vol. 46, no. 7, pp.2596-2604, 2000.
- [24] D. Peleg and A. Wool, "Crumbling wall: a class of practical and efficient quorum system," *Distributed Computing*, vol. 10, no. 2, pp. 87-98, 1997.
- [25] A. Shamir, "How to share a secret," *Comm. ACM*, vol. 22, no. 11, pp. 612-613, 1979.
- [26] D. R. Stinson, "Decomposition constructions for secret sharing schemes," *IEEE Trans. on Information Theory*, vol. 40, pp.118-125, 1994.
- [27] H.-M. Sun and S.-P. Shieh, "Constructing perfect secret sharing schemes for general and uniform access structures," *Journal of Information Science and Engineering*, vol. 15, pp. 679-689, 1999.

On the Selection of Robust Tag SNPs

Yao-Ting Huang and Kun-Mao Chao

Department of Computer Science and Information Engineering

National Taiwan University

Taipei, Taiwan

Email: {d92023, kmchao}@csie.ntu.edu.tw

Abstract

Recent studies have shown that the chromosome recombination only takes places at some narrow hotspots. Within segments between these hotspots, called haplotype block, little or even no recombination occurs and a small subset of SNPs, called tag SNPs, are sufficient to capture the entire block pattern. However, the tag SNP may be genotyped as missing data if it does not pass the threshold of data quality, and the DNA sample may fail to be identified due to the ambiguity caused by missing data. In this paper, we formulate this problem as finding a set of SNPs, called auxiliary tag SNPs, which is able to resolve the ambiguity caused by missing data. In addition, we also consider another set of SNPs, called robust tag SNPs, which guarantees no ambiguity regardless of the occurrence of missing data at any tag SNP. Both problems of finding minimum auxiliary and robust tag SNPs are shown to be NP-complete. Our study indicates that auxiliary tag SNPs can be found efficiently when robust tag SNPs have been computed in advance. To find robust tag SNPs, we propose two greedy approximation algorithms. These two approximation algorithms have approximation ratio $(m+1) \ln \frac{K(K-1)}{2}$ and $\ln((m+1) \frac{K(K-1)}{2})$ respectively, where m is the number of missing data and K is the number of distinct block patterns.

1 Introduction

In recent years, *Single Nucleotide Polymorphisms* (SNPs) [1] have become more and more popular for association studies¹ of genetic diseases

¹To perform association study, scientists first collect DNA samples and extract (genotype) SNPs from diseased and non-diseased individuals. Next, the SNPs from diseased individuals are compared to those from non-diseased ones. Eventually, a profile that contains SNP patterns corresponding to diseases will be established.

or traits. Although the cost of genotyping SNPs is gradually decreasing, it is still uneconomical to genotype all SNPs for association study [2]. However, recent findings showed that the chromosomal recombination only occurs at some narrow hotspots. The chromosomal region between these hotspots is called a “haplotype block.” Within a haplotype block, there is little or even no recombination occurred, and the SNPs (in the block) tend to be inherited together. Due to the low diversity of SNPs in a haplotype block, the information they carry is highly redundant. Thus, a small subset of SNPs (called “tag SNPs”) is sufficient to capture the entire SNP pattern of the haplotype block. Haplotype blocks with corresponding tag SNPs are quite useful and cost-effective in association studies as it does not require genotyping all SNPs. Many studies have tried to minimize the number of tag SNPs required in each block. In a large-scale study of chromosome 21, Patil *et al.* [6] developed a greedy algorithm to partition the haplotypes into 4,135 blocks with 4,563 tag SNPs. Zhang *et al.* [7, 8] used a dynamic programming approach to reduce the number of blocks and tag SNPs to 2,575 and 3,562, respectively. Bafna *et al.* [2] showed that the problem of minimizing tag SNPs is NP-hard and gave efficient algorithms for special cases of this problem.

When identifying an unknown DNA sample, the tag SNPs of the DNA sample are genotyped and compared to those of diseases. However, the genotyped tag SNP is considered as the missing data if it does not pass the threshold of data quality [3, 6, 9]. In this case, the DNA sample may fail to be identified. Figure 1 illustrates the influence of the missing data on DNA samples. In this figure, a haplotype block² (Figure 1 (A)) defined

²This haplotype block is redrawn from the haplotype database of chromosome 21 published by Patil *et al.* [6] at <http://www.perlegen.com/haplotype/>. We follow the same assumption as Patil *et al.*, Zhang *et al.*, and Bafna *et al.* that all SNPs are biallelic (i.e., taking on only two values).

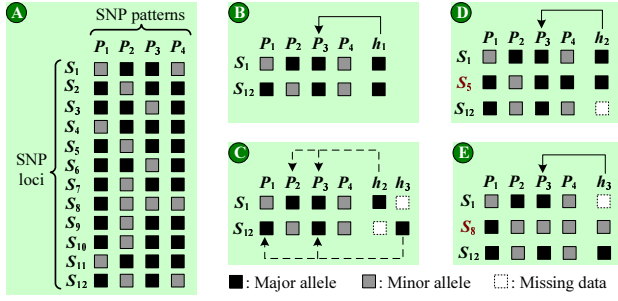


Figure 1: The influence of the missing data on DNA samples and corresponding auxiliary tag SNPs

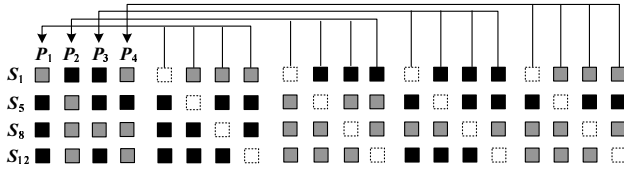


Figure 2: The robust tag SNPs and haplotype samples with missing data occurred at any locus

by 12 SNPs on chromosome 21 is presented. Each column represents a SNP pattern (P_1 , P_2 , P_3 , and P_4) and each row represents a SNP locus (S_1 , S_2 , ..., and S_{12}). The black and grey boxes stands for the major and minor alleles at the SNP locus, respectively. Suppose we select SNPs S_1 and S_{12} as tag SNPs. The DNA sample h_1 is identified as SNP pattern P_3 unambiguously (Figure 1 (B)). Consider DNA samples D_2 and D_3 with one tag SNP genotyped as the missing data (Figure 1 (C)). h_2 can be identified as SNP patterns P_2 or P_3 , and h_3 can be identified as P_1 or P_3 . As a result, the missing data results in ambiguity when identifying DNA samples.

Although the selected tag SNPs fail to identify the sample due to missing data, the remaining SNPs within the haplotype block may provide abundant information to resolve the ambiguity. For example, suppose we genotype an additional SNP S_5 for h_2 (Figure 1 (D)). h_2 is identified as SNP pattern P_3 unambiguously. On the other hand, if SNP S_8 is genotyped (Figure 1 (E)), h_3 is also identified unambiguously. These additional SNPs are referred to “auxiliary tag SNPs,” which can be found from the remaining SNPs in the block and are able to resolve the ambiguity caused by the missing data.

Alternatively, instead of re-genotyping auxiliary tag SNPs each time when encountering missing data, we can work on a set of SNPs which

is not affected by the the occurrence of missing data. For example, suppose we select SNPs S_1 , S_5 , S_8 , and S_{12} to be genotyped. Note that no matter which SNP is genotyped as missing data, the remaining three SNPs can still identify the DNA sample unambiguously (see Figure 1). We refer to these SNPs as “robust tag SNPs,” which correctly identify the DNA sample regardless of the missing data occurred at any SNP locus. When the occurrence of missing data is frequently, the cost of re-genotyping processes can be reduced by using robust tag SNPs instead of auxiliary tag SNPs.

This paper studies the problems of finding robust and auxiliary tag SNPs. Our study indicates that auxiliary tag SNPs can be found efficiently if robust tag SNPs have been computed in advance. The result of the paper is organized as follows. In Section 2, we formulate the problem of finding the robust tag SNPs mathematically and prove its NP-hardness. Section 3 gives an efficient approximation algorithm to find robust tag SNPs. Its approximation ratio is $(m + 1) \ln(\frac{K(K-1)}{2})$, where m is the number of SNPs genotyped as missing data and K is the number of patterns in the block. Section 4 illustrates the second approximation algorithm which achieves a better approximation ratio $\ln((m + 1) \frac{K(K-1)}{2})$. In Section 5, we show the NP-hardness of finding auxiliary tag SNPs and describe an efficient algorithm when robust tag SNPs have been computed in advance. Section 6 presents the experimental result of our algorithms applied to the public haplotype database. Finally, concluding remarks are given in Section 7.

2 Finding Robust Tag SNPs

Assume we are given a haplotype block consisting of N SNPs and K SNP patterns, which is denoted by an $N \times K$ matrix M_h (see Figure 2 (A)). Let $M_h[i, j] \in \{1, 2\}$ for each $i \in 1..N$ and $j \in 1..K$, where 1 and 2 represent the major and minor alleles, respectively. The set of robust tag SNPs C' which allows m SNPs genotyped as missing data must satisfy the following two properties: (1) each sample can be identified unambiguously (as one of the K patterns) by SNPs in C' ; (2) when m SNPs in C' are genotyped as missing data, (1) still holds. This problem is referred to as *Minimum Robust Tag SNPs* (MRTS) and the formal definition is given below.

Problem: Minimum Robust Tag SNPs

Input: An $N \times K$ matrix M_h and an integer m .

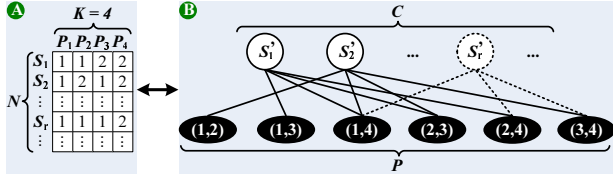


Figure 3: The haplotype matrix M_h and the corresponding bipartite graph G

Output: The minimum subset of rows (SNPs) C' in M_h which satisfies:

- (1) for each pair of patterns P_i and P_j , there is a row k in C' such that $M_h[k, i] \neq M_h[k, j]$ ³;
- (2) when m rows are discarded from C' arbitrarily, (1) still holds.

Now we show that the MRTS problem can be reformulated as a variation of the set covering problem [5]. Each row k in M_h is reformulated as a set $S'_k = \{(i, j) \mid M[k, i] \neq M[k, j] \text{ and } i < j\}$. For example, suppose the row k in M_h is $\{1, 1, 1, 2\}$. The corresponding set $S'_k = \{(1, 4), (2, 4), (3, 4)\}$. Let C be the collection of S'_k , where $1 \leq k \leq N$. Let P be the set that contains each pair of these K patterns (i.e., $P = \{(i, j) \mid 1 \leq i < j \leq K\} = \{(1, 2), (1, 3), \dots, (K-1, K)\}$). The following lemma implies that the set of robust tag SNPs C' is a collection such that each element in P is covered by corresponding sets of C' for at least $(m+1)$ times.

Lemma 1 $C' \subseteq C$ is the set of robust tag SNPs which allow m SNPs genotyped as missing data iff each element in P is covered by the sets in C' for at least $(m+1)$ times.

Proof:

Consider each element (i, j) in P and each set S'_k in C as nodes in an undirected bipartite graph G (see Figure 2 (B)). There is an edge connecting the node (i, j) and S'_k iff $(i, j) \in S'_k$. Consider a subset of nodes $C' \subseteq C$ such that each node in P has at least $(m+1)$ edges connected to some node in C' (i.e., C' covers P for at least $(m+1)$ times). Suppose the SNP S_r of the sample is genotyped as missing data, where $S'_r \in C'$. This implies that the row r in M_h can not be used to distinguish the sample, which has the same effect as the removal of the node S'_r and its edges from C' . If m nodes in C' are removed (i.e., m SNPs genotyped as missing data), each node in P still connects to some

³To identify the sample unambiguously, each pair of patterns must be distinguished by some row in C' . For example (see Figure 2 (A)), the patterns P_1 and P_2 can be distinguished by SNP S_2 since $M_h[2, 1] \neq M_h[2, 2]$.

node in C' (i.e., each pair of patterns can still be distinguished by the remaining SNPs in C'). Thus, the SNPs corresponding to C' are the robust tag SNPs which allow m SNPs genotyped as missing data. The proof of the other direction is similar. $\square$

Now we show that the NP-hardness of the MRTS problem.

Theorem 1 *The MRTS problem is NP-hard.*

Proof:

By Lemma 1, the set covering problem [5] can be reduced to a special case of MRTS when $m = 0$. Since the set covering problem is NP-hard, MRTS is NP-hard. $\square$

From Theorem 1, there is no polynomial time algorithm for solving MRTS unless $P = NP$. In Sections 3 and 4, we give two efficient approximation algorithms to solve MRTS.

3 The First Approximation Algorithm

In this section, we describe an approximation algorithm to solve MRTS by a greedy approach. By Lemma 1, we can solve MRTS by finding a subcollection $C' \subseteq C$ that covers (distinguishes) each element (pair of patterns) in P for at least $(m+1)$ times. Assume that each element in P and the corresponding SNPs that distinguish it are stored in a $(m+1) \times |P|$ table (see Figure 3 (A)). If a SNP S_k is picked, S_k is written into the grid of the column (i, j) , where (i, j) is the pair of patterns distinguished by S_k . At each step, the first algorithm picks a SNP that can write most grids in the row by a row-by-row manner. Figure 3 illustrates an example for this algorithm to cover P twice, where SNPs S_1, S_4, S_2 , and S_3 are picked in order. Let R_i be the set of unwritten grids at row i . While writing R_i , this algorithm picks a set $S \in C$ that maximizes $|S \cap R_i|$ (i.e., the set that writes most unwritten grids in R_i). Then, S is added to C' , and the rest elements in S (i.e., $S - R_i$) are written into the remaining unwritten grids in other rows. When all grids in this table are covered, C' is thus the collection that can cover P for $(m+1)$ times. The detail of this algorithm is given below.

Algorithm: GREEDY-1(C, P, m)

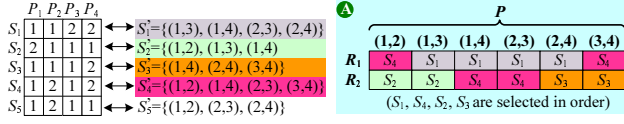


Figure 4: The sets picked by the first greedy algorithm

```

1   $R_i \leftarrow P$ , for each  $1 \leq i \leq m+1$ 
2   $C' \leftarrow \phi$ 
3  for  $i = 1$  to  $m+1$  do
4    while  $R_i \neq \phi$  do
5      select an  $S \in C$  that maximizes  $|S \cap R_i|$ 
6       $C' \leftarrow C' \cup S$ 
7       $j \leftarrow i$ 
8      while  $S \neq \phi$  and  $j \leq m+1$  do
9         $S_{tmp} \leftarrow S \cap R_j$ 
10        $R_j \leftarrow R_j - S_{tmp}$ 
11        $S \leftarrow S - S_{tmp}$ 
12        $j \leftarrow j+1$ 
13 endfor
14 return  $C'$ 

```

The time complexity of this algorithm is analyzed as follows. At Line 4, the number of iterations of the intermediate loop is bounded by $|R_i| \leq |P|$. Within the loop body (Lines 5-12), Line 5 takes $O(|P||C|)$ because we need to check each set in C and compare with each element in $R_i \leq |P|$. The inner loop (Lines 8-12) takes only $O(|S|)$. Thus, the entire program runs in $O(m|C||P|^2)$.

We now calculate the approximation ratio of $|C'|$ to $|C^*|$, where C^* is the collection of the optimal solution. The approximation ratio is proved by assigning different scores to the sets picked by the greedy algorithm [4]. Let $|S_k^w|$ be the number of grids written by S_k in the row where S_k is picked by the algorithm. For example (see Figure 3), $|S_4^w| = 2$ since S_4 writes two grids, (1,2) and (3,4), in the first row where the greedy algorithm picks S_4 . The score C_j^i is assigned to each grid put at the i th row and j th column, where

$$C_j^i = \begin{cases} \frac{1}{|S_k^w|} & \text{if the grid is written by } S_k \text{ while} \\ & \text{working the } i\text{th row;} \\ 0 & \text{Otherwise.} \end{cases}$$

Under this score assignment, the summation of the score C_j^i for each grid in the table is equal to $|C|$, that is,

$$\sum_{i=1}^{m+1} \sum_{j=1}^{\frac{K(K-1)}{2}} C_j^i = |C|. \quad (1)$$

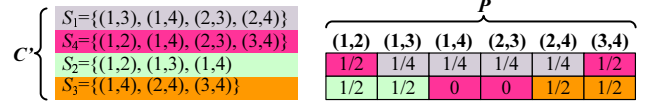


Figure 5: The score C_j^i for each set picked by the first greedy algorithm

Let R_k^i be the number of grids in the i th row remaining unwritten before the k th iteration (i.e., $S_1, S_2, \dots, S_{k-1}$ have been picked by the algorithm). Similar to [4], the summation of the score for each grid can be calculated as

$$\sum_{i=1}^{m+1} \sum_{j=1}^{\frac{K(K-1)}{2}} C_j^i = \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} (R_{k-1}^i - R_k^i) \frac{1}{|S_k^w|}. \quad (2)$$

Lemma 2 $|S_k^w| \geq \frac{R_k^i}{|C^*|}$.

Proof:

Consider the beginning of the k th iteration. Let C_k^* be the collection of sets in C^* that has been picked by the greedy algorithm before the k th iteration, and the collection of remaining sets in C^* be C_k^{*} . We claim that if this algorithm subsequently picks all sets in C_k^* , the remaining unwritten grids in the table will be all written. Otherwise (i.e., some grids remain unwritten), since $C_k^* \cup C_k^{*} = C^*$, this implies sets in C^* fail to write all grids in the table, which is a contradiction. By the pigeonhole principle, there exists one set in C_k^* with size at least $\frac{R_k^i}{|C_k^*|}$.⁴ Because sets in C_k^* are candidates to the greedy algorithm, $|S_k^w|$ must be at least $\frac{R_k^i}{|C_k^*|}$, which implies $|S_k^w| \geq \frac{R_k^i}{|C^*|}$ since $|C^*| \geq |C_k^*|$. $\square$

Theorem 2 The approximation ratio of the first greedy algorithm is $(m+1) \ln \frac{K(K-1)}{2}$.

Proof:

From (2) and Lemma 2, we have

$$\begin{aligned} \sum_{i=1}^{m+1} \sum_{j=1}^{\frac{K(K-1)}{2}} C_j^i &= \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} (R_{k-1}^i - R_k^i) \frac{1}{|S_k^w|} \\ &\leq \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} (R_{k-1}^i - R_k^i) \frac{C^*}{R_{k-1}^i} \end{aligned}$$

⁴If each set in C_k^* has size less than $\frac{R_k^i}{|C_k^*|}$, the summation

of the size of all sets in C_k^* is $< \frac{R_k^i}{|C_k^*|} \times |C_k^*| = R_k^i$. Since $C_k^* \cup C_k^{*} = C^*$, this means that C^* can not write all grids in R_k^i , which is a contradiction.

$$\begin{aligned}
&= \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} \left(\sum_{l=R_k^i+1}^{R_{k-1}^i} \right) \frac{C^*}{R_{k-1}^i} \\
&\leq C^* \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} \sum_{l=R_k^i+1}^{R_{k-1}^i} \frac{1}{l} \\
&= C^* \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} \left(\sum_{l=1}^{R_{k-1}^i} \frac{1}{l} - \sum_{l=1}^{R_k^i} \frac{1}{l} \right) \\
&\leq C^* \sum_{i=1}^{m+1} \sum_{k=1}^{|C|} (H(R_{k-1}^i) - H(R_k^i)) \\
&= C^* \sum_{i=1}^{m+1} [(H(R_0^i) - H(R_1^i)) + \dots \\
&\quad + (H(R_{|C|-1}^i) - H(R_{|C|}^i))] \\
&= C^* \sum_{i=1}^{m+1} [(H(R_0^i) - H(R_{|C|}^i))] \\
&\leq C^*(m+1) \max(H(R_0^i)) \\
&\leq C^*(m+1) \ln |P|. \tag{3}
\end{aligned}$$

Combining (1) and (3), we get

$$\frac{C}{C^*} \leq (m+1) \ln |P| = (m+1) \ln \frac{K(K-1)}{2}.$$

□

4 The Second Approximation Algorithm

This section gives a greedy algorithm which achieves a better approximation ratio than that in Section 3. Let U be the collection of R_i , which is defined in Section 3. In other words, U contains $m+1$ elements corresponding to each element in P . The greedy algorithm works by picking a set S that can cover the most uncovered elements from the union of R_i at each step (i.e., the S that maximizes $|S \cap (R_1 \cup \dots \cup R_{m+1})|$) and adds S to C . Then, the elements in S are used to cover the remaining elements in R_i and we remove the R_i with no more uncovered elements from U . When all sets in U are covered, C can cover P $m+1$ times. The algorithm is described as follows:

Algorithm: GREEDY-2(C, P, m)

- 1 $R_i \leftarrow P$, for each $1 \leq i \leq m+1$
- 2 $U \leftarrow \{R_1, R_2, \dots, R_{m+1}\}$
- 3 $C \leftarrow \emptyset$

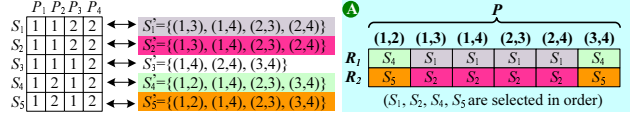


Figure 6: The sets picked by the second greedy algorithm



Figure 7: The score C_j^i for each grid in the table

```

4  while  $U \neq \emptyset$  do
5      select and removed an  $S \in C$  that maximizes
       $|S \cap (R_1 \dots \cup R_{m+1})|$ 
6       $C \leftarrow C \cup S$ 
7      for each  $R_i \in U$  do
8           $S_{tmp} \leftarrow S \cap R_i$ 
9           $R_i \leftarrow R_i - S_{tmp}$ 
10          $S \leftarrow S - S_{tmp}$ 
11         if  $R_i = \emptyset$  then  $U \leftarrow U - R_i$ 
12     endfor
13 return  $C$ 

```

Figure 4 illustrates an example for this algorithm to cover P twice. The greedy algorithm picks the set S_1 , S_2 , S_4 , and S_5 , in order. At Line 4, the number of iterations of the loop is bounded by the number of elements contained in U , which is $(m+1)|P|$. Within the loop, Line 5 can be implemented to take $O(|P||C|)$ because the union of R_i only need to be calculated once and then stored in a separated variable. The inner loop (Lines 7-12) is also bounded at $O(|S| < |P|)$. Thus, the running time of this program is $O(m|C||P|^2)$.

We analyze the approximation ratio of the second greedy algorithm. Let C be the collection of sets chosen by this greedy algorithm, and C^* be the collection of sets chosen by the optimal solution. Let $|S'_k|$ be the number of elements covered by S_k which contribute to the greedy algorithm. For example (see Figure 4), $|S'_4| = 2$ since S_4 covers two elements, (1,2) and (3,4), in the table. With similar techniques as in Section 3, assign the score $C_j^i = \frac{1}{|S'_k|}$ to the grid positioned at the i th row and j th column.

Let T_k be the number of grids in the table remaining uncovered before the k th iteration. We have the following lemma:

Lemma 3 $|S'_k| \geq \frac{T_k}{|C^*|}$.

Proof:

C_k^* and C_k^* are defined the same as in the proof of Lemma 2. We claim that there exists a set in C_k^* which has size at least $\frac{T_k}{|C_k^*|}$. Since the greedy algorithm always pick a set that covers maximal uncovered grids and all sets in C_k^* are available, $|S'_k| \geq \frac{T_k}{|C_k^*|} \geq \frac{T_k}{|C^*|}$. $\square$

Theorem 3 *The approximation ratio for the second greedy algorithm is $\ln((m+1)\frac{K(K-1)}{2})$.*

Proof:

The summation of the score for each grid would be

$$\begin{aligned}
C &= \sum_{i=1}^{m+1} \sum_{j=1}^{\frac{K(K-1)}{2}} C_j^i \\
&= \sum_{k=1}^{|C|} (T_{k-1} - T_k) \frac{1}{|S'_k|} \\
&\leq \sum_{k=1}^{|C|} (T_{k-1} - T_k) \frac{C^*}{T_{k-1}} \\
&\leq C^* H(T_0) \\
&\leq C^* \ln((m+1)\frac{K(K-1)}{2}). \quad (4)
\end{aligned}$$

From (4), we get

$$\frac{C}{C^*} \leq \ln((m+1)\frac{K(K-1)}{2}).$$

$\square$

5 Finding Auxiliary Tag SNPs

This section describes the problem of finding the auxiliary tag SNPs corresponding to a set of tag SNPs S with missing data. M_h is defined as in Section 2. This problem is referred to as *Minimum Auxiliary Tag SNPs* (MATS) and defined as follows.

Problem: Minimum Auxiliary Tag SNPs

Input: An $N \times K$ matrix M_h , and a set S .

Output: The minimum set of auxiliary tag SNPs A such that $A \cup S$ can identify the haplotype sample without ambiguity.

Note that when N and K become larger and the number of missing data increases, it is more difficult to find the auxiliary tag SNPs.

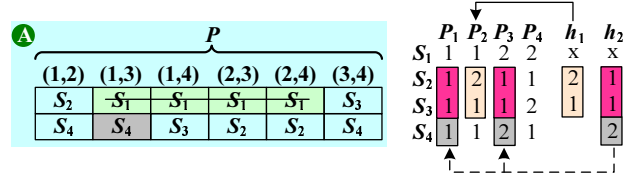


Figure 8: Examples to find the auxiliary tag SNPs by robust tag SNPs

Theorem 4 *MATS is NP-hard.*

Proof: Consider that all tag SNPs are genotyped as missing data. This problem is just like finding another set of tag SNPs to distinguish those K patterns, which is known as NP-hard. $\square$

Although the MATS problem is NP-complete, we show that auxiliary tag SNPs can be found efficiently when robust tag SNPs have been computed in advance. According to Lemma 1, each element in P must be covered for $m+1$ times by these robust tag SNPs. Without loss of generality, assume that these robust tag SNPs are implemented and stored in an $(m+1) \times |P|$ matrix M_r (see Figure 5). Each column in M_r represents an element in P and each $M_r\{*, j\}$ stores the SNP that covers the j th element in P . With this matrix, we can apply the following algorithm to find auxiliary tag SNPs with respect to S . At first, we compare the rest SNPs (which are not missing data) in S with each of the K patterns. If there is only one pattern matched (e.g., h_1 in Figure 5), the haplotype sample is identified as that block pattern (e.g., P_2) and we are done. Otherwise (e.g., h_2 in Figure 5), each pair of the matched patterns (e.g., P_1 and P_3) stands for an uncovered element in P and requires further disambiguation. Then, for each pair of the ambiguous patterns, traverse the corresponding column in M_r to find a set which can distinguish the pair of patterns (e.g., S_4 can distinguish P_1 and P_3). Let the collection of these sets for each ambiguous pair of patterns be A . According to Lemma 1, since all uncovered elements in P can be covered by $A \cup S$, A is the set of auxiliary tag SNPs corresponding to S . The detail of this algorithm is described as follows:

Algorithm: FINDING-AUXILIARY-SNPs(M_h, M_r, S)

```

1   $A \leftarrow \phi$ 
2   $counter \leftarrow 0$ 
3  for each haplotype pattern  $h \in M_h$  do
4      if there is no mismatch between each SNP in  $S$  and  $h$  then

```

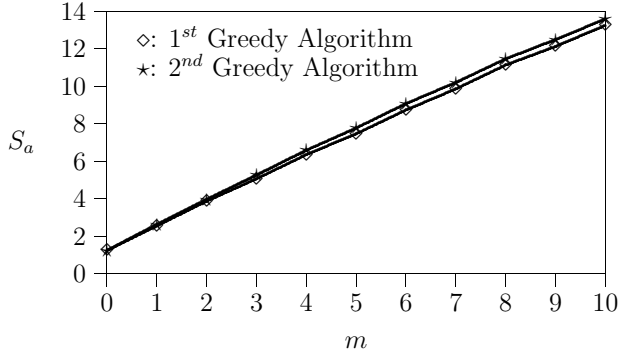


Figure 9: The average number of tag SNPs for each block with respect to m

```

5         counter ← counter + 1
6     endif
7 endfor
8 if counter = 1 then
9     return “No more SNPs required”;
10 else
11     for each pair of the matched pattern  $j$  do
12          $i \leftarrow 1$ 
13         while  $M_r[i, j] \in S$  and  $i \leq m$  do
14              $i \leftarrow i + 1$ 
15             Add  $M_r[i, j]$  to  $A$ 
16         endfor
17     endif
18 return  $A$ 

```

As for the running time of this algorithm, the worst case is that all SNPs in S are genotyped as missing data, and we need to traverse each column in the matrix M_r (Lines 11-16). Note that the size of P is $C_2^K = \frac{K(K-1)}{2}$. Thus, the running time of this algorithm is $O(m|P|) = O(m \frac{K(K-1)}{2}) = O(mK^2)$.

6 Experimental Result

We apply our two approximation algorithms mentioned in Sections 3 and 4 on the public haplotype data of Human Chromosome 21 [6]. This data set includes 20 haplotypes of 24,047 SNPs spanning over about 32.4MB. We first compare the number of tag SNPs found by our algorithms with the optimal number found by Patil *et al.* over the same 4,135 blocks. The first and second algorithms both find 4,610 tag SNPs, where the optimal number they found is 4,563. Thus, the ratio is $\frac{4610}{4563} \simeq 1.01$, which indicates our approximation algorithms are quite close to the optimal solution.

Next, we evaluate these two algorithms with respect to m (i.e., the number of SNPs genotyped as missing data). Let S_a be the average number of robust tag SNPs for each block found by these two algorithms. Figure 6 plots S_a with respect to m . We observe that S_a grows linearly for both algorithms. Note that the lower bound of robust tag SNPs is $(m + \lg |K|)$, where K is the number of patterns. This phenomenon indicates that the number of robust tag SNPs does not grow too much when m increases and is quite close to the lower bound.

Although the theoretical approximation ratio of the second algorithm is better than that of the first one, our experimental result indicates that the first algorithm slightly outperforms the second one when m becomes large in the longer blocks⁵. This is because we search for SNPs to pick from the beginning of the long block but the SNPs within do not vary too much (e.g., the SNP with pattern (1,1,2,2) is repeated continuously at many loci). The second algorithm tends to pick the former SNPs in the block, where the first algorithm tends to pick the latter ones since it finds different patterns at each step. Thus, for long blocks where SNPs of the optimal solution distributed at two ends, the first algorithm is slightly better.

7 Conclusion

In this paper, we study the problems of finding robust and auxiliary tag SNPs. We describe two greedy approximation algorithms for finding robust tag SNPs. An efficient algorithm is presented to find auxiliary tag SNPs when robust tag SNPs have been computed in advance. Our experimental result shows that the solution found by both greedy algorithms is quite close to the optimal solution even when the number of SNPs allowed for missing data increases. Note that the first greedy algorithm tries to optimize SNPs in the first row of the table structure. Therefore, if the occurrence of missing data is infrequently, we can select the SNPs in the first row to genotype, and re-genotype auxiliary tag SNPs only when encountering missing data. Since the solution found by both greedy algorithms is similar, the first greedy algorithm may be more useful than the second one in practice.

⁵In the data set from Patil, there are many short blocks with very few SNPs, which are not biologically meaningful. We discard blocks that do not contain enough SNPs for the solution of MRTS in the experiment.

Acknowledgements. We thank Ting Chen, Chia-Yu Su, and the reviewer for helpful comments. Yao-Ting Huang and Kun-Mao Chao were supported in part by an NSC grant 92-2213-E-002-059.

- [9] Zhao, J.H., Lissarrague, S., Essioux, L., and Sham, P.C. GENECOUNTING: haplotype analysis with missing genotypes. *Bioinformatics*, 18(12):1694–1695, 2002.

References

- [1] <http://www.ncbi.gov/dbsnp>.
- [2] Bafna, V., Halldorsson, B.V., Schwartz, R., Clark, A.G., and Istrail, S. Haplotypes and Informative SNP Selection Algorithms: Don't Block Out Information. *Proceedings of the Seventh Annual International Conference on Research in Computational Molecular Biology*, pages 19–27, 2003.
- [3] Bourgain, C., Genin, E., Ober, C., and Clerget-Darpoux, F. Missing data in haplotype analysis: a study on the MILC method. *Annals of Human Genetics*, 66(1):99–108, 2002.
- [4] Cormen T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. *Introduction to Algorithms*. The MIT Press, 2001.
- [5] Garey, M.R. and Johnson, D.S. *Computers and Intractability*. Freeman, New York, 1979.
- [6] Patil, N., Berno, A.J., Hinds, D.A., Barrett, W.A., Doshi, J.M., Hacker C.R., Kautzer, C.R., Lee, D.H., Marjoribanks, C., McDonough, D.P., Nguyen, B.T.N., Norris, M.C., Sheehan, J.B., Shen, N., Stern, D., Stokowski, R.P., Thomas, D.J., Trulson, M.O., Vyas, K.R., Frazer, K.A., Fodor, S.P.A., and Cox, D.R. Blocks of limited haplotype diversity revealed by high-resolution scanning of human chromosome 21. *Science*, 294:1719–1723, 2001.
- [7] Zhang, K., Deng, M., Chen, T., Waterman, M.S., and Sun, F. A Dynamic Programming Algorithm for Haplotype Block Partitioning. *Proceedings of the National Academy of Sciences of the United States of America*, 99:7335–7339, 2002.
- [8] Zhang, K., Sun, F., Waterman, M.S., and Chen, T. Dynamic Programming Algorithms for Haplotype Block Partitioning: Applications to Human Chromosome 21 Haplotype Data. *Proceedings of the Seventh Annual International Conference on Research in Computational Molecular Biology*, pages 332–340, 2003.

A Case Study of Predicting the Class of the Unknown Viruses of ssRNA positive-strand in NCBI

Jing-Doo Wang

Department of Information Technology
Taichung Healthcare and Management University
Wufeng Shiang, Taichung, Taiwan 413, R.O.C.
jdwang@thmu.edu.tw

Abstract

The purpose of this paper is to predict the class label of unknown virus in the class of ssRNA positive-strand in NCBI. Each virus (sequence) was presented as one vector via representative patterns. The representative patterns are distinct patterns with biased distribution of the frequency among classes. We use WithinCoverage and PatternEntropy as thresholds to select representative patterns. The WithinCoverage of one pattern is the percentage of the sequences within one class in which that pattern appears. The PatternEntropy is the entropy of term frequency of one pattern across different classes. Note that the length of these patterns are variable. We experimented with the ssRNA positive-strand viruses in NCBI in which consisted of 33 classes with 396 viruses as training data and the unknown class with 8 viruses as testing data. We adapted the linear classifier with Rocchio algorithm to measure the relationship between the existed class and the unknown viruses. Experimental results showed the class achieved the highest value of cosine similarity for each unknown virus.

Keywords: *representative patterns, classification.*

1 Introduction

The DNA sequences of entire genomes are being determined at a rapid rate. As more and more whole genomes are available, there is a need for new methods to compare large sequences and transfer biological knowledge from annotated genomes to related new ones. These changes in genome sequencing strategies have created a strong need for new methods to compare genomic sequences. There are sequences alignment

methods for sequence comparison, like Smith-Waterman algorithm [17], FASTA algorithm [13] and BLAST algorithm [4], to measure the relationship between different sequences. However, these methods usually suffer from the limitations of the computational time and memory available to handle large amount of DNA sequences data.

The purpose of this paper is to predict the class label of unknown virus in the class of ssRNA positive-strand in NCBI [3]. We present each virus as one vector using representative patterns such that many machine learning algorithms in vector space model [14] could be applied to those bioinformatics problems [9], such as inter or intra-chromosomal comparison. The representative patterns according to the classification, determined by domain specific experts, are distinct patterns with biased distribution of the frequency among classes.

We select representative patterns from the significant patterns. First of all, we adapted our Chinese term extraction approach [18, 22] for extracting the significant patterns from DNA sequences. Note that the length of these patterns are variable. To obtain the distribution of the frequency of each pattern among classes, first we sort all of the suffix strings [7] from the sequences. Then, we scan the sorted suffix strings in lexicographic order and extract the longest common prefix and, with our significant term extraction approach, calculate the statistics of the frequency distribution among classes. The patterns extracted from above processes are candidate significant patterns with right boundary verification. We do the same processes with the reversed strings of the original sequences to have patterns with the left boundary verification. The significant patterns are candidate significant patterns that passed both right and left boundary verification. Finally, the representative patterns are selected from those significant pat-

terms with biased frequency distributions among classes.

After extracting the significant patterns, we use *WithinCoverage* and *PatternEntropy* as thresholds to select representative patterns from the significant patterns. The *WithinCoverage* of one pattern is the percentage of the sequences within one class in which that pattern appears. The *PatternEntropy* is the entropy of term frequency of one pattern across different classes. Intuitively, we select the patterns with the value of *WithinCoverage* as high as possible such that these patterns appear in most of the sequences within the same class. Therefore, these patterns selected are representative of that class. On the other hand, we select the patterns with the value of *PatternEntropy* as low as possible such that the distributions of term frequency of these patterns among classes were biased. That is, these patterns could be applied to distinguish one class from the others. After selecting the representative patterns, we can present a sequence (a virus) as one vector. To determine the weighting for each dimension (representative pattern) in one vector, we use the combination of the length of representative pattern and the entropy of term frequency distribution of that pattern among the classes existed.

In this paper, for simplicity, we have linear classifier using Rocchio algorithm [8] to predict the class label of unknown virus, although there are many well-known classifiers [10]. We experimented with the ssRNA positive-strand viruses [3] (Revised: March 5, 2004.) from NCBI [1]. There were 33 classes which consisted of 396 viruses as training data and the unknown ssRNA positive-strand viruses which consisted 8 viruses as testing data. Experimental results showed the class achieved the highest value of cosine similarity for each unknown virus.

Because there were some classes containing few virus in the training set, we didn't have the measure of precision and recall [14] in classification to evaluate the performance of virus classification, but only predicted the unknown virus. We are looking for the classes containing more sequences to further investigate our approach for sequence classification [11, 2] such that make our experiments more robust in the future.

The remainder of this paper is organized as follows. Section 2 describes the processes of extracting representative patterns. Section 3 gives the weighting method of representative patterns for each sequence. Section 4 reviews the linear classifier with Rocchio algorithm. Section 5 gives ex-

perimental results. Section 6 gives conclusions and discussions.

2 Extracting Representative Patterns

We adapted our Chinese term extraction approach [18, 22] for extracting the representative patterns to distinguish one class from the others. Note that the length of these patterns are variable. The process of representative patterns extraction consists of three steps: (i) sorting the suffix strings of genomic sequences, (ii) determining the boundaries of significant patterns and (iii) selecting representative patterns from significant patterns. First, we sorted the suffix strings of genomic sequences. Then, we have significant patterns which pass both right boundary and left boundary verification by scanning the suffix strings lexicographically and extracting the longest common prefix from adjacent strings. Finally we select representative patterns from significant patterns according to two parameters. One is the percentage of species within one class that pattern appeared and another is the entropy of the pattern frequency distribution among classes. Intuitively, we aim to extract those representative patterns which appear in the species within the same class as many as possible, and whose frequency distributions among classes were biased such that they could be applied to distinguish different classes with proper weighting method. We described these three steps in section 2.1, section 2.2 and section 2.3, respectively.

2.1 Sorting Suffix Strings

The suffix strings of the sequences from the genome of each species are sorted such that we can scan these strings in lexicographical order to verify the boundaries of significant patterns and, at the same time, to have the statistics of attributes related to those sequences. Note that we assume the maximum length of patterns is 50 in this paper. As shown in Fig. 1, for example, there are suffix strings of one sequence "TATAAAAAGGAT". Note that the reversed sequence of "TATAAAAAGGAT" is "TAGGAAAAATAT". The suffix strings of the sequences and those of the reversed sequences are individually sorted such that all the suffix strings are in lexicographic order and are used to determining the right or left boundaries of significant patterns which was de-

- (1) TATAAAAAGGAT
- (2) ATAAAAAGGAT
- (3) TAAAAAGGAT
- (4) AAAAAGGAT
- (5) AAAAGGAT
- (6) AAAGGAT
- (7) AAGGAT
- (8) AGGAT
- (9) GGAT
- (10) GAT
- (11) AT
- (12) T

Figure 1: Suffix strings of sequence "TATAAAAAGGAT"

defined in section 2.2. A partial sorted suffix strings, for example, are shown in Fig. 2, where the value of "SID", for simplicity, are 1 for the identifier of one sequence, and the value of "Position" represents the starting position of suffix string in that sequence.

2.2 Determining the Boundaries of Significant Patterns

First of all, we give the definition of significant pattern [21] as following.

Definition 1 Significant Pattern

Let w be a string. For any character x , let $Porb(wx|w)$ be the probability that w is followed by x , and $Porb(xw|w)$ be the probability that w is preceded by x . The string w passes the right boundary verification if $Porb(wx|w) < \theta_1$ for all x , and passes the left boundary verification if $Porb(xw|w) < \theta_2$ for all x , where θ_1 and θ_2 are threshold determined manually. The probability $Porb(wx|w)$, resp. $Porb(xw|w)$, is estimated by $\frac{TF(wx)}{TF(w)}$, resp. $\frac{TF(xw)}{TF(w)}$, where $TF(y)$ is the term frequency of string y . The string w is identified as a *significant term* if it passes both right and left boundary verifications and its term frequency $TF(w)$ exceeds a threshold, δ . This study simply sets $\theta_1 = \theta_2 = 1$ and $\delta = 2$. That is, w is identified as a significant term if it has at least two distinct succeeding and preceding characters, and if $TF(w)$ exceeds 2.

The sorted suffix strings are scanned lexicographically to extract the longest common prefix of adjacent strings yielded candidate significant patterns with right and left boundary verification

via our term extraction approach in [18, 22]. As shown in Fig. 1, the pattern "TATAAA", for example, is a significant pattern which pass both right and left boundary verification. However, As shown in Fig. 1 (a), the pattern "TATAAAGT" with dotted rectangle, for example, didn't pass right boundary verification because it was always covered by the pattern "TATAAAGTT". Similarly, as shown in Fig. 1 (b), the pattern "AAATATTAATATT" with dotted rectangle, for example, didn't pass left boundary verification because it was always covered by the pattern "AAATATTAATATTA".

2.3 Selecting Representative Patterns from Significant Patterns

The representative patterns are used to distinguish one class from the others. We use *WithinCoverage* and *PatternEntropy* as thresholds to select representative patterns from the significant patterns. We define *WithinCoverage* and *PatternEntropy* in the following.

Definition 2 WithinCoverage

Let the *WithinCoverage* WC_{P_i, C_j} of pattern P_i respect to class C_j be $\frac{S_{P_i, C_j}}{|C_j|}$, where S_{P_i, C_j} is the number of sequences within class C_j that contain pattern P_i , and $|C_j|$ is the number of sequences within class C_j .

Definition 3 PatternEntropy

Let the *PatternEntropy* $PE(P_i)$ of pattern P_i be $\frac{\sum_{k=1}^{|C|} Prob(P_{i,k}) * \log_{|C|} Prob(P_{i,k})}{\log_{|C|}}$, where the $|C|$ is the number of classes and the probability $Prob(P_{i,j})$ is $\frac{TF_{i,j}}{\sum_{k=1}^{|C|} TF_{i,k}}$ and $TF_{i,j}$ is the term frequency of pattern P_i in class C_j . Note that $0 \leq PE(P_i) \leq 1$

The *WithinCoverage* of one pattern is the percentage of the sequences within one class in which that pattern appears. The *PatternEntropy* is the entropy of term frequency of one pattern across different classes. Intuitively, we select the patterns with the value of *WithinCoverage* as high as possible such that these patterns appear in most of the sequences within the same class. Therefore, these patterns selected are representative of that class. On the other hand, we select the patterns with the value of *PatternEntropy* as low as possible such that the distributions of term frequency of these patterns among classes were biased. That is, these patterns could be applied to distinguish one class from the others.

(SID :Position) Suffix String of Sequences

•
•
•
1 7801 TAGTTTTTGATGGCAAGTCCAAATGCGACGAGTCTGCTTCTAAGTCTGCT
1 5877 TATAAA AAGGATAATGCTTACTATACAGAGCAGCCTATAGACCTTGTACC
1 4880 TATAAA AGTGTTCACAACTGTGGACAACACTAATCTCCACACACAGCTTG
1 109 TATAAA CAATAATAAAATTTTACTGTCGTTGACAAGAAACGAGTAACTCGT
1 5823 TATAAA CTCGATGGAGTTACTTACACAGAGATTGAACCAAAATTGGATGG
1 1718 TATAAA GAGTCTTGATTACAAGTCTTTCAAAACCATTGTTGAGTCCTGCG
1 21040 TATAAA GCAAAAAGTAGCCCTGGGTGGTTCTATAGCTGTAAAGATAACAG
1 4392 TATAAA GGAATTAATAATCAAGAGGGCATCGTTGACTATGGTGTCCGATT
1 18413 TATAAA GGCTTGCCCTGGAATGTAGTGCCTATTAAGATAGTACAAATGCT
1 1773 TATAAA GTTACCAAGGGAAAGCCCGTAAAAGGTGCTTGAACATTGGACA
1 18043 TATAAA GTTCAAGACTGAAGGATTATGTGTTGACATACCAGGCATACCAA
1 22787 TATAAA TATAGGTATCTTAGACATGGCAAGCTTAGGCCCTTTGAGAGAGA
1 26988 TATAAA TTAAATACAGACCACGCCGTAGCAACGACAATATTGCTTTGCT
1 5619 TATAAA TTACAGCAAGGTACATTCTTATGTGCGAATGAGTACACTGGTAA
1 22703 TATAAA TTGCCAGATGATTTTCATGGGTGTGTCTTGCTTGAATACTAG
1 10269 TATAAA TTTGTCCGTATCCAACCTGGTCAAACATTTTCAGTTCTAGCATG
1 9810 TATAACAAGTACAAGTATTTTCAGTGGAGCCTTAGATACTACCAGCTATCG
•
•
•

(a) sorted suffix strings

(SID :Position) Suffix String of Sequences

•
•
•
1 12550 AAATAGTTTTGTGAGATGCGCGCGTGCGCGTCCATACTAAGATGATGTAA
1 11687 AAATAT AGTTGCGACTCCACACATCCACGGACACATCCTACTTCTGGTCA
1 2742 AAATAT CAAAGGTTATGCCATCGCCAACATACGTCGTTTTGGACTTAGTC
1 27957 AAATAT CAATGGCGTCTCTGAGTTGTTACCAAACTTTCTGAACATTAGTT
1 28012 AAATAT CACAGTTATTTCCGTGAACATCTTCGTCTCTTTCTACGGTTTTA
1 24850 AAATAT CAGAATTGGAGGGCGTCCCTATTCTCTGAGAAATCAAACAGTTC
1 19461 AAATAT GAACCCACAGAATCCCAATCTTCATAGTTGAAATTCGGATTTCGT
1 25338 AAATAT GAATGCAACCTACCAACGGTAATACCGAGATTGTAGGTACGTAT
1 29621 AAATAT GACGCATCCACGTGATCCGTACGTCGGCTCGCTGTGATGTGTC
1 24111 AAATAT GAGTCGTCCACCACGTCTGTAGTATTGTTTTCTTCTGAGAACAA
1 23907 AAATAT GCTGTGTCCGAACACCAACATCACATTCTACAAAGGAACATCT
1 11317 AAATAT GTAATCACTACCATATTCTACAAATTTGACCAGTGGACCACCTCAA
1 6943 AAATAT TAATATTAATGGTCAACTTCATCGTAGTTACAAGGATCATAAGG
1 7027 AAATAT TAATATTAATGTCGTTATTGTGGTCAAACAGGACCGCGATAAACAG
1 23853 AAATAT TATGGGTAGGTTAAAACCAAGTTAGAGACACATTCATTGAGGTA
1 8690 AAATAT TTAGGTGTGTCTATTCACTTTTTGGGAAGAAATCTCAGTAAGAG
1 24886 AAATCAAACAGTTCACCTTTCTTGGAGTGGCAGTTCTACTTTGAGCTGCCC
•
•
•

(b) sorted suffix strings(reversed)

Figure 2: Sorted Suffix Strings

3 The Weighting of Representative Patterns for Each Sequence

In this study, we have the weighting w_j of representative pattern RP_j for the sequence W as $w_j = wtf(RP_j) \cdot length(RP_j) \cdot \frac{1}{PE(RP_j)+0.001}$, where $wtf(RP_j)$ is the term frequency of the pattern RP_j in the sequence W , and the PatternEntropy $PE(RP_j)$ be the entropy [10] of the pattern RP_j among classes, and the value 0.001 is set to avoid the value of PatternEntropy being zero. Therefore, we have each sequence $W = (w_1, w_2, \dots, w_n)$ be represented as a vector, where n is the number of representative patterns selected.

4 Linear classifier using Rocchio Algorithm

Linear classifier is a simple approach for classification [8]. The main idea of linear classifier is to construct a feature vector as one representative for each class (category). For each class C_i , linear classifier computes prototype vector $G_i = (g_{i,1}, \dots, g_{i,n})$, where n is the dimension of the pattern space and each element $g_{i,j}$ corresponds to the weight of the pattern P_i of G_i . The elements in the vector G_i are learned from positive examples tuned by negative examples. Positive examples are those documents belonging to that class while negative examples are those documents not belonging to that class. To classify a request sequence X , we compute the cosine similarity between X and each prototype vector G_i , and assign to X the class whose prototype vector has the highest degree of cosine similarity with X . Cosine similarity is defined as follows:

$$CosSim(X, G_i) = \frac{\sum_{j=1}^n x_j \cdot g_{i,j}}{\sqrt{\sum_{j=1}^n x_j^2} \sqrt{\sum_{j=1}^n g_{i,j}^2}}$$

In this study, we use the Rocchio algorithm [15] to construct the representative G_i for class C_i . Let P and N be the set of positive and negative examples with respect to class C_i in the training corpus. $|P|$ and $|N|$ are the number of examples in P and N , respectively. The prototype vector G_i is defined as follows:

$$G_i = \frac{\sum_{W \in P} W}{|P|} - \eta \frac{\sum_{W \in N} W}{|N|}$$

where η is the parameter that adjusts the relative impact of positive and negative examples. In this

study, we choose $\eta = 0.25$ according to our experience in [19].

5 Experimental Results

We experimented with the ssRNA positive-strand viruses [3] in NCBI [1]. As shown in Table 1, we had 33 classes which consisted of 396 viruses as training data and, as shown in Table 2, the unknown class which consisted of 8 viruses as testing data. Note that we have one sequence represent for one virus.

First of all, we selected 19465 representative patterns from the significant patterns extracted from the training data. The representative patterns were the significant patterns which achieved the value of WithinCoverage over 95% for one class and the value of PatternEntropy under 0.2. The value of 95% and 0.2 were determined manually according to the average of non-zero items in vectors. We used these patterns to present the sequences of viruses as vectors. The average of non-zero items in one vector for training data and testing data are 105.5 and 35.5, respectively. The minimum and maximum length of representative patterns are 9 and 50, respectively. For simplicity, we only showed the distribution of the length of representative patterns whose lengths ranged from 9 to 30 in Figure 5. As shown in Table 3, we had the class achieved the highest value of cosine similarity for each unknown virus. For example, the value of cosine similarity of the "Botrytis virus F" and class "HepatitisE-likeviruses" was 0.316871.

6 Conclusions and Discussions

In this paper we predicted the unknown virus of the ssRNA positive-strand [3] in NCBI [1]. We extracted representative patterns from virus and presented the sequences as vectors such that makes the linear classifier for predicting unknown virus possible. Note that the length of representative patterns are variable.

We are looking for more sequences which are well classified and contain more sequences to further investigate our approach for DNA sequence classification such that make our experiments more robust in the future[2]. DNA sequence classification is an effective aid of the comprehension, summarization and compression of DNA sequences [11]. There are studies about the feature selection for sequence classification [6, 12, 20, 5].

Table 1: Training Data : ssRNA positive-strand viruses, no DNA stage(Revised: March 5, 2004.)

	Class	# of Virus	TotalLength(bp)
1	Astroviridae	6	41148
2	Barnaviridae	1	4009
3	Benyvirus	2	30143
4	Bromoviridae	18	151486
5	Caliciviridae	9	70075
6	Closteroviridae	11	182663
7	Comoviridae	17	187716
8	Dicistroviridae	11	103124
9	Flaviviridae	31	336855
10	Flexiviridae	42	316405
11	Furovirus	5	52760
12	HepatitisE-likeviruses	1	7176
13	Hordeivirus	1	10223
14	Idaeovirus	1	7681
15	Iflavirus	5	47483
16	Leviviridae	10	38908
17	Luteoviridae	16	91598
18	Narnaviridae	7	18166
19	Nidovirales	12	292492
20	Nodaviridae	8	35723
21	Pecluvirus	2	20751
22	Picornaviridae	30	230782
23	Pomovirus	4	46735
24	Potyviridae	46	453903
25	Sequiviridae	9	68920
26	Sobemovirus	9	37791
27	Tetraviridae	2	14416
28	Tobamovirus	15	96365
29	Tobravirus	3	29723
30	Togaviridae	15	173184
31	Tombusviridae	32	137821
32	Tymoviridae	11	70771
33	Umbravirus	4	16625
	Total	396	3423621

Table 2: Testing Data : unclassified ssRNA positive-strand viruses (Revised: March 5, 2004.)

	Taxonomy ID	ACCESSION	unclassified ssRNA positive-strand viruses
1	20451015	NC_003780.1	Acyrtosiphon pisum virus
2	21557557	NC_004045.1	Beet western yellows ST9 associated virus
3	11125722	NC_002604.1	Botrytis virus F
4	37595815	NC_005132.1	Botrytis virus X
5	9634052	NC_001278.1	Diaporthe ambigua RNA virus 1
6	18875445	NC_003412.1	Euprosteria elaeasa virus
7	38707888	NC_005281.1	Heterosigma akashiwo RNA virus SOG263
8	28261417	NC_004560.1	Oyster mushroom spherical virus

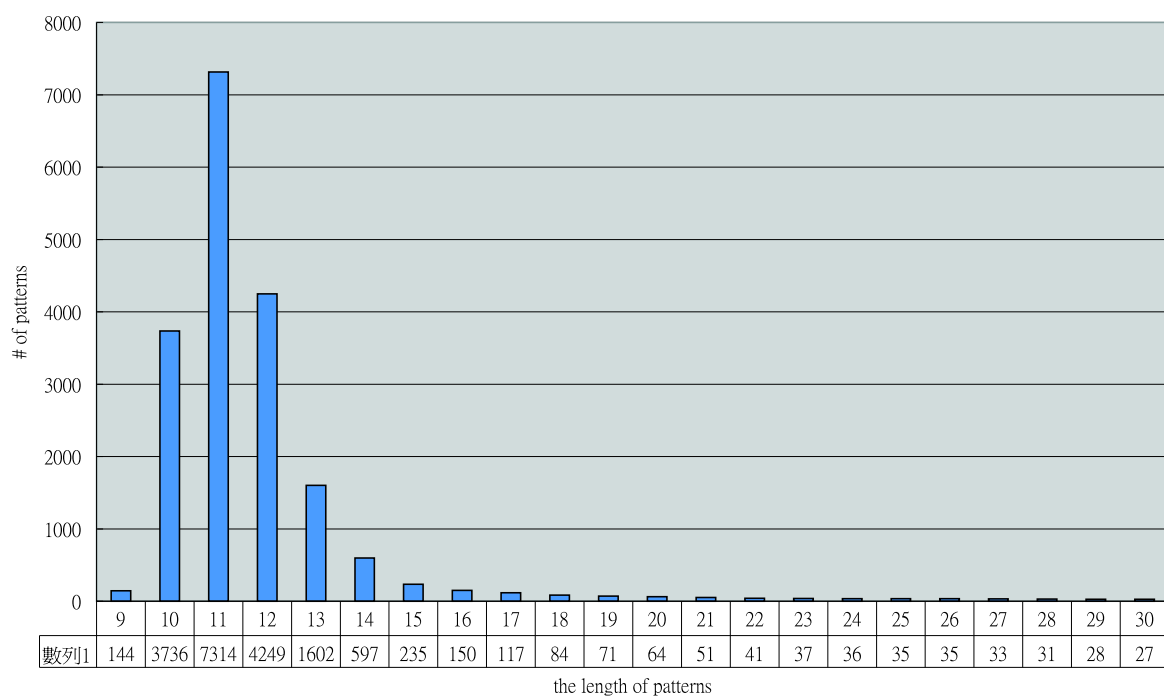


Figure 3: The distribution of the length of representative patterns

Table 3: The unknown viruses with the class achieved the highest cosine similarity

	unclassified ssRNA positive-strand viruses	Class	cosine similarity
1	Acyrtosiphon pisum virus	Benyvirus	0.014302
2	Beet western yellows ST9 associated virus	Tombusviridae	0.007297
3	Botrytis virus F	HepatitisE_likeviruses	0.316871
4	Botrytis virus X	Flaviviridae	0.002858
5	Diaporthe ambigua RNA virus 1	Flaviviridae	0.003824
6	Euprosterna elaeasa virus	Flexiviridae	0.002339
7	Heterosigma akashiwo RNA virus SOG263	Potyviridae	0.006173
8	Oyster mushroom spherical virus	HepatitisE_likeviruses	0.002388

However, most of these approaches used n -grams model. On the other hand, the approach of representative patterns extraction could be used to mine for the distinct patterns for one class is interesting for the design of identifying the marker of that class.

Acknowledgment. Thank Dr. Chang Pai-Jun, Dr. Liu Hsiang-Chuan and Dr. Lee Ming-Ming for valuable suggestions, and Horng Xiong-Ru for gathering the sequences of ssRNA positive-strand viruses.

References

- [1] NCBI. <http://www.ncbi.nlm.nih.gov/>.
- [2] Taxonomy and classification of viruses. <http://www.ncbi.nlm.nih.gov/ICTVdb/MCM8.pdf>.
- [3] Viral Genomes. <http://www.ncbi.nlm.nih.gov/genomes/VIRUSES/viruses.html>.
- [4] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215:403–410, 1990.
- [5] Jeong-Hyeon Choi and Hwan-Gue Cho. Analysis of common k -mers for whole genome sequences using ssb-tree. *Genome Informatics*, 13:30–41, 2002.
- [6] NA Chuzhanova, AJ Jones, and S Margetts. Feature selection for genetic sequence classification. *Bioinformatics*, 14(2):139–143, 1998.
- [7] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences : computer science and computational biology*. Cambridge University Press, 1997.
- [8] David D. Lewis, Robert E. Schapire, James P. Callan, and Ron Papka. Training algorithms for linear text classifiers. In *Proceedings of the 19th Ann Int ACM SIGIR Conference on Research and Development in Information Retrieval(SIGIR'96)*, pages 298–306, 1996.
- [9] Webb Miller. Comparison of genomic dna sequences: solved and unsolved problems. *Bioinformatics*, 17(5):391–397, 2001.
- [10] Tom M. Mitchell. *Machine Learning*. The McGraw-Hill Companies, Inc, 1997.
- [11] Snehasis Mukhopadhyay, Changhong Tang, Jeffery Huang, and Mathew Palakal. Genetic sequence classification and its application to cross-species homology detection. *The Journal of VLSI Signal Processing : Systems for Signal, Image, and Video Technology*, 35:273–285, 2003.
- [12] Mohammed J. Zaki Neal Lesh and Mitsunori Ogi-hara. Mining features for sequence classification. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 342–346, 1999.
- [13] W R Pearson and D J Lipman. Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences of the United States of America*, 85:2444–2448, 1988.
- [14] Baeza-Yates Ricardo and Ribeiro-Neto Berthier. *Modern Information Retrieval*. Addison Wesley, 1999.
- [15] Rocchio and J.J. Jr. *Relevance feedback in information retrieval*, chapter The SMART Retrieval System: Experiments in Automatic Document Processing, pages 313–323. Prentic Hall, g.saltan edition, 1971.
- [16] Amitabh Kumar Singhal. *Term Weighting Revisited*. PhD thesis, Cornell University, 1997.
- [17] T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147:195–197, 1981.
- [18] Jyh-Jong Tsay and Jing-Doo Wang. A scalable approach for Chinese term extraction. In *2000 International Computer Symposium(ICS2000)*, Taiwan, R.O.C, pages 246–253, 2000.
- [19] Jyh-Jong Tsay and Jing-Doo Wang. Improving linear classifier for Chinese text categorization. In *2001 National Computer Symposium, Taiwan, R.O.C*, pages B178–B188, 2001.
- [20] Jason Tsong-Li Wang, Steve Rozen, Bruce A. Shapiro, Dennis Shasha, Zhiyuan Wang, and Maisheng Yin. New techniques for DNA sequence classification. *Journal of Computational Biology*, 6(2):209–218, 1999.
- [21] Jing-Doo Wang. *Design and Evaluation of Approaches for Automatic Chinese Text Categorization*. PhD thesis, Department of Computer Science and Information Engineering, National Chung Cheng University, Chiayi, Taiwan 62107, R.O.C., 2002.
- [22] Jing-Doo Wang and Jyh-Jong Tsay. Minging periodic events from restorpective Chinese news. *International Journal of Computer Processing of Oriental Languages, A Special Issue on "Web and WAP Oriental Languages Multimedia Computing"*, 15(4):361–377, 2002.

An Optimal Algorithm for the Range Maximum-Sum Segment Query Problem

Kuan-Yu Chen and Kun-Mao Chao

Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan 106
{r92047, kmchao}@csie.ntu.edu.tw

Abstract

We are given a sequence A of n real numbers which is to be preprocessed. In the Range Maximum-Sum Segment Query (RMSQ) problem, a query is comprised of two intervals $[i, j]$ and $[k, l]$ and our goal is to return the maximum-sum segment of A where the starting index of the segment lies in $[i, j]$ and the ending index lies in $[k, l]$. We provide the first known optimal algorithm with $O(n)$ preprocessing time and $O(1)$ query time.

Keywords: RMQ, maximum sum interval, sequence analysis

1 Introduction

Sequence analysis in bioinformatics has been studied for decades. One important line of investigation in sequence analysis is to locate the biologically meaningful segments, like conserved regions or GC-rich regions in DNA sequences. A common approach is to assign a real number (also called scores) to each residue, and then look for the maximum-sum or maximum-average segment [3, 5, 10].

Ruzzo and Tompa [12] proposed a linear time algorithm for finding all maximal scoring subsequences. Huang [9] extended the well-known recurrence relation used by Bentley [2] for solving the maximum sum consecutive subsequence problem, and derived a linear-time algorithm for computing the optimal segments with lengths at least L . Lin, Jiang, and Chao [10] and Fan *et al.* [5] studied the maximum-sum segment problem with length at least L and at most U .

In this paper, we consider a more general problem in which we wish to find the maximum-sum

segment whose starting and ending indices of the segment lie in given intervals. By preprocessing the sequence in $O(n)$ time, each query can be answered in $O(1)$ time. This also yields an alternative linear-time algorithm for computing the maximum-sum segment with length constraints.

The rest of the paper is organized as follows. Section 2 gives the formal definition of the RMSQ problem and introduces the RMQ techniques [6]. Section 3 coping with the special case of the RMSQ problem (called the SRMSQ problem). Section 4 gives the optimal algorithm for the RMSQ problem.

2 Problem Definition and Preliminaries

The input is a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$ of (not necessarily positive) real numbers. The maximum-sum segment of A is the contiguous subsequence having the greatest total sum, where the sum of a subsequence $S(i, j) = \langle a_i, \dots, a_j \rangle$ is simply $w(S(i, j)) = \sum_{k=i}^j a_k$. For simplicity, throughout the paper, the term "subsequence" will be taken to mean "contiguous subsequence". To avoid ambiguity, we disallow nonempty, zero-sum prefix or suffix (also called ties) in the maximum-sum segments. We define $c[i] = \sum_{k=1}^i a_k$ as the cumulative sum of A for all $1 \leq i \leq n$ and $c[0] = 0$. Notice that $w(S(i, j)) = c(j) - c(i - 1)$.

As an example, consider the input sequence $A = \langle 4, -5, 2, -2, 4, 3, -2, 6 \rangle$. The maximum-sum segment of A is $M = \langle 4, 3, -2, 6 \rangle$, with a total sum of 11. There is another subsequence tied for this sum by appending $\langle 2, -2 \rangle$ to the left end of M , but this subsequence is not the maximum-sum segment, since it has a nonempty zero-sum prefix.

2.1 Problem Definition

We start with a special case of the RMSQ problem, called the SRMSQ problem.

Problem 1. *A Special Case of the RMSQ problem (SRMSQ)*

Structure to Preprocess: $A = \langle a_1, a_2, \dots, a_n \rangle$ is a sequence of n real numbers.

Query: For an interval $I = (i, j)$, $1 \leq i \leq j \leq n$, $SRMSQ_A(i, j)$ returns a pair of indices (x, y) , $i \leq x \leq y \leq j$, such that segment $S(x, y)$ is the maximum-sum segment of the subsequence $\langle a_i, \dots, a_j \rangle$.

A naïve algorithm is to build a $n \times n$ table storing answers to all of the $O(n^2)$ possible queries. Each entry (i, j) in the table represents an interval (i, j) so we only have to fill in the upper-triangular part of the table. By applying the well known linear time algorithm for finding the maximum-sum segments of a sequence, we have an $O(n^3)$ preprocessing algorithm. Notice that answering a SRMSQ query now requires just one lookup to the table. To achieve $O(n^2)$ preprocessing rather than the $O(n^3)$ naïve preprocessing, we use the online property of the algorithm, filling the table row-by-row. In the paper, our algorithm can achieve $O(n)$ time and space preprocessing, and $O(1)$ time for each query.

Problem 2. *the Range Maximum-Sum Segment Query problem (RMSQ)*

Structure to Preprocess: $A = \langle a_1, a_2, \dots, a_n \rangle$ is a sequence of n real numbers.

Query: For two intervals $S = (i, j)$ and $E = (k, l)$, $1 \leq i \leq j \leq n$ and $1 \leq k \leq l \leq n$, $RMSQ_A(i, j, k, l)$ returns a pair of indices (x, y) , such that $w(S(x, y))$ is maximized for $i \leq x \leq j$ and $k \leq y \leq l$.

This is a generalized version of the SRMSQ problem because when $i = k$ and $j = l$ we are actually querying $SRMSQ_A(i, j)$. The naïve algorithm will have to build a 4-dimensional table and the time complexity for preprocess will achieve $\Omega(n^4)$. This is definitely inefficient for practical use. We will also provide an algorithm with $O(n)$ preprocessing time and $O(1)$ query time.

2.2 The RMQ Techniques

We next introduce an important technique, called RMQ, used in our algorithm. We are given a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$ to be preprocessed. A Range Minima Query (RMQ) specifies an interval I and the goal is to find the index k with minimum value a_k for $k \in I$.

Lemma 1. *The RMQ problem can be solved in $O(n)$ time and space preprocessing and $O(1)$ time per query. [1, 6]*

The well known algorithm for the RMQ problem is to first construct the Cartesian tree (defined by Vuillemin 1980) of the sequence, then be preprocessed for LCA (Least Common Ancestor) queries [8, 4]. This algorithm can be easily modified to output the index k for which a_k achieves the minimum or the maximum. We denote RMQ_{min} as the minimum query and RMQ_{max} as the maximum query. That is, $RMQ_{min}(A, i, j)$ will return index k such that a_k achieves the minimum for $i \leq k \leq j$, and $RMQ_{max}(A, i, j)$ will return index k such that a_k achieves the maximum. For the correctness of our algorithm if there are more than two minimums (maximums) in the query interval, it always outputs the rightmost (leftmost) index k for which a_k achieves the minimum (maximum). This can be done by constructing the Cartesian tree in a particular order.

3 Coping with the SRMSQ Problem

A key idea to solve the SRMSQ problem is to view the problem in the sense of the cumulative sum. For convenience of later proof, we give the following definition.

Definition 1. *Let A be any nonempty real number sequence. We define $l[j]$ for each index j of A to be the largest index k such that $c[k] \geq c[j]$ and $1 \leq k \leq j - 1$. But if no such k exists, that is, $c[j] > c[k]$ for all $1 \leq k \leq j - 1$, we assign $l[j] = 0$.*

Such largest index $l[j]$ and the cumulative sum $c[j]$ for each index j of A can be computed by the PREPROCESS1 algorithm as illustrated in Figure 1.

Algorithm PREPROCESS1

Input: An array of n real numbers $A[1 \dots n]$.

Output: A length n array $l[\cdot]$ and a length $n + 1$ array $c[\cdot]$

```
1   $c[0] \leftarrow 0$ ;  
2  for  $j \leftarrow 1$  to  $n$  do  
3     $c[j] \leftarrow c[j - 1] + A[j]$ ;  
4     $l[j] \leftarrow j - 1$ ;  
5    while  $c[l[j]] < c[j]$  and  $l[j] > 0$  do  
6       $l[j] \leftarrow l[l[j]]$ ;  
7    end while  
8  end for
```

Figure 1: Algorithm for computing $c[\cdot]$ and $l[\cdot]$

Lemma 2. *Let A be any nonempty real number sequence. If segment $S(i, j)$ is the maximum-sum segment of A , then index i is the largest index such that the cumulative sum $c[i - 1]$ is minimized for $l[j] \leq i - 1 < j$ (that is, $c[i - 1]$ is the rightmost minimum).*

Proof. Suppose not, then either (1) index $i - 1$ lies in the interval $[0, l[j] - 1]$ or (2) index $i - 1$ lies in the interval $[l[j], j - 1]$ but is not the largest index such that the cumulative sum $c[i - 1]$ is minimized for $l[j] \leq i - 1 < j$. We discuss it as follows.

(1) Suppose index $i - 1$ lies in the interval $[0, l[j] - 1]$:

When $l[j] = 0$, it's obvious that $i - 1$ cannot lie in the interval $[0, l[j] - 1]$. When $l[j] > 0$, we have $w(S(i, l[j])) = c[l[j]] - c[i - 1] \geq c[j] - c[i - 1] = w(S(i, j))$. If equality holds, then $w(S(l[j] + 1, j)) = c[j] - c[l[j]] = 0$. So $S(l[j] + 1, j)$ would be a zero-sum prefix of $S(i, j)$. Thus, $w(S(i, l[j]))$ must be strictly greater than $w(S(i, j))$. But, this contradicts to the fact that $S(i, j)$ is the maximum-sum segment of A . So index $i - 1$ cannot lie in the interval $[0, l[j] - 1]$.

(2) Suppose index $i - 1$ lies in the interval $[l[j], j - 1]$:

If $c[i - 1]$ is not minimized for $l[j] \leq i - 1 < j$. That is, there exists an index k , $k \neq i$ and $l[j] \leq k - 1 < j$, such that $c[k - 1] < c[i - 1]$. Then we have $w(S(k, j)) = c[j] - c[k - 1] > c[j] - c[i - 1] = w(S(i, j))$. This contradicts to the fact that $S(i, j)$ is the maximum-sum segment of A . So the cumulative sum $c[i - 1]$ must be minimized for $l[j] \leq i - 1 < j$. We further sup-

pose that $c[i - 1]$ is not the rightmost minimum, that is, there exists an index $k' > i$ such that $c[k' - 1]$ is also a minimum. Then $w(S(i, k' - 1)) = c[k' - 1] - c[i - 1] = 0$. So $S(i, j)$ has a zero-sum prefix $S(i, k' - 1)$ which also contradicts to the definition of the maximum-sum segment. Hence, index i must be the largest index such that $c[i - 1]$ is minimized for $l[j] \leq i - 1 < j$. $\square$

Definition 2. *Let A be any nonempty real number sequence. We define the "good partner", $p[j]$, of each index j in A as the largest index k such that $c[k - 1]$ is minimized for $l[j] \leq k - 1 < j$. And segment $S(p[j], j)$ is called a candidate segment of A at j . We also define $m[j]$ to be the sum of the candidate segment of A at j , that is $m[j] = w(S(p[j], j))$.*

By Lemma 2, we know that each pair $(p[j], j)$ constitutes a candidate solution of the maximum-sum segment of A , that is, segment $S(p[j], j)$. The relationship between $l[j]$ and $p[j]$ as defined above is illustrated in Figure 2. The left side of the figure shows the case that there exists a largest index $l[j]$ such that $c[l[j]] \geq c[j]$ and $1 \leq l[j] \leq j$. And the right side of the figure shows the case that $c[j]$ is the unique maximum of $c[k]$ for all $1 \leq k \leq j$.

The good partner $p[j]$ and the sum of the candidate segment $m[j]$ for each index j of A can be computed by the PREPROCESS2 algorithm as illustrated in Figure 3.

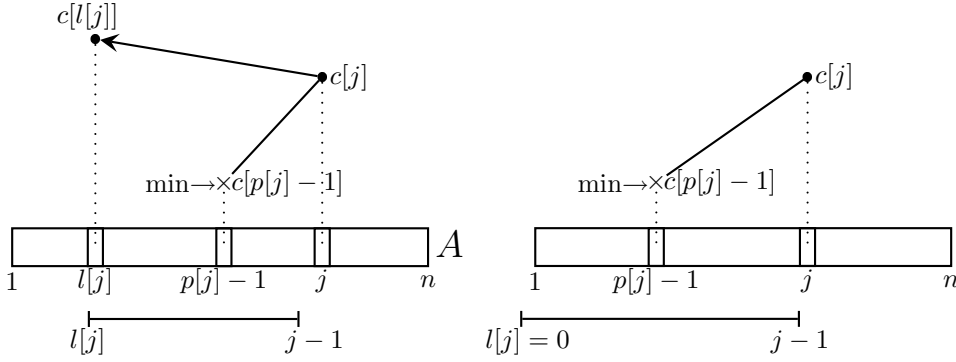


Figure 2: An illustration for $l[\cdot]$ and $p[\cdot]$.

Algorithm PREPROCESS2

Input: An array of n real numbers $A[1 \dots n]$, array $c[\cdot]$ and array $l[\cdot]$.

Output: Two length n arrays, $p[\cdot]$ and $m[\cdot]$.

- 1 Apply RMQ_{min} preprocess to array $c[\cdot]$.
 - 2 **for** $j \leftarrow 1$ **to** n **do**
 - 3 $p[j] \leftarrow RMQ_{min}(c, l[j], j-1) + 1$;
 - 4 $m[j] \leftarrow c[j] - c[p[j]-1]$;
 - 5 **end for**
-

Figure 3: Algorithm for computing $p[\cdot]$ and $m[\cdot]$

Lemma 3. *Let A be any nonempty real number sequence of length n . If index i satisfies $m[i] \geq m[k]$ for all $1 \leq k \leq n$, then $S(p[i], i)$ is the maximum-sum segment.*

Proof. Suppose on the contrary, segment $S(s, t)$, $(s, t) \neq (p[i], i)$, is the maximum-sum segment. By Lemma 2, we have $s = p[t]$. So $m[t] = w(S(p[t], t)) = w(S(s, t)) > w(S(p[i], i)) = m[i]$ which contradicts to $m[i]$ is the maximum value. $\square$

Lemma 3 tells us, once we have computed $m[j]$ and $p[j]$ for each index j of A . To find the maximum-sum segment of A , we only have to retrieve the index i such that $m[i] \geq m[k]$ for all $1 \leq k \leq n$. Then, $S(p[i], i)$ is the maximum-sum segment of A . Lemma 4-6 will show some important properties of the candidate segments.

Lemma 4. *Let A be any nonempty real number sequence. If $p[j]$ is the good partner of index j and $p[j] < j$, then $c[p[j]-1] < c[k] < c[j]$ for all $p[j]-1 < k < j$.*

Proof. Suppose not. That is, there exists an index k' , $p[j]-1 < k' < j$, such that $c[k'] \leq c[p[j]-1]$ or $c[k'] \geq c[j]$. (1) If $c[k'] \leq c[p[j]-1]$. By definition of $p[j]$, we know that $l[j] \leq p[j] < j$. Since $p[j]-1 < k'$, we have $l[j] < k' < j$ and $c[k'] \leq c[p[j]-1]$. This contradicts to the definition of $p[j]$ to be the largest index and $c[p[j]-1]$ is minimized for $l[j] \leq p[j]-1 < j$. (2) If $c[k'] \geq c[j]$, then by definition of $p[j]$, we have $k' \leq p[j]-1 < j$. So $k' \leq p[j]-1 < k'$. A contradiction occurs. $\square$

That is, $c[p[j]-1]$ is the unique minimum and $c[j]$ is the unique maximum of $c[k]$ for all $p[j]-1 \leq k \leq j$. The following lemma shows the nesting property of the candidate segments. See Figure 5. Notice that, the pointer of each index j points to the position of $p[j]$.

Algorithm PREPROCESS OF SRMSQ_A(i, j)

- 1 Run algorithm PREPROCESS1 to compute array $c[\cdot], l[\cdot]$ of A .
- 2 Run algorithm PREPROCESS2 to compute array $p[\cdot], m[\cdot]$ of A .
- 3 Apply RMQ_{max} preprocess to array $m[\cdot]$.
- 4 Apply RMQ_{min} preprocess to array $c[\cdot]$.

Algorithm QUERY OF SRMSQ_A(i, j)

- 1 $r \leftarrow \text{RMQ}_{\max}(m, i, j)$
 - 2 **if** $p[r] < i$ **then**
 - 3 $r_1 \leftarrow \text{RMQ}_{\min}(c, i - 1, r - 1) + 1$;
 - 4 $r_2 \leftarrow \text{RMQ}_{\max}(m, r + 1, j)$;
 - 5 **if** $c[r] - c[r_1 - 1] < m[r_2]$ **then**
 - 6 OUTPUT ($p[r_2], r_2$);
 - 7 **else**
 - 8 OUTPUT (r_1, r);
 - 9 **end if**
 - 10 **else**
 - 11 OUTPUT ($p[r], r$);
 - 12 **end if**
-

Figure 4: Algorithm for the SRMSQ problem.

Lemma 5. *Let A be any nonempty real number sequence. For two indices i and j , $i < j$, if $p[i]$ is the good partner of i and $p[j]$ is the good partner of j , then it cannot be the case that $p[i] < p[j] \leq i < j$.*

Proof. Suppose $p[i] < p[j] \leq i < j$ holds. By Lemma 4, we have

$$(1) \ c[p[i] - 1] < c[k'] < c[i] \quad \forall \ p[i] - 1 < k' < i$$

$$(2) \ c[p[j] - 1] < c[k''] < c[j] \quad \forall \ p[j] - 1 < k'' < j$$

Since $p[j] - 1 < i$, we can substitute $p[j] - 1$ for k' in (1) $\Rightarrow c[p[i] - 1] < c[p[j] - 1] < c[i]$.

Similarly, we can substitute i for k'' in (2) $\Rightarrow c[p[j] - 1] < c[i] < c[j]$.

Then we have

$$(3) \ c[p[i] - 1] < c[p[j] - 1] < c[k''] < c[j] \quad \forall \ p[j] - 1 < k'' < j$$

$$(4) \ c[p[i] - 1] < c[k'] < c[i] < c[j] \quad \forall \ p[i] - 1 < k' < i$$

By (3) and (4), we have

$$(5) \ c[p[i] - 1] < c[k'''] < c[j] \quad \forall \ p[i] - 1 < k''' < j$$

So, if there exists an index k such that $c[k] \geq c[j]$, then $k < p[i] - 1$. We also have by (5) that $c[p[i] - 1] < c[p[j] - 1]$ which is a contradiction to that $c[p[j] - 1]$ minimizes $c[l]$ for $k < l < j$. If there is no such index k , then $c[p[i] - 1] < c[p[j] - 1]$ is a contradiction to that $c[p[j] - 1]$ minimizes $c[l]$ for $l < j$. $\square$

Now, we are about to establish the relationship

between sequence A and its subsequence Q . The following key lemma will show that a candidate segment S of A is still a candidate segment of Q if Q contains the whole candidate segment S .

Lemma 6. *Let $A = \langle a_1, \dots, a_n \rangle$ be any nonempty real number sequence and $Q = \langle a_s, \dots, a_t \rangle$ be any subsequence of A . If $p[j]$ is the good partner of index j for sequence A and $s \leq p[j] \leq j \leq t$, then $p[j]$ is still the good partner of index j for sequence Q .*

Proof. Let $c[j]$ be the cumulative sum of each index j in Q . Then, $c[j] = \sum_{k=s}^j a_k = \sum_{k=1}^j a_k - \sum_{k=1}^{s-1} a_k = c[j] - c[s-1]$ for all $s-1 \leq j \leq r$. Let $l[j]$ be the largest index k such that $c[k] \geq c[j]$ for all $s \leq k \leq j-1$. If no such k exists, we assign $l[j] = s-1$. Our goal is to prove that $p[j]$ is the largest index k that minimizes $c[k-1]$ for all $l[j] \leq k-1 < j$.

(1) If $l[j] \geq s$. Since $l[j]$, by definition, is the largest index k such that $c[k] \geq c[j]$ for all $1 \leq k \leq j-1$. So $l[j]$ is the largest index k such that $c[k] = c[k] - c[s-1] \geq c[j] - c[s-1] = c[j]$ for all $1 \leq s \leq k \leq j-1$. Hence, we have $l[j] = l[j]$. And $p[j]$, by definition, is the largest index k that minimizes $c[k-1]$ for all $l[j] \leq k-1 < j$. So $p[j]$ is the largest index k that minimizes $c[k-1] = c[k-1] - c[s-1]$ for all $l[j] = l[j] \leq k-1 < j$.

(2) If $l[j] < s$. That is, $c[k] < c[j]$ for all

$s \leq k \leq j-1$. So, $c[k] = c[k] - c[s-1] < c[j] - c[s-1] = c[j]$ for all $s \leq k \leq j-1$. Hence, we have $l[j] = s-1$. Since $p[j]$ is the largest index k that minimizes $c[k-1]$ for all $l[j] \leq k-1 < j$. So, $p[j]$ is the largest index k that minimizes $c[k-1] = c[k-1] - c[s-1]$ for all $l[j] \leq s-1 \leq k-1 < j$. That is, $p[j]$ is the largest index k that minimizes $c[k-1]$ for all $l[j] \leq k-1 < j$. $\square$

Corollary 1. Let $A = \langle a_1, \dots, a_n \rangle$ be any nonempty real number sequence and $Q = \langle a_s, \dots, a_t \rangle$ be any subsequence of A . If $p[j]$ is the good partner of index j for sequence A , $s \leq p[j] \leq j \leq t$, and $m[j]$ is the sum of the candidate segment of A at j . Then $m[j]$ is also the sum of the candidate segment of Q at j .

Proof. A direct result of Lemma 6. $\square$

Now, we are ready to present our algorithm for the SRMSQ problem (See Figure 4). Let A be a sequence of n real numbers and $[i, j]$ is the query interval, where $1 \leq i \leq j \leq n$.

For instance, see Figure 5, the input sequence A has 15 elements. suppose we are querying $\text{SRMSQ}_A(3, 7)$. The QUERY OF SRMSQ algorithm in Figure 4 will first query the index r such that $m[r]$ is maximized for $3 \leq r \leq 7$ (line 1). In this case, $r = 5$, which means candidate segment $S(p[5], 5)$ has the largest sum compared to other segments. The left end of $S(p[5], 5)$, $p[5] = 3$, lies in the interval $[3, 7]$. The algorithm executes line 11, output $(p[5], 5)$, which means segment $S(3, 5)$ is the maximum-sum segment of the subsequence $A[3 \dots 7]$.

Suppose we are querying $\text{SRMSQ}_A(6, 12)$, $\text{RMQ}_{\max}(m, 6, 12)$ will return index 9 (line 1). Since $p[9] = 3 < 6$, line 3-9 will be executed. In line 3, $\text{RMQ}_{\min}(c, 5, 8)$ will return index 8. In line 4, $\text{RMQ}_{\max}(m, 10, 12)$ will return index 11. In line 5, since $c[9] - c[8] = 6 < m[11] = 8$, the QUERY OF RMSQ algorithm will output $(p[11], 11)$, which means $S(11, 11)$ is the maximum-sum segment of the subsequence $A[6 \dots 12]$.

Theorem 1. Algorithm QUERY OF $\text{SRMSQ}_A(i, j)$ will output the maximum-sum segment of the subsequence $S(i, j)$.

Proof. Let $m[k]$ be the sum of the candidate segment of $S(i, j)$ at k and $c[k]$ be the cumulative

sum of $S(i, j)$ for $i-1 \leq k \leq j$. We have $c[k] = c[k] - c[i-1]$, $i-1 \leq k \leq j$. Let $p[k]$ be the good partner of index k of Q for all $i \leq k \leq j$. Let index r satisfy $m[r] \geq m[k]$ for all $i \leq k \leq j$ (line 1).

(A) If $p[r] \geq i$: Our goal is to show that $m[r] \geq m[k]$ for all $i \leq k \leq j$, and then apply Lemma 3 to complete the proof. We first consider each index k' such that $i \leq k' \leq j$ and $i \leq p[k'] \leq k' \leq j$. By corollary 1, we can deduce that $m[k'] = m[k'] \leq m[r] = m[r]$. We next consider each index k'' such that $i \leq k'' \leq j$ and $p[k''] < i$. Since $p[k''] < i \leq k''$, we can apply Lemma 4 and get $c[p[k'']-1] < c[k] < c[k''] \quad \forall \quad p[k'']-1 < k < k''$ (1)

Since $i \leq p[k''] < k''$ by definition of $p[k'']$, we have $p[k'']-1 < i-1 \leq p[k'']-1 < k''$. So, we can substitute $p[k'']-1$ for k in (6) and get $c[p[k'']-1] < c[p[k'']-1]$. Hence, we can deduce that $m[k''] = c[k''] - c[p[k'']-1] = c[k''] - c[p[k'']-1] < c[k''] - c[p[k'']-1] = m[k'']$. Moreover, $m[k''] \leq m[r] = m[r]$. So, we have $m[k''] < m[r]$. Till now, we have shown that $m[r] \geq m[k]$ for all $i \leq k \leq j$. By Lemma 3, $S(p[r], r)$ is the maximum-sum segment of $S(i, j)$ (line 11).

(B) If $p[r] < i$: We first consider each index k' such that $i \leq k' < r$. Since $p[r] < i \leq r$, we can apply Lemma 4 and obtain

$c[p[r]-1] < c[k] < c[r] \quad \forall \quad p[r]-1 < k < r$ (2)
Since $c[k] < c[r]$ for all $p[r]-1 < i \leq k < r$, we have $c[k] = c[k] - c[i-1] < c[r] - c[i-1] = c[r] \quad \forall \quad i \leq k < r$. For any segment $S(k, k')$, $i \leq k < r$, since $w(S(k, k')) = c[k'] - c[k] < c[r] - c[k] = w(S(k, r))$, it's not hard to see that $S(k, k')$ cannot be the maximum-sum segment.

We next consider each index k'' such that $r < k'' \leq j$. By Lemma 5, we know that it cannot be the case $p[r] < p[k''] \leq r < k''$. If $p[k''] \leq r$, then it must be the case that $p[k''] \leq p[r] < r < k''$. Since $p[k'']-1 \leq r < k''$ and $p[k'']-1 \leq p[r]-1 < k''$, we can apply Lemma 4 and get $c[p[k'']-1] < c[r] < c[k'']$ and $c[p[k'']-1] < c[p[r]-1] < c[k'']$. So, $m[k''] = c[k''] - c[p[k'']-1] > c[r] - c[p[r]-1] = m[r]$ which contradicts to that $m[r] \geq m[k]$ for all $i \leq k \leq j$. So, $p[k''] \not\leq r$, that is, $p[k''] > r$. Hence, by corollary 1, we have $m[k''] = m[k'']$ for $r < k'' \leq j$. Let index r_2 satisfies $m[r_2] \geq m[k]$ for all $r+1 \leq k \leq j$ (line 4). It's not hard to see that either $S(r_1, r)$ or $S(p[r_2], r_2)$ is the maximum-sum segment of $S(i, j)$. By Lemma 3, the one with greater sum is the maximum-sum segment of $S(i, j)$ (line 5-9). $\square$

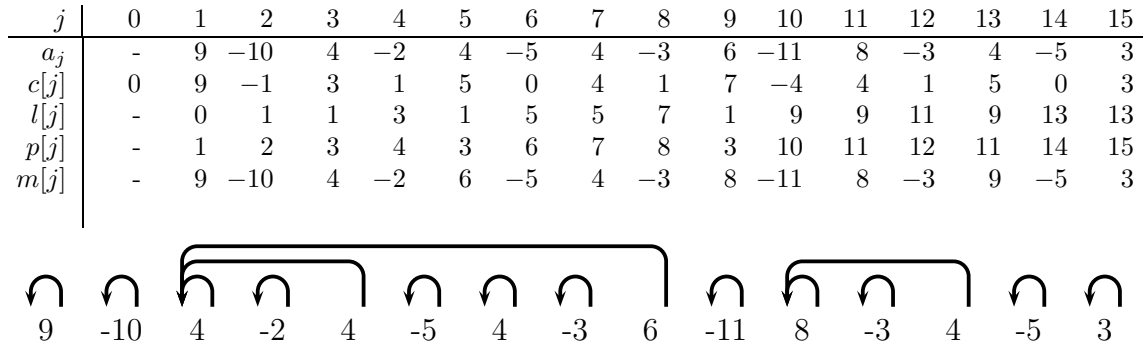


Figure 5: The candidate segment $S(p[j], j)$ of each index j .

Algorithm PREPROCESS OF RMSQ

- 1 Apply SRMSQ preprocess.
- 2 Apply $\text{RMQ}_{\max}$ preprocess to $c[\cdot]$.

Algorithm QUERY OF $\text{RMSQ}_A(i, j, k, l)$

- 1 **if** $j \leq k$ **then**
 - 2 OUTPUT ($\text{RMQ}_{\min}(c, i-1, j-1) + 1, \text{RMQ}_{\max}(c, k, l)$)
 - 3 **else**
 - 4 $(r_1, r'_1) \leftarrow (\text{RMQ}_{\min}(c, i-1, k-1) + 1, \text{RMQ}_{\max}(c, k, l));$
 - 5 $(r_2, r'_2) \leftarrow (\text{RMQ}_{\min}(c, k, j-1) + 1, \text{RMQ}_{\max}(c, j, l));$
 - 6 $(r_3, r'_3) \leftarrow \text{SRMSQ}_A(k, j);$
 - 7 OUTPUT (r_m, r'_m) such that $c[r_m] - c[r'_m]$ is maximized for $1 \leq m \leq 3;$
 - 8 **end if**
-

Figure 6: Algorithm for the RMSQ problem.

Lemma 7. *Algorithm PREPROCESS1 runs in $O(n)$ time.*

Proof. It can be shown by a simple amortized analysis. The total number of operations of the algorithm is clearly bounded by $O(n)$ except for the while-loop body of Steps 5-7. In the following, we show that the amortized cost of the while-loop is a constant. Therefore, the overall time required by the loop is $O(n)$. We define the *potential function* of A after the i th iteration of the for-loop to be $\Phi(i)$, i.e. the number of times pointer $l[i]$ may be advanced most. So we have $\Phi(i) \geq 0$ always holds in every iteration. Suppose that the pointer $l[\cdot]$ is advanced c_i times in this period. Then the actual cost of the operations is $c_i + 1$. Observing that $\Phi(i) = \Phi(i-1) - c_i + 1$, the change of the potential of A during the i th iteration is $\Phi(i) - \Phi(i-1) = \Phi(i-1) - c_i + 1 - \Phi(i-1) = 1 - c_i$. The amortized cost is therefore calculated as $\hat{c}_i = c_i + 1 + \Phi(i) - \Phi(i-1) = 2$. Since exactly

n iterations would be executed in the entire process, the while-loop spends at most overall $O(n)$ time. $\square$

Lemma 8. *Algorithm PREPROCESS2 runs in $O(n)$ time.*

Proof. By Lemma 1, the preprocessing time for RMQ is $O(n)$ and the query time is $O(1)$. So, the cost of each step in the for-loop is a constant and the overall time required is $O(n)$. $\square$

Theorem 2. *The SRMSQ problem can be solved in $O(n)$ preprocessing time and $O(1)$ query time.*

Proof. By Lemma 1, Lemma 7, and Lemma 8, the algorithms for preprocess all run in $O(n)$ time. So, the time complexity for the PREPROCESS OF SRMSQ algorithm is $O(n)$. Since each RMQ query in the QUERY OF SRMSQ algorithm takes $O(1)$ time, one can easily see that the query time for the QUERY OF SRMSQ algorithm is $O(1)$.

It's not hard to verify that the space complexity for these algorithms is also $O(n)$. $\square$

4 Coping with the RMSQ Problem

The RMSQ problem is to answer the queries comprising of two intervals $[i, j]$ and $[k, l]$, where $[i, j]$ specifies the range of the starting index of the maximum-sum segment, and $[k, l]$ specifies the range of the ending index. Since it is meaningless if the range of the starting index is in front of the range of the ending index. So, without loss of generality, we assume that $i \leq k$ and $j \leq l$. We presents our algorithm as follows.

Theorem 3. *Algorithm QUERY OF $RMSQ_A(i, j, k, l)$ will output the correct answer.*

Proof. We discuss it under two possible conditions.

(1) Nonoverlapping($j \leq k$): Suppose the intervals $[i, j]$ and $[k, l]$ do not overlap. Since $w(S(x, y)) = c(y) - c(x - 1)$, to maximize $S(x, y)$ is equivalent to maximize $c(y)$ and minimize $c(x - 1)$ for $i \leq x \leq j$ and $k \leq y \leq l$. By applying the RMQ technique to preprocess $c[\cdot]$, the maximum-sum segment can be easily located.(line 2)

(2) Overlapping($j > k$): When it comes to the overlapping case, just to find the maximum cumulative sum and the minimum cumulative sum might go wrong if the minimum is on the right of the maximum. We discuss it under 3 possible conditions for the maximum-sum segment $S(x, y)$. (a) Suppose $i \leq x \leq k$ and $k \leq y \leq l$, this is an nonoverlapping case. So we find the minimum cumulative sum and the maximum cumulative sum.(line 4) (b) Suppose $k + 1 \leq x \leq j$ and $j \leq y \leq l$, this is also an nonoverlapping case. We find the minimum cumulative sum and the maximum cumulative sum.(line 5) (c) Otherwise, $k + 1 \leq x \leq j$ and $k + 1 \leq y \leq j$, this is exactly the same as a $SRMSQ_A(k + 1, j)$ query.(line 6) The maximum sum segment $S(x, y)$ must be one of these three possible cases and have the largest sum(line 7). $\square$

Theorem 4. *The RMSQ problem can be solved in $O(n)$ preprocessing time and $O(1)$ query time.*

Proof. The time for the RMQ_{max} and the $SRMSQ$ preprocesses are $O(n)$. So the PREPROCESS OF RMSQ algorithm costs $O(n)$ time. And the query time is $O(1)$ since each step in the QUERY OF RMSQ algorithm costs $O(1)$ time. $\square$

Acknowledgements. We thank Yu-Ru Huang and An-Chiang Chu for helpful conversations. Quan-Yu Chen and Kun-Mao Chao were supported in part by an NSC grant 92-2213-E-002-059.

References

- [1] M. A. Bender, and M. Farach-Colton. The LCA Problem Revisited. In *Proceedings of the 4th Latin American Symposium on Theoretical Informatics*, 17: 88–94, 2000.
- [2] J. Bentley, Programming Pearls - Algorithm Design Techniques, *CACM*, 865-871, 1984.
- [3] K. Chung and H.-I. Lu. An Optimal Algorithm for the Maximum-Density Segment Problem. In *Proceedings of the 11th Annual European Symposium on Algorithms (ESA 2003)*, LNCS 2832, pp. 136-147, 2003.
- [4] D. Harel and R. E. Tarjan. Fast Algorithms for Finding Nearest Common Ancestors. *SIAM J Comput.* Vol. 13, No 2, 1984.
- [5] T.-H. Fan, S. Lee, H.-I. Lu, T.-S. Tsou, T.-C. Wang, and A. Yao. An Optimal Algorithm for Maximum-Sum Segment and Its Application in Bioinformatics. *CIAA*, LNCS 2759, pp. 251-257, 2003.
- [6] H. Gabow, J. Bentley, and R. Tarjan. Scaling and Related Techniques for Geometry Problems. *Proc. Symp Theory of Computing (STOC)*, 1984, 135-143.
- [7] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1999.
- [8] B. Schieber and U. Vishkin. On Finding Lowest Common Ancestors: Simplification and Parallelization. *SIAM J Comput.* Vol. 17, No 6, 1988.
- [9] Huang, X. (1994) An algorithm for identifying regions of a DNA sequence that satisfy a content requirement. *CABIOS*, **10**, 219-225.
- [10] Y.-L. Lin, T. Jiang, and K.-M. Chao. Efficient Algorithms for Locating the Length-constrained Heaviest Segments with Applications to Biomolecular Sequence Analysis. *Journal of Computer and System Sciences*, 65, 570-586, 2002.

- [11] S. Muthukrishnan, Efficient Algorithms for Document Retrieval Problems. In *Proceedings of the 13th Annual ACM-SIAM Symposium on Discrete Algorithms*, 657–666, 2002.
- [12] W. L. Ruzzo and M. Tompa. A Linear Time Algorithm for Finding All Maximal Scoring Subsequences. In *7th Intl. Conf. Intelligent Systems for Molecular Biology, Heidelberg, Germany*, Aug. 1999.

Efficient Algorithms for Some Variants of the Farthest String Problem

Chih Huai Cheng, Ching Chian Huang, Shu Yu Hu, Kun-Mao Chao
Department of Computer Science and Information Engineering
National Taiwan University
Taipei, Taiwan 106
{b89010, b89037, b89117, kmchao}@csie.ntu.edu.tw

Abstract

The farthest string problem (FARTHEST STRING) is one of the core problems in the field of consensus word analysis and several biological problems such as discovering potential drugs, universal primers, or unbiased consensus sequences. Given k strings of the same length L and a nonnegative integer d , FARTHEST STRING is to find a string s such that none of the given strings has a Hamming distance that is smaller than d from s . It has been shown to be NP-complete. In this paper, we study two variants of FARTHEST STRING. One is to search for a string s satisfying the condition that the hamming distances between s and all the given strings are greater than d . We give an $O(|\Sigma|(L-d)^{(L-d)})$ -time algorithm, where $|\Sigma|$ is the alphabet size. The other variant is to find a string s such that the sum of the hamming distances between s and all the given strings are maximized. We solve this problem in $O(kL)$ time.

1 Introduction

With the development of genetic technology, more and more problems are solved by using genetic information. We can distinguish related species from other species by comparing the DNA difference

between one another [7]. All these problems can be reduced to finding a pattern which occurs in one set of strings (the Closest String problem) but does not occur in another set (the Farthest String problem) [6]. FARTHEST STRING is an important problem in computational biology. This paper proposes efficient algorithms to solve the following two problems. The first one is the farthest string problem with fixed parameters. Where k strings with length L and a non-negative number d is given, the problem is to determine whether a string s that satisfies the condition that the hamming distance between s and all the given strings are greater than d . The second problem is to find the farthest string s under the condition that s must reside in the set of the given strings and has the maximum sum of hamming distance with respect to all other strings.

The farthest string problem is proven to be NP-complete [8] and there is a polynomial-time approximation scheme (PTAS) for the farthest string problems based on a linear programming relaxation technique [8]. Now, consider the two most natural parameters concerning farthest strings: the minimum Hamming distance d and the number of given input strings k . In some applications d is (very) large, so it is important to ask whether efficient algorithms are possible when d or k is fixed. FARTHEST STRING,

although NP-complete in general, is tractable [2, 5] for fixed parameters d and Σ by using the bounded search tree algorithm [1, 3, 4].

The rest of the paper is organized as follows. In Section 2, we give the general strategy and the main idea of the algorithm for the farthest string problem with fixed parameters and show the running time and correctness under the condition of binary strings first. Next, we extend the result to arbitrary alphabet instances. In Section 3, we give a formal definition of the variant of the farthest string problem and propose an efficient algorithm to solve the problem in $O(kL)$. We then prove the correctness and time complexity of the algorithm.

2 Farthest string with fixed parameter

As we stated in the introduction, the farthest string problem here is given k strings with the same length L and a non-negative number d , and trying to find a string s that satisfies the condition that the hamming distance between s and all the given strings are greater than d . We give the formal definition of the farthest string problem in mathematical terms:

Input: Strings $s_1, s_2, \dots, s_k$ over alphabet Σ of length L , and a nonnegative integer d .

Question: Is there a string s of length L such that $d_H(s, s_i) \geq d$ for all $i = 1, \dots, k$?

Before presenting the algorithm for this problem, we give some definitions and minor results that will be used in the algorithm.

Lemma 1. *Given a set of strings $S = \{s_1, s_2, \dots, s_k\}$ and a positive integer d . If there are $i, j \in \{1, \dots, k\}$ with $d_H(s_i, s_j) > x$, then there is no string s with*

$$\min_{i=1, \dots, k} \{d_H(s, s_i)\} > L-x/2.$$

Proof. Suppose the hamming distance between s_i and s_j is greater than x , this means that there are at most $L-x$ positions with the same symbol and at least x positions with different symbols at the same relative positions on s_i and s_j . To achieve the greatest hamming distance between s_i and s_j simultaneously, the complement of the symbol on all of the positions where s_i and s_j contain the same symbol is chosen. Thus we increase the hamming distance for s_i and s_j by at most $L-x$, on the other hand, changing a symbol on the other positions where s_i and s_j have different alphabets can increase the hamming distance by only at most $x/2$. Therefore the maximum hamming distance that an arbitrary string may have between s_i and s_j is $L-x + x/2 = L-x/2$ when $d_H(s_i, s_j) = x$. Hence, when $d_H(s_i, s_j) > x$ the maximum hamming distance on both s_i, s_j cannot exceed $L-x/2$.

Corollary 1. *Given a set of strings $S = \{s_1, s_2, \dots, s_k\}$ and a positive integer d . If there are $i, j \in \{1, \dots, k\}$ with $d_H(s_i, s_j) > 2L-2d$, then there is no string s with $\min_{i=1, \dots, k} \{d_H(s, s_i)\} \geq d$.*

Proof. By lemma1, we set $L-x/2 = d$ and then the value of x can be deduced easily. We can find that $x = 2d-L$, and therefore no string s exist that satisfies the condition that $\min_{i=1, \dots, k} d_H(s, s_i) \geq d$.

Given a string s then $\bar{s}$ is the string defined by taking the complement alphabet of every position on string s . If s is a binary string then the job is to simply exchange the 1's to 0's and the 0's to 1's. If the alphabet set of s contains more than two elements, then for each string the complement string may have more than one choice. E.g. when the alphabet set = $\{0,1,2\}$ then the complement of alphabet 0 may be 1 or 2.

Lemma 2. Given a set of strings $S = \{s_1, s_2, \dots, s_k\}$ and a positive integer d . If there are $i, j \in \{1, \dots, k\}$ with $d_H(\bar{s}_i, s_j) < 2d - L$, then there is no string s with $\min_{i=1, \dots, k} \{d_H(s, s_i)\} \geq d$.

Proof. The hamming distance between complement of s_i and s_j is smaller than $2d - L$, it means that there are at most $2d - L$ positions with different symbols at the same relative positions on $\bar{s}_i$ and s_j . In other words, there are at least $L - (2d - L) = 2L - 2d$ positions with different symbols at the same relative positions on s_i and s_j . By the corollary mentioned above, lemma2 is proved.

The idea of our algorithm is to start with one of the given strings, e.g., $\bar{s}_1$, as a “candidate string.” If there is a string s_i , $i = 2, \dots, k$, that differs from the candidate string in less than d positions, we recursively try several ways to move the candidate string “away from” s_i ; the term “moving away” means that we select a position in which the candidate string and s_i have the same alphabet and set this position in the candidate string to the complement of the character of s_i at this position. We stop either if we moved the candidate “too far away” from $\bar{s}_1$ or if we found a solution. By a careful selection of subcases of this recursion we can limit the size of this search tree to $O((\sum (L-d))^{(L-d)})$, as will be shown in the following theorem. In Figure 1 we outline a recursive procedure solving FARTHEST STRING. It is based on the bounded search tree paradigm that is frequently successfully applied in the development of fixed-parameter algorithms [1, 4, 6].

Recursive procedure $FSD(s, \Delta d)$

Global variables: Set of strings $S = \{s_1, s_2, \dots, s_k\}$, integer d .
Input: Candidate string s and integer Δd .
Output: A string $\hat{s}$ with $\min_{i=1, \dots, k} \{d_H(\hat{s}, s_i)\} \geq d$ if it exists, and “not found,” otherwise.
(D0) If $\Delta d < 0$, then return “not found”;
(D1) If $d_H(s, s_i) < 2d - L$ for some $i \in \{1, \dots, k\}$, then return “not found”;
(D2) If $d_H(s, s_i) \geq d$ for all $i = 1, \dots, k$, then return s ;
(D3) Choose any $i \in \{1, \dots, k\}$ such that $d_H(s, s_i) < d$:
 $P := \{p \mid s[p] = s_i[p]\}$;
Choose any $P' \subset P$ with $|P'| = L - d + 1$;
For every $p \in P'$ do
 $s' := s$;
 $s'[p] := \overline{s_i[p]}$;
 $s_{ret} := FSD(s', \Delta d - 1)$;
If $s_{ret} \neq \text{“not found”}$, then return s_{ret} ;
(D4) Return “not found”

Fig.1 The algorithm we propose to search the farthest string. The recursive procedure creates a bounded search tree that traverses to discover the farthest string.

Theorem 1 Given a set of binary strings $S = \{s_1, s_2, \dots, s_k\}$ of length L , and an integer d , Algorithm FSD (Figure 1) determines in $O(kL(L-d)^{(L-d)})$ time whether there is a string with $\min_{i=1, \dots, k} \{d_H(s, s_i)\} \geq d$ and it computes such an s if one exists.

Proof.

Running time. We consider the recursive part of the algorithm. Parameter Δd is initialized to $L - d$. Every recursive call decreases Δd by one. The algorithm stops when $\Delta d < 0$. Therefore, the algorithm builds a

search tree of height at most d . In one step of the recursion, the algorithm chooses, given the current candidate string s , a string s_i such that $d_H(s, s_i) < d$. It creates a subcase for $L-d+1$ of the positions in which s and s_i have the same alphabet. This yields an upper bound of $(L-d+1)^{L-d} = O((L-d)^{L-d})$ on the search tree size.

Correctness. Let us consider the situation when we first enter the recursive procedure. In the first recursive call the candidate string s is $\bar{s}_1$. If $\bar{s}_1$ satisfies the condition that the hamming distance between $\bar{s}_1$ and s_i is no lesser than d , then we have our answer; If $\bar{s}_1$ does not satisfy the above condition and a furthest string s does indeed exist, then there exists some string s_i such that $d_H(\bar{s}_1, s_i) < d$. So $\bar{s}_1$ and s_i have at most $d-1$ positions that have different alphabets (one symbol is 0 and the other one is 1); in other words there are at least $L-d+1$ positions that contain the same alphabet. Since by assumption s is the farthest string, therefore s must satisfy the below conditions: s and s_i have at least d positions that contain different alphabets; in other words there at most $L-d$ positions containing same alphabets. So by the pigeonhole theorem, if we select $L-d+1$ positions from $\bar{s}_1$, and substitute it with the complement (choose 0 when alphabet is 1 ; choose 1 when alphabet is 0), then one of the changes must certainly make $\bar{s}_1$ closer to the farthest string s . In the each following recursive procedure will make the candidate string s (the parameter of *FSD*) closer to the farthest string (if the farthest string exists). So after the execution of the algorithm we will find the farthest string s if it exists.

Theorem 2 Given a set of $|\Sigma|$ alphabet strings $S = \{ s_1, s_2, \dots, s_k \}$ of length L , and an integer d , Algorithm FSD (Figure 1) determines in $O(kL(|\Sigma|^{L-d}))$ time whether there is a string with $\max_{i=1, \dots, k} d_H(s, s_i) \geq d$ and it computes such an s if one exists.

Proof. We extend the result to arbitrary symbols. The basic pseudo-code of the algorithm does not change, but in the case of choosing the complement of the alphabet, the branch factor of the bounded search tree is increased. In the case of binary alphabets, the p th position of s_i 's complement is either a change from 0 to 1 or 1 to 0. When we extend the case to non-binary alphabets, the complement may have more than one choice possible, for example, when the alphabet set = $\{0,1,2\}$ then the complement of 0 may be 1 or 2. Similarly in the beginning of the procedure call, if $\bar{s}_1$ satisfies the condition that the hamming distance between $\bar{s}_1$ and s_i is no lesser than d , then we have our answer; if $\bar{s}_1$ does not satisfy the above condition and a farthest string s does indeed exist, then there exists some string s_i such that $d_H(\bar{s}_1, s_i) < d$. Like the proof above, $\bar{s}_1$ and s_i have at least $L-d+1$ positions that share the same alphabet, hence s and s_i have at most $L-d$ positions with the same alphabet, then according to the pigeonhole theorem there exists a position on $\bar{s}_1$ that is the same with s_i but differs from the farthest string s , we take the complement of the alphabet on that position, then one of the complements must be the same with the alphabet on s , therefore that change will make the candidate string closer to the farthest string.

The change made in the FSD procedure will be made to recursively call $(|\Sigma|-1) * (L-d)$ FSD procedures

3 Farthest String Defined by Maximum Hamming Distance Sum

Since the term “farthest string” can be defined to either be restricted in a given string set or can be any arbitrary string constituted by the alphabet set.

Input: Strings $s_1, s_2, \dots, s_k$ over alphabet Σ of length L .

Question: Find a string $s \in \{s_1, s_2, \dots, s_k\}$ that

$$\text{maximizes } \sum_{i=1}^k d_H(s, s_i) ?$$

Given a set of k strings of length L , we can think of these strings as a $k \times L$ character matrix. Then we refer to the columns of this matrix as columns of the set of strings. Note that, given a set of length L strings $S = \{s_1, s_2, \dots, s_k\}$, a created string with the largest hamming distance can be easily computed by choosing in every column a letter occurring the least times. This way of selecting the letters is called the minority vote; yet this technique does not necessarily produce a unique solution.

Lemma 3. *By using the minority vote technique to solve this problem still can not find the exact result.*

Proof. We can to use the minority vote technique to find the pseudo farthest string, and then decide which existing string is closest to this string by computing the hamming distance sum. Yet, this algorithm does not yield a correct solution for two reasons. First, since the result of the minority vote technique is not unique. Selecting different alphabets can have different results. Secondly, when searching for a string that most

resembles the pseudo farthest string there may be more than one choice available. Apparently, this string does not yield the correct solution but an approximate result of the farthest sting bounded by the given string set problem.

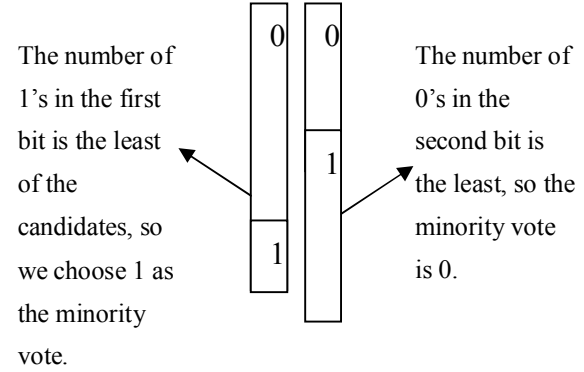


Fig. 2 The result of using the minority vote technique, the output is “00”, we can see that “00” and “11” have the same hamming distance with the ideal solution “10”, therefore we choose the first choice “00”. But the best solution in this example is “11”, so we introduce the concept of weighted votes.

Let $num(a, p)$ denote the number of times the alphabet a appears on the p th column. We assign a weight on every alphabet in a column, which is defined as k subtract the times an alphabet appears in the column. Every different combination of alphabets will yield different weights, and selecting the string with the largest combined weight is then the solution of this problem. The weight of the p th position of the i th string is defined as $w_i[p] = k - num(s_i[p], p)$

$$\textbf{Theorem 3. } \sum_{j \neq i} d_H(s_i, s_j) = \sum_{p=1}^L w_i[p]$$

Proof.

$$\begin{aligned}
\sum_{j \neq i} d_H(s_i, s_j) &= \sum_{j \neq i} \sum_{p=1}^L d_H(s_i[p], s_j[p]) \\
&= \sum_{p=1}^L \sum_{j \neq i} d_H(s_i[p], s_j[p]) \\
&= \sum_{p=1}^L k - \text{num}(s_i[p], p) \\
&= \sum_{p=1}^L w_i[p]
\end{aligned}$$

In other words, the sum of the hamming distance of the i th string and the other strings in the set will equal the sum of weights of every position on the i th string.

Algorithm Implementation

The concept of our algorithm is to calculate the weighted sum of every string and examine which string will yield the largest weighted sum, the designated string is then the solution to this problem.

```

1 for p=0 to L
2   for i=0 to k
3     num[si[p]] += 1
4 farthest = 0
5 farthest_distance = 0
6 for i=0 to k
7   temp_distance = 0
8   for p=0 to L
9     temp_distance += k - num[si[p]]
10  if temp_distance > farthest_distance
11    farthest_distance = temp_distance
12    farthest = i
return sfarthest

```

Fig. 3 The above algorithm calculates the weighted sum of every string in the set, and selects the string with the largest sum.

Note that if the alphabet size is not binary and

exceeds the number k , in [6] it has been shown that it is possible to find an isomorphic mapping that transforms the set of strings into a set that uses at most k alphabets.

Also note that, with slight modifications, the algorithm may be implemented to compute for a farthest string that does not necessarily be in the given set of strings.

Theorem 4 Given a set of binary strings $S = \{s_1, s_2, \dots, s_k\}$, the algorithm in figure 3 determines in $O(kL)$ time finding a string that maximizes

$$\sum_{i=1}^k d_H(s, s_i).$$

Proof.

Running time. The loops on line 1-3 will take $O(kL)$ loops and each loop takes $O(1)$ time, and the same situation occurs on line 8-12. Rest of the pseudo-code takes $O(1)$ time. Thus the upper bound on this algorithm is $O(kL)$.

Correctness. By Theorem 3, the string with the maximum sum of weights is the string that we are searching for. The algorithm that we propose calculates the sum of weights for every string in the set, and selects the string with the maximum sum. Therefore, by Theorem 3 we indeed obtain the farthest string.

Conclusion

In this paper we present two definitions in which the farthest string may be defined. Either via the maximum hamming distance sum between the set of given strings or via the minimum distance for a farthest string. We present algorithms to achieve the answer for both of these problems.

Both of our algorithms are proved to be linear, if

the minimum hamming distance d is a fixed constant. This result is very useful in the field of bio-information. Yet, still much effort is still needed. Although we have proved the algorithm to be linear, the coefficient of the algorithm may be very large depending on the value of d that we select. Ways to cut down on the coefficient of the algorithm will yield a better practical use of our algorithm.

References

- [1] J. Alber, J. Gramm, and R. Niedermeier. Faster exact solutions for hard problems: a parameterized point of view. *Discrete Mathematics* 229:3-27, 2001.
- [2] S. Arora, C. Lund, R. Motwani, M. Sudan, M. Szegedy, Proof verification and intractability of approximation problems, In *Proceedings of the 33th Annual IEEE Symposium on Foundations of Computer Science*, 13–22, 1992.
- [3] R. G. Downey and M. R. Fellows. *Parameterized Complexity*. Springer-Verlag, New York, 1999.
- [4] M. R. Fellows. Parameterized complexity: the main ideas and connections to practical computing. In *Experimental Algorithmics, Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 51–77, 2002.
- [5] M. Garey, D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H. Freeman, San Francisco, 1979.
- [6] J. Gramm, R. Niedermeier, and P. Rossmanith , Fixed-Parameter Algorithms for CLOSEST STRING and Related Problems, *Algorithmica* 37: 25–42, 2003.
- [7] T. Jiang, C. Trendall, S. Wang, T. Wareham, X. Zhang, Drug target identification using Gibbs sampling techniques, in: *Pacific Symposium on Biocomputing*, 389–400, 2000.
- [8] J. K. Lanctot, M. Li , B. Ma, S. Wang, and L. Zhang, Distinguishing string selection problems, *Information and Computation* 185: 41–55, 2003.

Local Rules for Protein Folding on Triangular Lattice in the 2D HP Model

(Extended Abstract)

*Sheng-Lung Peng[†], Yung-Sen Liu[†], Hui-Chun Wu[†], Chuan Yi Tang[‡]

[†]Department of Computer Science and Information Engineering
National Dong Hwa University, Hualien, Taiwan, R.O.C.

[‡]Department of Computer Science, National Tsing Hua University
Hsinchu, Taiwan, R.O.C.

Abstract

Agarwala et al. proposed two main folding rules, namely, backbone folding and arrow folding, on triangular lattice in 2D HP model [1]. However, these folding rules are not suitable for sequences with sparse H's. In this paper, we propose two folding rules, which can support the weakness of their folding rules. The first rule is called starlike folding which is better than others for the sequences in the pattern of $(HP^+)^+$. The other rule is called compact starlike folding which is suitable for the sequences in the combined patterns of $(HP^+)^+$ and H^n . Together with backbone folding and arrow folding, we propose a general strategy of protein folding which suggests a suitable usage for all possible sequences.

1 Introduction

A protein is a sequence of amino acids linked together. There are twenty different amino acids used in the formation of proteins. The length of a protein sequence ranges from 50 residues to about 2000. Understanding the behavior and properties of proteins is one of the most fascinating challenges in molecular biology.

After its discovery, many scientists attempt to answer two questions: How should people predict the structure of a protein given its sequence? Is it possible to find a protein sequence that folds into a desired structure? Both questions, known as *direct* and *inverse foldings*, remain unanswered. Much effort (about thirty years worth) has been

expended trying to understand how proteins fold up in nature. Understanding structures allows biologists to make an educated guess about protein functions. It would also enhance our ability for drugs design by seeing the three dimensional molecular details of the proteins in our body with which almost all kinds of drugs interact.

In this paper, we study the Hydrophobic-hydrophilic (Polar) model introduced by Dill [9]. It is one of the most studied models in this domain [3, 4, 6, 10, 15, 17]. The HP model abstracts the problem by dividing the 20 amino acids into two classes: hydrophobic and hydrophilic residues. We can classify all amino acids in a protein as hydrophobic or hydrophilic [7, 20].

In this model, we only consider a protein sequence from $\{H, P\}$, where H represents hydrophobic residues, and P represents polar residues. For convenience, we use regular expressions to describe these sequences. For example, H^n indicates the sequence with n H's in sequence, $H^3(P^+H)^+$ indicates 3 H's following at least one subsequence with (P^+H) pattern where (P^+H) means that the subsequence contains at least one P and ending on an H.

The Hydrophobic-Polar model is a free energy model. The energy function for this system favors interactions between H-H monomer types and is indifferent to all other interactions (P-P and H-P). This is known to highlight the importance of the hydrophobic effect in protein folding. A nonadjacent H-H bond is called a *contact*.

The HP model is also a lattice model because those HP sequences are placed in a lattice. The lattice nodes are responsible for keeping the nodes of sequence at the fixed lattice node position. The characters of lattice model are as follows.

*Corresponding author: lung@csie.ndhu.edu.tw

- Monomers are represented using uniform size.
- Bond length is uniform.
- Positions are restricted to a regular lattice.
- Every conformation is a self-avoiding walk in 2D or 3D.

There are several types of lattice models. For example, 2D square, 2D triangular, 2D hexagonal, 3D cubic, 3D triangular and 3D diamond, etc [14]. The most used lattice is square lattice. However, square lattice has parity constraint, i.e., the sequence $(PH)^n$, has 0 contact, but has $2n - 3$ contacts on the triangular lattice. This sequence is the worst case on the square lattice but is the best case on the triangular lattice. Thus, the triangular lattice is more closer to the real world than the square lattice. The triangular lattice does not exhibit the parity problem [1, 12].

The *protein folding problem* is to find a folding (conformation) of a protein sequence such that the number of contacts is maximum (or the overall energy is minimum). For a variety of such simple models, this minimization problem is NP-complete [5, 8, 19].

For a protein sequence S , let N_h denote the number of H's in S , N_p denote the number of P's in S , and N_{ph} be the number of (PH)'s in S . If $N_p \geq N_h$ and $N_{ph} > \frac{N_h}{2}$, then we call S a *sparse sequence*; otherwise, it is called a *dense sequence*.

In [1], Agarwala *et al.* proposed a set of folding rules on a triangular lattice and analyzed their approximation ratios. In particular, on 2D triangular lattice, they proposed some folding rules, namely, *backbone folding* which has $\frac{1}{4}$ -approximation, *improved backbone folding* which has $\frac{1}{2}$ -approximation, *arrow folding* which has $\frac{1}{2}$ -approximation, and *improved arrow folding* which has $\frac{6}{11}$ -approximation. Unfortunately, the worse cases of these foldings occur in sparse sequences. Besides, by exploring PDB (Protein Data Bank), we find that there are 10788 (among 24615) sequences which are sparse. It motivates this study.

In this paper, we propose two folding rules, namely, *starlike folding* and *compact starlike folding* for sparse sequences. The starlike folding yields $\frac{7}{12}$ -approximation and the compact starlike folding yields better than $\frac{7}{12}$ -approximation. The rest of this paper is organized as follows: Section 2 provides a necessary background. Section 3 presents our main results. Finally, we give some concluding remarks in Section 4.

2 Preliminaries

A graph of triangular lattice is a placement of vertices on distinct points of the regular triangular lattice such that each edge of the graph maps to adjacent node on the lattice. Every node in the lattice has 6 neighbors.

A protein sequence in HP model is represented as $S = s_1 s_2 s_3 s_4 \cdots s_n$, where $s_i \in \{H, P\}$ for all $1 \leq i \leq n$. A folding of a protein in the 2D HP model is an embedding of S in the 2D lattice. Two H-nodes in lattice form a *non-local H-H bond* if they are adjacent in the lattice but not adjacent in S . This non-local H-H bond is called a *contact* [2]. That is, if s_i and s_j are adjacent H-nodes in lattice and $|i - j| > 1$, then there is a contact between s_i and s_j . It is not hard to see that an H-node has at most 4 contacts except its neighbors in the sequence. If an H-node has 4 contacts, this node is called a *complete node*.

For each of the neighboring lattice points that is occupied by neither an adjacent node in the sequence nor an H-node, we call this lattice edge a *missing contact*. Therefore, for each node of the sequence on triangular lattice, the sum of the contacts and missing contacts is at most 4, and is 5 if this node is the endpoint of the sequence.

2.1 Backbone Folding

Backbone folding is a simple folding strategy based on laying out two backbones anti-parallel to each other such that most of the contacts are achieved among the H's along a backbone and the H's across the backbones [1]. It is according to the following rules to fold a protein sequence.

- Every H is placed on the backbone.
- Two P's on the backbone never form contacts.

Each node of the backbone must have two neighbors from the other backbone and at least one of which must be an H-node. All the H-nodes are on the backbone and each one must create at least one bond. It gives a minimum of $\frac{N_h}{2}$ bonds out of a maximum of $2N_h$ bonds. Therefore, the approximation ratio of the backbone folding is at least $\frac{1}{4}$ [1].

For some special sequences, the backbone folding has a near optimal solution. For example, consider the sequence $S = (HP)^n$, we can use the backbone folding to layout this sequence. In this folding, most of the H-nodes have 4 contacts. The four nodes at the endpoints of the backbone strip

have only 2 or 3 contacts. Thus, the number of contacts is $C(S) = 2N_h - 3$. It almost yields an optimal folding in the 2D HP model on the triangular lattice for large values of n .

2.2 Arrow Folding

The following definitions come from [1]. A *strip* is a maximal substring of a sequence containing only H's. For example, HHHH is a 4-strip starting at position 2 of $S = \text{PHHHHPHPPPPHH-PHHHHH}$. A *separator* is a maximal substring of a sequence containing only P's and a *k-separator* likewise. For example, PPPP is a 4-separator between 1-strip and 2-strip in S .

Define levels L^{+1} , L^0 and L^{-1} , a sequence is folded along this pathway such that all the H-nodes are distributed only in L^{+1} , L^0 , and L^{-1} . All the residues on level L^0 are H-nodes. Arrow folding yields a $\frac{1}{2}$ -approximation in the 2D HP model on triangular lattice for large values of N_h [1].

In an arrow folding, the arrow can be divided into three groups, head, body, and tail. It is not hard to see that the arrow tail is a sparse sequence of H's and most of the missing contacts are in the arrow tail. In this area, all H's are placed in L^0 level and each H has only 2 contacts according to the rule of arrow folding. In summary, the arrow folding is suitable for dense sequences.

3 Main Results

In this section, we propose some local folding rules for the 2D HP model on triangular lattices. Together with backbone and arrow folding rules, we propose a general strategy for using these rules to fold a protein sequence.

3.1 Circle Folding and Hexagon Folding

At first, we introduce a local rule for folding H^n with a large n . If an H-node is on the border of a folding graph of H^n , then this H-node is a non-complete node. All of the complete nodes must be in the internal part of a folding graph. Therefore, if we want to increase the number of contacts, we have to reduce the number of H-nodes on the border of the folding graph.

If we apply arrow folding on H^n , then its shape looks like a bar. It is known that the circumference of a circle is smaller than the perimeter of any

other shape with the same area [2]. That is, if H^n is folded as a circle, it can get the maximum number of contacts. Thus, the *circle folding* is a folding such that the resulting folding graph is a circle.

Though circle folding is better than arrow folding in long H^n , it is hard to implement. We now propose another folding similar to the circle folding. We call it *hexagon folding*. The hexagon folding is easier to construct than circle folding because the circle folding has to calculate the length of each chord. Hexagon folding only surrounds the central node and change its direction on different level from center to outside. Furthermore, its folding graph is equivalent to circle folding in short sequence. So we use hexagon folding instead of circle folding.

3.2 Starlike Folding

In this section, we focus on the sparse sequences. Consider the case that a lot of (HP^+) 's or (H^2P^+) 's are in a sequence. If we use the backbone folding or the arrow folding, then we obtain a result near $\frac{1}{2}$ -approximation. Suppose that we apply the arrow folding to a sparse sequence of H's like $(HP^+)^+$, it is not hard to see that all the H's are placed on a line

Now we introduce *starlike folding* for sparse sequences. It is named because its folding shape looks like “*.” The idea is to put H's together and put P's outside H's. It is similar to backbone folding, with a slight modification for sequences of (HP^+) 's. The difference between starlike folding and backbone folding is the rule for folding P's. It defines six directions like a star, one for pathway in and out, the other directions for folding the P's. A sequence is folded along this pathway such that all the H's are placed only in central area.

Note that for each direction of a star determines a ray for a sequence of P's. In the case of long P chains, the rays from multiple stars will intersect. It can be handled by operations similar to *pull moves* proposed by Lesh *et al.* [13].

Now we analyze the approximation ratio of starlike folding for sparse sequences. Let S be such a sequence. Divide S into k subsequences $S_1, S_2, \dots, S_k$, i.e., $S = S_1 S_2 \dots S_k$. Each S_i contains 6 H's except S_k which may contain less than 6 H's. Let $N_{h(i)}$ (respectively, $N_{ph(i)}$) be the number of H's (respectively, (PH)'s) in S_i . Let $N_{d(i)} = N_{h(i)} - N_{ph(i)}$. For each S_i , $1 \leq i \leq k$, the number of contacts is $C(S_i) = 2N_{h(i)} - 3 - N_{d(i)}$. By the definition, $N_{ph(i)} > \frac{N_{h(i)}}{2}$. Thus, $N_{d(i)} \leq$

$\frac{N_{h(i)}}{2} - 1$. It leads to $C(S_i) = 2N_{h(i)} - 3 - N_{d(i)} \geq 2N_{h(i)} - 3 - \frac{N_{h(i)}}{2} + 1 = \frac{3N_{h(i)}}{2} - 2$. Since $k \leq \frac{N_h}{6} + 1$, $C(S) = \sum_{i=1}^k (\frac{3N_{h(i)}}{2} - 2) = \frac{3N_h}{2} - 2k \geq \frac{3N_h}{2} - 2(\frac{N_h}{6} + 1) = \frac{7N_h}{6} - 2$. Therefore, the approximation ratio is $\frac{C(S)}{2N_h} \geq \frac{\frac{7N_h}{6} - 2}{2N_h} \approx \frac{7}{12}$. Thus, we have the following theorem.

Theorem 3.1 *Starlike folding yields a $\frac{7}{12}$ -approximation for sparse sequences.*

It shows that the starlike folding is better than arrow folding on sparse sequences. Note that the starlike folding is not suitable for dense sequences. An $H^n (n > 1)$ subsequence in a dense sequence may decrease the approximation. In this case, there exists an $H^n, n > 2$, between two starlike foldings. This H^n is too short to be folded, so it may decrease the approximation. Thus, arrow folding is a good choice. In the following section we propose the compact starlike folding to tackle this problem.

3.3 The Compact Starlike Folding

Although the starlike folding increases the approximation for sparse sequences, a short H^n ($3 \leq n$) between two starlike folding may decrease the approximation. For tackling this problem, we use starlike folding to surround this short H^n in order to increase the folding approximation. We call this rule *compact starlike folding*.

Now we describe the rules of the compact starlike folding. In the case of $3 \leq n \leq 7$, then we place the short H^n in a hexagon. We select one node on the hexagon as a start point and select one neighboring node as end point in order to place more HP^+ surrounding the short H^n . This little adjustment is different from the rule of hexagon folding in the first level of hexagon.

Second, we place the other H 's (with the form of HP^+) outside the H^n . The P 's are placed outside these H 's according to the rules of starlike folding. Every H in HP^+ must be kept close to the inner part of starlike folding.

Consider an example to compare the effect of these folding rules. Let $S = PHP^2HP^2H^2P^2HP^3H^{19}P^2HP^2H^2P^2HP^2HP^2$. Table 1 shows the result.

The approximation of the compact starlike folding is hard to measure. However, in our experiment, its approximation is better than others for

Folding rules	Contacts	App.
Arrow folding	32	0.55
starlike folding	36	0.62
Compact starlike folding	47	0.81

Table 1: A result of different folding rules.

Sequence types	Folding rules
$N_h > N_p$	Arrow
$N_h = N_p, N_t = 1$	Arrow
$N_h = N_p = N_t$	Backbone
$N_h < N_p, N_t \div 1$	Arrow
$N_h < N_p, N_t \div N_h$	Starlike
$N_h < N_p, N_t \div N_h, N_{H^3} > 0$	Compact starlike

Table 2: N_h : no. of H nodes, N_p : no. of P nodes, N_t : no. of (PH) 's, N_{H^3} : no. of (H^3) 's.

sparse sequences. Hence, we conjecture that its approximation is better than others for sparse sequences.

3.4 A General Strategy

Finally, we integrated these rules mentioned above and propose a general strategy for folding a protein sequence on triangular lattice in the 2D HP model. Our strategy is as follows.

- Step 1. Find all the subsequences H^n with $n \geq 8$ and fold them by hexagon folding or arrow folding.
- Step 2. For each remaining subsequence, choose a folding rule to fold it according to the suggestion listed in Table 2.
- Step 3. Check each pair of adjacent foldings and try to use the compact starlike folding to combine them.
- Step 4. Combine each folded sequence sequentially.

4 Conclusion

In this paper, we propose the starlike folding and the compact starlike folding on triangular lattice in the 2D HP model. For sparse sequences, starlike folding yields $\frac{7}{12}$ -approximation and compact starlike folding seems better than starlike folding. Both of them are better than the previous folding rules for sparse sequences. Together with backbone folding and arrow folding, we suggest a

general strategy for folding any protein sequence on triangular lattice in the 2D HP model. For the future, we will consider the same problem on 3D triangular lattice and consider feasible global rules.

References

- [1] R. Agarwala, S. Batzoglou, V. Dancik, S. Decatur, M. Farach, S. Hannenhalli, and S. Skiena, Local rules for protein folding on a triangular lattice and generalized hydrophobicity in the HP model, *Journal of Computational Biology*, 4(1997), 275–296.
- [2] O. Aichholzer, D. Bremner, E. D. Demaine, H. Meijer, V. Sacristau, and M. Soss, Long Proteins with Unique Optimal Foldings in the H-P Model, *Computational Geometry: Theory and Applications*, 2003, 139–159.
- [3] E. Bornberg-Bauer, Simple Folding Model for HP Lattice Proteins, *Computer Science*, 1996, 125–136.
- [4] E. Bornberg-Bauer, How are model protein structures distributed in sequence space?, *Journal of Biophysical*, 1997, 2393–2403.
- [5] B. Berger and T. Leighton, Protein folding in the hydrophobic-hydrophilic (HP) model is NP-complete, *Journal of Computational Biology*, 5(1998), 27–40.
- [6] R. Backofen, S. Will and E. Bornberg-Bauer, Application of Constraint Programming techniques for Structure Prediction of Lattice Proteins with Extended Alphabets, *Bioinformatics*, 1999, 234–242.
- [7] G. Caldarelli and P. De Los Rios, Cold and Warm Denaturation of Proteins, *Journal of Biological Physics*, 2001, 229–241.
- [8] P. Crescenzi, D. Goldman, C. Papadimitriou, A. Piccolboni, and M. Yannakakis, On the complexity of protein folding, *Journal of Computational Biology*, 5(1998), 423–465.
- [9] K. A. Dill, S. Bromberg, K. Yue, K. M. Fiebig, D. P. Yee, P.D. Thomas, and H. S. Chan, Principles of protein folding - a perspective from simple exact models. *Protein Science*, 1995, 561–602.
- [10] W. E. Hart and S. Istrail, Fast protein folding in the hydrophobic-hydrophilic model within three-eighths of optimal, *Journal of Computational Biology*, 1996, 53–96.
- [11] A. Irback, C. Peterson, F. Potthast, and O. Sommelius, Local Interactions and Protein Folding: A 3D Off-Lattice Approach, *Journal of Chemical Physics*, 1997, 273–282.
- [12] A. Irback and E. Sandelin, Local Interactions and Protein Folding: A Model Study on the Square and Triangular Lattices, *Journal of Chemical Physics*, 1998, 2245–2250.
- [13] N. Lesh, M. Mitzenmacher, S. Whitesides, A Complete and Effective Move Set for Simplified Protein Folding, *Proceedings of 7th Annual International Conference on Research in Computational Molecular Biology (RECOMB)*, 2003, 188–195.
- [14] H. Li, C. Tang, and N. S. Wingreen, The Designability of Protein Structures: A Lattice-Model Study using the Miyazawa-Jernigan Matrix, *Proteins*, 49(2002), 403–413.
- [15] R. B. Lyngso and C. N. S. Pedersen, Protein Folding in the 2D HP Model, *BRICS RS-99-16*, 1999, 1–18.
- [16] A. Nayak, A. Sinclair, and U. Zwick, Spatial codes and the hardness of string folding problems, *Proceedings of the Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 1998, 639–648.
- [17] D. Pelta, N. Krasnogor, A. B. Moron, and J. L. Verdegay, F.A.N.S. for the Protein Folding Problem: Comparing Encodings and Search Modes, *Proceedings of the 4th Metaheuristics International Conference (MIC2001)*, 2001, 327–332.
- [18] A. Radzicka, and R. Wolfenden, Comparing the Polarities of the Amino Acids: Side-Chain Distribution Coefficients between the Vapor Phase, Cyclohexane, 1-Octanol, and Neutral, *Biochemistry*, 1988, 1664–1670.
- [19] A. S. Fraenkel, Complexity of protein folding, *Bulletin of Mathematical Biology*, 1993, 1190–1200.
- [20] E. Sandelin, Thermodynamics of Protein Folding and Design (Ph.D. Thesis) Date of issue: 2000-09-08, ISBN 91-628-4305-2.